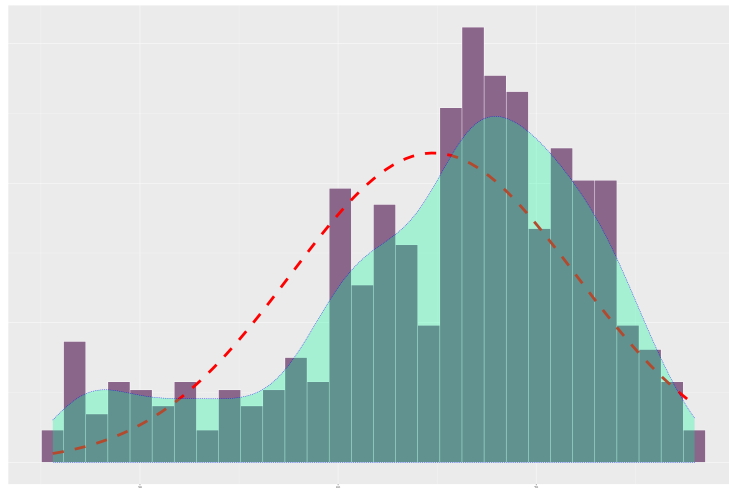




Statistik mit R und RStudio



EIN NACHSCHLAGEWERK FÜR
GESUNDHEITSBERUFE

zusammengestellt von
Jörg große Schlarmann

Version vom 14. Februar 2023

Das Logo der Hochschule Niederrhein sowie die Logos von R, RStudio und quarto sind Eigentum der jeweiligen Rechteinhaber. Für alles weitere gelten die folgenden Bedingungen.



Dieses Werk ist unter der Creative Commons Lizenz CC BY-NC-SA 4.0 verfügbar, siehe <https://creativecommons.org/licenses/by-nc-sa/4.0/deed.de>

Sie dürfen

- das Material in jedwedem Format oder Medium vervielfältigen und weiterverbreiten.
- das Material remixen, verändern und darauf aufbauen.

Unter folgenden Bedingungen:

Namensnennung: Sie müssen angemessene Urheber¹-und Rechteangaben machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen vorgenommen wurden. Diese Angaben dürfen in jeder angemessenen Art und Weise gemacht werden, allerdings nicht so, dass der Eindruck entsteht, der Lizenzgeber unterstütze gerade Sie oder Ihre Nutzung besonders.

Nicht kommerziell: Sie dürfen das Material nicht für kommerzielle Zwecke nutzen.

Weitergabe unter gleichen Bedingungen: Wenn Sie das Material remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen Sie Ihre Beiträge nur unter derselben Lizenz wie das Original verbreiten.

Keine weiteren Einschränkungen: Sie dürfen keine zusätzlichen Klauseln oder technische Verfahren einsetzen, die anderen rechtlich irgendetwas untersagen, was die Lizenz erlaubt.

Eine aktuelle epub-Version dieses Buches finden Sie unter:

<https://www.produnis.de/R/rbuch.epub>

Eine aktuelle PDF-Version dieses Buches finden Sie unter:

<https://www.produnis.de/R/rbuch.pdf>

Dieses Buch steht zudem als Webseite bereit unter:

<https://www.produnis.de/R/>

Kritik und Diskussion sind per Mastodon möglich:

[🐙 https://mastodon.social/@rbuch](https://mastodon.social/@rbuch)

¹**Zitationsvorschlag:** große Schlarman, J (2023): "*Statistik mit R und RStudio - Ein Nachschlagewerk für Gesundheitsberufe*", Hochschule Niederrhein, Krefeld,
<https://www.produnis.de/R/>

Inhaltsverzeichnis

1	Einleitung	1
1.1	Installation von R und RStudio	2
1.2	Kurze Einführung in R und RStudio	3
1.2.1	R	3
1.2.2	RStudio	4
1.2.2.1	RStudio anpassen	5
1.2.2.2	Eine Sitzung starten	8
1.2.2.3	Projekte in RStudio	13
1.2.3	Grundlegende Befehle in R	16
1.2.3.1	Rechnen mit R	17
1.2.3.2	Variablen	22
1.2.3.3	Wertesequenzen erzeugen	27
1.2.3.4	Datentypen	32
1.2.3.5	Datenklassen	35
1.2.3.5.1	Vektoren	36
1.2.3.5.2	Matrizen	38
1.2.3.5.3	Faktoren	45
1.2.3.5.4	ordinale Faktoren	50
1.2.3.5.5	Datenframes	51
1.2.3.5.6	Listen	66
1.2.3.6	Fehlende Daten	68
1.2.3.7	Umgang mit Datensätzen	70
1.2.3.7.1	Sortieren	71
1.2.3.7.2	Teilgruppen erstellen	73
1.2.3.8	Pipe	75
1.2.3.9	Daten laden und speichern	77
1.2.3.10	Arbeitsverzeichnis	79
1.2.3.11	Daten importieren	80
1.2.3.11.1	Import aus Textdateien	80
1.2.3.11.2	Import aus CSV-Datei	83

1.2.3.11.3	Import aus SPSS-Datei	85
1.2.3.11.4	Import aus Excel-Datei	87
1.2.3.11.5	Import aus .ods-Datei	88
1.2.3.11.6	Import mit RStudio	89
1.2.3.12	Daten exportieren	91
1.2.3.12.1	Export als CSV-Datei	91
1.2.3.12.2	Export als Textdatei-Datei	91
1.2.3.12.3	Export als SPSS-Datei	92
1.2.3.13	Zusatzpakete installieren	92
1.2.3.13.1	Zusatzpaket zu diesem Lehrbuch	97
1.2.3.14	Ausgabe in Datei umleiten	98
1.2.4	Verteilungsfunktionen	101
1.2.4.1	Normalverteilung	101
1.2.4.2	t-Verteilung	105
1.2.4.3	Chiquadrat-Verteilung	107
1.2.4.4	Poisson-Verteilung	108
1.2.4.5	Binomial-Verteilung	110
1.2.5	Eigene Funktionen programmieren	112
1.2.5.1	Beispiele für Funktionen	113
1.2.5.1.1	z-Transformation	113
1.2.5.1.2	Sensitivität / Spezifität	114
1.2.5.1.3	Kenngößen	114
1.2.5.1.4	Häufigkeitstabellen	115
1.2.5.2	Bedingungen	115
1.2.5.2.1	Beispiel Zusatzpakete	117
1.2.5.3	Funktionen in Dateien speichern	119
1.3	Markdown	120
1.4	RMarkdown	121
1.4.1	Syntax	125
1.5	quarto	132
1.5.1	Installation	132
1.5.2	Dokument erstellen	133
1.5.3	Editor-Ansicht	137
1.5.4	Dokumentenaufbau	137
1.5.5	Syntax	139
1.5.5.1	Textformat	139
1.5.5.2	Überschriften	139

1.5.5.3	Links und Bilder	139
1.5.5.4	Listen	140
1.5.5.5	Tabellen	140
1.5.5.6	Gleichungen	141
1.5.6	R-Code integrieren	141
1.5.6.1	Chunk-Optionen	144
1.5.7	PDF-Dokumente erzeugen	145
1.5.8	quarto-Dokumentation	146
1.6	Tidyverse	147
1.6.1	Pakete	150
1.6.2	Pipe	151
1.6.3	Unterschiede zwischen %>% und >	152
1.6.4	tibbles	153
1.6.5	Funktionsaufrufe	156
1.6.6	Schritt 1: Daten importieren	157
1.6.6.1	gelabelte SPSS-Daten	157
1.6.7	Schritt 2: Daten <i>tidy</i> machen	160
1.6.7.0.1	verschachtelte wide table	162
1.6.7.1	Datentypen korrigieren	164
1.6.7.2	fehlende Werte	168
1.6.7.3	Variablen umbenennen	172
1.6.8	Schritt 3: Umgang mit Datensätzen	172
1.6.8.1	Daten filtern und sortieren	173
1.6.8.2	Fälle auswählen	176
1.6.8.2.1	Gruppierungen	180
1.6.8.3	Variablen hinzufügen	182
1.6.8.4	Fälle hinzufügen	182
1.6.8.5	Variablen auswählen	185
1.6.8.6	Variablen erzeugen	186
1.6.8.7	statistische Berechnungen	187
1.6.8.8	Umgang mit Faktoren	191
1.6.8.8.1	Levels umbenennen	191
1.6.8.8.2	Levelreihenfolge ändern	192
1.6.8.8.3	Levels zusammenfassen	198
1.6.8.9	Daten visualisieren	201
1.6.9	Schritt 4: Ergebnisse kommunizieren	201
1.6.10	Die Macht der Pipe	201

1.7	Weitere Hilfsmittel	202
1.7.1	Cheatsheets	202
1.7.2	Youtube	203
1.7.3	freie Lehrbücher	203
1.7.4	Internetforen	204
2	Statistik mit R und RStudio	205
2.1	Deskriptive Statistik	206
2.1.1	Häufigkeiten	206
2.1.2	Lagekenngrößen	207
2.1.3	Streuungskenngrößen	212
2.1.4	Kreuztabellen	213
2.1.5	z-Transformation	214
2.1.6	Korrelation	215
2.1.7	Regressionen	219
2.1.7.1	multiple lineare Regressionen	221
2.1.7.2	Logistische Regression	223
2.1.7.3	Ordinale Regression	225
2.1.8	Überlebenszeitanalysen	238
2.1.8.1	Kaplan-Meier-Analysen	238
2.1.8.2	Cox-Regression	246
2.1.9	Faktorenanalyse	248
2.1.9.1	explorative Faktorenanalyse	248
2.1.9.2	konfirmatorische Faktorenanalyse	252
2.1.9.3	Hauptkomponentenanalyse	256
2.2	Schließende Statistik	258
2.2.1	Konfidenzintervalle	258
2.2.1.1	Normalverteilung	258
2.2.1.2	Binomialverteilung	264
2.2.2	Signifikanztests	270
2.2.2.1	Chiquadrat-Test	270
2.2.2.2	Fisher's Exakttest	270
2.2.2.3	t-Test	271
2.2.2.3.1	Einstichproben t-Test	271
2.2.2.3.2	unabhängiger t-Test	272
2.2.2.3.3	abhängiger t-Test	274
2.2.2.4	F-Test	275

2.2.2.5	Levene-Test	275
2.2.2.6	Mann-Whitney-U-Test	276
2.2.2.7	Wilcoxon-Test	277
2.2.2.8	Shapiro-Wilk-Normalverteilungstest	278
2.2.3	Varianzanalysen	280
2.2.3.1	ANOVA mit Messwiederholung	280
2.2.3.2	Friedman ANOVA	287
2.2.3.3	einfaktorielle ANOVA	293
2.2.4	Fallzahlkalkulation	295
2.2.4.1	klinische Studien	295
2.2.4.2	Umfragen	296
2.3	Diagramme plotten	298
2.3.1	Punktwolke	300
2.3.2	verfügbare Farben	308
2.3.3	Histogramm	309
2.3.4	Kreisdiagramm	312
2.3.5	Säulendiagramm	315
2.3.6	Balkendiagramm	317
2.3.7	Boxplot	319
2.3.8	Diagramme speichern	321
2.4	ggplot	322
2.4.1	Satzbau	322
2.4.2	Erstes Plot	324
2.4.2.1	Aesthetics	325
2.4.2.2	Geome	325
2.4.2.3	mehr Aesthetics	327
2.4.2.4	mehr Geome	328
2.4.2.5	Facetten	330
2.4.2.6	Geom-Parameter	333
2.4.3	Punktwolke	335
2.4.4	Histogramm	337
2.4.4.1	Histogramm und Dichteverteilung	339
2.4.5	Säulendiagramme	344
2.4.6	Balkendiagramme	352
2.4.7	Boxplots	354
2.4.8	Kreisdiagramme	359

2.4.9	Diagramme speichern	361
2.4.9.1	R-Objekt	361
2.4.9.2	in Datei speichern	362
2.4.10	Aussehen ändern	363
2.4.10.1	Titel und Überschriften	363
2.4.10.2	Achsen	364
2.4.10.3	Legendenbox	367
2.4.10.4	Farben	370
2.4.10.5	Linien	374
2.4.10.6	Text	375
2.4.10.7	Sonderzeichen	376
2.4.10.8	Themes	376
3	Was willst du machen?	380
3.1	Daten manipulieren	381
3.1.1	Einer Kreuztabelle Prozentwerte hinzufügen	381
3.1.2	Eine gemeinsame Kreuztabelle mit mehrere Variablen erstellen	382
3.1.2.1	Zeilen- und Spaltensummen	384
3.1.2.2	Prozenttabellen	385
3.1.2.3	Signifikanztests	386
3.1.2.4	eine große Kreuztabelle	387
3.1.3	Zeilen und Spalten tauschen	388
3.1.4	Klassen für Variablen bilden (klassieren)	389
3.2	Diagramme plotten	392
3.2.1	Normalverteilung	392
3.2.2	t-Verteilung	403
3.2.3	Chiquadrat-Verteilung	406
3.2.4	Poisson-Verteilung	409
3.3	Pakete verwalten	412
3.3.1	Pakete beim Start automatisch laden	412
3.4	RMarkdown	413
3.4.1	Ausgabeformat per <code>.css</code> -Datei ändern	413
3.5	quarto	414
3.5.1	eigene L ^A T _E X Vorlagen erstellen	414
3.5.1.1	<code>.tex</code> -Date	414
3.5.1.2	Vorlage einbinden	416

3.5.1.3	YAML-Variablen	416
3.5.2	quarto am Handy oder Tablet	417
3.6	Referenztabellen erstellen	418
3.6.1	Fallzahlen nach Effektstärke und Power	418
3.6.2	Verteilungsfunktion und Quantile der Standardnormal- verteilung	422
3.6.3	Kritische Werte t^* der t -Verteilung	426
3.6.4	Kritische Werte der Chiquadrat-Verteilung	428
3.7	COVID19 Fallzahlen analysieren	429
Literaturverzeichnis		436
Stichwortverzeichnis		438
Verzeichnis der verwendeten R-Befehle		441

1 Einleitung

Willkommen zu diesem Nachschlagewerk.

Dieses Buch widmet sich an Studierende und Beschäftigte im Gesundheitswesen und soll als Nachschlagewerk für statistische Fragestellungen bezüglich der freien Software  dienen. Es ist **nicht** dafür geeignet, Einsteigern Statistik zu erklären, sondern es richtet sich an jene, die ihre statistischen Auswertungen mit  fahren möchten. Das heisst, Sie müssen schon wissen was Sie eigentlich tun wollen. Dieses Buch möchte Ihnen zeigen, wie Sie Ihre Vorhaben in  umsetzen können.

Dieses Buch ist in drei Teile gegliedert:

1. Im ersten Teil (ab Seite 2) geben wir eine kurze Einführung in  und RStudio.
2. Im zweiten Teil (ab Seite 205) betreiben wir Statistik mit .
3. Der dritte Teil (ab Seite 380) zeigt konkrete Anwendungsbeispiele. Hierbei folgen wir der Ordnung “Was willst du machen?” und zeigen an “echten” Fragen, wie wir das erlernte -KungFu in der Praxis anwenden können.

Eine aktuelle epub-Version dieses Buches finden Sie unter:

<https://www.produnis.de/R/rbuch.epub>

Eine aktuelle PDF-Version dieses Buches finden Sie unter:

<https://www.produnis.de/R/rbuch.pdf>

Dieses Buch steht zudem als Webseite bereit unter:

<https://www.produnis.de/R/>

Kritik und Diskussion sind per Mastodon möglich:

 <https://mastodon.social/@rbuch>

1.1 Installation von R und RStudio



® ist Open Source Software und kann daher weltweit kostenfrei auf vielen Betriebssystemen, z.B. Windows, Apple und Linux, installiert werden. Gehen Sie hierzu auf die Webseite des Projektes unter <https://r-project.org> und laden sich das passende Installationspaket für Ihr Betriebssystem herunter und führen Sie die Installation durch.

Für Windows lautet der Link auf das Installationspaket

<https://cran.r-project.org/bin/windows/base/>

Für MacOSX lautet der Link

<https://cran.r-project.org/bin/macosx/>

Bei fast allen Linuxdistributionen liegt R im Repository, so dass es z.B. unter Ubuntu mit `apt-get install r-base` installiert werden kann.

Von RStudio steht ebenfalls eine quelloffene (sowie eine kommerzielle) Version für diese Systeme zur Verfügung. Gehen Sie auch hier auf die Projektseite <https://rstudio.com/products/rstudio/download/> und laden sich die kostenfreie Variante für Ihr Betriebssystem herunter. Sie können RStudio erst installieren, wenn R bereits installiert ist.

Falls Sie nicht wissen, wie sich Software auf Ihrem PC installieren lässt, empfehlen wir dieses Youtube-Video

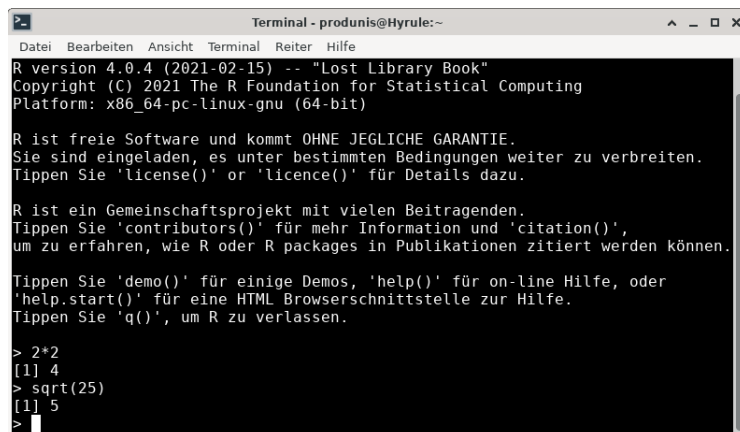
https://www.youtube.com/watch?v=X_Mxya2Fis0

Es zeigt den gesamten Installationsprozess von ® und RStudio am Beispiel “Windows” (weitere Youtube-Videos sammeln wir in Abschnitt 1.7.2 auf Seite 203).

1.2 Kurze Einführung in R und RStudio

1.2.1 R

 ist eine Programmiersprache für Statistik. Mit ihr lassen sich statistische Berechnung durchführen und graphisch darstellen. Die klassische -Software läuft in einem Terminalfenster (Abbildung 1.1) über welches die Befehle abgesetzt werden.



```
Terminal - produnis@Hyrule:~
Datei Bearbeiten Ansicht Terminal Reiter Hilfe
R version 4.0.4 (2021-02-15) -- "Lost Library Book"
Copyright (C) 2021 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu (64-bit)

R ist freie Software und kommt OHNE JEGLICHE GARANTIE.
Sie sind eingeladen, es unter bestimmten Bedingungen weiter zu verbreiten.
Tippen Sie 'license()' or 'licence()' für Details dazu.

R ist ein Gemeinschaftsprojekt mit vielen Beitragenden.
Tippen Sie 'contributors()' für mehr Information und 'citation()',
um zu erfahren, wie R oder R packages in Publikationen zitiert werden können.

Tippen Sie 'demo()' für einige Demos, 'help()' für on-line Hilfe, oder
'help.start()' für eine HTML Browserschnittstelle zur Hilfe.
Tippen Sie 'q()', um R zu verlassen.

> 2*2
[1] 4
> sqrt(25)
[1] 5
>
```

Abb. 1.1:  Konsole

Dies kann am Anfang abschreckend wirken, da wir es eher gewöhnt sind, Programme über eine graphische Benutzeroberfläche zu steuern.

Seit 2012 wird dafür **RStudio** entwickelt, welches eine funktionsreiche Entwicklungsumgebung für  bereitstellt. Mittlerweile hat sich **RStudio** als Open Source Standardanwendung für statistische Analysen durchgesetzt.

Sie werden nicht drum herumkommen, Ihre Befehle weiterhin aufzuschreiben und auszuführen, aber **RStudio** bietet Ihnen eine sehr hilfreiche und komfortable Arbeitsumgebung, die viele Arbeitsschritte erleichtern wird.

Auch in  hat sich einiges getan, vor allem in den letzten Jahren. Mit der Paketesammlung *Tidyverse* (siehe Kapitel 1.6 auf Seite 147) und dessen strenger Umsetzung der *Tidy Data* sowie kosequenter Nutzung der *Pipe* (siehe Abschnitt 1.6.2) hat sich ein eigenständiger Dialekt entwickelt, der immer mehr Anhänger findet. Die grundlegenden Schritte der Datenmanipulation und graphischen Darstellung sind im *Tidyverse* viel einfacher als mit klassischen .

Hausmitteln. Die Funktionalität der Pipe wurde mit Version 4.1 ins klassische  übernommen (siehe Abschnitt 1.2.3.8).

In diesem Buch lernen Sie zuerst das “klassische” , denn die grundlegenden Konzepte sind immer noch gültig, und auch die Funktionen für statistische Analysen sind immer noch die selben. Die Abschnitte zum Umgang mit Datensätzen sowie zur `plot()`-Funktion sind aber nur rudimentär gehalten, da dies mit den modernen Paketen `dplyr` und `ggplot2` wesentlich einfacher funktioniert.

1.2.2 RStudio

Die Nutzeroberfläche von **RStudio** besteht aus vier Bereichen (siehe Abbildung 1.2), die nach ihrer hauptsächlichen Funktion benannt sind. Jeder Bereich bietet aber über seine Reiter weitere Anwendungsmöglichkeiten (die Sie jetzt evtl. noch nicht verstehen).

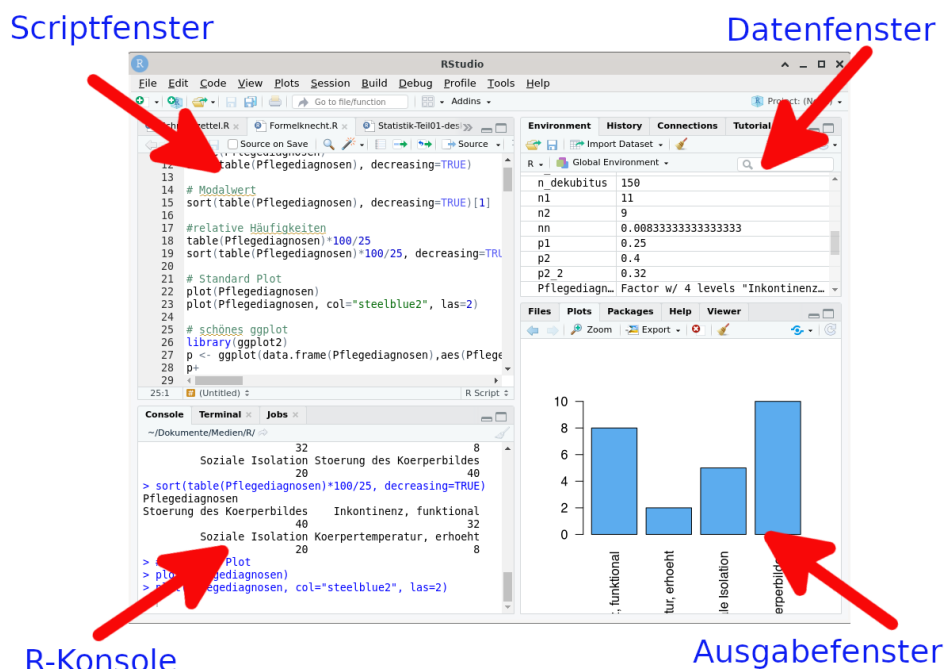


Abb. 1.2: RStudio Bereiche

Oben links befindet sich das “Scriptfenster”. In diesem Bereich werden Ihre Scriptdateien angezeigt und Sie können sie dort editieren. In den Scriptdateien schreiben Sie Ihre -Befehle nacheinander auf, je nachdem, welche Auswertungen oder Datentransformationen Sie fahren, oder welche Diagramme Sie plotten möchten (siehe Kapitel 2). Ihr Code wird dabei von **RStudio** farblich aufgehübscht, was ihn wesentlich übersichtlicher macht.

Rechts daneben ist das “Datenfenster”. Hier werden alle Variablen und Datenframes angezeigt, die derzeit im Speicher geladen sind. Durch Doppelklick auf einen Datensatz werden die Daten im “Scriptfenster” angezeigt. Über den Reiter **History** können Sie alle Befehle, die Sie in der -Konsole abgesetzt haben, einsehen. Über den Reiter **Tutorials** haben Sie Zugriff auf zahlreiche Anleitungsangebote zu **RStudio**.

Unten links befindet sich die -Konsole. Dieser Bereich entspricht der “normalen” -Installation. Hier können Sie von Hand Befehle eingeben, oder aber Befehle aus dem Scriptfenster ausführen lassen. Die Konsole unterscheidet farblich zwischen Eingabe und Ausgabe, was sehr zur Übersicht beiträgt. Der Reiter **Terminal** öffnet eine Systemkonsole (unter Linux sehr hilfreich), und unter **Jobs** werden z.B. Installationsprozesse von Zusatzpaketen dargestellt.

Unten rechts ist das “Ausgabefenster”. Hier werden unter anderem Plots und Hilfetexte angezeigt. Das Fenster verfügt zudem über einen Dateibrowser (Reiter **Files**), von dem aus Sie bequem Dateien oder Datensätze aus Ihrem Arbeitsverzeichnis laden können. Über den Reiter **Packages** können Sie Zusatzpakete aktivieren und deaktivieren.

Für die ersten Schritte in **RStudio** empfehlen wir folgendes Youtube-Video:
<https://www.youtube.com/watch?v=tyvEHQszZJs>

1.2.2.1 RStudio anpassen

RStudio lässt eine Menge an Anpassungen zu, so dass Sie die Software möglichst gut an Ihren Workflow anpassen können. Folgende zwei Einstellungen empfehlen wir nach der Installation vorzunehmen.

- **Arbeitsverzeichnis setzen:** Unter Tools → Global Options gelangen Sie ins allgemeine Einstellungs Menü. Unter dem Reiter  **General** → **Basic** lässt sich das Arbeitsverzeichnis (working directory) festlegen (Abbildung 1.3). In dieses Verzeichnis legt  standardmäßig alle Diagramme, Scripte sowie den aktuellen Workspace (also alle Variablen und Datensätze, die derzeit geladen sind). Es bietet sich an, ein eigenes Verzeichnis für  zu erstellen und hier in RStudio auszuwählen.

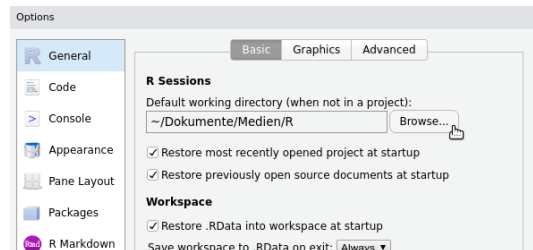


Abb. 1.3: Arbeitsverzeichnis setzen

- **Tastaturkürzel:** Unter Tools → Modify Keyboard Shortcuts öffnet sich das Dialogfenster zum Festlegen der Tastaturkürzel. Hier geben wir im Feld Filter “exec” ein (Abbildung 1.4). Bei der Aktion **Execute Current Line** setzen wir das Tastaturkürzel **[STRG]+[R]** (Abbildung 1.5). Die Einstellung bewirkt, dass im Scriptfenster die aktuelle Zeile per **[STRG]+[R]** ausgeführt wird. Dies ist ziemlich hilfreich.

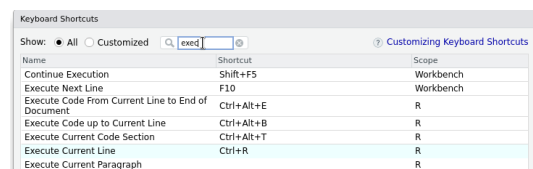


Abb. 1.4: Keyboard Shortcuts filtern

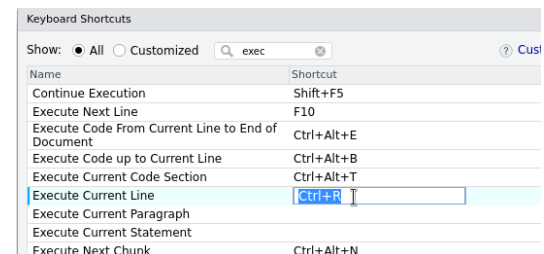


Abb. 1.5: **[STRG]+[R]** setzen

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

An dieser Stelle sei darauf hingewiesen, dass das Aussehen von RStudio angepasst werden kann. Vielleicht möchten Sie auf einen “augenfreundlicheren” Modus umschalten, in welchem der Hintergrund dunkler ist. Klicken Sie hierfür in der Menüleiste auf **Tools** → **Global Options** klicken.

Wählen Sie hier den Bereich **Appearance** aus (Abbildung 1.6).

Es werden Ihnen zahlreiche “Themes” zur Auswahl gestellt. Probieren Sie alle einmal aus. Als dunkles Theme verwende ich persönlich gerne **Dracula** (Abbildung 1.7).

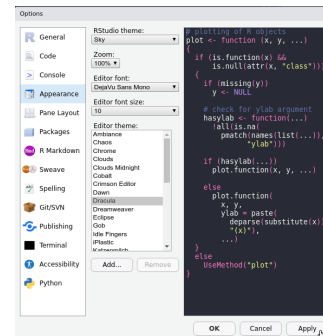


Abb. 1.6: Erscheinungsbild ändern

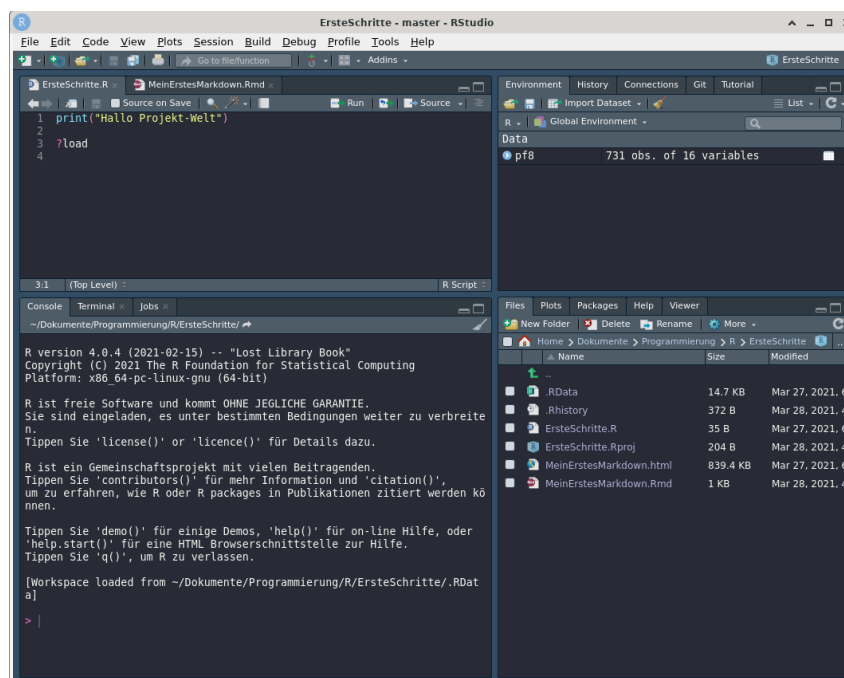


Abb. 1.7: Theme Dracula

1.2.2.2 Eine Sitzung starten

Starten Sie RStudio. Es begrüßt Sie die RStudio Arbeitsfläche. Das Scriptfenster oben links ist noch geschlossen, so dass der gesamte linke Bereich von der R-Konsole eingenommen wird (Abb. 1.8).

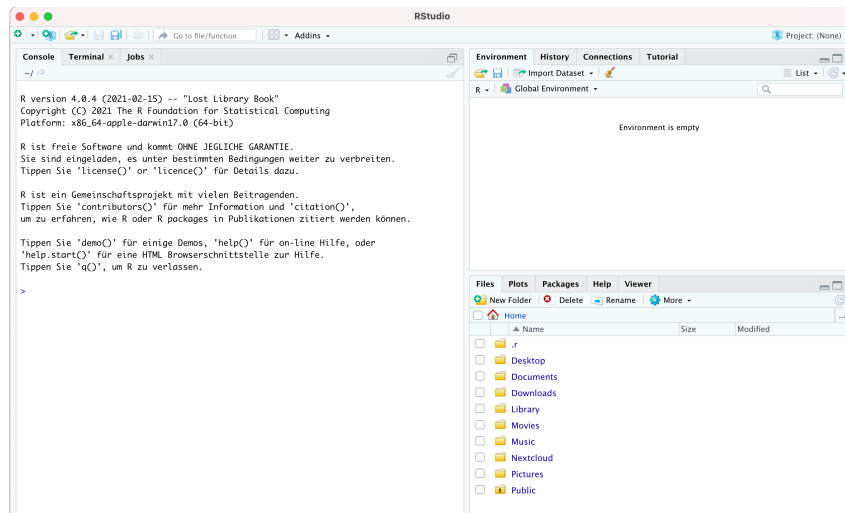


Abb. 1.8: Neue Sitzung gestartet

Als erstes legen Sie eine neue Scriptdatei an. Klicken Sie oben links auf das grüne + Zeichen und wählen Sie R Script (Abbildung 1.9).

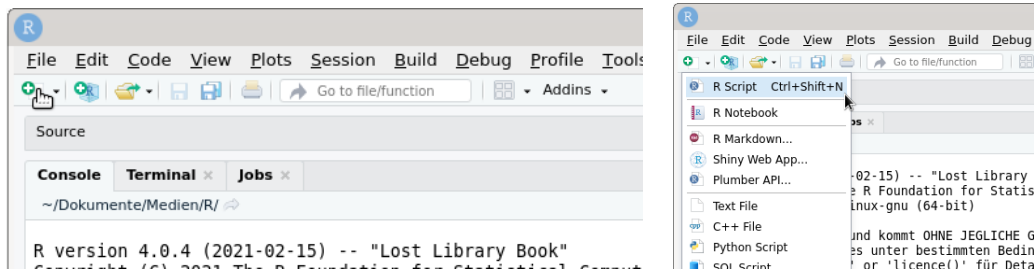


Abb. 1.9: eine neue Scriptdatei anlegen

Wie Abbildung 1.9 zeigt, können Sie auch die Tastenkombination [STRG] + [SHIFT] + [N] verwenden, um eine neue Scriptdatei zu erzeugen.

In dieser Scriptdatei werden wir alle Befehle hinterlegen. Für gewöhnlich beginnt man bei Programmierübungen mit einem “Hallo Welt” Beispiel. Schreiben Sie in die Scriptdatei den Befehl:

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 print("Hallo Welt")
```

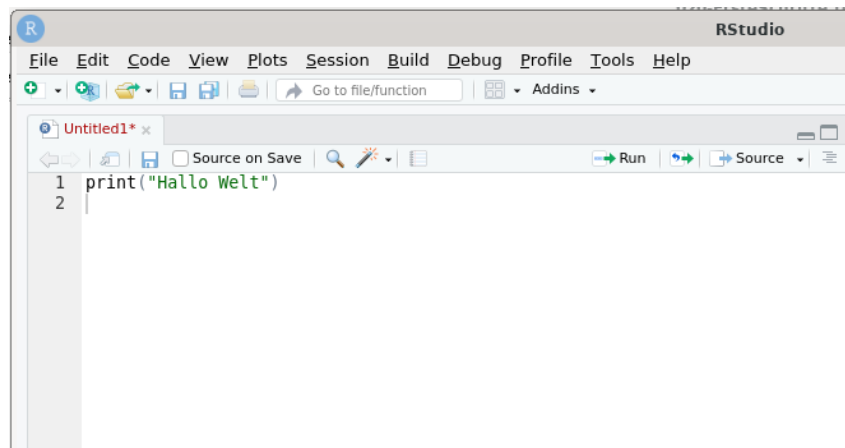


Abb. 1.10: "Hallo Welt" -Beispiel

Diesen Befehl wollen wir nun in der  -Konsole ausführen. Das kann auf mehrere Arten geschehen:

- gehen Sie mit dem Cursor in die Zeile des Befehls. Drücken Sie die (auf Seite 5 angelegte) Tastenkombination [STRG] + [R].
- oder gehen Sie mit dem Cursor in die Zeile des Befehls und klicken Sie auf den →Run-Knopf (siehe Abbildung 1.11).

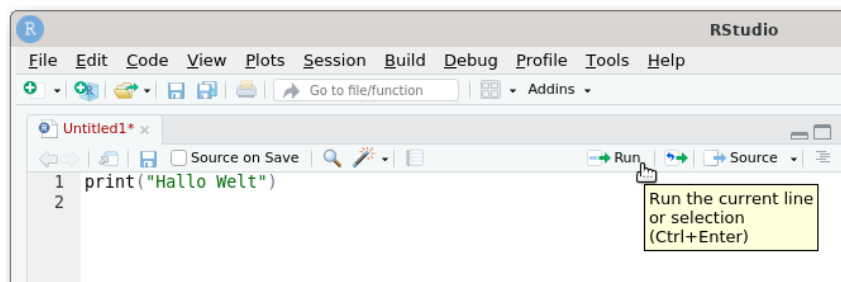


Abb. 1.11: Run-Knopf

- oder gehen Sie mit dem Cursor in die Zeile des Befehls und drücken Sie die (voreingestellte) Tastenkombination [STRG] + [ENTER].

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

Alle drei Aktionen bewirken, dass der Befehl `print("Hallo Welt")` in der R-Konsole unten links ausgeführt wird (siehe Abbildung 1.12).

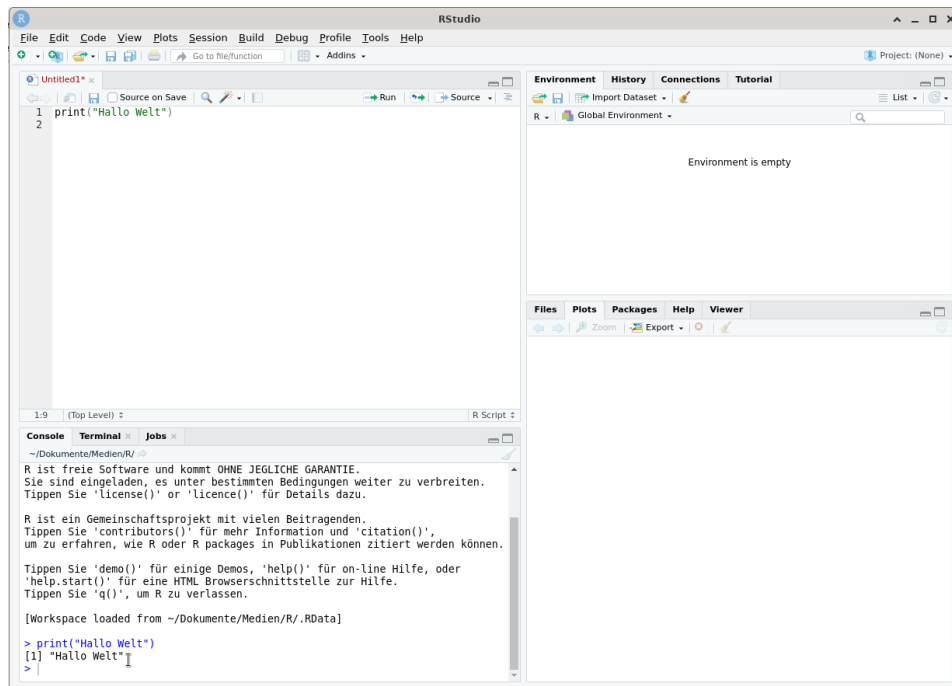


Abb. 1.12: “Hallo Welt” -Ausgabe in der R-Konsole unten links

Und R antwortet entsprechend in der Konsole mit

```
[1] "Hallo Welt"
```

Sie können die Befehle auch jederzeit direkt unten in die R-Konsole eingeben und ausführen.

Prinzipiell sollten Sie aber alles in der Scriptdatei aufschreiben und von dort ausführen. So können Sie leicht Ihre Befehle anpassen oder auch komplizierte R-Sprüche nach und nach aufbauen. Manchmal kann es aber auch nützlich sein, ein kurzes `?print` (dazu später mehr) direkt unten in der Konsole einzugeben. Sie werden “den Dreh” sehr schnell heraus haben.

Wir sollten so häufig wie möglich unsere Scriptdatei abspeichern. Hierzu drücken Sie die Tastenkombination [STRG] + [S], oder Sie klicken auf das Diskettensymbol (siehe Abbildung 1.13).

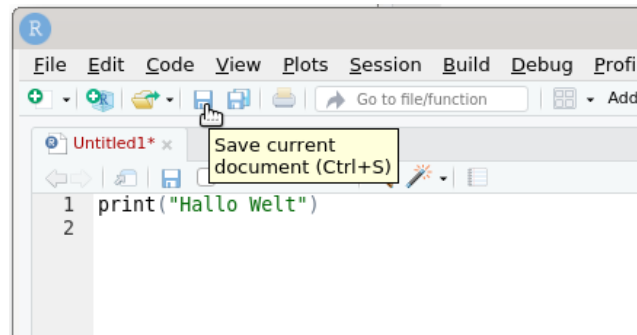


Abb. 1.13: Scriptdatei speichern

Derzeit heisst unsere Scriptdatei noch `Untitled1`. Geben Sie Ihrer Scriptdatei einen schönen Dateinamen mit der Dateiendung `.R`, also z.B. `ErsteSchritte.R`. Der neue Dateiname wird nach dem Speichern über dem Script angezeigt (Abbildung 1.14).

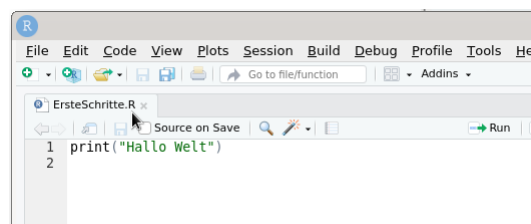


Abb. 1.14: neuer Dateiname `ErsteSchritte.R`

Mit dem `#`-Zeichen können Zeilen *auskommentiert* werden. Das bedeutet, dass alles, was hinter einem `#` steht, von `R` ignoriert wird (siehe Abbildung 1.15). `RStudio` hebt auskommentierte Stelle farblich hervor (Abbildung 1.15, alles Auskommentierte ist hellgrün dargestellt), so dass sie leicht erkannt werden können.

Das Auskommentieren mit `#` ist hilfreich, um den Code zu *kommentieren*, also zu beschreiben, was die nachfolgenden Zeilen machen sollen. So bleibt Ihr Code verständlich und wird auch für andere nachvollziehbar. Zum anderen kann es hilfreich sein, einzelne Befehlszeilen mittels `#` zu “deaktivieren”, z.B. wenn man auf Fehlersuche ist.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

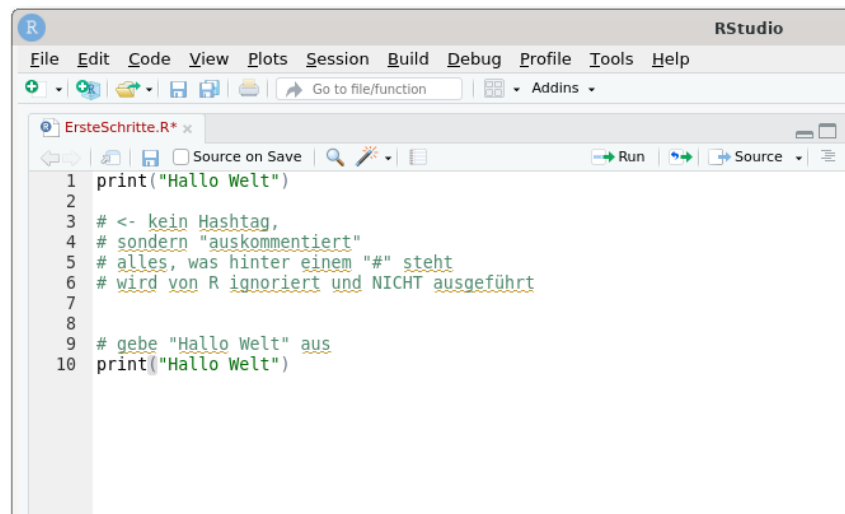


Abb. 1.15: auskommentieren mit #

In Abbildung 1.15 sehen Sie auch, dass der Dateiname der Scriptdatei `ErsteSchritte.R*` in rot dargestellt ist und mit einem `*` endet. Auf diese Weise teilt RStudio Ihnen mit, dass es ungespeicherte Änderungen an der Scriptdatei gibt, die Sie evtl. speichern möchten. Das verwundert nicht, denn wir haben ja gerade jede Menge Kommentare in unser Script geschrieben. Mit der Tastenkombination `[STRG] + [S]` speichern wir die Scriptdatei. Wie Abbildung 1.16 zeigt, ist der Dateiname nun nicht mehr rot und hat auch kein `*` mehr.

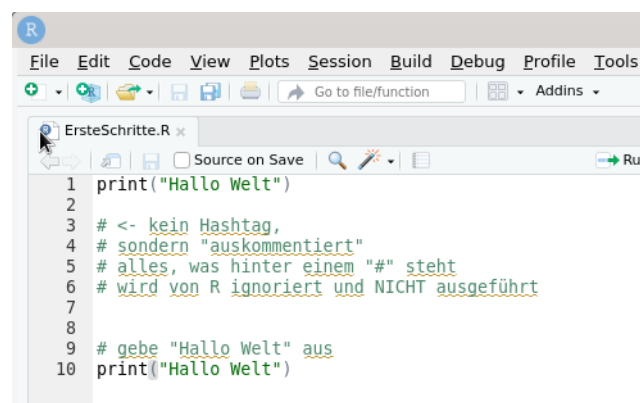


Abb. 1.16: alle Änderungen gespeichert

1.2.2.3 Projekte in RStudio

RStudio erlaubt es Ihnen, so genannte “Projekte” zu erstellen.

Ein neues Projekt startet mit einer leeren -Konsole und einem (fast) leeren Arbeitsverzeichnis. Sie können wie gewohnt Daten laden, bearbeiten und Ihre Auswertungen fahren. Wenn Sie RStudio schließen, wird Ihr Arbeitsstand im Projektordner gespeichert. Beim nächsten Öffnen steht dann alles wieder so zur Verfügung, wie Sie es verlassen haben.

Es empfiehlt sich sehr, für jedes “Projekt”, das Sie mit  durchführen möchten, ein eigenes RStudio-Projekt zu erstellen. Dies erleichtert die Arbeit unheimlich. Da jedes Projekt in seinem eigenen Dateiordner “lebt”, behalten Sie die Übersicht über Daten, Dateien und Diagramme. Sie können zwischen den Projekten wechseln, ohne dass Ihre Arbeitsspeicher durcheinandergeraten, oder Variablen überschrieben werden. Sofern Sie konsequent alle Dateien in den jeweiligen Ordnern gespeichert lassen, können Sie sich lästige Dateipfadangaben sparen, und Sie können den Projektordner einfach mit Kolleginnen und Kollegen teilen (indem Sie ihn kopieren), und in *deren* RStudio wird alles so funktionieren wie bei Ihnen.

Ich persönlich habe in meinem Dokumenteordner einen Unterordner R, in welchem sich alle Projekte befinden. Es ist aber natürlich auch möglich, Ihre einzelnen Projektordner an beliebigen Stellen des Dateisystems zu platzieren. Projektordner können auch “im Nachhinein” beliebig verschoben werden.

Legen Sie nun ein neues Projekt in RStudio an. Klicken Sie in der Menüleiste auf **File** → **New Project**....

Es öffnet sich ein neues Fenster, in welchem Sie gefragt werden, ob Sie einen neuen Projektordner im Dateisystem anlegen möchten, oder ob Sie einen bestehenden Ordner nutzen möchten. Da Sie (wahrscheinlich) noch keine -Projekte durchgeführt haben, klicken Sie auf **New Directory** (Abbildung 1.17).

In RStudio können Sie viel machen, deshalb ist die Auswahlliste des **Project Type** so lang. Wählen Sie den ersten Eintrag **New Project** aus. Geben Sie dem Projekt einen Namen. In Abbildung 1.17 haben ich “ErsteSchritte” genannt. Darunter wird der Dateipfad angegeben. Ignorieren Sie die weiteren Häkchenoptionen und klicken Sie unten auf den **Create Project**-Knopf.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

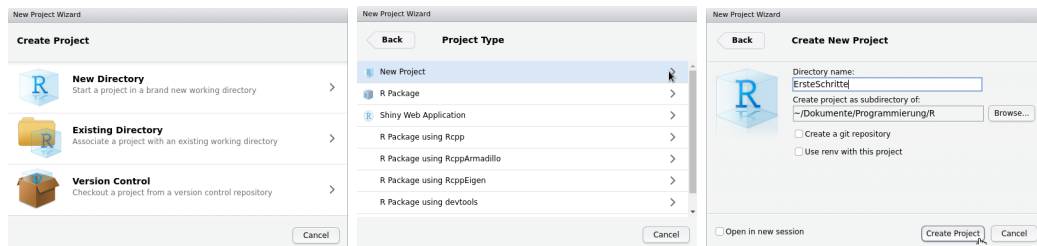


Abb. 1.17: Neues Projekt erstellen

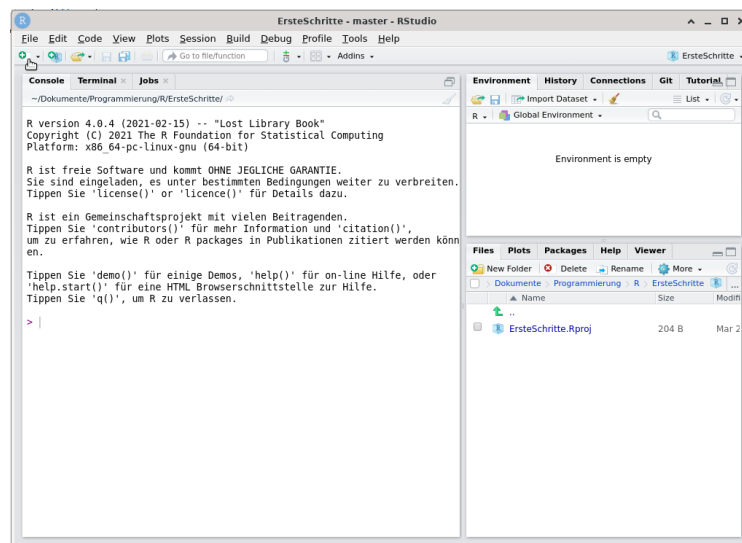


Abb. 1.18: Neuerstelltes Projekt

Es begrüßt Sie ein “frisches” RStudio-Fenster (Abbildung 1.18). Wiederholen Sie nun die Schritte aus dem “Hallo Welt”-Beispiel. Das heisst, legen Sie eine neue Scriptdatei an, indem Sie auf das grüne + klicken (siehe Mauszeiger in Abbildung 1.18), und schreiben Sie den Befehl hinein:

```
1 print("Hallo Projekt-Welt")
```

Führen Sie den Befehl mittel Run-Knopf oder der Tastenkombination [STRG] + [R] aus, und speichern Sie Ihre Scriptdatei anschließend unter dem Namen **ErsteSchritte.R** ab, indem Sie auf das Diskettensymbol klicken oder [STRG] + [S] drücken.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

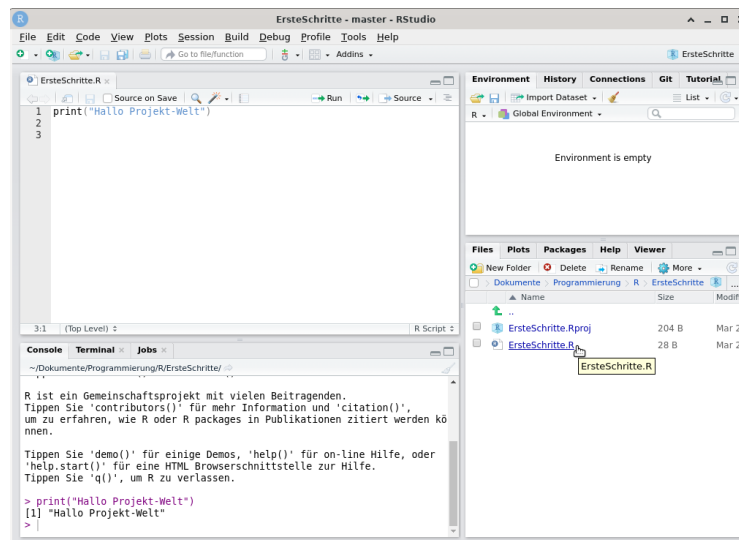


Abb. 1.19: Neues Projekt mit Scriptdatei

Wie Sie sehen, wird Ihre Scriptdatei nun im Ausgabefenster angezeigt (Abbildung 1.19). Daneben sehen Sie die Projektdatei `ErsteSchritte.Rproj`. Wenn Sie aus Ihrem Dateibrowser auf diese Datei klicken, öffnet sich automatisch RStudio und es wird der Arbeitsstand des Projektes geladen.

Rechts über dem Datenfenster wird Ihr derzeit aktives Projekt angezeigt (in Abbildung 1.20 `ErsteSchritte`).

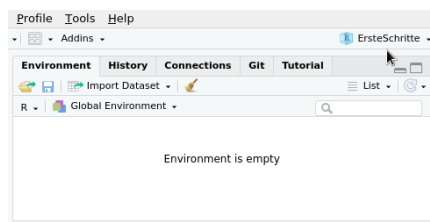


Abb. 1.20: Projekte verwalten

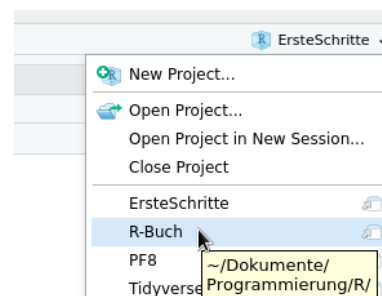


Abb. 1.21: letzte Projekte

Wenn Sie hier klicken, können Sie neue Projekte anlegen, und ihre zuletzt geöffneten Projekte werden Ihnen angezeigt. So können Sie bequem zwischen Ihren Projekten wechseln (Abbildung 1.21).

1.2.3 Grundlegende Befehle in R

Zu jedem -Befehl lässt sich eine Hilfeseite anzeigen. Diese gibt grundlegende Informationen über Funktionsweisen des Befehls sowie die zugelassenen Befehlsoptionen. Die meisten Hilfeseiten enden mit ein paar Beispielen. Die Hilfeseiten erreicht man mit dem Befehl `help()` oder kurz `?`, wobei der Befehl, dessen Hilfsseite wir aufrufen möchten, innerhalb der Befehlsklammern stehen muss.

```
1 # gib Hilfeseite für Funktion "mean" aus
2 help(mean)
3
4 # oder einfach
5 ?mean
```

Wenn Sie den Befehl `help(mean)` z.B. mit [STRG] + [R] ausführen, oder direkt unten in die -Konsole eingeben, zeigt RStudio die Hilfeseiten im unteren rechten Viertel an (siehe Abbildung 1.22).

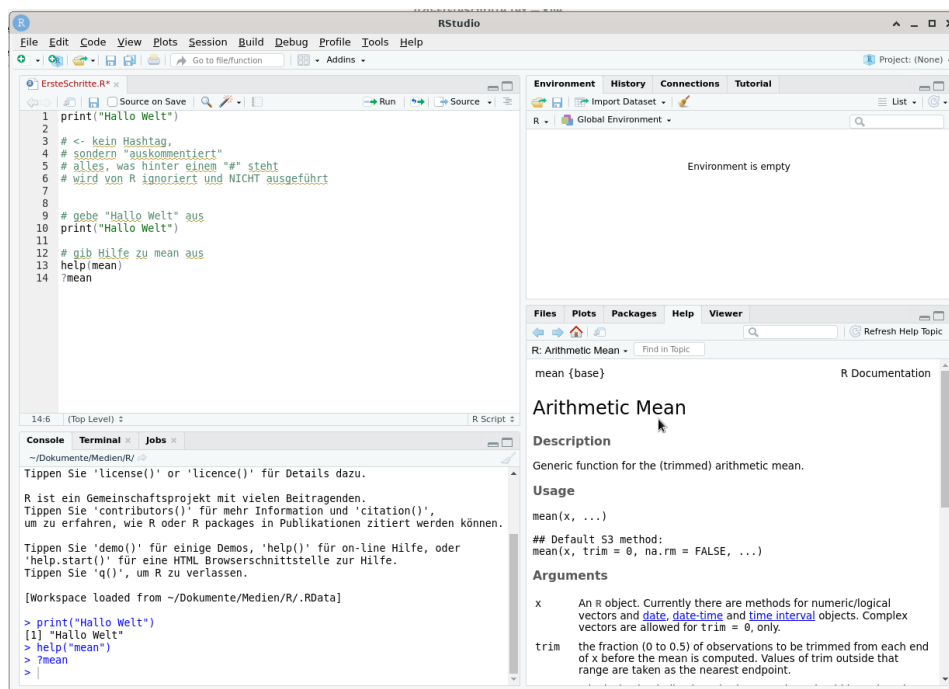


Abb. 1.22: Hilfeseiten stehen unten rechts

In diesem Buch werden die  Befehle nie vollständig mit all ihren Optionen und Einstellungsmöglichkeiten vorgestellt. Daher bietet es sich an, bei jedem “neuen” Befehl, den man lernt oder verwendet, per `help()` oder `?` die Anleitungssseite zumindest zu überfliegen.

```

1  # bei help() steht die Funktion in Klammern
2  # gib Hilfeseite für Funktion "c()" aus
3  help(c)
4
5  # Bei ? wird sie ohne Klammern direkt drangeschrieben
6  ?c
7  ?mean
8  ?median

```

Mit `quit()` oder kurz `q()` wird die -Sitzung beendet.

```

1  # Beende die R-Sitzung
2  quit()
3
4  # Beende die R-Sitzung
5  q()
6
7  # Rufe Hilfeseite auf
8  ?quit

```

Wenn Sie den Befehl `quit()` in der -Konsole von RStudio ausführen, wird auch RStudio beendet.

1.2.3.1 Rechnen mit R

 funktioniert unter anderem wie ein mächtiger Taschenrechner. Folgende grundlegende mathematische Operationen können mit  ausgeführt werden:

Operation	 -Symbol
Addition	+
Subtraktion	−
Multiplikation	*
Division	/
Potenzieren	^ oder **
Wurzel	<code>sqrt(x)</code>

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

Hier ein paar Beispiele:

```
1 # Addition
2 2+8
```

```
[1] 10
```

```
1 # Multiplikation
2 4*4
```

```
[1] 16
```

```
1 # Subtraktion
2 9-7
```

```
[1] 2
```

```
1 # Division
2 6/2
```

```
[1] 3
```

```
1 # Exponent
2 2^3
```

```
[1] 8
```

```
1 # Quadratwurzel
2 sqrt(25)
```

```
[1] 5
```

```
1 # Exponent anders notiert
2 2**3
```

```
[1] 8
```

```
1 # Lange Formel
2 sqrt( (2*10) / (20/2) ) - 9
```

```
[1] -7.585786
```

```
1 # Befehl schlägt fehl!
2 2,3 - 9
```

```
Fehler: Unerwartete(s) ',', in "2,"
```



Achtung, Anfängerfehler!



An der letzten Ausgabe zeigt sich ein wesentliches Merkmal von :

Dezimalstellen werden mit einem Punkt UND NICHT mit einem Komma dargestellt.

Dies ist ein häufiger Anfängerfehler und meist der Grund, warum am Anfang etwas nicht funktioniert.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # Dezimalstellen mit .  
2 4.25 * 6.25
```

```
[1] 26.5625
```

Vielleicht haben Sie sich schon gefragt, wieso vor jeder Ausgabe eine [1] steht. In  beginnt jede Ausgabezeile mit der Position des Wertes, der als nächstes ausgegeben wird. Da wir derzeit nur Rechenoperationen durchführen, die genau ein Ergebnis haben, ist die Position unseres Ergebniswertes immer [1]. Hat man längere Datenreihen, z.B. das Gewicht von 10 Probanden in kg, dann könnte die Ausgabe so aussehen:

```
[1] 102.20 76.83 98.50 86.55 121.93  
[6] 89.21 123.22 78.23 98.33 83.37
```

Die zweite Ausgabezeile beginnt mit einer [6], was bedeutet, dass nachfolgend die 6. Position der Wertereihe angezeigt wird.

Das Ziehen einer Wurzel erfolgt über die Funktion `sqrt()`. Damit Funktionen arbeiten können, benötigen sie ein “Argument”. Das Argument ist in unserem Fall die Zahl 25, da wir die Wurzel aus 25 ziehen wollen.

```
1 # Wurzel ziehen  
2 sqrt(25)
```

```
[1] 5
```

Argumente werden der Funktion in runden Klammern übergeben. So erklärt sich der Befehl `sqrt(25)`.

Weitere mathematische Funktionen sind:

Funktion	Befehl	Beispiel	Ausgabe
Exponentialfunktion	<code>exp()</code>	<code>exp(2)</code>	<code>[1] 7.389056</code>
natürlicher Logarithmus	<code>log()</code>	<code>log(2)</code>	<code>[1] 0.6931472</code>
Logarithmus zur Basis 2	<code>log2()</code>	<code>log2(3)</code>	<code>[1] 1.584963</code>
Logarithmus zur Basis 10	<code>log10()</code>	<code>log10(3)</code>	<code>[1] 0.4771213</code>
Sinus	<code>sin()</code>	<code>sin(2)</code>	<code>[1] 0.9092974</code>
Kosinus	<code>cos()</code>	<code>cos(2)</code>	<code>[1] -0.4161468</code>
Tangens	<code>tan()</code>	<code>tan(2)</code>	<code>[1] -2.18504</code>
Absolutwert	<code>abs()</code>	<code>abs(-7)</code>	<code>[1] 7</code>

Nachkommastellen

 zeigt standardmäßig 6 Nachkommastellen an. Intern rechnet  mit “dopelter Genauigkeit”. Damit sind Nummern mit ca. 17 Stellen (1 Ziffer + 16 Nachkommastellen oder eben eine 17 stellige Zahl ohne Nachkommastellen) repräsentierbar. Möchte man die Nachkommastellen entsprechend angezeigt bekommen, so kann man dies mit der `options`-Funktion einstellen:

```
1 # gib ab jetzt 16 Nachkommastellen aus
2 options(digits=17)
3 # Wurzel aus 3
4 sqrt(3)
```

```
[1] 1.7320508075688772
```

Nach dieser Eingabe liefert  Ergebnisse mit bis zu 16 Nachkommastellen. Wir stellen den Wert wieder zurück auf 6 Nachkommastellen.

```
1 # gib ab jetzt wieder "nur" 6 Nachkommastellen aus
2 options(digits=7)
```

Runden

Ziehen wir die Wurzel aus 3, erhalten wir ein Ergebnis mit 6 Nachkommastellen.

```
1 # Wurzel aus 3
2 sqrt(3)
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] 1.732051
```

Dieses Ergebnis kann mit der Funktion `round()` gerundet werden.

```
1 # Runde auf 2 Nachkommastellen
2 round(sqrt(3), digits=2)
3
4 # oder einfach
5 round(sqrt(3), 2)
```

```
[1] 1.73
```

```
1 # Runde auf ganze Zahl
2 round(sqrt(3))
```

```
[1] 2
```

Neben `round()` existieren noch die Spezialfunktionen `ceiling()` und `floor()`.

Die Funktion `ceiling()` rundet immer zur nächsten ganzen Zahl **auf**, und die Funktion `floor()` rundet immer zur nächsten ganzen Zahl **ab**.

```
1 # ceiling rundet immer auf
2 ceiling(sqrt(3))
```

```
[1] 2
```

```
1 # floor rundet immer ab
2 floor(sqrt(3))
```

```
[1] 1
```

```
1 # ceiling rundet immer auf
2 ceiling(1.0008)
```

```
[1] 2
```

```
1 # floor rundet immer ab
2 floor(1.9999)
```

```
[1] 1
```

 nutzt die wissenschaftliche Schreibweise sehr großer und kleiner Zahlen (so genannte *Zehnerpotenzen*).

```
1 # erzeuge eine sehr große Zahl
2 987654321 * 987654321
```

```
[1] 9.754611e+17
```

Um die Ergebniszahl vor der Zehnerpotenz zu runden, kann die Funktion `format()` genutzt werden.

```
1 # runde vor der Zehnerpotenz
2 format(987654321 * 987654321, digits=2)
```

```
[1] "9.8e+17"
```

1.2.3.2 Variablen

Variablen dienen der Speicherung von Werten oder komplexen Daten (streng genommen spricht man in $\mathbb{R}$ aber immer von “Objekten”). Variablen bestehen aus einem Namen und dem zugewiesenen Wert (bzw. Werten). Der Name einer Variable in $\mathbb{R}$ darf nicht mit einer Zahl beginnen, das heisst Variablennamen wie `1stTry` sind nicht erlaubt. Ein Wert kann einer Variable auf zwei Weisen zugewiesen werden:

```
1 # weise Variable "x" den Wert 8 zu
2 x <- 8
```

Die in $\mathbb{R}$ gängige Variante ist das Zuweise mittels “Pfeil”, den man mit `<` und - nachahmt.

Die Zuweisung per Gleichheitszeichen funktioniert aber ebenso.

```
1 # weise Variable "x" den Wert 8 zu
2 x <- 8
3
4 # funktioniert auch mit "="
5 x = 8
```

Wir haben nun eine Variable `x` mit dem Wert `8`. Beachten Sie, dass $\mathbb{R}$ zwischen Groß- und Kleinschreibung unterscheidet. Unsere Variable heisst `x` und nicht `X`!

In **RStudio** wird uns nun im Datenfenster (oberes rechtes Viertel) die Variable `x` mit ihrem Wert `8` angezeigt (Abbildung 1.23).

Den Wert einer Variable können wir ausgeben lassen, indem wir einfach ihren Namen in die $\mathbb{R}$ -Konsole schreiben

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

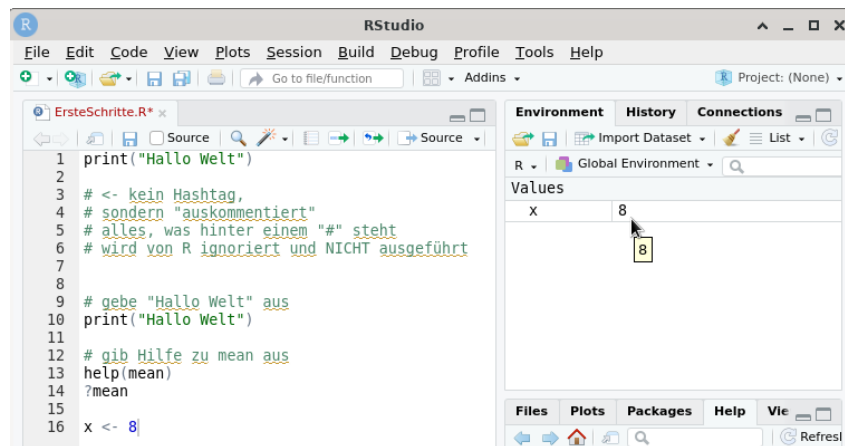


Abb. 1.23: Datenfenster mit Variable x

```
1 # gib Wert von x aus
2 x
```

```
[1] 8
```

Wie erwähnt unterscheidet R zwischen Groß- und Kleinschreibung, und so erhalten wir einen Fehler wenn wir eingeben:

```
1 # großes "X", kein kleines "x"
2 X
```

```
Fehler: Objekt 'X' nicht gefunden
```

Mit Variablen kann auch gerechnet werden:

```
1 # erzeuge Variable "y" mit Wert 10
2 y <- 10
3 # erzeuge Variable "z" mit Wert 5
4 z <- 5
5 # rechne ein bisschen
6 x+y+z
```

```
[1] 23
```

```
1 # noch ein bisschen rechnen
2 x/(y+z)
```

```
[1] 0.5333333
```

```
1 # speichere Ergebnis in weitere Variable "Dummy"
2 Dummy <- x+(y^z)
3 # Gib Wert(e) von Dummy aus
4 Dummy
```

```
[1] 100008
```

Wertereihen

Wir können einer Variable auch eine Wertereihe (also mehrere Werte) zuweisen - spätestens jetzt sollten wir nicht mehr von einer “Variable” sprechen, sondern von einem “Objekt”. Dies erfolgt für gewöhnlich mit der Funktion `c()` (für *concatenate*, “aneinanderreihen”), indem man die einzelnen Werte mit einem Komma getrennt der Funktion übergibt.

```
1 # weise Wertereihe 1 bis 10 mit c() zu
2 a <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
3 # Gib Werte von "a" aus
4 a
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Mit `c()` kann auch Text, also z.B. Wörter, übergeben werden.

```
1 # es müssen keine Zahlen sein
2 b <- c("eins", "zwei", "drei", "vier", "fünf", "sechs", "sieben", "acht", "neun", "zehn")
3 # Gib Werte von "b" aus
4 b
```

```
[1] "eins"  "zwei"  "drei"  "vier"
[5] "fünf"  "sechs" "sieben" "acht"
[9] "neun"  "zehn"
```

Der Text muss in Anführungszeichen gesetzt werden, da  ansonsten nicht das Wort "fünf" versteht, sondern nach einem Objekt namens `fünf` sucht. Denn Variablen und Objekte können ebenfalls aneinandergereiht werden.

```
1 # Variablen aneinanderreihen
2 c(x, y, z)
```

```
[1] 8 10 5
```



Achtung, Anfängerfehler!



Wichtig ist, dass innerhalb der `c()`-Kette nur Werte des selben Typs (entweder Zahlen oder Text) angegeben werden. Mischt man Text mit Zahlen, wandelt `c()` die Wertereihe auf den “kleinsten gemeinsamen Nenner” um, und das ist immer “Text”.

```
1 # c() fällt auf den "kleinsten gemeinsamen Nenner" zurück
2 c(x, y, z, b)
```

```
[1] "8"      "10"     "5"      "eins"   "zwei"   "drei"   "vier"
[8] "fünf"   "sechs"  "sieben" "acht"   "neun"   "zehn"
```

Aus den Zahlen 8, 10 und 5 sind die *Wörter* "8", "10" und "5" geworden, und mit denen kann man nun keine Rechenoperationen mehr durchführen. Das Mischen der Wertetypen ist ein weiterer häufiger Anfängerfehler.

Mit der Funktion `length()` kann die Länge der Wertereihe bestimmt werden

```
1 # länge von Objekt "a"
2 length(a)
```

```
[1] 10
```

Objekt `a` besteht aus 10 Werten.

```
1 # länge von Objekt "Dummy"
2 length(Dummy)
```

```
[1] 1
```

Objekt `Dummy` besteht aus 1 Wert.

Im Datenfenster von **RStudio** (rechtes oberes Viertel) werden alle Objekte angezeigt (Abbildung 1.24).

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

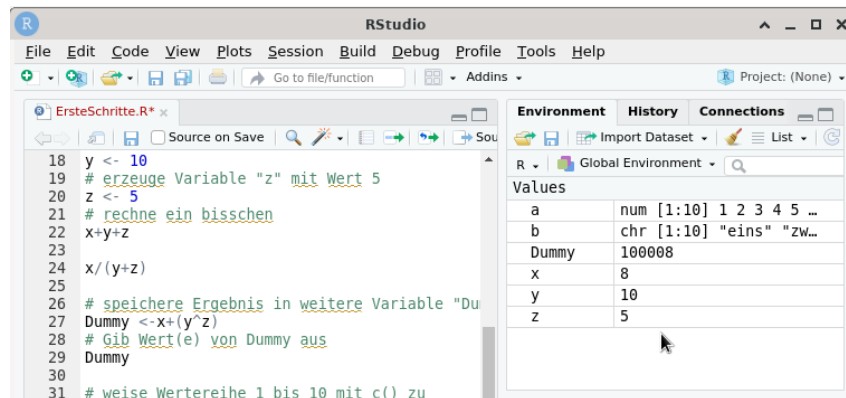


Abb. 1.24: Datenfenster zeigt alle Objekt

Wir sehen dort unsere Objekte a, b, Dummy, x, y und z. Rechts daneben zeigt RStudio zusammenfassende Informationen zu den Objekte an. So sieht man direkt, dass der Wert von Dummy = 100008 ist, oder dass a zehn Werte enthält.

In der -Konsole können alle vorhandenen Objekte mit der Funktion `ls()` angezeigt werden.

```
1 # zeige alle Objekte an
2 ls()
[1] "a" "b" "Dummy" "x" "y" "z"
```

Objekte lassen sich mit dem Befehl `rm()` löschen.

```
1 # lösche Objekt "Dummy"
2 rm(Dummy)
3
4 # zeige alle Objekte an
5 ls()
[1] "a" "b" "x" "y" "z"
```

Um *alle* Objekte zu löschen, verknüpfen wir den `ls()`-Befehl mit `rm()`. So ändert sich der Befehl in:

```
1 # lösche ALLE Objekte
2 rm(list=ls())
3
4 # zeige alle Objekte an
5 ls()
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
character(0)
```

In RStudio löschen Sie alle Objekte aus dem Arbeitsspeicher, indem Sie im Datenfenster auf den kleinen Besen klicken (Abbildung 1.25).

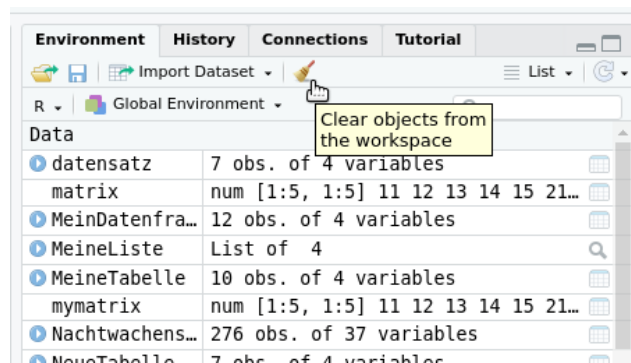


Abb. 1.25: der Besen leert den Workspace

1.2.3.3 Wertesequenzen erzeugen

Bislang haben wir unsere Daten von Hand eingegeben. Für die Zahlenreihe von 1 bis 10 sah das so aus:

```
1 # Zahlenreihe von 1 bis 10
2 zahlen <- c(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
3
4 # ausgeben
5 zahlen
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

In R lassen sich solch kontinuierliche Reihen mit einem `:` erzeugen:

```
1 # auch Zahlenreihe von 1 bis 10
2 zahlen <- 1:10
3
4 # ausgeben
5 zahlen
```


1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] 1 2 3 4 5 6 7 8 9 10
```

In Kombination mit `c()` geht auch:

```
1 # neuer Vektor aus 2 Zahlenreihen
2 zahlen2 <- c(1:10, 100:110)
3
4 # ausgeben
5 zahlen2
```

```
[1] 1 2 3 4 5 6 7 8 9 10 100 101 102 103 104 105
[17] 106 107 108 109 110
```

Mit der Funktion `seq()` lassen sich **Sequenzen** erstellen. Wir übergeben der Funktion Start- und Endpunkt, und in welchen Schritten gezählt werden soll. Eine Zahlenreihe von 1 bis 10 in den Schritten 0.5 erzeugt man so:

```
1 # eine Sequenz erstellen
2 sequenz <- seq(from=1, to=10, by=0.5)
3
4 # ausgeben
5 sequenz
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0
[14] 7.5 8.0 8.5 9.0 9.5 10.0
```

oder schlicht

```
1 # erstellt die selbe Sequenz
2 sequenz <- seq(1, 10, 0.5)
3
4 # ausgeben
5 sequenz
```

```
[1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0
[14] 7.5 8.0 8.5 9.0 9.5 10.0
```

Mit der Funktion `rep()` (für **repeat**) lassen sich wiederholte Werte erzeugen

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # erzeuge 20 wiederholte Werte,  
2 # 20 mal "weiblich"  
3 geschlecht <- rep("weiblich", 20)  
4  
5 # ausgeben  
6 geschlecht
```

```
[1] "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich"  
[7] "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich"  
[13] "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich" "weiblich"  
[19] "weiblich" "weiblich"
```

In Kombination mit `c()` geht auch:

```
1 # erzeuge 20 wiederholte Werte,  
2 # 10 mal "weiblich" und 10 mal "maennlich"  
3 geschlecht <- c(rep("weiblich", 10), rep("maennlich", 10))  
4  
5 # ausgeben  
6 geschlecht
```

```
[1] "weiblich" "weiblich" "weiblich" "weiblich" "weiblich"  
[6] "weiblich" "weiblich" "weiblich" "weiblich" "weiblich"  
[11] "maennlich" "maennlich" "maennlich" "maennlich" "maennlich"  
[16] "maennlich" "maennlich" "maennlich" "maennlich" "maennlich"
```

Oder anders verschachtelt:

```
1 # erzeuge 20 wiederholte Werte, anders verschachtelt  
2 # 10 mal das Paar "weiblich" "maennlich"  
3 geschlecht <- rep(c("weiblich", "maennlich"), 10)  
4  
5 # ausgeben  
6 geschlecht
```

```
[1] "weiblich" "maennlich" "weiblich" "maennlich" "weiblich"  
[6] "maennlich" "weiblich" "maennlich" "weiblich" "maennlich"  
[11] "weiblich" "maennlich" "weiblich" "maennlich" "weiblich"  
[16] "maennlich" "weiblich" "maennlich" "weiblich" "maennlich"
```

Zufallszahlen

In **R** lassen sich auch Zufallszahlen generieren. Hierfür steht die Funktion `sample()` zur Verfügung.

```
1 # erzeuge 20 Zufallswerte
2 sample(20)
```

```
[1] 4 1 14 17 15 2 18 13 20 6 19 16 9 8 11 12 10 3 5 7
```

Der Bereich, aus welchem die Zahlen gezogen werden sollen, kann ebenfalls angegeben werden.

```
1 # erzeuge 20 Zufallswerte zwischen 150 und 250
2 sample(150:250, 20)
```

```
[1] 161 234 199 210 225 236 247 232 217 179 246 150 220 226
[15] 214 177 188 202 160 224
```

Über die Option `replace` kann angegeben werden, ob Werte auch doppelt vorkommen können (so genanntes *zurücklegen*).

```
1 # erzeuge 20 Zufallswerte zwischen 150 und 250
2 # MIT zurücklegen
3 sample(150:250, 20, replace=TRUE)
```

```
[1] 242 163 187 198 204 190 152 184 243 176 170 170 179 217
[15] 150 198 202 227 166 223
```

Zudem bieten die in **R** implementierten Wahrscheinlichkeitsverteilungen über ihre **r**-Funktionen (für *random*, siehe Abschnitt 1.2.4 auf Seite 101) die Möglichkeit, zufällige Werte zu erzeugen.

Die Funktion `runif()` erzeugt Zufallswerte aus der “stetigen Gleichverteilung”. Standardmäßig liegen die Zufallswerte zwischen 0 und 1.

```
1 # erzeuge 20 Zufallswerte zwischen 0 und 1
2 runif(20)
```

```
[1] 0.62383493 0.48679031 0.78736781 0.56125648 0.95977623
[6] 0.14850696 0.35799878 0.67937020 0.09689319 0.63402240
[11] 0.36186202 0.19484200 0.94330547 0.60192248 0.04309927
[16] 0.70540669 0.75177372 0.72113117 0.60635434 0.50857776
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

Man kann aber (ähnlich wie bei `sample()`) den Bereich festlegen, aus welchem die Werte gezogen werden sollen.

```
1 # erzeuge 20 Zufallswerte zwischen 30 und 150
2 runif(20, min=30, max=150)
```

```
[1] 92.79867 90.43651 51.79427 137.82589 40.12666
[6] 123.10785 43.78025 101.20329 147.89779 81.56550
[11] 115.65866 48.43188 109.07900 114.22313 90.78145
[16] 77.35028 43.79792 74.98960 103.78252 45.64845
```

Für ganzzahlige Werte können die Rundungsfunktionen `round()`, `floor()` und `ceiling()` eingeschoben werden.

```
1 # erzeuge 20 GANZZAHLIGE Zufallswerte zwischen 30 und 150
2 floor(runif(20, min=30, max=150))
```

```
[1] 127 52 142 73 137 59 130 60 101 75 35 119 113 111
[15] 83 138 86 148 113 138
```

Um Zufallszahlen aus der Standardnormalverteilung zu generieren eignet sich die Funktion `rnorm()`.

```
1 # erzeuge 40 zufällige Werte aus der
2 # Standardnormalverteilung
3 normal <- rnorm(40)
4
5 # ausgeben
6 normal
```

```
[1] -1.30683973 0.08581781 0.52898949 0.12316733 1.96814473
[6] -0.03224878 1.24255982 0.80833688 0.12617009 -1.68422729
[11] 2.07087237 0.70833213 0.90960225 1.52042129 -0.61109432
[16] -0.56164887 0.87728035 -0.07644280 -1.09885688 -1.80537158
[21] 0.34978431 -0.93592539 -0.66504186 -0.95879982 -1.60549156
[26] -1.01631415 -0.94082868 0.90715833 0.32633218 -0.11079465
[31] -0.92969431 -1.67418761 -0.20824912 1.21117314 -0.59430724
[36] 0.03055659 0.65196122 0.58226606 -1.33694530 1.16788244
```

Somit erzeugt `rt()` zufällige Werte aus der t-Verteilung.

```
1 # erzeuge 40 zufällige Werte aus der
2 # t-Verteilung bei 10 Freiheitsgraden
3 t <- rt(40, df=10)
4
5 # ausgeben
6 t
```

```
[1] 0.52311923 -0.67621471 1.04927080 -0.81329198 -0.01622871
[6] -0.88733849 -0.55410697 -0.37071397 0.79783044 -2.05919751
[11] 0.03762458 -0.09059438 0.49890986 1.02610386 0.72289407
[16] 0.83527344 -1.92007156 -0.00191527 1.51483848 -0.02774711
[21] 1.66549996 0.38796390 0.67036334 -0.72107346 -0.77602159
[26] -0.50466444 0.20603820 -4.16495836 -0.86765434 -2.75438947
[31] 0.71361286 -0.35672527 0.06796026 0.21855036 0.32415552
[36] 1.03790635 -1.53014553 -1.18606832 -0.40752377 -0.54694851
```

1.2.3.4 Datentypen

 ist eine objektorientierte Sprache und kann verschiedene Formen von Datenklassen und -typen verarbeiten.

Die für uns wichtigsten Datentypen sind:

- numerisch (Zahlen, mit denen man rechnen kann)
- character (Zeichenkette, also Text und Wörter)
- logisch (Wahr/Falsch, bzw. TRUE und FALSE)

Wir haben im oberen Abschnitt (1.2.3.2) schon zwei *Datentypen* kennengelernt, zum einen den Typ **numerisch**, denn wir haben den Variablen **x**, **y** und **z** jeweils Zahlenwerte zugeordnet, mit denen wir rechnen konnten.

```
1 # numerischer Datentyp
2 v_num <- c(0, 8, 15)
3
4 # rechnen ist möglich
5 v_num * 100
```

```
[1] 0 800 1500
```

Zum anderen hatten wir der Variable **b** Wörter zugewiesen. Dies entspricht dem Datentyp “character”.

```
1 # Datentyp "Character"
2 v_char <- c("Hallo", "ihr", "lieben", "Leute")
3
4 # rechnen ist nicht möglich
5 v_char * 100
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

Fehler in `v_char * 100` : nicht-numerisches Argument für binären Operator

Variablen vom Typ `logisch` enthalten die logischen Aussagen `TRUE` und `FALSE` bzw. deren Abkürzungen `T` und `F`.

Eine logische Aussage kann zum Beispiel durch einen Vergleich erzeugt werden

```
1 # ist 50 = 100
2 50==100
```

```
[1] FALSE
```

Da 50 eben nicht gleich 100 ist, erhalten wir die logische Aussage `FALSE` zurück.

```
1 # ist 50 kleiner als 100
2 50<100
```

```
[1] TRUE
```

Die Zahl 50 ist kleiner als 100, darum erhalten wir die logische Antwort `TRUE` zurück.

Das kann man auch auf Wertereihen anwenden.

```
1 # erzeuge 20 werte
2 reihe <- seq(1, 100, 5)
3 # ausgeben
4 reihe
```

```
[1] 1 6 11 16 21 26 31 36 41 46 51 56 61 66 71 76 81 86 91 96
```

```
1 # Prüfe jeden Wert in "reihe", ob er kleiner als 75 ist
2 reihe<75
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[12] TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

```
1 # Prüfe jeden Wert in "reihe", ob er kleiner als 75 ist
2 # und speichere das Ergebnis in Variable "logical"
3 logical <- reihe<75
4
5 # zeige den 7. und 18. Wert von "logical" an
6 logical[c(7,18)]
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] TRUE FALSE
```

Mit der Funktion `class()` kann der Datentyp (und auch die Datenklasse, dazu später mehr) angezeigt werden.

```
1 # welcher Datentyp ist Variable "logical"
2 class(logical)
```

```
[1] "logical"
```

```
1 # welcher Datentyp ist Variable "v_char"
2 class(v_char)
```

```
[1] "character"
```

```
1 # welcher Datentyp ist Variable "v_num"
2 class(v_num)
```

```
[1] "numeric"
```

Mit den Funktionen `is.numeric()`, `is.logical()` und `is.character()` können wir den Datentyp ebenfalls überprüfen. Als Antwort erhalten wir ein logisches `TRUE` oder `FALSE`.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # ist Variable "v_num" numerisch?  
2 is.numeric(v_num)
```

```
[1] TRUE
```

```
1 # ist Variable "v_char" numerisch?  
2 is.numeric(v_char)
```

```
[1] FALSE
```

```
1 # ist Variable "v_num" logical?  
2 is.logical(v_num)
```

```
[1] FALSE
```

```
1 # ist Variable "v_char" logical?  
2 is.logical(v_char)
```

```
[1] FALSE
```

```
1 # ist Variable "v_num" character?  
2 is.character(v_num)
```

```
[1] FALSE
```

```
1 # ist Variable "v_char" character?  
2 is.character(v_char)
```

```
[1] TRUE
```

```
1 # ist Variable "logical" character?  
2 is.character(logical)
```

```
[1] FALSE
```

```
1 # ist Variable "logical" logical?  
2 is.logical(logical)
```

```
[1] TRUE
```

1.2.3.5 Datenklassen

Die Datentypen können wiederum in *Datenklassen* gespeichert werden. Für uns wichtige Datenklassen sind:

- Vektoren
- Matrizen
- Faktoren (Gruppen, Rangfolge)
- Datenframes
- Listen

1.2.3.5.1 Vektoren

Vektoren haben wir schon kennengelernt. Ein Vektor ist eine einfache Wertereihe vom selben Datentyp. So erzeugt die Funktion `c()` einen Datenvektor. Alle Werte der Datenklasse **Vektor** müssen Werte des selben *Wertetyps* enthalten. Kombinieren wir **numerische**, **character** und **logische** Werte in einem Vektor, so wandeln sich alle Werte in den kleinsten gemeinsamen Datentyp (nämlich **character**) um.

```
1 # c() fällt auf den "kleinsten gemeinsamen Nenner" zurück
2 c(1, 2, 3, 4, "fünf", "sechs", TRUE, TRUE, FALSE)
```

```
[1] "1"      "2"      "3"      "4"      "fünf"   "sechs"  "TRUE"   "TRUE"
[9] "FALSE"
```

Alle Werte sind nun vom Datentyp **character**, erkennbar an den Anführungszeichen.

Auf die Werte kann man zugreifen, indem man den Variablennamen eingibt:

```
1 # erzeuge einen Vektor
2 vektor <- seq(1, 20, 1)
3
4 # anzeigen
5 vektor
```

```
[1] 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

Auf die einzelnen Werte des Vektors kann man zugreifen, indem man die gewünschte Position in eckigen Klammern an den Variablennamen anhängt.

```
1 # Zeige den ersten Wert von "vektor"
2 vektor[1]
```

```
[1] 1
```

Auch hier können wir *Positionsbereiche* mit einem `:` angeben

```
1 # Zeige die Werte an Position 4 bis 15
2 vektor[4:15]
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1]  4  5  6  7  8  9 10 11 12 13 14 15
```

Mit einem Minuszeichen können auch bestimmte Werte oder Wertbereiche ausgelassen werden.

```
1 # Zeige "vektor" OHNE den ersten Wert
2 vektor[-1]
```

```
[1]  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

```
1 # Zeige "vektor" OHNE die Werte 10 bis 14
2 vektor[-(10:14)]
```

```
[1]  1  2  3  4  5  6  7  8  9 15 16 17 18 19 20
```

Wieviele Werte ein Vektor enthält erfährt man mit der Funktion `length()`.

```
1 # wieviele Werte hat "vektor"?
2 length(vektor)
```

```
[1] 20
```

Der Vektor `vektor` beinhaltet 20 Werte.

Mit der Funktion `is.vector()` kann geprüft werden, ob ein Objekt ein Vektor ist.

```
1 # ist "vektor" ein Vektor?
2 is.vector(vektor)
```

```
[1] TRUE
```

1.2.3.5.2 Matrizen

Matrizen sind zweidimensionale Strukturen (Tabellen) und werden von  intern durch Vektoren dargestellt. Dies impliziert, dass alle Werte der Matrix vom selben Datentyp (z.B. **numerisch**) sein müssen, genau so wie bei Vektoren. Um besser zu erklären, wie Matrizen funktionieren, erzeugen wir zunächst ein paar Beispielvektoren.

```

1  # Erzeuge Testwertereihen
2  a <- c(11, 12, 13, 14, 15)
3  b <- c(21, 22, 23, 24, 25)
4  c <- c(31, 32, 33, 34, 35)
5  d <- c(41, 42, 43, 44, 45)
6  e <- c(51, 52, 53, 54, 55)
7  f <- c("eins", "zwei", "drei", "vier", "fünf")
8
9  # Füge alle Zahlen zu einem Vektor zusammen
10 alle <- c(a, b, c, d, e)
11
12 # anzeigen
13 alle

```

```

[1] 11 12 13 14 15 21 22 23 24 25 31 32 33 34 35 41 42 43 44 45
[21] 51 52 53 54 55

```

Die Funktion `matrix()` setzt aus einem Vektor eine Matrix zusammen. Lässt man alle Parameter im Funktionsaufruf `matrix()` leer, wird eine Matrix mit 1 Spalte erzeugt.

```

1  # Erzeuge eine Matrix aus Vektor "a"
2  matrix(a)

```

```

      [,1]
[1,]   11
[2,]   12
[3,]   13
[4,]   14
[5,]   15

```

Mit dem Parameter `ncol` kann die gewünschte Anzahl an Spalten übergeben werden:

```

1  # Erzeuge eine Matrix aus Vektoren "a" und "b"
2  # mit 2 Spalten
3  matrix(c(a,b), ncol=2)

```

```

      [,1] [,2]
[1,]  11  21
[2,]  12  22
[3,]  13  23
[4,]  14  24
[5,]  15  25

```

Mit dem Parameter **nrow** kann die gewünschte Anzahl an Zeilen übergeben werden:

```

1 # Erzeuge eine Matrix aus Vektoren "a" und "b"
2 # mit 2 Zeilen
3 matrix(c(a,b), nrow=2)

```

```

      [,1] [,2] [,3] [,4] [,5]
[1,]  11  13  15  22  24
[2,]  12  14  21  23  25

```



Achtung, Anfängerfehler!



Achten Sie auf die Reihenfolge, in der die Werte in der Matrix angelegt wurden. Das ist wahrscheinlich nicht das Ergebnis, das Sie erwartet haben. Die Funktion **matrix()** arbeitet standardmäßig die Werte pro Spalte (*spaltenorientiert*) ab. Wir können das mit dem Parameter **byrow** ändern:

```

1 # Erzeuge eine Matrix aus Vektoren "a" und "b"
2 # mit 2 Zeilen, diesmal zeilenorientiert
3 matrix(c(a,b), nrow=2, byrow=TRUE)

```

```

      [,1] [,2] [,3] [,4] [,5]
[1,]  11  12  13  14  15
[2,]  21  22  23  24  25

```

Sobald eine Matrix mit 2 oder mehr Spalten angelegt werden soll, muss der Datenvektor so viele Werte enthalten, dass die gewünschte Matrix vollständig erstellt werden kann. Sollten zu wenige Werte vorhanden sein, gibt  eine Warnmeldung aus. Für eine Matrix mit 2 Spalten muss also eine **gerade** Anzahl an Werten vorhanden sein.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # Matrix mit 2 Spalten benötigt gerade Anzahl an Werten
2 # daher gibt R (mit nur 9 Werten) eine Warnmeldung aus
3 matrix(1:9, nrow=2)
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8    1
Warnmeldung:
In matrix(1:9, nrow = 2) :
  Datenlänge [9] ist kein Teiler oder Vielfaches der Anzahl der Zeilen [2]
```

Wie Sie sehen versucht **R** selbstständig die Matrix zu vervollständigen. Hierfür wiederholt **R** die angegebenen Werte so lange, bis die Matrix “voll” ist. Im obigen Beispiel wurde nach der 9 wieder eine 1 eingetragen.

Wie bereits erwähnt müssen die Werte vom selben Datentyp sein. Mischt man **numeric** mit **character**, fällt auch Matrix auf den kleinsten gemeinsamen Datentyp (**character**) zurück.

```
1 # Erzeuge eine Matrix aus Vektoren "a" und "f"
2 # fällt auch Typ "char" zurück
3 matrix(c(a,f), nrow=2, byrow=TRUE)
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,] "11"  "12"  "13"  "14"  "15"
[2,] "eins" "zwei" "drei" "vier" "fünf"
```

Neben **matrix()** können auch die Befehle **rbind()** (für *rowbind*) und **cbind()** (für *columnbind*) verwendet werden, um eine Matrix zu erzeugen. Der Befehl **cbind()** fügt die übergebenen Vektoren *spaltenorientiert* zu einer Matrix zusammen.

```
1 # Erzeuge eine Matrix aus Vektoren "c" und "d"
2 # spaltenorientiert
3 cbind(c, d)
```

```
      c d
[1,] 31 41
[2,] 32 42
[3,] 33 43
[4,] 34 44
[5,] 35 45
```

Der Befehl `rbind()` fügt die übergebenen Vektoren *zeilenorientiert* zu einer Matrix zusammen.

```
1 # Erzeuge eine Matrix aus Vektoren "c" und "d"
2 # zeilenorientiert
3 rbind(c, d)
```

```
  [,1] [,2] [,3] [,4] [,5]
c   31  32  33  34  35
d   41  42  43  44  45
```

Wir speichern eine Matrix in einer Variable:

```
1 # Erzeuge eine Matrix aus Vektor "alle"
2 # mit 5 Spalten
3 mymatrix <- matrix(alle, ncol=5)
4
5 # ausgeben
6 mymatrix
```

```
  [,1] [,2] [,3] [,4] [,5]
[1,]  11  21  31  41  51
[2,]  12  22  32  42  52
[3,]  13  23  33  43  53
[4,]  14  24  34  44  54
[5,]  15  25  35  45  55
```

An der Ausgabe der Spalten- und Zeilentitel lässt sich erahnen, wie die einzelnen Werte einer Matrix referenziert werden können. Bei Vektoren können die einzelnen Werte abgerufen werden, indem in eckigen Klammern die gewünschte Position angegeben wird. Dies funktioniert bei Matrizen ähnlich, jedoch muss innerhalb der eckigen Klammer zwischen *Zeilen* und *Spalten* unterschieden werden. Dies geschieht mit einem Komma, wobei vor dem Komma die Zeilen, und nach dem Komma die Spalten referenziert werden.

```
1 # zeige die 1. Zeile der Matrix
2 # die Zahl vor dem Komma repräsentiert die Zeilen
3 mymatrix[1,]
```

```
[1] 11 21 31 41 51
```

```
1 # zeige die 1. Spalte der Matrix
2 # die Zahl nach dem Komma repräsentiert die Spalten
3 mymatrix[,1]
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] 11 12 13 14 15
```

Wie in einem Koordinatensystem können nun gezielt einzelne Werte oder Wertbereiche referenziert werden.

```
1 # zeige den Wert in Zeile 3 und Spalte 2
2 mymatrix[3,2]
```

```
[1] 23
```

```
1 # zeige die Werte aus Zeile 2 bis 4
2 # und Spalte 1 bis 3
3 mymatrix[2:4,1:3]
```

```
  [,1] [,2] [,3]
[1,] 12  22  32
[2,] 13  23  33
[3,] 14  24  34
```

Mit der Funktion `t()` kann die Matrix transponiert werden, das bedeutet, es werden Zeilen und Spalten diagonal gespiegelt.

```
1 # zeige meine Matrix
2 mymatrix
```

```
  [,1] [,2] [,3] [,4] [,5]
[1,] 11  21  31  41  51
[2,] 12  22  32  42  52
[3,] 13  23  33  43  53
[4,] 14  24  34  44  54
[5,] 15  25  35  45  55
```

```
1 # transponiere meine Matrix
2 t(mymatrix)
```

```
  [,1] [,2] [,3] [,4] [,5]
[1,] 11  12  13  14  15
[2,] 21  22  23  24  25
[3,] 31  32  33  34  35
[4,] 41  42  43  44  45
[5,] 51  52  53  54  55
```

Mit den Funktionen `colnames()` und `rownames()` können Spalten und Zeilen noch betitelt werden.

Wir benennen unsere Spalten von “a” bis “e”:

```
1 # benenne Spalten der Matrix
2 colnames(mymatrix) <- c("a", "b", "c", "d", "e")
3
4 # anzeigen
5 mymatrix
```

```
      a b c d e
[1,] 11 21 31 41 51
[2,] 12 22 32 42 52
[3,] 13 23 33 43 53
[4,] 14 24 34 44 54
[5,] 15 25 35 45 55
```

Wir benennen unsere Zeilen von römisch I bis V:

```
1 # benenne Spalten der Matrix
2 rownames(mymatrix) <- c("I", "II", "III", "IV", "V")
3
4 # anzeigen
5 mymatrix
```

```
      a b c d e
I    11 21 31 41 51
II   12 22 32 42 52
III  13 23 33 43 53
IV   14 24 34 44 54
V    15 25 35 45 55
```

Mit der Funktion `class()` kann die Datenklasse angezeigt werden.

```
1 # welcher Datentyp ist Variable "mymatrix"
2 class(mymatrix)
```

```
[1] "matrix" "array"
```

Mit der Funktion `is.matrix()` kann zudem geprüft werden, ob ein Objekt eine Matrix ist.

```
1 # ist "mymatrix" eine Matrix?
2 is.matrix(mymatrix)
```


1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] TRUE
```

```
1 # ist "vector" eine Matrix?
2 is.matrix(vector)
```

```
[1] FALSE
```

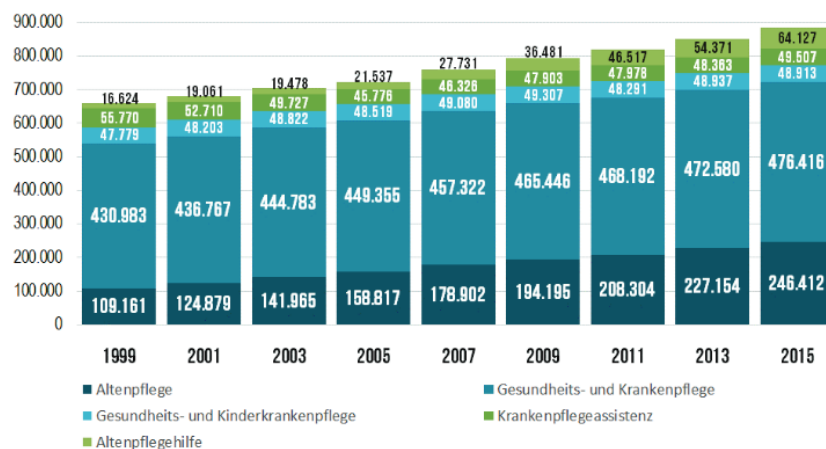


Abb. 1.26: Beschäftigte in Pflegeberufen (aus [3, S.31])

Als Übung können wir nun beispielsweise die Anzahl der Beschäftigten in Pflegeberufen aus dem “Pflegethermometer 2018” [3, S.31] (siehe Abbildung 1.26), als Matrix in R übertragen.

```
1 # Schreibe die Zahlen reihenweise aus der Grafik ab
2 Pflegeberufe <- c(16624, 19061, 19478, 21537, 27731, 36481, 46517, 54371, 64127,
3                 55770, 52710, 49727, 45776, 48326, 47903, 47978, 48363, 49507,
4                 47779, 48203, 48822, 48519, 49080, 49307, 48291, 48937, 48913,
5                 430983, 436767, 444783, 449355, 457322, 465446, 468192, 472580, 476416,
6                 109161, 124879, 141965, 158817, 178902, 194195, 208304, 227154, 246412
7                 )
8 # überführe in Matrix mit 9 Spalten
9 # Die Werte kommen reihenmäßig
10 Pflegeberufe <- matrix(Pflegeberufe, byrow=T, ncol=9)
11
12 # benenne die Spalten
13 colnames(Pflegeberufe) <- c(1999, 2001, 2003, 2005, 2007, 2009, 2011, 2013, 2015)
```

```

14
15 # benenne die Reihen
16 rownames(Pflegeberufe) <- c("Krankenpflegeassistenz", "Altenpflegehilfe",
17 "Kinderkrankenpflege", "Krankenpflege", "Altenpflege")
18
19 # zeige Tabelle
Pflegeberufe

```

	1999	2001	2003	2005	2007	2009	2011	2013	2015
Krankenpflegeassistenz	16624	19061	19478	21537	27731	36481	46517	54371	64127
Altenpflegehilfe	55770	52710	49727	45776	48326	47903	47978	48363	49507
Kinderkrankenpflege	47779	48203	48822	48519	49080	49307	48291	48937	48913
Krankenpflege	430983	436767	444783	449355	457322	465446	468192	472580	476416
Altenpflege	109161	124879	141965	158817	178902	194195	208304	227154	246412

Wir werden auf diese Matrix später noch zurückkommen.

1.2.3.5.3 Faktoren

Die Datenklasse **factor** beschreibt gruppierte (nominale) oder ranggeordnete (ordinale) Werte. Gruppierte Werte sind beispielsweise “Beruf”, “Konfession”, “Familienstand” oder “Geschlecht”. Es liegt keine Reihenfolge unter den Gruppen vor. Sie werden mit der Funktion **factor()** erstellt.

```

1 # nominalen Faktoren erstellen
2 v_fac1 <- factor(c("maennlich", "weiblich", "divers"))
3
4 # ausgeben
5 v_fac1

```

```

[1] maennlich weiblich divers
Levels: divers maennlich weiblich

```

Unsere Variable **v_fac1** besteht aus 3 Gruppen, nämlich **divers**, **maennlich** und **weiblich** mit je einem Wert. Erzeugen wir die Daten erneut, diesmal mit mehr Werten und einer abgekürzten Schreibweise für das Geschlecht.

```

1 # Faktoren neu erstellen, mit mehr Werten
2 v_fac1 <- factor(c("m", "w", "d", "m", "w", "d", "m", "w", "d", "m", "w", "d"))
3
4 # ausgeben
5 v_fac1

```

```
[1] m w d m w d m w d m w d
Levels: d m w
```

In der ersten Zeile sehen wir unsere Datenreihe, in der zweiten Zeile alle Gruppen (Levels), in diesem Falle `d`, `m` und `w`.

Für ordinale (also ranggeordnete) Werte nehmen wir ebenfalls die `factor()`-Funktion. Versuchen wir also, klassische Schulnoten abzubilden. Wir stellen uns eine virtuelle Liste mit Schulnoten von 100 SchülerInnen vor. Wir übertragen die Noten von der virtuellen Liste in $\mathbb{R}$, und zwar in der Reihenfolge, wie sie auf unserer virtuellen Notenliste stehen könnten (sprich: *unsortiert*).

```
1 # Faktoren aus Notenliste erstellen
2 v_fac2 <- factor(c("gut", "ausreichend", "sehr gut", "ausreichend", "befriedigend",
3 "mangelhaft", "ungenügend", "gut", "gut", "sehr gut"))
4 # ausgeben
5 v_fac2
```

```
[1] gut      ausreichend sehr gut      ausreichend befriedigend mangelhaft
[7] ungenügend gut      gut      sehr gut
Levels: ausreichend befriedigend gut mangelhaft sehr gut ungenügend
```

In der ersten Zeile sehen wir unsere Datenreihe, in der letzten Zeile alle Ränge (Levels). Ein Blick auf die Levels zeigt aber auch, dass diese in einer falschen Reihenfolge angelegt wurden. Der erste Rang ist hier `ausreichend`, und der zweite `befriedigend` (es sollte ja eigentlich so sein, dass `sehr gut` der erste Rang ist, und `gut` der zweite, usw). Das liegt daran, dass wir die Noten unsortiert aneinandergereiht haben. $\mathbb{R}$ erstellt die Reihenfolge der Levels anhand der Reihenfolge, in der sie eintreffen.



Achtung, Anfängerfehler!



Dies ist ein häufiger Anfängerfehler bei der Erstellung von Faktoren!

Um unsere Schulnoten in der richtigen Levelreihenfolge anzulegen, müssen wir dem Befehl `factor()` eben diese Reihenfolge über die Option `levels` mitgeben (mehr Informationen erhalten Sie über die Hilfeseite `?factor`). Konkret übergeben wir per `levels=c()` die Levelnamen von `sehr gut` bis `ungenügend`.

```

1 # Faktoren aus Notenliste erstellen
2 # diesmal Levelreihenfolge mit "levels=" vorgeben
3 v_fac2 <- factor(c("gut", "ausreichend", "sehr gut", "ausreichend", "befriedigend",
4 "mangelhaft", "ungenügend", "gut", "gut", "sehr gut"), levels=c("sehr gut", "gut",
5 "befriedigend", "ausreichend", "mangelhaft", "ungenügend"))
6
7 # ausgeben
8 v_fac2

```

```

[1] gut      ausreichend sehr gut      ausreichend befriedigend mangelhaft
[7] ungenügend gut      gut      sehr gut
Levels: sehr gut gut befriedigend ausreichend mangelhaft ungenügend

```

Die Levels sind nun in der korrekten Reihenfolge.

Wir hätten die Daten aber gar nicht neu eingeben müssen. Der Befehl lässt sich verkürzen, indem man einfach die bestehende Variable als Input nutzt und neu überschreibt:

```

1 # Levelreihenfolge in "v_fac2" reparieren und überschreiben
2 v_fac2 <- factor(v_fac2, levels=c("sehr gut", "gut", "befriedigend",
3 "ausreichend", "mangelhaft", "ungenügend"))
4
5 # ausgeben
6 v_fac2

```

```

[1] gut      ausreichend sehr gut      ausreichend befriedigend mangelhaft
[7] ungenügend gut      gut      sehr gut
Levels: sehr gut gut befriedigend ausreichend mangelhaft ungenügend

```

So kann man auch nachträglich die Levelreihenfolge korrigieren.

Mit dem Befehl `revalue()` aus dem `plyr`-Zusatzpaket¹ lassen sich die Werte und Levels von Faktoren umändern. Wir ändern unsere Schulnoten von den ausgeschriebenen Noten hin zu Zahlenwerten. Hierfür erzeugen wir eine neue Variable `v_fac3`.

```

1 # Lade Zusatzpaket "plyr"
2 library(plyr)
3
4 # Ändere Levelnamen
5 v_fac3 <- revalue(v_fac2, c("sehr gut"="1",
6 "gut"="2", "befriedigend"="3", "ausreichend"="4", "mangelhaft"="5", "ungenügend"="6"))
7
8 # Werte ausgeben
9 v_fac3

```

¹zur Installation von Zusatzpaketen siehe Abschnitt 1.2.3.13 auf Seite 92

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] 2 4 1 4 3 5 6 2 2 1
Levels: 1 2 3 4 5 6
```

Beachten Sie, dass die “Zahlen”werte nur nominaler Natur sind. Wir können mit ihnen nicht rechnen!

```
1 # rechnen ist mit factor nicht möglich!
2 v_fac3 * 100
```

```
[1] NA NA NA NA NA NA NA NA NA NA
Warnmeldung:
In Ops.factor(v_fac3, 100) : '*' not meaningful for factors
```

Die Werte innerhalb einer Faktorenreihe referenziert man so wie bei Vektoren, indem man die gewünschte Position in eckigen Klammern an den Variablenamen anhängt.

```
1 # zeige die Werte von 3 bis 7 von "v_fac3"
2 v_fac3[3:7]
```

```
[1] 1 4 3 5 6
Levels: 1 2 3 4 5 6
```

Welche Levels in einem Faktor existieren erfährt man mit der Funktion `levels()`.

```
1 # welche Levels hat "v_fac3"?
2 levels(v_fac3)
```

```
[1] "1" "2" "3" "4" "5" "6"
```

Durch Kombination mit der Funktion `length()` können wir die Anzahl der Levels erfahren.

```
1 # wieviele Levels hat "v_fac3"?
2 length(levels(v_fac3))
```

```
[1] 6
```

Die Variable hat 6 Levels.

Welche Level welche Häufigkeit hat erfahren wir mit der Funktion `table()`.

```
1 # welche Level hat welche Häufigkeit?  
2 table(v_fac2)
```

```
v_fac2  
sehr gut      gut befriedigend  ausreichend  mangelhaft  ungenügend  
      2          3          1          2          1          1
```

Mit der Funktion `class()` kann die Datenklasse angezeigt werden.

```
1 # welcher Datentyp ist Variable "v_fac2"  
2 class(v_fac2)
```

```
[1] "factor"
```

Mit der Funktion `is.factor()` kann zudem geprüft werden, ob ein Objekt ein Faktor ist.

```
1 # ist "v_fac2" ein Faktor?  
2 is.factor(v_fac2)
```

```
[1] TRUE
```

```
1 # ist "mymatrix" ein Faktor?  
2 is.factor(mymatrix)
```

```
[1] FALSE
```

1.2.3.5.4 ordinale Faktoren

Die bislang von uns erzeugten Faktoren haben zwar “scheinbar” eine korrekte Levelreihenfolge. Für  handelt es sich dabei aber weiterhin um “einfache” Faktoren, also *nominale* Daten. Um die Faktoren in *ordinale* Faktoren umzuwandeln kann die Funktion `ordered()` verwendet werden. Nutzen wir hierfür unsere Variable `v_fac2` mit den Schulnoten.

```
1 # wandle in ordinalen Faktor um
2 ordered(v_fac2)
```

```
[1] gut      ausreichend sehr gut      ausreichend befriedigend
[6] mangelhaft ungenügend gut      gut      sehr gut
6 Levels: sehr gut < gut < befriedigend < ausreichend < ... < ungenügend
```

Die Levels haben nun eine Rangfolge, welche durch das Kleinerzeichen `<` dargestellt wird. Leider sind die Levelränge genau verkehrt herum, denn im obigen Falle ist “**sehr gut**” die kleinste und “**ungenügend**” die größte Note. Wir ändern also nochmal die Levelreihenfolge. Hierzu übergeben wir die Levelnamen in einen Vektor, und kehren mit der Funktion `rev()` dessen Reihenfolge um (die Funktion `fct_rev()` aus dem `forcats`-Paket dreht die Levelreihenfolge ebenfalls um, aber derzeit wissen wir ja noch nicht, wie man Zusatzpakete installiert (siehe Abschnitt 1.2.3.13 auf Seite 92)).

```
1 # zeige Levelnamen an
2 levels(v_fac2)
```

```
[1] "sehr gut"      "gut"          "befriedigend" "ausreichend"  "mangelhaft"
↪ "ungenügend"
```

```
1 # kehre den Vektor um
2 rev(levels(v_fac2))
```

```
[1] "ungenügend"  "mangelhaft"  "ausreichend" "befriedigend" "gut"         "sehr gut"
```

In Kombination mit `ordered()` erhalten wir so die korrekte ordinale Darstellung.

```

1 # überführe die ordinale Reihenfolge in Variable 'v_ord'
2 v_ord <- ordered(factor(v_fac2,
3                       levels= rev(levels(v_fac2))))
4
5 v_ord

```

```

[1] gut          ausreichend sehr gut          ausreichend befriedigend
[6] mangelhaft  ungenügend  gut          gut          sehr gut
6 Levels: ungenügend < mangelhaft < ausreichend < befriedigend < gut < sehr gut

```

Die Funktion `factor()` nimmt zudem den Parameter `ordered=TRUE` entgegen, der direkt ein ordinale Objekt erzeugt:

```

1 # erzeuge direkt einen ordinalen factor
2 # mit Parameter 'ordered=TRUE'
3 v_ord <- factor(v_fac2,
4               levels=rev(levels(v_fac2)),
5               ordered=TRUE)
6
7 v_ord

```

```

[1] gut          ausreichend sehr gut          ausreichend befriedigend
[6] mangelhaft  ungenügend  gut          gut          sehr gut
6 Levels: ungenügend < mangelhaft < ausreichend < befriedigend < gut < sehr gut

```

1.2.3.5.5 Datenframes

Die Datenklasse Datenframe (Datensatz) ist wohl die wichtigste in $\mathbb{R}$. Datenframes sind ebenso wie Matrizen zweidimensional. Im Unterschied zu einer Matrix können in einem Datenframe unterschiedliche Datentypen, also z.B. `numeric`, `character` und `factor`, zusammengeführt werden. Das Datenframe folgt dabei der Logik “ein Fall pro Zeile” (so genanntes *tidy data* Format[6], siehe Abschnitt 1.6 auf Seite 147). Das bedeutet, dass jede Beobachtung (auch Wiederholungen) in einer eigenen Zeile stehen und die jeweiligen Variablen durch die Spalten repräsentiert werden.

Erzeugen wir uns ein paar Beispielvektoren unterschiedlichen Typs mit je 12 Werten.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # erzeuge Testvektoren "factor", "char", "numeric", "logical"
2 geschlecht <- factor(rep(c("m", "w", "d"), 4))
3 spitzname <- c("Hasi", "Ide", "Moms", "Ryu", "Dave", "Zoid", "Adu", "Efi", "Ole", "Ray",
4               "Sam", "Emi")
5 hausnummer <- 1:12
   angemeldet <- c(TRUE, TRUE, FALSE, T, F, F, F, T, T, T, F, T)
```

Aus den Variablen setzen wir nun mit der Funktion `data.frame()` ein Datenframe zusammen.

```
1 # erzeuge ein Datenframe aus den Testvektoren
2 data.frame(geschlecht, spitzname, hausnummer, angemeldet)
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Moms	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE

Das Datenframe speichern wir in die Variable `MeinDatenframe`.

```
1 # speicher Datenframe in Variable
2 MeinDatenframe <- data.frame(geschlecht, spitzname, hausnummer, angemeldet)
3
4 # zeige Datenklasse an
5 class(MeinDatenframe)
```

```
[1] "data.frame"
```

Die Funktion `class()` weist unsere Variable als Datenframe aus.

Ähnlich wie bei Matrizen müssen die Vektoren jeweils die selbe Anzahl an Werten (die selbe *Länge*) besitzen, damit das Datenframe vollständig aufgebaut werden kann. Entfernen wir z.B. einen Wert aus der Reihe `hausnummer`, schlägt der Befehl fehl.

```
1 # Datenframe, "hausnummer" ist einen Wert kürzer
2 data.frame(geschlecht, spitzname, hausnummer[-1], angemeldet)
```

```
Fehler in data.frame(geschlecht, spitzname, hausnummer[-1], angemeldet) :  
  Argumente implizieren unterschiedliche Anzahl Zeilen: 12, 11
```

Wenn zwei Datenframes mit den **selben** Spaltennamen existieren, können diese per `rbind()` zusammengefasst werden. In unserem Beispiel verdoppeln wir einfach unser Datenframe.

```
1 # füge 2 Datenframes mittels rbind() zusammen  
2 rbind(MeinDatenframe, MeinDatenframe)
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE
13	m	Hasi	1	TRUE
14	w	Ide	2	TRUE
15	d	Momsi	3	FALSE
16	m	Ryu	4	TRUE
17	w	Dave	5	FALSE
18	d	Zoid	6	FALSE
19	m	Adu	7	FALSE
20	w	Efi	8	TRUE
21	d	Ole	9	TRUE
22	m	Ray	10	TRUE
23	w	Sam	11	FALSE
24	d	Emi	12	TRUE



Achtung, Anfängerfehler!



Es funktioniert **nicht**, wenn eine neue Zeile mit einem Datenvektor übergeben wird, denn in einem Vektor können nur Werte des selben Datentyps vorkommen.

```
1 # füge einzelne Zeile mit rbind() hinzu  
2 rbind(MeinDatenframe, c("m", "Joe", 99, TRUE))
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE
13	m	Joe	99	TRUE

Zwar sieht es so aus, als sei die neue Zeile korrekt eingetragen worden, wenn wir jedoch das neue Datenframe in einer Variable abspeichern und die Datenklassen überprüfen, stellen wir fest, was falsch gelaufen ist.

```

1 # füge einzelne Zeile mit rbind() hinzu
2 new <- rbind(MeinDatenframe, c("m", "Joe", 99, TRUE))
3
4 # überprüfe Datentyp für Spalte "hausnummer"
5 class(new$hausnummer)

```

```
[1] "character"
```

Der Datentyp in Spalte `hausnummer` ist auf den “kleinsten gemeinsamen Nenner” (`character`) zurückgefallen. Das liegt daran, dass zunächst der Vektor in `c()` auf `character` zurückfällt. Somit sind alle Werte in der `c()`-Funktion vom Typ `character`. Bei `hausnummer` zieht nun dieser neue Wert die gesamte Spalte auf den Typ `character` zurück. Ebenso verhält es sich bei Variable `angemeldet`, die eigentlich mal vom Typ `logical` war.

```

1 # überprüfe Datentyp für Spalte "geschlecht"
2 class(new$angemeldet)

```

```
[1] "character"
```

Wenn wir mit der Spalte `hausnummer` rechnen wollen, schlägt dies fehl.

```

1 # multipliziere Spalte "hausnummer" mit 2
2 new$hausnummer * 2

```

```
Fehler in new$hausnummer * 2 :
nicht-numerisches Argument für binären Operator
```

Ein falscher `rbind()`-Befehl kann Ihnen also das gesamte Datenframe “zer-schießen”. Dies ist ein häufiger Anfängerfehler, **seien Sie sorgsam**, wenn Sie einem Datenframe neue Zeilen hinzufügen!

Um also eine neue Zeile korrekt dem Datenframe hinzuzufügen, muss diese neue Zeile ebenfalls als Datenframe in der selben Struktur (also mit den selben Variablen (Spalten) vorliegen. Die

```
1 # neue Zeile
2 neuezeile <- data.frame( factor("m"), "Joe", 99, TRUE)
3
4 # übergebe die Spaltennamen an die neue Zeile
5 colnames(neuzeile) <- colnames(MeinDatenframe)
6
7 # füge zu Datenframe hinzu
8 new <- rbind(MeinDatenframe, neuezeile)
9
10 # anzeigen
11 new
```

1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE
13	m	Joe	99	TRUE

Mit dem Befehl `cbind()` können dem Datenframe neue Spalten hinzugefügt werden. Hierbei ist wichtig, dass die neue Spalte die selbe Anzahl an Werten aufweist wie die anderen Spalten des Datenframes. Wir erzeugen eine neue Variable und fügen diese als neue Spalte dem Datenframe hinzu.

```
1 # neue Variable mit 12 Werten
2 kinder <- c(1, 4, 3, 1, 2, 3, 2, 1, 4, 2, 3, 4)
3
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
4 # neues Datenframe mit dieser Spalte
5 new <- cbind(MeinDatenframe, kinder)
6
7 # anzeigen
8 new
```

	geschlecht	spitzname	hausnummer	angemeldet	kinder
1	m	Hasi	1	TRUE	1
2	w	Ide	2	TRUE	4
3	d	Momsi	3	FALSE	3
4	m	Ryu	4	TRUE	1
5	w	Dave	5	FALSE	2
6	d	Zoid	6	FALSE	3
7	m	Adu	7	FALSE	2
8	w	Efi	8	TRUE	1
9	d	Ole	9	TRUE	4
10	m	Ray	10	TRUE	2
11	w	Sam	11	FALSE	3
12	d	Emi	12	TRUE	4

Ebenso wie bei der Matrix lassen sich die einzelnen Spalten und Zeilen referenzieren, indem wir in eckigen Klammern die gewünschte Position angeben

```
1 # Zeige nur die erste Spalte
2 MeinDatenframe[,1]
```

```
[1] m w d m w d m w d m w d
Levels: d m w
```

Die einzelnen Spalten des Datenframes lassen sich auch über ihren Namen referenzieren. Hierfür schreiben wir ein Dollarzeichen \$ und hängen den Spaltennamen daran.

```
1 # Zeige nur Spalte "geschlecht"
2 MeinDatenframe$geschlecht
```

```
[1] m w d m w d m w d m w d
Levels: d m w
```

```
1 # Zeige nur Spalte "angemeldet"
2 MeinDatenframe$angemeldet
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] TRUE TRUE FALSE TRUE FALSE FALSE FALSE TRUE TRUE TRUE FALSE TRUE
```

So können wir auch die jeweiligen Datentypen der Spaltenwerte überprüfen.

```
1 # welcher Datentyp liegt in Spalte "angemeldet" vor?  
2 class(MeinDatenframe$angemeldet)
```

```
[1] "logical"
```

```
1 # welcher Datentyp liegt in Spalte "spitzname" vor?  
2 class(MeinDatenframe$spitzname)
```

```
[1] "character"
```

```
1 # welcher Datentyp liegt in der 1. Spalte vor?  
2 class(MeinDatenframe[, 1])
```

```
[1] "factor"
```

Möchten wir uns die *Fälle* (also die Zeilen) ausgeben lassen, erfolgt dies mit

```
1 # Zeige Fall Nummer 4  
2 MeinDatenframe[4,]
```

```
  geschlecht spitzzname hausnummer angemeldet  
4          m      Ryu           4         TRUE
```

oder für Fallserien per

```
1 # Zeige Fälle Nummer 2 bis 5  
2 MeinDatenframe[2:5,]
```

```
  geschlecht spitzzname hausnummer angemeldet  
2          w      Ide           2         TRUE  
3          d     Momsi           3        FALSE  
4          m      Ryu           4         TRUE  
5          w      Dave           5        FALSE
```

Wir können auch bedingte Ausgaben erzeugen.

```
1 # Zeige nur die Fälle mit "hausnummer" kleiner als 5  
2 MeinDatenframe[MeinDatenframe$hausnummer<5, ]
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE

```
1 # Zeige nur die Fälle mit "angemeldet" = TRUE
2 MeinDatenframe[MeinDatenframe$angemeldet==T, ]
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
4	m	Ryu	4	TRUE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
12	d	Emi	12	TRUE

Dies kann auch verknüpft werden

```
1 # Zeige nur die Fälle mit "angemeldet" = TRUE
2 # und "hausnummer" größer 6
3 MeinDatenframe[(MeinDatenframe$angemeldet==T) & (MeinDatenframe$hausnummer>6), ]
```

	geschlecht	spitzname	hausnummer	angemeldet
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
12	d	Emi	12	TRUE



Achtung, Anfängerfehler!



Beachten Sie, dass wir jedes Mal, wenn wir auf eine Spalte des Datenframes referenzieren möchten, den Datenframe-Namen mit einem Dollarzeichen \$ schreiben müssen. Tun wir das nicht, sucht R nach einer Variable im Workspace, und nutzt dann *deren* Werte. Dies ist ein häufiger Anfängerfehler!

Die ständige Angabe des Datenframes macht die Befehle recht lang. Mit der Funktion `with()` können wir uns die Referenzierung mit \$ sparen. Wir übergeben der Funktion `with()` unser Datenframe, und sagen dann, was damit getan werden soll.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # mit Funktion with() wird es leichter
2 with(MeinDatenframe, MeinDatenframe[hausnummer>4 & angemeldet==F, ])
```

```
geschlecht spitzname hausnummer angemeldet
5          w      Dave          5      FALSE
6          d      Zoid          6      FALSE
7          m      Adu           7      FALSE
11         w      Sam          11      FALSE
```

```
1 # mit Funktion with() wird es leichter
2 with(MeinDatenframe, spitzname[hausnummer>4 & angemeldet==F])
```

```
[1] "Dave" "Zoid" "Adu"  "Sam"
```

Das funktioniert in Kombination mit jeder anderen Funktion.

```
1 # Summe der Hausnummern
2 with(MeinDatenframe, sum(hausnummer))
```

```
[1] 78
```

```
1 # Häufigkeit von geschlecht
2 with(MeinDatenframe, table(geschlecht))
```

```
geschlecht
d m w
4 4 4
```

Ähnlich wie bei Matrizen können wir die Zeilen- und Spaltentitel anpassen. Mit `colnames()` können wir die Spalten umbenennen.

```
1 # benenne Spalten des Datenframes
2 colnames(MeinDatenframe) <- c("Sex", "Nickname", "House", "confirmed")
3
4 # anzeigen
5 MeinDatenframe
```


1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

	Sex	Nickname	House	confirmed
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE

Dementsprechend können mit `rownames()` die Zeilen umbenannt werden.

```
1 # benenne Spalten des Datenframes
2 rownames(MeinDatenframe) <- c("Eins", "Zwei", "Drei", "Vier", "Fünf", "Sechs", "Sieben",
3 "Acht", "Neun", "Zehn", "Elf", "Zwölf")
4
5 # anzeigen
MeinDatenframe
```

	Sex	Nickname	House	confirmed
Eins	m	Hasi	1	TRUE
Zwei	w	Ide	2	TRUE
Drei	d	Momsi	3	FALSE
Vier	m	Ryu	4	TRUE
Fünf	w	Dave	5	FALSE
Sechs	d	Zoid	6	FALSE
Sieben	m	Adu	7	FALSE
Acht	w	Efi	8	TRUE
Neun	d	Ole	9	TRUE
Zehn	m	Ray	10	TRUE
Elf	w	Sam	11	FALSE
Zwölf	d	Emi	12	TRUE

Das sollte bei einem Datenframe nach dem *Tidy Data* Prinzip (siehe Kapitel 1.6 auf Seite 147) aber niemals notwendig sein.

Mit der Funktion `class()` kann die Datenklasse angezeigt werden.

```
1 # welcher Datentyp ist "MeinDatenframe"
2 class(MeinDatenframe)
```

```
[1] "data.frame"
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

Mit der Funktion `is.data.frame()` kann zudem geprüft werden, ob ein Objekt ein Faktor ist.

```
1 # ist "MeinDatenframe" ein Datenframe?  
2 is.data.frame(MeinDatenframe)
```

```
[1] TRUE
```

```
1 # ist "mymatrix" ein Datenframe?  
2 is.data.frame(mymatrix)
```

```
[1] FALSE
```

In RStudio werden die Variablen und Datensätze im Datenfenster oben rechts angezeigt (Abbildung 1.27).

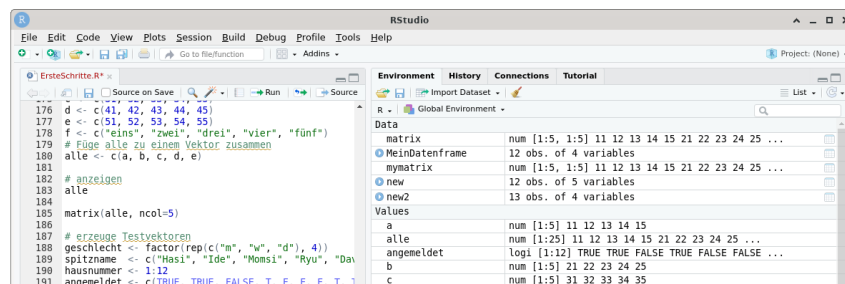


Abb. 1.27: Klicken Sie oben rechts auf einen Datensatz

Wenn Sie hier auf einen Datensatz klicken, z.B. auf `MeinDatenframe`, so werden Ihnen die *Inhalte* (also die Werte) des Datensatzes im Scriptfenster angezeigt (Abbildung 1.28).

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

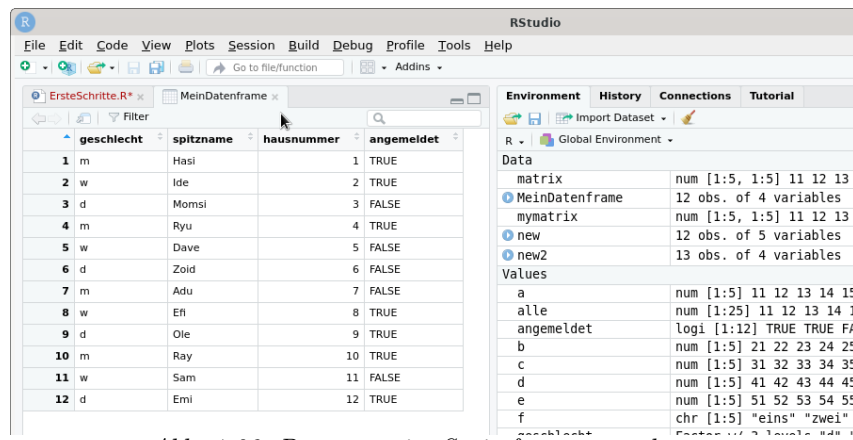


Abb. 1.28: Datensatz im Scriptfenster anschauen

Wir können mit der Funktion `as.data.frame()` Objekte in ein Datenframe umwandeln. Schauen wir uns erneut die auf Seite 45 erzeugte Matrix der Beschäftigten in den Pflegeberufen an.

```
1 # zeige Matrix
2 Pflegeberufe
```

	1999	2001	2003	2005	2007	2009	2011	2013	2015
Krankenpflegeassistentz	16624	19061	19478	21537	27731	36481	46517	54371	64127
Altenpflegehilfe	55770	52710	49727	45776	48326	47903	47978	48363	49507
Kinderkrankenpflege	47779	48203	48822	48519	49080	49307	48291	48937	48913
Krankenpflege	430983	436767	444783	449355	457322	465446	468192	472580	476416
Altenpflege	109161	124879	141965	158817	178902	194195	208304	227154	246412

Mit der Funktion `as.data.frame()` wandeln wir die Matrix in ein Datenframe um.

```
1 # wandle Matrix in Datenframe um
2 Pflegeframe <- as.data.frame(Pflegeberufe)
3
4 # anzeigen
5 Pflegeframe
```

	1999	2001	2003	2005	2007	2009	2011	2013	2015
Krankenpflegeassistentz	16624	19061	19478	21537	27731	36481	46517	54371	64127
Altenpflegehilfe	55770	52710	49727	45776	48326	47903	47978	48363	49507
Kinderkrankenpflege	47779	48203	48822	48519	49080	49307	48291	48937	48913
Krankenpflege	430983	436767	444783	449355	457322	465446	468192	472580	476416
Altenpflege	109161	124879	141965	158817	178902	194195	208304	227154	246412

Kommen wir noch einmal auf das Konzept *Tidy Data*^[6] zurück (siehe Abschnitt 1.6 auf Seite 147). Ein Datenframe sollte möglichst so aufgebaut sein, dass jeweils ein Fall pro Zeile abgebildet wird. Das ist bei unserem Datenframe `Pflegeframe` aber nicht der Fall.

Ein *Datenframe* würde sich aus der Matrix jede Beschäftigtenzahl einzeln vornehmen, um dann zu fragen “aus welchem Jahr stammt diese Zahl?” und “aus welcher Berufsgruppe stammt diese Zahl?”. Das heisst, es würde hinterher die-
ser Struktur folgen:

	Jahr	Berufsgruppe	Wert
1	1999	Krankenpflege	430983
2	2001	Krankenpflege	436767
3	2001	Altenpflege	124879
4	2003	Altenpflegehilfe	49727
		(. . .)	

Man spricht in diesem Zusammenhang von `long table` und `wide table`. Die Matrix der Pflegeberufe stellt dabei die *wide table*, die *breite* Tabelle dar. “Breit” bedeutet, dass die Tabelle, wenn wir ihr nun 10 weitere Jahrgänge hinzufügen würden, immer breiter und breiter werden würde.

Unser angestrebtes Tidy-Data-Datenframe ist vom Typ *long table*, da die Tabelle, wenn wir ihr Daten hinzufügen würden, immer *länger* und länger werden würde.

Wie formen wir unsere Matrix in ein *Tidy Data*-Datenframe, also in eine *long table*, um?

Mit der Funktion `expand.grid()` kann ein Datenframe mit Wertepaaren erzeugt werden. Für unser Beispiel mit den Pflegeberufen brauchen wir im Datenframe je eine Zeile für alle möglichen Kombinationen aus `Jahr` und `Berufsgruppe`. Die Funktion `expand.grid()` erzeugt genau solche Paarungen. Idealerweise sind die benötigten Werte (alle `Jahre` und alle `Berufsgruppen`) als Zeilen- und Spaltennamen in der Matrix `Pflegeberufe` vorhanden.

```

1 # erzeuge ein Tidy-Data-Datenframe
2 # mit allen möglichen Kombinationen
3 # aus Jahren und Berufsgruppen
4 new <- expand.grid( colnames(Pflegeberufe), rownames(Pflegeberufe))

```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
5  
6 # anzeigen  
7 new
```

```
  Var1      Var2  
1 1999 Krankenpflegeassistenz  
2 2001 Krankenpflegeassistenz  
3 2003 Krankenpflegeassistenz  
4 2005 Krankenpflegeassistenz  
5 2007 Krankenpflegeassistenz  
6 2009 Krankenpflegeassistenz  
7 2011 Krankenpflegeassistenz  
8 2013 Krankenpflegeassistenz  
9 2015 Krankenpflegeassistenz  
10 1999      Altenpflegehilfe  
11 2001      Altenpflegehilfe  
12 2003      Altenpflegehilfe  
   ( . . . )
```

Mit der Funktion `cbind()` können wir nun die Zahlenwerte aus der Matrix als neue Spalte an das Datenframe anhängen. Dafür müssen die Werte in Form eines Vektors vorliegen.

Um die Matrix als Vektor auszugeben nutzen wir die Funktion `as.vector()`

```
1 # stelle die Matrix als Vektor dar  
2 as.vector(Pflegeberufe)
```

```
[1] 16624 55770 47779 430983 109161 19061 52710 48203 436767 124879 19478  
[12] 49727 48822 444783 141965 21537 45776 48519 449355 158817 27731 48326  
[23] 49080 457322 178902 36481 47903 49307 465446 194195 46517 47978 48291  
[34] 468192 208304 54371 48363 48937 472580 227154 64127 49507 48913 476416  
[45] 246412
```

Wie Sie sehen, überführt  die Matrix spaltenweise in den Vektor. Für unser neues Datenframe bräuchten wir aber einen zeilenorientierten Vektor, damit er mit der Reihenfolge der Einträge (Paarung aus **Jahr** und **Berufsgruppe**) übereinstimmt. Um einen reihenorientierten Vektor zu erzeugen muss die Matrix mit der Funktion `t()` transponiert werden.

```
1 # stelle die Matrix als Vektor dar  
2 # zeilenorientiert  
3 as.vector(t(Pflegeberufe))
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[1] 16624 19061 19478 21537 27731 36481 46517 54371 64127 55770 52710
[12] 49727 45776 48326 47903 47978 48363 49507 47779 48203 48822 48519
[23] 49080 49307 48291 48937 48913 430983 436767 444783 449355 457322 465446
[34] 468192 472580 476416 109161 124879 141965 158817 178902 194195 208304 227154
[45] 246412
```

Diesen Vektor fügen wir nun per `cbind()` dem Datenframe als neue Spalte hinzu

```
1 # füge Spalte hinzu
2 Pflegeframe <- cbind(new, as.vector(t(Pflegeberufe)))
3
4 # benenne die Spalten neu
5 colnames(Pflegeframe) <- c("Jahr", "Berufsgruppe", "Anzahl")
6 # zeige an
7 Pflegeframe
```

	Jahr	Berufsgruppe	Anzahl
1	1999	Krankenpflegeassistent	16624
2	2001	Krankenpflegeassistent	19061
3	2003	Krankenpflegeassistent	19478
4	2005	Krankenpflegeassistent	21537
5	2007	Krankenpflegeassistent	27731
6	2009	Krankenpflegeassistent	36481
7	2011	Krankenpflegeassistent	46517
8	2013	Krankenpflegeassistent	54371
9	2015	Krankenpflegeassistent	64127
10	1999	Altenpflegehilfe	55770
11	2001	Altenpflegehilfe	52710
12	2003	Altenpflegehilfe	49727
	(. . .)		

Der vollständige Code, ohne Hilfsdatenframe `new`, zur Überführung der Matrix `Pflegeberufe` in das Tidy-Data-Datenframe `Pflegeframe` sieht also so aus:

```
1 # füge Spalte hinzu
2 Pflegeframe <- cbind(expand.grid( colnames(Pflegeberufe), rownames(Pflegeberufe)),
3 as.vector(t(Pflegeberufe)))
4
5 # benenne die Spalten neu
6 colnames(Pflegeframe) <- c("Jahr", "Berufsgruppe", "Anzahl")
7 # zeige erste 12 Fälle an
8 head(Pflegeframe, 12)
```

	Jahr	Berufsgruppe	Anzahl
1	1999	Krankenpflegeassistent	16624
2	2001	Krankenpflegeassistent	19061
3	2003	Krankenpflegeassistent	19478
4	2005	Krankenpflegeassistent	21537

```
5 2007 Krankenpflegeassistentz 27731
6 2009 Krankenpflegeassistentz 36481
7 2011 Krankenpflegeassistentz 46517
8 2013 Krankenpflegeassistentz 54371
9 2015 Krankenpflegeassistentz 64127
10 1999 Altenpflegehilfe 55770
11 2001 Altenpflegehilfe 52710
12 2003 Altenpflegehilfe 49727
```

1.2.3.5.6 Listen

In der Datenklasse Listen können beliebige Datenobjekte (Vektor, Faktor, Datenframe, Matrizen) zusammengefasst werden. Listen sind also eine Ansammlung an Datenobjekten, ähnlich wie ein Schrank oder ein Koffer, in welchem man “sein Zeug” ablegt. Wir generieren testweise eine Liste aus den Datenobjekten, die wir bislang erzeugt haben. Dies erfolgt in  mit der Funktion `list()`.

```
1 # erzeuge eine Liste aus den Datenobjekten
2 # "MeinDatenframe", "mymatrix", "geschlecht" und "logical"
3 MeineListe <- list(MeinDatenframe, mymatrix, geschlecht, logical)
4
5 # anzeigen
6 MeineListe
```

```
[[1]]
  geschlecht spitzname hausnummer angemeldet
1          m      Hasi           1        TRUE
2          w       Ide           2        TRUE
3          d     Momsi           3       FALSE
4          m       Ryu           4        TRUE
5          w      Dave           5       FALSE
6          d      Zoid           6       FALSE
7          m       Adu           7       FALSE
8          w       Efi           8        TRUE
9          d      Ole           9        TRUE
10         m      Ray          10        TRUE
11         w      Sam          11       FALSE
12         d      Emi          12        TRUE

[[2]]
      a  b  c  d  e
I  11 21 31 41 51
II 12 22 32 42 52
III 13 23 33 43 53
IV 14 24 34 44 54
V 15 25 35 45 55
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
[[3]]
[1] m w d m w d m w d m w d
Levels: d m w

[[4]]
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[13] TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

Wie Sie sehen, werden die einzelnen Positionen der Datenobjekte durch doppelte eckige Klammer angezeigt (`[[1]]` ist unser Datenframe, `[[2]]` unsere Matrix, usw.) und können über diese auch referenziert werden.

```
1 # zeige Objekt 1 (= unser Datenframe)
2 MeineListe[[1]]
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE

```
1 # zeige Objekt 3 (= variable "geschlecht")
2 MeineListe[[3]]
```

```
[1] m w d m w d m w d m w d
Levels: d m w
```

Die Werte der jeweiligen Objekte können anschließend wie gewohnt referenziert werden.

```
1 # zeige Objekt 3, aber nur den 5. Wert
2 MeineListe[[3]][5]
```

```
[1] w
Levels: d m w
```



```
1 # zeige Objekt 2, aber nur die 3. Spalte
2 MeineListe[[2]][, 3]
```

```
  I  II III  IV  V
31 32 33 34 35
```

1.2.3.6 Fehlende Daten

Fehlende Daten werden in  mit dem Ausdruck `NA` dargestellt. Angenommen Sie erfragen von zehn Personen das Alter, und zwei Personen geben keine Auskunft, dann könnten man das wie folgt in einem Vektor darstellen.

```
1 # Alter von 10 Personen
2 alter <- c(28, 30, NA, 21, 27, 43, NA, 35, 22, 18)
3 # anzeigen
4 alter
```

```
[1] 28 30 NA 21 27 43 NA 35 22 18
```

Möchten wir nun die Summe der Alterswerte bestimmen, meldet  uns `NA` zurück

```
1 # Summe der Altersangaben
2 sum(alter)
```

```
[1] NA
```

Die Berechnung wird nicht durchgeführt, da fehlende Werte in der Reihe vorkommen. Möchte man die Berechnung erzwingen, kann (bei fast jeder -Funktion) der Parameter `na.rm` (für *NA remove*; entferne NA) auf `TRUE` gesetzt werden. So werden alle fehlenden Daten ignoriert und die Berechnungen nur mit der Restmenge durchgeführt.

```
1 # Summe der Altersangaben OHNE NA
2 sum(alter, na.rm=TRUE)
```

```
[1] 224
```

Mit der Funktion `is.na()` kann geprüft werden, ob fehlende Werte enthalten sind

```
1 # sind Werte in "alter" NA?  
2 is.na(alter)
```

```
[1] FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
```

Möchten wir wissen, an welcher Position sich die NA-Einträge befinden können wir die Funktion `which()` mit `is.na()` kombinieren

```
1 # welche Position haben die NAs in "alter"  
2 which(is.na(alter))
```

```
[1] 3 7
```

Sind *keine* NAs enthalten, meldet  ein `integer(0)` zurück

```
1 # NAs in "Pflegeframe"  
2 which(is.na(Pflegeframe))
```

```
integer(0)
```

1.2.3.7 Umgang mit Datensätzen

Mit der Funktion `str()` (für *structure*) erhalten wir eine erste Beschreibung des Datensatzes.

```
1 # Übersicht von "MeinDatenframe"
2 str(MeinDatenframe)
```

```
'data.frame':      12 obs. of  4 variables:
 $ geschlecht: Factor w/ 3 levels "d","m","w": 2 3 1 2 3 1 2 3 1 2 ...
 $ spitzname : chr  "Hasi" "Ide" "Moms" "Ryu" ...
 $ hausnummer: int  1 2 3 4 5 6 7 8 9 10 ...
 $ angemeldet: logi  TRUE TRUE FALSE TRUE FALSE FALSE ...
```

Das Datenframe `MeinDatenframe` enthält 12 Beobachtungen von 4 Variablen. Variable `geschlecht` ist ein Faktor mit 3 Ausprägungen, usw.

In RStudio werden diese Informationen im Datenfenster angezeigt (Abbildung 1.29).

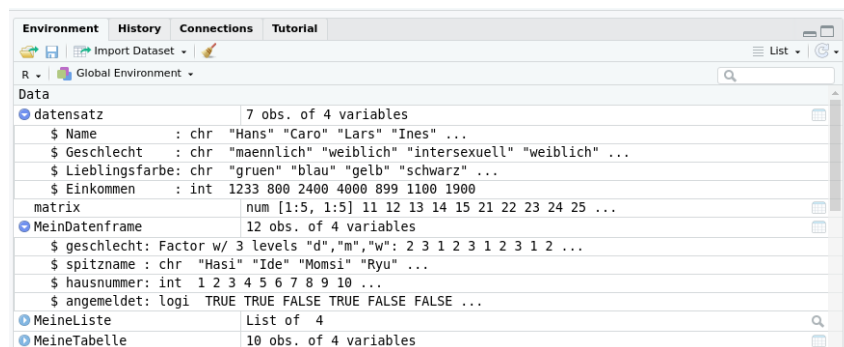


Abb. 1.29: Datenfenster zeigt `str()`-Ausgabe

Mit der Funktion `head()` können Sie sich einen weiteren Überblick über den Datensatz verschaffen. Die Funktion `head()` gibt die ersten 6 Zeilen des Datensatzes aus.

```
1 # zeige die ersten 6 Reihen von "MeinDatenframe"
2 head(MeinDatenframe)
```

```
  geschlecht spitzname hausnummer angemeldet
1          m      Hasi           1         TRUE
2          w       Ide           2         TRUE
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE

Möchten wir mehr als 6 Einträge sehen, können wir dies an `head()` übergeben:

```
1 # zeige die ersten 10 Reihen von "MeinDatenframe"  
2 head(MeinDatenframe, 10)
```

	geschlecht	spitzname	hausnummer	angemeldet
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Momsi	3	FALSE
4	m	Ryu	4	TRUE
5	w	Dave	5	FALSE
6	d	Zoid	6	FALSE
7	m	Adu	7	FALSE
8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE

Im Gegensatz dazu zeigt `tail()` die letzten Reihen des Datenframes an.

```
1 # zeige die letzten 5 Reihen von "MeinDatenframe"  
2 tail(MeinDatenframe, 5)
```

8	w	Efi	8	TRUE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
11	w	Sam	11	FALSE
12	d	Emi	12	TRUE

1.2.3.7.1 Sortieren

Mit der Funktion `sort()` können Datenreihen auf- und absteigend sortiert werden. Standardmäßig wird aufsteigend sortiert

```
1 # sortiere "hausnummer" von "MeinDatenframe"  
2 # aufsteigend  
3 sort(MeinDatenframe$hausnummer)
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12
```

Um absteigend zu sortieren, muss der Parameter `decreasing` auf `TRUE` gesetzt werden

```
1 # sortiere "spitzname" von "MeinDatenframe"
2 # absteigend
3 sort(MeinDatenframe$spitzname, decreasing=T)
```

```
[1] "Zoid" "Sam" "Ryu" "Ray" "Ole" "Moms" "Ide" "Hasi"
[9] "Emi" "Efi" "Dave" "Adu"
```

Mit der Funktion `order()` werden die *Zeilenpositionen* ausgegeben, an deren Stelle sich die von `sort()` sortierten Daten befinden.

```
1 # sortiere "spitznamen" von "MeinDatenframe"
2 # aufsteigend, und zeige Positionen an
3 order(MeinDatenframe$spitzname)
```

```
[1] 7 5 8 12 1 2 3 9 10 4 11 6
```

Das kann man nun verwenden, um den kompletten Datensatz in dieser Ordnung anzuzeigen. Wir können ja durch Anhängen von eckigen Klammern die gewünschten Zeilen und Spalten des Datenframes referenzieren. Also übergeben wir die Positionsreihenfolge von `order()` als Argument in die eckige Klammer.

```
1 # sortiere "spitznamen" von "MeinDatenframe"
2 # aufsteigend, zeige ganzes Datenframe an
3 MeinDatenframe[ order(MeinDatenframe$spitzname), ]
```

	geschlecht	spitzname	hausnummer	angemeldet
7	m	Adu	7	FALSE
5	w	Dave	5	FALSE
8	w	Efi	8	TRUE
12	d	Emi	12	TRUE
1	m	Hasi	1	TRUE
2	w	Ide	2	TRUE
3	d	Moms	3	FALSE
9	d	Ole	9	TRUE
10	m	Ray	10	TRUE
4	m	Ryu	4	TRUE
11	w	Sam	11	FALSE
6	d	Zoid	6	FALSE

Oder andersherum

```
1 # sortiere "spitznamen" von "MeinDatenframe"
2 # absteigend, zeige ganzes Datenframe an
3 MeinDatenframe[ order(MeinDatenframe$spitzname, decreasing=T), ]
```

	geschlecht	spitzname	hausnummer	angemeldet
6	d	Zoid	6	FALSE
11	w	Sam	11	FALSE
4	m	Ryu	4	TRUE
10	m	Ray	10	TRUE
9	d	Ole	9	TRUE
3	d	Momsi	3	FALSE
2	w	Ide	2	TRUE
1	m	Hasi	1	TRUE
12	d	Emi	12	TRUE
8	w	Efi	8	TRUE
5	w	Dave	5	FALSE
7	m	Adu	7	FALSE

1.2.3.7.2 Teilgruppen erstellen

Mit der Funktion `subset()` können Teilgruppen aus einem Datenframe erstellt werden.

```
1 # erstelle eine Teilgruppe aus MeinDatenframe
2 # für "geschlecht" = d
3 subset(MeinDatenframe, geschlecht=="d")
```

	geschlecht	spitzname	hausnummer	angemeldet
3	d	Momsi	3	FALSE
6	d	Zoid	6	FALSE
9	d	Ole	9	TRUE
12	d	Emi	12	TRUE

Über den Parameter `select` kann angegeben werden, welche Spalten übernommen werden sollen.

```
1 # erstelle eine Teilgruppe aus MeinDatenframe
2 # für "geschlecht" = d
3 # aber nur Spalte "spitzname" und "hausnummer"
4 subset(MeinDatenframe, geschlecht=="d", select = c(spitzname, hausnummer))
```

	spitzname	hausnummer
3	Momsi	3
6	Zoid	6
9	Ole	9
12	Emi	12

Eine andere Möglichkeit zur Erstellung von Teilgruppen kann über die Referenzierung der Zeilen und Spalten des Datenframes (eckige Klammern) erfolgen

```
1 # erstelle eine Teilgruppe aus MeinDatenframe
2 # für "geschlecht" = w
3 MeinDatenframe[ MeinDatenframe$geschlecht == "w", ]
```

	geschlecht	spitzname	hausnummer	angemeldet
2	w	Ide	2	TRUE
5	w	Dave	5	FALSE
8	w	Efi	8	TRUE
11	w	Sam	11	FALSE

Auf diese Weise können die Bedingungen auch mit einem & kombiniert werden

```
1 # erstelle eine Teilgruppe aus MeinDatenframe
2 # für "geschlecht" = w und "hausnummer" größer 5
3 MeinDatenframe[ (MeinDatenframe$geschlecht == "w") & (MeinDatenframe$hausnummer > 5) , ]
```

	geschlecht	spitzname	hausnummer	angemeldet
8	w	Efi	8	TRUE
11	w	Sam	11	FALSE

Über Angabe der Spaltennummern können wir das Teilset noch weiter zusammenschrumpfen lassen

```
1 # erstelle eine Teilgruppe aus MeinDatenframe
2 # für "geschlecht" = w und "angemeldet" = TRUE
3 # aber nur Spalte "spitzname" und "hausnummer"
4 MeinDatenframe[(MeinDatenframe$geschlecht == "w") & (MeinDatenframe$angemeldet == T),
  c(2,3)]
```

	spitzname	hausnummer
2	Ide	2
8	Efi	8

Möchte man auf die Referenzierung per Dollarzeichen \$ verzichten, kann die Funktion `with()` verwendet werden.

```
1 # with() macht es leichter
2 with(MeinDatenframe, MeinDatenframe[(geschlecht == "w") & (angemeldet == T), c(2,3)])
```

	spitzname	hausnummer
2	Ide	2
8	Efi	8

1.2.3.8 Pipe

Ein wichtiger Operator für den Umgang mit Datensätzen ist die **Pipe**. Diese ist fester Bestandteil des Tidyverse (siehe Abschnitt ??) und hat sich dort so bewährt, dass sie seit  Version 4.1 ebenfalls im “Standard”- implementiert ist.

Die **Pipe** erlaubt es, *Datenströme* weiterzuleiten.

Hierfür wurde ein eigener Operator implementiert, die Zeichenkette `|>`

Sie bedeutet so viel wie “*und dann*”.

Zur Erklärung sei der "klassische Weg" der Arbeitsschritte in  aufgezeigt:

```

1  # wir machen etwas, und speichern es in ein Objekt
2  habe.getan <- mache.etwas(Datensatz)
3
4  # dann machen wir etwas weiteres,
5  # und speichern wieder in ein Objekt
6  habe.weiteres.getan <- mache.weiteres(habe.getan)
7
8  # und kommen schließlich zum Endergebnis
9  endergebnis <- mache.noch.letzte.Sache(habe.weiteres.getan)

```

Mit Einsatz der **Pipe** wird dieser Prozess quasi umgekehrt in

```

1  # Speichere Endergebnis
2  endergebnis <- Datensatz |>
3      gruppriere.nach.geschlecht() |>
4      sortiere Alter aufsteigend() |>
5      nimm die letzten 4 Werte()

```

Die liest sich in etwa so:

1. “Speichere etwas ins Objekt `endergebnis`, und das geht so...
2. nimm den `Datensatz` *und dann*
3. gruppriere die Daten nach `geschlecht` *und dann*

4. sortiere nach `Alter` aufsteigend *und dann*
5. nimm die letzten 4 Werte.”

Die `Pipe` reicht das jeweilige Ergebnis (den *Datenstrom*) an die nächste Code-Zeile weiter. Erst wenn die letzte Zeile durchgelaufen ist, wird das Resultat in `endergebnis` gespeichert.

Dies liest sich zu Beginn evtl. etwas ungewohnt, aber Sie können erkennen, dass die Befehle so wesentlich übersichtlicher und die einzelnen *Manipulationsschritte* nachvollziehbarer geworden sind. Auch kann man sich diese Art der “Grammatik” relativ leicht merken.

Standardmäßig übergibt die `Pipe` das Objekt auf der linken Seite an das erste Argument der Funktion auf der rechten Seite, so dass `x |> f(y)` äquivalent ist zu `f(x, y)`, und `x |> f(y) |> g(z)` ist äquivalent zu `g(f(x, y), z)`.

Mehr über die “Original”-`Pipe` `%>%` des Tidyverse sowie zu den Unterschieden zur Standard-`Pipe` `|>` finden Sie im Abschnitt 1.6.2 auf Seite 151.

In RStudio können Sie einstellen, ob Sie mit der Tastenkombination `[STRG] + [ALT] + [M]` die Tidyverse-`Pipe` oder die Standard-`Pipe` erzeugen möchten. Wählen Sie hierzu im Menü “Tools” → “Global Options” → “Code” aus, siehe Abbildung 1.30.

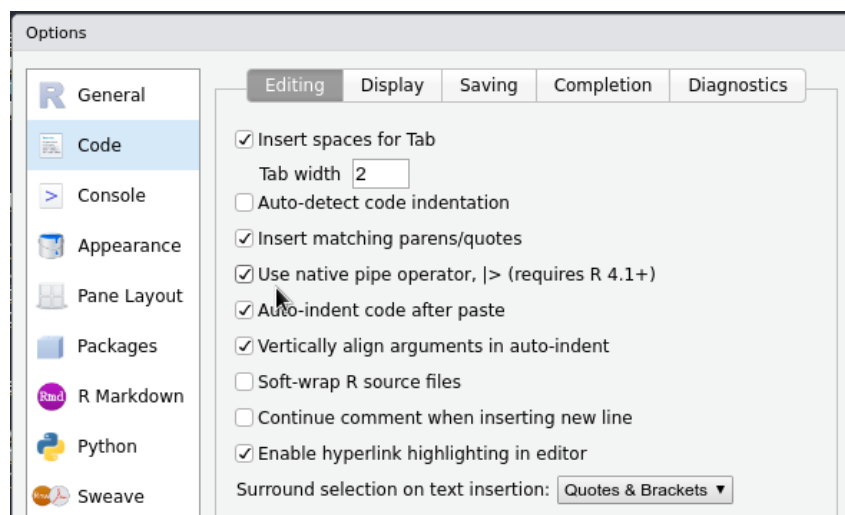


Abb. 1.30: Datei Tastenkombination für Pipe

1.2.3.9 Daten laden und speichern

In  können Daten leicht geladen und gespeichert werden. Eine -Datei hat die Dateiendung `.RData`. In der Datei können “beliebig” viele Objekte (Variablen, Datenframes, usw.) enthalten sein.

Mit der Funktion `save.image()` werden alle Objekte im Speicher, also die aktuelle “Worksession”, als “unsichtbare” Datei `.RData` ins aktuelle Arbeitsverzeichnis geschrieben.

```
1 # speichere die aktuelle Sitzung
2 save.image()
```

In RStudio können Sie im Ausgabefenster (unten rechts) auf den Reiter **Files** klicken. Sie sehen Ihr aktuelles Arbeitsverzeichnis, und darin die -Datei `.RData` (siehe Abbildung 1.31).

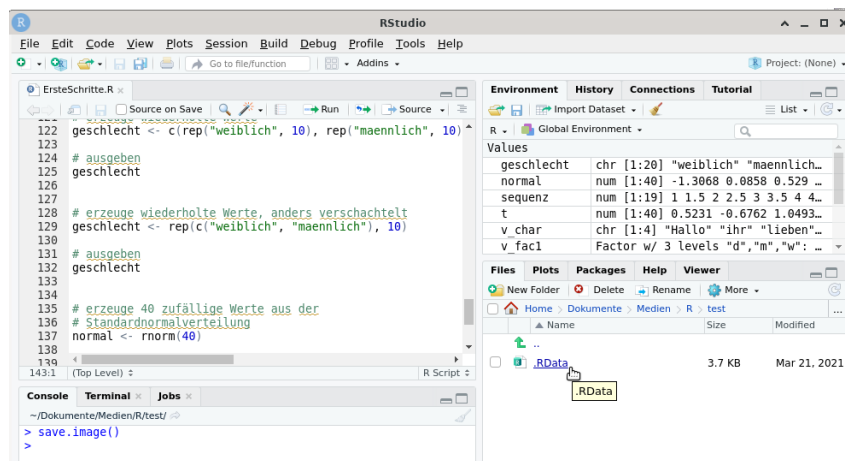


Abb. 1.31: Dateifenster *Files* unten rechts

Das ist übrigens die selbe Datei, die angelegt wird, wenn Sie RStudio beenden, und “speichere Arbeitsspeicher” klicken. Das heisst, dass diese Datei unter Umständen bald wieder überschrieben wird.

Der Funktion `save.image()` kann auch ein gewünschter Dateiname übergeben werden.

```
1 # speichere die aktuelle Sitzung in Datei "MeineSitzung.RData"
2 save.image("MeineSitzung.RData")
```

Die Datei `MeineSitzung.RData` wird in RStudio im Ausgabefenster angezeigt (siehe Abbildung 1.32).



Abb. 1.32: Datei `MeineSitzung.RData`

Wenn nur einzelne Objekte gespeichert werden sollen, wird die `save()`-Funktion angewandt. Mit dem `file`-Parameter wird die gewünschte Zielfile angegeben.

```
1 # speichere nur die Variable "sequenz"
2 # in Datei "Sequenz.RData"
3 save(sequenz, file="Sequenz.RData")
```

```
1 # speichere die Objekte "sequenz" "normal" und "geschlecht"
2 # in Datei "MeineVariablen.RData"
3 save(sequenz, normal, geschlecht, file="MeineVariablen.RData")
```

Daten können mit dem `load()`-Befehl geladen werden.

```
1 # lade Datei "MeineVariablen.RData"
2 load("MeineVariablen.RData")
```

In RStudio kann im Ausgabefenster unter dem Reiter **Files** einfach auf eine `R`-Datei geklickt werden, um diese zu laden (Abbildung 1.33).

Wenn die Datei im Internet liegt, muss sie mit Hilfe der `url()`-Funktion geladen werden

```
1 # Test-Datensatz aus dem Internet laden
2 # (dieser enthält das Datenframe "ordinalSample")
3 load(url("https://www.produnis.de/R/OrdinalSample.RData"))
4
5 # Daten anschauen mit head(ordinalSample)
6 head(ordinalSample)
```

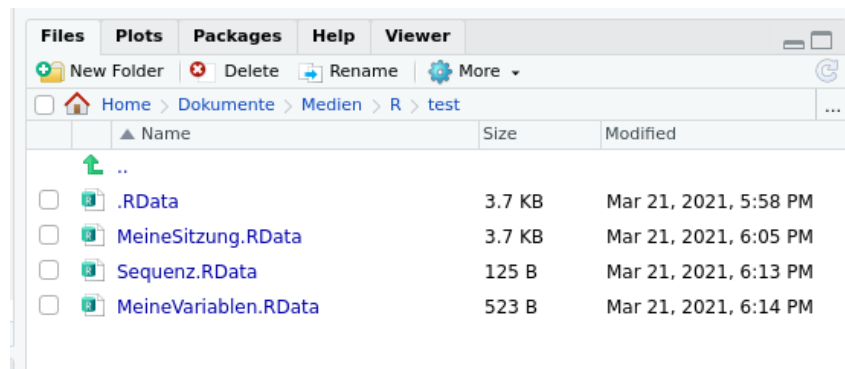


Abb. 1.33: verfügbare R-Dateien

1.2.3.10 Arbeitsverzeichnis

Das aktuelle Arbeitsverzeichnis kann per `getwd()` angezeigt werden.

```
1 # Arbeitsverzeichnis ausgeben
2 getwd()
```

```
[1] "/home/produnis/Dokumente/Medien/R/test"
```

Mit dem Befehl `dir()` werden die Dateien des aktuellen Arbeitsverzeichnisses angezeigt.

```
1 # Inhalt des Arbeitsverzeichnis ausgeben
2 dir()
```

```
[1] "MeineSitzung.RData" "MeineVariablen.RData" "Sequenz.RData"
```

Mit dem Befehl `setwd()` kann das aktuelle Arbeitsverzeichnis gewechselt werden.

```
1 # Arbeitsverzeichnis wechseln
2 setwd("/tmp/R/")
```

Auf Dauer kann es einfacher sein, ein eigenes Arbeitsverzeichnis beim Startvorgang einzustellen (siehe Abschnitt 1.2.2.1 auf Seite 5).

1.2.3.11 Daten importieren

In  können Daten auf vielfältige Weise importiert werden. Das Grundproblem beim “Tausch” von Daten zwischen verschiedenen Anwendungen (z.B. SPSS oder Statistica) liegt im verwendeten Format. So können native SPSS-Dateien nur mit einem Zusatzpaket importiert werden, für Statistica gibt es ein solches Zusatzpaket (derzeit) nicht. In beiden Fällen ist es aber möglich, die Daten über das CSV-Format zu tauschen. Hierfür speichert man in SPSS den Datensatz als CSV-Datei ab und liest diese in  ein.



Achtung, Anfängerfehler!



Bedenken Sie ebenfalls, dass  Dezimalstellen mit einem Punkt angibt, und nicht mit einem Komma. So kann es zu Fehlern kommen, wenn die zu importierenden Werte nicht ebenfalls die Dezimalstellen mit einem Punkt angeben! Dies ist ein weiterer häufiger Anfängerfehler!

1.2.3.11.1 Import aus Textdateien

In einer Textdatei (mit Dateiondung `.txt`) liegen unsere Daten wie folgt vor:

```
Geschlecht Alter Gewicht Groesse
m 28 80 170
w 18 55 174
w 25 74 183
m 29 101 190
m 21 84 185
w 19 74 178
w 27 65 169
w 26 56 163
m 31 88 189
m 22 78 184
```

Angenommen, diese Textdatei heiße `Datentabelle.txt`, und läge in unserem Arbeitsverzeichnis, dann können wir die Daten mit der Funktion `read.table()` in  einlesen.

```
1 # importiere Daten aus "Datentabelle.txt"
2 MeineTabelle <- read.table("Datentabelle.txt", header=TRUE)
3
4 # ausgeben
5 MeineTabelle
```

	Geschlecht	Alter	Gewicht	Groesse
1	m	28	80	170
2	w	18	55	174
3	w	25	74	183
4	m	29	101	190
5	m	21	84	185
6	w	19	74	178
7	w	27	65	169
8	w	26	56	163
9	m	31	88	189
10	m	22	78	184

Liegt die Datei im Internet, kann die Adresse mit der Funktion `url()` übergeben werden.

```

1 # importiere Daten aus "Datentabelle.txt"
2 MeineTabelle <- read.table(url("https://www.produzis.de/R/Datentabelle.txt"), header=TRUE)
3
4 # ausgeben
5 MeineTabelle

```

	Geschlecht	Alter	Gewicht	Groesse
1	m	28	80	170
2	w	18	55	174
3	w	25	74	183
4	m	29	101	190
5	m	21	84	185
6	w	19	74	178
7	w	27	65	169
8	w	26	56	163
9	m	31	88	189
10	m	22	78	184

Der Parameter `header=TRUE` gibt an, dass in der ersten Zeile der Textdatei die Spaltennamen (Variablennamen) angegeben sind.

Da wir sonst keine Parameter übergeben haben, geht  davon aus, dass die Werte durch ein Leerzeichen getrennt sind, was in unserem Beispiel auch stimmt. Sollte die Tabelle in diesem Format vorliegen (die Daten sind mit einem Komma getrennt) ...

```

Geschlecht,Alter,Gewicht,Groesse
m,28,80,170
w,18,55,174
w,25,74,183
m,29,101,190
m,21,84,185
w,19,74,178

```

```
w,27,65,169
w,26,56,163
m,31,88,189
m,22,78,184
```

...dann müssen wir mit der Option `sep` (für *Datenseparator*) angeben, dass die Daten mit einem Komma getrennt sind.

```
1 # importiere Daten aus "Datentabelle.txt"
2 # Daten sind mit Komma getrennt
3 MeineTabelle <- read.table("Datentabelle.txt", sep=",", header=TRUE)
```

In beiden Fällen wurde die Texttabelle als Datenframe in  importiert. Die Funktion `class()` bestätigt uns dies.

```
1 # welche Datenklasse wurde erzeugt?
2 class(MeineTabelle)
```

```
[1] "data.frame"
```

Aber welche Datentypen wurden in den *Spalten* erzeugt?

```
1 # welcher Datentyp liegt in Spalte 1 vor?
2 class(MeineTabelle[,1])
```

```
[1] "character"
```

```
1 # welcher Datentyp liegt in Spalte "Alter" vor?
2 class(MeineTabelle$Alter)
```

```
[1] "integer"
```

Die Spalte `Geschlecht` wurde als `character` angelegt, die Variable `Alter` ist numerisch.

Möchten wir `Geschlecht` in einen Faktor umwandeln, “überschreiben” wir einfach die Spalte entsprechend:

```
1 # "Geschlecht" soll factor werden
2 MeineTabelle$Geschlecht <- factor(MeineTabelle$Geschlecht)
3
4 # überprüfen, ob es geklappt hat
5 class(MeineTabelle$Geschlecht)
```

```
[1] "factor"
```

1.2.3.11.2 Import aus CSV-Datei

Das CSV-Format (für **c**omma-**s**eparated-**v**alues) wird von vielen Anwendungen unterstützt. So können auch Tabellen, die in **LibreOffice** oder **Excel** angelegt wurden, als CSV-Datei gespeichert werden. Wenn Sie eine Datei im CSV-Format speichern, fragt Sie Ihre Anwendung nach dem gewünschten *Datenseparator*, also nach dem Zeichen, durch welches die Werte voneinander getrennt sind. Standardmäßig wird hier ein Komma “,” verwendet (wie der Name **CSV** vermuten lässt, und was zusätzlich sicherstellt, dass die zu importierenden Daten ihre Dezimalstellen mit einem **Punkt** abbilden, denn sonst würde nach jedem Dezimalkomma ein neuer Wert beginnen!) oder ein Semikolon “;”. Ich persönlich verwende gerne das Semikolon.

Der Import in **R** erfolgt ebenfalls über die Funktion `read.table()`. Neben dem Parameter `header` muss ihr der eingestellte *Datenseparator* übergeben werden.

Angenommen, die CSV-Datei hieße `DieDaten.csv`, läge in unserem Arbeitsverzeichnis und hätte den Datenseparator “;”, dann erfolgt der Import per `read.table()` mit dem Befehl:

```
1 # importiere Daten aus "DieDaten.csv"
2 NeueTabelle <- read.table("DieDaten.csv", sep=";", header=TRUE)
```

Liegt die Datei im Internet, kann die Adresse per `url()` Funktion übergeben werden.

```
1 # importiere Daten aus dem Internet
2 NeueTabelle <- read.table(url("http://www.produkt.de/R/DieDaten.csv"), sep=";",
3 header=TRUE)
4 # anzeigen
5 NeueTabelle
```

	Name	Geschlecht	Lieblingsfarbe	Einkommen
1	Hans	maennlich	gruen	1233
2	Caro	weiblich	blau	800
3	Lars	intersexuell	gelb	2400
4	Ines	weiblich	schwarz	4000
5	Samira	weiblich	gelb	899

6	Peter	maennlich	gruen	1100
7	Sarah	weiblich	blau	1900

Sollte die CSV-Datei mit dem Datenseparator TAB (für *Tabulator*) erstellt worden sein, lautet der Befehl:

```
1 # importiere Daten
2 # Daten sind mit TAB separiert!
3 NeueTabelle <- read.table("DieDaten.csv", sep="\t", header=TRUE)
```

(Separieren Sie Ihre Daten möglichst niemals mittels Tabulator, da es hier zu ungewollten Effekten kommen kann.)

Sollte `read.table()` keine brauchbaren Daten erzeugen (z.B. wegen Sonderzeichen in den Daten oder wenn die Daten per TAB separiert wurden), kann auf die Funktion `read.csv()` zurückgegriffen werden. Letztendlich ruft diese Funktion intern auch “nur” `read.table()` auf, jedoch mit jeder Menge vor-eingestellter Parameter.

```
1 # importiere Daten mit read.csv()
2 # Daten sind mit TAB separiert!
3 NeueTabelle <- read.csv("DieDaten.csv", sep="\t", header=TRUE)
```

1.2.3.11.3 Import aus SPSS-Datei

Um native SPSS-Dateien in  zu importieren, müssen zunächst Zusatzpakete installiert werden, siehe hierzu den Abschnitt 1.2.3.13 auf Seite 92.

Zum Import von SPSS-Dateien installieren und aktivieren wir das Paket **haven**.

```
1 # installiere das Zusatzpaket "haven"
2 install.packages("haven")
3
4 # aktiviere das Paket "haven"
5 library("haven")
```

Das Paket **haven** bietet für jeden SPSS-Dateityp die passende Funktion an. Angenommen, die SPSS-Datei heiße `alteDaten.sav` und läge in unserem Arbeitsverzeichnis, dann kann sie mit der Funktion `read_sav()` in  importiert werden.

```
1 # importiere SPSS-Datei "alteDaten.sav"
2 alteDaten <- read_sav("alteDaten.sav")
```

In SPSS werden kategoriale Daten häufig mit *Labels* versehen.

```
1 # Lade Test-SPSS-Datei mit Labels
2 spss <- read_sav(url("https://www.produnis.de/R/alteDaten-lang.sav"))
3 head(spss)
```

```
# A tibble: 6 x 6
      v1      v2      v3      v4      v5      thetruth
  <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl>
1 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
2 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
3 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
4 4 [strongly... 3 [agree]    3 [agree]    4 [strongl... 4 [strongl... 4 [strongl...
5 3 [agree]    3 [agree]    2 [disagre... 2 [disagre... 4 [strongl... 4 [strongl...
6 2 [disagree] 3 [agree]    3 [agree]    2 [disagre... 4 [strongl... 4 [strongl...
```

Wir sehen, dass die Werte der Spalten gelabelt sind (4=*strongly agree*, 3=*agree*, usw.). Die Labels stehen in eckigen Klammern neben dem eigentlichen Wert der Variable.

Die enthaltenen Labels kann man sich mit der Funktion `attr()` anzeigen lassen.

```
1 # Namen der Variablen
2 attr(spss, "names")
```

```
[1] "v1"      "v2"      "v3"      "v4"
[5] "v5"      "thetruth"
```

Das Label einer Variable wird angezeigt mit

```
1 # Label der Variablen
2 attr(spss$v1, "label")
```

```
[1] "Blue Question"
```

Die Labels innerhalb einer Variable mit

```
1 # Label der Variablenwerte für "v1"
2 attr(spss$v1, "labels")
```

strongly disagree	disagree	agree
1	2	3
strongly agree	no answer	
4	9	

In  ist die Verwendung von Wertelabels eher untypisch. Es empfiehlt sich, die Ausprägungsstufen *so wie sie sind* als Werte einzutragen, also z.B. direkt "männlich" - "weiblich" - "divers" anstatt 0 - 1 - 2 und anschließender Labelung. So werden die Ausprägungen auch auf den Grafiken entsprechend "aussagekräftig" angezeigt.

Wir wandeln die Variablen in Faktoren um, welche die Labels als Levelnamen nutzen².

```
1 library(tidyverse)
2 spss2 <- spss %>%
3   mutate(as_factor(.))
4 head(spss2, 10)
```

```
# A tibble: 20 x 6
  v1      v2      v3      v4      v5      thetruth
  <fct>   <fct>   <fct>   <fct>   <fct>   <fct>
1 strongly ag... strongly ag... strongly a... NA      NA      strongly agr...
2 strongly ag... strongly ag... strongly a... NA      NA      strongly agr...
3 strongly ag... strongly ag... strongly a... NA      NA      strongly agr...
4 strongly ag... agree      agree      strongly ... strongly a... strongly agr...
```

²Den folgenden Code können Sie *jetzt* noch nicht verstehen. Er stammt aus dem *Tidyverse*, dessen Funktionen in Kapitel 1.6 ab Seite 147 beschrieben werden.

5	agree	agree	disagree	disagree	strongly a...	strongly agr...
6	disagree	agree	agree	disagree	strongly a...	strongly agr...
7	disagree	strongly di...	agree	disagree	strongly d...	strongly agr...
8	disagree	agree	agree	agree	agree	agree
9	strongly ag...	strongly ag...	strongly a...	NA	NA	strongly dis...
10	agree	agree	agree	strongly ...	strongly a...	strongly agr...

Weitere Informationen zum Umgang mit gelabelten Datensätzen erhalten Sie im Tidymverse-Abschnitt 1.6.6.1 auf Seite 157.

1.2.3.11.4 Import aus Excel-Datei

Um `xlsx`-Dateien in `R` zu importieren, muss das Zusatzpaket `readxl` installiert werden. Der Datenimport erfolgt dann mit der Funktion `read_excel()`:

```
1 # installiere das Zusatzpaket "readxl"
2 install.packages("readxl")
3
4 # aktiviere das Paket "readxl"
5 library("readxl")
6
7 # importiere Excel-Datei "alteDaten.xlsx"
8 alteDaten <- read_excel("alteDaten.xlsx")
```

Soll ein bestimmtes Tabellenblatt der `xlsx`-Datei importiert werden, kann diese mit dem Parameter `sheet` übergeben werden.

```
1 # importiere Tabelle "Tabelle1" aus Excel-Datei "alteDaten.xlsx"
2 alteDaten <- read_excel("alteDaten.xlsx", sheet="Tabelle1")
```

Soll nur ein bestimmter Bereich der Tabelle importiert werden, kann diese mit dem Parameter `range` übergeben werden.

```
1 # importiere aus Excel-Datei "alteDaten.xlsx"
2 # Tabellenblatt "Tabelle1"
3 # im Bereich A2 bis D3
4 alteDaten <- read_excel("alteDaten.xlsx", sheet="Tabelle1", range="A2:D3")
```

1.2.3.11.5 Import aus .ods-Datei

Um ods-Dateien (z.B. aus Libreoffice oder OpenOffice) in  zu importieren, muss das Zusatzpakete readODS installiert werden. Der Datenimport erfolgt dann mit der Funktion read_ods():

```
1 # installiere das Zusatzpaket "readODS"
2 install.packages("readODS")
3
4 # aktiviere das Paket "readODS"
5 library("readODS")
6
7 # importiere ods-Datei "tolleDaten.ods"
8 tolleDaten <- read_ods("tolleDaten.ods")
```

Soll ein bestimmtes Tabellenblatt der ods-Datei importiert werden, kann diese mit dem Parameter **sheet** übergeben werden.

```
1 # importiere Tabelle "Tabelle1" aus ods-Datei "cooleDaten.ods"
2 cooleDaten <- read_ods("cooleDaten.ods", sheet="Tabelle1")
```

Soll nur ein bestimmter Bereich der Tabelle importiert werden, kann diese mit dem Parameter **range** übergeben werden.

```
1 # importiere aus ods-Datei "cooleDaten.ods"
2 # Tabellenblatt "Tabelle1"
3 # im Bereich A2 bis D3
4 alteDaten <- read.ods("cooleDaten.ods", sheet="Tabelle1", range="A2:D3")
```

1.2.3.11.6 Import mit RStudio

Mit RStudio können Daten sehr leicht importiert werden. Im Datenfenster klicken Sie oben auf Import Dataset.

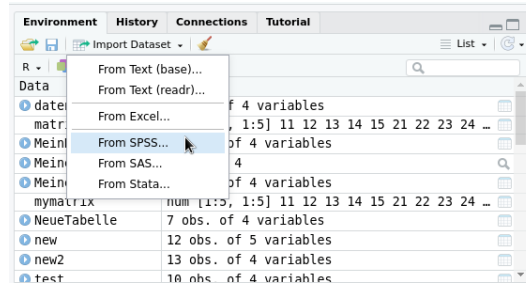


Abb. 1.34: Import Dataset

Es öffnet sich ein kleines Fenster, in welchem Sie das gewünschte Import-Format auswählen können (Abbildung 1.34). Wie Sie sehen, werden native Dateien aus Excel, SPSS, SAS und Stata unterstützt.

Wir nehmen in diesem Beispiel From SPSS.

Da der Datenimport das Zusatzpaket `haven` erfordert, wird es zur Installation vorgeschlagen, sofern es noch nicht installiert ist (Abbildung 1.35). Wir bestätigen in diesem Fall mit **Yes**.

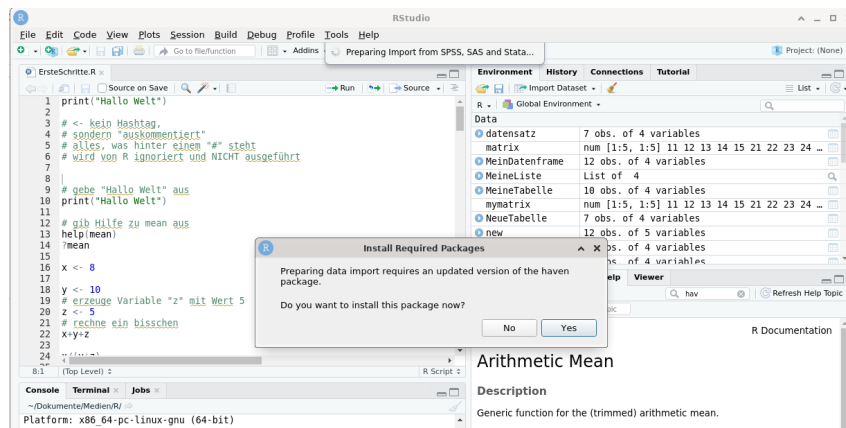


Abb. 1.35: schlägt Installation von `haven` vor

Nun öffnet sich das eigentliche Importierfenster (Abbildung 1.36).

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

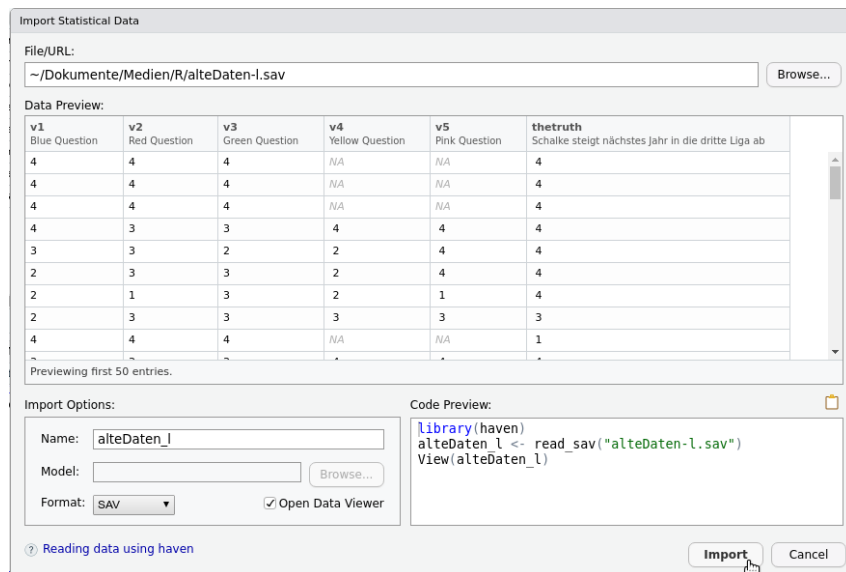


Abb. 1.36: Datensatz *alteDaten1-l.sav* importieren

Klicken Sie oben rechts auf den **Browse**-Knopf und wählen Sie die SPSS-Datei aus, die Sie importieren möchten. Für Abbildung 1.36 habe ich die Datei *alteDaten-l.sav* in meinem Arbeitsverzeichnis ausgewählt. Sobald Sie eine Datei ausgewählt haben, wird unter **Data Preview** eine Vorschau des zu importierenden Datensatzes angezeigt. Wie Sie sehen hat mein alter Datensatz die Variablen *v1*, *v2*, *v3* Darunter sehen Sie die Ausprägungen der Variablen.

Unter dem Vorschauenfenster können Sie im linken Abschnitt (**Import Options**) den Objektnamen eintragen, den der Datensatz in R haben soll. Ich habe in Abbildung 1.36 den Namen *alteDaten_l* gewählt. Das **Format** ist schon automatisch auf **SAV** eingestellt (denn der Dateiname endet mit *.sav*). Der Haken bei **Open Data Viewer** bewirkt, dass der Datensatz nach dem Import im Scriptfenster angezeigt wird.

Im rechten Abschnitt (**Code Preview**) sehen Sie die R-Befehle, die in der Konsole ausgeführt werden, um den Import durchzuführen.

Klicken Sie unten rechts auf **Import**.

1.2.3.12 Daten exportieren

In Abschnitt 1.2.3.9 auf Seite 77 wird beschrieben, wie Sie Ihre -Objekte im `.RData`-Format abspeichern können.

Es ist aber natürlich ebenso möglich, die Daten in anderen Formaten zu exportieren.

1.2.3.12.1 Export als CSV-Datei

Mit dem CSV-Format kommen die meisten Anwendungen zurecht. Daher empfehlen wir, dieses zum Tausch zwischen Anwendungen zu verwenden. Wir benutzen hierfür die Funktion `write.csv()` und speichern das Datenframe `MeinDatenframe` in die Datei `MeinDatenframe.csv`. Als Datenseparator wird standardmäßig ein Leerzeichen verwendet. Soll ein anderer Separator verwendet werden, kann dies über den Parameter `sep` angegeben werden.

```
1 # exportiere "MeinDatenframe" als CSV-Datei
2 # mit "," als Separator
3 write.csv(MeinDatenframe, "MeinDatenframe.csv", sep=",")
```

1.2.3.12.2 Export als Textdatei-Datei

Mit der Funktion `write.table()` können die Daten als simple Textdatei gespeichert werden. Als Datenseparator wird auch hier standardmäßig ein Leerzeichen verwendet. Soll ein anderer Separator verwendet werden, kann dies über den Parameter `sep` angegeben werden.

```
1 # exportiere "MeinDatenframe" als Text-Datei
2 # mit "," als Separator
3 write.table(MeinDatenframe, "MeinDatenframe.txt", sep=",")
```

Sollen die Daten an eine bestehende Datei angehängt werden, muss der Parameter `append` auf `TRUE` gesetzt werden.

1.2.3.12.3 Export als SPSS-Datei

Mit dem Zusatzpaket `haven` können -Objekte als SPSS-Datei abgespeichert werden.

Wir benutzen hierfür die Funktion `write_sav()` und speichern das Datenframe `MeinDatenframe` in die Datei `MeinDatenframe.sav`.

```
1 # exportiere "MeinDatenframe" als SPSS-Datei
2 write_sav(MeinDatenframe, "MeinDatenframe.sav")
```

Um eine komprimierte `.zsav`-Datei zu erstellen, setzen wir den Parameter `compress` auf `TRUE`.

```
1 # exportiere "MeinDatenframe" als komprimierte SPSS-Datei
2 write_sav(MeinDatenframe, "MeinDatenframe.sav", compress=TRUE)
```

1.2.3.13 Zusatzpakete installieren

In  stehen viele Zusatzpakete bereit, die den Funktionsumfang erweitern. Über das *Comprehensive R Archive Network*, kurz `CRAN`, können diese mit dem Befehl `install.packages()` installiert werden.

Möchte man beispielsweise “`ggplot`” (siehe Abschnitt 2.4 auf Seite 322) installieren, lautet der Befehl entsprechend (Achtung, das Paket für `ggplot` heisst `ggplot2` !)

```
1 # ggplot installieren, ACHTUNG, das Paket heisst ggplot2 !!!
2 install.packages("ggplot2")
```

Manche Pakete benötigen weitere Zusatzpakete, um vollständig laufen zu können. Man spricht hier von “Abhängigkeiten”, da das eine Paket vom anderen abhängt. Möchte man das gewünschte Paket inklusive aller Abhängigkeiten installieren, ändert sich der Befehl in:

```
1 # Paket ggplot mit allen Abhängigkeiten installieren
2 install.packages("ggplot2", dependencies=TRUE)
```

Sollen mehrere Pakete auf einmal installiert werden, können diese mit der `c()`-Funktion übergeben werden.

```
1 # Pakete ggplot, wikibooks, pwr mit allen Abhängigkeiten installieren
2 install.packages(c("ggplot2", "wikibooks", "pwr"), dependencies=TRUE)
```

Um die installierten Zusatzpakete benutzen zu können, müssen sie in einer -Sitzung zunächst geladen bzw. aktiviert werden. Dies erfolgt mit dem Befehl `library()`.

```
1 # aktiviere das ggplot2 Paket
2 library(ggplot2)
```

Mit dem Befehl `detach()` können die Zusatzpakete wieder deaktiviert werden.

```
1 # aktiviere das ggplot2 Paket
2 detach("package:ggplot2")
```

Ein anderer Weg besteht darin, den Paketnamen mit zwei Doppelpunkten getrennt vor die Funktion zu schreiben.

```
1 # nutze Funktion "read_sav" aus dem Paket "haven"
2 haven::read_sav("SPSSdaten.sav")
```

Die beiden Doppelpunkte weisen  an, die Funktion `read_sav()` aus dem Paket `haven` zu nutzen. So kann man sich den `library(haven)`-Aufruf sparen. Dies bietet sich an, wenn man “nur mal kurz” eine bestimmte Funktion eines bestimmten Pakets nutzen möchte. Hat man mehrere Funktionsaufrufe, kann das Laden per `library()` die Schreibarbeit verkürzen.

In **RStudio** lassen sich Pakete auf mehrere Arten installieren. Zum einen kann man in der Menüleiste auf **Tools** → **Install packages** klicken. Es öffnet sich ein neues Fenster, in welches wir einfach die Paketnamen eintragen können (Abbildung 1.37).

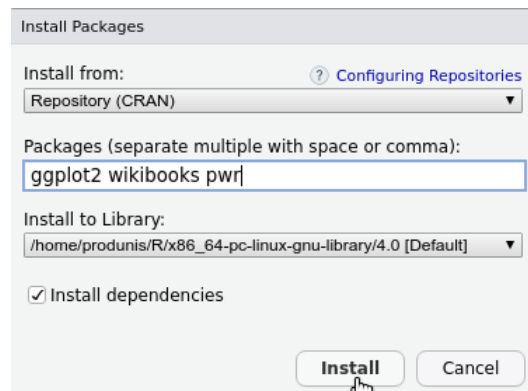


Abb. 1.37: Zusatzpakete mit RStudio

RStudio bietet eine weitere recht einfache und nützliche Methode zur Installation von Zusatzpaketen. Die Software durchsucht Ihre Scriptdatei nach dem Befehl `library()`. Ist das dort referenzierte Paket noch nicht installiert, schlägt RStudio die Installation vor (siehe gelber Balken in Abbildung 1.38). Klicken Sie im gelben Balken auf **Install** um das Paket zu installieren. Dies

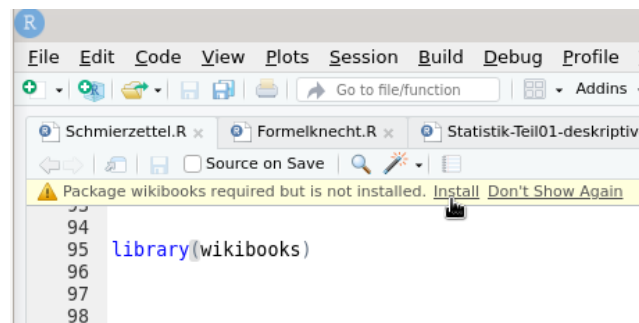


Abb. 1.38: Zusatzpakete per `library()`

ist nützlich, wenn man z.B. an einem “neuen” PC *alte* R-Scriptdateien öffnet, oder z.B. *fremde* R-Skripte verwendet. Nach dem Öffnen schlägt RStudio alle fehlenden Pakete zur Installation vor.

Natürlich können Sie auch einfach `install.packages("ggplot2")` (oder jeden anderen Paketnamen) in Ihre Scriptdatei schreiben und diese dann in der R-Konsole ausführen lassen.

Das Aktivieren von installierten Zusatzpaketen wird von RStudio ebenfalls ermöglicht. Klicken Sie im Ausgabefenster auf den Reiter **Packages**. So werden Ihnen alle installierten Zusatzpakete angezeigt (Abbildung 1.39).

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

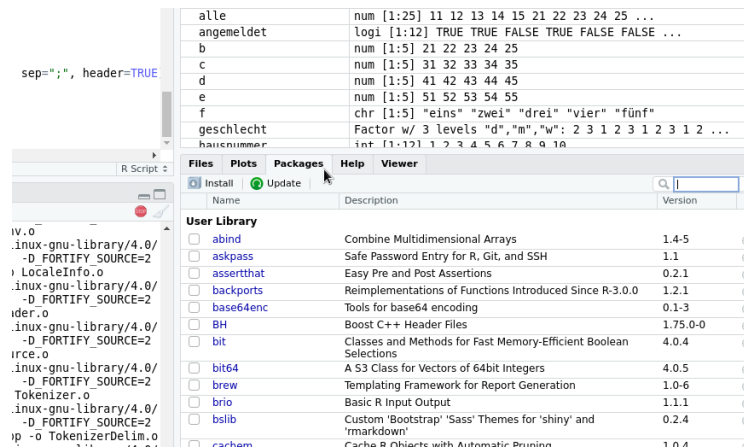


Abb. 1.39: Reiter Packages

Wenn Sie ein Paket aktivieren möchten, klicken Sie einfach dessen Häkchen an.

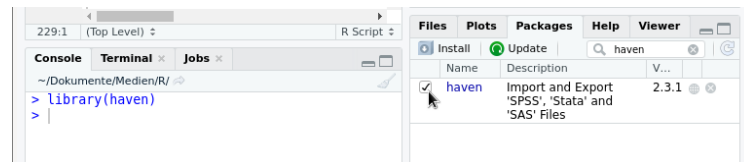


Abb. 1.40: aktiviere Paket haven

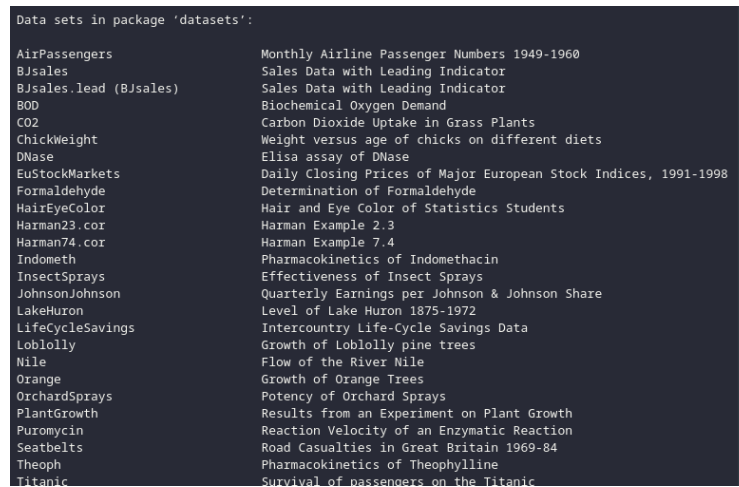
Wie Abbildung 1.40 zeigt, wurde durch den Klick automatisch der Befehl `library(haven)` in die R-Konsole übertragen und ausgeführt (linke Seite). Zum Deaktivieren müssen Sie einfach wieder auf das Kästchen klicken und den Haken entfernen.

Viele Paket enthalten Datensätze, die für Übungs- und Auswertungsbeispiele genutzt werden können.

Die enthaltenen Datensätze lassen sich mit dem Befehl `data(package="PAKETNAME")` auflisten. In R ist das Basispaket `datasets` enthalten, dessen Inhalte sich anzeigen lassen per:

```
1 # zeige die Datensätze des Pakets "datasets" an
2 data(package = "datasets")
```

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO



Data sets in package 'datasets':

AirPassengers	Monthly Airline Passenger Numbers 1949-1960
BJsales	Sales Data with Leading Indicator
BJsales.lead (BJsales)	Sales Data with Leading Indicator
BOD	Biochemical Oxygen Demand
CO2	Carbon Dioxide Uptake in Grass Plants
ChickWeight	Weight versus age of chicks on different diets
DNase	Elisa assay of DNase
EuStockMarkets	Daily Closing Prices of Major European Stock Indices, 1991-1998
Formaldehyde	Determination of Formaldehyde
HairEyeColor	Hair and Eye Color of Statistics Students
Harman23.cor	Harman Example 2.3
Harman74.cor	Harman Example 7.4
Indometh	Pharmacokinetics of Indomethacin
InsectSprays	Effectiveness of Insect Sprays
JohnsonJohnson	Quarterly Earnings per Johnson & Johnson Share
LakeHuron	Level of Lake Huron 1875-1972
LifeCycleSavings	Intercountry Life-Cycle Savings Data
Loblolly	Growth of Loblolly pine trees
Nile	Flow of the River Nile
Orange	Growth of Orange Trees
OrchardSprays	Potency of Orchard Sprays
PlantGrowth	Results from an Experiment on Plant Growth
Puromycin	Reaction Velocity of an Enzymatic Reaction
Seatbelts	Road Casualties in Great Britain 1969-84
Theoph	Pharmacokinetics of Theophylline
Titanic	Survival of passengers on the Titanic

Abb. 1.41: Datensätze im Basispaket *datasets*

Um einen Datensatz einzubinden, wird er mit der `data()`-Funktion aufgerufen.

```
1 # aktiviere den Datensatz "iris"
2 data(iris)
3
4 # zeige den Inhalt von 'iris'
5 head(iris)
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

Eine alternative Zugriffsweise besteht darin, die Datensätze per `paketname::datensatz` einem Objekt zuzuweisen.

```
1 # weise den Datensatz psych::bfi dem Objekt testdf zu
2 testdf <- psych::bfi
3
4 # zeige erste Zeilen an
5 head(testdf)
```

	A1	A2	A3	A4	A5	C1	C2	C3	C4	C5	E1	E2	E3	E4	E5	N1	N2	N3	N4	N5
61617	2	4	3	4	4	2	3	3	4	4	3	3	3	4	4	3	4	2	2	3
61618	2	4	5	2	5	5	4	4	3	4	1	1	6	4	3	3	3	3	5	5
61620	5	4	5	4	4	4	5	4	2	5	2	4	4	4	5	4	5	4	2	3
61621	4	4	6	5	5	4	4	3	5	5	5	3	4	4	4	2	5	2	4	1
61622	2	3	3	4	5	4	4	5	3	2	2	2	5	4	5	2	3	4	4	3
61623	6	6	5	6	5	6	6	6	1	3	2	1	6	5	6	3	5	2	2	3

	O1	O2	O3	O4	O5	gender	education	age
61617	3	6	3	4	3	1	NA	16
61618	4	2	4	3	3	2	NA	18
61620	4	2	5	5	2	2	NA	17
61621	3	3	4	3	5	2	NA	17
61622	3	3	4	3	3	1	NA	17
61623	4	3	5	6	1	2	3	21

1.2.3.13.1 Zusatzpaket zu diesem Lehrbuch

Zu diesem Lehrbuch existiert ein eigenes Zusatzpaket, in welchem alle verwendeten Datensätze und Zusatzfunktionen enthalten sind. Das Paket heisst `jgsbook` und kann über das CRAN³ installiert werden:

```
1 # installiere das Zusatzpaket dieses Lehrbuchs
2 install.packages("jgsbook")
```

Die enthaltenen Datensätze lassen sich mit dem Befehl `data()` auflisten:

```
1 # zeige die Datensätze des Pakets "jgsbook" an
2 data(package = "jgsbook")
```

R data sets

Data sets in package 'jgsbook':

Pflegeberufe	Matrix of Pflegeberufe by Isfort et al. 2018
epa	Datatable of the epa Example
mma	Dataset of a work sampling study
nw (Nachtwachen)	Dataset of the German Nachtwachen study with labelled variables
nw_labelled (nw)	Dataset of the German Nachtwachen study with labelled variables
ordinalSample (OrdinalSample)	Datatable of an Ordinal Sample
pf8	Dataset of the PF8 example.

Abb. 1.42: Datensätze im Paket `jgsbook`

...und in die Arbeitsumgebung laden:

³<https://cran.r-project.org/package=jgsbook>

```
1 # aktiviere jgsbook
2 library(jgsbook)
3
4 # aktiviere den Datensatz "epa"
5 data(epa)
6
7 # zeige die ersten 6 Datenreihen von "epa"
8 head(epa)
```

	age	sex	cms	risk	expert	decu
1	50	m	40	0	0	0
2	75	m	40	0	0	0
3	86	w	13	1	1	0
4	77	w	24	1	1	0
5	66	w	40	0	0	0
6	51	m	37	0	0	0

Sie können auch über die Kombination `paketname::datensatz` auf die Datensätze zugreifen. So muss das Paket nicht erst per `library()` aktiviert werden.

```
1 # Datensatz "epa" aus dem Paket "jgsbook"
2 epa <- jgsbook::epa
3
4 # zeige die ersten 6 Datenreihen von "epa"
5 head(epa)
```

	age	sex	cms	risk	expert	decu
1	50	m	40	0	0	0
2	75	m	40	0	0	0
3	86	w	13	1	1	0
4	77	w	24	1	1	0
5	66	w	40	0	0	0
6	51	m	37	0	0	0

1.2.3.14 Ausgabe in Datei umleiten

Das Umleiten der -Ausgabe in eine Textdatei ist evtl. ein etwas angestaubtes Verfahren, welches vor allem in Zeiten *vor* RStudio Anwendung gefunden hat. Heutzutage steht mit `quarto` eine komfortable Lösung zur Verfügung, hübsche

R-Auswertungen z.B. in einem ansprechenden `html`-Dokument (sprich einer Webseite) zu präsentieren (siehe hierzu Abschnitt 1.5 auf Seite 132).

Standardmäßig schreibt R seine Ausgaben in das Konsolfenster. Manchmal kann es aber auch vorteilhaft sein, die Ausgaben in eine Datei umzulenken. Dies kann mit der Funktion `sink()` erreicht werden. Der folgende Befehl leitet alle Ausgaben in die Textdatei `Ausgabe.txt` im Arbeitsverzeichnis um.

```
1 # leite ab jetzt alle Ausgaben in die Datei "Ausgabe.txt" um
2 sink("Ausgabe.txt")
```

Soll die Umleitung aufgehoben werden, erfolgt dies mit einem leeren Funktionsaufruf.

```
1 # beende die Umleitung
2 sink()
```

So kann man in der Scriptdatei genau festlegen, ab wann die Ausgaben in eine Datei geschrieben werden sollen, und wann nicht. Beachten Sie, dass R die Textdatei immer wieder überschreiben wird, z.B. wenn der `sink()` Aufruf beendet und dann erneut gestartet wird. Um die Ausgabe an eine bestehende Datei *anzuhängen* (ohne diese zu überschreiben) muss der Parameter `append` auf `TRUE` gesetzt werden.

```
1 # hänge ab jetzt alle Ausgaben an die Datei "Ausgabe.txt" an
2 sink("Ausgabe.txt", append=TRUE)
```

An dieser Stelle sei auf die Befehle `print()` und `cat()` hingewiesen. Wenn Sie die R-Ausgabe in eine Textdatei umleiten, möchten Sie evtl. ebenfalls Kommentare oder anderen Text in die Textdatei schreiben lassen, z.B. bevor Sie Befehle ausführen (oder nachdem).

Der Befehl `print()` gibt den Text dabei in der R-bekannten Weise aus.

```
1 # schreibe mit print() in die Textdatei
2 print("Guten Tag, beginnen wir mit der Analyse.")
```


In die Textdatei wird geschrieben:

```
[1] "Guten Tag, beginnen wir mit der Analyse."
```

Mit dem Befehl `cat()` wird tatsächlich nur der Text ausgegeben (ohne den Positionsanzeiger `[1]`).

```
1  # schreibe mit cat() in die Textdatei
2  cat("Guten Tag, beginnen wir mit der Analyse.\nLos geht es!")
```

In die Textdatei wird geschrieben:

```
Guten Tag, beginnen wir mit der Analyse.
Los geht es!
```

Der Parameter `\n` gibt an, dass eine neue Zeile beginnen soll.

1.2.4 Verteilungsfunktionen

In  sind zahlreiche Verteilungsfunktionen implementiert. Wir beschränken uns in diesem Buch auf die für uns wichtigen Wahrscheinlichkeitsverteilungen. Eine vollständige Liste aller implementierten Verteilungen findet sich unter <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/Distributions.html>

Der Funktionsumfang folgt dabei einer gewissen Logik, für jede Verteilung existiert je eine **d**, **p**, **q** und **r**-Funktion. Für die Normalverteilung (siehe Näheres in Abschnitt 1.2.4.1) sind das `dnorm()`, `pnorm()`, `qnorm()` und `rnorm()`. Für die t-Verteilung (siehe Abschnitt 1.2.4.2) heißen die Equivalente `dt()`, `pt()`, `qt()` und `rt()`, für χ^2 entsprechend `dchisq()`, `pchisq()`, `qchisq()` und `rchisq()`.

d ist jeweils die Dichtefunktion der Verteilung

p berechnet die Werte der Verteilungsfunktion (kumulierte Dichte)

q bestimmt die Quantile der Verteilung

r erzeugt Zufallswerte aus der Verteilung

Dies möchten wir am Beispiel der Normalverteilung verdeutlichen.

1.2.4.1 Normalverteilung

Die **Normalverteilung** ist mit den oben beschriebenen Funktionen in  implementiert.

Mit der Funktion `dnorm()` berechnet man die Werte der Wahrscheinlichkeitsdichte. Möchte man beispielsweise die Wahrscheinlichkeitsdichte für $x = 2$ aus der Standardnormalverteilung berechnen lautet der Befehl:

```
1  # Wahrscheinlichkeitsdichte für x=2
2  # in der Standardnormalverteilung
3  dnorm(2)
```

```
[1] 0.05399097
```

Standardmäßig stammen diese Werte aus der Standardnormalverteilung, sie streuen also mit einer Standardabweichung von $s = 1$ um den Mittelwert $\bar{x} = 0$.

Möchte man dies ändern, können der Funktion entsprechende Werte für Mittelwert und Standardabweichung übergeben werden:

```
1 # Wahrscheinlichkeitsdichte für x=4
2 # aus einer Verteilung mit Mittelwert 8 und Standardabweichung 2
3 dnorm(4, mean=8, sd=2)
```

```
[1] 0.02699548
```

Die Werte der Verteilungsfunktion lassen sich mit `pnorm()` berechnen. Die Funktion gibt die Wahrscheinlichkeit an, dass ein Zufallsvariable einen Wert von $\leq x$ (= “höchstens” x) annimmt.

```
1 # Wahrscheinlichkeit dass Wert kleiner-gleich 2
2 # in Standardnormalverteilung
3 pnorm(2)
```

```
[1] 0.9772499
```

Für andere Normalverteilungen ändert sich der Befehl entsprechend:

```
1 # Wahrscheinlichkeit dass Wert kleiner-gleich 8
2 # in Normalverteilung mit xquer=12 und s=3
3 pnorm(8, mean=12, s=3)
```

```
[1] 0.09121122
```

Quantile lassen sich mit der Funktion `qnorm()` berechnen. Quantile stellen Grenzen dar, die ein zufälliger Wert mit einer vorgegebenen Wahrscheinlichkeit nicht *überschreiten* wird. Wenn das 95%-Quantil beispielsweise den Wert 5 hat, wird eine zufällig gezogene Zahl mit 95% Sicherheit *kleiner* als 5 sein.

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # Grenze des 95% Quantils
2 # in Standardnormalverteilung
3 qnorm(p=0.95)
```

```
[1] 1.644854
```

Auch `qnorm()` können Werte für andere Normalverteilungen übergeben werden.

```
1 # Grenze des 95% Quantils
2 # in Normalverteilung mit xquer=9 und sd=5
3 qnorm(p=0.95, mean=9, sd=5)
```

```
[1] 17.22427
```

Mit `rnorm()` lassen sich normalverteilte Zufallswerte erzeugen:

```
1 # erzeuge "zufällig" 10 Werte
2 # aus der Standardnormalverteilung
3 rnorm(10)
```

```
[1] -0.0127120  1.2604385  1.7311170  1.5315658  1.2866661
[6] -2.2716082  0.7971483  0.6404524  0.1764541 -0.1808511
```

Auch mit `rnorm()` können wir andere Normalverteilungen für unsere Zufallszahlen angeben:

```
1 # erzeuge "zufällig" 10 normalverteilte Werte
2 # mit Mittelwert 8 und Standardabweichung 2
3 rnorm(10, mean=8, sd=2)
```

```
[1] 12.904742  9.272356  8.503763  4.955420  9.617479  7.018170
[7]  9.160690  5.292520  7.270711  7.172105
```

Plot-Beispiele

```

1  # erzeuge Werte von -3 bis 6
2  x <- seq(-3,6, by=0.005)
3  # übergebe in ein data.frame
4  df <- data.frame(x)
5  # ggplot erstellen
6  p <- ggplot(data=df, aes(x)) +
7      xlim(-3,6) + ylim(0, 0.8) +
8      xlab("x") + ylab("Dichtefunktion") +
9      ggtitle("Normalverteilungen")
10 p + stat_function(fun=dnorm, args=(c(mean=0,sd=1)), colour="black")+
11     annotate(geom="text", x=0, y=0.42, label="N(0;1)", color="black")+
12     stat_function(fun=dnorm, args=(c(mean=2,sd=0.5)), colour="blue") +
13     annotate(geom="text", x=3, y=0.6, label="N(2;0.5)", color="blue")+
14     stat_function(fun=dnorm, args=(c(mean=2,sd=2)), colour="darkorchid") +
15     annotate(geom="text", x=5, y=0.1, label="N(2;2)", color="darkorchid")

```

Der Befehl erzeugt Abbildung 1.43.

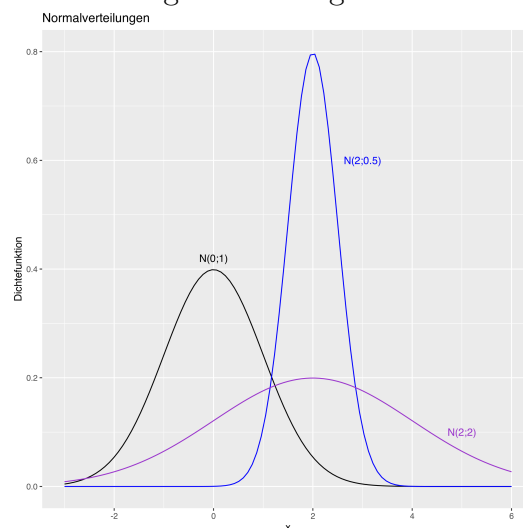


Abb. 1.43

Für weitere Plot-Beispiele zur Normalverteilung siehe Abschnitt 3.2.1 auf Seite 392.

1.2.4.2 t-Verteilung

Die “kleine Schwester” der Standardnormalverteilung ist die **Student-t-Verteilung** (oder einfach kurz **t-Verteilung**). Für sie stehen die selben Funktionstypen zur Verfügung wie für die Normalverteilung (siehe Abschnitt 1.2.4.1). Diese heissen entsprechend `dt()`, `pt()`, `qt()` und `rt()` und funktionieren genau wie ihre Pendants der Standardnormalverteilung. Jedoch muss ihnen noch die Anzahl der Freiheitsgrade (*df*) übergeben werden:

```
1 # Wahrscheinlichkeitsdichte für x=2
2 # in der t-Verteilung mit 4 Freiheitsgraden
3 dt(2, df=4)
```

```
[1] 0.06629126
```

```
1 # Wahrscheinlichkeit dass Wert kleiner-gleich 2
2 # in der t-Verteilung mit 3 Freiheitsgraden
3 pt(2, df=3)
```

```
[1] 0.930337
```

```
1 # Grenze des 95% Quantils
2 # in der t-Verteilung mit 3 Freiheitsgraden
3 qt(p=0.95, df=3)
```

```
[1] 2.353363
```

```
1 # erzeuge "zufällig" 10 Werte
2 # aus der t-Verteilung mit 13 Freiheitsgraden
3 rt(10, df=13)
```

```
[1] -0.9328004  1.4917239  0.0955167 -0.3438326 -0.7593989
[6] -0.0556377  0.1085506  1.5149383  0.4664587 -2.8477079
```

Möchte man die t-Verteilung plotten, kann das mit `ggplot` (siehe Kapitel 2.4 ab Seite 322) so erfolgen.

```
1 # Erzeuge x-werte
2 df <- data.frame(x=seq(-3,3, by=0.005))
3
4 # Grundlegende Plotangaben
5 p <- ggplot(data=df, aes(x)) +
6   # begrenze die Achsen
7   xlim(-3,3) + ylim(0, 0.4) +
8   # Achsen-Titel
9   xlab("x") + ylab("Dichtefunktion") +
10  # Plot-Titel
11  ggtitle("t-Verteilungen", subtitle = "nach Freiheitsgraden")
12
13 # t-Verteilung plotten
14 p +
15   stat_function(fun=dt, args=list(df=1), col="black") +
16   # Textfeld hinzufügen
17   annotate(geom="text", x=0, y=0.25, label="df=1", color="black")
```

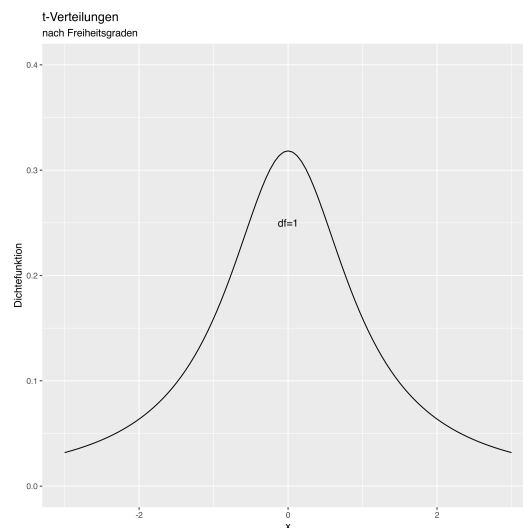


Abb. 1.44

Für weitere Plot-Beispiele zur t-Verteilung siehe Abschnitt 3.2.2 auf Seite 403.

1.2.4.3 χ^2 -Verteilung

Die χ^2 -Verteilung ist mit den oben beschriebenen Funktionen in $\mathbb{R}$ implementiert. Diese heissen entsprechend `dchisq()`, `pchisq()`, `qchisq()` und `rchisq()` und funktionieren genau wie ihre Pendanten der Standardnormalverteilung. Jedoch muss ihnen (wie bei der t-Verteilung) noch die Anzahl der Freiheitsgrade (*df*) übergeben werden:

```
1 # Wahrscheinlichkeitsdichte für x=2
2 # in der Chi^2-Verteilung mit 4 Freiheitsgraden
3 dchisq(2, df=4)
```

```
[1] 0.1839397
```

Möchte man die χ^2 -Verteilung plotten, kann das mit `ggplot` (siehe Kapitel 2.4 ab Seite 322) so erfolgen.

```
1 # Erzeuge x-werte
2 x=seq(0,25, by=0.005)
3 df <- data.frame( x,
4   df01 = dchisq(x, df=1),
5   df05 = dchisq(x, df=5),
6   df10 = dchisq(x, df=10),
7   df15 = dchisq(x, df=15)
8 )
9
10 # erzeuge long-table
11 df <- pivot_longer(df, cols=c(df01, df05, df10, df15))
12
13 p <- ggplot(data=df, aes(x, value, fill=name)) +
14   xlim(0,25) + ylim(0, 0.2) +
15   xlab("x") + ylab("Dichtefunktion") +
16   ggtitle("Chi^2-Verteilungen", subtitle = "nach Freiheitsgraden")
17 p + geom_line(aes(col=name, linetype=name))+
18   labs(col="Freiheitsgrade", linetype="")
```

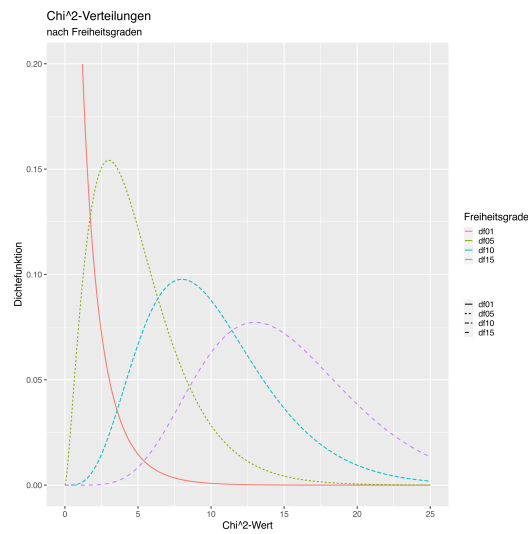



Abb. 1.45

Für weitere Plot-Beispiele zur χ^2 -Verteilung siehe Abschnitt 3.2.3 auf Seite 406.

1.2.4.4 Poisson-Verteilung

Die **Poisson-Verteilung** ist mit den oben beschriebenen Funktionen in  implementiert. Diese heißen entsprechend `dpois()`, `ppois()`, `qpois()` und `rpois()` und funktionieren genau wie ihre Pendanten der Standardnormalverteilung. Jedoch muss ihnen der Wert für λ übergeben werden.

```
1 # Wahrscheinlichkeitsdichte für x=2
2 # in der Poisson-Verteilung mit Lambda=4
3 dpois(2, lambda=4)
```

```
[1] 0.1465251
```

Möchte man die Poisson-Verteilung plotten, kann das mit `ggplot` (siehe Kapitel 2.4 ab Seite 322) so erfolgen. Wir plotten mit $\lambda = 1, 2, 4$ und 9 .

1.2. KURZE EINFÜHRUNG IN R UND RSTUDIO

```
1 # Erzeuge x-Werte
2 x=seq(0,25)
3 df <- data.frame( x,
4                   l1 = dpois(x, 1),
5                   l2 = dpois(x, 2),
6                   l4 = dpois(x, 4),
7                   l9 = dpois(x, 9)
8 )
9
10 # erzeuge eine long-table
11 df <- pivot_longer(df, cols=c(l1, l2, l4, l9))
12
13 # plot vorbereiten
14 p <- ggplot(data=df, aes(x, value, fill=name)) +
15   xlim(0,17) + ylim(0, 0.4) +
16   xlab("x") + ylab("Dichtefunktion") +
17   ggtitle("Poisson-Verteilungen", subtitle = "nach Lambda")
18 p + geom_line(aes(col=name,linetype=name))+
19   labs(col="Lambda",linetype="")
```

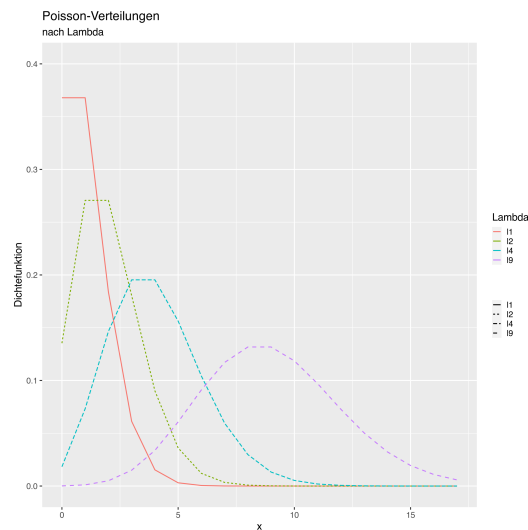


Abb. 1.46

Für weitere Plot-Beispiele zur Poisson-Verteilung siehe Abschnitt 3.2.4 auf Seite 409.

1.2.4.5 Binomial-Verteilung

Die Binomial-Verteilung ist mit den oben beschriebenen Funktionen in **R** implementiert. Diese heissen entsprechend `dbinom()`, `pbinom()`, `qbinom()` und `rbinom()` und funktionieren genau wie ihre Pendants der Standardnormalverteilung. Als Parameter werden übergeben:

- `x` = *günstige* Werte
- `n` = Anzahl der Beobachtungen
- `prob` = Wahrscheinlichkeit der Ereignisse

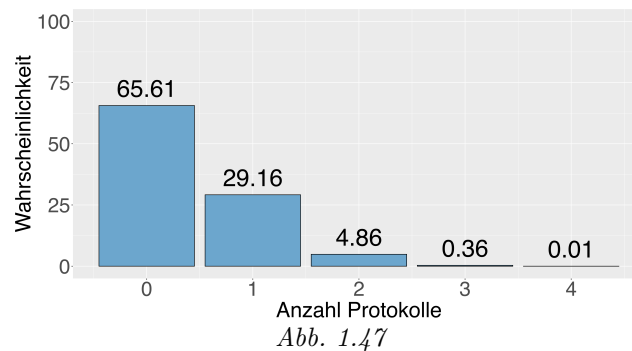
Die Wahrscheinlichkeit, dass ich in einem 4-tägigen Seminar mit insgesamt 10 Teilnehmern 2mal das Tagesprotokoll schreiben muss^[4] errechnet sich wie folgt:

```
1 # Wahrscheinlichkeit, insgesamt 2mal das Tagesprotokoll
2 # schreiben zu müssen,
3 # bei 10 Teilnehmern und 4 Seminartagen
4 dbinom(x=2, size=4, prob=0.1)
```

```
[1] 0.0486
```

Die Wahrscheinlichkeitswerte können für jedes Ereignis “Protokoll schreiben” errechnet und geplottet werden:

```
1 # Erzeuge Ereignisraum
2 protokoll <- c(0:4)
3
4 # überführe in Datenframe und berechne alle
5 # Wahrscheinlichkeiten, das Tagesprotokoll schreiben zu müssen,
6 # bei 10 Teilnehmern an 4 Seminartagen
7 df <- data.frame(protokoll, y=dbinom(x=protokoll, size=4, prob=0.1)*100)
8
9 # plotten
10 ggplot(df, aes(x=protokoll, y=y)) + geom_bar(stat="identity",
11 col="black", fill="skyblue3")+
12   geom_text(aes(label = y), size = 5, vjust = -0.5)+
13   xlab("Anzahl Protokolle")+
14   ylab("Wahrscheinlichkeit")+ ylim(0,100)
```

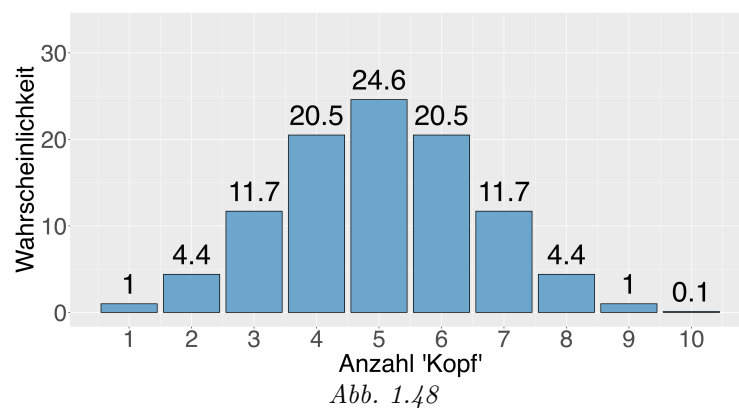


Die Wahrscheinlichkeit, bei 10 Münzwürfen k -mal Kopf zu werfen, errechnet sich wie folgt:

```

1  # Ereignisraum erzeugen
2  münze <- c(1:10)
3
4  # überführe in Datenframe und berechne
5  # alle Wahrscheinlichkeiten, k-mal "Kopf" zu werfen
6  df <- data.frame(münze, y=round(dbinom(x=münze, size=10, prob=0.5)*100,1))
7
8  # plotten
9  ggplot(df, aes(x=münze, y=y)) +
10   geom_bar(stat="identity", col="black", fill="skyblue3")+
11   geom_text(aes(label = y),size = 5,vjust = -0.5)+
12   xlab("Anzahl 'Kopf')+
13   ylab("Wahrscheinlichkeit")+ ylim(0,33) +
14   scale_x_continuous(breaks = 0:10)

```



1.2.5 Eigene Funktionen programmieren

In  lassen sich auch eigene Funktionen integrieren. Die wohl “einfachste” Methode hierbei ist, die Funktionen in der -Sprache selber zu schreiben. Es besteht aber auch die Möglichkeit, neue Funktionen in Programmiersprachen wie z.B. C++ zu programmieren.

Zum Erstellen einer eigenen Funktion steht der Befehl `function()` zur Verfügung. Dieser ist wie folgt aufzurufen:

```
1  function(Parameter1, Parameter2, ...) {
2      FUNKTIONSWEISE
3  }
```

In den Klammern direkt nach `function` können Parameter benannt werden, welche dann beim Funktionsaufruf der Funktion zur Verfügung stehen. Innerhalb der geschweiften Klammern wird die eigentliche Funktionsweise geschrieben.

Um eine neue Funktion in  aufrufen zu können, wird die Funktionsweise einem neuen Objekt (z.b. `myfunc`) zugeordnet.

```
1  myfunc <- function(x,y){ # Neue Funktion mit den Parametern x und y
2      z <- x+y             # Die Summe von x und y wird in z gespeichert
3      return(z)           # z wird zurückgemeldet
4  }
```

Der obigen Dummy-Funktion werden im Funktionsaufruf die Parameter `x` und `y` übergeben. Innerhalb der Funktion wird die Summe aus `x` und `y` gebildet, und per `return()` an die -Konsole zurückgemeldet. Jetzt können wir diese Funktion aufrufen:

```
1  myfunc(4,6)
```

```
[1] 10
```

Es besteht ferner die Möglichkeit, Standardwerte für die Parameter zu setzen, welche verwendet werden, falls der Funktionsaufruf ohne Parameter erfolgt.

```
1 myfunc <- function(x=3,y=7){ # Neue Funktion mit Standardwerten für die Parameter x und y
2   z <- x+y                  # Die Summe von x und y wird in z gespeichert
3   return(z)                 # z wird zurückgemeldet
4 }
```

Rufen wir die Funktion ohne Parameter auf, so werden die Standardwerte genutzt.

```
1 # Funktionsaufruf OHNE Parameter
2 myfunc()
```

```
[1] 10
```

Rufen wir die Funktion mit Parametern auf, werden diese verwendet.

```
1 myfunc(12,4)
```

```
[1] 16
```

1.2.5.1 Beispiele für Funktionen

Alle Funktionen (und Datensätze) dieses Lehrbuches sind auch im Zusatzpaket `jgsbook` enthalten, siehe Abschnitt 1.2.3.13.1 auf Seite 97.

1.2.5.1.1 z-Transformation

Mit dieser Funktion wird die z-Transformation für gegebene Werte errechnet. Die Funktion folgt der Formel

$$z_i = \frac{x_i - \mu}{\sigma}$$

Als Standardparameter werden die Werte der Standardnormalverteilung gesetzt.

```
1 ztrans <- function(x, mu=0, sd=1){
2   z = (x-mu)/sd
3   return(z)
4 }
5
6 # x ist 120, aus einer normalverteilten Reihe mit
7 # Mittelwert 118 und Standardabweichung 20
8 ztrans(120,mu=118,sd=20)
```

```
[1] 0.1
```

1.2.5.1.2 Sensitivität / Spezifität

Die folgende Funktion errechnet Sensitivität, Spezifität sowie positiv- und negativ-prädiktive Werte für gegebene Werte.

- `rp` = Anzahl richtig positive
- `rn` = Anzahl richtig negative
- `fp` = Anzahl falsch positive
- `fn` = Anzahl falsch negative

```
1 sens.spec <- function(rp, rn, fp, fn){
2   x <- data.frame(
3     sens=round(rp/(rp+fn)*100, 2),
4     spec=round(rn/(rn+fp)*100, 2),
5     ppw =round(rp/(rp+fp)*100, 2),
6     npw =round(rn/(rn+fn)*100, 2)
7   )
8   return(x)
9 }
10
11
12 sens.spec(40, 17, 85, 4)
```

```
      sens spec ppw  npw
1 90.91 16.67 32 80.95
```

1.2.5.1.3 Kenngrößen

Die folgende Funktion gibt die gebräuchlichsten Kenngrößen einer Wertereihe zurück:

```
1 kenngroessen <- function(werte){
2   bla <- data.frame(0)
3   bla$modus=paste(as.character(statip::mfv(werte)), collapse="|")
4   bla$mean=mean(werte, na.rm=T)
5   bla$median=median(werte, na.rm=T)
6   bla$p25=quantile(werte,0.25,type=6)
7   bla$p75=quantile(werte,0.75,type=6)
8   bla$iqr=IQR(werte,type=6)
```

```

9   bla$sd=sd(werte, na.rm=T)
10  bla$var=var(werte, na.rm=T)
11  bla$VK= (sd(werte, na.rm=T)/mean(werte,na.rm=T))
12  return(bla[-1])
13 }
14
15 # erzeuge zufällige Werte
16 x <- ceiling(rnorm(100, 10,5))
17
18 # Kenngrößen anzeigen
19 kenngrößen(x)

```

	modus	mean	median	p25	p75	iqr	sd	var	VK
1	6 11	10.65	11	7	14	7	4.710562	22.18939	0.4423063

1.2.5.1.4 Häufigkeitstabellen

Die folgende Funktion gibt eine vollständige Häufigkeitstabelle mit absoluten und relativen Häufigkeiten sowie kummulierten Werten zurück.

```

1   freqTable <- function(werte){
2     x <- table(werte)
3     tabelle <- data.frame(x)
4     tabelle$freqcum <- cumsum(x)
5     tabelle$relfreq <- round(x/length(werte)*100,2)
6     tabelle$relcum <- cumsum(round(x/length(werte)*100,2))
7     colnames(tabelle) <- c("Wert", "Häufig", "Hkum", "Relativ", "Rkum")
8     tabelle$Wert <- as.numeric(as.vector(tabelle$Wert))
9     return(tabelle)
10  }
11
12  x <- ceiling(rnorm(13, 10,2))
13  freqTable(x)

```

	Wert	Häufig	Hkum	Relativ	Rkum
1	8	1	1	7.69	7.69
2	9	3	4	23.08	30.77
3	10	4	8	30.77	61.54
4	12	4	12	30.77	92.31
5	13	1	13	7.69	100.00

1.2.5.2 Bedingungen

Innerhalb der Funktion können Variablenbedingungen mit dem `if()`-Befehl abgefragt werden. Der Aufruf erfolgt etwa so:


```
1 if(VARIABLENBEDINGUNG) {FUNKTIONSWEISE}
```

Innerhalb der Klammern des `if()`-Befehls werden die Variablenbedingungen gesetzt. Falls diese Bedingungen erfüllt sind, wird der Code innerhalb der geschweiften Klammern ausgeführt. Folgende Bedingungen können abgefragt werden

Zeichen	Bedingung
<code>==</code>	gleich
<code>!=</code>	ungleich
<code><</code>	kleiner
<code><=</code>	kleiner-gleich
<code>></code>	größer
<code>>=</code>	größer-gleich
<code>&</code>	UND
<code> </code>	ODER

Innerhalb von Funktionen kann man dies wie folgt anwenden

```
1 myfunc <- function(x=3,y=7){
2   z <- x+y                                # Die Summe von x und y wird in z gespeichert
3   if(z>20) {                              # Abfrage, ob die Summer größer als 20 ist
4     z <- "wow, bist du gross"             # wenn ja, dann schreibe einen Text in das Objekt z
5   }
6   return(z)                              # z wird zurückgemeldet
7 }
```

Mehrere Bedingungen können verknüpft werden, z.B. so:

```
1 myfunc <- function(x=3,y=7){
2   if(x<0 & y<0) {                         # Abfrage, ob x und y negativ sind
3     x <- x*(-1)                           # wenn ja, dann mache beide positiv
4     y <- y*(-1)
5   }
6   z <- x+y                                # Die Summe von x und y wird in z gespeichert
7   if(z==0 | z>50) {                       # Abfrage, ob z gleich 0 oder größer 50 ist
8     z <- "Summe ist 0 oder größer 50"     # wenn ja, schreibe einen Text...
9   }
10  return(z)                              # z wird zurückgemeldet
11 }
```

1.2.5.2.1 Beispiel Zusatzpakete

Die folgende Funktion installiert die vorgegebenen Pakete, sofern sie noch nicht installiert sind. Dies ist hilfreich, wenn z.B. auf eine höhere -Version geupdatet wurde, und alle Zusatzpakete neu installiert werden müssen.

```

1  install.my.packages <- function(){
2    # Liste meiner favorisierten Pakete
3    my_packages <- c("blogdown", "bookdown",
4                     "car",
5                     "foreign",
6                     "gghighlight", "ggplot2",
7                     "haven",
8                     "likert",
9                     "prettyR", "psych",
10                     "reshape", "reshape2",
11                     "samplingbook", "scales", "statip",
12                     "tidyverse",
13                     "VGAM",
14                     "xtable"
15  )
16
17  #-----
18  # Überprüfe, ob die Pakete bereits installiert sind
19  not_installed <- my_packages[!(my_packages %in% installed.packages()[ , "Package"])]
20  # installiere solche, die noch nicht installiert sind
21  if(length(not_installed)) install.packages(not_installed, dependencies = TRUE)
22  return(paste(length(not_installed), "Pakete wurden installiert (plus dependencies)."))
23  }

```

Die Funktion kann dann wie folgt aufgerufen werden:

```
1  install.my.packages()
```

```
[1] "0 Pakete wurden installiert (plus dependencies)."
```

Bei mir ist alles up-to-date, so dass kein Paket installiert werden muss.

Mit einer leichten Änderung können weitere Pakete an den Parameter `p` übergeben werden:

```

1  install.my.packages <- function(p=""){
2    # Liste meiner favorisierten Pakete
3    my_packages <- c("blogdown", "bookdown",
4                     "car",
5                     "foreign",
6                     "gghighlight", "ggplot2",
7                     "haven",
8                     "likert",
9                     "prettyR", "psych",
10                     "reshape", "reshape2",
11                     "samplingbook", "scales", "statip",
12                     "tidyverse",
13                     "VGAM",
14                     "xtable"
15  )
16
17  # Falls Pakete über den Parameter "p" übergeben wurden,
18  # füge sie der Liste hinzu
19  if(p!=""){
20    my_packages <- c(p, my_packages)
21  }
22  #-----
23  # Überprüfe, ob die Pakete bereits installiert sind
24  not_installed <- my_packages[!(my_packages %in% installed.packages()[, "Package"])]
25  # installiere solche, die noch nicht installiert sind
26  if(length(not_installed)) install.packages(not_installed, dependencies = TRUE)
27  return(paste(length(not_installed), "Pakete wurden installiert (plus dependencies)."))
28  }

```

Der Funktion kann so ein Vektor weiterer Pakete übergeben werden, die zusätzlich zur vorgegebenen Liste installiert werden, falls sie noch nicht installiert sind:

```

1  install.my.packages(c("ggpubr", "qqplotr"))

```

```

[1] "2 Pakete wurden installiert (plus dependencies)."
```

1.2.5.3 Funktionen in Dateien speichern

Eigene Funktionen sind Objekte wie alle anderen Variablen im Workspace. Wenn sie nicht abgespeichert werden, stehen Sie u.U. nach einem Neustart (oder wenn der Workspace überschrieben oder geleert wird) nicht mehr zur Verfügung.

Es bietet sich an, die eigenen Funktionen in einer .R-Datei zu speichern. Mit dem `source()`-Befehl kann die Datei dann in neuen Projekten eingelesen werden, und die Funktionen stehen zur Verfügung.

1

```
source("/Pfad/zu/MeineFunktionen.R")
```

1.3 Markdown

Markdown ist eine “Auszeichnungssprache”, die es erlaubt, mit einer recht einfachen Syntax ansprechende Dokumente zu erstellen. Sie wurde dabei so konzipiert, dass schon der *Quellcode* leicht lesbar ist.

In Kombination mit **RMarkdown** oder **quarto** lassen sich Berichte, Artikel, Präsentationen und Bücher erstellen, die -Code und -Diagramme verwenden.

Mehr Informationen erhalten Sie hier:

<https://quarto.org/docs/authoring/markdown-basics.html>

1.4 RMarkdown

Im Sommer 2022 wurde **RMarkdown** durch den Nachfolger **quarto** abgelöst, weshalb dieses Kapitel eher historische Informationen enthält. Wenn Sie **RMarkdown** noch nie benutzt haben, sollten Sie dieses Kapitel überspringen und direkt mit **quarto** einsteigen, siehe Kapitel 1.5 auf Seite 132.

RMarkdown ermöglicht es, **R**-Code, **R**-Output und -Diagramme, sowie Formeln und externe Quellen (z.B. Bilder aus dem Internet) in ansprechenden Dokumenten (z.B. Webseite, Word, Powerpoint, PDF) zu exportieren. Es ist sehr gut dafür geeignet, die eigenen statistischen Auswertungen und Ergebnisse für *Endanwender* (z.b. Auftragsgeber oder Dozenten) darzustellen.

Die Funktionalität hat sich so sehr weiterentwickelt, dass mittlerweile auch Abschlussarbeiten (Bachelor, Master) komplett mit **RMarkdown** geschrieben werden.

Tatsächlich ist Abschnitt 2.1.7.3 zur Ordinalen Regression zunächst mit **RMarkdown** erstellt und veröffentlicht worden⁴, das ist aber schon sehr lange her.

Wir erstellen in unserem Projekt **ErsteSchritte** nun das erste Markdown-Dokument. Klicken Sie hierzu wieder oben im Scriptfenster auf das grüne + Symbol und wählen Sie **R Markdown...** (Abbildung 1.49).

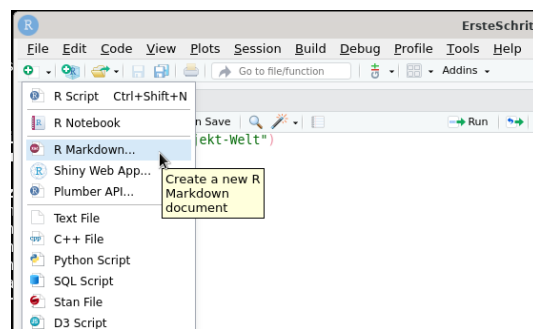


Abb. 1.49

Falls Sie dies zum ersten Mal tun, schlägt **RStudio** die notwendigen Zusatzpaket zur Installation vor (Abbildung 1.50). Die Installation dauert ein paar

⁴<https://rpubs.com/produnis/oreg>

Minuten, und Sie können den Prozess unten im Konsolfenster verfolgen (Abbildung 1.51, siehe grüner Balken).

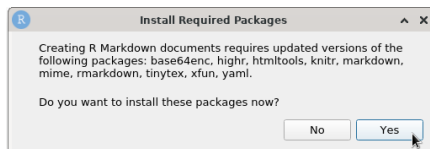


Abb. 1.50



Abb. 1.51: Installationsprozess

Es öffnet sich ein neues Fenster (Abbildung 1.52). Wir erstellen hier ein Dokument. Sie sehen aber schon, dass RMarkdown weitere Formate (z.B. Präsentationen) ebenfalls unterstützt.

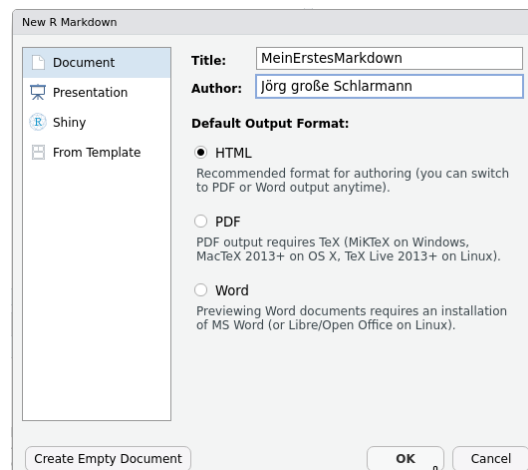


Abb. 1.52

Tragen Sie den Titel des Dokumentes sowie den Autoren ein. Wir belassen in diesem Beispiel die Default Output Option auf HTML. Sie können aber auch Word oder PDF wählen.

Es wird nun eine Standardvorlage des Dokumentes erzeugt (Abbildung 1.53). Bevor wir uns dem Inhalt zuwenden speichern wir das Dokument zunächst ab. Klicken Sie hierzu auf das Diskettensymbol (Mauszeiger in Abbildung 1.53) oder drücken Sie die Tastenkombination [STRG] + [S]. Geben Sie Ihrem Dokument einen Dateinamen mit der Endung .Rmd (für **R**Markdown). Ich habe meine Datei MeinErstesMarkdown.Rmd genannt.

Die Datei ist nun ebenfalls im Dateiverzeichnis zu sehen (Abbildung 1.54) und kann von dort aus geöffnet werden.

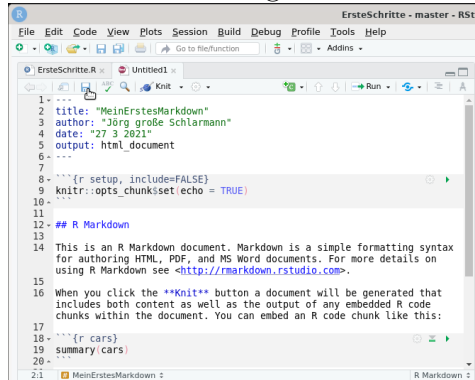


Abb. 1.53

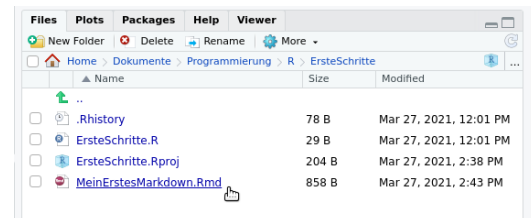


Abb. 1.54

Über dem Scriptfenster befindet sich der Knit-Knopf (Abbildung 1.55, Mauszeiger).

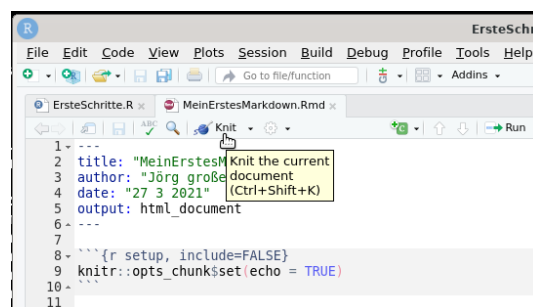


Abb. 1.55: Knit-Knopf

Wenn Sie hier klicken, oder die Tastenkombination [STRG] + [SHIFT] + [K] benutzen, wird aus Ihrem RMarkdown die gewünschte Ausgabedatei (in diesem Beispiel `html`, also eine Webseite) erzeugt und im Arbeitsverzeichnis gespeichert.

Da die Standardvorlage alle wichtigen Dinge enthält können wir direkt auf den Knit-Knopf klicken. Das RMarkdown-Dokument wird nun “übersetzt”, und Sie können den Prozess im Konsolenfenster verfolgen (Abbildung 1.56).

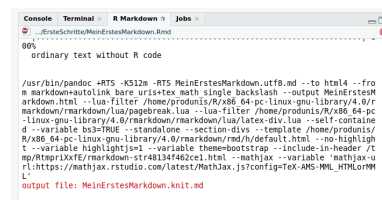


Abb. 1.56: knit-Prozess

Ist dieser Schritt erfolgreich, öffnet sich ein neues Fenster mit dem Endresultat (Abbildung 1.57).

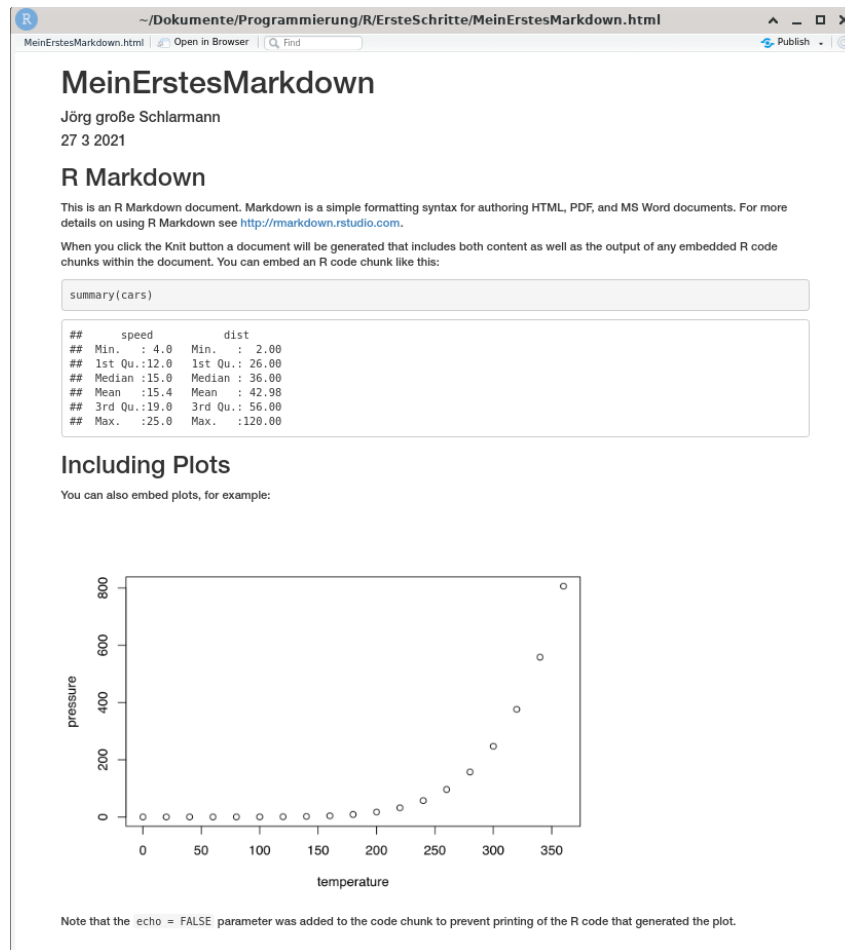
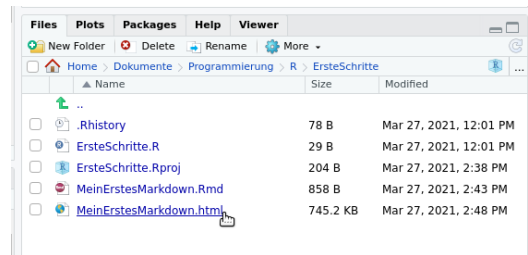


Abb. 1.57: Unser übersetztes *R*Markdown als *html*-Webseite

In unserem Arbeitsverzeichnis sehen wir ebenfalls die soeben erzeugte Ausgabedatei, in Abbildung 1.58 ist das `MeinErstesMarkdown.html`.

Beachten Sie, dass in der `html`-Datei bereits alle nötigen Ressourcen (z.B. alle Bilder) gespeichert sind. Möchten Sie Ihr `html`-Dokument mit Kolleginnen und Kollegen teilen, brauchen Sie keine weiteren Dateien kopieren oder versenden.

Abb. 1.58: erzeugte *html*-Datei

Die *html*-Webseite wird auf allen Geräten “gleich” aussehen, und es werden z.B. keine Grafiken oder Bilder fehlen.

1.4.1 Syntax

Die Syntax von **RMarkdown** ist recht leicht, und es existiert zudem ein Cheatsheet (siehe Abschnitt 1.7.1 auf Seite 202) unter <https://raw.githubusercontent.com/rstudio/cheatsheets/master/rmarkdown-2.0.pdf>.

Schauen wir uns unsere eben erzeugte Markdowndatei noch einmal an. Wir löschen alles, bis auf die ersten Zeilen, und beginnen ganz von vorne.

```

1 ---
2 title: "Ein neuer Titel"
3 author: "Jörg große Schlarmann"
4 date: "27 3 2021"
5 output: html_document
6 ---

```

Dies ist die *Kopfzeile* des Dokuments. Wir können hier den Titel, Autor und das Datum ändern. Unter den drei Strichen -- beginnen wir unser Dokument. Schreiben Sie folgende Zeilen.

```

1 ---
2 title: "Dies ist ein Test"
3 author: "Timm Thaler"
4 date: "27 3 2021"
5 output: html_document
6 ---
7
8 # Überschrift
9
10 Willkommen zu meinem RMarkdown-Dokument
11

```

```

12 - dies ist ein Punkt
13 - dies ein weitere
14
15 1. erstens
16 2. zweitens
17 3. drittens
18
19 ## Unter-Überschrift
20
21 Dies ist fett, dies kursiv, dies durchgestrichen.

```

Drücken Sie den Knit-Knopf und erzeugen so einen Testoutput.

Das Zeichen # steht für Überschriften und deren Hierarchie. Die Anzahl der # legt die Überschriftenhierarchie fest. Aus den Spiegelstrichen ist eine Liste geworden, die Zahlen wurden in eine nummerierte Aufzählungsliste umgewandelt. Schauen Sie im RMarkdown-Cheatsheet nach, welche weiteren Zeichen für welchen Effekt stehen.

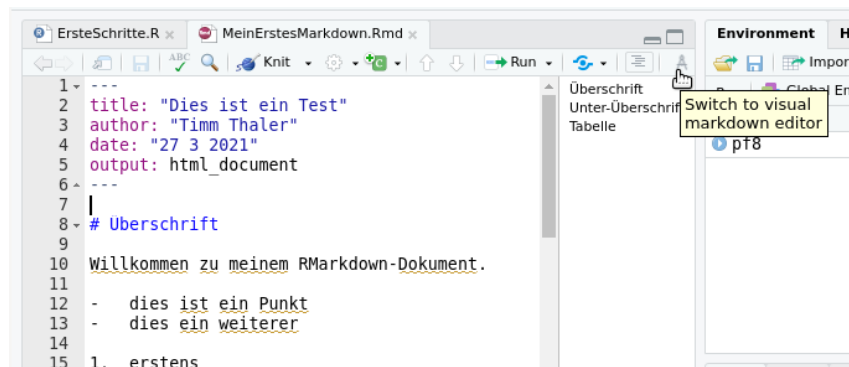


Abb. 1.59: mit A-Knopf zum visuellen Editor

In RStudio gibt es auch einen visuellen Markdown-Modus, der so ähnlich funktioniert wie Word oder LibreOffice. So können Sie Ihren Text “wie gewohnt” gestalten. Klicken Sie hierzu auf das A-Symbol rechts oben (siehe Mauszeiger in Abbildung 1.59).

Der Text sieht nun “fast” schon so aus, wie auf dem Endoutput (Abbildung 1.60).

Sie haben zudem oben eine Menüleiste, mit der Sie die Schriftart wechseln, externe Bilder einbinden oder Tabellen erzeugen können.

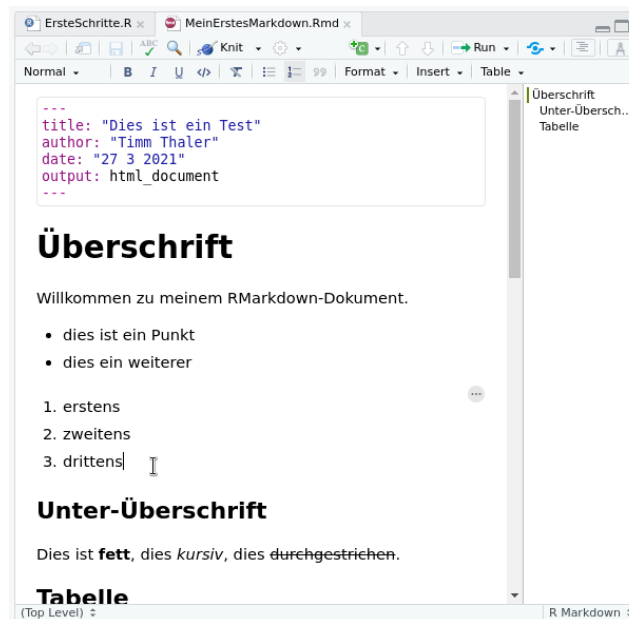


Abb. 1.60

Durch Klick auf das A-Symbol wechseln Sie zwischen visueller und “klassischer” Ansicht.



Achtung, Anfängerfehler!



Denken Sie daran, in die *klassische* Ansicht zurück zu wechseln, bevor Sie Markdown-Code kopieren und einfügen. Dies ist ein häufiger Anfängerfehler, und der visuelle Editor kann mit dem Code nichts anfangen.

In Markdown können sehr leicht Tabellen erzeugt werden. Versuchen Sie diesen Code aus (vorher vom visuellen Editor zurückwechseln!), und beachten Sie, wie “gut lesbar” und “leicht zu merken” das ist.

```

1  ## Tabelle
2
3  | Name   | Beruf       | Alter |
4  |-----|-----|-----|
5  | Heinz  | Kraftfahrer | 46    |
6  | Lisa   | Professorin | 32    |
7  | Jürgen | Klemptner  | 48    |
8
9  : Meine Tabelle

```

Alles weitere finden Sie auf den erwähnten Cheatsheets.

Kommen wir nun zum eigentlichen Kernpunkt: der Darstellung von -Code und -Output.

Hierfür muss im Markdownokument der Bereich für den -Code festgelegt werden. Dies erfolgt über die Zeichenkette `````, die den Codebereich einschließt.

```
1  ```{r}
2
3  # hier steht der R-Code
4
5  ```
```

Die Anführungsstriche grenzen den Codebereich ein. Dieser wird *Chunk* genannt. Das `{r}` bedeutet, dass -Code verwendet wird.



Achtung, Anfängerfehler!



Achten Sie darauf, die korrekten ````` zu verwenden, und nicht etwa `'''` oder `````. Dies ist ein häufiger Anfängerfehler.

In RStudio können Sie einfach die Tastenkombination `[STRG] + [ALT] + [i]` drücken, oder im Scriptfenster oben auf das grüne `c` klicken (Abbildung 1.61).

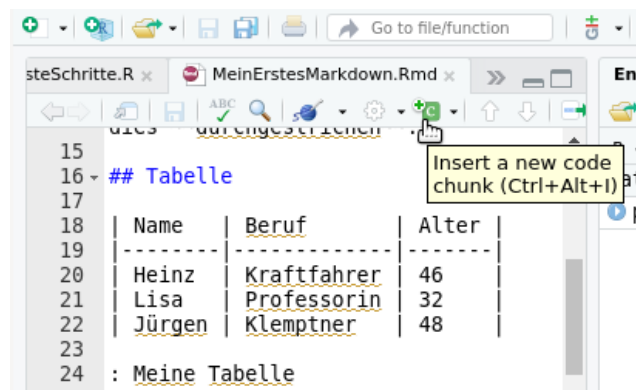


Abb. 1.61: `c` erstellt neuen *Chunk*

Beides fügt einen neuen *Chunk* (Codeblock) ein.

Schreiben Sie in Ihr Dokument:

```
1 Ich lade den pf8-Datensatz mit:
2 ```{r}
3 load(url("https://www.produnis.de/R/pf8.RData"))
4 ```
```

Wenn Sie den Knit-Knopf drücken, wird der `load()`-Befehl im Dokument angezeigt und in der Konsole ausgeführt.

Ergänzen Sie jeweils die nun folgende Zeilen, und schauen Sie sich das Ergebnis nach einem Knit an.

```
1 Ich lade den mma-Datensatz,
2 der Befehl wird aber im Dokument nicht angezeigt.
3 ```{r include=FALSE}
4 load(url("https://www.produnis.de/R/pf8.RData"))
5 ```
```

Durch den Parameter `include=FALSE` wird der Befehl zwar *intern* ausgeführt, jedoch nicht im Dokument angezeigt. Sollte der Befehl einen Output erzeugen, wird dieser ebenfalls nicht angezeigt.

Wenn Sie nur das Ergebnis anzeigen möchten, aber nicht die -Befehle, setzen Sie den Parameter `echo=FALSE`.

```
1 Wir plotten Alter und gewicht aus dem pf8-Datensatz
2 ```{r echo=FALSE}
3 plot(pf8$Alter, pf8$Gewicht)
4 ```
```

In der "Standardeinstellung" werden immer Befehl und Output des Chunks im Dokument angezeigt. Sie können dieses Verhalten auch erzwingen durch den Parameter `echo=TRUE`

```
1 Wir plotten Alter und Gewicht aus dem pf8-Datensatz
2 ```{r echo=TRUE}
3 plot(pf8$Alter, pf8$Gewicht)
4 ```
```

nur Code, kein Output	results="hide"
nur Code, nicht ausführen	eval=FALSE
kein Code, nur Output	echo=FALSE
zeige beides	echo=TRUE
zeige nichts	include=FALSE
mache auch nichts	eval=FALSE, include=FALSE

Tabelle 1.1: Darstellungsoptionen der Chunks

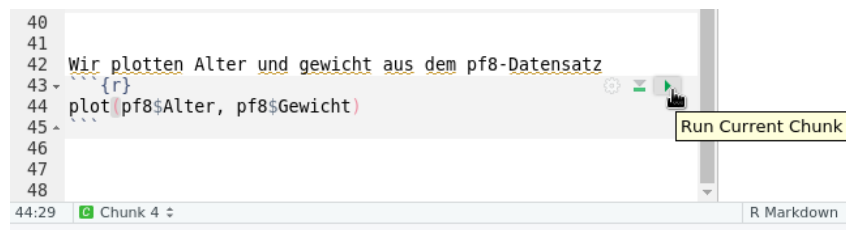


Abb. 1.62: Chunks können im Editor "aktiviert" werden

Die *Chunks* werden im Editorfenster farblich hervorgehoben (Abbildung 1.62). Sie haben hier die Möglichkeit, den Code "ohne Knit" auszuführen und sich das Ergebnis anzeigen zu lassen. Klicken Sie dazu rechts auf das grüne Dreieck.

In diesem Beispiel wird der Plot nun direkt im Markdownfenster angezeigt (Abbildung 1.63). Dies ist wirklich hilfreich beim Erstellen des Dokuments, denn Sie müssen nicht immer erst das gesamte Dokument knitten oder überhaupt geöffnet haben, um zu sehen, was  an dieser Stelle ausgeben würde.

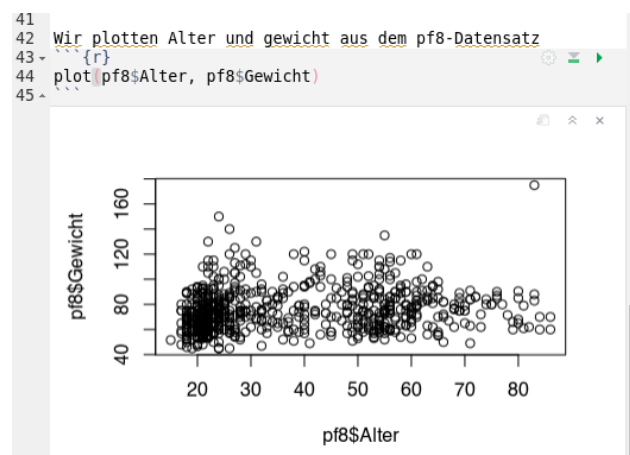


Abb. 1.63: Der Chunk-Output wird bereits im Editor angezeigt

Über das Zahnrad gelangen Sie zu weiteren Einstellungsmöglichkeiten. So können Sie die Chunks intern mit Namen versehen, so dass Sie später im Dokument darauf referenzieren können.

Die vielen weiteren Möglichkeiten von **RMarkdown** werden in diesem Buch nicht weiter behandelt, weitere Informationen erhalten Sie im **RMarkdown**-Cheatsheet (siehe Seite 202) oder im frei verfügbaren Buch *R Markdown: The Definitive Guide*^[9], welches Sie unter <https://bookdown.org/yihui/rmarkdown/> lesen können.

Grundsätzlich sollten Sie es so halten, dass Sie in den Scriptdateien Ihre Auswertungen ausprobieren und mit den Befehlen "herumspielen". Die Quintessenz Ihrer Analyseschritte können Sie dann in "aufgeräumter" Form in einem **RMarkdown** festhalten und mit anderen teilen.

1.5 quarto

quarto ermöglicht es,  -Code, -Output und -Diagramme, sowie Formeln und externe Quellen (z.B. Bilder aus dem Internet) in ansprechenden Dokumenten (z.B. Webseite, Word, Powerpoint, PDF) zu exportieren. Es ist sehr gut dafür geeignet, die eigenen statistischen Auswertungen und Ergebnisse für Endanwender (z.B. Auftraggeber oder Dozenten) darzustellen.



quarto ist der Nachfolger von **RMarkdown** und wurde 2022 veröffentlicht. Basierend auf den Erkenntnissen der letzten 10 Jahre mit **RMarkdown** wurde es von Grund auf neu aufgebaut, um mehr Sprachen und Umgebungen zu unterstützen.

quarto unterstützt die Publikationsformate:

- Artikel, Berichte
- Präsentationen
- Webseiten, Blogs
- Bücher

in den Ausgabeformaten HTML, PDF, ePub und Office.

Dieses Buch wurde jahrelang mit **RMarkdown** ([bookdown](#)) erzeugt und ist im September 2022 auf **quarto** umgezogen.

Mehr Infos erhalten Sie unter <https://www.quarto.org>.

1.5.1 Installation

In einer aktuellen Installation von **RStudio** ist **quarto** bereits enthalten. Unter Linux muss zusätzlich das Paket **quarto-cli** installiert werden. Für Ubuntu steht ein **.deb**-Paket unter <https://quarto.org/docs/get-started/> bereit, unter Archlinux lautet der Befehl:

```
1 yay -S quarto-cli-bin
```

1.5.2 Dokument erstellen

Wir erstellen in unserem Projekt ErsteSchritte nun das erste Markdown-Dokument. Klicken Sie hierzu wieder oben im Scriptfenster auf das grüne + Symbol und wählen Sie **Quarto Document**, siehe Abbildung 1.64.

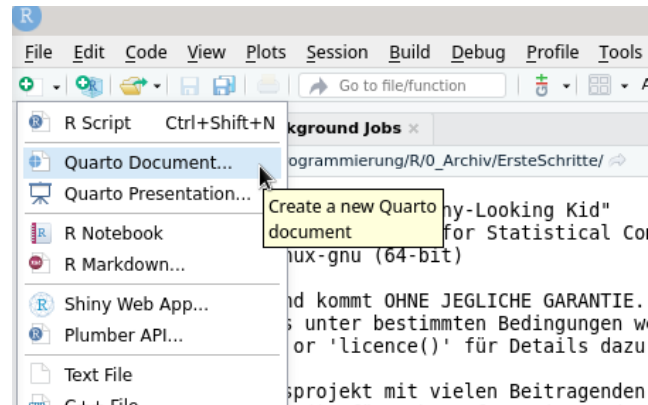


Abb. 1.64: neues Quarto Dokument

Falls Sie dies zum ersten Mal tun, schlägt RStudio die notwendigen Zusatzpakete zur Installation vor. Die Installation dauert ein paar Minuten, und Sie können den Prozess unten im Konsolfenster verfolgen.

Es öffnet sich ein neues Fenster (Abbildung 1.65).

Wir erstellen hier ein Dokument, Sie sehen aber schon, dass **quarto** weitere Formate (z.B. Präsentationen) ebenfalls unterstützt. Tragen Sie den Titel des Dokumentes sowie den Autoren ein. Wir belassen in diesem Beispiel die Default Output Option auf HTML.

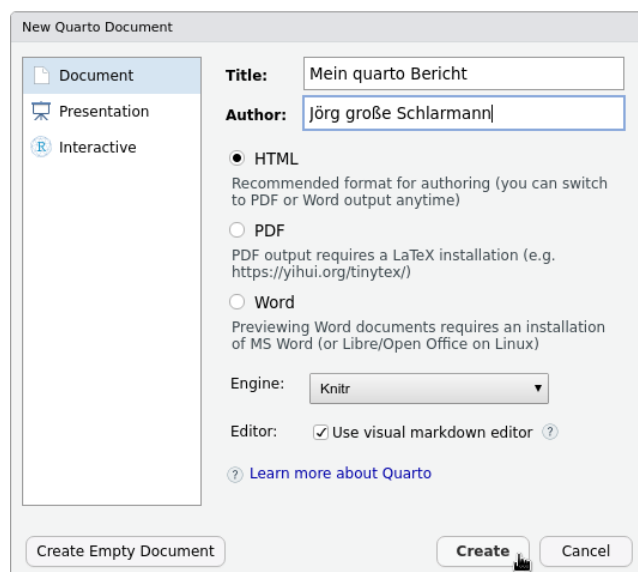


Abb. 1.65: neues HTML Dokument

Es wird nun eine Standardvorlage des Dokumentes erzeugt (Abbildung 1.66). Bevor wir uns dem Inhalt zuwenden speichern wir das Dokument zunächst ab. Klicken Sie hierzu auf das Diskettensymbol (Mauszeiger in Abbildung 1.66) oder drücken Sie die Tastenkombination [STRG] + [S]. Geben Sie Ihrem Dokument einen Dateinamen mit der Endung `.qmd` (für **q**uarto-**M**arkdown). Ich habe meine Datei `Mein-quarto-Dokument.qmd` genannt.

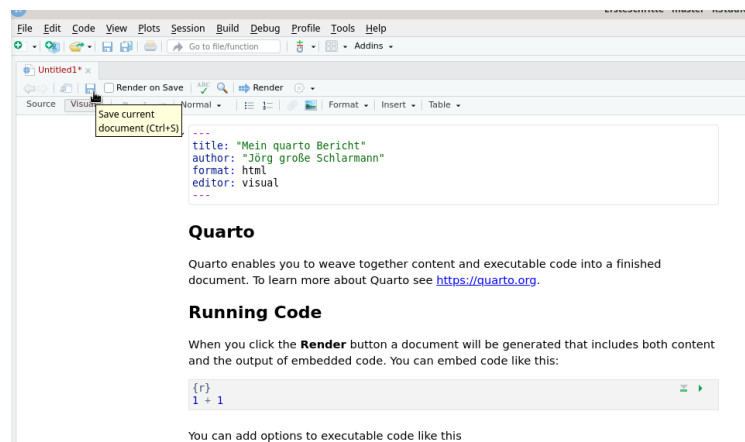


Abb. 1.66: speichern über Diskettensymbol

Die Datei ist nun ebenfalls im Dateiverzeichnis zu sehen (Abbildung 1.67) und kann von dort aus geöffnet werden.

Über dem Scriptfenster befindet sich der **Render**-Knopf (Abbildung 1.68, Mauszeiger).

Wenn Sie hier klicken, oder die Tastenkombination [STRG] + [SHIFT] + [K] benutzen, wird aus Ihrem quarto-Markdown die gewünschte Ausgabe-datei (in diesem Beispiel `html`, also eine Webseite) erzeugt und im Arbeitsverzeichnis gespeichert.

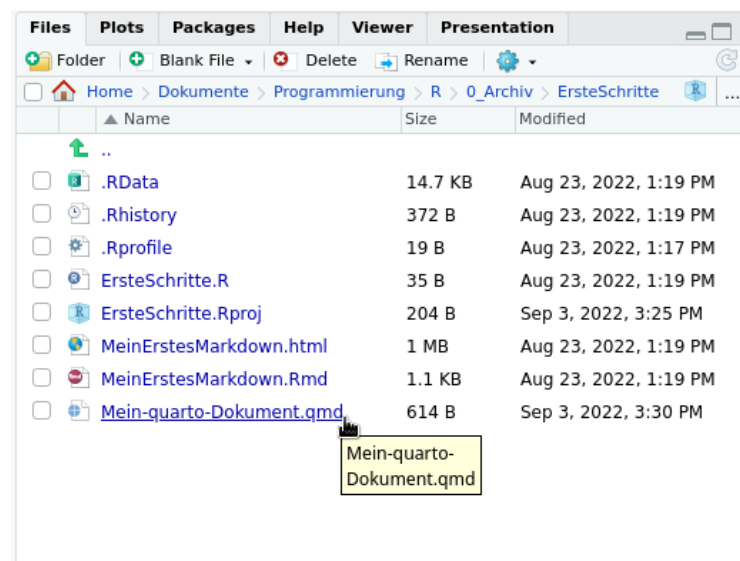


Abb. 1.67: quarto-Dokument im Arbeitsverzeichnis

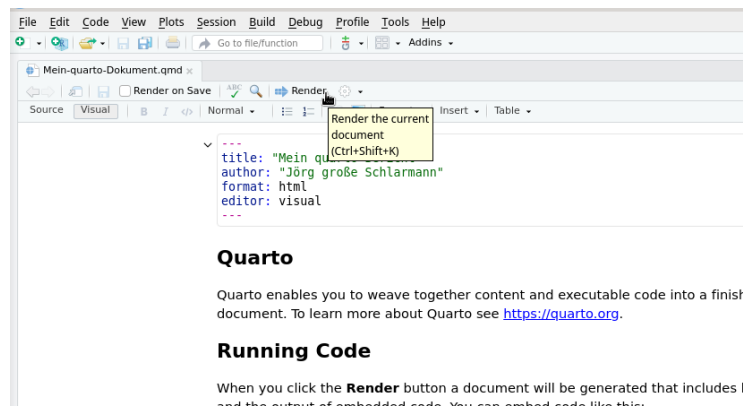


Abb. 1.68: Render-Knopf

Da die Standardvorlage alle wichtigen Dinge enthält können wir direkt auf den **Render-Knopf** klicken. Das `quarto-` Dokument wird nun “übersetzt”, und Sie können den Prozess im Konsolenfenster verfolgen (Abbildung 1.69).

Ist dieser Schritt erfolgreich, öffnet sich ein neues Fenster mit dem Endresultat (Abbildung 1.71).

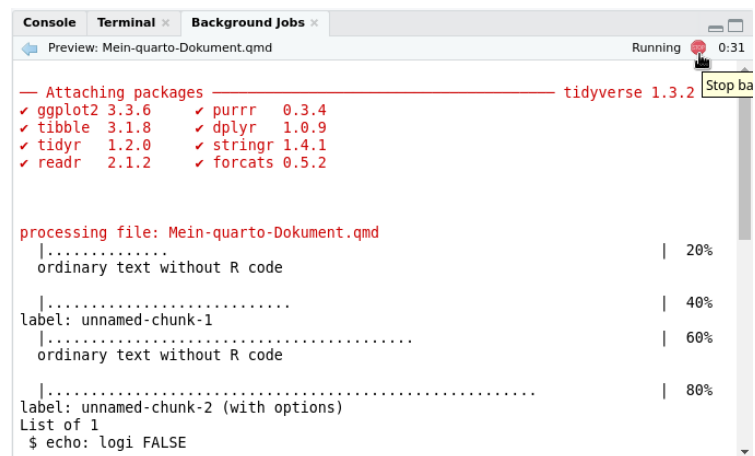


Abb. 1.69: Render-Prozess

In unserem Arbeitsverzeichnis sehen wir ebenfalls die soeben erzeugte Ausgabedatei, in Abbildung 1.70 ist das `Mein-quarto-Dokument.html`.

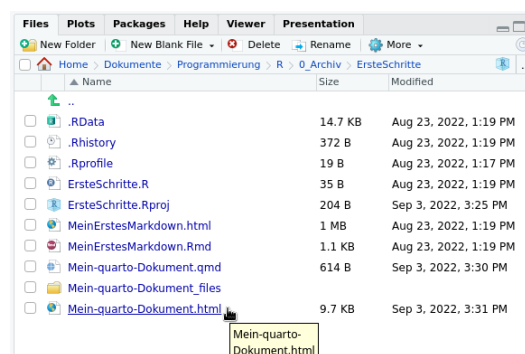


Abb. 1.70: fertige HTML-Datei im Arbeitsverzeichnis

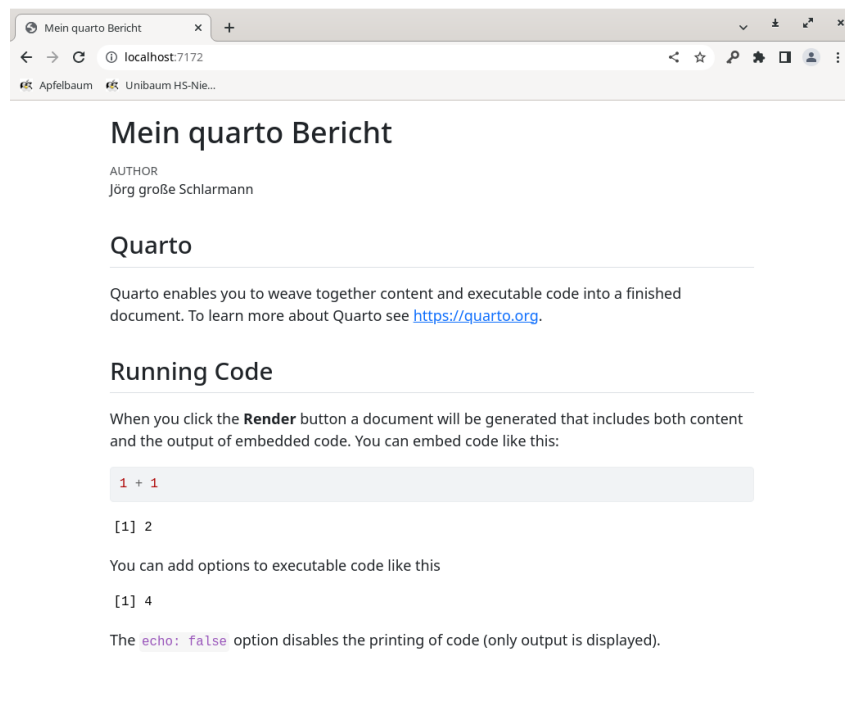
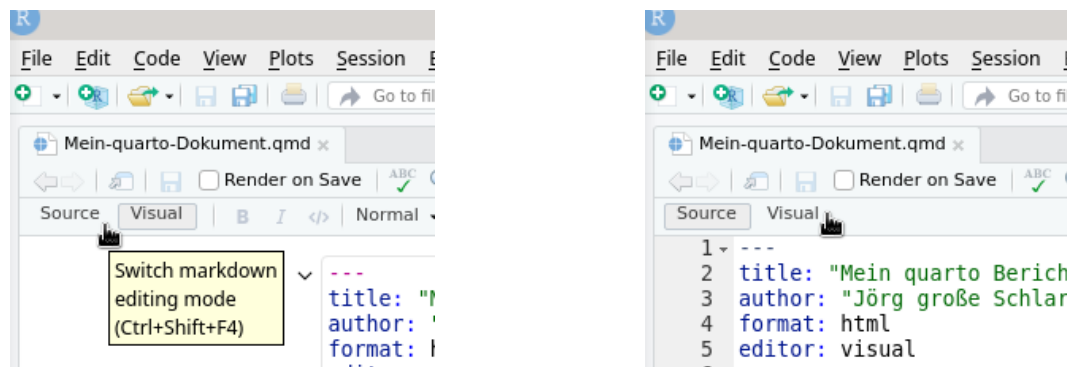


Abb. 1.71: fertiges *quarto* Dokument

1.5.3 Editor-Ansicht

Das quarto-Dokument startet im “visuellen Editor”, der so ähnlich funktioniert wie Word oder LibreOffice. So können Sie Ihren Text “wie gewohnt” gestalten. Sie haben zudem oben eine Menüzeile, mit der Sie die Schriftart wechseln, externe Bilder einbinden oder Tabellen erzeugen können.

Auf der linken Seite können Sie zwischen dem visuellen Editor und der Code-Ansicht wechseln.



☠ Achtung, Anfängerfehler! ☠

Denken Sie daran, in die klassische Code-Ansicht zurück zu wechseln, bevor Sie Markdown-Code kopieren und einfügen. Dies ist ein häufiger Anfängerfehler, und der visuelle Editor kann mit dem Code nichts anfangen.

1.5.4 Dokumentenaufbau

Schauen wir uns unsere eben erzeugte Markdowndatei noch einmal an. Wechseln Sie hierzu in die Code-Ansicht.

```

1 ---
2 title: "Mein quarto Bericht"
3 author: "Jörg große Schlarmann"
4 format: html
5 editor: visual
6 ---

```

Dies ist die *Kopfzeile* des Dokuments, die im **yaml**-Format angegeben ist (so genannter *yaml-Header*). Wir können hier den Titel, Autor und das Datum

ändern. Alle Parameter des Headers finden Sie unter <https://quarto.org/docs/reference/formats/html.html>

Unter den drei Strichen `--` beginnen wir unser eigentliches Dokument. Schreiben Sie folgende Zeilen in Ihr Dokument:

```

1  ---
2  title: "Mein quarto Bericht"
3  author: "Jörg große Schlarmann"
4  date: "01 09 2022"
5  format: html
6  editor: visual
7  ---
8
9  # Überschrift
10
11 Willkommen zu meinem quarto-Markdown-Dokument
12
13 - dies ist ein Punkt
14
15 - dies ein weiterer
16
17 1. erstens
18
19 2. zweitens
20
21 3. drittens
22
23 ## Unter-Überschrift
24
25 Dies ist fett, dies kursiv, dies durchgestrichen.
```

Drücken Sie den **Render-Knopf** und erzeugen so einen Testoutput (vgl. Abbildung 1.72).

Das Zeichen `#` steht für Überschriften und deren Hierarchie. Die Anzahl der `#` legt die Überschriftenhierarchie fest. Aus den Spiegelstrichen ist eine Liste geworden, die Zahlen wurden in eine nummerierte Aufzählungsliste umgewandelt.

Mein quarto Bericht

AUTHOR
Jörg große Schlarmann

PUBLISHED
September 1, 2022

Überschrift

Willkommen zu meinem quarto-Markdown-Dokument

- dies ist ein Punkt
- dies ein weiterer

1. erstens

2. zweitens

3. drittens

Unter-Überschrift

Dies ist **fett**, dies *kursiv*, dies ~~durchgestrichen~~.

Abb. 1.72: fertig gerendertes Testdokument

1.5.5 Syntax

Die Syntax von *Markdown* ist recht leicht, die folgenden Beispiele wurden aus dem [Quarto-Guide](#) hierhin übernommen:

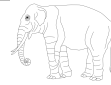
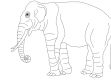
1.5.5.1 Textformat

<code>*kursiv*</code> und <code>**fett**</code>	<i>kursiv</i> und fett
<code>superscript²</code> und <code>subscript₂</code>	superscript ² und subscript ₂
<code>~~durchgestrichen~~</code>	durchgestrichen
<code>`verbatim code`</code>	verbatim code

1.5.5.2 Überschriften

# Header 1	Überschrift 1
## Header 2	Überschrift 2
### Header 3	Überschrift 3
#### Header 4	Überschrift 4
##### Header 5	Überschrift 5
##### Header 6	Überschrift 6

1.5.5.3 Links und Bilder

<code><https://www.produnis.de></code>	https://www.produnis.de
<code>[Produnis] (https://www.produnis.de)</code>	Produnis
<code>! [Caption] (elephant.png)</code>	
<code>! [Caption] (elephant.png) (https://www.produnis.de)</code>	
<code>! [Caption] (elephant.png) (https://www.produnis.de "Ein Elefant")</code>	

1.5.5.4 Listen

```

1  * ungeordnete Liste
2    + sub-Item
3    + sub-Item 2
4      - sub-sub-Item

```

- ungeordnete Liste
 - sub-Item
 - sub-Item 2
 - * sub-sub-Item

```

1  1. nummerierte Liste
2  2. zweites Item
3    i) sub-Item
4      A. sub-sub-Item

```

1. nummerierte Liste
2. zweites Item
 - i) sub-Item
 - A. sub-sub-Item

1.5.5.5 Tabellen

```

1  ## Tabelle
2
3  | Name   | Beruf      | Alter |
4  |-----|-----|-----|
5  | Heinz  | Kraftfahrer | 46    |
6  | Lisa   | Professorin | 32    |
7  | Jürgen | Klemptner  | 48    |
8
9  : Meine Tabelle

```

Name	Beruf	Alter
Heinz	Kraftfahrer	46
Lisa	Professorin	32
Jürgen	Klemptner	48

Tabelle 1.2: Meine Tabelle

1.5.5.6 Gleichungen

Mit `$` können In-Text-Formeln erzeugt werden, mit `$$` schließt man Matheumgebungen ein:

<code>inline math: \$E = mc^2\$</code>	<code>inline math: $E = mc^2$</code>
<code>display math: \$\$E = mc^2\$\$</code>	<code>display math:</code>

$$E = mc^2$$

Mehr Informationen erhalten Sie hier:

<https://quarto.org/docs/authoring/markdown-basics.html>

1.5.6 R-Code integrieren

Kommen wir nun zum eigentlichen Kernpunkt: der Darstellung von -Code und -Output.

Hierfür muss im Markdownokument der Bereich für den Code festgelegt werden. Dies erfolgt über die Zeichenkette "```", die den Codebereich einschließt.

```

1  ```{r}
2
3  # hier steht der R-Code
4
5  ```
```

Die Anführungsstriche grenzen den Codebereich ein. Dieser wird *Chunk* genannt. Das `{r}` bedeutet, dass -Code verwendet wird.

 Achtung, Anfängerfehler! 

Achten Sie darauf, die korrekten ````` zu verwenden, und nicht etwa `'''` oder `````. Dies ist ein häufiger Anfängerfehler.

In 'RStudio' können Sie einfach die Tastenkombination `[STRG] + [ALT] + [i]` drücken, oder im Scriptfenster oben auf das grüne `c` klicken, siehe Mauszeiger in Abbildung 1.73.

Beides fügt einen neuen *Chunk* (Codeblock) ein.

Die *Chunks* werden im Editorfenster farblich hervorgehoben. Tragen Sie folgenden Code in den Chunk ein:

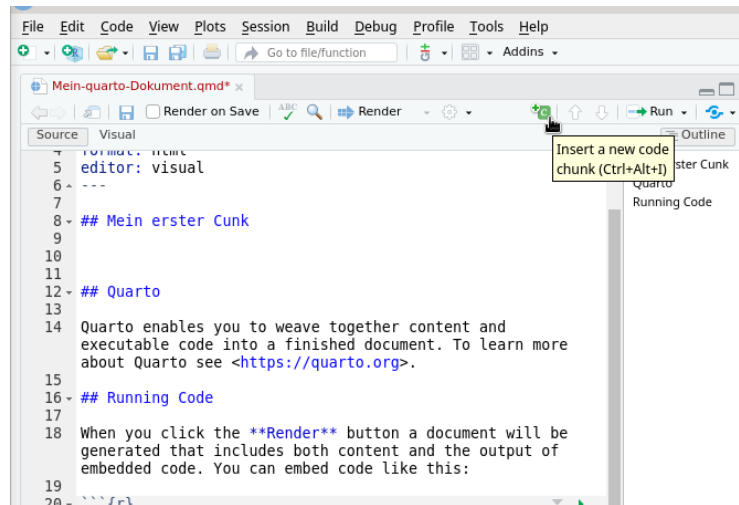


Abb. 1.73: einen neuen 'Chunk' erstellen

```
1 load(url("https://www.produnis.de/R/pf8.RData"))
2 plot(pf8$Alter, pf8$Gewicht)
```

Wenn Sie den *Render*-Knopf drücken, wird der *load()*-Befehl im Dokument angezeigt und in der Konsole ausgeführt. Das neu gerenderte HTML-Dokument öffnet sich automatisch.

Sie haben auch die Möglichkeit, den Code auszuführen und sich das Ergebnis anzeigen zu lassen, ohne das komplette Dokument zu *Rendern*. Klicken Sie dazu rechts auf das grüne Dreieck (Abbildung 1.74).

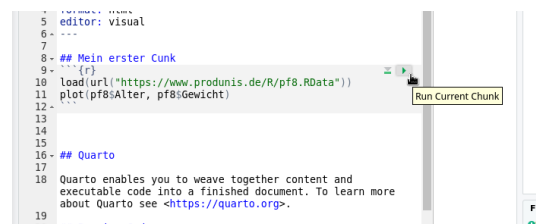


Abb. 1.74: Chunk-Code ausführen

In diesem Beispiel wird der Plot nun direkt im Markdownfenster angezeigt (Abbildung 1.76). Dies ist wirklich hilfreich beim Erstellen des Dokuments, denn Sie müssen nicht immer erst das gesamte Dokument rendern oder überhaupt geöffnet haben, um

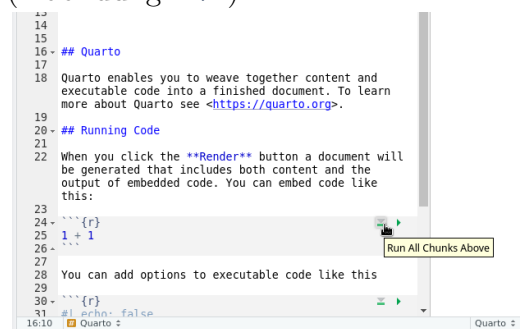


Abb. 1.75: Chunks bis hier ausführen

zu sehen, was  an dieser Stelle ausgeben würde.

Über das Symbol links daneben (Abbildung 1.75, siehe Mauszeiger) lassen sich alle Chunks bis hier hinausführen.

Dies ist zum Beispiel hilfreich, wenn in den vorherigen Chunks Objekte oder Variablen erzeugt wurden, auf die nun zugegriffen werden soll.

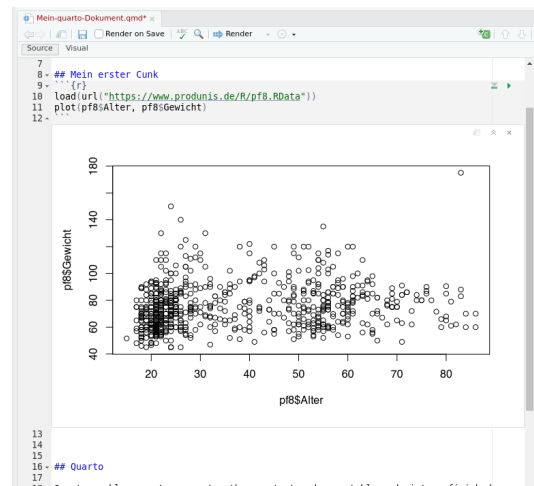


Abb. 1.76: Chunk-Output im Editor

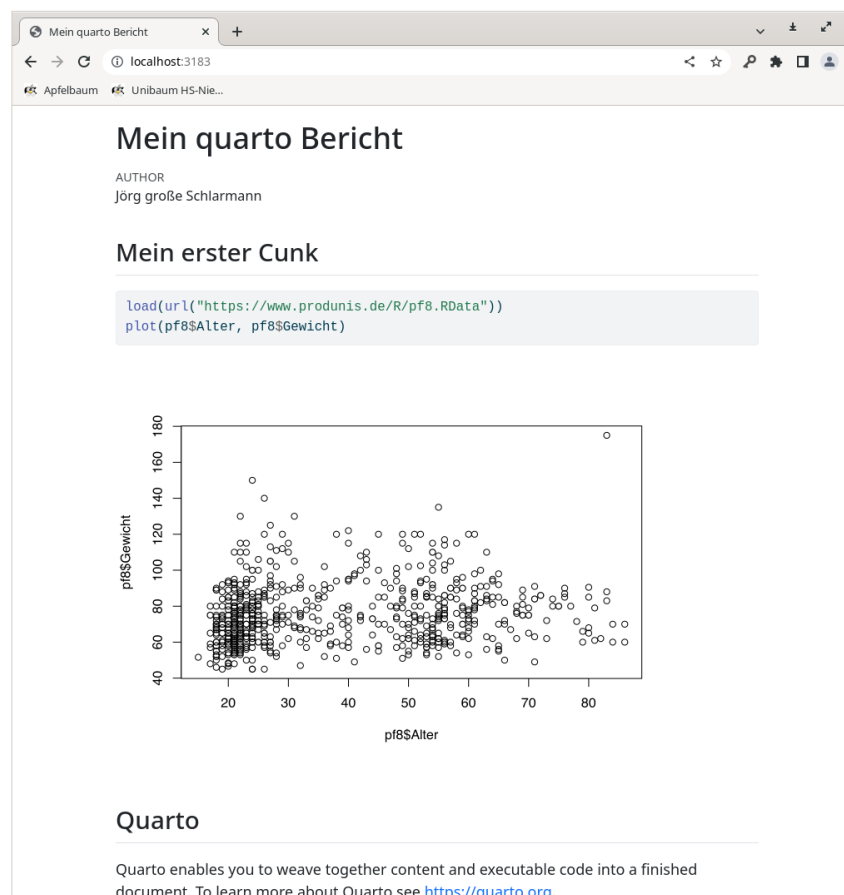


Abb. 1.77: Dokument mit Plot

1.5.6.1 Chunk-Optionen

Den Chunks können Parameter übergeben, um ihr Verhalten zu steuern. Dies erfolgt über die Zeichenfolge `#|` direkt am Zeilenanfang.

```

1  ```{r}
2  #| label: fig-testplot # zum verlinken der Plots
3  #| fig-cap: "Testplot" # Plotunterschrift
4  #| echo: true          # R-Befehle anzeigen
5  #| output: true        # R-Output anzeigen
6  #| warning: false      # Warnung unterdrücken
7  #| include: true       # Chunk anzeigen
8  #| eval: true          # Code ausführen
9  load(url("https://www.produnis.de/R/pf8.RData"))
10 plot(pf8$Alter, pf8$Gewicht)
11 ```

```

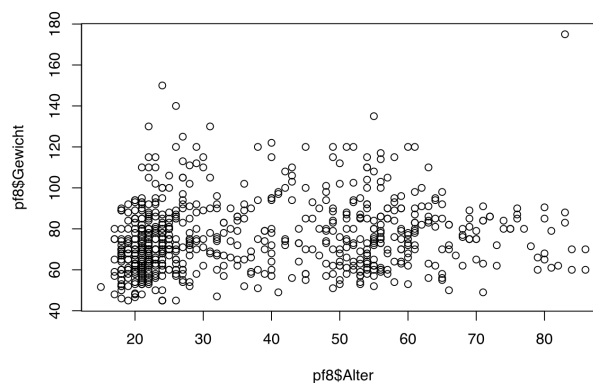


Abb. 1.78: Testplot

Der Plot ist über den Referenzierer `@fig-testplot` ansprechbar. Dieser wird beim rendern umgewandelt in `→` "Abbildung 1.78".

Die Standard-Parameter lauten:

- `label` - gibt dem Chunk einen Namen. Dies ist beim Debugging sehr hilfreich. Soll auf die Plots des Chunks referenziert werden, muss das Label zwingend mit `fig-` anfangen.
- `fig-cap` - gibt den Plottitel an.

- **echo** - gibt an, ob die Befehle des Chunks im Dokument angezeigt werden sollen (**true** / **false**).
- **output** - gibt an, ob die R-Ausgaben im Dokument angezeigt werden sollen (**true** / **false**).
- **warning** - gibt an, ob die Warnungen, die möglicherweise beim Ausführen des Codes ausgegeben werden, im Dokument angezeigt werden sollen (**true** / **false**).
- **include** - gibt an, ob der Chunk (Befehle und Ausgabe) im Dokument angezeigt werden soll (**true** / **false**).
- **eval** - gibt an, ob der Chunk ausgeführt (**true**) werden, oder nur der Chunkinhalt angezeigt werden soll (**false**).

1.5.7 PDF-Dokumente erzeugen

Da PDF-Dokumente von **quarto** intern mittels $\text{\LaTeX}$ erzeugt werden, müssen Sie auf Ihrem PC eine $\text{\LaTeX}$ Distribution installiert haben. Falls dies nicht der Fall ist, können Sie **TinyTeX** mit folgendem Befehl im RStudio Terminal installieren:

```
1 quarto install tool tinytex
```

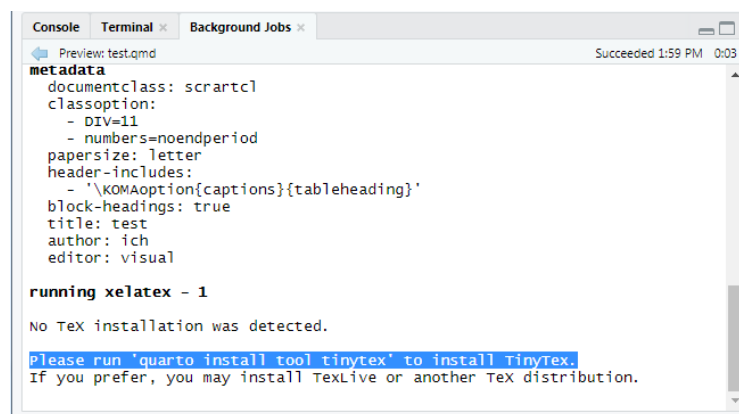


Abb. 1.79: *TinyTeX*

Mit dieser Methode installiert quarto alle weiteren Pakete automatisch, sofern diese benötigt werden. Sollten Sie bereits eine L^AT_EX-Distribution installiert haben, müssen Sie das Paket `fontawesome` installieren, damit alles funktioniert.

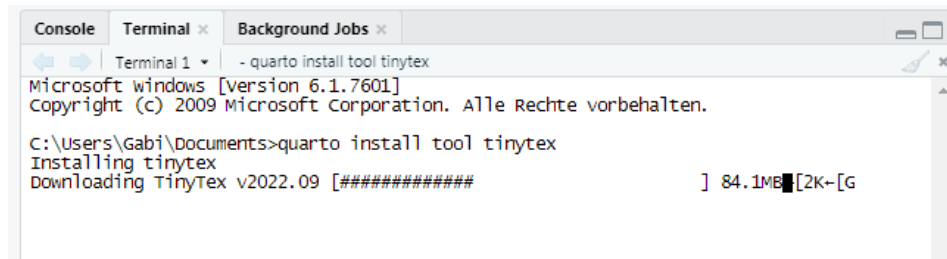


Abb. 1.80: TinyTeX installieren

1.5.8 quarto-Dokumentation

Die Webseite von quarto bietet umfangreiche Anleitungs- und Referenzierungsdokumente an.

- Der **Guide** leitet Sie durch die Erstellung Ihrer Dokumente, siehe <https://quarto.org/docs/guide/>, und ist für Einsteiger gut geeignet.
 - Für HTML z.B. <https://quarto.org/docs/output-formats/html-basics.html>
- Die **References** listen alle Parameter und Einstellungsmöglichkeiten der quarto-Dokumente auf, siehe <https://quarto.org/docs/reference/>.
 - Für HTML z.B. <https://quarto.org/docs/reference/formats/html.html>
- Auf Youtube gibt es eine offizielle Quarto-Playlist <https://www.youtube.com/playlist?list=PL9HYL-VRX0oRupficE2l5DGgVIzpyPTHs>

In der *Awesome Quarto List* auf Github <https://github.com/mcanouil/awesome-quarto> werden quartobezogene Anleitungen, Präsentationen, Erweiterungen und Beispiele gesammelt.

Beachten Sie auch Kapitel 3.5 "Was willst du machen" ab Seite 414.

1.6 Tidymverse

Das Tidymverse ist eine Weiterentwicklung von **R**, die maßgeblich von Hadley Wickham vorangetrieben wurde (weshalb auch die Bezeichnung *Hadleyverse* noch gebräuchlich ist. Wickham ist mittlerweile Chefentwickler von **RStudio**). Das Standardwerk zum Tidymverse “*R for Data Science*” (kurz *R4DS*) stammt ebenfalls von Wickham[7], und ist online frei verfügbar unter <https://r4ds.had.co.nz/> (eine physische Kopie des Buches kann ebenfalls gekauft werden).

Das Tidymverse folgt streng dem Konzept *Tidy Data*[6], welches besagt, dass ein Datensatz so aufgebaut sein muss, dass jeweils ein Fall pro Zeile abgebildet wird. Das bedeutet, dass jede Beobachtung (auch Wiederholungen) in einer eigenen Zeile steht, und die jeweiligen Variablen durch die Spalten repräsentiert werden.

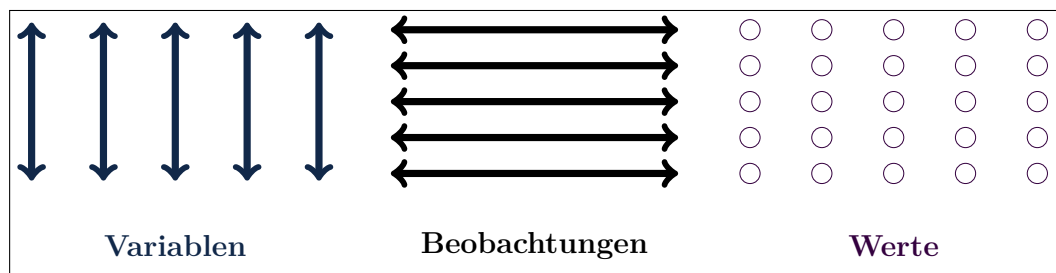


Abb. 1.81: Tidy Data Konzept

Abbildung 1.81 zeigt die Sicht auf ein Datenobjekt nach dem Tidy Data Konzept. Jede Variable wird als eigene Spalte dargestellt. Jede Reihe entspricht einer Beobachtung. Somit entsprechen die einzelnen Werte in der Tabelle genau einer Variable zu einer Beobachtung.

Man spricht in diesem Zusammenhang von “long table” und “wide table”. Die Matrix der Pflegeberufe, die wir aus Abbildung 1.26 übernommen haben, stellt dabei die *wide table*, die *breite* Tabelle dar.

	1999	2001	2003	2005	2007	2009	2011	2013	2015
Krankenpflegeassistenz	16624	19061	19478	21537	27731	36481	46517	54371	64127
Altenpflegehilfe	55770	52710	49727	45776	48326	47903	47978	48363	49507
Kinderkrankenpflege	47779	48203	48822	48519	49080	49307	48291	48937	48913
Krankenpflege	430983	436767	444783	449355	457322	465446	468192	472580	476416
Altenpflege	109161	124879	141965	158817	178902	194195	208304	227154	246412

“Breit” bedeutet, dass die Tabelle, wenn wir ihr nun 10 weitere Jahrgänge hinzufügen würden, immer breiter und breiter werden würde. Die *wide table* ist als *Darstellung* der Daten sicherlich gut geeignet, und die Leser erhalten eine gute Übersicht. Für die *Datenanalyse* und -verarbeitung ist dieses Format eher ungeeignet, da mit jedem Wert in der Tabelle eigentlich *zwei* Variablen adressiert werden (konkrete Berufsgruppe in einem konkreten Jahr). Welche beiden Variablen das nun genau sind, lässt sich am Wert alleine nicht ablesen, und wir hätten große Probleme, wenn Zeilen und Spalten nicht benannt wären!

Als Faustregel kann man sich merken, dass eine *wide table* dann vorliegt, wenn auch (bedeutsame) Zeilennamen im Datensatz vergeben wurden (was bei einer **matrix** häufig der Fall ist).

Ein Datensatz nach dem Tidy Data-Konzept ist vom Typ *long table*. Für die Matrix der Pflegeberufe sähe die **long table**-Version des Datensatzes so aus:

	Jahr	Berufsgruppe	Anzahl
1	1999	Krankenpflegeassistentz	16624
2	2001	Krankenpflegeassistentz	19061
3	2003	Krankenpflegeassistentz	19478
4	2005	Krankenpflegeassistentz	21537
5	2007	Krankenpflegeassistentz	27731
6	2009	Krankenpflegeassistentz	36481
7	2011	Krankenpflegeassistentz	46517
8	2013	Krankenpflegeassistentz	54371
9	2015	Krankenpflegeassistentz	64127
10	1999	Altenpflegehilfe	55770
11	2001	Altenpflegehilfe	52710
12	2003	Altenpflegehilfe	49727
	(. . .)		

Jede Spalte repräsentiert eine Variable, und jede Zeile repräsentiert eine Beobachtung. Auch ohne die Spaltennamen (Zeilennamen gibt es gar keine) könnten wir erkennen, worum es in dem Datensatz geht.

Der Name **long table** leitet sich davon ab, dass die Tabelle, wenn wir ihr mehr Daten hinzufügen würden, immer *länger* und länger werden würde.

Die Tatsache, dass die Verarbeitung solcher **long tables** wesentlich einfacher ist, bildet den Grundgedanken des Tidyverse, und es wurden spezielle

-Erweiterungen implementiert, die speziell an Tidy Data angepasst sind. Die Erweiterungen wurden nach dieser gemeinsame Philosophie entworfen, sie verwenden die selbe “Grammatik” und greifen auf die selbe Datenstruktur (*tibbles*, dazu später mehr) zurück.

Abbildung 1.82 zeigt den typischen Ablauf einer Datenauswertung in 4 Schritten.

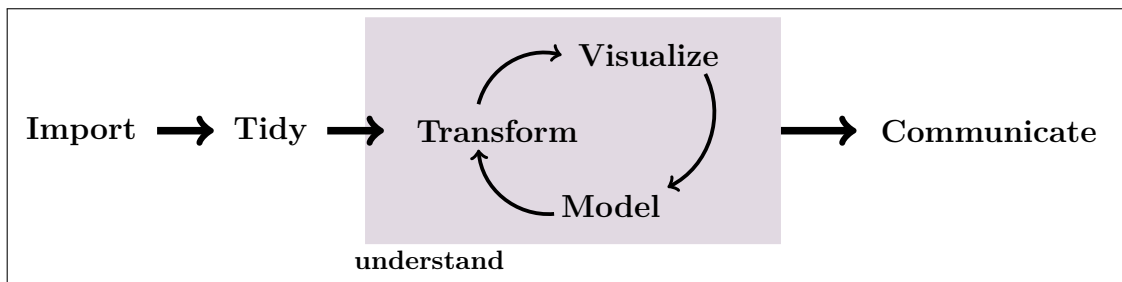


Abb. 1.82: Ablauf der Datenverarbeitung nach dem Tidy Data Konzept (in [7])

1. Die Daten müssen importiert und
2. gegebenenfalls angepasst (ins `tidy data` bzw. `long table` Format überführt werden).
3. Anschließend werden die Daten “statistisch ausgewertet”, d.h. wir versuchen zu verstehen, was wir in den Daten sehen und welche spezifischen Informationen extrahiert werden können. Das heisst die Daten werden transformiert (z.B. indem man Untergruppen bildet) und visualisiert (z.B. in Form von Diagrammen und Tabellen) oder wir rechnen Modelle wie Korrelationen, Regressionen und Signifikanztests.
4. Unsere Erkenntnisse nutzen wir dann zur Kommunikation (Präsentation) unserer Forschungsergebnisse.

Für all diese Arbeitsschritte stellt das Tidyverse passende Pakete und Funktionen in  zur Verfügung, und wir werden die Schritte nun einzeln durchlaufen.

1.6.1 Pakete

In  besteht das Tidyverse im Wesentlichen aus dem Zusatzpaket `tidyverse`, welches alle notwendigen Unterpakete als Abhängigkeit mit-installiert:

1. `Tibble`, die Datenklasse im Tidyverse, im Paket

- `tibble`, mit den Funktionen
`tibble()` `as_tibble()`

2. Daten importieren:

- `readr`, mit den Funktionen
`read_csv()`, `read_tsv()`, `read_delim()`, `read_table()`

3. Daten *tidy* machen:

- `tidyr`, mit den Funktionen
`pivot_longer()`, `pivot_wider()`, `separate()`, `unite()`, `drop_na()`, `replace_na()`, `fill()`,
`complete()`, `expand()`
- `forcats`, mit den Funktionen
`as_factor()`

4. Umgang mit Datensätzen:

- `dplyr`, mit den Funktionen
`arrange()`, `filter()`, `select()`, `pull()`, `mutate()`, `transmute()`, `rename()`, `summarise()`,
`bind_cols()`, `bind_rows()`
- `purrr`, mit den Funktionen
`modify()`, `map()`, `reduce()`, `nest()`
- `stringr`, mit den Funktionen
`str_sub()`, `bind_locate()`, `bind_extract()`, `bind_count()`, `bind_match()`

5. Daten visualisieren:

- `ggplot`, mit den Funktionen
`ggplot()`, `qplot()`

Dazu später mehr, denn um die Funktionen vorzustellen, müssen wir uns zunächst der *Pipe* zuwenden.

1.6.2 Pipe

Ein wichtiger Operator für die Arbeit im Tidyverse ist die **Pipe** aus dem **magrittr**-Zusatzpaket⁵. Sie erlaubt es, *Datenströme* weiterzuleiten. Ihre Funktionalität hat so sehr überzeugt, dass sie im Standard-R ab Version 4.1 enthalten ist, siehe Abschnitt 1.2.3.8.

Hierfür wurde ein eigener Operator implementiert, die Zeichenkette `%>%` (im Standard-R lautet die Zeichenkette `|>`).

Sie bedeutet so viel wie “*und dann*”.

Zur Erklärung sei nochmals der “klassische Weg” der Arbeitsschritte in  aufgezeigt:

```
1 # wir machen etwas, und speichern es in ein Objekt
2 habe.getan <- mache.etwas(Datensatz)
3
4 # dann machen wir etwas weiteres,
5 # und speichern wieder in ein Objekt
6 habe.weiteres.getan <- mache.weiteres(habe.getan)
7
8 # und kommen schließlich zum Endergebnis
9 endergebnis <- mache.noch.letzte.Sache(habe.weiteres.getan)
```

Mit Einsatz der Pipe wird dieser Prozess quasi umgekehrt in

```
1 # Speichere Endergebnis
2 endergebnis <- Datensatz %>%
3   gruppieren.nach.geschlecht %>%
4     sortiere Alter aufsteigend %>%
5       nimm die letzten 4 Werte
```

Die liest sich in etwa so:

- “Speichere etwas ins Objekt `endergebnis`, und das geht so...
- nimm den `Datensatz` *und dann*
- gruppieren die Daten nach `geschlecht` *und dann*
- sortiere nach `Alter` aufsteigend *und dann*

⁵Die Pipe wird mit **tidyverse** aktiviert, Sie müssen **magrittr** nicht extra installieren.

- nimm die letzten 4 Werte. ”

Die **Pipe** reicht das jeweilige Ergebnis (den *Datenstrom*) an die nächste Code-Zeile weiter. Erst wenn die letzte Zeile durchgelaufen ist, wird das Resultat in **endergebnis** gespeichert.

Dies liest sich zu Beginn evtl. etwas ungewohnt, aber Sie können erkennen, dass die Befehle so wesentlich übersichtlicher und die einzelnen *Manipulationsschritte* nachvollziehbarer geworden sind. Auch kann man sich diese Art der “Grammatik” relativ leicht merken.

Benötigt eine Funktion die Angabe eines Datensets, so kann mit einem Punkt **.** auf den Pipe-Datenstrom verwiesen werden.

```
1 # Der Punkt übergibt den Pipe-Strom an die Funktion sum()
2 endergebnis <- Datensatz %>%
3     wähle Spalte "Alter" %>%
4     # berechne Summe
5     sum(.)
```

Die Pipe funktioniert bei jedem **R**-Objekt, egal in welchem Format (**long table** vs. **wide table**, Faktor, Vektor, Matrix, usw.) es vorliegt.

1.6.3 Unterschiede zwischen %>% und |>

Während sich **|>** und **%>%** in einfachen Fällen identisch verhalten, gibt es ein paar wichtige Unterschiede. Diese betreffen Sie aber nur, wenn Sie **%>%** schon lange nutzen und einige der fortgeschrittenen Funktionen verwendet haben.

1. Standardmäßig übergibt die Pipe das Objekt auf ihrer linken Seite an das erste Argument der Funktion auf der rechten Seite. Mit **%>%** können Sie die Platzierung des Datenstroms mit dem Platzhalter “.” ändern. Zum Beispiel ist **x %>% f(1)** äquivalent zu **f(x, 1)**, aber **x %>% f(1, .)** ist äquivalent zu **f(1, x)**. Seit **R** 4.2.0 ist ein ähnlicher Platzhalter über **_** zur Basis-Pipe hinzugefügt worden, jedoch muss hier immer ein Argument benannt werden. Zum Beispiel ist **x |> f(1, y = _)** äquivalent zu **f(1, y = x)**.
2. Mit **%>%** kann man die Klammern weglassen, wenn man eine Funktion ohne weitere Argumente aufruft; **|>** erfordert immer die Klammern.

3. Der Platzhalter `|>` ist absichtlich einfach gehalten und kann viele Eigenschaften des Platzhalters `%>%` nicht wiedergeben. Sie können ihn nicht an mehrere Argumente übergeben, und er hat kein besonderes Verhalten, wenn er innerhalb einer anderen Funktion verwendet wird. Zum Beispiel ist `df %>% split(.$var)` äquivalent zu `split(df, df$var)` und `df %>% {plot(.$x, .$y)}` ist äquivalent zu `plot(df$x, df$y)`.
4. Mit `%>%` können Sie den Punkt `.` auf der linken Seite von Operatoren verwenden, so dass Sie eine einzelne Spalte aus einem Datenframe mit (z. B.) `mtcars %>% .$cyl` extrahieren können. Dies geht mit `|>` nicht.

Glücklicherweise gibt es keinen Grund, sich ganz auf die eine oder andere Pipe festzulegen - Sie können die Basis-Pipe für die meisten Fälle verwenden, und die Magrittr-Pipe benutzen, wenn Sie deren speziellen Eigenschaften wirklich benötigen.

1.6.4 tibbles

Da dieses Konzept der `long table` so essenziell ist, wurde im Tidyverse eine eigene Datenklasse eingeführt, die `tibble` heisst⁶.

Sie entspricht in etwa dem Datenframe, hat aber intern noch weitere wichtige Änderungen eingebaut.

Tibbles werden so wie Datenframes erzeugt, indem man gleichlange Vektoren übergibt. Der Funktionsname ist `tibble()`.

```
1 # aktiviere Tidyverse
2 library(tidyverse)
3
4 # erzeuge ein tibble
5 tibble(a=c(1,2,3), b=c("a", "b", "c"), c=c(T, F, T))
```

```
# A tibble: 3 x 3
  a b      c
<dbl> <chr> <lgl>
1     1 a    TRUE
2     2 b   FALSE
3     3 c    TRUE
```

⁶Das kommt daher, dass `tidy data` immer Tabellen sind, was häufig per `tbl` abgekürzt wird. Liest man im Englisch `tbl` laut und schnell, klingt das wie - genau - *tibble*.

Die Schwesterfunktion `tribble()` (mit *r*) erlaubt diese Schreibweise

```
1 # erzeuge ein tibble mit tribble
2 tribble(
3   ~Text, ~Zahl, ~Logic,
4   "a",    1,    T,
5   "b",    2,    F,
6   "c",    3,    T
7 )
```

```
# A tibble: 3 x 3
  Text   Zahl Logic
<chr> <dbl> <lgl>
1 a         1 TRUE
2 b         2 FALSE
3 c         3 TRUE
```

Sind die übergebenen Reihen nicht gleich lang, werden sie als `list` gespeichert.

```
1 # Datenreihen NICHT gleich lang
2 tribble(
3   ~Text, ~Zahl, ~Logic,
4   "a",    1:3,    T,
5   "b",    4:6,    F,
6   "c",    7:9,    T
7 )
```

```
# A tibble: 3 x 3
  Text   Zahl      Logic
<chr> <list>    <lgl>
1 a    <int [3]> TRUE
2 b    <int [3]> FALSE
3 c    <int [3]> TRUE
```

Die meisten Funktionen des Tidddyverse funktionieren auch mit Datenframes, eben weil sie sich so ähnlich sind. Dennoch ergibt es (schon ideologisch) Sinn, die Datenframes in `tibbles` umzuwandeln.

Schauen wir uns hierfür die Daten der Pflegeberufe als `long table` an:

```
1 # Pflegeframe
2 head(Pflegeframe)
```

```
  Jahr   Berufsgruppe Anzahl
1 1999 Krankenpflegeassistent 16624
2 2001 Krankenpflegeassistent 19061
3 2003 Krankenpflegeassistent 19478
4 2005 Krankenpflegeassistent 21537
5 2007 Krankenpflegeassistent 27731
6 2009 Krankenpflegeassistent 36481
```

```
1 # Übersicht von "Pflegeframe"
2 str(Pflegeframe)
```

```
'data.frame':      45 obs. of  3 variables:
 $ Jahr       : Factor w/  9 levels "1999","2001",...: 1 2 3 4 5 6 7 8 9 1 ...
 $ Berufsgruppe: Factor w/  5 levels "Krankenpflegeassistenz",...: 1 1 1 1 1 1 1 1 1 2 ...
 $ Anzahl      : num  16624 19061 19478 21537 27731 ...
```

Wir erhalten eine Zusammenstellung von wichtigen Informationen. Unser Pflegeframe hat 45 Beobachtungen (Reihen) von 3 Variablen (Spalten). Die Variablen `Jahr` und `Berufsgruppe` liegen als Faktoren vor, `Anzahl` ist numerisch.

```
1 # Datenklasse von "Pflegeframe"
2 class(Pflegeframe)
```

```
[1] "data.frame"
```

Jetzt wandeln wir das Pflegeframe in ein Pflegetibble um.

```
1 # erzeuge Pflegetibble aus Pflegeframe
2 Pflegetibble <- tibble(Pflegeframe)
3
4 # Datenklasse von "Pflegetibble"
5 class(Pflegetibble)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

Wie Sie sehen, ist unser Tibble immernoch ein Datenframe. Es ist aber auch ein “Tibble Datenframe” (`tbl_df`) und einfach ein Tibble (`tbl`).

Die Funktion zum Einsehen von Tibbles heisst `glimpse()` (englisch für *Blick*).

```
1 # werfe einen Blick auf "Pflegetibble"
2 glimpse(Pflegetibble)
```

```
Rows: 45
Columns: 3
 $ Jahr       <fct> 1999, 2001, 2003, 2005, 2007, 2009, 2011, 2013, 2015, 199...
 $ Berufsgruppe <fct> Krankenpflegeassistenz, Krankenpflegeassistenz, Krankenpf...
 $ Anzahl      <dbl> 16624, 19061, 19478, 21537, 27731, 36481, 46517, 54371, 6...
```


Die Ausgabe sieht fast genau so aus wie der `str()`-Aufruf. Neben den Variablennamen steht der Datentyp, wobei `<fct>` für Faktor, `<dbl>` für Dezimalzahlen und `<lgl>` für logical steht.

Im klassischen `R` hat man ein `data.frame` mit `str()` angeschaut, im Tidyverse schaut man mit `glimpse()` auf ein `tibble`.

1.6.5 Funktionsaufrufe

In `R` gibt es jede Menge Zusatzpakete, und obschon die Programmierer sich Mühe geben, einen Funktionsnamen nicht doppelt zu vergeben (vielleicht ist Ihnen schon aufgefallen, dass im Tidyverse viele Funktionen mit einem Unterstrich `_` geschrieben werden, z.B. `read_sav()` oder `as_factor()`), kann dies durchaus vorkommen.

Wenn wir das Tidyverse per `library()` aktivieren, erhalten wir folgende Informationen:

```
1 library(tidyverse)
```

```
> library(tidyverse)
— Attaching packages — tidyverse 1.3.1 —
✓ ggplot2 3.3.5      ✓ purrr  0.3.4
✓ tibble  3.1.6      ✓ dplyr  1.0.7
✓ tidyr   1.1.4      ✓ stringr 1.4.0
✓ readr   2.1.1      ✓ forcats 0.5.1
— Conflicts — tidyverse_conflicts() —
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
>
```

Abb. 1.83: Tidyverse aktivieren

Wie Sie sehen, gibt es nun Konflikte, da die Funktionen `filter()` und `lag()` jeweils in den Paketen `tidyverse` und `stats` (`R`-Basispaket) existieren. Das heisst, möchte man die Funktion `filter()` verwenden, muss man `R` mitteilen, aus welchem Paket sie stammt. Dies erfolgt in `R` indem man den Paketnamen getrennt durch zwei Doppelpunkte vor die Funktion schreibt.

```
1 # nutze Funktion filter() aus dem Paket "dplyr"
2 dplyr::filter()
3
4 # nutze Funktion filter() aus dem "stats" Paket
5 stats::filter()
```

Im Tidyverse ist es daher durchaus üblich, den Paketnamen mit zwei Doppelpunkten getrennt vor den Funktionsnamen zu setzen. So ist immer klar, welche Funktion aus welchem Paket denn nun aufgerufen werden soll.

```
1 # Paketname::Funktionsname
2 haven::read_sav("spss.sav")
```

Durch die Referenzierung des Paketnamens sparen Sie sich das Einbinden des gesamten Paketes per `library()`.

Dies funktioniert auch bei den zahlreichen Datensätzen, die  mitliefert. Möchten Sie den Datensatz `flights` aus dem Paket `nycflights13` nutzen, können Sie (nachdem das Paket `nycflights13` installiert wurde) wie folgt referenzieren:

```
1 # Installiere Paket "nycflights13"
2 install.packages("nycflights13")
3
4 # Paketname::Datensatz
5 head(nycflights13::flights)
```

1.6.6 Schritt 1: Daten importieren

Das Importieren von Daten unterscheidet sich nicht von den Methoden aus Abschnitt 1.2.3.11. Tidyverse bringt mit `readr()` ein eigenes Paket zum Importieren mit, dies ist aber nur für Textdateien wirklich geeignet. Mit den Zusatzpaketen `haven` und `readxl` können Sie wie in 1.2.3.11 beschrieben über die Funktionen `read_sav()`, `read_xlsx()` und `read_csv()` Ihre Daten einlesen.

In RStudio können Sie einfach im Datenfenster auf **Import Dataset** klicken, siehe Seite 89.

1.6.6.1 gelabelte SPSS-Daten

Hier sei noch einmal erwähnt, dass in SPSS kategoriale Daten häufig mit *Labels* versehen werden.

```

1 # Lade Test-SPSS-Datei mit Labels
2 spss <- haven::read_sav(url("https://www.produkt.de/R/alteDaten-lang.sav"))
3 head(spss)

```

```

# A tibble: 6 x 6
      v1      v2      v3      v4      v5 thetruth
  <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl> <dbl+lbl>
1 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
2 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
3 4 [strongly... 4 [strongly... 4 [strongl... NA      NA      4 [strongl...
4 4 [strongly... 3 [agree]      3 [agree]      4 [strongl... 4 [strongl... 4 [strongl...
5 3 [agree]      3 [agree]      2 [disagre... 2 [disagre... 4 [strongl... 4 [strongl...
6 2 [disagree] 3 [agree]      3 [agree]      2 [disagre... 4 [strongl... 4 [strongl...

```

Mit `head()` sehen wir, dass die Werte der Spalten gelabelt sind (4=*strongly agree*, 3=*agree*, usw.). Die Labels stehen in eckigen Klammern neben dem eigentlichen Wert der Variable.

Mit `glimpse()` werden die Labels ignoriert

```

1 # glimpse() zeigt KEINE Labels
2 glimpse(spss)

```

```

Rows: 45,278
Columns: 6
$ v1      <dbl+lbl> 4, 4, 4, 4, 3, 2, 2, 2, 4, 3, 2, 3, 2, 2, 4, 4, 3, 2, ...
$ v2      <dbl+lbl> 4, 4, 4, 3, 3, 3, 1, 3, 4, 3, 2, 2, 2, 2, 4, 3, 1, 2, ...
$ v3      <dbl+lbl> 4, 4, 4, 3, 2, 3, 3, 3, 4, 3, 2, 2, 2, 3, 4, 2, 3, 3, ...
$ v4      <dbl+lbl> NA, NA, NA, 4, 2, 2, 2, 3, NA, 4, 2, 4, 4, 2...
$ v5      <dbl+lbl> NA, NA, NA, 4, 4, 4, 1, 3, NA, 4, 4, 1, 4, 4...
$ thetruth <dbl+lbl> 4, 4, 4, 4, 4, 4, 4, 3, 1, 4, 4, 4, 3, 4...

```

In  ist die Verwendung von Wertelabels eher untypisch. Es empfiehlt sich, die Ausprägungsstufen *so wie sie sind* als Werte einzutragen, also z.B. direkt "männlich" - "weiblich" - "divers" anstatt 0 - 1 - 2 und anschließender Labelung. So werden die Ausprägungen auch auf den Grafiken entsprechend "aussagekräftig" angezeigt.

Wir wandeln alle Variablen in Faktoren um, welche die Labels als Levelnamen nutzen⁷.

```

1 spss %>%
2   mutate(as_factor(.))

```

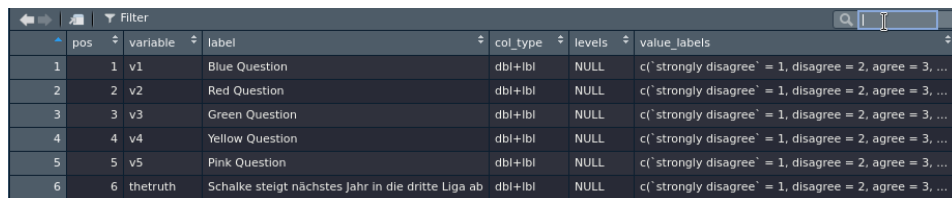
⁷Die Funktion `mutate()` wird später genauer erläutert.

```
# A tibble: 20 x 6
  v1          v2          v3          v4          v5          thetruth
<fct>      <fct>      <fct>      <fct>      <fct>      <fct>
1 strongly ag... strongly ag... strongly a... NA          NA          strongly agr...
2 strongly ag... strongly ag... strongly a... NA          NA          strongly agr...
3 strongly ag... strongly ag... strongly a... NA          NA          strongly agr...
4 strongly ag... agree        agree        strongly ... strongly a... strongly agr...
5 agree        agree        disagree    disagree    strongly a... strongly agr...
6 disagree     agree        agree        disagree    strongly a... strongly agr...
7 disagree     strongly di... agree        disagree    strongly d... strongly agr...
8 disagree     agree        agree        agree        agree        agree
9 strongly ag... strongly ag... strongly a... NA          NA          strongly dis...
10 agree        agree        agree        strongly ... strongly a... strongly agr...
```

Wenn die Daten umfangreich gelabelt sind, kann es hilfreich sein, sich ein "Wörterbuch" der Labels anzulegen. Hierfür nutzt man die Funktion `generate_dictionary()` aus dem `labelled`-Paket.

```
1 dict <- labelled::generate_dictionary(spss)
```

Das Objekt `dict` ist ein Datenframe, in welchem die Labels der Variablen gespeichert sind. In RStudio kann man die Labels so im Datenviewer durchsuchen.



	pos	variable	label	col_type	levels	value_labels
1	1	v1	Blue Question	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)
2	2	v2	Red Question	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)
3	3	v3	Green Question	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)
4	4	v4	Yellow Question	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)
5	5	v5	Pink Question	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)
6	6	thetruth	Schalke steigt nächstes Jahr in die dritte Liga ab	dbl+lbl	NULL	c('strongly disagree' = 1, disagree = 2, agree = 3, ...)

Abb. 1.84: Label-Wörterbuch

Da wir die SPSS-Datei mit den Funktionen des `haven`-Pakets importiert haben, liegen gelabelte Werte und Variablen als Klasse `haven_labelled` vor. Diese Eigenschaft machen wir uns in den nächsten Code-Schnipseln zu Nutze.

Mit diesen Befehlen werden alle Werte der im Datensatz enthaltenen Variablen, die als `haven_labelled` klassifiziert ist, in einen Factor umgewandelt:

```
1 spssfct <- spss %>%
2   mutate_if(haven::is_labelled, haven::as_factor)
3 glimpse(spssfct)
```

```

Rows: 45,278
Columns: 6
$ v1      <fct> strongly agree, strongly agree, strongly agree, strongly agree,...
$ v2      <fct> strongly agree, strongly agree, strongly agree, agree, agree, ...
$ v3      <fct> strongly agree, strongly agree, strongly agree, agree, disagree,...
$ v4      <fct> NA, NA, NA, strongly agree, disagree, disagree, disagree, agree,...
$ v5      <fct> NA, NA, NA, strongly agree, strongly agree, strongly agree, ...

```

Und folgende Befehle wandeln Variablen*labels* in Variablen*namen* um, hierzu muss das Paket `sjlabelled` installiert sein:

```

1 spssn <- spssfct %>%
2   sjlabelled::label_to_colnames()
3 glimpse(spssn)

```

```

Rows: 45,278
Columns: 6
$ `Blue Question`      <fct> strongly agree, st...
$ `Red Question`       <fct> strongly agree, st...
$ `Green Question`     <fct> strongly agree, st...
$ `Yellow Question`    <fct> NA, NA, NA, strong...
$ `Pink Question`      <fct> NA, NA, NA, strong...

```

1.6.7 Schritt 2: Daten *tidy* machen

Der zweite Schritt besteht darin, die Daten *tidy* zu machen, das heisst, sie ins *long table* Format zu überführen. Schauen wir uns hierfür die Matrix der Pflegeberufe an, die ja im *wide* Format vorliegt:

```

1 Pflegeberufe

```

	1999	2001	2003	2005	2007	2009	2011	2013	2015
Krankenpflegeassistentz	16624	19061	19478	21537	27731	36481	46517	54371	64127
Altenpflegehilfe	55770	52710	49727	45776	48326	47903	47978	48363	49507
Kinderkrankenpflege	47779	48203	48822	48519	49080	49307	48291	48937	48913
Krankenpflege	430983	436767	444783	449355	457322	465446	468192	472580	476416
Altenpflege	109161	124879	141965	158817	178902	194195	208304	227154	246412

Die Reihennamen repräsentieren Variablen.

Da im Tidyverse fast alles über `tibbles` bzw. Datenframes abläuft, muss die Matrix zunächst in ein `tibble` überführt werden. Hierfür verwenden wir die Funktion `as_tibble()`

```
1 tbl <- as_tibble(Pflegeberufe)
2 head(tbl)
```

```
# A tibble: 5 x 9
  `1999` `2001` `2003` `2005` `2007` `2009` `2011` `2013` `2015`
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 16624 19061 19478 21537 27731 36481 46517 54371 64127
2 55770 52710 49727 45776 48326 47903 47978 48363 49507
3 47779 48203 48822 48519 49080 49307 48291 48937 48913
4 430983 436767 444783 449355 457322 465446 468192 472580 476416
5 109161 124879 141965 158817 178902 194195 208304 227154 246412
```

Wie Sie sehen, wurden die Spalten (Jahre) als Variablen übernommen. Die Informationen aus den Zeilen (Berufsgruppe) ist verloren gegangen. Für fügen sie daher per `cbind()` unserem `tibble` hinzu.

```
1 tbl <- cbind(tbl, Berufsgruppe = rownames(Pflegeberufe))
2 head(tbl)
```

	1999	2001	2003	2005	2007	2009	2011	2013	2015	Berufsgruppe
1	16624	19061	19478	21537	27731	36481	46517	54371	64127	Krankenpflegeassistenz
2	55770	52710	49727	45776	48326	47903	47978	48363	49507	Altenpflegehilfe
3	47779	48203	48822	48519	49080	49307	48291	48937	48913	Kinderkrankenpflege
4	430983	436767	444783	449355	457322	465446	468192	472580	476416	Krankenpflege
5	109161	124879	141965	158817	178902	194195	208304	227154	246412	Altenpflege

Zur Überführung in ein korrektes tibble vom Format *long table* wird die Funktion `pivot_longer()` aus dem Tidyverse-Paket `tidyr` verwendet. Die Logik der Funktion besteht darin, die Variable mit den ehemaligen Reihennamen (die per `cbind()` hinzugefügt wurde), bestehen zu lassen. Aus den *übrigen* Spalten werden neue Variablen erzeugt, deren Werte passend zu den Reihenwerten (der `cbind()`-Spalte) eingetragen werden. In unserem Beispiel ist die (neue) Variable `Berufsgruppe` in Spalte 10. Wir belassen also Spalte 10, und wenden die Funktion auf die Spalten 1-9 an.

```
1 Pflegeetibble <- pivot_longer(tbl,
2   # wähle die Spalten 1-9 aus
3   cols=1:9,
4   # speichere Spaltennamen als Werte in neuer Variable "Jahr"
5   names_to="Jahr",
6   # Die derzeitigen Werte werden in der neuen Variable "Anzahl" gespeichert
7   values_to = "Anzahl")
8
9 head(Pflegeetibble)
```

	Berufsgruppe	Jahr	Anzahl
	<chr>	<chr>	<dbl>
1	Krankenpflegeassistenz	1999	16624
2	Krankenpflegeassistenz	2001	19061
3	Krankenpflegeassistenz	2003	19478
4	Krankenpflegeassistenz	2005	21537
5	Krankenpflegeassistenz	2007	27731
6	Krankenpflegeassistenz	2009	36481

Mit der Funktion `pivot_wider()` kann im Gegenzug eine *wide table* erstellt werden. Ihr wird übergeben, aus welcher Variable die neuen Spalten gebildet werden sollen, und aus welcher Variable die anzuzeigenden Werte zu nehmen sind. Für unser Pflege-tibble lautet der Befehl:

```
1 pivot_wider(Pflege_tibble, names_from = Jahr, values_from = Anzahl)
```

```
# A tibble: 5 x 10
  Berufsgruppe `1999` `2001` `2003` `2005` `2007` `2009` `2011` `2013` `2015`
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Krankenpflegeassistenz 16624 19061 19478 21537 27731 36481 46517 54371 64127
2 Altenpflegehilfe      55770 52710 49727 45776 48326 47903 47978 48363 49507
3 Kinderkrankenpflege    47779 48203 48822 48519 49080 49307 48291 48937 48913
4 Krankenpflege          430983 436767 444783 449355 457322 465446 468192 472580 476416
5 Altenpflege            109161 124879 141965 158817 178902 194195 208304 227154 246412
```

1.6.7.0.1 verschachtelte wide table

Schauen wir uns folgende Tabelle über ausgedachte Einkommen an:

```
1 untidy_df
```

	Altersgruppe	Maenner_2018	Frauen_2018	Maenner_2019	Frauen_2019	Maenner_2020	Frauen_2020
1	< 18	22000	20000	22000	20000	23000	21000
2	18-30	36000	35000	36000	35000	38000	36000
3	31-50	50000	40000	50000	40000	52000	47000
4	51-60	62000	60000	62000	60000	65000	62000
5	> 60	75000	72000	75000	72000	78000	75000

Diese Tabelle ist mehrfach verschachtelt. Zum einen haben wir “bedeutsame Reihennamen” in Form der **Altersgruppe**. Zum anderen sind auch in den Spaltennamen jeweils 2 Variablen “versteckt”, nämlich das Geschlecht und das Jahr.

Um diese Tabelle *tidy* zu machen, müssen wir in 2 Schritten vor gehen.

Mit der Funktion `pivot_longer()` bringen wir die Daten in eine erste **long table**-Version.

Für den Datensatz `untidy_df` belassen wir die Variable `Altersgruppe` und wenden die Funktion auf die übrigen Spalten an.

```
1 untidy_df %>%
2   pivot_longer(cols = Maenner_2018:Frauen_2020,
3               names_to="Geschlecht_Jahr",
4               values_to = "Einkommen") %>%
5   head(10)
```

```
# A tibble: 10 x 3
  Altersgruppe Geschlecht_Jahr Einkommen
  <chr>         <chr>         <dbl>
1 < 18         Maenner_2018     22000
2 < 18         Frauen_2018      20000
3 < 18         Maenner_2019     22000
4 < 18         Frauen_2019      20000
5 < 18         Maenner_2020     23000
6 < 18         Frauen_2020      21000
7 18-30        Maenner_2018     36000
8 18-30        Frauen_2018      35000
9 18-30        Maenner_2019     36000
10 18-30        Frauen_2019      35000
```

Wie Sie sehen, hat `pivot_longer()` die Altersgruppen ebenfalls richtig zugeordnet.

Im zweiten Schritt teilen wir die Variable `Geschlecht_Jahr` in zwei eigene Variablen `Geschlecht` und `Jahr` auf. Hierfür steht die Funktion `separate()` zur Verfügung.

```
1 untidy_df %>%
2   pivot_longer(cols = Maenner_2018:Frauen_2020,
3               names_to="Geschlecht_Jahr",
4               values_to = "Einkommen") %>%
5   separate(Geschlecht_Jahr,
6           into = c("Geschlecht", "Jahr")) %>%
7   head(10)
```

```
# A tibble: 10 x 4
  Altersgruppe Geschlecht Jahr Einkommen
  <chr>         <chr>   <chr>   <dbl>
1 < 18         Maenner 2018     22000
2 < 18         Frauen  2018     20000
```


3 < 18	Maenner	2019	22000
4 < 18	Frauen	2019	20000
5 < 18	Maenner	2020	23000
6 < 18	Frauen	2020	21000
7 18-30	Maenner	2018	36000
8 18-30	Frauen	2018	35000
9 18-30	Maenner	2019	36000
10 18-30	Frauen	2019	35000

Die Daten liegen nun vollständig als *tidy data* vor.

1.6.7.1 Datentypen korrigieren

Gerade bei importierten Daten sollte überprüft werden, ob den Variablen der korrekte Datentyp (numerisch, faktorial, logisch) zugewiesen ist. Dies ist wichtig, wenn wir mit ihnen später die Statistikfunktionen aufrufen.

Schauen wir unser Pflegeetibble an

```
1 glimpse(Pflegeetibble)
```

```
Rows: 45
Columns: 3
$ Berufsgruppe <chr> "Krankenpflegeassistenz", "Krankenpflegeassistenz", "Krankenpf1
$ Jahr <chr> "1999", "2001", "2003", "2005", "2007", "2009", "2011", "2013",
$ Anzahl <dbl> 16624, 19061, 19478, 21537, 27731, 36481, 46517, 54371, 64127,
```

Die Variablen `Berufsgruppe` und `Jahr` sind beide vom Typ `chr`, was falsch ist. Überführen wir die Berufsgruppen in einen Faktor, und die Jahre in numerische Werte. Im klassischen `R` geht dies so:

```
1 Pflegeetibble$Berufsgruppe <- as.factor(Pflegeetibble$Berufsgruppe)
2 Pflegeetibble$Jahr <- as.numeric(Pflegeetibble$Jahr)
3 glimpse(Pflegeetibble)
```

```
Rows: 45
Columns: 3
$ Berufsgruppe <fct> Krankenpflegeassistenz, Krankenpflegeassistenz, Krankenpflegeas...
$ Jahr <dbl> 1999, 2001, 2003, 2005, 2007, 2009, 2011, 2013, 2015, 1999, 200...
$ Anzahl <dbl> 16624, 19061, 19478, 21537, 27731, 36481, 46517, 54371, 64127, ...
```

Im Tidyverse kann man die Datentypen mit den Funktionen `mutate()` (wird später erleutert) und `across()` umwandeln. Letztere ist das Pendant zum klassischen `apply()` und erlaubt es, Funktionen auf ausgewählte Spalten (Variablen) des Datensatzes anzuwenden.

```

1 Pflgetibble <- Pflgetibble %>%
2   # wende auf Spalte "Berufsgruppe" die Funktion as.factor an.
3   mutate(across(Berufsgruppe, as.factor)) %>%
4   # wende auf Spalte "Jahr" die Funktion as.numeric an.
5   mutate(across(Jahr, as.numeric))
6
7 glimpse(Pflgetibble)

```

```

Rows: 45
Columns: 3
$ Berufsgruppe <fct> Krankenpflegeassistenz, Krankenpflegeassistenz, Krankenpflegeas...
$ Jahr         <dbl> 1999, 2001, 2003, 2005, 2007, 2009, 2011, 2013, 2015, 1999, 200...
$ Anzahl      <dbl> 16624, 19061, 19478, 21537, 27731, 36481, 46517, 54371, 64127, ...

```

Sollen der auszuführenden Funktion Parameter übergeben werden, ändert sich der Aufruf - z.B. um einen ordinalen Faktore zu erzeugen, wie folgt:

```

1 Pflgetibble %>%
2   # wende auf Spalte "Berufsgruppe" die Funktion factor an.
3   # Um Parameter an 'factor()' übergeben zu können,
4   # muss folgende Notation erfolgen.
5   # Erstelle ordinale Faktoren mit Parameter "ordered=T"
6   mutate(across(Berufsgruppe, ~factor(.x, ordered=T))) %>%
7   head()

```

```

# A tibble: 6 × 3
  Berufsgruppe      Jahr Anzahl
  <ord>          <dbl>   <dbl>
1 Krankenpflegeassistenz 1999 16624
2 Krankenpflegeassistenz 2001 19061
3 Krankenpflegeassistenz 2003 19478
4 Krankenpflegeassistenz 2005 21537
5 Krankenpflegeassistenz 2007 27731
6 Krankenpflegeassistenz 2009 36481

```

Schauen wir uns als weiteres Beispiel den gelabelten Datensatz der Nachtwachenstudie an.

```

1 # lade den Nachtwachendatensatz
2 load(url("https://www.produnis.de/R/nw.RData"))
3 #nutze die gelabelte Version
4 test <- nw_labelled
5
6 # wenn Sie das Paket jgsbook installiert haben,
7 # können Sie den Datensatz auch so laden:
8 test <- jgsbook::nw_labelled
9 glimpse(test)

```

```

Rows: 276
Columns: 49
$ sex                <fct> männlich, weiblich, weiblich, weiblich, weiblich...
$ age                <labelled> 56, 53, 55, 28, 53, 62, 55, 61, 34, 50, 54,...
$ land               <fct> Nordrhein-Westfalen, Nordrhein-Westfalen, Nordrh...
$ host               <fct> "privater Träger", "Wohlfahrtsverband (z.B. AWO,...
$ bed                <fct> weniger als 10, 75 - 100, 100 - 150, 100 - 150, ...
$ patient            <fct> 10 - 20, 75 - 100, 100 - 150, 100 - 150, 75 - 10...
$ liability           <labelled> 5, 30, 34, 23, 30, 41, 79, 79, 52, 32, 71, ...
$ Stufe0             <labelled> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 25, 25, 0, 0,...
$ Stufe1             <labelled> 3, 0, 10, 4, 0, 14, 15, 8, 6, 9, 15, 15, 9,...
$ Stufe2             <labelled> 9, 10, 19, 17, 11, 19, 29, 14, 32, 14, 19, ...
$ Stufe3             <labelled> 12, 20, 5, 2, 19, 7, 23, 18, 14, 10, 12, 12...
$ Demenz             <labelled> 7, 20, 16, 8, 28, 7, 60, 31, 32, 22, 21, 21...
$ Bettgitter         <labelled> 2, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1...
$ Bettgurt           <labelled> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Schlafmittel       <labelled> 7, 6, 5, 0, 5, 9, 7, 2, 15, 4, 4, 4, 4, 0, ...
$ colleagueall       <labelled> 2, 3, 5, 3, 3, 3, 2, 2, 2, 3, 3, 3, 3, 2, 2...
$ colleague          <labelled> 1, 1, 1, 0, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 2...
$ pause             <fct> selten, immer, häufig, immer, immer, manchmal, i...
$ pausecolleague     <fct> nie, immer, nie, immer, immer, nie, immer, immer...
$ hintergrunddienst <fct> NA, Ja, Ja, Ja, Ja, NA, NA, Ja, Ja, Ja, Ja, Ja, ...
$ daynights          <fct> "hauptsächlich im Nachtdienst, mit vereinzelten ...
$ whynight           <fct> NA, die Arbeitszeiten sagen mir zu, NA, familiär...
$ nights             <labelled> 6, 7, 8, 5, 7, 7, 7, 7, 5, 5, 7, 5, 9, 9, 8...
$ timework           <fct> Vollzeit, Teilzeit, NA, Teilzeit, Teilzeit, NA, ...
$ nightstarts        <labelled> "1970-01-01 20:33:00", "1970-01-01 20:30:00...
$ nightstops         <labelled> "1970-01-01 07:07:00", "1970-01-01 07:00:00...
$ MagNacht           <fct> eher ja, ja, ja, ja, ja, ja, ja, ja, ja, ja, ja, ...
$ WiederTag          <fct> neutral, nein, nein, neutral, nein, nein, nein, ...
$ ueberfordert       <fct> selten, nie, nie, nie, selten, selten, nie, selt...
$ Zwischenfall       <fct> selten, selten, häufig, selten, häufig, selten, ...
$ Bew_herumirren     <fct> sehr oft, häufig, selten, häufig, häufig, selten...
$ Bew_sterben        <fct> selten, häufig, häufig, selten, häufig, selten, ...
$ muede             <fct> selten, nie, nie, häufig, selten, nie, nie, selt...
$ schwer_wach        <fct> nie, nie, nie, selten, selten, nie, nie, nie, ni...
$ Demenz_aktiv       <labelled> 10, 7, 5, 5, 7, 7, 3, 5, NA, 5, 5, 5, 4, 4,...
$ Demenz_stoeren     <labelled> 8, 7, 3, 3, 7, 0, 3, 2, NA, 5, 5, 5, 2, 1, ...
$ Demenz_sediert     <labelled> 3, 7, 1, 0, 2, 0, 0, 1, NA, 2, 2, 2, 0, 0, ...
$ Demenz_nv          <labelled> 10, 4, 4, 0, 1, 7, 0, 0, NA, 0, 0, 0, 0, 0,...
$ Sorge_weglaufen    <fct> eher ja, ja, eher ja, nein, eher nicht, eher nic...
$ Sorge_Sturz        <fct> ja, nein, ja, ja, eher ja, eher nicht, neutral, ...
$ Sorge_FEM          <fct> ja, NA, NA, nein, neutral, eher ja, NA, neutral,...
$ Sorge_nv           <fct> ja, neutral, eher ja, eher nicht, nein, eher nic...
$ Sorge_sterben      <fct> eher ja, ja, ja, eher nicht, eher ja, ja, eher n...
$ workyear           <labelled> 35, 37, 37, 10, 30, 34, 13, 28, 4, 32, 10, ...
$ worknight          <labelled> 12, 11, 32, 1, 14, 29, 7, 13, 2, 5, 10, 5, ...
$ meeting            <fct> selten, immer, NA, nie, immer, manchmal, selten,...
$ fortbildung         <fct> Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, ...
$ wannfort           <labelled> 2004, 2014, 2013, 2013, 2014, 2015, 2014, 2...
$ edu                <fct> dreijährige Pflegeausbildung, zweijährige Pflege...

```

Wir sehen, dass viele Variablen vom Typ `labelled` sind. Schauen wir uns die

Variablen `age` und `sex` genauer an.

```
1 class(test$age)
```

```
[1] "labelled" "numeric"
```

```
1 class(test$sex)
```

```
[1] "labelled" "factor"
```

Die Variablen ist sowohl vom Typ `labelled` als auch vom Typ `numeric` bzw. `factor`. Manche -Funktionen haben mit dieser Doppelbelegung ihre Probleme, z.B. `vtable::sumtable()`.

```
1 vtable::sumtable(test$age)
```

```
Fehler in vtable::sumtable(test$age) :  
It doesn't look like you have any variables that belong in a sumtable.  
Check your data. Use vars to explicitly choose variables, or convert  
things to numeric or factor before sending to sumtable.
```

Wir sollten daher alle Variablentypen eindeutig setzen. Die Funktion `mutate_if()` wendet eine Funktion an, wenn die gesetzten Bedingungen erfüllt sind. So möchten wir im ersten Schritt nur solche Spalten, die (zusätzlich zum Typ `labelled`) vom Typ `numeric` sind, den eindeutigen Typ `numeric` zuweisen. Im zweiten Schritt wiederholen wir dies für Faktoren.

```
1 test %>%  
2   # wende nur auf labelled-numeric an  
3   mutate_if(is.numeric, as.numeric) %>%  
4   # wende nur auf labelled-factor an  
5   mutate_if(is.factor, as.factor) %>%  
6   # wende nur auf die Spalten nightstarts nightstops an  
7   mutate(across(nightstarts:nightstops, as.character)) %>%  
8   glimpse()
```

```
Rows: 276  
Columns: 49  
$ sex      <fct> männlich, weiblich, weiblich, weiblich, weiblich...  
$ age      <dbl> 56, 53, 55, 28, 53, 62, 55, 61, 34, 50, 54, 50, ...  
$ land     <fct> Nordrhein-Westfalen, Nordrhein-Westfalen, Nordrh...  
$ host     <fct> "privater Träger", "Wohlfahrtsverband (z.B. AWO,...  
$ bed      <fct> weniger als 10, 75 - 100, 100 - 150, 100 - 150, ...
```

```

$ patient      <fct> 10 - 20, 75 - 100, 100 - 150, 100 - 150, 75 - 10...
$ liability    <dbl> 5, 30, 34, 23, 30, 41, 79, 79, 52, 32, 71, 71, 4...
$ Stufe0      <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 25, 25, 0, 0, 0, 0...
$ Stufe1      <dbl> 3, 0, 10, 4, 0, 14, 15, 8, 6, 9, 15, 15, 9, 10, ...
$ Stufe2      <dbl> 9, 10, 19, 17, 11, 19, 29, 14, 32, 14, 19, 19, 2...
$ Stufe3      <dbl> 12, 20, 5, 2, 19, 7, 23, 18, 14, 10, 12, 12, 14,...
$ Demenz      <dbl> 7, 20, 16, 8, 28, 7, 60, 31, 32, 22, 21, 21, 20,...
$ Bettgitter  <dbl> 2, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 11, 1,...
$ Bettgurt    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
$ Schlafmittel <dbl> 7, 6, 5, 0, 5, 9, 7, 2, 15, 4, 4, 4, 4, 0, 26, 1...
$ colleagueall <dbl> 2, 3, 5, 3, 3, 3, 2, 2, 2, 3, 3, 3, 3, 2, 2, 6, ...
$ colleague   <dbl> 1, 1, 1, 0, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 2, 2, ...
$ pause       <fct> selten, immer, häufig, immer, immer, manchmal, i...
$ pausecolleague <fct> nie, immer, nie, immer, immer, nie, immer, immer...
$ hintergrunddienst <fct> NA, Ja, Ja, Ja, Ja, NA, NA, Ja, Ja, Ja, Ja, ...
$ daynights    <fct> "hauptsächlich im Nachtdienst, mit vereinzelten ..."
$ whynight     <fct> NA, die Arbeitszeiten sagen mir zu, NA, familiär...
$ nights       <dbl> 6, 7, 8, 5, 7, 7, 7, 7, 5, 5, 7, 5, 9, 9, 8, 8, ...
$ timework     <fct> Vollzeit, Teilzeit, NA, Teilzeit, Teilzeit, NA, ...
$ nightstarts  <chr> "1970-01-01 20:33:00", "1970-01-01 20:30:00", "1...
$ nightstops   <chr> "1970-01-01 07:07:00", "1970-01-01 07:00:00", "1...
$ MagNacht     <fct> eher ja, ja, ja, ja, ja, ja, ja, ja, ja, ja, ja, ...
$ WiederTag    <fct> neutral, nein, nein, neutral, nein, nein, nein, ...
$ ueberfordert <fct> selten, nie, nie, nie, selten, selten, nie, selt...
$ Zwischenfall <fct> selten, selten, häufig, selten, häufig, selten, ...
$ Bew_herumirren <fct> sehr oft, häufig, selten, häufig, häufig, selten...
$ Bew_sterben  <fct> selten, häufig, häufig, selten, häufig, selten, ...
$ muede        <fct> selten, nie, nie, häufig, selten, nie, nie, selt...
$ schwer_wach  <fct> nie, nie, nie, selten, selten, nie, nie, nie, ni...
$ Demenz_aktiv <dbl> 10, 7, 5, 5, 7, 7, 3, 5, NA, 5, 5, 5, 4, 4, 7, 7...
$ Demenz_stoeren <dbl> 8, 7, 3, 3, 7, 0, 3, 2, NA, 5, 5, 5, 2, 1, 7, 7,...
$ Demenz_sediert <dbl> 3, 7, 1, 0, 2, 0, 0, 1, NA, 2, 2, 2, 0, 0, 5, 7,...
$ Demenz_nv    <dbl> 10, 4, 4, 0, 1, 7, 0, 0, NA, 0, 0, 0, 0, 0, 7, 7...
$ Sorge_weglaufen <fct> eher ja, ja, eher ja, nein, eher nicht, eher nic...
$ Sorge_Sturz  <fct> ja, nein, ja, ja, eher ja, eher nicht, neutral, ...
$ Sorge_FEM    <fct> ja, NA, NA, nein, neutral, eher ja, NA, neutral,...
$ Sorge_nv     <fct> ja, neutral, eher ja, eher nicht, nein, eher nic...
$ Sorge_sterben <fct> eher ja, ja, ja, eher nicht, eher ja, ja, eher n...
$ workyear     <dbl> 35, 37, 37, 10, 30, 34, 13, 28, 4, 32, 10, 32, 2...
$ worknight    <dbl> 12, 11, 32, 1, 14, 29, 7, 13, 2, 5, 10, 5, 17, 3...
$ meeting      <fct> selten, immer, NA, nie, immer, manchmal, selten,...
$ fortbildung  <fct> Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, Ja, ...
$ wannfort     <dbl> 2004, 2014, 2013, 2013, 2014, 2015, 2014, 2015, ...
$ edu          <fct> dreijährige Pflegeausbildung, zweijährige Pflege...

```

1.6.7.2 fehlende Werte

Fehlende Werte (NA) können mit der Funktion `drop_na()` entfernt werden. Die Funktion “dropt” dabei die komplette Datenreihe, wenn ein NA in einer angegebenen Spalte enthalten ist.

```

1 # lade Beispieldatensatz
2 load(url("https://www.produnis.de/R/pf8.RData" ))
3
4 # Schaue Variable "Gewicht" an
5 glimpse(pf8$Gewicht)

```

```
num [1:731] 69 67 NA 90 68 60 80 60 NA 60 ...
```

Variable **Gewicht** enthält 731 Werte, und wir sehen, dass auch NA enthalten sind.

Jetzt "dropen" wir alle Reihen, in denen bei der Variable **Gewicht** ein NA steht

```

1 pf8 %>%
2   drop_na(Gewicht) %>%
3   glimpse(.)

```

```

Rows: 720
Columns: 16
$ Standort      <fct> Münster, Münster, Münster, Münster, Münster, Münster, Münster...
$ Alter         <int> 18, 67, 61, 24, 21, 59, 56, 52, 79, 22, 19, 18, 17, 21, 28, 7...
$ Geschlecht    <fct> weiblich, weiblich, männlich, männlich, weiblich, weiblich, w...
$ Größe         <int> 172, 165, 182, 173, 177, 168, 156, 166, 161, 206, 163, 172, 1...
$ Gewicht       <dbl> 69, 67, 90, 68, 60, 80, 60, 60, 66, 130, 52, 68, 48, 54, 70, ...
$ Bildung       <fct> Abitur, mittlere Reife, mittlere Reife, Abitur, Abitur, mittl...
$ Beruf         <fct> Inspektor*in, Rentner*in, Beamter*in, Student*in, Student*in,...
$ Familienstand <fct> Partnerschaft, geschieden, ledig, ledig, Partnerschaft, verhe...
$ Kinder        <int> 0, 0, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Wohnort       <fct> ländlich, ländlich, ländlich, städtisch, städtisch, ländlich,...
$ Rauchen       <fct> nein, nein, nein, nein, ja, nein, ja, nein, nein, nein, nein,...
$ SportHäufig   <dbl> NA, 2.0, 4.0, 4.0, 1.0, 2.0, 1.0, 2.0, NA, 4.0, 5.0, 2.5, 1.0...
$ SportMinuten  <dbl> NA, 60, 120, 60, 60, 45, 90, 45, NA, 90, 20, 120, 120, 80, NA...
$ SportWie      <fct> Allein, Gruppe, Allein, Allein, Gruppe, Gruppe, beides, Allei...
$ SportWarum    <fct> Arbeitsweg, Vorbeugung, Fitness, Gewichtsreduktion, Fitness, ...
$ LebenZufrieden <dbl> 5, 7, 2, 9, 8, 5, 8, 8, 8, 8, 7, 8, 7, 8, 7, 7, 8, 7, 10, 8, ...

```

Der Datensatz **pf8** wird mittels **Pipe** weitergeleitet. In der zweiten Zeile dropen wir alle Reihen mit **Gewicht** NA und geben das Resultat per Pipe weiter. Mit **glimpse()** sehen wir das Resultat. Datensatz **pf8** hat jetzt nur noch 720 Reihen, und nicht mehr 731.

Sollen *alle* NAs in allen Spalten entfernt werden, lautet der Befehl

```

1 pf8 %>%
2   drop_na(.) %>%
3   glimpse(.)

```

```

Rows: 398
Columns: 16
$ Standort      <fct> Münster, Münster, Münster, Münster, Münster, Münster, Münster...
$ Alter         <int> 67, 61, 24, 21, 59, 56, 52, 22, 19, 18, 17, 21, 72, 19, 69, 2...
$ Geschlecht    <fct> weiblich, männlich, männlich, weiblich, weiblich, weiblich, w...
$ Größe         <int> 165, 182, 173, 177, 168, 156, 166, 206, 163, 172, 158, 163, 1...
$ Gewicht       <dbl> 67, 90, 68, 60, 80, 60, 60, 130, 52, 68, 48, 54, 86, 65, 89, ...
$ Bildung       <fct> mittlere Reife, mittlere Reife, Abitur, Abitur, mittlere Reif...
$ Beruf         <fct> Rentner*in, Beamter*in, Student*in, Student*in, arbeitslos, Pf...
$ Familienstand <fct> geschieden, ledig, ledig, Partnerschaft, verheiratet, geschie...
$ Kinder        <int> 0, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Wohnort       <fct> ländlich, ländlich, städtisch, städtisch, ländlich, städtisch...
$ Rauchen       <fct> nein, nein, nein, ja, nein, ja, nein, nein, nein, nein, nein,...
$ SportHäufig   <dbl> 2.0, 4.0, 4.0, 1.0, 2.0, 1.0, 2.0, 4.0, 5.0, 2.5, 1.0, 4.0, 4...
$ SportMinuten  <dbl> 60, 120, 60, 60, 45, 90, 45, 90, 20, 120, 120, 80, 160, 120, ...
$ SportWie      <fct> Gruppe, Allein, Allein, Gruppe, Gruppe, beides, Allein, Grupp...
$ SportWarum    <fct> Vorbeugung, Fitness, Gewichtsreduktion, Fitness, Vorbeugung, ...
$ LebenZufrieden <dbl> 7, 2, 9, 8, 5, 8, 8, 8, 7, 8, 7, 8, 7, 8, 10, 8, 7, 9, 8, 8, ...

```

Mit dieser radikalen Methode schrumpft der Datensatz pf8 auf 398 vollständige Datenreihen zusammen.

Sie können mit den Funktionen `fill()` und `replace_na()` fehlende Werte auffüllen oder ersetzen. Hierbei ersetzt `replace_na()` alle NA durch einen vorgegebenen Wert, und `fill()` versucht, die NA durch “wahrscheinliche” Werte zu ersetzen. Dies ist hilfreich, wenn die Daten z.B. nach einem Kriterium sortiert sind.

```

1 test <- tibble(Beruf = factor(c("Bäcker", NA, "Bäcker", "Ärztin", NA, "Ärztin")))
2
3 # vorher
4 test

```

```

## # A tibble: 6 x 1
##   Beruf
##   <fct>
## 1 Bäcker
## 2 <NA>
## 3 Bäcker
## 4 Ärztin
## 5 <NA>
## 6 Ärztin

```

```

1 # ersetze NA
2 test %>%
3   tidyr::fill(Beruf)

```

```
## # A tibble: 6 x 1
##   Beruf
##   <fct>
## 1 Bäcker
## 2 Bäcker
## 3 Bäcker
## 4 Ärztin
## 5 Ärztin
## 6 Ärztin
```

Aufsteigende Werte können **nicht** erzeugt werden.

```
1 test <- tibble(Jahr = c(2014, NA, NA, NA, 2018, 2019, NA , NA , 2022))
2
3 # vorher
4 test
```

```
## # A tibble: 9 x 1
##   Jahr
##   <dbl>
## 1  2014
## 2    NA
## 3    NA
## 4    NA
## 5  2018
## 6  2019
## 7    NA
## 8    NA
## 9  2022
```

```
1 test %>%
2   tidyr::fill(Jahr)
```

```
## # A tibble: 9 x 1
##   Jahr
##   <dbl>
## 1  2014
## 2  2014
## 3  2014
## 4  2014
## 5  2018
## 6  2019
## 7  2019
## 8  2019
## 9  2022
```


1.6.7.3 Variablen umbenennen

Mit der Funktion `rename()` können Sie die Variablennamen ändern.

```
1 Pflegetibble %>%
2   rename(Neu=Berufsgruppe) %>%
3   head()
```

```
# A tibble: 6 x 3
  Neu      Jahr Anzahl
<fct>   <dbl> <dbl>
1 Krankenpflegeassistenz 1999 16624
2 Krankenpflegeassistenz 2001 19061
3 Krankenpflegeassistenz 2003 19478
4 Krankenpflegeassistenz 2005 21537
5 Krankenpflegeassistenz 2007 27731
6 Krankenpflegeassistenz 2009 36481
```

```
1 Pflegetibble %>%
2   rename(graduate=Berufsgruppe,
3         year=Jahr,
4         n=Anzahl) %>%
5   head()
```

```
# A tibble: 6 x 3
  graduate      year      n
<fct>       <dbl> <dbl>
1 Krankenpflegeassistenz 1999 16624
2 Krankenpflegeassistenz 2001 19061
3 Krankenpflegeassistenz 2003 19478
4 Krankenpflegeassistenz 2005 21537
5 Krankenpflegeassistenz 2007 27731
6 Krankenpflegeassistenz 2009 36481
```

1.6.8 Schritt 3: Umgang mit Datensätzen

Das Paket `dplyr` bietet zahlreiche Funktionen, die den Umgang mit Datensätzen erleichtern.

1.6.8.1 Daten filtern und sortieren

Mit der Funktion `filter()` kann der Datensatz nach Kriterien gefiltert werden. Beachten Sie, dass die Funktion mit `stats::filter()` kollidiert. Rufen Sie sie daher sicherheitshalber mit `dplyr::filter()` auf.

```
1 pf8 %>%
2   # Nur Daten von Personen jünger 40
3   dplyr::filter(Alter<40) %>%
4   glimpse()
```

```
## Rows: 443
## Columns: 16
## $ Standort      <fct> Münster, Münster, Münster, Münster, Münster, Münster, M...
## $ Alter          <int> 18, 24, 21, 22, 19, 18, 17, 21, 28, 19, 21, 28, 26, 21,...
## $ Geschlecht     <fct> weiblich, männlich, weiblich, männlich, weiblich, weibl...
## $ Größe          <int> 172, 173, 177, 206, 163, 172, 158, 163, 181, 175, 186, ...
## $ Gewicht        <dbl> 69, 68, 60, 130, 52, 68, 48, 54, 70, 65, 70, 85, 60, 65...
## $ Bildung        <fct> Abitur, Abitur, Abitur, Fachabitur, Abitur, mittlere Re...
## $ Beruf          <fct> Inspektor*in, Student*in, Student*in, Kaufmann/frau, St...
## $ Familienstand  <fct> Partnerschaft, ledig, Partnerschaft, ledig, ledig, ledi...
## $ Kinder         <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
## $ Wohnort        <fct> ländlich, städtisch, städtisch, ländlich, städtisch, st...
## $ Rauchen        <fct> nein, nein, ja, nein, nein, nein, nein, nein, nein, nei...
## $ SportHäufig    <dbl> NA, 4.0, 1.0, 4.0, 5.0, 2.5, 1.0, 4.0, NA, 0.5, 1.0, 5...
## $ SportMinuten   <dbl> NA, 60, 60, 90, 20, 120, 120, 80, NA, 120, 60, 50, 30, ...
## $ SportWie       <fct> Allein, Allein, Gruppe, Gruppe, Allein, Gruppe, Gruppe,...
## $ SportWarum     <fct> Arbeitsweg, Gewichtsreduktion, Fitness, Fitness, Vorbeu...
## $ LebenZufrieden <dbl> 5, 9, 8, 8, 7, 8, 7, 8, 7, 8, 7, 8, 7, 9, 8, 8, 6, 7, 8...
```

Die Kriterien können auch kombiniert werden

```
1 pf8 %>%
2   dplyr::filter(Alter>20 , Geschlecht == "weiblich", Bildung=="Abitur") %>%
3   glimpse()
```

```
## Rows: 56
## Columns: 16
## $ Standort      <fct> Münster, Münster, Münster, Bahn, Bahn, Rheine, Rheine, ...
## $ Alter          <int> 21, 21, 21, 21, 21, 22, 23, 22, 30, 52, 45, 23, 49, 41,...
## $ Geschlecht     <fct> weiblich, weiblich, weiblich, weiblich, weiblich, weibl...
## $ Größe          <int> 177, 163, 175, 170, 170, 170, 168, 163, 173, 174, 168, ...
## $ Gewicht        <dbl> 60, 54, 61, 58, 80, 59, 70, 63, 71, 72, 70, 56, 70, 49,...
## $ Bildung        <fct> Abitur, Abitur, Abitur, Abitur, Abitur, Abitur, Abitur,...
## $ Beruf          <fct> Student*in, Student*in, Student*in, Student*in, Student...
## $ Familienstand  <fct> Partnerschaft, Partnerschaft, Partnerschaft, NA, ledig,...
## $ Kinder         <int> 0, 0, 0, NA, 0, 0, 0, 0, 0, 1, 2, 0, 2, 0, 3, 2, 0, 1, ...
## $ Wohnort        <fct> städtisch, städtisch, städtisch, ländlich, ländlich, st...
## $ Rauchen        <fct> ja, nein, nein, nein, nein, nein, nein, nein, nein, nei...
```

```
## $ SportHäufig <dbl> 1.0, 4.0, 3.0, 5.0, 1.0, 1.0, 2.0, 4.0, 3.0, 4.0, 1.0, ...
## $ SportMinuten <dbl> 60, 80, 60, 75, 60, 90, 45, 60, 60, 20, 60, 90, 30, 120...
## $ SportWie <fct> Gruppe, beides, Gruppe, Allein, Allein, Allein, beides,...
## $ SportWarum <fct> Fitness, Vorbeugung, Fitness, Fitness, Fitness, Gewicht...
## $ LebenZufrieden <dbl> 8, 8, 8, 9, 8, 8, 10, 8, 7, 6, 7, 8, 7, 7, 7, 8, 8, ...
```

Beachten Sie, dass wir doppelte Gleichheitszeichen (==) verwenden müssen, da mit *einem* Gleichheitszeichen (=) Variablen zugewiesen werden (so wie mit <-).

Möchte man nach mehreren bestimmten Werte filtern, kann der `%in%` Operator verwendet werden. Hierdurch übergeben Sie einen Vektor an möglichen Ausprägungen. Angenommen, Sie möchten nur solche Fälle auswählen, in denen das Alter 20 *oder* 25 *oder* 40 Jahre beträgt, lautet der Aufruf:

```
1 # nach mehreren Werten filtern
2 # Suche Fälle, die 20 oder 25 oder 40 Jahre alt sind
3 pf8 %>%
4   dplyr::filter(Alter %in% c(20, 25, 40)) %>%
5   head(10)
```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand
1	Münster	25	männlich	182	75.0	Ausbildung	Finanzwirt	ledig
2	Bahn	25	männlich	180	72.0	Hochschule	etriebswirt	ledig
3	Rheine	25	weiblich	168	75.0	Hochschule	Student*in	ledig
4	Rheine	40	weiblich	163	68.0	Hauptschule	Angestellte*r	ledig
5	Rheine	20	weiblich	158	70.0	Hauptschule	Schüler*in	ledig
6	Rheine	20	weiblich	170	85.0	Hochschule	Azubi	ledig
7	Rheine	20	weiblich	170	65.0	Abitur	Azubi	ledig
8	Rheine	20	weiblich	160	48.5	Abitur	Student*in	ledig
9	Rheine	25	männlich	186	70.0	Hochschule	Student*in	ledig
10	Bahn	40	weiblich	181	64.0	Hochschule	gestellte*r	verheiratet

Über den Operator `!=` können Fälle ausgeschlossen werden. Wir schließen für dieses Beispiel alle Fälle aus, die verheiratet sind:

```
1 # schließe alle verheirateten Fälle aus
2 pf8 %>%
3   dplyr::filter(Familienstand != "verheiratet") %>%
4   head(10)
```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand
1	Münster	18	weiblich	172	69	Abitur	Inspektor*in	Partnerschaft
2	Münster	67	weiblich	165	67	mittlere Reife	Rentner*in	geschieden
3	Münster	60	weiblich	175	NA	Hochschule	Ergotherapeut*in	Partnerschaft
4	Münster	61	männlich	182	90	mittlere Reife	Beamter*in	ledig
5	Münster	24	männlich	173	68	Abitur	Student*in	ledig
6	Münster	21	weiblich	177	60	Abitur	Student*in	Partnerschaft
7	Münster	56	weiblich	156	60	Ausbildung	Pflegehelfer*in	geschieden
8	Münster	22	männlich	206	130	Fachabitur	Kaufmann/frau	ledig
9	Münster	19	weiblich	163	52	Abitur	Student*in	ledig
10	Münster	18	weiblich	172	68	mittlere Reife	Schüler*in	ledig

Mit der Funktion `arrange()` können die Daten sortiert werden. Dabei wird standardmäßig *aufsteigend* sortiert.

```

1 # sortiere Datensatz "pf8"
2 pf8 %>%
3   arrange(Alter, Größe, Gewicht) %>%
4   head(.)

```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand
1	Internet	15	weiblich	165	51.6	<NA>		Partnerschaft
2	Münster	17	weiblich	158	48.0	mittlere Reife	Schüler*in	ledig
3	Internet	17	weiblich	160	75.0	<NA>		Partnerschaft
4	Internet	17	weiblich	163	57.0	<NA>		Partnerschaft
5	Internet	17	weiblich	168	59.0	<NA>		ledig
6	Internet	17	weiblich	175	80.0	<NA>		ledig

Die Daten werden zunächst aufsteigend nach `Alter` sortiert. Gibt es Fälle mit dem selben `Alter`, wird nach dem zweiten Kriterium (in diesem Falle `Größe`) sortiert. Gibt es Fälle mit der selben `Größe` und dem selben `Alter`, wird innerhalb dieser Gruppe nach `Gewicht` sortiert.

Für eine *absteigende* Sortierung muss die Funktion `desc()` verwendet werden

```

1 # sortiere Datensatz "pf8"
2 # diesmal absteigend
3 pf8 %>%
4   arrange(desc(Alter), desc(Größe), desc(Gewicht)) %>%
5   head(.)

```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand
1	Münster	86	weiblich	162	70	Hauptschule	Rentner*in	verwitwet
2	Rheine	86	weiblich	160	60	Abitur	Postbote*in	verwitwet
3	Rheine	84	weiblich	164	70	Hauptschule	Verkäufer*in	verwitwet
4	Rheine	84	weiblich	160	60	Hauptschule	Rentner*in	verwitwet

5	Ladbergen	83	männlich	182	88	Hauptschule	Rentener*in	verheiratet
6	Münster	83	<NA>	165	175	<NA>	Rentner*in	ledig

1.6.8.2 Fälle auswählen

Mit der Funktion `slice()` können gewünschte Datenreihen (Fälle) ausgegeben werden

```
1 pf8 %>%
2   # zeige Fälle (Reihen) 98 bis 105
3   slice(98:105) %>%
4   glimpse()
```

```
## Rows: 8
## Columns: 16
## $ Standort      <fct> Rheine, Rheine, Rheine, Rheine, Rheine, Rheine, Rheine,...
## $ Alter          <int> 23, 30, 26, 35, 22, 50, 35, 22
## $ Geschlecht     <fct> männlich, weiblich, männlich, männlich, männlich, männl...
## $ Größe          <int> 194, 173, 189, 184, 181, 204, 172, 175
## $ Gewicht        <dbl> 110, 71, 89, 79, 92, 102, 86, 75
## $ Bildung        <fct> Abitur, Abitur, Hauptschule, mittlere Reife, mittlere R...
## $ Beruf          <fct> Außendienst, Innendienst, Angestellte*r, Produktionsle...
## $ Familienstand  <fct> Partnerschaft, verheiratet, Partnerschaft, verheiratet,...
## $ Kinder         <int> 0, 0, 0, 3, 0, 1, 2, 0
## $ Wohnort        <fct> ländlich, ländlich, städtisch, ländlich, ländlich, länd...
## $ Rauchen        <fct> ja, nein, ja, ja, nein, nein, ja, nein
## $ SportHäufig    <dbl> 1, 3, 1, 3, 2, 4, 1, 0
## $ SportMinuten   <dbl> 30, 60, 60, 90, 30, 30, 60, 0
## $ SportWie       <fct> Allein, Allein, Gruppe, Gruppe, Gruppe, beides, Gruppe,...
## $ SportWarum     <fct> Fitness, Fitness, Freizeit, Fitness, Fitness, Vorbeugun...
## $ LebenZufrieden <dbl> 9, 7, 6, 6, 7, 8, 7, 7
```

Mit `slice_min()` und `slice_max()` können die kleinsten und größten Werte eingesehen werden.

```
1 pf8 %>%
2   # zeige die leichtesten Gewichte
3   slice_min(Gewicht)%>%
4   glimpse()
```

```
## Rows: 5
## Columns: 16
## $ Standort      <fct> Münster, Münster, Münster, Internet, Internet
## $ Alter          <int> 26, 24, 24, 24, 19
## $ Geschlecht     <fct> männlich, weiblich, weiblich, weiblich, weiblich
## $ Größe          <int> 180, 164, 160, 159, 155
```

```
## $ Gewicht      <dbl> 45, 45, 45, 45, 45
## $ Bildung      <fct> Hauptschule, Abitur, Abitur, NA, NA
## $ Beruf        <fct> Kaufmann/frau, Student*in, Student*in, ,
## $ Familienstand <fct> ledig, Partnerschaft, Partnerschaft, Partnerschaft, led...
## $ Kinder       <int> 0, 0, 0, 0, 0
## $ Wohnort      <fct> ländlich, städtisch, städtisch, ländlich, städtisch
## $ Rauchen      <fct> ja, nein, nein, nein, nein
## $ SportHäufig  <dbl> NA, 2.0, 3.0, 4.0, 2.5
## $ SportMinuten <dbl> NA, 60, 60, 60, 90
## $ SportWie     <fct> NA, Gruppe, Allein, Allein, Allein
## $ SportWarum   <fct> NA, Vorbeugung, Vorbeugung, NA, NA
## $ LebenZufrieden <dbl> 10, 8, 9, 10, 9
```

Die Tabelle wird nach **Gewicht** sortiert ausgegeben. Im Datensatz **pf8** haben insgesamt 5 Fälle das kleinste **Gewicht**.

Der Parameter **n** gibt an, wieviele “Stufen” gezählt werden sollen, also z.B. der *kleinste* ($n=1$) oder auch der *zweit-kleinste* Wert ($n=2$) usw..

Intern zählt **n** allerdings mit, wieviele Fälle (Reihen) bereits ausgegeben wurde. Das ist am Anfang etwas un-intuitiv.

```
1 pf8 %>%
2   # zeigen die leichtesten Gewichte
3   slice_min(Gewicht, n=3)%>%
4   glimpse()
```

```
## Rows: 5
## Columns: 16
## $ Standort      <fct> Münster, Münster, Münster, Internet, Internet
## $ Alter         <int> 26, 24, 24, 24, 19
## $ Geschlecht    <fct> männlich, weiblich, weiblich, weiblich, weiblich
## $ Größe        <int> 180, 164, 160, 159, 155
## $ Gewicht       <dbl> 45, 45, 45, 45, 45
## $ Bildung       <fct> Hauptschule, Abitur, Abitur, NA, NA
## $ Beruf         <fct> Kaufmann/frau, Student*in, Student*in, ,
## $ Familienstand <fct> ledig, Partnerschaft, Partnerschaft, Partnerschaft, led...
## $ Kinder        <int> 0, 0, 0, 0, 0
## $ Wohnort       <fct> ländlich, städtisch, städtisch, ländlich, städtisch
## $ Rauchen       <fct> ja, nein, nein, nein, nein
## $ SportHäufig   <dbl> NA, 2.0, 3.0, 4.0, 2.5
## $ SportMinuten  <dbl> NA, 60, 60, 60, 90
## $ SportWie      <fct> NA, Gruppe, Allein, Allein, Allein
## $ SportWarum    <fct> NA, Vorbeugung, Vorbeugung, NA, NA
## $ LebenZufrieden <dbl> 10, 8, 9, 10, 9
```

Das Ergebnis ist evtl. anders, als Sie es erwarten. Es werden weiterhin 5 Fälle angezeigt, obwohl **n** auf 3 gesetzt wurde. Das liegt daran, dass 5 Fälle das kleinste **Gewicht** haben, und diese werden vollständig angezeigt. Intern ist **n**

dabei auf 5 angewachsen, weshalb keine *zweit-kleinsten* Gewichte mehr ausgegeben werden.

Lassen wir uns die Älteste Probanden auswählen.

```
1 pf8 %>%
2   # zeige die ältesten
3   slice_max(Alter, n=2) %>%
4   glimpse()
```

```
## Rows: 2
## Columns: 16
## $ Standort      <fct> Münster, Rheine
## $ Alter          <int> 86, 86
## $ Geschlecht     <fct> weiblich, weiblich
## $ Größe          <int> 162, 160
## $ Gewicht        <dbl> 70, 60
## $ Bildung        <fct> Hauptschule, Abitur
## $ Beruf          <fct> Rentner*in, Postbote*in
## $ Familienstand  <fct> verwitwet, verwitwet
## $ Kinder         <int> 0, 1
## $ Wohnort        <fct> ländlich, städtisch
## $ Rauchen        <fct> nein, nein
## $ SportHäufig    <dbl> 2, 2
## $ SportMinuten   <dbl> 45, 45
## $ SportWie       <fct> Gruppe, Gruppe
## $ SportWarum     <fct> Fitness, Freizeit
## $ LebenZufrieden <dbl> 7, 8
```

Die Tabelle besteht zwar aus 2 Fällen, aber beide Fälle haben das selbe **Alter**. Die *zweit-ältesten* werden nicht mit angezeigt, da **n** intern bereits auf 2 angewachsen ist.

Wir erhöhen **n** auf 3:

```
1 pf8 %>%
2   # zeige die ältesten
3   slice_max(Alter, n=3)%>%
4   glimpse()
```

```
## Rows: 4
## Columns: 16
## $ Standort      <fct> Münster, Rheine, Rheine, Rheine
## $ Alter          <int> 86, 86, 84, 84
## $ Geschlecht     <fct> weiblich, weiblich, weiblich, weiblich
## $ Größe          <int> 162, 160, 160, 164
## $ Gewicht        <dbl> 70, 60, 60, 70
## $ Bildung        <fct> Hauptschule, Abitur, Hauptschule, Hauptschule
## $ Beruf          <fct> Rentner*in, Postbote*in, Rentner*in, Verkäufer*in
```

```
## $ Familienstand <fct> verwitwet, verwitwet, verwitwet, verwitwet
## $ Kinder <int> 0, 1, 0, 0
## $ Wohnort <fct> ländlich, städtisch, städtisch, städtisch
## $ Rauchen <fct> nein, nein, nein, nein
## $ SportHäufig <dbl> 2, 2, 2, NA
## $ SportMinuten <dbl> 45, 45, 60, NA
## $ SportWie <fct> Gruppe, Gruppe, Allein, NA
## $ SportWarum <fct> Fitness, Freizeit, Fitness, NA
## $ LebenZufrieden <dbl> 7, 8, 8, 5
```

Jetzt haben wir insgesamt 4 Fälle. Es werden zuerst die *ältesten* angezeigt. Da es 2 Fälle sind, steigt `n` intern auf 2 an. Da wir `n` mit 3 aufgerufen haben, ist noch “Platz” für die *zweit-ältesten*. Da dies ebenfalls 2 Fälle sind, werden beide ausgegeben. Insgesamt sehen wir also 4 Fälle, obwohl wir `n` mit 3 aufgerufen haben.

Ähnlich wie `head()` und `tail()` zeigen `slice_head()` und `slice_tail()` die ersten bzw. letzten Fälle an.

```
1 # zeige die ersten 5 Fälle
2 pf8 %>% slice_head(n=5)
```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand
1	Münster	18	weiblich	172	69	Abitur	Inspektor*in	Partnerschaft
2	Münster	67	weiblich	165	67	mittlere Reife	Rentner*in	geschieden
3	Münster	60	weiblich	175	NA	Hochschule	Ergotherapeut*in	Partnerschaft
4	Münster	61	männlich	182	90	mittlere Reife	Beamter*in	ledig
5	Münster	24	männlich	173	68	Abitur	Student*in	ledig

```
1 # zeige die letzten 2 Fälle
2 pf8 %>% slice_tail(n=2)
```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand	Kinder	Wohnort
1	Internet	20	weiblich	168	58	<NA>		ledig	0	städtisch
2	Internet	40	männlich	183	78	<NA>		verheiratet	2	städtisch

Mit `slice_sample()` können zufällig Fälle aus dem Datensatz gezogen werden.

```
1 # ziehe zufällig 7 Fälle
2 pf8 %>% slice_sample(n=7)
```


	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand	Kinder
1	Internet	70	weiblich	158	75	<NA>		geschieden	0
2	Internet	27	männlich	184	83	<NA>		ledig	0
3	Internet	28	weiblich	168	74	<NA>		verheiratet	0
4	Münster	86	weiblich	162	70	Hauptschule	Rentner*in	verwitwet	0
5	Münster	25	weiblich	180	77	Fachabitur	Pflege	ledig	0
6	Münster	20	weiblich	165	85	Abitur	Azubi	ledig	0
7	Münster	57	männlich	161	60	Hauptschule	Sicherheitsdienst	verheiratet	0

1.6.8.2.1 Gruppierungen

Mit der Funktion `group_by()` können Gruppierungen vorgenommen werden. In Kombination mit `group_by()` zeigen die `slice()`-Funktionen die jeweilige Auswahl an Fällen pro Gruppe.

```

1 pf8 %>%
2   drop_na() %>%
3   # Gruppieren nach Geschlecht
4   group_by(Geschlecht) %>%
5   # zeige
6   slice(1)

```

```

# A tibble: 3 x 16
# Groups:   Geschlecht [3]
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand Kinder
  <fct>    <int> <fct>    <int>    <dbl> <fct>    <fct> <fct>    <int>
1 Münster     61 männlich    182      90 mittler... Beam... ledig      0
2 Münster     67 weiblich    165      67 mittler... Rent... geschieden 0
3 Rheine      32 divers     186      92 Hochsch... Sozi... verheiratet 0

```

```

1 pf8 %>%
2   drop_na() %>%
3   # Gruppieren nach Geschlecht
4   group_by(Geschlecht) %>%
5   # sortiere nach Alter
6   arrange(desc(Alter)) %>%
7   # zeige die ersten 3 Fälle pro Gruppe
8   slice(1:3)

```

```

# A tibble: 7 x 16
# Groups:   Geschlecht [3]
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand Kinder
  <fct>    <int> <fct>    <int>    <dbl> <fct>    <fct> <fct>    <int>
1 Ladbergen  83 männlich    182      88 Haupts... Rent... verheiratet 0
2 Münster    78 männlich    178     71.5 Hochsc... Lehr... verheiratet 0
3 Bahn       76 männlich    180      90 Hochsc... Arzt... verheiratet 0

```

4 Münster	86 weiblich	162	70	Haupts... Rent... verwitwet	0
5 Rheine	86 weiblich	160	60	Abitur Post... verwitwet	1
6 Rheine	84 weiblich	160	60	Haupts... Rent... verwitwet	0
7 Rheine	32 divers	186	92	Hochsc... Sozi... verheiratet	0

Innerhalb der Gruppierungen kann auch gefiltert werden.

```

1 pf8 %>%
2   # gruppieren nach Standort
3   group_by(Geschlecht) %>%
4   # filtere "divers"
5   dplyr::filter(Geschlecht != "divers") %>%
6   # filtere "Internet"
7   dplyr::filter(Standort != "Internet") %>%
8   # sortiere nach "Gewicht"
9   arrange(desc(Gewicht)) %>%
10  # zeige die ersten 2 Fälle pro Gruppe
11  slice(1,2)

```

```

# A tibble: 4 x 16
# Groups:   Geschlecht [2]
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand Kinder
  <fct>    <int> <fct>    <int>    <dbl> <fct>    <fct> <fct>    <int>
1 Münster    22 männlich    206    130 Fachabi... Kauf... ledig      0
2 Bahn       31 männlich    184    130 Fachabi... Scha... Partnerschaft 0
3 Münster    40 weiblich    171    122 Hochsch... Pres... Partnerschaft 0
4 Rheine     45 weiblich    168    120 mittler... kauf... geschieden 1

```

```

1 pf8 %>%
2   group_by(Standort) %>%
3   dplyr::filter(Familienstand == "ledig" & Geschlecht == "männlich") %>%
4   arrange(desc(Alter)) %>%
5   head()

```

```

# A tibble: 6 x 16
# Groups:   Standort [2]
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand Kinder
  <fct>    <int> <fct>    <int>    <dbl> <fct>    <fct> <fct>    <int>
1 Rheine     81 männlich    179     79 Hauptsc... "Han... ledig      0
2 Rheine     74 männlich    175     84 mittler... "Koc... ledig      0
3 Münster    72 männlich    175     86 mittler... "Sic... ledig      0
4 Münster    69 männlich    179     89 mittler... "Ren... ledig      0
5 Münster    61 männlich    182     90 mittler... "Bea... ledig      0
6 Rheine     59 männlich    175     70 Abitur    "" ledig      1

```

1.6.8.3 Variablen hinzufügen

Variablen können mit der Funktion `bind_cols()` hinzugefügt werden. Dabei muss die Länge der enthaltenen Werte mit der Länge des `tibble` übereinstimmen, ansonsten wiederholt  die Wertereihe so lange, bis sie mit der Länge des `tibble` übereinstimmt.

```
1 pf8 %>%
2   # Fügt eine Variable "test" hinzu, in der alle Werte TRUE sind.
3   bind_cols(test = TRUE) %>%
4   glimpse()
```

```
Rows: 731
Columns: 17
$ Standort      <fct> Münster, Münster, Münster, Münster, Münster, Münster, Münster...
$ Alter         <int> 18, 67, 60, 61, 24, 21, 59, 56, 82, 52, 79, 22, 19, 18, 17, 2...
$ Geschlecht    <fct> weiblich, weiblich, weiblich, männlich, männlich, weiblich, w...
$ Größe         <int> 172, 165, 175, 182, 173, 177, 168, 156, 184, 166, 161, 206, 1...
$ Gewicht       <dbl> 69, 67, NA, 90, 68, 60, 80, 60, NA, 60, 66, 130, 52, 68, 48, ...
$ Bildung       <fct> Abitur, mittlere Reife, Hochschule, mittlere Reife, Abitur, A...
$ Beruf         <fct> Inspektor*in, Rentner*in, Ergotherapeut*in, Beamter*in, Stude...
$ Familienstand <fct> Partnerschaft, geschieden, Partnerschaft, ledig, ledig, Partn...
$ Kinder        <int> 0, 0, 0, 0, 0, 0, 0, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Wohnort       <fct> ländlich, ländlich, städtisch, ländlich, städtisch, städtisch...
$ Rauchen       <fct> nein, nein, nein, nein, nein, ja, nein, ja, nein, nein, nein,...
$ SportHäufig   <dbl> NA, 2.0, 2.0, 4.0, 4.0, 1.0, 2.0, 1.0, 1.0, 2.0, NA, 4.0, 5.0...
$ SportMinuten  <dbl> NA, 60, 45, 120, 60, 60, 45, 90, NA, 45, NA, 90, 20, 120, 120...
$ SportWie      <fct> Allein, Gruppe, beides, Allein, Allein, Gruppe, Gruppe, beide...
$ SportWarum    <fct> Arbeitsweg, Vorbeugung, Vorbeugung, Fitness, Gewichtsreduktio...
$ LebenZufrieden <dbl> 5, 7, 7, 2, 9, 8, 5, 8, 10, 8, 8, 8, 7, 8, 7, 8, 7, 7, 8, 7, ...
$ test          <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, T...
```

Unsere neue Variable `test` sehen wir ganz unten. Alle Werte stehen auf `TRUE`.

1.6.8.4 Fälle hinzufügen

Neue Fälle können mit der Funktion `bind_rows()` hinzugefügt werden. Dies klappt nur, wenn die neuen Fälle als `tibble` vorliegen, das über die selben Variablen verfügt wie der Originaldatensatz. Die Reihenfolge der Spalten ist egal, da über die Spaltennamen gematcht wird.



Achtung, Anfängerfehler!



Wichtig ist jedoch, dass innerhalb der Variablen auch der selbe Datentyp (numerisch, faktor, logisch) vorliegt, da ansonsten die Datentypen auf den kleinsten gemeinsamen Nenner (`character`) zurückfallen. Wir machen es hier einmal falsch:

```

1 # erzeuge neuen Fall
2 # ohne Angabe des Datentyps
3 neu <- tibble("Internet", 44, "weiblich", 166, 70, NA, "Pilot", "verheiratet", 0,
4 "ländlich", "ja")
5 # kopiere die Spaltennamen
6 colnames(neu) <- colnames(pf8)
7
8 # füge zum Datensatz hinzu
9 # Achtung, zerschießt die Datentypen des tibble!
10 pf8 %>%
11   bind_rows( neu ) %>%
12   tail(pf8)

```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf	Familienstand	Kinder
727	Internet	19	weiblich	158	58	<NA>		ledig	0
728	Internet	37	weiblich	160	59	<NA>		verheiratet	1
729	Internet	56	männlich	185	79	<NA>		Partnerschaft	0
730	Internet	20	weiblich	168	58	<NA>		ledig	0
731	Internet	40	männlich	183	78	<NA>		verheiratet	2
732	Internet	44	weiblich	166	70	<NA>	Pilot	verheiratet	0
	Wohnort	Rauchen	SportHäufig	SportMinuten	SportWie	SportWarum	LebenZufrieden		
727	städtisch	nein	7.0	10	Allein	<NA>	7		
728	ländlich	nein	3.0	60	Allein	<NA>	9		
729	ländlich	nein	2.0	90	Gruppe	<NA>	9		
730	städtisch	nein	1.0	60	Allein	<NA>	8		
731	städtisch	ja	2.5	120	Gruppe	<NA>	10		
732	ländlich	ja	NA	NA	<NA>	<NA>	NA		

Unsere Zeile ist ganz unten zu sehen. Die fehlenden Werte wurden mit `NA` aufgefüllt. Alles scheint gut gelaufen zu sein. Wie ein Blick mit `glimpse()` jedoch zeigt, haben wir die Datentypen unseres `tibbles` zerschossen, da unsere hinzugefügte Zeile nicht die korrekten *Datentypen* beinhaltet.

```

1 pf8 %>%
2   bind_rows( neu ) %>%
3   glimpse()

```

```

Rows: 732
Columns: 16
$ Standort      <chr> "Münster", "Münster", "Münster", "Münster", "Münster", "Münst...
$ Alter         <dbl> 18, 67, 60, 61, 24, 21, 59, 56, 82, 52, 79, 22, 19, 18, 17, 2...
$ Geschlecht    <chr> "weiblich", "weiblich", "weiblich", "männlich", "männlich", "...
$ Größe         <dbl> 172, 165, 175, 182, 173, 177, 168, 156, 184, 166, 161, 206, 1...
$ Gewicht       <dbl> 69, 67, NA, 90, 68, 60, 80, 60, NA, 60, 66, 130, 52, 68, 48, ...

```

```

$ Bildung      <fct> Abitur, mittlere Reife, Hochschule, mittlere Reife, Abitur, A...
$ Beruf        <chr> "Inspektor*in", "Rentner*in", "Ergotherapeut*in", "Beamter*in...
$ Familienstand <chr> "Partnerschaft", "geschieden", "Partnerschaft", "ledig", "led...
$ Kinder       <dbl> 0, 0, 0, 0, 0, 0, 0, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Wohnort      <chr> "ländlich", "ländlich", "städtisch", "ländlich", "städtisch",...
$ Rauchen      <chr> "nein", "nein", "nein", "nein", "nein", "ja", "nein", "ja", "...
$ SportHäufig  <dbl> NA, 2.0, 2.0, 4.0, 4.0, 1.0, 2.0, 1.0, 1.0, 2.0, NA, 4.0, 5.0...
$ SportMinuten <dbl> NA, 60, 45, 120, 60, 60, 45, 90, NA, 45, NA, 90, 20, 120, 120...
$ SportWie     <fct> Allein, Gruppe, beides, Allein, Allein, Gruppe, Gruppe, beide...
$ SportWarum   <fct> Arbeitsweg, Vorbeugung, Vorbeugung, Fitness, Gewichtsreduktio...
$ LebenZufrieden <dbl> 5, 7, 7, 2, 9, 8, 5, 8, 10, 8, 8, 8, 7, 8, 7, 8, 7, 7, 8, 7, ...

```

Wie Sie sehen, sind unsere ehemaligen Faktoren `Standort` und `Geschlecht` auf `<chr>` zurückgefallen.

Ein falscher `bind_rows()`-Befehl kann Ihnen also das gesamte tibble “zerschießen”!

Fügen wir nun eine **korrekte** neue Zeile hinzu

```

1 # erzeuge korrekte neue Zeile
2 neu <- tibble(factor("Internet"), 44, factor("weiblich"), 166, 70, NA, factor("Pilot"),
3 factor("verheiratet"), 0, factor("ländlich"), factor("ja"))
4 # Spaltennamen übernehmen
5 colnames(neu) <- colnames(pf8)
6
7 pf8 %>%
8   bind_rows( neu ) %>%
9   glimpse()

```

```

Rows: 732
Columns: 16
$ Standort      <fct> Münster, Münster, Münster, Münster, Münster, Münster, Münster...
$ Alter         <dbl> 18, 67, 60, 61, 24, 21, 59, 56, 82, 52, 79, 22, 19, 18, 17, 2...
$ Geschlecht    <fct> weiblich, weiblich, weiblich, männlich, männlich, weiblich, w...
$ Größe        <dbl> 172, 165, 175, 182, 173, 177, 168, 156, 184, 166, 161, 206, 1...
$ Gewicht       <dbl> 69, 67, NA, 90, 68, 60, 80, NA, 60, 66, 130, 52, 68, 48, ...
$ Bildung       <fct> Abitur, mittlere Reife, Hochschule, mittlere Reife, Abitur, A...
$ Beruf         <fct> Inspektor*in, Rentner*in, Ergotherapeut*in, Beamter*in, Stude...
$ Familienstand <fct> Partnerschaft, geschieden, Partnerschaft, ledig, ledig, Partn...
$ Kinder        <dbl> 0, 0, 0, 0, 0, 0, 0, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ Wohnort       <fct> ländlich, ländlich, städtisch, ländlich, städtisch, städtisch...
$ Rauchen       <fct> nein, nein, nein, nein, nein, ja, nein, ja, nein, nein, nein,...
$ SportHäufig   <dbl> NA, 2.0, 2.0, 4.0, 4.0, 1.0, 2.0, 1.0, 1.0, 2.0, NA, 4.0, 5.0...
$ SportMinuten  <dbl> NA, 60, 45, 120, 60, 60, 45, 90, NA, 45, NA, 90, 20, 120, 120...
$ SportWie      <fct> Allein, Gruppe, beides, Allein, Allein, Gruppe, Gruppe, beide...
$ SportWarum    <fct> Arbeitsweg, Vorbeugung, Vorbeugung, Fitness, Gewichtsreduktio...
$ LebenZufrieden <dbl> 5, 7, 7, 2, 9, 8, 5, 8, 10, 8, 8, 8, 7, 8, 7, 8, 7, 7, 8, 7, ...

```

Die Datentypen sind erhalten geblieben.

1.6.8.5 Variablen auswählen

Mit der Funktion `select()` können Variablen (Spalten) des `tibble` ausgewählt werden.

```
1 # wähle Variablen "Alter", "Größe" und "Gewicht"
2 # aus Datensatz pf8
3 pf8 %>%
4   select(Alter, Größe, Gewicht) %>%
5   head(.)
```

	Alter	Größe	Gewicht
11	18	172	69
12	67	165	67
13	60	175	NA
14	61	182	90
15	24	173	68
16	21	177	60

In Kombination mit der Funktion `everything()` kann die Reihenfolge der Variablen im Datensatz geändert werden. Die Funktion `everything()` hängt alle weiteren Variablen an unsere Auswahl an. Angenommen, wir möchten `Größe` und `Gewicht` “als erstes” sehen, und alle anderen Variablen danach, so lautet der Befehl:

```
1 # zeige erst "Größe" und "Gewicht"
2 # und danach alles andere.
3 pf8 %>%
4   select(Größe, Gewicht, everything()) %>%
5   as_tibble()
```

```
# A tibble: 731 x 16
  Größe Gewicht Standort Alter Geschlecht Bildung  Beruf  Familienstand
  <int>   <dbl>   <fct>   <int> <fct>   <fct>   <fct>   <fct>
1  172     69 Münster    18 weiblich Abitur   Inspekt... Partnerschaft
2  165     67 Münster    67 weiblich mittlere... Rentner... geschieden
3  175     NA Münster    60 weiblich Hochschu... Ergothe... Partnerschaft
4  182     90 Münster    61 männlich mittlere... Beamter... ledig
5  173     68 Münster    24 männlich Abitur   Student... ledig
6  177     60 Münster    21 weiblich Abitur   Student... Partnerschaft
7  168     80 Münster    59 weiblich mittlere... arbeitl... verheiratet
8  156     60 Münster    56 weiblich Ausbildu... Pflegeh... geschieden
9  184     NA Münster    82 männlich mittlere... Kaufman... verheiratet
10 166     60 Münster    52 weiblich Fachabit... Kaufman... verheiratet
```

Mit der Funktion `pull()` werden die Spaltenwerte als Vektor wiedergegeben

```

1 # gib Werte von "Alter" als Vektor
2 pf8 %>%
3   pull(Alter) %>%
4   head(10)

```

```
[1] 18 67 60 61 24 21 59 56 82 52
```

1.6.8.6 Variablen erzeugen

Die Funktion `mutate()` erlaubt es, neue Variablen (Spalten) zu erzeugen, die durch Interaktion mit den anderen Variablen entstanden sind. So können wir im Datensatz `pf8` aus den Variablen `Größe` und `Gewicht` den Body-Maß-Index errechnen, und diese Werte als neue Variable (Spalte) speichern.

```

1 # erzeuge BMI
2 pf8 %>%
3   mutate(BMI = Gewicht/Größe*100) %>%
4   head()

```

	Standort	Alter	Geschlecht	Größe	Gewicht	Bildung	Beruf
11	Münster	18	weiblich	172	69	Abitur	Inspektor*in
12	Münster	67	weiblich	165	67	mittlere Reife	Rentner*in
13	Münster	60	weiblich	175	NA	Hochschule	Ergotherapeut*in
14	Münster	61	männlich	182	90	mittlere Reife	Beamter*in
15	Münster	24	männlich	173	68	Abitur	Student*in
16	Münster	21	weiblich	177	60	Abitur	Student*in

	Sport	Warum	Leben	Zufrieden	BMI
11		Arbeitsweg		5	40.11628
12		Vorbeugung		7	40.60606
13		Vorbeugung		7	NA
14		Fitness		2	49.45055
15		Gewichtsreduktion		9	39.30636
16		Fitness		8	33.89831

Mit der Schwesterfunktion `transmute()` wird nur die neue Variable in einem eigenen tibble zurückgegeben

```

1 # erzeuge BMI und gib nur BMI zurück
2 pf8 %>%
3   transmute(BMI = Gewicht/Größe*100) %>%
4   head()

```

```

      BMI
11 40.11628
12 40.60606
13      NA
14 49.45055
15 39.30636
16 33.89831

```

1.6.8.7 statistische Berechnungen

Mit der Funktion `summarise()` können “zusammenfassenden Statistiken” berechnet werden. Sie fungiert als eine Art “Vermittlungsfunktion”, um Statistikfunktionen (siehe Kapitel 2 ab Seite 205) in den Tidyverse-*Workflow* zu integrieren.

Klassische zusammenfassende Kennzahlen sind Mittelwert (über Funktion `mean()`), Median (über Funktion `median()`) und Standardabweichung (über Funktion `sd()`). Es funktioniert aber auch mit allen anderen Statistikfunktionen. Wir beschränken uns im Weiteren auf diese drei, alle anderen werden in Kapitel 2 vorgestellt.

Berechnen wir mit `mean()` den Mittelwert für `Alter`.

```

1 # Mittelwert von "Alter"
2 pf8 %>%
3   summarise(Mittelwert = mean(Alter))

```

```

Mittelwert
1      NA

```

Wir erhalten ein `NA` zurück, weil in der Variable `Alter` fehlende Werte enthalten sind. Fast alle statistischen Funktionen erwarten von uns, dass `NAs` weggefilter wurden. Nutzen wir die Macht der Pipe und ändern den Befehl in

```

1 # Mittelwert von "Alter"
2 pf8 %>%
3   drop_na() %>%
4   summarise(Mittelwert = mean(Alter))

```

```

Mittelwert
1  38.25126

```


Mit der Funktion `group_by()` können Gruppierungen vorgenommen werden.

```
1 # Gruppiere nach Geschlecht
2 pf8 %>%
3   group_by(Geschlecht) %>%
4   drop_na() %>%
5   summarise(Mittelwert = mean(Alter))
```

```
# A tibble: 3 x 2
  Geschlecht Mittelwert
  <fct>         <dbl>
1 männlich      37.3
2 weiblich      39.1
3 divers        32
```

Die Liste der “zusammenfassenden Statistiken” lässt sich beliebig erweitern.

```
1 pf8 %>%
2   group_by(Geschlecht) %>%
3   drop_na() %>%
4   summarise(Mittelwert = mean(Alter),
5             Median = median(Alter),
6             Stdabw = sd(Alter))
```

```
# A tibble: 3 x 4
  Geschlecht Mittelwert Median Stdabw
  <fct>         <dbl>   <dbl>   <dbl>
1 männlich      37.3     29    17.5
2 weiblich      39.1     33    18.9
3 divers        32      32     NA
```

Auch, indem Gruppierungen komplexer definiert werden.

```
1 # mit mehr Gruppierungen
2 pf8 %>%
3   group_by(Geschlecht, Standort) %>%
4   drop_na() %>%
5   summarise(Mittelwert = mean(Alter),
6             Median = median(Alter),
7             Stdabw = sd(Alter))
```

```
# A tibble: 9 x 5
# Groups:   Geschlecht [3]
  Geschlecht Standort Mittelwert Median Stdabw
  <fct>         <fct>         <dbl>   <dbl>   <dbl>
1 männlich Rheine      34.3     26    15.4
2 männlich Münster     37.9     28    18.5
3 männlich Bahn       39.9     36    19.8
4 männlich Ladbergen  41.4     40    17.6
```

5	weiblich	Rheine	37.6	33	17.8
6	weiblich	Münster	38.8	27	20.0
7	weiblich	Bahn	41.3	37.5	19.4
8	weiblich	Ladbergen	43	40.5	18.5
9	divers	Rheine	32	32	NA

Die Funktion `count()` zählt die Häufigkeit der Variablenwerte. Dies ist nicht nur bei Faktoren hilfreich.

```

1 # mit mehr Gruppierungen
2 pf8 %>%
3   count(Geschlecht)

```

	Geschlecht	n
1	männlich	287
2	weiblich	437
3	divers	1
4	<NA>	6

In Kombination mit `group_by()` lassen sich Ausgaben ähnlich der Kreuztabelle erstellen.

```

1 pf8 %>%
2   drop_na() %>%
3   group_by(Familienstand, Geschlecht) %>%
4   count()

```

```

# A tibble: 13 x 3
# Groups:   Familienstand, Geschlecht [13]
  Familienstand Geschlecht    n
  <fct>         <fct>    <int>
1 ledig        männlich    74
2 ledig        weiblich    82
3 Partnerschaft männlich    32
4 Partnerschaft weiblich    40
5 verheiratet  männlich    61
6 verheiratet  weiblich    78
7 verheiratet  divers       1
8 geschieden  männlich     5
9 geschieden  weiblich     7
10 verwitwet   männlich     4
11 verwitwet   weiblich     9
12 getrennt    männlich     3
13 getrennt    weiblich     2

```

Mit der Funktion `add_count()` werden die Häufigkeitswerte als eigene Variable im Datensatz gespeichert.

```

1 pf8 %>%
2   # Speichere die Levelhäufigkeit als eigene Variable
3   add_count(Geschlecht, name = "AnzahlGeschlecht") %>%
4   # Zeige diese Variablen als erstes, dann den Rest
5   select(Geschlecht, AnzahlGeschlecht, everything()) %>%
6   # Ausgabe als tibble ist "schöner"
7   as_tibble()

```

```

# A tibble: 731 x 17
  Geschlecht AnzahlGeschlecht Standort Alter Größe Gewicht Bildung
  <fct>          <int> <fct>    <int> <int>   <dbl> <fct>
1 weiblich           437 Münster     18   172    69 Abitur
2 weiblich           437 Münster     67   165    67 mittlere Re...
3 weiblich           437 Münster     60   175    NA Hochschule
4 männlich           287 Münster     61   182    90 mittlere Re...
5 männlich           287 Münster     24   173    68 Abitur
6 weiblich           437 Münster     21   177    60 Abitur
7 weiblich           437 Münster     59   168    80 mittlere Re...
8 weiblich           437 Münster     56   156    60 Ausbildung
9 männlich           287 Münster     82   184    NA mittlere Re...
10 weiblich          437 Münster     52   166    60 Fachabitur

```

Die Funktion `n()` liefert die Anzahl der Fälle.

```

1 pf8 %>%
2   group_by(Standort, Geschlecht) %>%
3   drop_na() %>%
4   summarise(xquer = mean(Alter),
5             sd = sd(Alter),
6             median = median(Alter),
7             n = n())

```

```

# A tibble: 9 x 6
# Groups:   Standort [4]
  Standort Geschlecht xquer  sd median  n
  <fct>      <fct>    <dbl> <dbl> <dbl> <int>
1 Rheine    männlich  34.3  15.4  26    67
2 Rheine    weiblich  37.6  17.8  33    83
3 Rheine    divers   32    NA    32     1
4 Münster   männlich  37.9  18.5  28    62
5 Münster   weiblich  38.8  20.0  27    83
6 Bahn      männlich  39.9  19.8  36    29
7 Bahn      weiblich  41.3  19.4  37.5   38
8 Ladbergen männlich  41.4  17.6  40    21
9 Ladbergen weiblich  43    18.5  40.5   14

```

1.6.8.8 Umgang mit Faktoren

Mit dem Paket `forcats` stehen im Tidyverse viele hilfreiche Funktionen für den Umgang mit Faktoren zur Verfügung.

1.6.8.8.1 Levels umbenennen

Mit der Funktion `fct_recode()` können Faktorenlevels umbenannt werden.

```
1 pf8 %>%
2   mutate(Geschlecht = fct_recode(Geschlecht,
3     m = "männlich",
4     w = "weiblich",
5     d = "divers")) %>%
6   as_tibble()
```

```
# A tibble: 731 x 16
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand
  <fct>    <int> <fct>      <int>    <dbl> <fct>    <fct> <fct>
1 Münster     18 w          172      69 Abitur Insp... Partnerschaft
2 Münster     67 w          165      67 middle... Rent... geschieden
3 Münster     60 w          175      NA Hochsc... Ergo... Partnerschaft
4 Münster     61 m          182      90 middle... Beam... ledig
5 Münster     24 m          173      68 Abitur Stud... ledig
6 Münster     21 w          177      60 Abitur Stud... Partnerschaft
7 Münster     59 w          168      80 middle... arbe... verheiratet
8 Münster     56 w          156      60 Ausbil... Pfle... geschieden
9 Münster     82 m          184      NA middle... Kauf... verheiratet
10 Münster    52 w          166      60 Fachab... Kauf... verheiratet
```

Enthalten die neuen Levelnamen Sonderzeichen (wie das Leerzeichen), müssen sie in Anführungsstrichen gesetzt werden.

```
1 pf8 %>%
2   mutate(Geschlecht = fct_recode(Geschlecht,
3     "ein Mann" = "männlich",
4     "eine Frau" = "weiblich",
5     "ein(e) Divers" = "divers")) %>%
6   as_tibble()
```

```
# A tibble: 731 x 16
  Standort Alter Geschlecht Größe Gewicht Bildung Beruf Familienstand
  <fct>    <int> <fct>      <int>    <dbl> <fct>    <fct> <fct>
1 Münster     18 eine Frau  172      69 Abitur Insp... Partnerschaft
2 Münster     67 eine Frau  165      67 middle... Rent... geschieden
3 Münster     60 eine Frau  175      NA Hochsc... Ergo... Partnerschaft
4 Münster     61 ein Mann  182      90 middle... Beam... ledig
5 Münster     24 ein Mann  173      68 Abitur Stud... ledig
6 Münster     21 eine Frau  177      60 Abitur Stud... Partnerschaft
```

7 Münster	59 eine Frau	168	80 middle...	arbe...	verheiratet
8 Münster	56 eine Frau	156	60 Ausbil...	Pfle...	geschieden
9 Münster	82 ein Mann	184	NA middle...	Kauf...	verheiratet
10 Münster	52 eine Frau	166	60 Fachab...	Kauf...	verheiratet

1.6.8.8.2 Levelreihenfolge ändern

Bei kategorialen Daten existiert keine geordnete Reihe der Werte. Dennoch kann es hilfreich sein, kategoriale Levels in eine bestimmte Reihenfolge zu bringen. Über die Hausfunktion `factor()` können über den Parameter `levels` die Levelreihenfolgen von Hand geändert werden.

```
1 x <- factor(c("vielleicht", "ja", "nein"))
2 x
```

```
[1] vielleicht ja      nein
Levels: ja nein vielleicht
```

```
1 # ändere Levelreihenfolge
2 factor(x, levels=c("nein", "vielleicht", "ja"))
```

```
[1] vielleicht ja      nein
Levels: nein vielleicht ja
```

Für häufige Anwendungsfälle bietet `forcats` Funktionen, die uns diese Arbeit abnehmen.

Schauen wir uns im Datensatz `pf8` die Anlässe für Sport an. Das Skalenniveau ist nominal. Wenn wir die Daten plotten (zu `ggplot` siehe Kapitel 2.4), werden die Daten (die Diagrammsäulen) in der Reihenfolge der Levels angezeigt, und nicht in der Reihenfolge der Häufigkeiten.

```
1 pf8 %>%
2   select(SportWarum) %>%
3   drop_na() %>%
4   # dies ist der Plotbefehl, den Sie jetzt noch nicht verstehen.
5   # Lesen Sie das Kapitel zu "ggplot".
6   ggplot(aes(x=SportWarum)) + geom_bar()
```

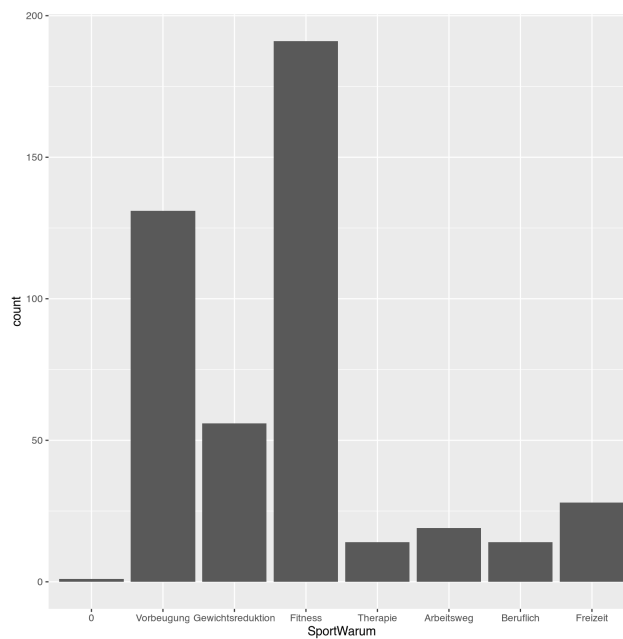


Abb. 1.85

Soll die Reihenfolge der Levels so verändert werden, dass die mit der höchsten Ausprägung zuerst angezeigt werden, kann die Funktion `fct_infreq()` verwendet werden.

```

1 pf8 %>%
2   drop_na() %>%
3   # Sortiere nach Häufigkeiten
4   mutate(SportWarum = fct_infreq(SportWarum)) %>%
5   # plotten
6   ggplot(aes(x=SportWarum)) + geom_bar()
```

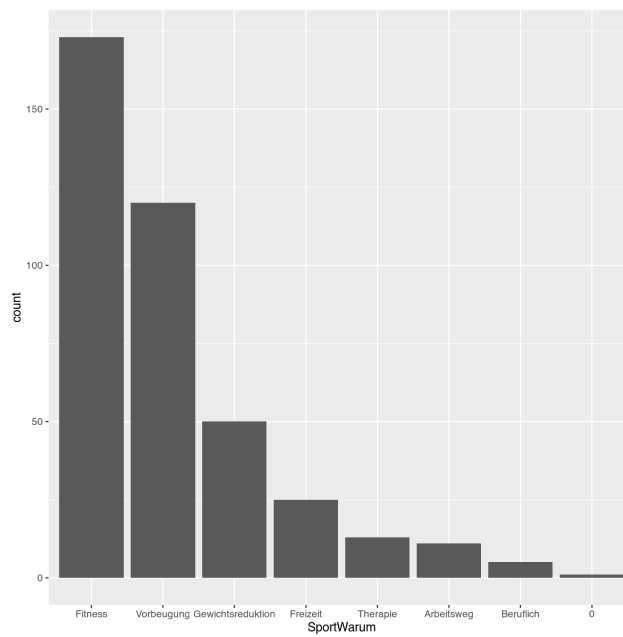


Abb. 1.86

Mit der Funktion `fct_rev()` wird die Levelreihenfolge umgekehrt.

```

1 pf8 %>%
2   drop_na() %>%
3   # Sortiere nach Häufigkeiten
4   mutate(SportWarum = fct_infreq(SportWarum)) %>%
5   # kehre Levelreihenfolge um
6   mutate(SportWarum = fct_rev(SportWarum)) %>%
7   # plotten
8   ggplot(aes(x=SportWarum)) + geom_bar()
```

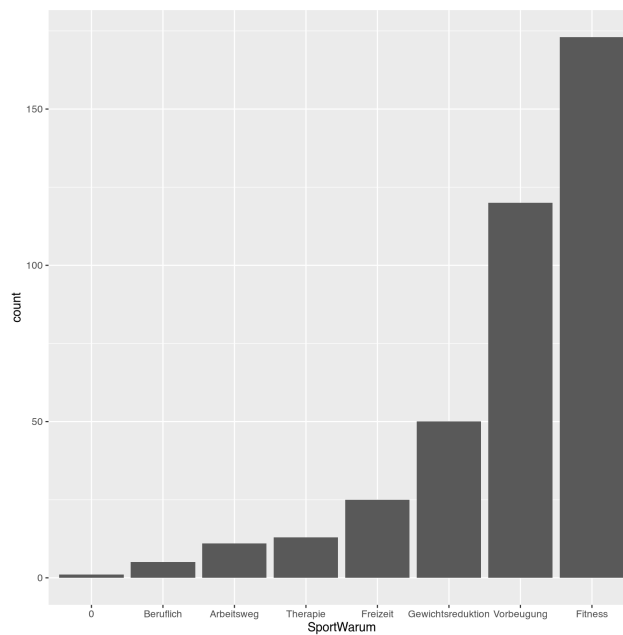


Abb. 1.87

Die Levelreihenfolge kann auch auf Grundlage einer anderen Variable erfolgen. Erstellen wir uns hierfür ein Subset des pf8-Datensatzes, indem wir für jede Sport-Kategorie die Mittelwerte von `Alter` und `Gewicht` ermitteln.

```

1 pf8sub <- pf8 %>%
2   group_by(SportWarum) %>%
3   summarise(
4     Alter = mean(Alter, na.rm=T),
5     Gewicht = mean(Gewicht, na.rm=T),
6     n = n()
7   )
8 pf8sub

```

```

# A tibble: 9 x 4
  SportWarum      Alter Gewicht      n
  <fct>         <dbl>   <dbl> <int>
1 0              19        65      1
2 Vorbeugung     42.9      78.1    131
3 Gewichtsreduktion 31.9      81.7     56
4 Fitness        37.4      73.2    191
5 Therapie       52.4      80.8     14
6 Arbeitsweg     34.6      82.1     19
7 Beruflich      43.9      80.6     14

```


8 Freizeit	36.7	74.0	28
9 NA	34.1	73.9	277

Wenn wir die Daten plotten, folgen die Datenpunkte auch hier der Levelreihenfolge.

```
1 pf8sub %>%
2   # plotten
3   ggplot(aes(x=Alter, y=SportWarum)) + geom_point()
```

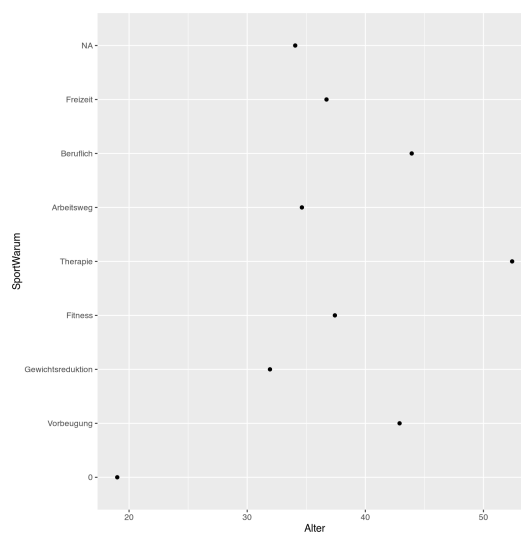


Abb. 1.88

Soll auch hier die Reihenfolge nach Mittelwerten erfolgen, kann die Funktion `fct_reorder()` verwendet werden.

```
1 pf8sub %>%
2   # überschreibe Variable "SportWarum"
3   # ordne die Levels von "SportWarum" nach "Alter"
4   mutate(SportWarum = fct_reorder(SportWarum, Alter)) %>%
5   # plotten
6   ggplot(aes(x=Alter, y=SportWarum)) + geom_point()
```

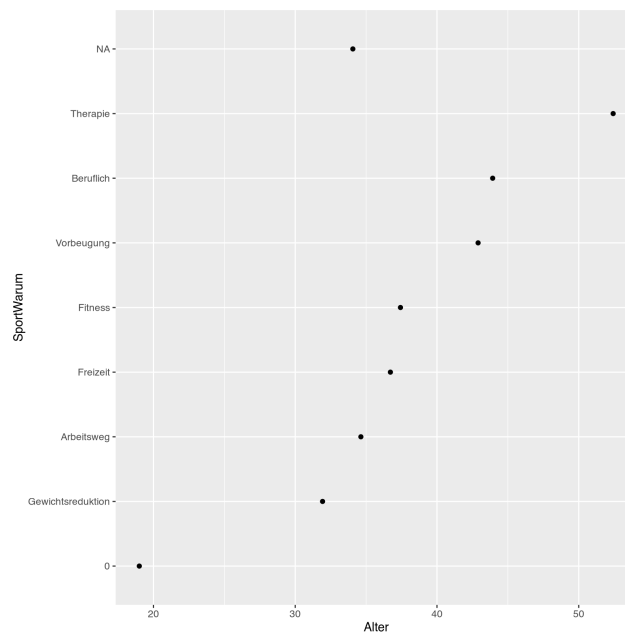


Abb. 1.89

Beachten Sie, dass ‘NA’ weiterhin ganz oben angezeigt werden.

Mit der Funktion `fct_relevel()` können einzelne Levels “nach vorne” geholt werden. In diesem Beispiel möchten wir, dass der Grund **Freizeit** als erstes angezeigt wird.

```

1 pf8sub %>%
2   # überschreibe Variable "SportWarum"
3   # ordne die Levels von "SportWarum" nach "Alter"
4   mutate(SportWarum = fct_reorder(SportWarum, Alter)) %>%
5   # hole Level "Freizeit" nach vorne
6   mutate(SportWarum = fct_relevel(SportWarum, "Freizeit")) %>%
7   # plotten
8   ggplot(aes(x=Alter, y=SportWarum)) + geom_point()

```

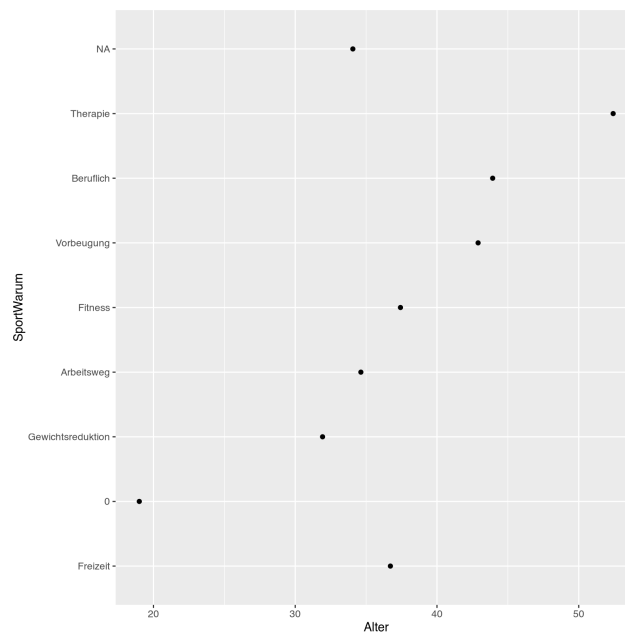


Abb. 1.90

1.6.8.8.3 Levels zusammenfassen

Schauen wir uns die Berufe im Datensatz pf8 an.

```
1 pf8 %>%
2   dplyr::select(Beruf) %>%
3   dplyr::count(Beruf) %>%
4   arrange(n) %>%
5   as_tibble()
```

```
# A tibble: 104 x 2
  Beruf          n
  <fct>        <int>
1 "Produktionsleiter" 1
2 "Angestellte"      1
3 "Bäcker*in"        1
4 "Bauzeichner*in"   1
5 "Berater*in"       1
6 "Betriebsprüfer*in" 1
7 "Bibliothekar*in"  1
8 "Büchereileiter*in" 1
9 "Controller"       1
```

```
10 "Ergotherapeut*in"      1
# ... with 94 more rows
```

Wie Sie sehen, haben wir sehr viele Levels, die nur einmal oder zweimal vorkommen.

Mit der Funktion `fct_lump_min()` können wir niedrigausgeprägte Levels zusammenfassen.

```
1 pf8 %>%
2   drop_na() %>%
3   # fasse alle Levels mit Ausprägung kleiner 4 zusammen
4   mutate(Beruf = fct_lump_min(Beruf, 4)) %>%
5   dplyr::select(Beruf) %>%
6   dplyr::count(Beruf) %>%
7   arrange(desc(n)) %>%
8   as_tibble()
```

```
# A tibble: 22 x 2
  Beruf      n
  <fct>    <int>
1 Other      101
2 Student*in  82
3 kaufmann/frau  29
4 Handwerker*in  19
5 Angestellte*r  18
6 Azubi       17
7 Pflege      16
8 Rentner*in   14
9 Schüler*in   14
10 Lehrer*in   12
# ... with 12 more rows
```

Alle Levels mit Ausprägungen kleiner 4 wurden in der neuen Level ‘Other’ zusammengefasst.

Mit der Funktion `fct_lump_n()` bleiben die höchsten `n` Levels erhalten, während der Rest unter `Other` zusammengefasst wird.

```
1 pf8 %>%
2   drop_na() %>%
3   # behalte die 3 höchstausgeprägtesten Levels
4   # und fasse den Rest zusammen
5   mutate(Beruf = fct_lump_n(Beruf, 3)) %>%
6   dplyr::select(Beruf) %>%
7   dplyr::count(Beruf) %>%
8   arrange(desc(n)) %>%
9   as_tibble()
```

```
# A tibble: 4 x 2
  Beruf      n
  <fct>    <int>
1 Other    268
2 Student*in 82
3 kaufmann/frau 29
4 Handwerker*in 19
```

Mit der Funktion `fct_collapse()` können die Levels “von Hand” zusammengefasst werden. Schauen wir uns den Familienstand im Datensatz `pf8` an.

```
1 levels(pf8$Familienstand)
```

```
[1] "ledig" "Partnerschaft" "verheiratet" "geschieden" "verwitwet" "getrennt"
```

Fassen wir nun alle “Verpartnerte” und alle “Singles” zu zwei Levels zusammen.

```
1 ```{r}
2 pf8 %>%
3   # fasse Singles und Partner in eigenen Levels zusammen
4   mutate(Partnerschaft = fct_collapse(Familienstand,
5     "Partnerschaft" = c("Partnerschaft", "verheiratet"),
6     "Single" = c("ledig", "geschieden", "verwitwet", "getrennt"))) %>%
7   dplyr::select(Partnerschaft, Familienstand, Alter) %>%
8   as_tibble()
```

```
# A tibble: 731 x 3
  Partnerschaft Familienstand Alter
  <fct>         <fct>         <int>
1 Partnerschaft Partnerschaft    18
2 Single       geschieden     67
3 Partnerschaft Partnerschaft    60
4 Single       ledig           61
5 Single       ledig           24
6 Partnerschaft Partnerschaft    21
7 Partnerschaft verheiratet     59
8 Single       geschieden     56
9 Partnerschaft verheiratet     82
10 Partnerschaft verheiratet     52
# ... with 721 more rows
```

1.6.8.9 Daten visualisieren

Zum Visualisieren von Daten sind neben Tabellen vor allem Diagramme geeignet. Wie Sie im Tidyverse Diagramme erstellen behandeln wir im Kapitel `ggplot` ab Seite 322.

1.6.9 Schritt 4: Ergebnisse kommunizieren

Ihre Ergebnisse können Sie Ihrem Publikum am besten mit `RMarkdown` kommunizieren. Lesen Sie hierzu Abschnitt 1.4 ab Seite 121.

1.6.10 Die Macht der Pipe

Die volle Macht entfaltet das Tidyverse durch die Aneinanderreihung von Funktionen und Befehlen mittels der Pipe. Im Vergleich zum klassischen  kommen Sie so viel leichter zum Ziel, und Ihr Code ist gut les- und nachvollziehbar.

Versuchen Sie den folgenden Code nachzuvollziehen

```
1 mma %>%  
2   drop_na() %>%  
3   dplyr::filter(ward == "Chirurgie") %>%  
4   group_by(day) %>%  
5   ggplot(aes(x=category, fill=category)) + geom_bar(aes(fill=qual))
```

```
1 pf8 %>%  
2   group_by(Geschlecht, Standort) %>%  
3   drop_na() %>%  
4   dplyr::filter(Rauchen == "ja") %>%  
5   summarise(Mittelwert = mean(Alter),  
6             Median = median(Alter),  
7             Stdabw = sd(Alter))
```

1.7 Weitere Hilfsmittel

1.7.1 Cheatsheets

In RStudio ist eine Liste mit “Cheatsheets” enthalten. Ein Cheatsheet (*Schummelzettel*) ist eine Seite, auf der (möglichst) alle wichtigen Befehle zu einem Thema (z.B. L^AT_EX oder  oder ggplot) enthalten sind. Diese Seite kann z.B. ausgedruckt und neben den PC bzw. ans Pinnboard geheftet werden. So hat man immer einen guten Überblick über die wichtigsten Befehle oder Funktionen.

Klicken Sie in der Menüleiste von RStudio auf **Help** → **Cheatsheets**, um eine Liste der enthaltenen Cheatsheets zu erhalten. Alle Sheets sind auf dieser Webseite verfügbar:

<https://www.rstudio.com/resources/cheatsheets/>

Folgende Cheatsheets sind besonders empfehlenswert:

- Klassisches R
<http://github.com/rstudio/cheatsheets/blob/main/base-r.pdf>
- Datentransformation mit dplyr
<https://raw.githubusercontent.com/rstudio/cheatsheets/master/data-transformation.pdf>
- Grafiken erstellen mit ggplot
<https://raw.githubusercontent.com/rstudio/cheatsheets/master/data-visualization-2.1.pdf>
- RMarkdown
<https://raw.githubusercontent.com/rstudio/cheatsheets/master/rmarkdown-2.0.pdf>
- RMarkdown references
<https://www.rstudio.com/wp-content/uploads/2015/03/rmarkdown-reference.pdf>

1.7.2 Youtube

Auf Youtube gibt es zahlreiche Anleitungsvideos zu  und RStudio. In der folgenden Liste möchten wir ein paar sehenswerte Videos vorschlagen.

- R und RStudio installieren
https://www.youtube.com/watch?v=X_Mxya2Fis0
- Erste Schritte in RStudio
<https://www.youtube.com/watch?v=tyvEHQszZJs>
- Einfaches Rechnen und Variablen in R
https://www.youtube.com/watch?v=qOVhiOu_ARs
- Zusatzpakete installieren
<https://www.youtube.com/watch?v=YD80LpvkHD8>
- Intro to the Tinyverse (englisch)
<https://www.youtube.com/watch?v=MKwyauo8nSI>

Ebenso gibt es empfehlenswerte Playlists:

- “Statistik in R” von Björn Walther
<https://www.youtube.com/playlist?list=PLJ8d6ddu0frpqxUw6Y7lGV4SoEkI9gIAi>
- “Statistik in R” von Daniela Keller
<https://www.youtube.com/watch?v=dEsm3PHrAHE&list=PLTMQeIvhjir5BEI7DfSAbTmHREI8Zqpnv>
- “Statistische Methoden 1” von Florian Heiss
<https://www.youtube.com/playlist?list=PLVOKtPjp4poWkwg7305ykfWu2mPviYnCq>
- Quarto-Playlist (englisch)
<https://www.youtube.com/playlist?list=PL9HYL-VRXOoRupficE2l5DGgVIzpypTHs>

1.7.3 freie Lehrbücher

Weitere freie -Bücher

- *Einführung in R* von Ellis & Mayer
<https://methodenlehre.github.io/einfuehrung-in-R/>
- *R für Psychos* von Burk & Anton
<https://r-intro.tadaa-data.de/book/>

1.7.4 Internetforen

- *Deutsches R Forum*
<http://forum.r-statistik.de/>



2 Statistik mit R und RStudio

Ebenso wie die Statistik, ist dieses Kapitel in folgende Bereiche unterteilt:

1. Deskriptive Statistik (ab Seite 206)
2. Schließende Statistik (ab Seite 258)

Die Funktionen sind alle in *klassischem* R-Code dargestellt, da sie so auch in einer Tidyverse-Befehlskette stehen würden.

Zu Demonstrationszwecken werden wir in diesem Kapitel auf Beispieldatensätze `epa`, `mma`, `nw` und `pf8` zurückgreifen, die Sie sich aus dem Internet in Ihre R-Session laden können.

```
1  # lade Datensatz "epa"
2  load(url("https://www.produnis.de/R/epa.RData"))
3  # lade Datensatz "mma"
4  load(url("https://www.produnis.de/R/mma.RData"))
5  # lade Datensatz "pf8"
6  load(url("https://www.produnis.de/R/pf8.RData"))
7  # lade Datensatz "nw"
8  load(url("https://www.produnis.de/R/nw.RData"))
9
10 # Wenn Sie das Paket 'jgsbook' installiert haben,
11 # können Sie auch wie folgt auf die Datensätze zugreifen
12 epa <- jgsbook::epa
13 mma <- jgsbook::mma
14 pf8 <- jgsbook::pf8
15 nw  <- jgsbook::nw
```

2.1 Deskriptive Statistik

Zur Beschreibung von Verteilungen eignen sich Tabellen, Lage- und Streuungskenngrößen.

2.1.1 Häufigkeiten

Eine Häufigkeitstabelle wird mit der Funktion `table()` erstellt

```
1 # Daten erzeugen
2 x <- c(1, 3, 5, 7, 7, 7, 6, 34)
3
4 # Häufigkeitstabelle
5 table(x)
```

```
x
 1  3  5  6  7 34
1  1  1  1  1  3  1
```

Für relative Häufigkeiten teilt man einfach durch die Länge der Variable `x`. So ändert sich der Befehl in:

```
1 # relative Häufigkeitstabelle
2 table(x)/length(x)
```

```
x
 1  3  5  6  7 34
0.125 0.125 0.125 0.125 0.375 0.125
```

Kumulierte Häufigkeiten erhält man mit der Funktion `cumsum()`

```
1 # kumulierte Häufigkeitstabelle
2 cumsum(table(x))
```

```
x
 1  3  5  6  7 34
1  2  3  4  7  8
```

Und entsprechend auch kumulierte relative Häufigkeiten

```
1 # kumulierte relative Häufigkeitstabelle
2 cumsum(table(x)/length(x))
```

```
x
  1    3    5    6    7   34
0.125 0.250 0.375 0.500 0.875 1.000
```

Eine weitere Option bietet die Funktion `freq()` aus dem `prettyR`-Paket. Sie erstellt eine Tabelle mit absoluten und relativen Häufigkeiten sowie relativen Häufigkeiten ohne fehlende Wert (NA).

```
1 # Paket "prettyR" installieren
2 install.packages("prettyR", dependencies=TRUE)
3
4 # Paket laden
5 library(prettyR)
6
7 # Häufigkeitstabelle mit absoluten und relative Werten erstellen
8 freq(x)
```

```
Frequencies for x
      7    1    3    5    6   34   NA
      3    1    1    1    1    1    0
%      37.5 12.5 12.5 12.5 12.5 12.5  0
%!NA 37.5 12.5 12.5 12.5 12.5 12.5
```

2.1.2 Lagekenngrößen

Mit dem Befehl `summary()` erhält man eine erste Zusammenfassung der Werteverteilung mit den wichtigsten Lagekenngrößen.

```
1 # Daten erzeugen
2 x <- c(1, 3, 5, 7, 7, 7, 6, 34)
3
4 # Zusammenfassung
5 summary(x)
```

```
Min.    1st Qu.  Median    Mean   3rd Qu.    Max.
1.00    4.50    6.50    8.75    7.00   34.00
```

2.1. DESKRIPTIVE STATISTIK

Die Funktion `describe()` aus dem `psych`-Zusatzpaket liefert noch detailliertere Übersichten:

```
1 # Zusammenfassung
2 psych::describe(x)
```

```
vars n mean sd median trimmed mad min max range
X1 1 8 8.75 10.43 6.5 8.75 1.48 1 34 33

skew kurtosis se
1.69 1.35 3.69
```

Das Funktioniert auch bei kompletten Datensätzen.

```
1 # Zusammenfassung von pf8
2 psych::describe(pf8)
```

```
vars n mean sd median trimmed mad min max
Standort* 1 731 2.85 1.61 2 2.81 1.48 1 5
Alter 2 730 37.00 18.00 28 34.98 12.60 15 86
Geschlecht* 3 725 1.61 0.49 2 1.63 0.00 1 3
Größe 4 730 174.04 9.62 173 173.84 10.38 151 206
Gewicht 5 720 75.53 16.88 73 73.93 16.31 45 175
Bildung* 6 506 4.88 1.77 5 4.98 2.97 1 7
Beruf* 7 731 40.42 37.04 34 38.58 48.93 1 104
Familienstand* 8 729 2.11 1.12 2 1.99 1.48 1 6
Kinder 9 727 0.38 0.79 0 0.18 0.00 0 5
Wohnort* 10 729 1.41 0.49 1 1.39 0.00 1 2
Rauchen* 11 729 1.21 0.40 1 1.13 0.00 1 2
SportHäufig 12 677 2.56 1.88 2 2.41 1.48 0 21
SportMinuten 13 667 62.42 40.14 60 60.47 44.48 0 400
SportWie* 14 656 1.64 0.69 2 1.55 1.48 1 3
SportWarum* 15 454 3.75 1.64 4 3.49 1.48 1 8
LebenZufrieden 16 724 7.91 1.59 8 8.07 1.48 1 11

range skew kurtosis se
Standort* 4 0.28 -1.53 0.06
Alter 71 0.74 -0.70 0.67
Geschlecht* 2 -0.40 -1.75 0.02
Größe 55 0.26 -0.26 0.36
Gewicht 130 1.12 2.40 0.63
Bildung* 6 -0.39 -1.26 0.08
Beruf* 103 0.26 -1.55 1.37
Familienstand* 5 0.83 0.48 0.04
Kinder 5 2.19 4.45 0.03
Wohnort* 1 0.36 -1.87 0.02
Rauchen* 1 1.45 0.11 0.01
SportHäufig 21 1.91 12.75 0.07
SportMinuten 400 1.53 8.64 1.55
```

2.1. DESKRIPTIVE STATISTIK

SportWie*	2	0.62	-0.77	0.03
SportWarum*	7	1.06	0.79	0.08
LebenZufrieden	10	-1.10	1.86	0.06

Die Lagewerte lassen sich auch einzeln bestimmen.

Mit der Funktion `min()` wird das Minimum der Verteilung ausgegeben.

```
1 # Minimum bestimmen
2 min(x)
```

```
[1] 1
```

Dementsprechend liefert `max()` das Maximum.

```
1 # Maximum bestimmen
2 max(x)
```

```
[1] 34
```

Die Spannweite wird mit `range()` abgefragt.

```
1 # Spannweite bestimmen
2 range(x)
```

```
[1] 1 34
```

Das arithmetische Mittel wird mit der Funktion `mean()` bestimmt

```
1 # x-quer bestimmen
2 mean(x)
```

```
[1] 8.75
```

Sind fehlende Werte in der Reihe enthalten, gibt  nur NA zurück

```
1 # mean für "Gewicht" im Datensatz "pf8"
2 mean(pf8$Gewicht)
```

```
[1] NA
```

Beheben kann man dies, indem der Parameter `na.rm` auf `TRUE` gesetzt wird.

```
1 # mean für "Gewicht" im Datensatz "pf8"
2 # entferne NAs
3 mean(pf8$Gewicht, na.rm=TRUE)
```

```
[1] 75.5325
```

Der Median wird mit der Funktion `median()` ermittelt.

```
1 # Median bestimmen
2 median(x)
```

```
[1] 6.5
```

Auch hier müssen enthaltene NAs ausgeblendet werden:

```
1 # Median bestimmen, OHNE NAs
2 median(pf8$Gewicht, na.rm=TRUE)
```

```
[1] 73
```

In den -Hausmitteln existiert keine eigene Funktion zur Bestimmung des Modalwertes. Wir können ihn daher “auf Umwegen” bestimmen. Wir lassen mittels `table()` eine Häufigkeitstabelle der Variable ausgeben. Diese sortieren wir mittels `sort()` absteigend. Der Modus ist dann der erste Wert der sortierten Tabelle.

```

1  # Daten als Häufigkeitstabelle
2  table(x)
3
4  # absteigend sortieren
5  sort(table(x), decreasing=TRUE)
6
7  # Modalwert
8  sort(table(x), decreasing=TRUE)[1]

```

```

7
3

```

Es ist der Wert 7 und er kommt 3 mal vor.

Eine weitere Möglichkeit bietet das Zusatzpaket **statip**. Es enthält die Funktion `mfv()`, mit welcher der Modalwert bestimmt werden kann.

```

1  # Daten als Häufigkeitstabelle
2  library(statip)
3
4  # Modalwert bestimmen
5  mfv(x)

```

```

[1] 7

```

Die vier Quartile werden mit der Funktion `quantile()` ermittelt (ja, der Befehl für Quartile heisst in  `quantile()`).

```

1  # Quartile bestimmen, ohne NAs
2  quantile(x, na.rm=T)

```

```

  0%  25%  50%  75% 100%
1.0  4.5  6.5  7.0 34.0

```

Es lassen sich beliebige Quantile berechnen.

```

1  # Quartile bestimmen, ohne NAs
2  quantile(x, probs = c(.333, .666))

```



```
33.3%  66.6%  
5.331  7.000
```

Und auch die Art der Berechnung lässt sich über den Parameter `type` festlegen. Beispielsweise verwendet `type=6` die Methode von SPSS.

```
1  # Quartile bestimmen, ohne NAs  
2  # so wie SPSS  
3  quantile(x, probs = c(.333, .666), type=6)
```

```
33.3% 66.6%  
4.994 7.000
```

2.1.3 Streuungskenngrößen

Die wichtigste Streuungskenngröße ist die Standardabweichung. Sie wird bestimmt mittels der Funktion `sd()`.

```
1  # Standardabweichung bestimmen  
2  sd(x, na.rm=T)
```

```
[1] 10.43004
```

Die Varianz der Stichprobe wird mit `var()` berechnet.

```
1  # Varianz von x bestimmen  
2  var(x, na.rm=T)
```

```
[1] 108.7857
```

Der Interquartilsabstand (oder einfach nur Quartilsabstand) wird mit der Funktion `IQR()` bestimmt.

```
1 # Quartilsabstand bestimmen
2 IQR(x, na.rm=T)
```

```
[1] 2.5
```

Ebenso wie bei `quantile()` lässt sich über den Parameter `type` die Berechnungsart ändern. Mit `type=6` nutzt R die Methode wie in SPSS:

```
1 # Quartilsabstand bestimmen
2 # Methode wie in SPSS
3 IQR(x, na.rm=T, type=6)
```

```
[1] 3.5
```

2.1.4 Kreuztabellen

Mit der Funktion `table()` lassen sich auch Kreuztabellen erstellen, indem einfach beide Variablen übergeben werden.

```
1 # erzeuge Daten
2 Punktwert <- c(2 , 12 , 7 , 15 , 10 , 4 , 13 , 9 , 16 , 19)
3 Geschlecht <- rep(c("maennlich" , "weiblich") ,5)
4
5 # Kreuztabelle
6 table(Punktwert, Geschlecht)
```

	Geschlecht	
Punktwert	maennlich	weiblich
2	1	0
4	0	1
7	1	0
9	0	1
10	1	0
12	0	1
13	1	0

15	0	1
16	1	0
19	0	1

Ebenso steht die Funktion `xtabs()` zur Verfügung.

```
1 # Kreuztabelle mit "xtabs()"
2 xtabs(~ Punktwert + Geschlecht)
```

	Geschlecht	
Punktwert	maennlich	weiblich
2	1	0
4	0	1
7	1	0
9	0	1
10	1	0
12	0	1
13	1	0
15	0	1
16	1	0
19	0	1

Siehe Kapitel 3.1.1 auf Seite 381 für weitere Beispiele zur Kreuztabelle.

2.1.5 z-Transformation

Die z-Transformation erfolgt mit der Funktion `scale()`.

```
1 # Wertereihe
2 Punktwerte <- c(2 , 12 , 7 , 15 , 10 , 4 , 13 , 9 , 16 , 19)
3
4 # z-Transformation
5 scale(Punktwerte)
```

	[,1]
[1,]	-1.6183421
[2,]	0.2418212
[3,]	-0.6882604

```
[4,] 0.7998702
[5,] -0.1302114
[6,] -1.2463094
[7,] 0.4278376
[8,] -0.3162278
[9,] 0.9858866
[10,] 1.5439356
attr(,"scaled:center")
[1] 10.7
attr(,"scaled:scale")
[1] 5.375872
```

2.1.6 Korrelation

Die allgemeine Funktion zur Berechnung von Korrelationen heisst `cor()`.

Die Korrelation nach Pearson berechnet sich mit der Funktion `cor()` und deren Parameter `method="pearson"`

```
1 # erzeuge Daten
2 Lesetest <- c(2 , 12 , 7 , 15 , 10 , 4 , 13 , 9 , 16 , 19)
3 Rechtschreibung <- c(3 , 14 , 9 , 17 , 12 , 4 , 16 , 12 , 18 , 20)
4
5 # Pearson Maßkorrelationskoeffizient
6 cor(Lesetest, Rechtschreibung, method="pearson")
```

```
[1] 0.9884616
```

Mit der Funktion `cor.test()` kann direkt ein Signifikanztest mitgeführt werden

```
1 # Pearson Maßkorrelationskoeffizient
2 # mit Signifikanztest
3 cor.test(Lesetest, Rechtschreibung, method="pearson")
```

```
Pearson's product-moment correlation
```

```
data: Lesetest and Rechtschreibung
t = 18.458, df = 8, p-value = 7.648e-08
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
```

```
0.9502085 0.9973659
sample estimates:
  cor
0.9884616
```

Sollten fehlende Wert (NA) enthalten sein, gibt der Befehl nur ein NA zurück.

```
1 # Pearson Maßkorrelationskoeffizient
2 # mit Datensatz "pf8", da sind NAs enthalten!!!
3 with(pf8, cor(Alter,Größe, method="pearson"))
```

```
[1] NA
```

Mit dem Parameter `use="complete.obs"` (nutze nur vollständige Beobachtungen) können wir dies beheben.

```
1 # Pearson Maßkorrelationskoeffizient
2 # mit Datensatz "pf8", verwende nur "komplette" Daten (ohne NA)
3 with(pf8, cor(Alter,Größe, method="pearson", use="complete.obs"))
```

```
[1] -0.1067484
```

Für die Korrelationsberechnung nach Spearman (Spearman's ρ) wird der Parameter `method` auf `spearman` gesetzt

```
1 # Spearman's Korrelationsberechnung
2 cor(Lesetest, Rechtschreibung, method="spearman")
```

```
[1] 0.9969651
```

Auch hier müssen NAs mit dem Parameter `use="complete.obs"` unterdrückt werden

```
1 # Spearman's Rho
2 # mit Datensatz "pf8", verwende nur "komplette" Daten (ohne NA)
3 with(pf8, cor(Alter,Größe, method="spearman", use="complete.obs"))
```

```
[1] -0.05920716
```

Auch für Spearman's Rho kann ein Signifikanztest mitgefahren werden, indem `cor.test()` aufgerufen wird.

```
1 # Spearman's Korrelationsberechnung
2 # mit Signifikanztest
3 cor.test(Lesetest, Rechtschreibung, method="spearman")
```

```
Spearman's rank correlation rho

data: Lesetest and Rechtschreibung
S = 0.50076, p-value = 3.698e-10
alternative hypothesis: true rho is not equal to 0
sample estimates:
rho
0.9969651

Warnmeldung:
In cor.test.default(Lesetest, Rechtschreibung, method = "spearman") :
  Kann exakten p-Wert bei Bindungen nicht berechnen
```

Die Warnmeldung kann ignoriert werden, der p-Wert wird in der Ausgabe angezeigt.

Da `cor.test()` ein Objekt der Klasse `Liste` zurückliefert, kann auf die einzelnen Komponenten des Tests zugegriffen werden.

```
1 # schreibe Testergebnis in Variable "test"
2 test <- cor.test(Lesetest, Rechtschreibung, method="spearman")
3
4 # zeige nur p-Wert an
5 test$p.value
```

```
[1] 3.698093e-10
```

2.1. DESKRIPTIVE STATISTIK

1
2

```
# zeige nur Teststatistik an  
test$statistic
```

```
      S  
0.5007599
```

1
2

```
# zeige nur Spearmans's Rho an  
test$estimate
```

```
      rho  
0.9969651
```

2.1.7 Regressionen

Die Funktion zur Berechnung linearer Regressionen ist `lm()` (für “linear model”). Sie folgt der Syntax *x erklärt durch y*, was durch eine Tilde `~` ausgedrückt wird.

```

1  # erzeuge Daten
2  Lesetest <- c(2 , 12 , 7 , 15 , 10 , 4 , 13 , 9 , 16 , 19)
3  Rechtschreibung <- c(3 , 14 , 9 , 17 , 12 , 4 , 16 , 12 , 18 , 20)
4
5  # lineare Regression "Rechtschreibung erklärt durch Lesetest"
6  lm(Rechtschreibung~Lesetest)

```

```

Call:
lm(formula = Rechtschreibung ~ Lesetest)

Coefficients:
(Intercept)      Lesetest
      1.208         1.055

```

Hierbei ist `Intercept` der Schnitt durch die Y-Achse.

Wenn Sie über den Parameter `data` das Datenframe angeben, können Sie auf die Variablen referenzieren, ohne `$` wenden zu müssen.

```

1  # lade Testdaten
2  load(url("https://www.produnis.de/R/nw.RData"))
3
4  # entweder Variablen per "$" referenzieren
5  lm(nw$workyear ~ nw$worknight)
6
7  # oder Datenframe mit angeben
8  lm(workyear ~ worknight, data=nw)
9
10 # speichere Modell in "fit"
11 fit <- lm(workyear~worknight, data=nw)
12 # schaue mit "summary()" an
13 summary(fit)

```

Es ist üblich, Modelle in ein Objekt zu speichern und über die `summary()`-Funktion aufzurufen. Die Informationen der Modellzusammenfassung sind so detaillierter.


```

1 # speichere Modell in "fit"
2 fit <- lm(workyear~worknight, data=nw)
3 # schaue mit "summary()" an
4 summary(fit)

```

```

Call:
lm(formula = workyear ~ worknight, data = nw)

Residuals:
    Min       1Q   Median       3Q      Max
-12.889  -5.743  -2.351   3.660  26.427

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  9.91425     0.74088   13.38  <2e-16 ***
worknight    0.82912     0.05958   13.91  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 7.463 on 274 degrees of freedom
Multiple R-squared:  0.4141,    Adjusted R-squared:  0.4119
F-statistic: 193.6 on 1 and 274 DF,  p-value: < 2.2e-16

```

Außerdem lässt sich das Modell so leicht plotten.

```

1 # plotte das Modell
2 plot(fit)

```

Dies erzeugt insgesamt 4 Plots (Residualplot, Q-Q, Scale-Location und Einfluss). Durch drücken der **[Enter]**-Taste schalten Sie zum nächsten Plot weiter. Sie können aber auch alle 4 Plots in einem ausgeben lassen.

```

1 # plotte alle 4 auf einer Seite
2 par(mfrow = c(2, 2), oma = c(0, 0, 2, 0))
3 plot(fit)

```

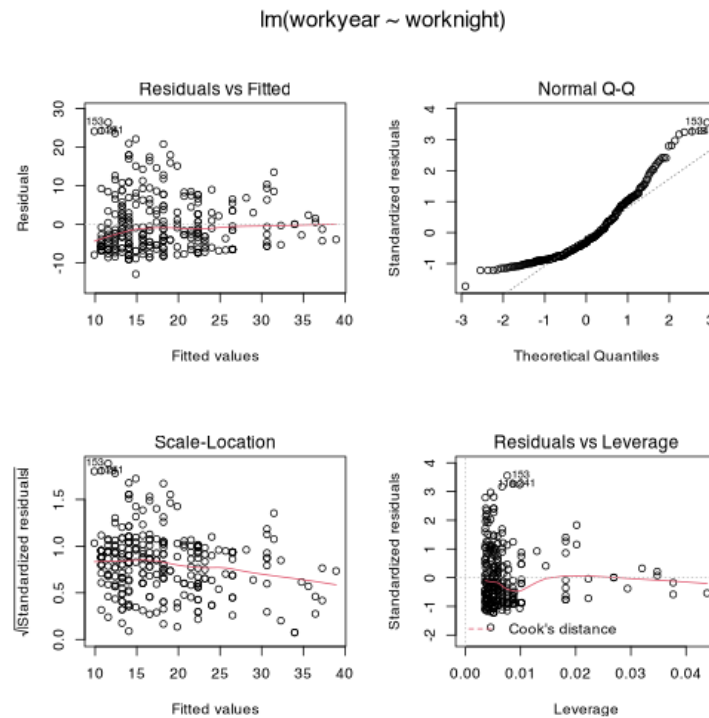


Abb. 2.1

2.1.7.1 multiple lineare Regressionen

Multiple lineare Regressionen werden ebenfalls mit der Funktion `lm()` berechnet. Hierbei werden die erklärenden Variablen per `+` Zeichen aneinandergereiht.

```
1 # multiple lineare Regression
2 # "workyear" erklärt durch "worknight" UND "age"
3 fit <- lm(workyear ~ worknight+age, data=nw)
4 summary(fit)#
```

```
Call:
lm(formula = workyear ~ worknight + age, data = nw)

Residuals:
    Min       1Q   Median       3Q      Max
-14.7077  -3.9748  -0.3066   3.5936  19.9569
```

2.1. DESKRIPTIVE STATISTIK

```
Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -5.99856    1.72557   -3.476 0.000591 ***
worknight    0.57707    0.05714   10.100 < 2e-16 ***
age          0.41614    0.04195    9.921 < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.41 on 273 degrees of freedom
Multiple R-squared:  0.5693,    Adjusted R-squared:  0.5662
F-statistic: 180.5 on 2 and 273 DF,  p-value: < 2.2e-16
```

Ebenfalls können hier die 4 Plots erzeugt werden.

```
1 # plotte alle 4 auf einer Seite
2 par(mfrow = c(2, 2), oma = c(0, 0, 2, 0))
3 plot(fit)
```

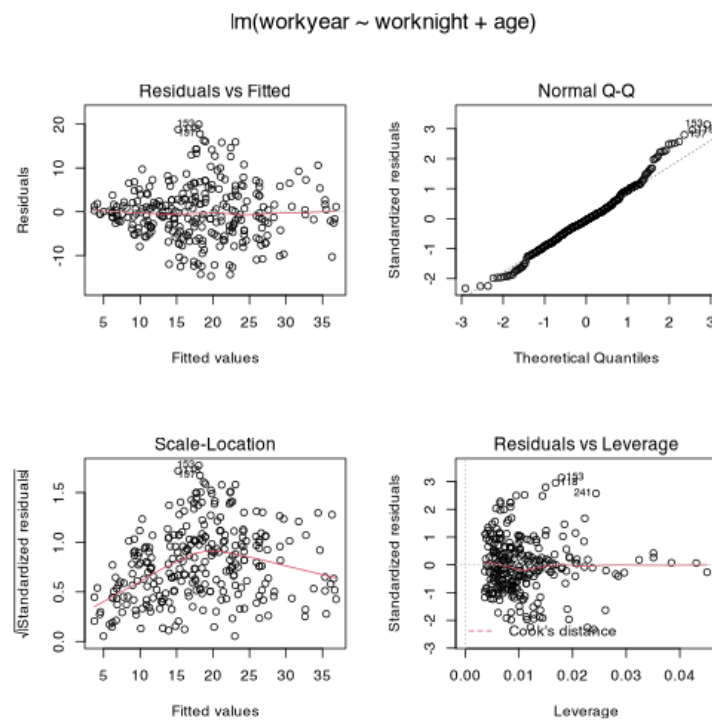


Abb. 2.2

2.1.7.2 Logistische Regression

Als Beispiel dienen Daten der Challenger-Katastrophe¹. Der Ausfall eines oder mehrerer Dichtungsringe in einer der seitlichen Feststoffrakten wurde als Grund der Katastrophe ermittelt. Probleme mit den Dichtungsringen sind auch vorher am Boden aufgetreten. So liegen Erhebungsdaten vor, die den Defekt eines Rings in Zusammenhang mit der Außentemperatur darstellen können: Je kälter es draußen ist, desto höher die Ausfallwahrscheinlichkeit eines Rings. Bedauerlicher Weise wurden diese Daten erst rückwirkend ausgewertet. Diese Daten übertragen wir wie folgt in :

```

1 Temp <- c(66, 67, 68, 70, 72, 75, 76, 79, 53, 58, 70, 75, 67, 67, 69, 70, 73, 76, 78,
2 81, 57, 63, 70)
3 Ausfall <- factor(c(0, 0,0 ,0 ,0 ,0 ,0 ,0 ,1 ,1 ,1 ,1 ,0 ,0 ,0 ,0 ,0 ,0 ,0 ,1 ,1, 1))
4 shuttle <- data.frame(Temp, Ausfall)
5 colnames(shuttle) <- c("Temp", "Ausfall")

```

Die Kodierung des factor **Ausfall** lautet: 0=kein Ausfall | 1=Ausfall

Die Einheit für **Temp** lautet Fahrenheit.

Der Aufruf zur logistischen Regression (**Ausfall** erklärt durch **Temp**) lautet:

```

1 fit <- glm(Ausfall~Temp,data=shuttle, family=binomial)
2 fit

```

Dies liefert die Ausgabe

```

Call:  glm(formula = Ausfall ~ Temp, family = binomial, data = shuttle)

Coefficients:
(Intercept)      Temp
    15.0429    -0.2322

Degrees of Freedom: 22 Total (i.e. Null);  21 Residual
Null Deviance:      28.27
Residual Deviance: 20.32    AIC: 24.32

```

Die Odds Ratio (OR) für $\beta_{Temp=-0,2322}$ errechnet sich mit der **exp()**-Funktion:

¹Am 28. Januar 1986 explodierte die Raumfähre Challenger 73 Sekunden nach dem Start.

```
1 exp(-0.2322)
```

```
[1] 0.7927875
```

Dies liefert das Ergebnis $OR = 0.7927875$. Die Interpretation lautet: Mit jedem höheren Grad an Temperatur ändert sich die Chance auf einen Ausfall um den Faktor 0.7927. Das bedeutet, dass die Chancen eines Ausfalls niedriger werden, je höher die Außentemperatur steigt.

In der Nacht vor dem Challengerstart konnte eine Temperatur von 37° Fahrenheit gemessen werden. Auf Grundlage der logistischen Regression lässt sich nun die Ausfallwahrscheinlichkeit bei 37° Außentemperatur berechnen

```
1 predict(fit, data.frame(Temp=37), type="response")
```

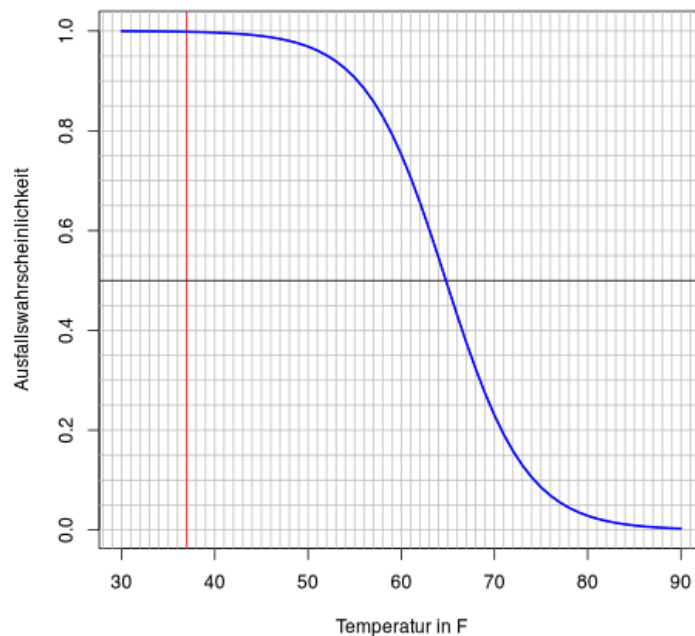
```
[1] 0.9984243
```

Das Modell sagt bei einer Außentemperatur von 37° eine Ausfallwahrscheinlichkeit von 99,84% voraus.

Jetzt kann man dies für jede mögliche Temperatur durchmachen, und schon erhält man dieses schöne Plot (Abbildung 2.3).

```
1 y <- predict(fit, data.frame(Temp=30:90), type="response")
2 plot(30:90, y, type="l", ylab="Ausfallswahrscheinlichkeit", xlab="Temperatur in F")
3 abline(v=seq(20,100,1), h=seq(-0.2,1.2,0.05),col="gray") #Mathegitter
4 abline(v=37, col="red") # 37-Grad
5 abline(h=0.5, col="black") # Mittellinie
6 points(30:90, y, type="l",lwd=2, col="blue") # Schätzwerte in "blau"
```

Der Befehl erzeugt Abbildung 2.3.

*Abb. 2.3*

2.1.7.3 Ordinale Regression

Zur Bestimmung des Einflusses von Prädiktoren auf ein ordinale Ziel stehen verschiedene Modelle der ordinalen Regression zur Verfügung. Namentlich werden hier vor allem “Proportional Odds Modelle” (siehe Seite 226) und “Continuation Ratio Modelle” (siehe Seite 233) verwendet. Eine verständliche Einführung in ordinale Modelle findet sich im Open Access Artikel von große Schlarmann und Galatsch [2].

Für die ordinalen Regressionen verwenden wir das -Paket **VGAM**. Für die Berechnungen laden wir mit `load()` einen Beispieldatensatz aus dem Internet:

```
1 # VGAM installieren
2 install.packages("VGAM", dependencies = T)
3
4 # Test-Datensatz aus dem Internet laden
5 load(url("http://www.produnis.de/R/OrdinalSample.RData"))
```

```

6
7   # kopiere Originaldaten für Testzwecke
8   mydata <- ordinalSample
9
10  # zeigen den Datensatz an
11  head(mydata)

```

	Konflikt	Zufriedenh	Geschlecht	Stimmung
1	4.2	2.50	1	maessig
6	1.8	3.00	1	sehr gut
15	3.4	1.75	1	maessig
16	4.0	2.50	2	maessig
22	2.2	2.50	1	gut
23	3.4	2.25	1	maessig

Der Test-Datensatz enthält das Datenframe `ordinalSample`. Dieses besteht aus den vier Variablen `Konflikt`, `Zufriedenh`, `Geschlecht` und `Stimmung`.

Die Variable `Stimmung` dient hier als ordinales Ziel mit den Ausprägungen `schlecht < maessig < gut < sehr gut`.

Die Variable `Konflikt` dient als Prädiktor. Sie gibt an, wieviele Konflikte derzeit im Arbeitsleben vorliegen.

Die Variable `Zufriedenh` soll ebenfalls Prädiktor sein. Sie beschreibt, wie zufrieden Probanden mit ihrem Job sind.

Wir wollen nun wissen, wie stark `Stimmung` durch `Konflikt` und `Zufriedenh` beeinflusst wird. Das heisst, in unserem Modell soll die Variable `Stimmung` durch `Konflikt` und `Zufriedenh` beschrieben werden.

Proportional Odds Modelle errechnet sich leicht mit dem `VGAM`-Paket und der Funktion `vglm()`:

```

1   # aktiviere VGAM
2   library(VGAM)
3
4   # erstelle Modell

```

2.1. DESKRIPTIVE STATISTIK

```
5 pom <- vglm(Stimmung ~ Konflikt + Zufriedenh, data = mydata, family =  
cumulative(parallel = T))  
6  
7 # oder abgekürzt  
8 pom <- vglm(Stimmung ~ Konflikt + Zufriedenh, data = mydata, family = propodds)  
9  
10 # gebe Modellzusammenfassung aus  
11 summary(pom)
```

```
Call:  
vglm(formula = Stimmung ~ Konflikt + Zufriedenh, family = propodds,  
      data = mydata)  
  
Coefficients:  
              Estimate Std. Error z value Pr(>|z|)  
(Intercept):1  0.6210      0.6482   0.958  0.3381  
(Intercept):2 -1.7703      0.6540  -2.707  0.0068 **  
(Intercept):3 -4.0578      0.6759  -6.004 1.93e-09 ***  
Konflikt      -0.5762      0.0970  -5.940 2.86e-09 ***  
Zufriedenh     1.3643      0.2021   6.751 1.47e-11 ***  
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
  
Names of linear predictors: logitlink(P[Y>=2]), logitlink(P[Y>=3]), logitlink(P[Y>=4])  
  
Residual deviance: 920.5493 on 1240 degrees of freedom  
  
Log-likelihood: -460.2746 on 1240 degrees of freedom  
  
Number of Fisher scoring iterations: 5  
  
No Hauck-Donner effect found in any of the estimates  
  
Exponentiated coefficients:  
      Konflikt Zufriedenh  
0.5620549  3.9129275
```

Mit dem Modell können nun weitere Parameter errechnet werden:

```
1 # Koeffizienten  
2 pom.coef <- (coef(summary(pom)))  
3 pom.coef
```

```
              Estimate Std. Error    z value    Pr(>|z|)  
(Intercept):1 0.6209962 0.64821816  0.9580049 3.380603e-01  
(Intercept):2 -1.7702692 0.65404383 -2.7066523 6.796539e-03  
(Intercept):3 -4.0577657 0.67586741 -6.0037895 1.927645e-09  
Konflikt      -0.5761558 0.09700358 -5.9395317 2.858375e-09  
Zufriedenh     1.3642858 0.20208974  6.7508909 1.469400e-11
```


Die Odds Ratio errechnet sich mit der `exp()`-Funktion:

```
1 # Odds Ratio
2 pom.odds =exp(coef(pom))
3 pom.odds
```

(Intercept):1	(Intercept):2	(Intercept):3	Konflikt	Zufriedenh
1.8607808	0.1702871	0.0172876	0.5620549	3.9129275

```
1 # Devianz
2 pom.devi =deviance(pom)
3 pom.devi
```

(Intercept):1	(Intercept):2	(Intercept):3	Konflikt	Zufriedenh
1.8607808	0.1702871	0.0172876	0.5620549	3.9129275

```
1 # AIC
2 pom.aic =AIC(pom)
3 pom.aic
```

```
[1] 930.5493
```

```
1 # logLikelihood
2 pom.ll =logLik(pom)
3 pom.ll
```

```
[1] -460.2746
```

```
1 # 0-Modell (fuer pseudo R^2)
2 p0 =vglm(Stimmung ~ 1, data = mydata, family = propodds)
3 p0.ll = logLik(p0)
```

```
1 # R^2 McFadden
2 pom.mcfad =as.vector(1 - (pom.ll/p0.ll))
3 pom.mcfad
```

```
[1] 0.1240613
```

```
1 # R2 Cow&Snell
2 N=length(mydata[, 1]) # Anzahl der Fälle
3 pom.cox = as.vector(1 - exp((2/N) * (p0.11 - pom.11)))
4 pom.cox
```

```
[1] 0.2696034
```

```
1 # R2 Nagelkerke
2 pom.nagel = as.vector((1 - exp((2/N) * (p0.11 - pom.11)))/(1 - exp(p0.11)^(2/N)))
3 pom.nagel
```

```
[1] 0.2928789
```

Das Proportional Odds Modell geht von der “equal slopes assumption” (auch: “proportional odds assumption”) aus. Diese Annahme muss geprüft werden, bevor das Modell als gültig angesehen werden kann. Dies geht mit dem VGAM-Paket recht einfach. Der Befehl

```
1 vglm(Stimmung ~ Konflikt+Zufriedenh, data=mydata, family=propodds)
```

ist eine Abkürzung für

```
1 vglm(Stimmung ~ Konflikt+Zufriedenh, data=mydata, family=cumulative(parallel=T))
```

Der Parameter `parallel=TRUE/FALSE` stellt ein, ob das Modell mit equal slopes (=TRUE), oder ohne equal slopes assumption (=FALSE) erstellt werden soll. Zur Überprüfung der “Equal Slopes Assumption” erstellt man jeweils ein TRUE und ein FALSE-Modell, und führt dann einen Likelihood-Test durch. Die Nullhypothese lautet, dass equal slopes vorliegen.

```

1  # Modell OHNE equal slopes
2  npom =vglm(Stimmung ~ Konflikt+Zufriedenh, data=mydata, family=cumulative(parallel=F))
3
4  # log-likelihood-Test auf Equal Slopes Assumption
5  lrtest(pom, npom) # Test

```

Likelihood ratio test

```

Model 1: Stimmung ~ Konflikt + Zufriedenh
Model 2: Stimmung ~ Konflikt + Zufriedenh
  #Df LogLik Df  Chisq Pr(>Chisq)
1 1240 -460.27
2 1236 -456.46 -4  7.6202    0.1065

```

Der Test ist mit $p = 0,1065$ nicht signifikant. Das heisst, es liegen equal slopes vor. Das Modell darf also verwendet werden. Alternativ kann dieser Test auch auf die Devianz der Modelle bestimmt werden:

```

1  # Devianz-Test auf Equal Slopes Assumption
2  pom.pdevi = (1 - pchisq(deviance(pom) - deviance(npom), df = df.residual(pom) -
3  df.residual(npom)))
  pom.pdevi

```

```
[1] 0.106526
```

Ist der Test signifikant, liegen keine equal slopes vor. Hier kann ein Partial Proportional Odds Modell erstellt werden, welches die equal slopes nur für bestimmte Prädiktoren annimmt.

Mit dem VGAM-Paket kann recht einfach bestimmt werden, wie ein solches Modell erstellt werden soll. Hierfür erweitert man die `parallel`-Option wie folgt:

```

1  # Equal Slopes NUR für 'Konflikt'
2  ppom =vglm(Stimmung ~ Konflikt+Zufriedenh, data=mydata, family=cumulative(parallel=T ~
3  Konflikt - 1))
  ppom

```

```
Call:
vglm(formula = Stimmung ~ Konflikt+Zufriedenh, family=cumulative(parallel=T ~ Konflikt
↪ - 1), data=mydata)

Coefficients:
(Intercept):1 (Intercept):2 (Intercept):3      Konflikt  Zufriedenh:1  Zufriedenh:2
↪ Zufriedenh:3
  0.6614816    1.5373699    2.7337389    0.5705897   -1.9684047   -1.2805318
↪ -0.8865225

Degrees of Freedom: 1245 Total; 1238 Residual
Residual deviance: 914.4929
Log-likelihood: -457.2464
```

```
1  # Nochmals EqualSlopes NUR für 'Konflikt'
2  ppom =vglm(Stimmung ~ Konflikt+Zufriedenh, data=mydata, family=cumulative(parallel=F
3  ~Zufriedenh))
   ppom
```

```
Call:
vglm(formula = Stimmung ~ Konflikt+Zufriedenh, family=cumulative(parallel=F ~
Zufriedenh), data = mydata)

Coefficients:
(Intercept):1 (Intercept):2 (Intercept):3      Konflikt  Zufriedenh:1  Zufriedenh:2
↪ Zufriedenh:3
  0.6614816    1.5373699    2.7337389    0.5705897   -1.9684047   -1.2805318
↪ -0.8865225

Degrees of Freedom: 1245 Total; 1238 Residual
Residual deviance: 914.4929
Log-likelihood: -457.2464
```

parallel=T Konflikt-1 bedeutet, dass equal slopes nur für Konflikt angenommen wird.

parallel=F Zufriedenh bedeutet, dass equal slopes nur für Zufriedenh **nicht** angenommen wird.

2.1. DESKRIPTIVE STATISTIK

Beide Befehle bedeuten also das selbe. Daher ist die R-Ausgabe bei beiden Befehlen gleich.

Eine Koeffizientenübersicht erhält man per

```
1 ppom.ce =coef(summary(ppom))
2 ppom.ce
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept):1	0.6614816	0.85866412	0.7703613	4.410856e-01
(Intercept):2	1.5373699	0.71997824	2.1353005	3.273647e-02
(Intercept):3	2.7337389	0.98084390	2.7871294	5.317723e-03
Konflikt	0.5705897	0.09715456	5.8730094	4.279541e-09
Zufriedenh:1	-1.9684047	0.34535219	-5.6997024	1.200167e-08
Zufriedenh:2	-1.2805318	0.23585229	-5.4293805	5.654999e-08
Zufriedenh:3	-0.8865225	0.32542569	-2.7241933	6.445877e-03

Mit einem kleinen Script können Konfidenzintervalle und andere Werte ausgegeben werden:

```
1 get.ci <- function(x){
2   back <- cbind( x[1], # estimate
3                 (x[1] - (1.96 * x[2])), #lCI
4                 (x[1] + (1.96 * x[2])), #uCI
5                 x[2], x[3], # SD und z
6                 (2*(1 -pnorm(abs(x[3])))) ),# p-wert
7                 exp(x[1]))
8   colnames(back) <- c("Estimate", "lCI", "uCI", "SD", "z", "p-value", "OR")
9   return(back)
10 }
```

Der Aufruf erfolgt z.B. so:

```
1 get.ci(as.numeric(ppom.ce[4, ]))
```

	Estimate	lCI	uCI	SD	z	p-value	OR
[1,]	0.5705897	0.3801667	0.7610126	0.09715456	5.873009	4.279541e-09	1.76931

Continuation Ratio Model lassen sich ebenfalls mit der Funktion `vglm()` erstellen. Um ein Continuation Ratio Modell zu rechnen, muss der Parameter `family` auf `cratio` gesetzt werden.

```

1  # aktiviere VGAM
2  library(VGAM)
3
4  # CR-Modell MIT PO-Assumption (Vorwärts)
5  crm =vglm(Stimmung ~ Konflikt + Zufriedenh, data = mydata, family = cratio(parallel = T,
6  reverse = F))
summary(crm)

```

```

Call:
vglm(formula = Stimmung ~ Konflikt + Zufriedenh, family = cratio(parallel = T,
  reverse = F), data = mydata)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept):1  0.92610    0.57405   1.613   0.1067
(Intercept):2 -1.14132    0.57924  -1.970   0.0488 *
(Intercept):3 -3.01660    0.60706  -4.969 6.72e-07 ***
Konflikt       -0.53645    0.08639  -6.209 5.32e-10 ***
Zufriedenh     1.16010    0.17944   6.465 1.01e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Names of linear predictors: logitlink(P[Y>1|Y>=1]), logitlink(P[Y>2|Y>=2]),
↪ logitlink(P[Y>3|Y>=3])

Residual deviance: 924.3824 on 1240 degrees of freedom

Log-likelihood: -462.1912 on 1240 degrees of freedom

Number of Fisher scoring iterations: 6

No Hauck-Donner effect found in any of the estimates

```

Dies berechnet standardmäßig die Vorwärts-Methode. Möchte man die Rückwärts-Methode rechnen, setzt man den Parameter `reverse` auf `TRUE`.

```

1  # CR-Modell MIT PO-Assumption (Rückwärts)
2  crm =vglm(Stimmung ~ Konflikt + Zufriedenh, data = mydata, family = cratio(parallel = T,
3  reverse = TRUE))
summary(crm)

```

```

Call:
vglm(formula = Stimmung ~ Konflikt + Zufriedenh, family = cratio(parallel = T,
  reverse = TRUE), data = mydata)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept):1  0.12822    0.57259   0.224 0.822809
(Intercept):2  2.08751    0.58094   3.593 0.000326 ***
(Intercept):3  4.04245    0.60636   6.667 2.62e-11 ***
Konflikt       0.45234    0.08488   5.329 9.88e-08 ***
Zufriedenh    -1.25631    0.18108  -6.938 3.98e-12 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Names of linear predictors: logitlink(P[Y<2|Y<=2]), logitlink(P[Y<3|Y<=3]),
↪ logitlink(P[Y<4|Y<=4])

Residual deviance: 920.4627 on 1240 degrees of freedom

Log-likelihood: -460.2314 on 1240 degrees of freedom

Number of Fisher scoring iterations: 5

No Hauck-Donner effect found in any of the estimates

```

Hat man die passende Methode gewählt, folgen die selben Befehle wie beim Proportional Odds Modell (siehe Seite 226). Zunächst muss überprüft werden, ob equal slopes vorliegen:

```

1  # CR-Modell OHNE PO-Assumption (Rückwärts)
2  ncrm =vglm(Stimmung ~ Konflikt + Zufriedenh, data = mydata, family = cratio(parallel =
3  F, reverse = T))
   summary(ncrm)

```

```

Call:
vglm(formula = Stimmung ~ Konflikt + Zufriedenh, family = cratio(parallel = F,
  reverse = T), data = mydata)

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept):1  2.3649    1.1740   2.014 0.043969 *
(Intercept):2  1.9998    0.8117   2.464 0.013753 *
(Intercept):3  2.4670    1.1190   2.205 0.027474 *
Konflikt:1     0.1465    0.1811   0.809 0.418633
Konflikt:2     0.4864    0.1194   4.073 4.65e-05 ***
Konflikt:3     0.6308    0.1689   3.736 0.000187 ***
Zufriedenh:1   -1.8008    0.3919  -4.596 4.32e-06 ***
Zufriedenh:2   -1.2602    0.2578  -4.889 1.01e-06 ***

```

2.1. DESKRIPTIVE STATISTIK

```
Zufriedenh:3    -0.8351      0.3382   -2.469 0.013543 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Names of linear predictors: logitlink(P[Y<2|Y<=2]), logitlink(P[Y<3|Y<=3]),
↪ logitlink(P[Y<4|Y<=4])

Residual deviance: 914.7924 on 1236 degrees of freedom

Log-likelihood: -457.3962 on 1236 degrees of freedom

Number of Fisher scoring iterations: 6

No Hauck-Donner effect found in any of the estimates
```

```
1  # loglikelihood test auf equal slopes assumption
2  lrtest(ncrm, crm) # Test
```

```
Likelihood ratio test

Model 1: Stimmung ~ Konflikt + Zufriedenh
Model 2: Stimmung ~ Konflikt + Zufriedenh
  #Df LogLik Df  Chisq Pr(>Chisq)
1 1236 -457.40
2 1240 -460.23  4  5.6703    0.2252
```

Der Test ist nicht signifikant ($p=0,23$). Das bedeutet, dass die Annahme der equal slopes für die hier getesteten Rückwärts-Modelle beibehalten werden kann.

Nun können weitere Modellparameter bestimmt werden:

```
1  # 0-Modell (fuer pseudo R^2) Vorbereitungen
2  c0 = vglm(Stimmung ~ 1, data = mydata, family = cratio(parallel = T))
3  c0.ll = logLik(c0)
4  crm.ll = logLik(crm)
5  N = length(mydata[, 1]) # Anzahl der Fälle
6
7  # R^2 McFadden
8  crm.mcfad = as.vector(1 - (crm.ll/c0.ll))
9  crm.mcfad
```

```
[1] 0.1241436
```



```

1  # R2 Cox&Snell
2  crm.cox =as.vector(1 - exp((2/N) * (c0.11 - crm.11)))
3  crm.cox

```

```
[1] 0.2697557
```

```

1  # R2 Nagelkerke
2  crm.nagel =as.vector(((1 - exp((2/N) * (c0.11 - crm.11)))/(1 - exp(c0.11)^(2/N)))
3  crm.nagel

```

```
[1] 0.2930443
```

```

1  # Devianz
2  crm.devi =deviance(crm)
3  crm.devi

```

```
[1] 920.4627
```

```

1  # AIC
2  crm.aic =AIC(crm)
3  crm.aic

```

```
[1] 930.4627
```

Mit unserer Funktion `get.ci()` von Seite 232 können wir uns die Modellparameter der Koeffizienten ausgeben lassen

```

1  # Konfidenzintervalle
2  crm.ce =coef(summary(crm))
3  crm.ce

```

2.1. DESKRIPTIVE STATISTIK

	Estimate	Std. Error	z value	Pr(> z)
(Intercept):1	0.1282227	0.57259420	0.2239329	8.228095e-01
(Intercept):2	2.0875126	0.58094099	3.5933299	3.264788e-04
(Intercept):3	4.0424508	0.60636000	6.6667504	2.615293e-11
Konflikt	0.4523353	0.08488357	5.3288915	9.881398e-08
Zufriedenh	-1.2563060	0.18108066	-6.9378255	3.981811e-12

```
1 get.ci(as.numeric(crm.ce[4, ]))
```

	Estimate	lCI	uCI	SD	z	p-value	OR
[1,]	0.4523353	0.2859635	0.6187071	0.08488357	5.328892	9.881398e-08	1.571979

```
1 get.ci(as.numeric(crm.ce[5, ]))
```

	Estimate	lCI	uCI	SD	z	p-value	OR
[1,]	-1.256306	-1.611224	-0.9013879	0.1810807	-6.937825	3.981704e-12	0.2847038

2.1.8 Überlebenszeitanalysen

2.1.8.1 Kaplan-Meier-Analysen

Zur Überlebenszeitanalyse nach dem Kaplan-Meier-Verfahren steht das Zusatzpaket `survival` zur Verfügung. Zum Ausprobieren der Funktionen liegt dem Paket der Datensatz `ovarian` bei.

```

1  # Beschreibung des Datensatzes lesen
2  ?survival::ovarian
3
4  # speichere Testdatensatz "ovarian" in Objekt "ov"
5  ov <- survival::ovarian
6  head(ov)

```

	futime	fustat	age	resid.ds	rx	ecog.ps
1	59	1	72.3315	2	1	1
2	115	1	74.4932	2	1	1
3	156	1	66.4658	2	1	2
4	421	0	53.3644	2	2	1
5	431	1	50.3397	2	1	1
6	448	0	56.4301	1	1	2

Bevor wir mit der Analyse beginnen, sollten wir den Datensatz noch aufbereiten. Wir überführen alle kategorialen Variablen in einen factor.

```

1  ### Datensatz aufbereiten
2  ## alle kategorialen Variablen
3  # rx ist die Behandlungsgruppe
4  ov$rx <- factor(ov$rx,
5                  levels = c("1", "2"),
6                  labels = c("A", "B"))
7
8  # resid.ds sind Nebenerkrankungen
9  ov$resid.ds <- factor(ov$resid.ds,
10                       levels = c("1", "2"),
11                       labels = c("nein", "ja"))
12
13 # ecog.ps ist der ECOG-Status
14 ov$ecog.ps <- factor(ov$ecog.ps,
15                      levels = c("1", "2"),
16                      labels = c("gut", "schlecht"))

```

Als ersten Analyseschritt erzeugen wir mit der Funktion `Surv()` ein "Survival-objekt". Der Funktion muss eine Zeitvariable mit der Beobachtungszeit sowie eine dichotome Statusvariable mit dem Ereignis (0 = nicht eingetreten; 1 =

eingetreten) übergeben werden. Im `ovarian`-Datensatz ist die Zeit in `futime` und der Status in `fustat` gespeichert. Der Funktionsaufruf lautet dementsprechend:

```
1 # Survivalobjekt erzeugen
2 survObjekt <- survival::Surv(time=ov$futime, event=ov$fustat)
```

Mit der Funktion `survfit()` wird nun das Kaplan-Meier-Modell gefittet. Der Funktion muss das eben erstellte Survivalobjekt übergeben werden.

```
1 ### Kaplan-Meier
2 # ~1 bedeutet "Gesamtmodell", ohne Gruppierung
3 km_fit <- survival::survfit(survObjekt ~ 1, data=ov)
```

Das Modell kann als Tabelle oder Plot ausgegeben werden.

```
1 ## Tabelle anschauen
2 # "survival"-Spalte ist die Überlebenswahrscheinlichkeit
3 summary(km_fit)
```

Call: `survfit(formula = survObjekt ~ 1, data = ov)`

time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
59	26	1	0.962	0.0377	0.890	1.000
115	25	1	0.923	0.0523	0.826	1.000
156	24	1	0.885	0.0627	0.770	1.000
268	23	1	0.846	0.0708	0.718	0.997
329	22	1	0.808	0.0773	0.670	0.974
353	21	1	0.769	0.0826	0.623	0.949
365	20	1	0.731	0.0870	0.579	0.923
431	17	1	0.688	0.0919	0.529	0.894
464	15	1	0.642	0.0965	0.478	0.862
475	14	1	0.596	0.0999	0.429	0.828
563	12	1	0.546	0.1032	0.377	0.791
638	11	1	0.497	0.1051	0.328	0.752

Die Spalte `survival` gibt die entsprechenden Überlebenswahrscheinlichkeiten an.

```
1 # erzeuge Testwerte
2 ## plot() (sehr hässlich)
3 plot(km_fit)
```

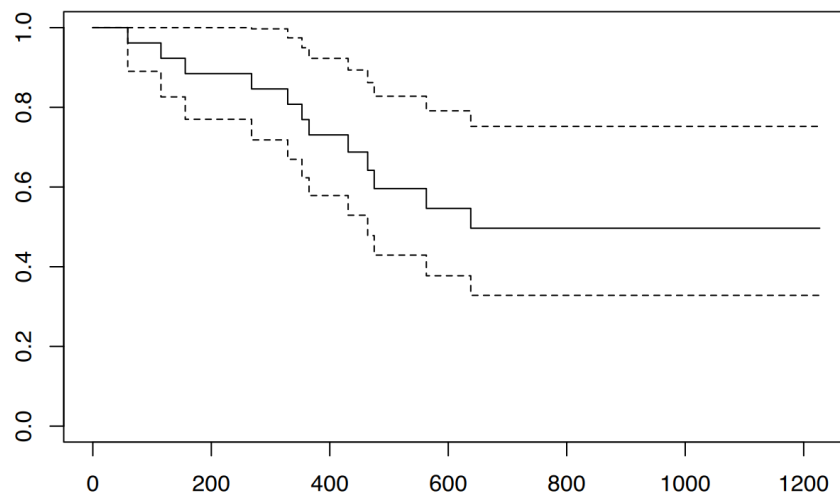


Abb. 2.4

Etwas hübschere Plots lassen sich mit der Funktion `autoplot()` erzeugen. Damit die Funktion die Modelldaten versteht, muss das Zusatzpaket `ggfortify` geladen werden.

```
1 # etwas hübscher
2 library(ggfortify)
3 autoplot(km_fit)
```

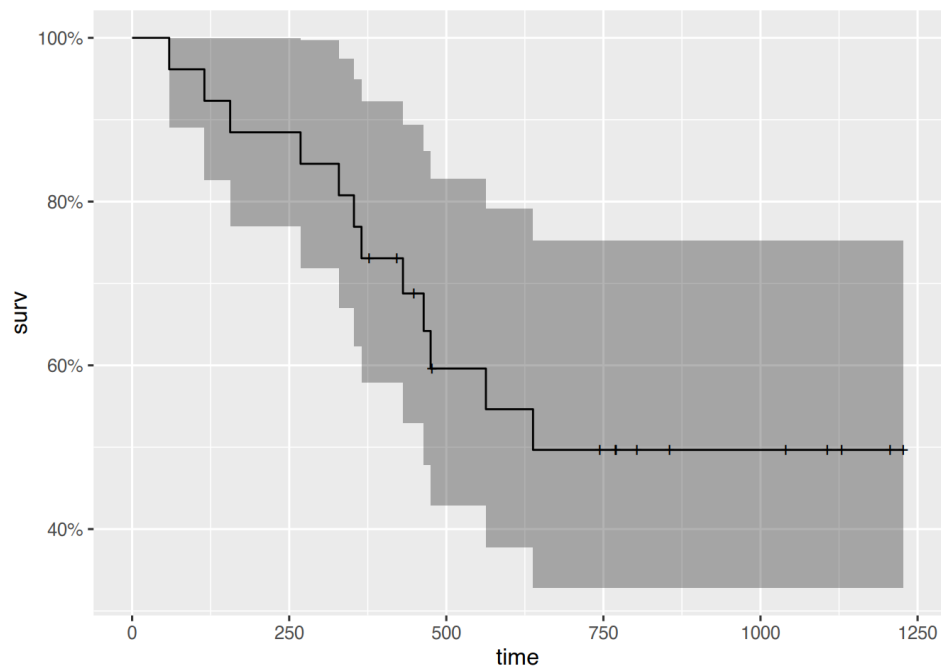


Abb. 2.5

Darüber hinaus bietet das Paket `survminer` die Funktion `ggsurvplot()`:

```
1 ## survminer ggsurvplot  
2 survminer::ggsurvplot(km_fit)
```

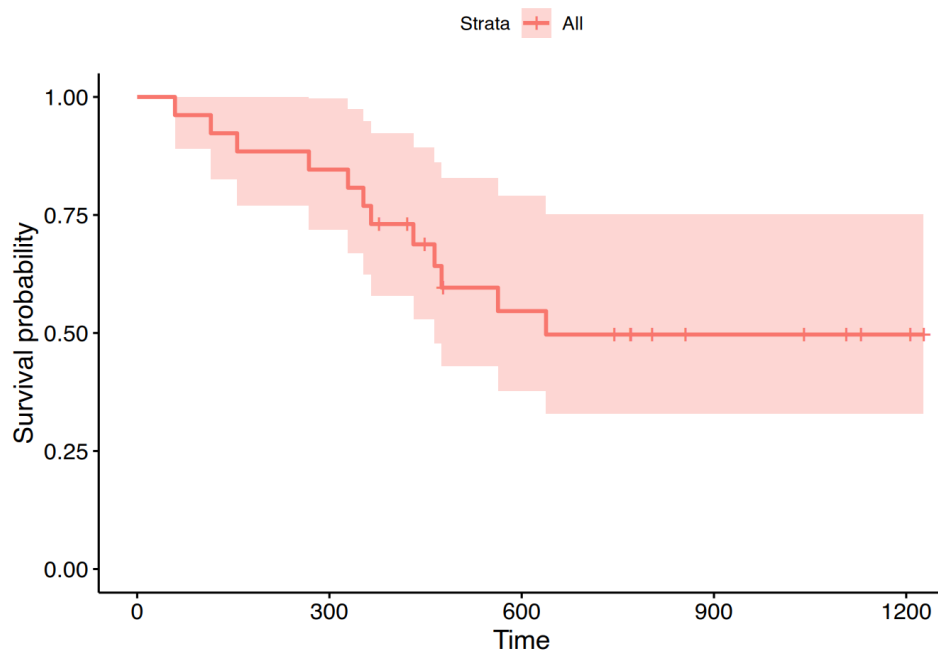


Abb. 2.6

Das Modell lässt sich auch für Gruppenvergleiche erstellen. Die Variable `ov$rx` unterteilt den Datensatz in zwei Behandlungsgruppen. Das Überlebensmodell pro Gruppe lässt sich wie folgt erzeugen:

```
1 ### Gruppierungen
2 # nach Behandlungsgruppe "rx" gruppieren
3 km_fit <- survival::survfit(survObjekt ~ rx, data=ov)
```

Mittels `summary()` werden die Überlebenstabellen pro Gruppe ausgegeben.

```
1 ### Gruppierungen
2 # nach Behandlungsgruppe "rx" gruppieren
3 km_fit <- survival::survfit(survObjekt ~ rx, data=ov)
```

```
Call: survfit(formula = survObjekt ~ rx, data = ov)
```

rx=A						
time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
59	13	1	0.923	0.0739	0.789	1.000

2.1. DESKRIPTIVE STATISTIK

115	12	1	0.846	0.1001	0.671	1.000
156	11	1	0.769	0.1169	0.571	1.000
268	10	1	0.692	0.1280	0.482	0.995
329	9	1	0.615	0.1349	0.400	0.946
431	8	1	0.538	0.1383	0.326	0.891
638	5	1	0.431	0.1467	0.221	0.840

rx=B						
time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
353	13	1	0.923	0.0739	0.789	1.000
365	12	1	0.846	0.1001	0.671	1.000
464	9	1	0.752	0.1256	0.542	1.000
475	8	1	0.658	0.1407	0.433	1.000
563	7	1	0.564	0.1488	0.336	0.946

Im Plot werden die Kaplan-Meier-Kurven beider Gruppen angezeigt.

```
1 # plot (hässlich)
2 plot(km_fit, conf.int=T) # mit Konfidenzintervallen
```

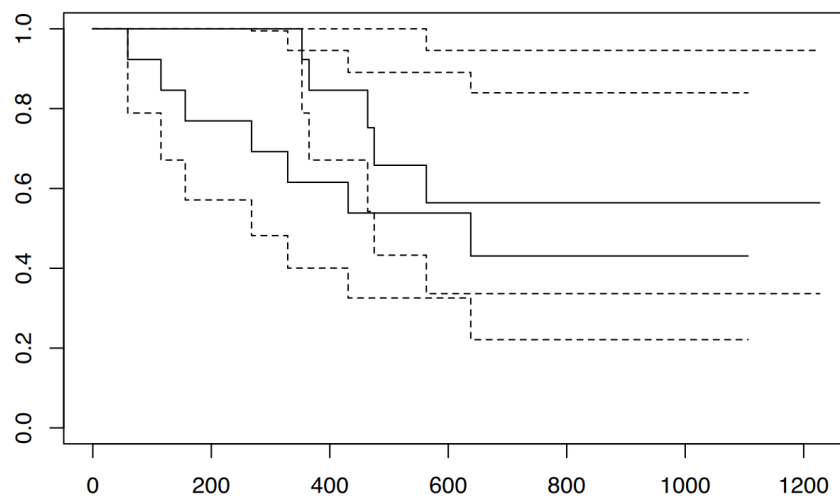


Abb. 2.7

Das Diagramm kann noch etwas aufgehübscht werden.

```
1 # plot (hübscher)
2 plot(km_fit, conf.int=TRUE, col=c("red", "blue"), yscale=100,
3       xlab = "Überlebenszeit in Tagen", ylab = "Prozent Überleben")
4 legend("bottomright", title="Gruppe", c("1", "2"),
5       fill=c("red", "blue"))
```

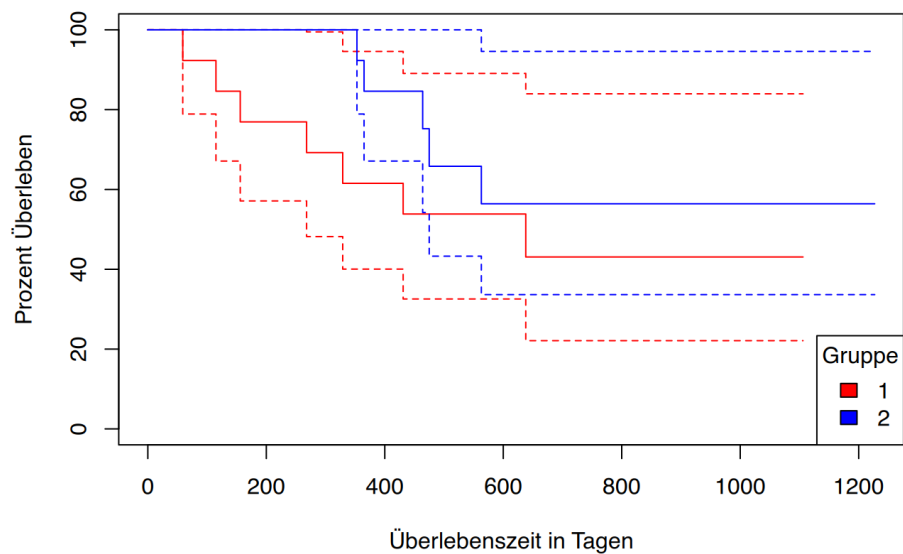


Abb. 2.8

Die Funktion `ggsurvplot()` liefert auf Wunsch den p-Wert des Gruppenunterschieds mit:

```
1 ## survminer ggsurvplot mit p-Wert
2 survminer::ggsurvplot(km_fit, pval=TRUE, conf.int = TRUE)
```

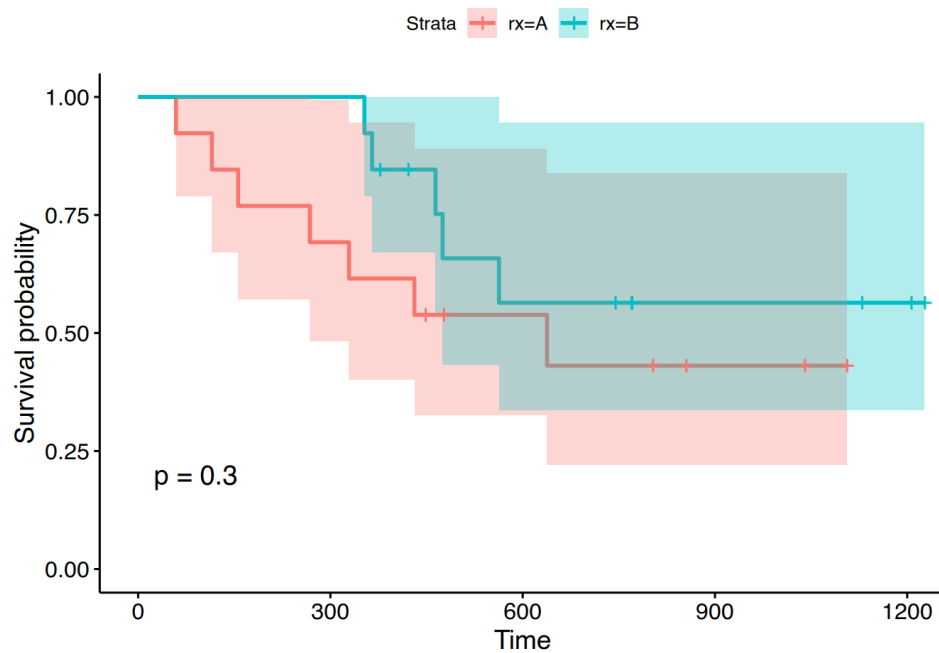


Abb. 2.9

Der p-Wert lässt sich mit dem Lograng-Test auch selbst berechnen. Hierfür steht die Funktion `survdif()` zur Verfügung.

```

1 # p-Wert selbst berechnen
2 ## Lograng-Test
3 survival::survdif(survObjekt ~ rx, data=ov)

```

```

Call:
survival::survdif(formula = survObjekt ~ rx, data = ov)

```

	N	Observed	Expected	(O-E) ² /E	(O-E) ² /V
rx=A	13	7	5.23	0.596	1.06
rx=B	13	5	6.77	0.461	1.06

Chisq= 1.1 on 1 degrees of freedom, p= 0.3

2.1.8.2 Cox-Regression

Neben der Kaplan-Meier-Analyse können die Daten auch per Cox-Regression analysiert werden. Der Vorteil besteht darin, dass Prädiktoren verschiedensten Datenniveaus in die Analyse integriert werden können. Das Paket `survival` bietet hierzu die Funktion `coxph()`.

```
1 cox <- survival::coxph(survObjekt ~ rx+resid.ds+age+ecog.ps, data=ov)
2 # Modell ausgeben
3 cox
```

```
Call:
survival::coxph(formula = survObjekt ~ rx + resid.ds + age +
  ecog.ps, data = ov)

              coef exp(coef) se(coef)      z      p
rx          -0.91450   0.40072  0.65332 -1.400 0.16158
resid.ds     0.82619   2.28459  0.78961  1.046 0.29541
age           0.12481   1.13294  0.04689  2.662 0.00777
ecog.ps      0.33621   1.39964  0.64392  0.522 0.60158

Likelihood ratio test=17.04 on 4 df, p=0.001896
n= 26, number of events= 12
```

Die Spalte `exp(coef)` entspricht der Hazard-Ratio, mit welcher Richtung und Stärke des jeweiligen Einflusses interpretiert werden kann.

Das Modell kann mittels `ggforest()` geplottet werden:

```
1 # Forestplot der Hazard-Ratio
2 survminer::ggforest(cox, data=ov)
```

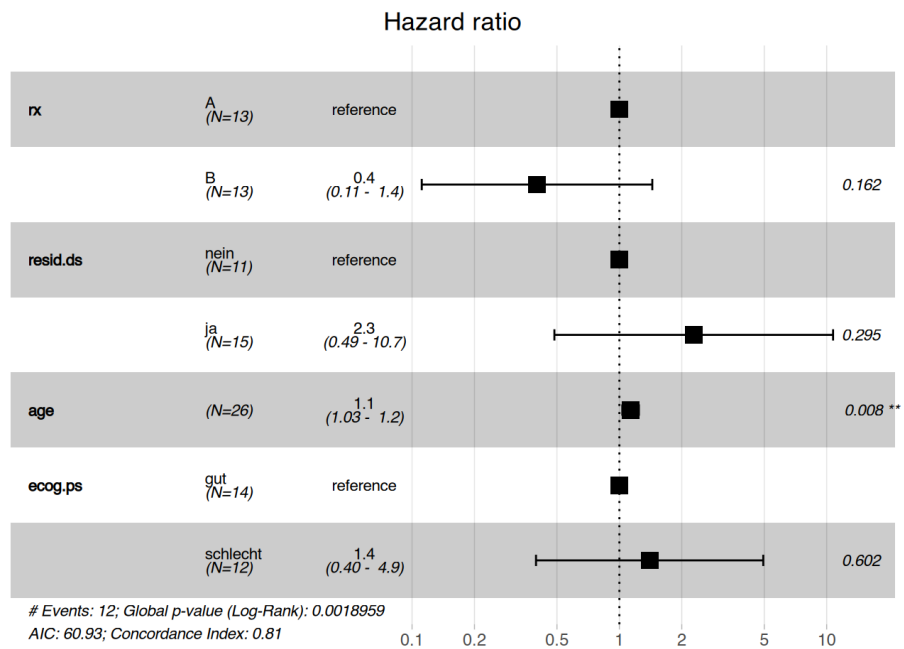


Abb. 2.10

2.1.9 Faktorenanalyse

Die beiden am Häufigsten genutzten Verfahren sind die *Hauptkomponentenanalyse* (PCA) und die *Hauptachsenanalyse* (PAF). Sie unterscheiden sich in den Varianzanteilen, die in den Items enthalten sein können. Die PCA versucht, die *gesamte* Varianz auf Faktoren zurückzuführen, die PAF die *gemeinsame* Varianz der Items. In psychologischen Anwendungen (= am Menschen) wird daher eher die PAF verwendet.

2.1.9.1 explorative Faktorenanalyse

Bei der explorativen Faktorenanalyse (EFA) erfolgt die Auswertung (häufig) nicht theoriegeleitet. Ziel ist es vielmehr zu erfahren, mit wievielen Faktoren die Daten bestmöglich erklärt werden können.

Wir verwenden den Datensatz **Faktorenbogen**, der wie folgt geladen werden kann.

```
1 # Datensatz laden
2 load(url("https://www.produnis.de/R/Faktorenbogen.rda"))
3
4 # anschauen
5 psych::describe(Faktorenbogen)
```

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew	kurtosis	se
age	1	150	53.11	18.82	53.0	54.16	17.05	1	101	100	-0.47	0.29	1.54
gender*	2	150	1.65	0.55	2.0	1.64	0.00	1	3	2	0.02	-0.87	0.04
A	3	150	4.41	2.03	5.0	4.40	2.97	0	10	10	0.05	-0.43	0.17
B	4	150	1.92	0.84	2.0	1.84	1.48	1	5	4	0.76	0.43	0.07
C	5	150	7.91	6.19	7.0	7.22	5.93	0	30	30	1.12	1.31	0.51
D	6	150	12.23	5.05	12.0	11.95	4.45	3	27	24	0.51	-0.04	0.41
E	7	150	14.77	2.39	15.0	14.69	2.97	10	21	11	0.28	-0.13	0.20
F	8	150	5.00	1.32	5.0	5.03	1.48	2	8	6	-0.12	-0.67	0.11
G	9	150	10.35	4.08	9.5	10.06	3.71	3	24	21	0.78	0.76	0.33
H	10	150	15.92	2.93	16.0	15.84	2.97	10	24	14	0.21	-0.39	0.24
I	11	150	85.02	68.19	65.0	75.93	51.15	0	356	356	1.40	2.05	5.57
J	12	150	5.02	2.25	5.0	4.93	1.48	1	11	10	0.33	-0.12	0.18
K	13	150	5.44	2.08	6.0	5.38	2.97	1	12	11	0.31	-0.10	0.17
L	14	150	1.47	0.65	1.0	1.36	0.00	1	4	3	1.20	0.86	0.05

Die Items für die Faktorenanalyse befinden sich in den Variablen A bis L.

```
1 items <- Faktorenbogen %>%
2   select(A:L)
```

Mit dem Bartlett-Test kann überprüft werden, ob die Items miteinander korrelieren (das sollten sie tun).

```
1 psych::cortest.bartlett(items)$p.value
```

```
R was not square, finding R from data
[1] 8.843226e-151
```

Das Ergebnis ist signifikant.

Mittels *Kaiser-Meyer-Olkin*-Verfahren (KMO) wird geprüft, wie gut die Daten für eine Faktorenanalyse geeignet sind, wobei Werte zwischen 0 (für “nicht geeignet”) und 1 (für “perfekt geeignet”) angenommen werden. Die *Minimum Average Partial* (MSA) ist ein Maß der internen Konsistenz, und nimmt ebenfalls Werte zwischen 0 (nicht konsistent) und 1 (perfekt konsistent) an. Beide Werte sollte größer als 0,5 sein.

```
1 # berechne KMO und MSA
2 psych::KMO(items)
```

```
Kaiser-Meyer-Olkin factor adequacy
Call: psych::KMO(r = items)
Overall MSA = 0.69
MSA for each item =
  A   B   C   D   E   F   G   H   I   J   K   L
0.36 0.72 0.66 0.68 0.60 0.73 0.24 0.73 0.75 0.62 0.78 0.61
```

Die “Overall MSA” entspricht dem KMO.

Mittels Screeplot kann die Faktorenzahl graphisch entschieden werden. Dabei sollte der letzte “gültige” Faktor vor dem deutlichen Knick zu einer abflachenden Gerade genutzt werden. Darüber hinaus sollten die Eigenwerte der Faktoren größer 1 sein.

Mit -Hausmitteln kann ein Screeplot wie folgt erzeugt werden.

```
1 plot(eigen(cor(items))$values, type="b")
2 abline(h=1)
```

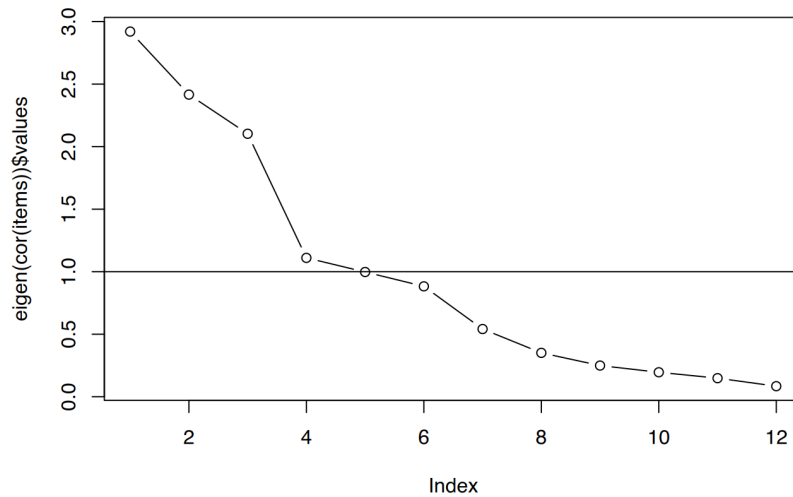


Abb. 2.11

Schöner geht es mit der Funktion `fa.parallel()`.

```
1 psych::fa.parallel(items)
```

```
Parallel analysis suggests that the number of factors = 3
and the number of components = 3
```

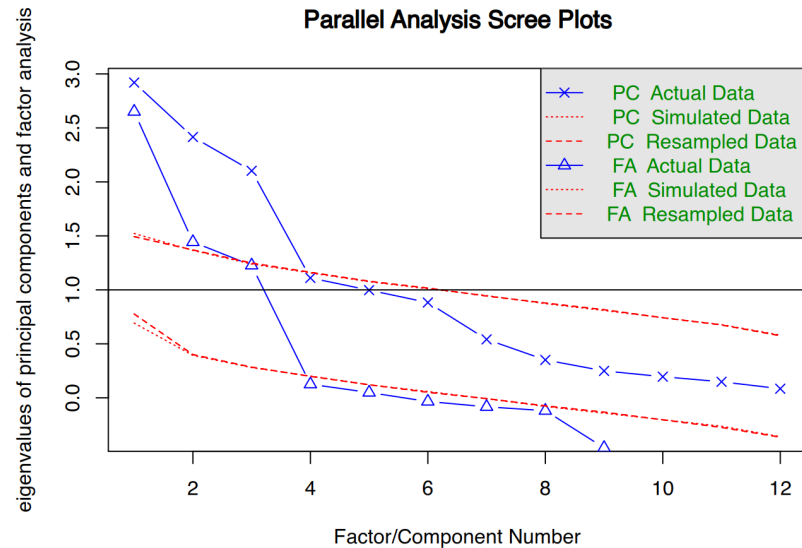


Abb. 2.12

Es werden 3 Faktoren/Komponenten vorgeschlagen.

Die Funktion berechnet das Screeplot sowohl für PAC (Sternchen) als auch PAF (Dreieck). Mit dem Parameter `fa = "fa"` kann die PAC-Methode entfernt werden.

```
1 psych::fa.parallel(items, fa = "fa")
```

```
Parallel analysis suggests that the number of factors = 3
```

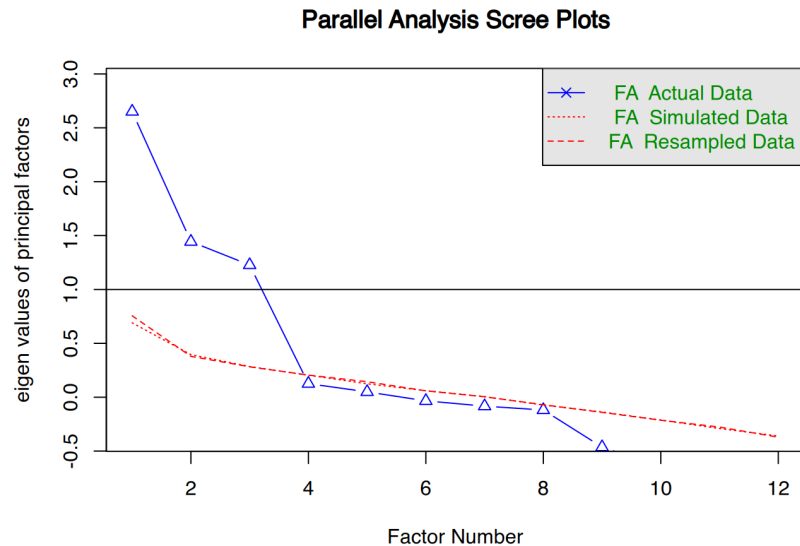



Abb. 2.13

Die Daten lassen sich am besten mit 3 Faktoren erklären.

2.1.9.2 konfirmatorische Faktorenanalyse

Ist die Anzahl der Faktoren bekannt (z.B. theoriegeleitet), wird die Hauptachsenanalyse wie folgt durchgeführt.

```

1 # Hauptachsen-Faktorenanalyse (PAF)
2 # mit 3 Faktoren, Varimax-Rotation
3 fit <- psych::fa(items, nfactors=3,
4                 rotate = "varimax")
5 fit

```

```

Factor Analysis using method = minres
Call: psych::fa(r = items, nfactors = 3, rotate = "varimax")
Standardized loadings (pattern matrix) based upon correlation matrix
  MR1  MR2  MR3   h2   u2 com
A  0.00  0.08  0.03 0.0069 0.993 1.2
B -0.01  0.83 -0.08 0.6997 0.300 1.0
C -0.05  0.88 -0.01 0.7770 0.223 1.0
D  0.96 -0.09 -0.07 0.9266 0.073 1.0
E -0.02  0.10  0.88 0.7820 0.218 1.0
F -0.04 -0.12  0.59 0.3655 0.635 1.1
G  0.00  0.00  0.07 0.0048 0.995 1.0

```

```
H  0.94 -0.02 -0.08 0.8855 0.115 1.0
I -0.03  0.78 -0.05 0.6107 0.389 1.0
J -0.08  0.08  0.88 0.7866 0.213 1.0
K  0.89  0.03 -0.01 0.7896 0.210 1.0
L -0.10  0.09  0.14 0.0393 0.961 2.6

          MR1  MR2  MR3
SS loadings      2.61 2.13 1.94
Proportion Var    0.22 0.18 0.16
Cumulative Var    0.22 0.39 0.56
Proportion Explained 0.39 0.32 0.29
Cumulative Proportion 0.39 0.71 1.00

Mean item complexity = 1.2
Test of the hypothesis that 3 factors are sufficient.

The degrees of freedom for the null model are 66 and the objective function was
        6.38 with Chi Square of 920.48
The degrees of freedom for the model are 33 and the objective function was 0.32

The root mean square of the residuals (RMSR) is 0.03
The df corrected root mean square of the residuals is 0.05

The harmonic number of observations is 150 with the empirical chi square 20.2
        with prob < 0.96
The total number of observations was 150 with Likelihood Chi Square = 45.01
        with prob < 0.079

Tucker Lewis Index of factoring reliability = 0.971
RMSEA index = 0.049 and the 90 % confidence intervals are 0 0.083
BIC = -120.34
Fit based upon off diagonal values = 0.99
Measures of factor score adequacy

          MR1  MR2  MR3
Correlation of (regression) scores with factors 0.98 0.94 0.94
Multiple R square of scores with factors        0.96 0.88 0.88
Minimum correlation of possible factor scores    0.92 0.76 0.77
```

Die Ausgabe von `fit` ist ziemlich lang. Wie gewohnt besteht die Möglichkeit, mittels `$` auf die einzelnen Ergebnisse zuzugreifen. Eine Übersicht erhält man mittels `names()`.

```
1 names(fit)
```

```
[1] "residual"      "dof"          "chi"          "nh"           "rms"
[6] "EPVAL"         "crms"         "EBIC"         "ESABIC"       "fit"
[11] "fit.off"       "sd"           "factors"      "complexity"   "n.obs"
[16] "objective"     "criteria"     "STATISTIC"    "PVAL"         "Call"
[21] "null.model"    "null.dof"     "null.chisq"   "TLI"          "RMSEA"
```

2.1. DESKRIPTIVE STATISTIK

```
[26] "BIC"          "SABIC"          "r.scores"        "R2"             "valid"
[31] "score.cor"    "weights"        "rotation"        "hyperplane"     "communality"
[36] "communalities" "uniquenesses"   "values"          "e.values"       "loadings"
[41] "model"        "fm"             "rot.mat"         "Structure"      "method"
[46] "scores"       "R2.scores"      "r"               "np.obs"         "fn"
[51] "Vaccounted"
```

```
1 fit$communality
```

```
      A      B      C      D      E      F
0.006940720 0.699704328 0.776989353 0.926609287 0.782014553 0.365479522
      G      H      I      J      K      L
0.004811051 0.885466837 0.610670890 0.786557566 0.789638724 0.039273937
```

```
1 fit$loadings
```

```
Loadings:
  MR1   MR2   MR3
A
B      0.833
C      0.880
D 0.956
E      0.103 0.878
F     -0.118 0.592
G
H 0.938
I      0.779
J           0.879
K 0.888
L -0.104      0.141

      MR1   MR2   MR3
SS loadings    2.605 2.129 1.940
Proportion Var 0.217 0.177 0.162
Cumulative Var 0.217 0.394 0.556
```

Mit diesen Informationen können die Items ein bisschen umsortiert...

- Faktor 1: D, H, K
- Faktor 2: B, C, I
- Faktor 3: E, F, J

... und ein Korrelationsmatrix erstellt werden.

```

1 # ändere Reihenfolge
2 items2 <- items %>%
3   select(D, H, K, B, C, I, E, F, J)
4
5 # Korrelationsmatrix plotten
6 corrrplot::corrplot(corr = cor(items2), # Korrelationsmatrix
7   method = "color", # Ausprägung farblich kodieren
8   addCoef.col = "black", # schreibe Werte in schwarz
9   number.cex = 0.7) # Schriftgröße

```

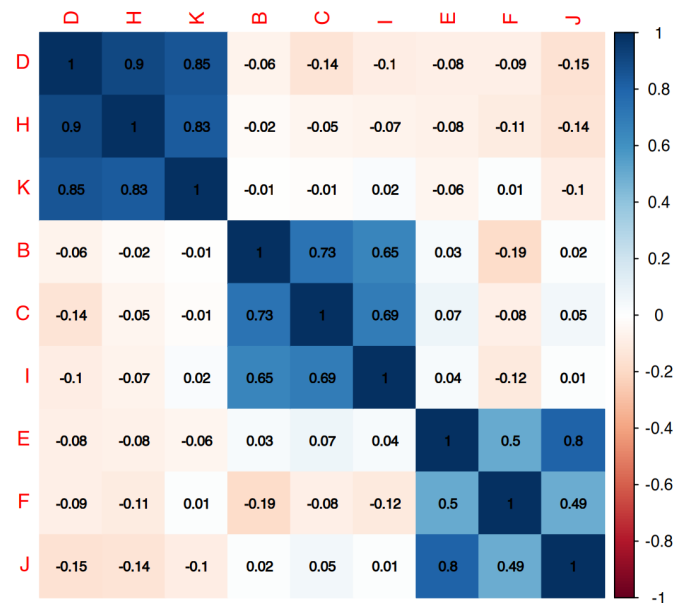


Abb. 2.14

2.1.9.3 Hauptkomponentenanalyse

Die Hauptkomponentenanalyse kann mit der Funktion `principal()` durchgeführt werden.

```

1  # Hauptkomponentenanalyse (PCA)
2  # mit 3 Hauptkomponenten und
3  # Varimax-Rotation
4  fit2 <- psych::principal(items, nfactors=3, rotate="varimax")
5
6  # anschauen
7  fit2

```

```

Principal Components Analysis
Call: psych::principal(r = items, nfactors = 3, rotate = "varimax")
Standardized loadings (pattern matrix) based upon correlation matrix
   RC1  RC2  RC3  h2  u2 com
A  0.01  0.13  0.05 0.019 0.981 1.3
B -0.01  0.89 -0.08 0.793 0.207 1.0
C -0.05  0.90  0.00 0.821 0.179 1.0
D  0.96 -0.08 -0.07 0.928 0.072 1.0
E -0.03  0.10  0.89 0.803 0.197 1.0
F -0.03 -0.15  0.76 0.606 0.394 1.1
G  0.01  0.00  0.12 0.015 0.985 1.0
H  0.95 -0.02 -0.07 0.916 0.084 1.0
I -0.04  0.86 -0.05 0.752 0.248 1.0
J -0.09  0.08  0.89 0.806 0.194 1.0
K  0.94  0.04  0.00 0.884 0.116 1.0
L -0.14  0.14  0.24 0.095 0.905 2.3

          RC1  RC2  RC3
SS loadings      2.74 2.44 2.26
Proportion Var    0.23 0.20 0.19
Cumulative Var    0.23 0.43 0.62
Proportion Explained 0.37 0.33 0.30
Cumulative Proportion 0.37 0.70 1.00

Mean item complexity = 1.2
Test of the hypothesis that 3 components are sufficient.

The root mean square of the residuals (RMSR) is 0.06
with the empirical chi square 61.27 with prob < 0.002

Fit based upon off diagonal values = 0.96

```

```

1  # welche $-Werte sind verfügbar?
2  names(fit2)

```

2.1. DESKRIPTIVE STATISTIK

```
[1] "values"      "rotation"    "n.obs"       "communality" "loadings"
[6] "fit"         "fit.off"     "fn"          "Call"        "uniquenesses"
[11] "complexity"  "chi"         "EPVAL"       "R2"          "objective"
[16] "residual"    "rms"         "factors"     "dof"         "null.dof"
[21] "null.model"  "criteria"    "STATISTIC"   "PVAL"        "weights"
[26] "r.scores"    "rot.mat"     "Vaccounted"  "Structure"   "scores"
```

```
1 # Faktorenladungen
2 fit2$loadings
```

```
Loadings:
  RC1    RC2    RC3
A      0.128
B      0.887
C      0.905
D 0.958
E      0.102 0.890
F     -0.146 0.764
G      0.121
H 0.954
I      0.865
J      0.890
K 0.940
L -0.135 0.143 0.238

          RC1    RC2    RC3
SS loadings 2.744 2.435 2.260
Proportion Var 0.229 0.203 0.188
Cumulative Var 0.229 0.432 0.620
```

Ein passendes Video von Daniela Keller findet sich bei Youtube unter <https://www.youtube.com/watch?v=NFPGQcq1f08>.

2.2 Schließende Statistik

2.2.1 Konfidenzintervalle

Konfidenzintervalle folgen der Logik, dass zum Punktschätzwert ein gewisser Fehlerbereich addiert und subtrahiert wird. Hierdurch wird ein Intervall mit Unter- und Obergrenze aufgezo-gen, das mit einer bestimmten Irrtumswahrscheinlichkeit den “wahren Wert” der Grundgesamtheit enthält.

2.2.1.1 Normalverteilung

Um ein Konfidenzintervall **für einen Mittelwert** aus einer normalverteilten Wertemenge zu berechnen wird folgende Formel angewendet:

$$\left. \begin{array}{l} \mu_u \\ \mu_o \end{array} \right\} = \bar{x} \pm t \cdot \frac{s}{\sqrt{n}} \quad (2.1)$$

$\bar{x}$ = arithmetisches Mittel
 n = Stichprobenumfang
 s = Standardabweichung
 t = Wert der t -Verteilung, (mit Freiheitsgraden $df = n - 1$)

Nehmen wir an, eine Zufalls-Erhebung über die jährlichen Ausgaben für Pflegehilfsmittel ergab bei 100 Befragten ($n = 100$) einen Mittelwert $\bar{x}=400,-\text{€}$ bei einer Standardabweichung von $s=20,-\text{€}$.

Das Konfidenzintervall für $\alpha = 0.05$ wird nun wie folgt bestimmt.

```
1  # schreibe gegebene Werte in Variablen
2  n      <- 100
3  xquer  <- 400
4  s      <- 20
5
6  # 95%KI-Fehler berechnen mit (alpha/2)
7  fehler95 <- qt(0.975, df=n-1)*s/sqrt(n)
8
9  # Fehlerwert ausgeben
10 fehler95
```

```
[1] 3.968434
```

Beachten Sie, dass in der Funktion `qt()` zur Bestimmung des kritischen t-Wertes die Irrtumswahrscheinlichkeit α halbiert werden muss, da es sich um eine zweiseitige Fragestellung handelt. So muss `qt()` nicht mit dem Wert 0.95 ($\alpha = 0.05$) aufgerufen werden, sondern mit 0.975 ($\frac{\alpha}{2} = 0.025$).

```
1 # untere Grenze des 95%KI
2 xquer - fehler95
```

```
[1] 396.0316
```

Die untere Grenze des 95%-Konfidenzintervalls liegt bei 396.0316 €.

```
1 # obere Grenze des 95%KI
2 xquer + fehler95
```

```
[1] 403.9684
```

Die obere Grenze des 95%-Konfidenzintervalls liegt bei 403.9684 €.

```
1 # Länge des 95%KI
2 (xquer + fehler95) - (xquer - fehler95)
3 # oder schlauer
4 2*fehler95
```

```
[1] 7.936868
```

Das 95%-Konfidenzintervall hat eine Länge von 7.936868 €.

Für das 99% Konfidenzintervall ändern wir entsprechend um. Auch hier muss, da es sich um eine zweiseitige Fragestellung handelt, mit $\frac{\alpha}{2}$, also 0.005, gerechnet werden. In der Funktion `qt()` ändert sich also der Wert auf 0.995.

```
1 # 99%KI-Fehler berechnen mit (alpha/2)
2 fehler99 <- qt(0.995, df=n-1)*s/sqrt(n)
3
4 # zeige fehler99 an
5 fehler99
```



```
[1] 5.252811
```

```
1 # untere Grenze des 99%KI
2 xquer - fehler99
```

```
[1] 394.7472
```

Die untere Grenze des 99%-Konfidenzintervalls liegt bei 394.7472 €.

```
1 # obere Grenze des 99%KI
2 xquer + fehler99
```

```
[1] 405.2528
```

Die obere Grenze des 99%-Konfidenzintervalls liegt bei 405.2528 €.

```
1 # Länge des 99%KI
2 (xquer + fehler99) - (xquer - fehler99)
3 # oder schlauer
4 2*fehler99
```

```
[1] 10.50562
```

Das 99%-Konfidenzintervall hat eine Länge von 10.50562 €.

Unterschied von Mittelwerten

Das Konfidenzintervall für den **Unterschied von Mittelwerten** normalverteilter Daten berechnet sich mit der Formel

$$\left. \begin{matrix} \mu_u \\ \mu_o \end{matrix} \right\} = \hat{d} \pm t \cdot \sqrt{\frac{s_{pool}^2}{n_x} + \frac{s_{pool}^2}{n_y}} \quad (2.2)$$

$$\begin{aligned} \hat{d} &= \text{Differenz der Mittelwerte } \bar{x}, \bar{y} \\ n_x, n_y &= \text{die Umfänge der beiden Stichproben} \\ s_{pool} &= \text{gepoolte Standardabweichungen} = \frac{(n_x-1) \cdot s_x^2 + (n_y-1) \cdot s_y^2}{n_x + n_y - 2} \\ t &= \text{Wert der } t\text{-Verteilung (mit Freiheitsgraden } df = n_x + n_y - 1) \end{aligned}$$

Nehmen wir an, in einer Studie wurde die Wirkung von zwei Medikamenten auf den Magnesiumgehalt des Blutserums (in mg/dl) untersucht. 13 Personen erhielten Medikament A und 10 Personen Medikament B. Nach 6 Wochen wurden folgende Daten bestimmt:

- $\bar{x}_A = 2,22$ mg/dl
- $\bar{x}_B = 2,70$ mg/dl
- $s_A = 0,255$ mg/dl
- $s_B = 0,306$ mg/dl
- $n_A = 13$
- $n_B = 10$

Das Konfidenzintervall für den Unterschied der Mittelwerte bei $\alpha = 0,05$ wird nun wie folgt bestimmt (Achtung, da es sich um eine zweiseitige Fragestellung handelt, müssen wir mit $\frac{\alpha}{2} = 0,025$ rechnen).

```

1  # erzeuge die Daten
2  xa <- 2.22
3  xb <- 2.7
4  sa <- 0.255
5  sb <- 0.306
6  na <- 13
7  nb <- 10
8
9  # Differenz der Mittelwerte
10 d <- xb - xa
11
12 # ausgeben
13 d

```

```
[1] 0.48
```

2.2. SCHLIESSENDE STATISTIK

```
1 # berechne die gepoolten Standardabweichungen
2 s_pool = ((na-1)*sa^2 + (nb-1)*sb^2) / (na+nb-2)
3
4 # ausgeben
5 s_pool
```

```
[1] 0.07728686
```

```
1 #Fehlerquote berechnen
2 fehler95 <- qt(0.975, df=na+nb-1) * sqrt(s_pool^2/na + s_pool^2/nb)
3
4 # ausgeben
5 fehler95
```

```
[1] 0.06741865
```

```
1 # untere Grenze des 95%KI
2 d - fehler95
```

```
[1] 0.4125813
```

Die untere Grenze des 95%-Konfidenzintervalls liegt bei 0.4125813 mg/dl

```
1 # obere Grenze des 95%KI
2 d + fehler95
```

```
[1] 0.5474187
```

Die obere Grenze des 95%-Konfidenzintervalls liegt bei 0.5474187 mg/dl

```
1 # Länge des Intervalls
2 2*fehler95
```

```
[1] 0.1348373
```

Das 95%-Konfidenzintervall hat eine Länge von 0.1348373 mg/dl

Für das 99%-Konfidenzintervall wird der Wert für die Funktion `qt()` auf 0.995 gesetzt (da es sich um eine zweiseitige Fragestellung handelt, müssen wir mit $\frac{\alpha}{2} = 0,005$ rechnen).

```
1 #Fehlerquote berechnen
2 fehler99 <- qt(0.995, df=na+nb-1) * sqrt(s_pool^2/na + s_pool^2/nb)
3
4 # ausgeben
5 fehler99
```

```
[1] 0.09163373
```

```
1 # untere Grenze des 99%KI
2 d - fehler99
```

```
[1] 0.3883663
```

Die untere Grenze des 99%-Konfidenzintervalls liegt bei 0.3883663 mg/dl

```
1 # obere Grenze des 99%KI
2 d + fehler99
```

```
[1] 0.5716337
```

Die obere Grenze des 99%-Konfidenzintervalls liegt bei 0.5716337 mg/dl

```
1 # Länge des Intervalls
2 2*fehler99
```

```
[1] 0.1832675
```

Das 99%-Konfidenzintervall hat eine Länge von 0.1832675 mg/dl

2.2.1.2 Binomialverteilung

Zur Berechnung von Konfidenzintervallen **für Anteilswerte** wird folgende Formel verwendet.

$$\left. \begin{matrix} p_o \\ p_u \end{matrix} \right\} = k \pm c \cdot \sqrt{\frac{k \cdot (1 - k)}{n}} \quad (2.3)$$

n = Umfang der Stichprobe
 k = geschätzter Anteil
 c = Quartilswert für Irrtumswahrscheinlichkeit α

Nehmen wir an, in einer klinischen Studie wurde an 150 Patienten die Dekubitusprävalenz ermittelt. Dabei wurde eine Prävalenz von 35% festgestellt. Die Konfidenzintervalle für den “wahren Wert” der Grundgesamtheit wird wie folgt bestimmt.

```
1  # Daten erzeugen
2  n <- 150
3  k <- 0.35
4
5  # Fehler berechnen mit (alpha/2)!
6  fehler95 <- qnorm(0.975)*sqrt(k*(1-k)/n)
7
8  # anzeigen
9  fehler95
```

```
0.07632963
```

Beachten Sie, dass in der Funktion `qnorm()` die Irrtumswahrscheinlichkeit α halbiert werden muss, da es sich um eine zweiseitige Fragestellung handelt. So muss `qnorm()` nicht mit dem Wert 0.95 ($\alpha = 0.05$) aufgerufen werden, sondern mit 0.975 ($\frac{\alpha}{2} = 0.025$).

```
1  # untere Grenze des KI
2  k - fehler95
```

2.2. SCHLIESSENDE STATISTIK

```
0.2736704
```

Die untere Grenze des 95%-Konfidenzintervalls liegt bei $0.2736704 = 27,36704\%$

```
1 # obere Grenze des KI
2 k + fehler95
```

```
[1] 0.4263296
```

Die obere Grenze des 95%-Konfidenzintervalls liegt bei $0.4263296 = 42,63296\%$

```
1 # Länge des KI
2 2*fehler95
```

```
[1] 0.1526593
```

Das 95%-Konfidenzintervall hat eine Länge von $0.1526593 = 15,26593\%$.

Für das 99%-Konfidenzintervall muss der Wert der Funktion `qnorm()` auf 0.995 ($\frac{\alpha}{2} = 0,005$) angepasst werden.

```
1 # Fehler berechnen mit (alpha/2)!
2 fehler99 <- qnorm(0.995)*sqrt(k*(1-k)/n)
3
4 # anzeigen
5 fehler99
```

```
[1] 0.1003141
```

```
1 # untere Grenze des KI
2 k - fehler99
```

0.2496859

Die untere Grenze des 99%-Konfidenzintervalls liegt bei $0.2496859 = 24,96859\%$

```
1 # obere Grenze des KI
2 k + fehler99
```

0.4503141

Die obere Grenze des 99%-Konfidenzintervalls liegt bei $0.4503141 = 45,03141\%$

```
1 # Länge des KI
2 2*fehler99
```

0.2006283

Das 99%-Konfidenzintervall hat eine Länge von $0.2006283 = 20,06283\%$.

Unterschied von Anteilswerten

Zur Berechnung der Konfidenzintervalle bei **Unterschieden von Anteilswerten** wird die folgende Formel verwendet.

$$\left. \begin{matrix} d_o \\ d_u \end{matrix} \right\} = \hat{d} \pm c \cdot \sqrt{\frac{\hat{p}_1 \cdot (1 - \hat{p}_1)}{n_1} + \frac{\hat{p}_2 \cdot (1 - \hat{p}_2)}{n_2}} \quad (2.4)$$

$\hat{d}$ = die Differenz der Anteile
 n_1, n_2 = die Umfänge der beiden Stichproben
 $\hat{p}_1, \hat{p}_2$ = die geschätzten Anteile in beiden Stichproben
 c = Quartilswert für Irrtumswahrscheinlichkeit α

Nehmen wir an, in Bundesland A gaben 25 von 100 Befragten an, Zuzahlungen bei Pflegehilfsmitteln leisten zu müssen. In Bundesland B wurden 150 Personen befragt, von denen 60 von Zuzahlungen betroffen waren. Bundesland A hat

2.2. SCHLIESSENDE STATISTIK

dementsprechend einen Anteil von 0,25 (25%) und Bundesland B einen Anteil von 0,40 (40%) Zuzählern. Der Punktschätzwert für die Differenz $\hat{d}$ beträgt folglich $0,40 - 0,25 = \mathbf{0,15}$.

Die Konfidenzintervalle für die wahre Differenz d kann nun wie folgt bestimmt werden.

```
1  # Daten erzeugen
2  n1 <- 100
3  p1 <- 0.25
4
5  n2 <- 150
6  p2 <- 0.4
7
8  # Differenz der Anteile
9  d <- p2-p1
```

Beachten Sie, dass in der Funktion `qnorm()` die Irrtumswahrscheinlichkeit α halbiert werden muss, da es sich um eine zweiseitige Fragestellung handelt. So muss `qnorm()` nicht mit dem Wert 0.95 ($\alpha = 0.05$) aufgerufen werden, sondern mit 0.975 ($\frac{\alpha}{2} = 0.025$).

```
1  # 95%KI-Fehler berechnen mit (alpha/2)
2  fehler95 <- qnorm(0.975)*sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
3
4  #anzeigen
5  fehler95
```

```
[1] 0.1155382
```

```
1  # untere Grenze des 95%KI
2  d - fehler95
```

```
[1] 0.03446183
```

Die untere Grenze des 95%-Konfidenzintervalls liegt bei $0.03446183 = 3,446183\%$

```
1  # obere Grenze des 95%KI
2  d + fehler95
```



```
[1] 0.2655382
```

Die obere Grenze des 95%-Konfidenzintervalls liegt bei $0.2655382 = 26,55382\%$

```
1 # Länge des KI
2 2*fehler95
```

```
[1] 0.2310763
```

Die Länge des 95%-Konfidenzintervalls liegt bei $0.2310763 = 23,10763\%$

Für das 99%-Konfidenzintervall muss der Wert der Funktion `qnorm()` auf 0.995 ($\frac{\alpha}{2} = 0,005$) angepasst werden.

```
1 # 99%KI-Fehler berechnen mit (alpha/2)
2 fehler99 <- qnorm(0.995)*sqrt(p1*(1-p1)/n1 + p2*(1-p2)/n2)
3
4 #anzeigen
5 fehler99
```

```
[1] 0.1518429
```

```
1 # untere Grenze des 99%KI
2 d - fehler99
```

```
[1] -0.001842898
```

Die untere Grenze des 99%-Konfidenzintervalls liegt bei $-0.0018429 = -0,18429\%$

```
1 # obere Grenze des 99%KI
2 d + fehler99
```

```
[1] 0.3018429
```

Die obere Grenze des 99%-Konfidenzintervalls liegt bei $0.3018429 = 30,18429\%$

```
1 # Länge des KI
2 2*fehler99
```

```
[1] 0.3036858
```

Die Länge des 99%-Konfidenzintervalls liegt bei $0.3036858 = 30,36858\%$

Wie Sie sehen, liegt die untere Grenze des 99%-Konfidenzintervalls im negativen Bereich. Errechnet sich ein Intervall, welches die 0 einschließt, bedeutet dies, dass der wahre Unterschied auch 0 sein könnte.

2.2.2 Signifikanztests

2.2.2.1 χ^2 -Test

Der χ^2 -Test wird mit der Funktion `chisq.test()` durchgeführt. Hierfür müssen die Daten als Kreuztabelle vorliegen.

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/nw.RData"))
3
4 # erstelle Kreuztabelle aus Alter und Geschlecht
5 xtabelle <- xtabs(~ nw$age + nw$sex)
6
7 # Führe Chiquadrat-Test durch
8 chisq.test(xtabelle)
```

```
Pearson's Chi-squared test

data:  xtabelle
X-squared = 43.886, df = 44, p-value = 0.4765

Warnmeldung:
In chisq.test(xtabelle) : Chi-Quadrat-Approximation kann inkorrekt sein
```

Der p-Wert ist nicht signifikant, das heisst, dass kein Zusammenhang gefunden werden konnte.

Die Warnung bedeutet, dass der p-Wert aus sehr kleinen Zellenhäufigkeiten (kleiner 5) geschätzt wurde, und daher fehlerhaft sein kann. Wir sollten daher einen *exakten* Test durchführen, zum Beispiel den *Fisher-Test*.

Ein gutes Anleitungsvideo zum χ^2 -Test findet sich bei Youtube:
<https://www.youtube.com/watch?v=4lFyRcXoJB8>

2.2.2.2 Fisher's Exakttest

Der *exakte Fisher-Test* wird mit der Funktion `fisher.test()` durchgeführt.

```
1 # Führe Fisher-Test durch
2 fisher.test(xtabelle)
```

```
Fisher's Exact Test for Count Data with simulated p-value (based on 2000 replicates)

data:  nw$sex and nw$age
p-value = 0.3848
alternative hypothesis: two.sided
```

Der exakte Fisher-Test bestätigt, dass es keinen Unterschied gibt.

2.2.2.3 t-Test

Der t-Test ist wohl einer der bekanntesten Signifikanztests. In  wird er mit der Funktion `t.test()` ausgeführt.

2.2.2.3.1 Einstichproben t-Test

Beim Einstichproben t-Test wird der Mittelwert der Grundgesamtheit über den Parameter `mu` mitgegeben.

Schauen wir, ob im Nachwachsendatensatz das Alter von einem gedachten Mittelwert 40 in der Grundgesamtheit abweicht.

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/nw.RData"))
3
4 # Führe t-Test für "Alter" auf Mittelwert 40 durch
5 t.test(nw$age, mu=40)
```

```
One Sample t-test

data:  nw$age
t = 6.8274, df = 275, p-value = 5.524e-11
alternative hypothesis: true mean is not equal to 40
95 percent confidence interval:
 43.00908 45.44744
sample estimates:
mean of x
 44.22826
```

Der Test ist signifikant, das bedeutet, das Alter im Datensatz `nw` weicht von einem Mittelwert in der Grundgesamtheit von 40 ab. Es liegt ein Unterschied vor.

Zur Beschreibung der Effektstärke des Unterschieds können wir *Cohen's d* berechnen. Dies kann über das Zusatzpaket `lsr` und der Funktion `cohensD()` erfolgen.

```
1 # aktiviere Zusatzpaket "lsr"
2 library(lsr)
3
4 # berechne Effektsärke Cohen's D
5 cohensD(nw$age, mu=40)
```

```
[1] 0.4109627
```

Dieser Wert kann nun nach Cohen's Angaben^[1] interpretiert werden (0.2=kleiner Effekt, 0.5=mittel, 0.8=stark). Unsere Effektgröße ist eher "mittel".

Ein gutes Anleitungsvideo zum Einstichproben t-Test findet sich bei Youtube: <https://www.youtube.com/watch?v=7vcyjuSwfzI>

2.2.2.3.2 unabhängiger t-Test

Für den Vergleich der Mittelwerte bei unabhängige Stichproben werden die Werte beider Gruppen als Variable übergeben. So können wir z.B. schauen, ob sich die Datensätze `nw` und `epa` hinsichtlich des Alters unterscheiden

```
1 # t-test auf Alter in Datensatz "epa" und "nw"
2 t.test(nw$age, epa$age)
```

```
Welch Two Sample t-test

data:  nw$age and epa$age
t = -19.316, df = 854.37, p-value < 2.2e-16
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -20.65610 -16.84545
sample estimates:
mean of x mean of y
 44.22826  62.97903
```

Das Ergebnis ist signifikant, die Gruppen unterscheiden sich also.

Standardmäßig wird ein zweiseitiger Test durchgeführt. Über den Parameter `alternative` kann dies geändert werden.

```
1 # t-test einseitig, kleiner
2 t.test(nw$age, epa$age, alternative = "less")
3 # t-test einseitig, größer
4 t.test(nw$age, epa$age, alternative = "greater")
5 # t-test zweiseitig
6 t.test(nw$age, epa$age, alternative = "two.sided")
```

Die Funktion nimmt auch die Syntax *erklärt durch* entgegen. Wir schauen, ob sich das Alter im Datensatz `epa` in den Geschlechtergruppen unterscheidet, also *Alter erklärt durch Geschlecht*.

```
1 # t-test Alter zwischen Geschlechtern im Datensatz "epa"
2 t.test(age ~ sex, data=epa)
```

```
Welch Two Sample t-test

data: age by sex
t = -4.7535, df = 595.03, p-value = 2.51e-06
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -9.923268 -4.120822
sample estimates:
mean in group m mean in group w
    59.79646      66.81851
```

Das Ergebnis ist signifikant. Auch hier lässt sich wieder die Effektstärke Cohen's D berechnen.

```
1 # Effektsärke
2 lsr::cohensD(age ~ sex, data=epa)
```

```
[1] 0.3837919
```

Der Effekt ist mit 0,38 eher klein.

Ein gutes Anleitungsvideo zum unabhängigen t-Test findet sich bei Youtube:
<https://www.youtube.com/watch?v=3Yz-XNRasGM>

2.2.2.3.3 abhängiger t-Test

Bei gepaarten Werten muss ein abhängiger t-Test berechnet werden. Dies erfolgt über den Parameter `paired`, der auf `TRUE` gesetzt wird. Auch hier können wir wieder die Richtung (größer, kleiner, zweiseitig) über den Parameter `alternative` verändern.

```

1  # erzeuge Testwerte
2  t0 <- rnorm(100, mean=12, sd=1)
3  t1 <- rnorm(100, mean=11, sd=3)
4
5  # gepaarter t-Test
6  t.test(t0, t1, paired=TRUE)
7
8  # gepaarter t-test einseitig, kleiner
9  t.test(t0, t1, paired=TRUE, alternative = "less")
10 # gepaarter t-test einseitig, größer
11 t.test(t0, t1, paired=TRUE, alternative = "greater")
12 # gepaarter t-test zweiseitig
13 t.test(t0, t1, paired=TRUE, alternative = "two.sided")

```

```

Paired t-test

data:  t0 and t1
t = 3.012, df = 99, p-value = 0.003295
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.3195899 1.5536199
sample estimates:
mean of the differences
      0.9366049

```

Die Effektstärke Cohen's D lässt sich entsprechend berechnen, indem der Parameter `method` auf `paired` gesetzt wird:

```

1  # Effektsärke
2  lsr::cohensD(t0, t1, method="paired")

```

```
[1] 0.3258625
```

Der Effekt ist mit 0,33 eher gering.

Ein gutes Anleitungsvideo zum verbundenen t-Test findet sich bei Youtube:
https://www.youtube.com/watch?v=H15M_8gh10k

2.2.2.4 F-Test

Der F-Test wird mit der Funktion `var.test()` berechnet. So können wir prüfen, ob die Varianz des Alters im Datensatz `epa` in den Geschlechtergruppen unterschiedlich ist.

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/epa.RData"))
3
4 # F-Test
5 var.test(age ~ sex , data=epa)
```

```
      F test to compare two variances

data:  age by sex
F = 0.98352, num df = 338, denom df = 280, p-value = 0.8816
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.7848842 1.2294811
sample estimates:
ratio of variances
      0.983521
```

Der Test ist nicht signifikant, es liegt kein Unterschied in den Varianzen vor.

2.2.2.5 Levene-Test

Ebenso kann mit dem Levene-Test auf Varianzunterschied geprüft werden. Hierfür rufen wir die Funktion `leveneTest()` aus dem Zusatzpaket `car` auf.

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/epa.RData"))
3
4 # Levene-Test
5 car::leveneTest(epa$age, epa$sex)
```

```
Levene's Test for Homogeneity of Variance (center = median)
      Df F value Pr(>F)
group  1  0.0232 0.8789
      618
```


Der Test ist nicht signifikant, es liegt kein Unterschied in den Varianzen vor.

Ein gutes Anleitungsvideo zum verbundenen Levene-Test findet sich bei YouTube:

<https://www.youtube.com/watch?v=717UWGnyQGA>

2.2.2.6 Mann-Whitney-U-Test

Dies ist die nicht-parametrische alternative zum t-Test für unabhängige Stichproben. Er wird mit der Funktion `wilcox.test()` durchgeführt, wobei der Parameter `paired` auf `FALSE` gesetzt wird (ist die Standardeinstellung).

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/epa.RData"))
3
4 # Man-Whitney-U-Test
5 wilcox.test(epa$age ~ epa$sex)
```

```
Wilcoxon rank sum test with continuity correction

data:  epa$age by epa$sex
W = 35862, p-value = 1.151e-07
alternative hypothesis: true location shift is not equal to 0
```

Das Ergebnis ist signifikant, das Alter im Datensatz `epa` unterscheidet sich zwischen den Geschlechtern.

Mit dem Parameter `conf.int` kann das Konfidenzintervall des Schätzwertes mit ausgegeben werden.

```
1 # Man-Whitney-U-Test mit Konfidenzintervall
2 wilcox.test(epa$age ~ epa$sex, conf.int=TRUE)
```

```
Wilcoxon rank sum test with continuity correction

data:  epa$age by epa$sex
W = 35862, p-value = 1.151e-07
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 -9.999986 -4.999952
```

```
sample estimates:
difference in location
-7.000029
```

Ein gutes Anleitungsvideo zum Mann-Whitney-U-Test finden Sie bei Youtube:
<https://www.youtube.com/watch?v=4lFyRcXoJB8>

2.2.2.7 Wilcoxon-Test

Dies ist die nicht-parametrische alternative zum abhängigen t-Test. Er wird mit der Funktion `wilcox.test()` durchgeführt, wobei der Parameter `paired` auf `TRUE` gesetzt werden muss. Andernfalls rechnet  einen Mann-Whitney-U-Test.

```
1 # erzeuge Testwerte
2 t0 <- sample(70:140, 100, replace=T)
3 t1 <- sample(74:170, 100, replace=T)
4
5 wilcox.test(t0, t1, paired=TRUE)
```

```
Wilcoxon signed rank test with continuity correction

data:  t0 and t1
V = 1036.5, p-value = 1.435e-06
alternative hypothesis: true location shift is not equal to 0
```

Mit dem Parameter `conf.int` kann das Konfidenzintervall des Schätzwertes mit ausgegeben werden.

```
1 # erzeuge Testwerte
2 t0 <- sample(70:140, 100, replace=T)
3 t1 <- sample(74:170, 100, replace=T)
4
5 wilcox.test(t0, t1, paired=TRUE, conf.int=TRUE)
```

```
Wilcoxon signed rank test with continuity correction

data:  t0 and t1
```

```
V = 1036.5, p-value = 1.435e-06
alternative hypothesis: true location shift is not equal to 0
95 percent confidence interval:
 -26.99994 -12.50000
sample estimates:
(pseudo)median
      -19.5
```

Ein gutes Anleitungsvideo zum Wilcoxon-Test finden Sie bei Youtube:

<https://www.youtube.com/watch?v=4lFyRcXoJB8>

2.2.2.8 Shapiro-Wilk-Normalverteilungstest

Zum Test auf Normalverteilung kann der *Shapiro-Wilk*-Test über die Funktion `shapiro.test()` verwendet werden. Die Nullhypothese lautet, *dass* eine Normalverteilung vorliegt. Testen wir, ob im Datensatz `nw` das Alter (`age`) normalverteilt ist.

```
1 # lade Testdaten
2 load(url("https://www.produnis.de/R/nw.RData"))
3
4 # Teste "age" auf Normalverteilung
5 shapiro.test(nw$age)
```

```
Shapiro-Wilk normality test

data:  nw$age
W = 0.96636, p-value = 4.677e-06
```

Das Ergebnis ist signifikant, die Daten sind **nicht** normalverteilt.

Erzeugen wir normalverteilte Werte zur Gegenprobe.

```
1 # Test mit 100 zufälligen und normalverteilten Werten
2 shapiro.test(rnorm(100))
```

```
Shapiro-Wilk normality test  
data:  rnorm(100)  
W = 0.98488, p-value = 0.3115
```

Das Ergebnis ist (wie erwartet) nicht signifikant, das heisst, unsere Zufallszahlen sind wirklich normalverteilt.

2.2.3 Varianzanalysen

2.2.3.1 ANOVA mit Messwiederholung

Eine ANOVA mit Messwiederholung kann wie folgt in  durchgeführt werden. Wir nutzen den Testdatensatz `Messwiederholung.rda`.

```
1 load(url("https://www.produnis.de/R/Messwiederholung.rda"))
```

Der Datensatz besteht aus 200 Probanden, bei denen zu drei Zeitpunkten ein Wert erhoben wurde.

```
1 head(Messwiederholung)
```

	Name	t1	t2	t3
1	Brycen	11.728110	8.190290	6.202859
2	Waaail	9.781605	8.646142	6.311455
3	Abigail	11.937967	9.386623	4.577478
4	Holly	4.597950	8.204762	7.250849
5	Sheila	8.277670	8.322449	6.952457
6	Rebekah	7.891258	9.104856	7.191639

Zunächst müssen wir die Daten ins ‘long table’-Format überführen.

```
1 # Überführe die Daten ins "long table"-Format
2 df <- Messwiederholung %>%
3   pivot_longer(cols=t1:t3,
4     names_to = "Zeitpunkt",
5     values_to = "Wert") %>%
6   mutate(Zeitpunkt = factor(Zeitpunkt),
7     Name = factor(Name))
8
9 # anschauen
10 head(df)
```

	Name	Zeitpunkt	Wert
	<fct>	<fct>	<dbl>
1	Brycen	t1	11.7
2	Brycen	t2	8.19
3	Brycen	t3	6.20
4	Waaail	t1	9.78
5	Waaail	t2	8.65
6	Waaail	t3	6.31

2.2. SCHLIESSENDE STATISTIK

Zunächst verschaffen wir uns einen deskriptiven Überblick über die Daten.

```
1 # mal deskriptiv anschauen
2 df %>%
3   group_by(Zeitpunkt) %>%
4   summarise(M = mean(Wert),
5             SD = sd(Wert),
6             Q1 = quantile(Wert, probs = 0.25),
7             Q2 = quantile(Wert, probs = 0.5),
8             Q3 = quantile(Wert, probs = 0.75),
9             IQR = IQR(Wert),
10            N = n())
```

```
# A tibble: 3 × 8
  Zeitpunkt      M    SD   Q1   Q2   Q3   IQR     N
  <fct>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
1 t1      9.88  2.14  8.41  9.77 11.3  2.91  200
2 t2      7.99  0.935 7.24  8.09  8.60  1.36  200
3 t3      6.92  0.868 6.31  6.89  7.51  1.21  200
```

Das geht auch graphisch:

```
1 # mal graphisch anschauen
2 boxplot(df$Wert ~ df$Zeitpunkt)
```

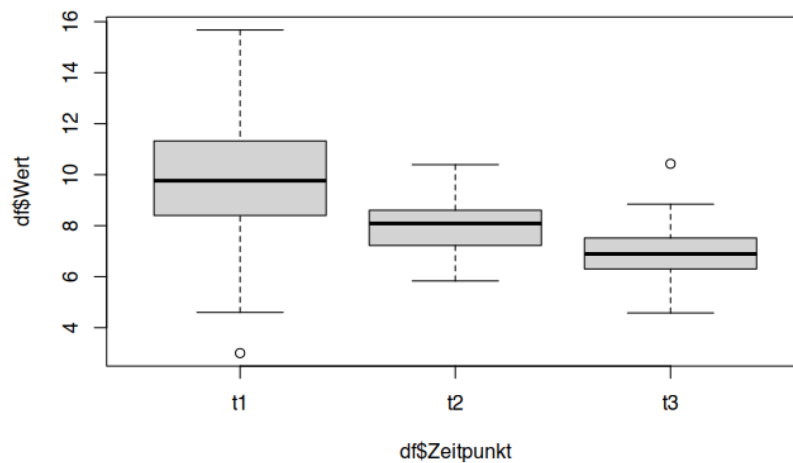


Abb. 2.15

```
1 ez::ezPlot(df, Wert, Name, within = Zeitpunkt, x = Zeitpunkt)
```

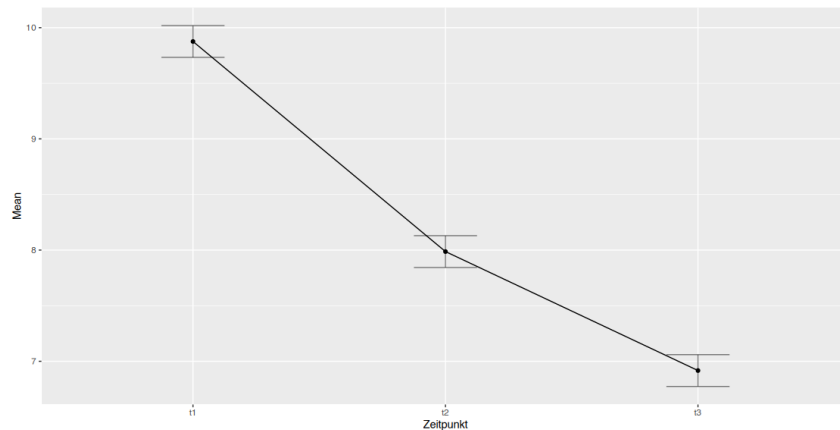


Abb. 2.16

```
1 # mit Regressionsgerade
2 ez::ezPlot(df, Wert, Name, within = Zeitpunkt, x = Zeitpunkt) +
3   geom_smooth(aes(x = as.numeric(Zeitpunkt)), method = 'lm', se = F)
```

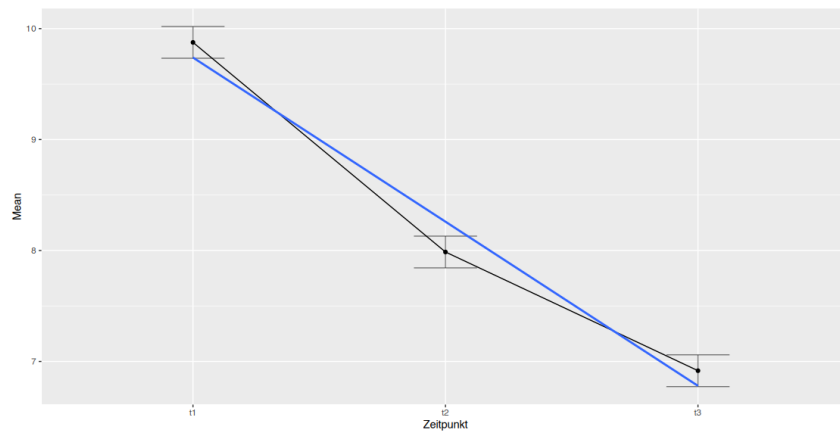


Abb. 2.17

Die Plots legen nahe, dass es einen Unterschied in den Zeitpunktgruppen gibt. Wir überprüfen mittels Shapiro-Test und QQ-Plot, ob die Daten normalverteilt sind.

```

1  ## qq-Plot (Normalverteilung)
2  ggpubr::ggqqplot(df, x="Wert",
3                    facet.by = "Zeitpunkt",
4                    color = "Zeitpunkt",
5                    scales = "free")

```

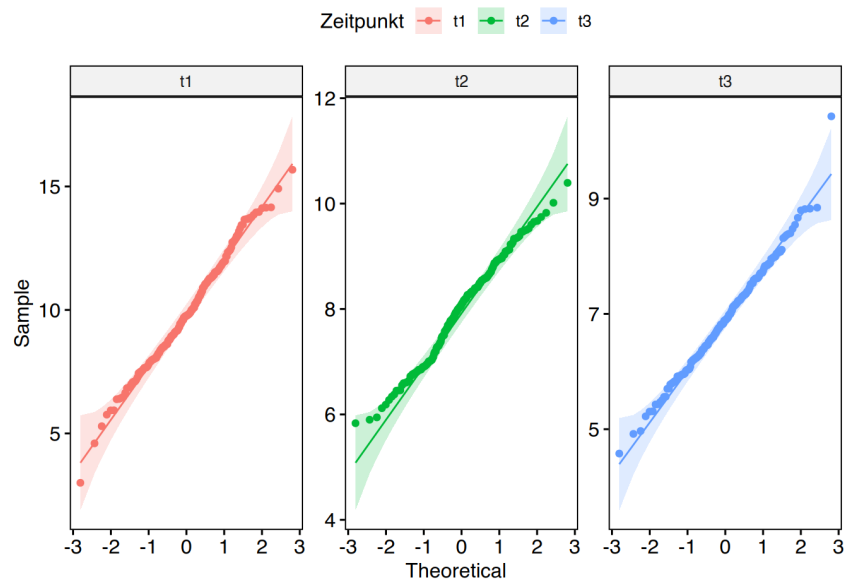


Abb. 2.18

```

1  # Teste "Wert" auf Normalverteilung
2  shapiro.test(df$Wert[df$Zeitpunkt=="t1"])

```

Shapiro-Wilk normality test

data: df\$Wert[df\$Zeitpunkt == "t1"]
W = 0.99364, p-value = 0.5481

```

1  # Teste "Wert" auf Normalverteilung
2  shapiro.test(df$Wert[df$Zeitpunkt=="t2"])

```



```
Shapiro-Wilk normality test

data: df$Wert[df$Zeitpunkt == "t2"]
W = 0.98766, p-value = 0.08025
```

```
1 # Teste "Wert" auf Normalverteilung
2 shapiro.test(df$Wert[df$Zeitpunkt=="t3"])
```

```
Shapiro-Wilk normality test

data: df$Wert[df$Zeitpunkt == "t3"]
W = 0.99116, p-value = 0.2625
```

Die Testergebnisse sind nicht signifikant, also liegt Normalverteilung vor. Wir dürfen eine ANOVA mit Messwiederholung rechnen. Mit -Hausmittel könnten wir so vorgehen:

```
1 # ANOVA mit Messwiederholung
2 fit <- aov(Wert ~ Zeitpunkt + Error(Name), data=df)
3 summary(fit)
```

```
Error: Name
      Df Sum Sq Mean Sq F value Pr(>F)
Residuals 199  395.8    1.989

Error: Within
      Df Sum Sq Mean Sq F value Pr(>F)
Zeitpunkt  2  898.0    449.0   212.3 <2e-16 ***
Residuals 398  841.6      2.1

---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Das Ergebnis ist signifikant. Mit der Funktion `pairwise.t.test()` können wir nun schauen, welche Gruppenunterschiede es konkret gibt:

```
1 # geht auch mit R-Hausmitteln so:
2 pairwise.t.test(df$Wert, df$Zeitpunkt,
3                 p.adjust="bonferroni")
```

```

Pairwise comparisons using t tests with pooled SD

data: df$Wert and df$Zeitpunkt

      t1      t2
t2 < 2e-16 -
t3 < 2e-16 1.1e-12

P value adjustment method: bonferroni

```

Die Unterschiede sind zwischen allen Gruppen signifikant.

Das Zusatzpaket `rstatix` bietet mit der Funktion `anova_test()` eine alternative Vorgehensweise.

```

1 # ANOVA mit Messwiederholung
2 fit <- rstatix::anova_test(data = df, dv = Wert, wid = Name,
3                             within = Zeitpunkt, effect.size = "pes")
4
5 # auf Sphärizität prüfen
6 fit$`Mauchly's Test for Sphericity`

```

```

      Effect      W      p p<.05
1 Zeitpunkt 0.668 4.29e-18      *

```

Mauchly's Test ist signifikant, das heisst, es liegt *keine* Sphärizität vor.  hat in diesem Falle bereits die Freiheitsgrade automatisch korrigiert (mittels Greenhouse-Geisser-Verfahren).

Schauen wir uns die ANOVA an:

```

1 rstatix::get_anova_table(fit) Das geht auch graphisch wie oben beschrieben:

```

```

ANOVA Table (type III tests)

      Effect DFn  DFd    F      p p<.05  pes
1 Zeitpunkt 1.5 298.73 212.34 2.18e-48      * 0.516

```

Die ANOVA ist signifikant. Jetzt können wir analysieren, welche konkreten Gruppenunterschiede vorliegen.

2.2. SCHLIESSENDE STATISTIK

```
1 # post-hoc Analyse: paarweise Gruppenvergleiche
2 df %>%
3   rstatix::pairwise_t_test(Wert ~ Zeitpunkt, paired = TRUE,
4                             p.adjust.method = "bonferroni") %>%
5   as.data.frame()
```

	.y.	group1	group2	n1	n2	statistic	df	p	p.adj	p.adj.signif
1	Wert	t1	t2	200	200	11.24384	199	5.02e-23	1.51e-22	****
2	Wert	t1	t3	200	200	18.28475	199	1.79e-44	5.37e-44	****
3	Wert	t2	t3	200	200	11.26870	199	4.23e-23	1.27e-22	****

Die Unterschiede sind zwischen allen Gruppen signifikant.
Die Effektstärke kann wie folgt berechnet werden.

```
1 # Effektstärke (nur solche, die post-hoc signifikant sind, interpretieren)
2 df %>%
3   rstatix::cohens_d(Wert ~ Zeitpunkt, paired = TRUE) %>%
4   as.data.frame()
```

	.y.	group1	group2	effsize	n1	n2	magnitude
1	Wert	t1	t2	0.7950598	200	200	moderate
2	Wert	t1	t3	1.2929274	200	200	large
3	Wert	t2	t3	0.7968171	200	200	moderate

Achtung, wir dürfen nur solche Werte interpretieren, die oben signifikant waren.
In unserem Beispiel sind alle Werte signifikant, darum dürfen wir alle Effekte interpretieren.

2.2.3.2 Friedman ANOVA

Sind die Werte nicht normalverteilt, kann ein Friedman-ANOVA gerechnet werden. Wir nutzen den Testdatensatz `MarioANOVA.rda`. Er enthält Informationen über 47 Super Mario Charaktere, deren Fehlerquote beim Spielen eines schweren Levels zu 3 Zeitpunkten erfasst wurde.

```
1 load(url("https://www.produnis.de/R/MarioANOVA.rda"))
2
3 # anschauen
4 head(MarioANOVA)
```

```
# A tibble: 6 × 8
  Name      Alter Kingdom Geschlecht BadGuy  t1    t2    t3
<fct>      <dbl> <fct>      <fct>    <lgl> <dbl> <dbl> <dbl>
1 Mario         35 Mushroom Kingdom männlich FALSE 0.372 0.472 0.257
2 Luigi         35 Mushroom Kingdom männlich FALSE 1.27  1.34  1.14
3 Prinzessin Peach 25 Mushroom Kingdom weiblich  FALSE 6.15  1.40  1.98
4 Bowser        45 Bowser's Castle männlich  TRUE  4.85  0.648 1.27
5 Toad          20 Mushroom Kingdom männlich  FALSE 1.34  0.174 1.77
6 Toadette      19 Mushroom Kingdom weiblich  FALSE 0.727 0.413 2.09
```

In den Variablen `t1` bis `t3` liegen die Werte der 3 Messzeitpunkte. Auch hier überführen wir die Daten ins `long table`-Format.

```
1 df <- MarioANOVA %>%
2   select(Name, t1:t3) %>%
3   pivot_longer(cols=t1:t3,
4                 names_to = "Zeitpunkt",
5                 values_to = "Wert")
```

Schauen wir uns die Daten zunächst deskriptiv an.

```
1 # mal deskriptiv anschauen
2 df %>%
3   group_by(Zeitpunkt) %>%
4   summarise(M = mean(Wert),
5             SD = sd(Wert),
6             Q1 = quantile(Wert, probs = 0.25),
7             Q2 = quantile(Wert, probs = 0.5),
8             Q3 = quantile(Wert, probs = 0.75),
9             IQR = IQR(Wert),
10            N = n())
```

```
# A tibble: 3 × 8
  Zeitpunkt     M     SD    Q1    Q2    Q3   IQR     N
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
1 t1        2.75  4.61  0.750  1.34  2.46  1.71    47
2 t2        0.762  0.585  0.356  0.509  1.15  0.791    47
3 t3        0.901  0.701  0.266  0.650  1.34  1.07    47
```

Das geht auch graphisch wie oben beschrieben:

```
1 # mal graphisch anschauen
2 boxplot(df$Wert ~ df$Zeitpunkt)
```

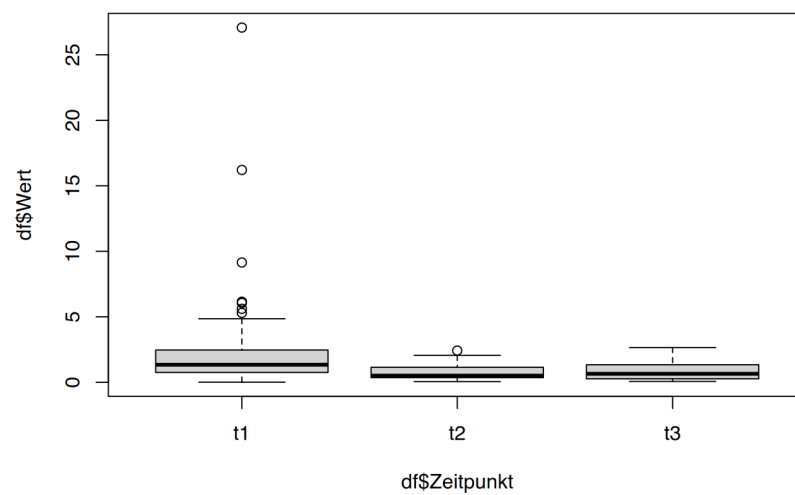


Abb. 2.19

```
1 ez::ezPlot(df, Wert, Name, within = Zeitpunkt, x = Zeitpunkt)
```

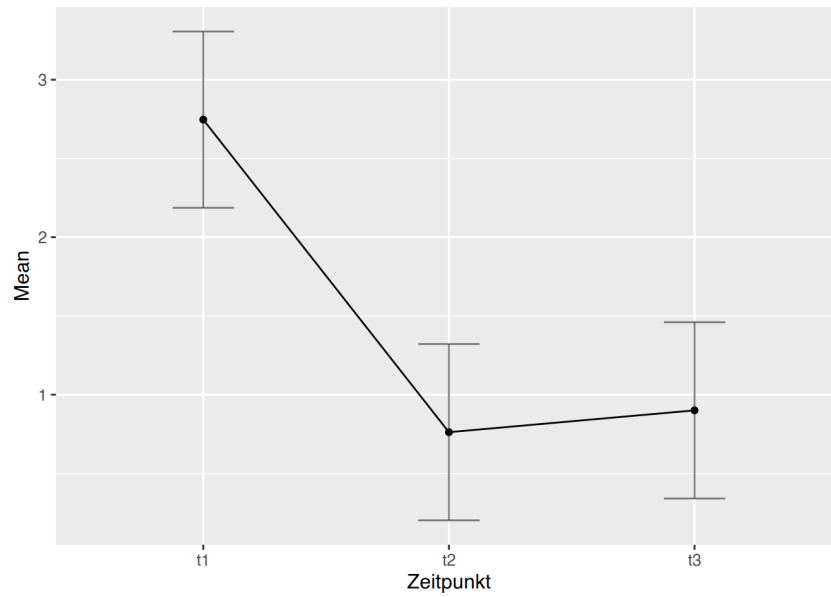


Abb. 2.20

```

1 # mit Regressionsgerade
2 ez::ezPlot(df, Wert, Name, within = Zeitpunkt, x = Zeitpunkt) +
3   geom_smooth(aes(x = as.numeric(Zeitpunkt)), method = 'lm', se = F)

```

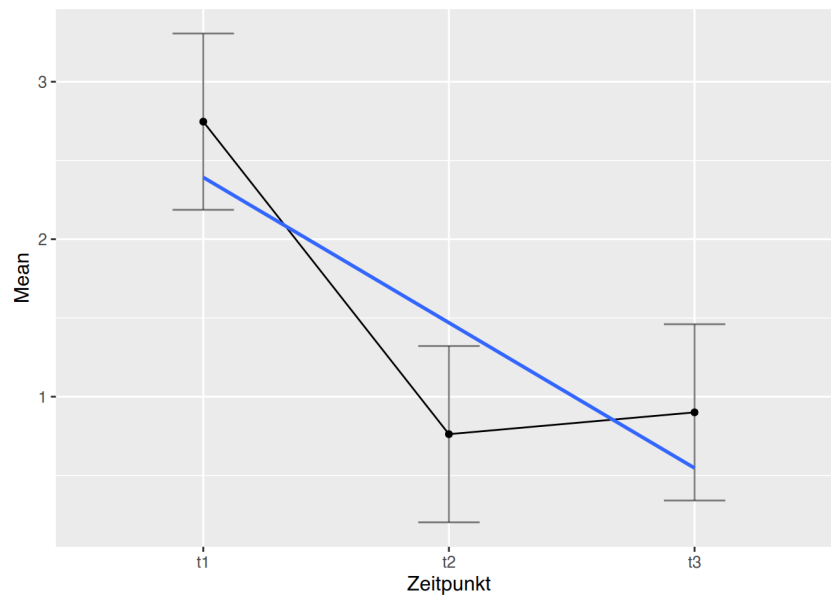


Abb. 2.21

Nun prüfen wir mittels QQ-Plot und Shapiro-Test auf Normalverteilung.

```
1 ## qq-Plot (Normalverteilung)
2 ggpubr::ggqqplot(df, x="Wert",
3                   facet.by = "Zeitpunkt",
4                   color = "Zeitpunkt",
5                   scales = "free")
```

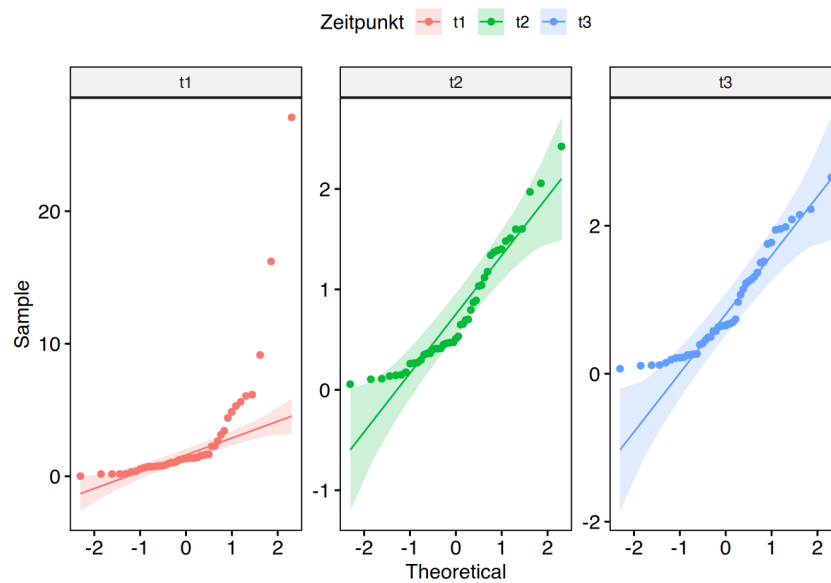


Abb. 2.22

```
1 # Teste "Wert" auf Normalverteilung
2 shapiro.test(df$Wert[df$Zeitpunkt=="t1"])
```

Shapiro-Wilk normality test

data: df\$Wert[df\$Zeitpunkt == "t1"]
W = 0.52189, p-value = 3.729e-11

```
1 # Teste "Wert" auf Normalverteilung
2 shapiro.test(df$Wert[df$Zeitpunkt=="t2"])
```

```
Shapiro-Wilk normality test

data: df$Wert[df$Zeitpunkt == "t2"]
W = 0.89427, p-value = 0.0004751
```

```
1 # Teste "Wert" auf Normalverteilung
2 shapiro.test(df$Wert[df$Zeitpunkt=="t3"])
```

```
Shapiro-Wilk normality test

data: df$Wert[df$Zeitpunkt == "t3"]
W = 0.90101, p-value = 0.0007756
```

Die Werte für t1 bis t3 sind signifikant, also **nicht** normalverteilt.

Wir führen daher eine Friedman-ANOVA mit Messwiederholung durch.

```
1 rstatix::friedman_test(Wert ~ Zeitpunkt | Name, data=df)
```

```
# A tibble: 1 × 6
  .y.      n statistic    df      p method
* <chr> <int>    <dbl> <dbl>    <dbl> <chr>
1 Wert     47     12.8      2 0.00165 Friedman test
```

Das Ergebnis ist signifikant, es gibt also einen Unterschied.

```
1 # post-hoc-test
2 df %>%
3   rstatix::wilcox_test(Wert ~ Zeitpunkt, paired=T,
4     p.adjust.method = "bonferroni")
```

```
# A tibble: 3 × 9
  .y. group1 group2    n1    n2 statistic      p    p.adj p.adj.signif
* <chr> <chr> <chr> <int> <int>    <dbl>    <dbl>    <dbl> <chr>
1 Wert  t1    t2      47    47     917 0.000102 0.000306 ***
2 Wert  t1    t3      47    47     882 0.000536 0.002    **
3 Wert  t2    t3      47    47     506 0.546    1      ns
```


Der Unterschied existiert zwischen t1 und t2 sowie zwischen t1 und t3.

Die Effektstärke wird wie folgt berechnet:

```
1 # Effektstärke
2 rstatix::friedman_effsize(Wert ~ Zeitpunkt | Name, data = df)
```

```
# A tibble: 1 × 5
  .y.      n effsize method      magnitude
* <chr> <int>   <dbl> <chr>      <ord>
1 Wert    47   0.136 Kendall W small
```

Der Effekt des Gesamtmodells ist eher gering. Schauen wir uns die einzelnen Gruppeneffekte an. Achtung, wir dürfen nur solche Werte interpretieren, die oben signifikant waren. In unserem Beispiel ist das der Unterschied von t1 zu t2 sowie von t1 zu t3.

```
1 # nur die signifikanten anschauen (t1 -> t2 und t1 -> t3)
2 df %>%
3   rstatix::wilcox_effsize(Wert ~ Zeitpunkt, paired = TRUE)
```

```
# A tibble: 3 × 7
  .y. group1 group2 effsize    n1    n2 magnitude
* <chr> <chr> <chr>   <dbl> <int> <int> <ord>
1 Wert  t1    t2    0.545    47    47 large
2 Wert  t1    t3    0.491    47    47 moderate
3 Wert  t2    t3    0.0895   47    47 small
```

Der Effekt von t1 nach t2 ist “groß”, der von t1 nach t3 ist “moderat”.

2.2.3.3 einfaktorielle ANOVA

Eine einfaktorielle Varianzanalyse (ANOVA) wird mit der Funktion `aov()` durchgeführt. Sie wird nach der Syntax *x erklärt durch y* aufgerufen. Wie bei anderen Modellen ist es ratsam, die Ergebnisse der Berechnung in ein Objekt zu speichern, und dieses per `summary()` anzuschauen

```
1 # Lade Testdatensatz
2 load(url("https://www.produnis.de/R/pf8.RData"))
3
4 # erstelle ANOVA "Alter" erklärt durch "Standort"
5 fit <- aov(Alter ~ Standort, data=pf8)
6
7 # schaue Modellzusammenfassung an
8 summary(fit)
```

```
      Df Sum Sq Mean Sq F value    Pr(>F)
Standort    4 10557  2639.3     8.48 1.09e-06 ***
Residuals 725 225633   311.2
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
1 observation deleted due to missingness
```

Das Ergebnis ist signifikant, wir haben unterschiedliche Varianzen im Alter erklärt durch Standort.

Wir wissen aber noch nicht, innerhalb welche Standorte die Unterschiede bestehen. Dazu führen wir post hoc Tests durch, in denen alle Gruppen paarweise verglichen werden. Wegen dieses multiplen Testens muss eine Fehlerkorrektur für Alpha durchgeführt werden. Dazu steht die Funktion `pairwise.t.test()` zur Verfügung, die mit den selben Angaben aufgerufen wird.

```
1 pairwise.t.test(pf8$Alter, pf8$Standort)
```

```
Pairwise comparisons using t tests with pooled SD

data:  pf8$Alter and pf8$Standort

      Rheine Münster Bahn  Ladbergen
Münster 0.5042 -      -      -
Bahn    0.4354 1.0000 -      -
Ladbergen 0.5432 1.0000 1.0000 -
Internet 0.0090 1.8e-05 0.0002 0.0059
```

```
P value adjustment method: holm
```

Die gewählte Fehlerkorrektur ist standardmäßig `holm`. Möchten Sie auf die Bonferroni-Korrekturmethode wechseln, übergeben Sie den Parameter `p.adjust="bonferroni"`

```
1 pairwise.t.test(pf8$Alter, pf8$Standort, p.adjust="bonferroni")
```

```
Pairwise comparisons using t tests with pooled SD

data: pf8$Alter and pf8$Standort

      Rheine  Münster Bahn   Ladbergen
Münster 1.00000 -        -        -
Bahn    0.72572 1.00000 -        -
Ladbergen 1.00000 1.00000 1.00000 -
Internet 0.01292 1.8e-05 0.00022 0.00733

P value adjustment method: bonferroni
```

Die signifikanten Unterschiede befinden sich zwischen dem Standort **Internet** und allen anderen Standorten.

Ein anschauliches Video zur ANOVA in  findet sich bei Youtube:
<https://www.youtube.com/watch?v=GctGncQrJew>

2.2.4 Fallzahlkalkulation

2.2.4.1 klinische Studien

Zur Fallzahlkalkulation bei klinischen Studien (z.B. RCTs) kann das Zusatzpaket `pwr` installiert werden. Mit der Funktion `pwr.t.test()` können alle Werte berechnet werden. Ihr müssen die entsprechenden Werte für das Signifikanzlevel, Power und Cohen's d (Effektstärke) übergeben werden.

```

1  # berechne Fallzahl mit
2  # Cohens d = 0.1
3  # alpha = 0.05
4  # Power = 0.8
5  pwr::pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="two.sample",
  alternative="two.sided")

```

```

Two-sample t test power calculation

      n = 1570.733
      d = 0.1
sig.level = 0.05
  power = 0.8
alternative = two.sided

NOTE: n is number in *each* group

```

Über die Parameter `type` und `alternative` lassen sich die Testbedingungen festlegen.

```

1  # Lade Paket
2  library(pwr)
3
4  # unabhängige Stichprobe, zweiseitig
5  pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="two.sample", alternative="two.sided")
6
7  # Einstichproben-Test, kleiner
8  pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="one.sample", alternative="less")
9
10 # gepaarte Stichprobe, größer
11 pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="paired", alternative="greater")

```

Möchten Sie nicht immer die gesamte Testausgabe angezeigt bekommen, sondern nur den Wert für `n`, können Sie dies mit einem Dollarzeichen `$` hinter dem Funktionsaufruf spezifizieren.

```
1 # zeige nur "n"
2 pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="two.sample")$n
```

```
[1] 1570.733
```

```
1 # runde auf nächste ganze Zahl
2 ceiling(pwr.t.test(d=0.1, sig.level=0.05, power=0.8, type="two.sample")$n)
```

```
[1] 1571
```

Sie benötigen 1571 Probanden pro Untersuchungsgruppe.

2.2.4.2 Umfragen

Soll eine Befragung (ein *Survey*) durchgeführt werden, kann die benötigte Fallzahl über das Zusatzpaket **samplingbook** errechnet werden.

Geht es um *numerische* Werte, wird die Funktion `sample.size.mean()` verwendet. Ihr übergibt man Fehlerspanne *e* (ein Präzisionswert, der besagt, wie breit das Konfidenzintervall - nach oben sowie nach unten - sein darf, für gewöhnlich 5% bzw. 0.05), das Konfidenzniveau, die Standardabweichung *S* innerhalb der Grundgesamtheit (für gewöhnlich 50% bzw. 0.5) sowie die Populationsgröße. Standardmäßig ist die Populationsgröße auf *N=Inf* (für unendlich) gesetzt.

```
1 # Berechne Fallzahl für einen Survey
2 # Konfidenzniveau auf 95%, Fehlerbereich 5%.
3 # In der Grundgesamtheit hat der Wert eine
4 # Standardabweichung von S=0.5
5 samplingbook::sample.size.mean(e=0.05, S=0.5, level=0.95 )
```

```
sample.size.mean object: Sample size for mean estimate
Without finite population correction: N=Inf, precision e=0.05 and standard deviation
↪ S=0.5

Sample size needed: 385
```

Ist die Grundgesamtheit bekannt, kann dies mit N angegeben werden.

```
1 # Die Grundgesamtheit ist 1.500 Personen groß
2 samplingbook::sample.size.mean(e=0.05, S=0.5, level=0.95, N=1500 )
```

```
sample.size.mean object: Sample size for mean estimate
With finite population correction: N=1500, precision e=0.05 and standard deviation
↪ S=0.5

Sample size needed: 306
```

Sollen Prozentwerte **kategorialer** Variablen erfragt werden (z.B. der Anteil weiblicher Pflegepersonen), wird die Funktion `sample.size.prop()` verwendet. Ihr werden ebenso `e`, `N` und `level` (Konfidenzniveau) übergeben. Statt der Standardabweichung wird über den Parameter `P` die prozentuale Verteilung in der Grundgesamtheit angegeben.

```
1 # Berechne Fallzahl für einen Survey
2 # unsere Werte dürfen maximal 5% abweichen
3 # Konfidenzniveau auf 95%
4 # In der Grundgesamtheit hat der Wert eine
5 # Verteilung von 65%
6 # Die Grundgesamtheit ist 1.500 Personen groß
7 samplingbook::sample.size.prop(e=0.05, P=0.65, N=1500)
```

```
sample.size.prop object: Sample size for proportion estimate
With finite population correction: N=1500, precision e=0.05 and expected proportion
↪ P=0.63

Sample size needed: 290
```

2.3 Diagramme plotten

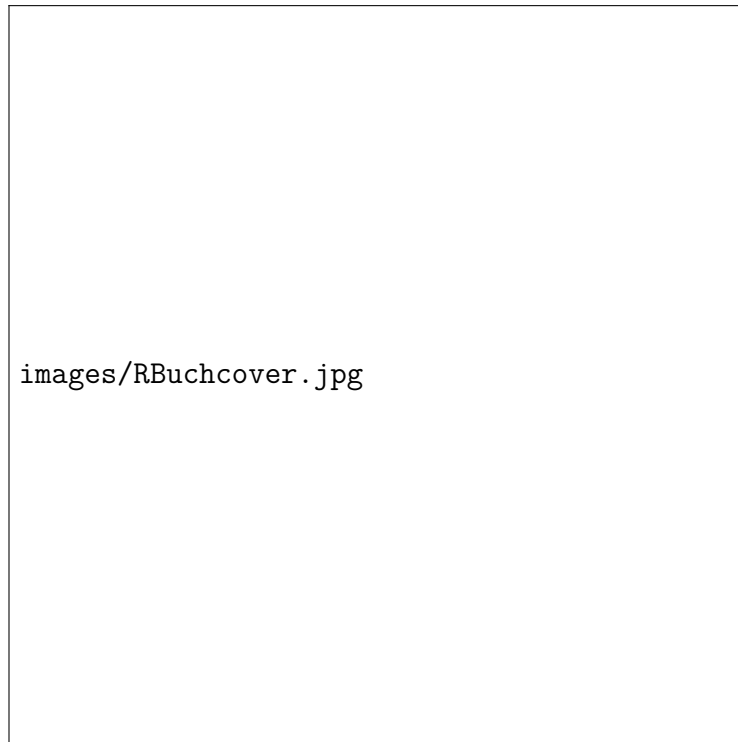


Abb. 2.23: Ein Plot mit klassischem  erstellt

In diesem Kapitel werden die Plotfunktionen des *klassischen*  vorgestellt. Sie erzeugen immernoch brauchbare Ergebnisse, und man kann viele Einstellungen vornehmen. Wenn Sie allerdings ein etwas komplexeres Diagramm aufbauen möchten, lohnt es sich nicht, die Spezialeinstellungen der klassischen Plotfunktionen zu durchstöbern. Schwenken Sie in diesem Falle lieber gleich zum moderneren `ggplot()` um, welches in Kapitel 2.4 ab Seite 322 vorgestellt wird. So kommen Sie viel “einfacher” an Ihre Ziel.

Die universelle Funktion zum erstellen von Diagrammen heisst `plot()`. Mit ihr werden ein Großteil der Diagramme erstellt, und viele Funktionen rufen letztendlich auch nur `plot()` auf.

Da `plot()` so universal ist, nimmt sie viele Parameter entgegen. Tabelle 2.1 (Seite 299) zeigt diese Parameter in einer Übersicht. Viele der Parameter sind

2.3. DIAGRAMME PLOTTEN

Parameter	Argumente / Beschreibung	Beispiel
type (mittlerweile veraltet, funktioniert aber noch)	legt den Diagrammtyp fest l = Liniendiagramm p = Punktdiagramm b = (both) Kombination aus l und p s = Stufendiagramm h = Histogramm- <i>artig(!)</i> n = keine Darstellung	type="p"
col	Füllfarbe (colors() zeigt alle Farben) gray0 = schwarz bis gray99 = weiß	col="blue" col="#112233" (RGB) col="gray77"
main	Überschrift	main="Mein Plot"
sub	Untertitel	sub="Abbildung 12"
xlab	Beschriftung der X-Achse	xlab="Jahrgänge"
ylab	Beschriftung der Y-Achse	ylab="Häufigkeit"
xlim	Start- und Endwert der X-Achse	xlim=c(2,10)
ylim	Start- und Endwert der Y-Achse	ylim=c(0,100)
xaxt	X-Achse komplett unbeschriftet	xaxt="n"
yaxt	Y-Achse komplett unbeschriftet	yaxt="n"
pch	Darstellungssymbol 0 □ 1 ○ 2 △ 3 + 4 × 5 ◇ 6 ▽ 7 ⊠ 8 ✱ 9 ⊕ 10 ⊗ 11 ⊛ 12 ⊞ 13 ⊠ 14 ⊞ 15 ■ 16 ● 17 ▲ 18 ◆ 19 ● 20 ● 21 ○ 22 □ 23 ◇ 24 △ 25 ▽	pch=4
cex	Symbolgröße	cex=2
lty	Linientyp 1 ————— 4 2 5 3 6	lty=1
lwd	Strichstärke der Linien	lwd=3
bty	Boxtyp, setzt Rahmen ums Plot n = kein Rahmen o = komplett C = oben, unten und links L = unten und links u = unten, rechts und links 7 = oben und rechts (die Buchstaben sehen so aus wie der Rahmen)	bty="o"
font	Fontformat 1 = normal 2 = fett 3 = kursiv 5 = griechische Symbole	font=2

Tabelle 2.1: Parameter der Funktion `plot()`

auch in weiteren Plotfunktionen implementiert. Es kann dennoch nie schaden, sich mit `?barplot` in der -Konsole die expliziten Parameter zu vergegenwärtigen.

2.3.1 Punktwolke

Die Punktwolke wird mit besagter Funktion `plot()` erstellt. Ihr werden zwei Vektoren mit den entsprechenden x und y Werten übergeben.

```

1  # erzeuge Testwerte
2  x <- sort(rnorm(100, mean=12, sd=2))
3  y <- sort(rnorm(100, mean=140, sd=15))
4
5  # Plotten
6  plot(x,y)

```

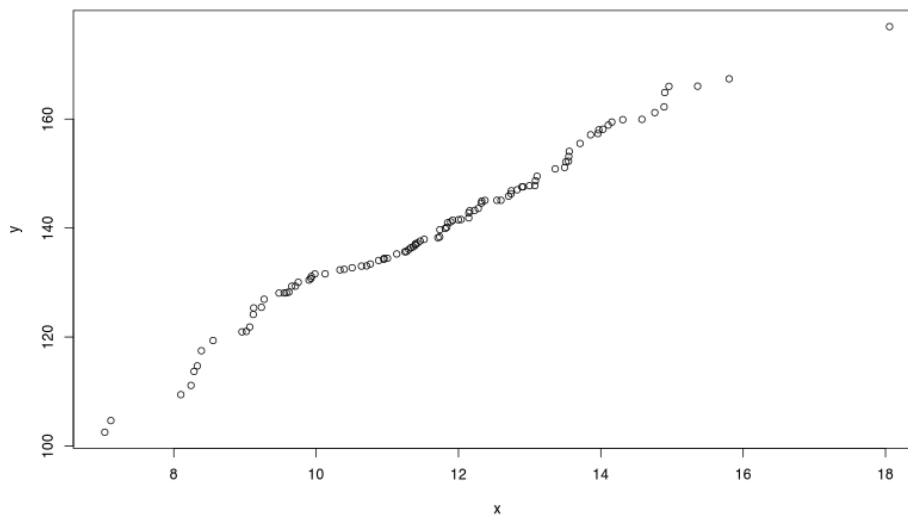


Abb. 2.24

Über die Parameter aus Tabelle 2.1 kann das Plot aufgehübscht werden

```

1  plot(x,y, col="blue",
2        ylab="Körpergröße", xlab="Alter",
3        main="Ein Plot mit R base",
4        sub="Dieses Plot wurde aus Zufallswerten erstellt",
5        pch=13)

```

2.3. DIAGRAMME PLOTTEN

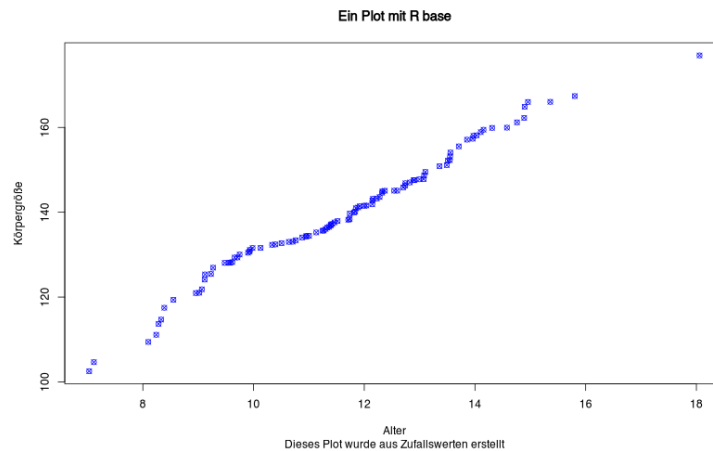


Abb. 2.25

Mit der Funktion `abline()` kann dem Plot eine Gerade hinzugefügt werden, z.B. eine Regressionsgerade. Die Funktion benötigt als Angaben den Schnitt durch die Y-Achse sowie den Steigungswert. Die Funktion `lm()` liefert für Regressionsgeraden genau diese Werte.

```
1 # Regressionswerte von Körpergröße ~ Alter
2 lm(y~x)
```

```
Call:
lm(formula = y ~ x)
```

```
Coefficients:
(Intercept)          x
      59.50         6.89
```

Das heisst, wir rufen die Funktion innerhalb von `abline()` auf

```
1 plot(x,y, col="blue",
2       ylab="Körpergröße", xlab="Alter",
3       main="Ein Plot mit R base",
4       sub="Dieses Plot wurde aus Zufallswerten erstellt",
5       pch=13)
6 abline(lm(y~x))
```

2.3. DIAGRAMME PLOTTEN

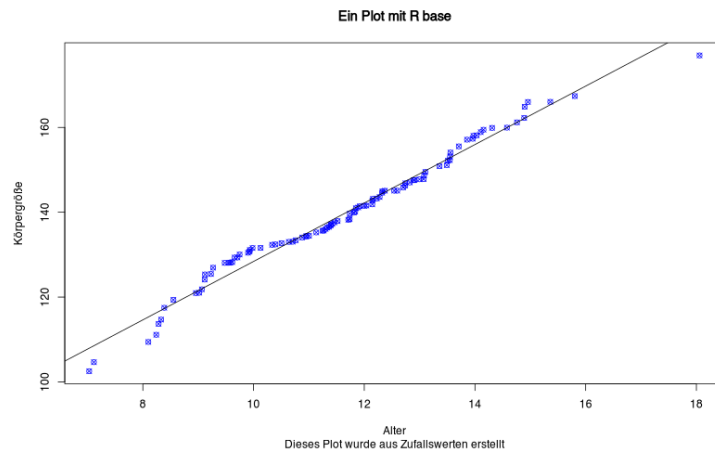


Abb. 2.26

In rot sieht sie evtl. besser aus:

```
1 plot(x,y, col="blue",  
2      ylab="Körpergröße", xlab="Alter",  
3      main="Ein Plot mit R base",  
4      sub="Dieses Plot wurde aus Zufallswerten erstellt",  
5      pch=13)  
6 abline(lm(y~x), col="red")
```

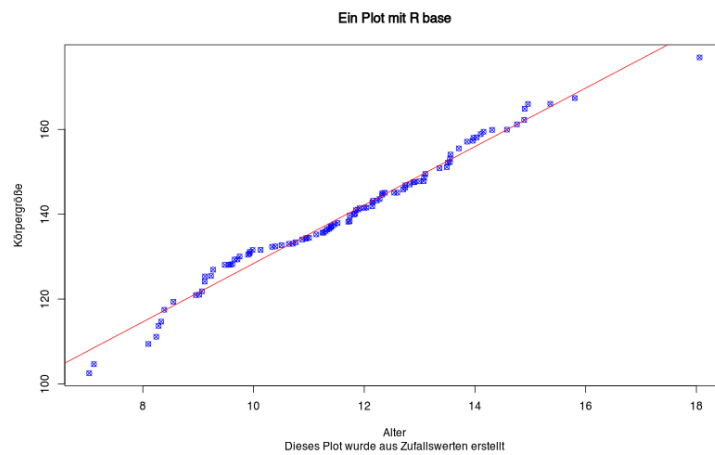


Abb. 2.27

Mit der Funktion `lines()` können Linien hinzugefügt werden. Dafür übergeben wir die entsprechenden Koordinaten des Start- und Endpunkts. So können wir z.B. den Mittelwert für `Alter` markieren

2.3. DIAGRAMME PLOTTEN

```
1  # Mittelwert als x-Koordinate
2  mx <- mean(x)
3
4  # Berechne Schnittpunkt mit Regressionsgeraden
5  ymx <- coef(lm(y~x))[1] + mx*coef(lm(y~x))[2]
6
7  plot(x,y, col="blue",
8       ylab="Körpergröße", xlab="Alter",
9       main="Ein Plot mit R base",
10      sub="Dieses Plot wurde aus Zufallswerten erstellt",
11      pch=13)
12  abline(lm(y~x), col="red")
13
14  # Füge Mittelwertlinie hinzu
15  lines(x=c(mx, mx), y=c(0, ymx), lty=3)
```

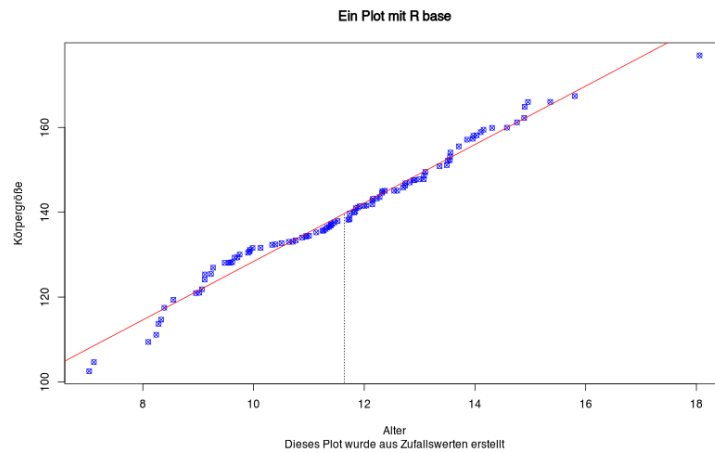


Abb. 2.28

Mit der Funktion `polygon()` können Polygone hinzugefügt werden.

```
1  # Mittelwert als x-Koordinate
2  mx <- mean(x)
3
4  # Berechne Schnittpunkt mit Regressionsgeraden
5  ymx <- coef(lm(y~x))[1] + mx*coef(lm(y~x))[2]
6
7  plot(x,y, col="blue",
8       ylab="Körpergröße", xlab="Alter",
9       main="Ein Plot mit R base",
10      sub="Dieses Plot wurde aus Zufallswerten erstellt",
11      pch=13)
12  abline(lm(y~x), col="red")
```

2.3. DIAGRAMME PLOTTEN

```
13
14 # Füge Mittelwertlinie hinzu
15 lines(x=c(mx, mx), y=c(0, ymx), lty=3)
16
17 # füge Polygon hinzu
18 polygon(x=c(x, max(x), min(x)), y=c(y, 0, 0), col="snow2")
```

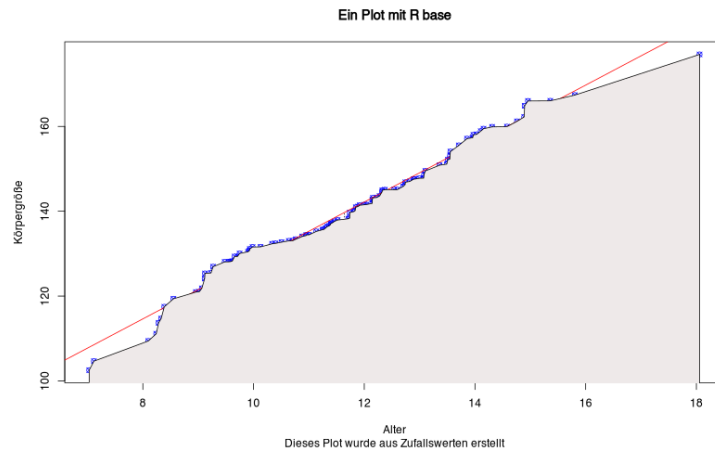


Abb. 2.29

So kann ein Diagramm der Standardnormalverteilung erstellt werden.

```
1 # erzeuge 10.000 Zufallswerte der Standardnormalverteilung
2 x <- sort(rnorm(10000))
3
4 # erzeuge die passenden Dichterwerte
5 y <- dnorm(x)
6
7 # beginne mit Plot
8 plot(x, y, ylab="Dichtefunktion", xlab="x", main="Die Standardnormalverteilung",
9      sub="Bereich von 1 Standardabweichung")
10
11 # füge gefülltes Polygon hinzu
12 polygon(x=c(x, max(x), min(x)), y=c(y, 0, 0), col="snow2")
13
14 # füge 1 Standardabweichung hinzu
15 lines(x=c(-1, -1), y=c(0, dnorm(-1)), lty=3)
16 lines(x=c(1, 1), y=c(0, dnorm(1)), lty=3)
17
18 # füge Polygon für 1. Standardabweichung hinzu
19 xsd1 <- x[x<1 & x>-1]
20 ysd1 <- dnorm(xsd1)
21 polygon(x=c(xsd1, max(xsd1), min(xsd1)), y=c(ysd1, 0, 0), col="green")
```

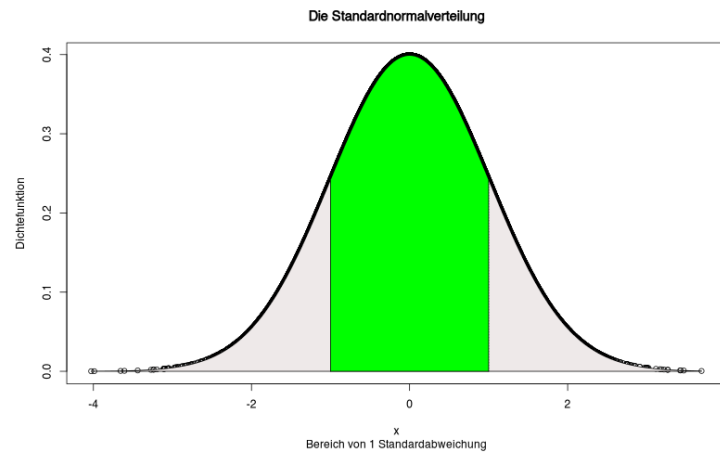


Abb. 2.30

Der Bereich von zwei Standardabweichungen kann wie folgt hinzugefügt werden. Dies “übermalt” jedoch die grüne Fläche.

```

1  # füge 2. Standardabweichung hinzu
2  lines(x=c(-2, -2), y=c(0, dnorm(-2)), lty=6)
3  lines(x=c(2, 2), y=c(0, dnorm(2)), lty=6)
4
5  # füge Polygon für 2. Standardabweichung
6  # dies übermalt das 1. Polygon
7  xsd2 <- x[x<2 & x>-2]
8  ysd2 <- dnorm(xsd2)
9  polygon(x=c(xsd2, max(xsd2), min(xsd2)), y=c(ysd2, 0, 0 ), col="blue")

```

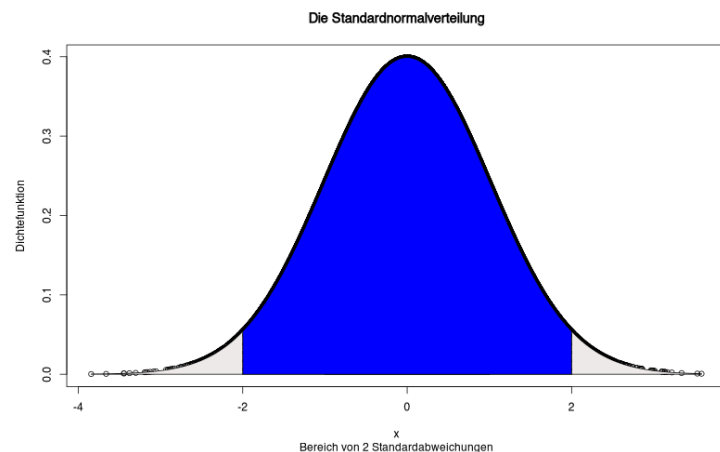


Abb. 2.31

2.3. DIAGRAMME PLOTTEN

Liegen mehrere Variablen in einem Datenframe vor, stellt `plot()` nach Kombinationen getrennt dar.

```
1 x <- data.frame(Lesetest = sort(rnorm(100, mean=20, sd=6)),
2                 Rechentest = rnorm(100, mean=15, sd=12),
3                 Rechtschreibung = sort(rnorm(100, mean=21, sd=4))
4 )
5
6 plot(x)
```

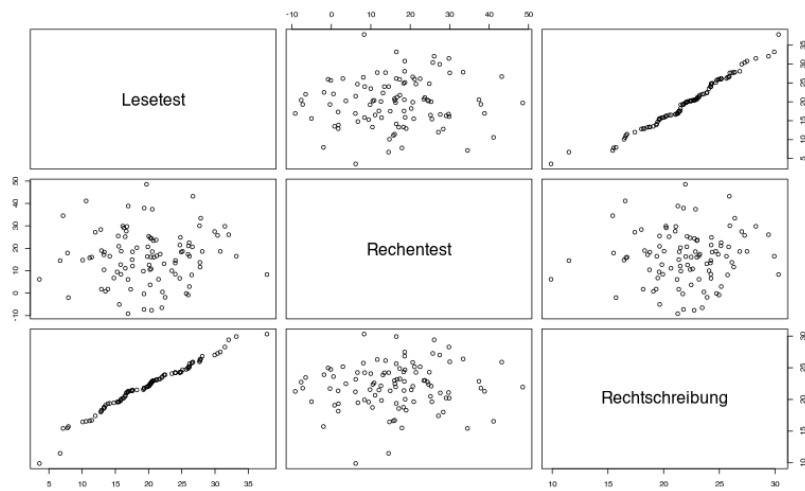


Abb. 2.32

Je mehr Variablen vorliegen, desto komplexer wird der Plot

```
1 x <- data.frame(Lesetest = sort(rnorm(100, mean=20, sd=6)),
2                 Rechentest = rnorm(100, mean=15, sd=12),
3                 Rechtschreibung = sort(rnorm(100, mean=21, sd=4)),
4                 Grammatikfehler = sort(rnorm(100, mean=21, sd=4), decreasing=T)
5 )
6
7 plot(x)
```

2.3. DIAGRAMME PLOTTEN

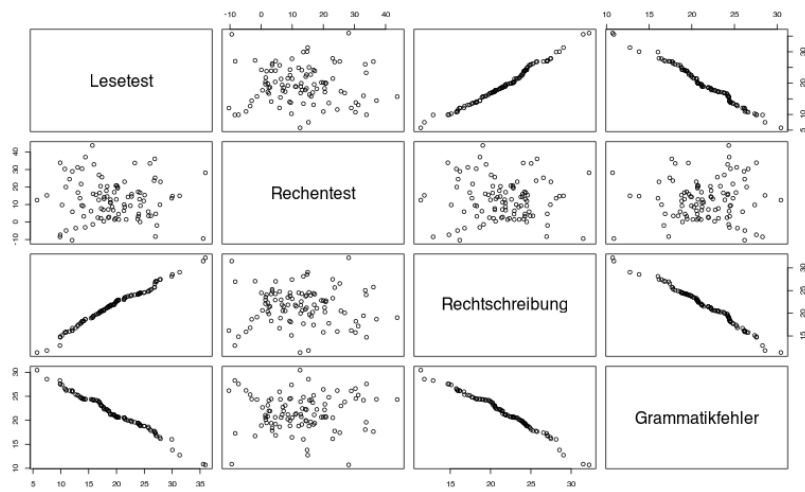


Abb. 2.33

Mit Farben kann man es etwas aufhübschen.

```
1 x <- data.frame(Lesetest = sort(rnorm(100, mean=20, sd=6)),
2                 Rechentest = rnorm(100, mean=15, sd=12),
3                 Rechtschreibung = sort(rnorm(100, mean=21, sd=4)),
4                 Grammatikfehler = sort(rnorm(100, mean=21, sd=4), decreasing=T)
5 )
6
7 plot(x, col=rainbow(3))
```

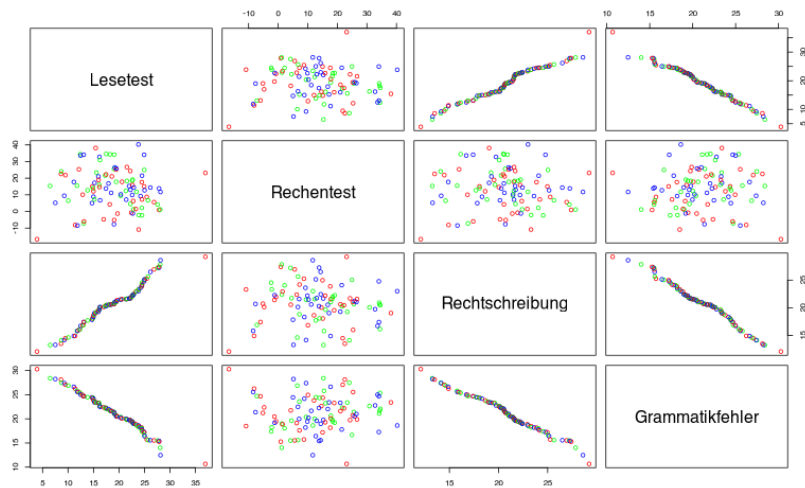


Abb. 2.34

2.3.2 verfügbare Farben

Die Funktion `colors()` zeigt die verfügbaren Farben an.

```
1 # zeige verfügbare Farben an
2 colors()
```

```
[1] "white"           "aliceblue"       "antiquewhite"
[4] "antiquewhite1"   "antiquewhite2"   "antiquewhite3"
[7] "antiquewhite4"   "aquamarine"      "aquamarine1"
[10] "aquamarine2"     "aquamarine3"     "aquamarine4"
[13] "azure"           "azure1"          "azure2"
[16] "azure3"          "azure4"          "beige"
[19] "bisque"          "bisque1"         "bisque2"
[22] "bisque3"         "bisque4"         "black"
[25] "blanchedalmond"  "blue"            "blue1"
[28] "blue2"           "blue3"           "blue4"
[31] "blueviolet"      "brown"           "brown1"
[34] "brown2"          "brown3"          "brown4"
[37] "burlywood"       "burlywood1"      "burlywood2"
[40] "burlywood3"      "burlywood4"      "cadetblue"
[43] "cadetblue1"      "cadetblue2"      "cadetblue3"
[46] "cadetblue4"      "chartreuse"       "chartreuse1"
[49] "chartreuse2"     "chartreuse3"     "chartreuse4"
[52] "chocolate"       "chocolate1"      "chocolate2"
[55] "chocolate3"     "chocolate4"      "coral"
( . . . )
```

Der Output ist 657 Farben lang.

```
1 # wieviele Farben gibt es?
2 length(colors())
```

```
[1] 657
```

Zu erwähnen ist noch die Farbe **gray**, die in den Abstufungen **gray0** (=schwarz) bis **gray99** (= weiß) knapp 99 Grauschattierungen ermöglicht. Dies ist bei schwarz-weiß Publikationen sehr hilfreich.

Soll es möglichst bunt sein, erzeugt die Funktion **rainbow()** die gewünschte Anzahl an Farben.

Eine Übersicht der Farben in “echten Farbtönen” findet sich bei Björn Walther: <http://www.bjoernwalther.com/farben-in-r-der-col-befehl/>

2.3.3 Histogram

Ein Histogramm wird mit der Funktion `hist()` erstellt. Plotten wir ein Histogramm der Variable `Alter` im Beispieldatensatz `pf8`.

```
1 # lade Datensatz "pf8"
2 load(url("https://www.produnis.de/R/pf8.RData"))
3
4 # erstelle Histogramm von Variable "Alter" aus Datensatz "pf8"
5 hist(pf8$Alter)
```

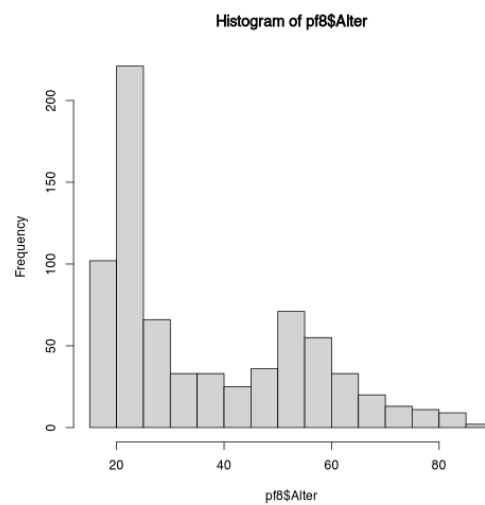


Abb. 2.35

```
1 # zum Vergleich plot()
2 plot(pf8$Alter, type="h")
```

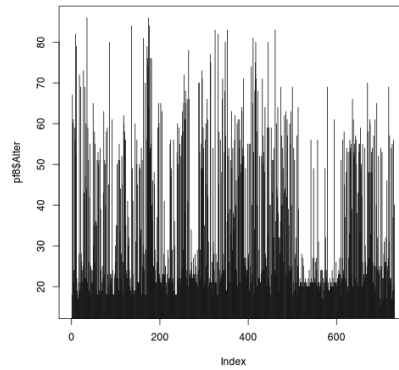


Abb. 2.36

Wie Sie sehen ist das `plot()`-Resultat nur entfernt dem Histogramm ähnlich.

Die Klassengröße des Histogramms kann mit dem Parameter `breaks()` übergeben werden. Entweder schreiben wir die Anzahl der “Balken”, dann versucht `R` das “möglichst gut” auszuführen, oder wir übergeben einen Vektor mit den konkreten Breakpunkten.

```
1 # Histogramm mit "möglichst" 4 Balken
2 hist(pf8$Alter, breaks=4)
```

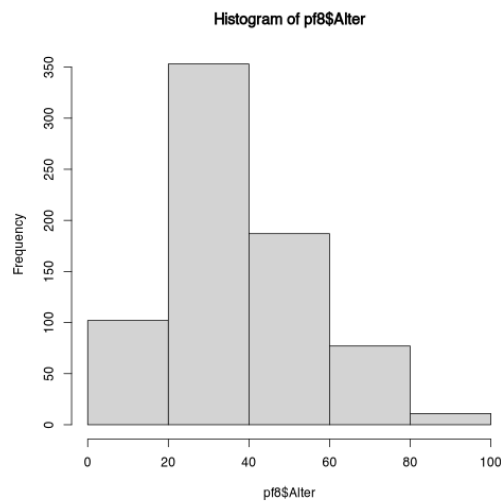


Abb. 2.37

Wie Sie sehen stellt  dennoch 5 Balken dar, weil es so “besser” aussieht als mit nur 4 Balken. Die Anzahl der Balken ist also kein *konkreter* Befehl, sondern eher ein *Serviovorschlag*.

Über die Parameter (siehe Tabelle 2.1 (Seite 299)) kann der Plot nun aufgehübscht werden.

```

1  # Histogramm mit "möglichst" 4 Balken
2  # in schön
3  hist(pf8$Alter, breaks=4,
4      # Überschrift
5      main="Mein Histogramm",
6      # Unterschrift
7      sub="Abbildung x.y: Ein Histogramm",
8      # X-Achse
9      xlab="Alter in Jahren",
10     # Y-Achse
11     ylab="Häufigkeit",
12     # blaue Balken
13     col="blue")

```

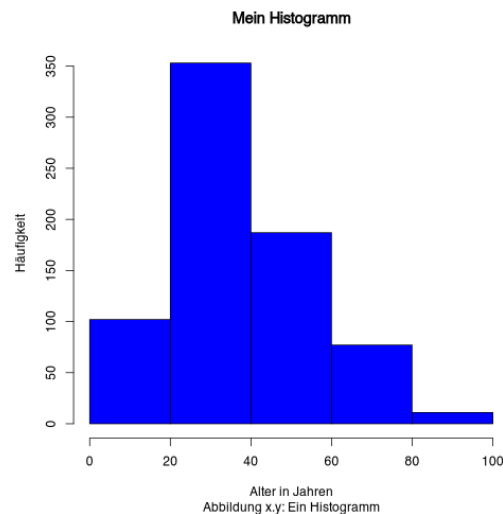


Abb. 2.38

2.3.4 Kreisdiagramm

Ein Kreisdiagramm wird mit der Funktion `pie()` erzeugt. Die Funktion benötigt einen Vektor mit den Anteilsgrößen.

```
1 # Kreisdiagramm 60 / 40  
2 pie(c(60,40))
```

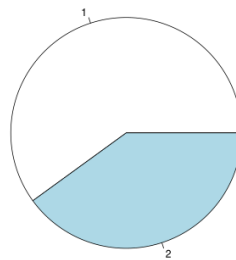


Abb. 2.39

Soll z.B. ein Kreisdiagramm aus einem Faktor erzeugt werden, gelingt dies über die Integration der Funktion `table()`

```
1 # Kreisdiagramm von "Geschlecht" in "pf8"  
2 pie(table(pf8$Geschlecht))
```

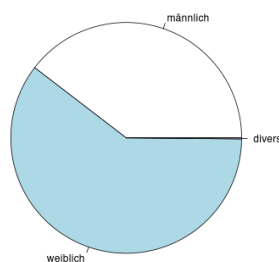


Abb. 2.40

2.3. DIAGRAMME PLOTTEN

Mit den Parametern `col`, `clockwise` und `labels` können Sie weitere Anpassungen vornehmen.

```
1 pie(c(60,40), labels=c("ja", "nein"),  
    col=c("azure", "coral"))
```

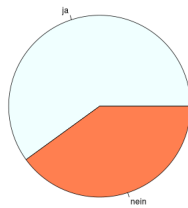


Abb. 2.41

```
1 pie(table(pf8$Geschlecht),  
    clockwise=TRUE, col=c("burlywood",  
    "chartreuse", "black"))
```

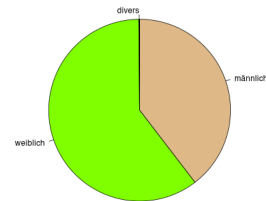
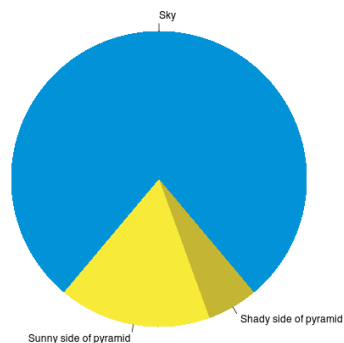


Abb. 2.42

Und auch lustige Dinge kann man plotten²:

```
1 par(mar = c(0, 1, 0, 1))  
2 pie(  
3   c(280, 60, 20),  
4   c('Sky', 'Sunny side of pyramid', 'Shady side of pyramid'),  
5   col = c('#0292D8', '#F7EA39', '#C4B632'),  
6   init.angle = -50, border = NA  
7 )
```



²<https://www.shirin-glander.de/2017/09/moving-my-blog-to-blogdown/>

Abb. 2.43

2.3.5 Säulendiagramm

Säulendiagramme werden mit der Funktion `barplot()` erstellt. Sie benötigt einen Vektor mit den jeweiligen Ausprägungen der Säulen sowie optional die Namen der Säulen über den Parameter `names.arg`:

```
1 # Säulendiagramm
2 barplot(c(10,40,20), names.arg=c("Dritter", "Erster", "Zweiter"))
```

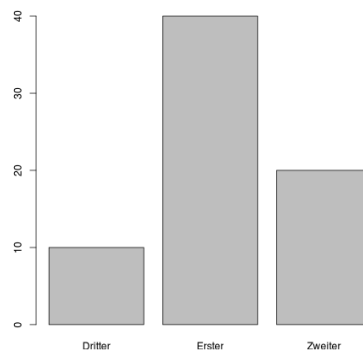


Abb. 2.44

Liegen die Daten als Faktor vor, können sie mit Hilfe der Funktion `table()` übergeben werden

```
1 # Säulendiagramm von "Standort" in "pf8"
2 barplot(table(pf8$Standort))
```

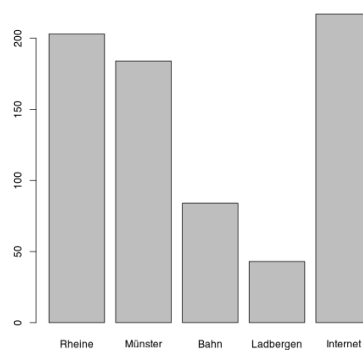


Abb. 2.45

2.3. DIAGRAMME PLOTTEN

Liegen die Daten als `wide table` vor (z.B als Matrix mit bedeutsamen `rownames`), so wie unsere Matrix der Pflegeberufe (siehe Seite 44), erstellt `barplot()` gruppierte Säulen. Über das Argument `legend.text` können die Reihennamen übergeben werden.

```
1 # Säulendiagramm der Matrix Pflegeberufe
2 barplot(Pflegeberufe, beside=T, col=rainbow(5), legend.text = rownames(Pflegeberufe))
```

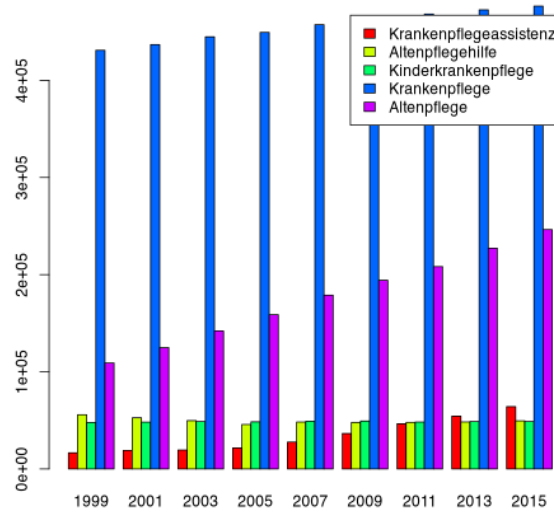


Abb. 2.46

Wird der Parameter `beside` weggelassen (entspricht `FALSE`), werden die Reihen in einer gemeinsamen Säule zusammengefasst.

```
1 # Säulendiagramm der Matrix Pflegeberufe
2 barplot(Pflegeberufe, col=rainbow(5), legend.text = rownames(Pflegeberufe))
```

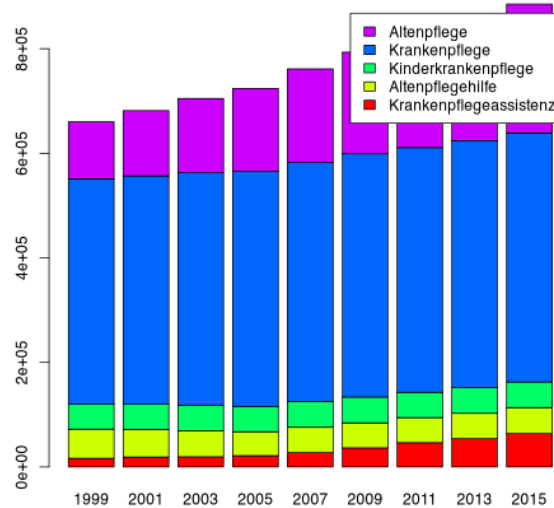


Abb. 2.47

Das entspricht vom Prinzip her der Abbildung auf Seite 44.

2.3.6 Balkendiagramm

Balkendiagramme werden genau so erstellt wie Säulendiagramme, nämlich über die Funktion `barplot()` mit dem Parameter `horiz=TRUE`.

```

1  # Balkendiagramm der Matrix Pflegeberufe
2  # diesmal Schwarz-weiss
3  barplot(Pflegeberufe,
4          # schalte um auf "Balkendiagramm"
5          horiz=TRUE,
6          # Graustufen    führende 0 kann weggelassen werden
7          col=gray(c(.1, .2, .3, .4, .5)),
8          legend.text = rownames(Pflegeberufe))

```

2.3. DIAGRAMME PLOTTEN

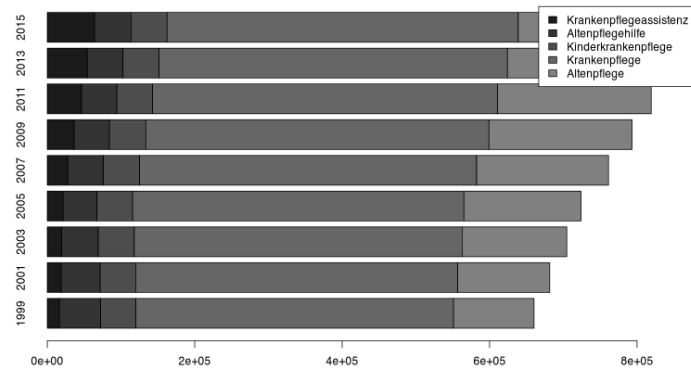


Abb. 2.48

```

1  # Balkendiagramm "category" aus "mma"
2  # diesmal Schwarz-weiss
3  barplot(table(mma$category),
4           horiz=TRUE,
5           col=rainbow(7),
6           # Legende aus Faktorenlevels
7           legend.text = levels(mma$category))
8

```

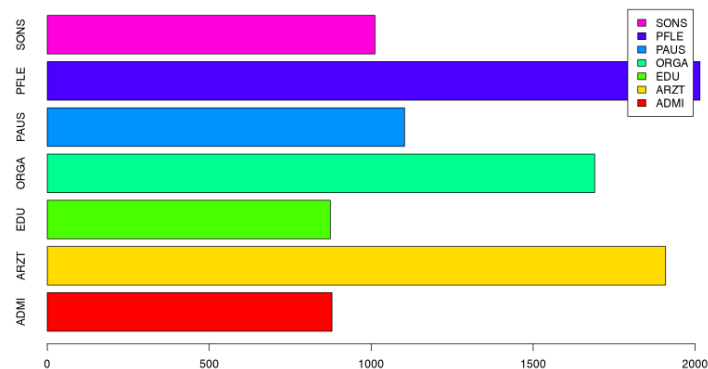


Abb. 2.49

2.3.7 Boxplot

Boxplots können mit der Funktion `boxplot()` erstellt werden.

```
1 # lade Datensatz
2 load(url("https://www.produnis.de/R/nw.RData"))
3
4 # Boxplot des Alters in Datensatz "nw"
5 boxplot(nw$age)
```

Die Funktion erkennt auch die Syntax “*erklärt durch*” (Abbildung 2.51).

```
1 # Boxplot des Alters in Datensatz "nw" erklärt durch "sex"
2 boxplot(age ~ sex, data=nw)
```

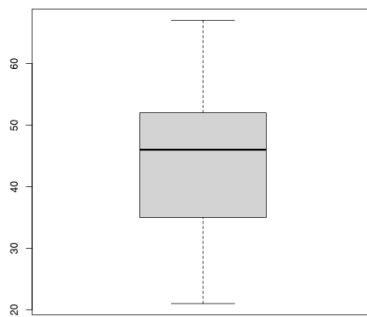


Abb. 2.50

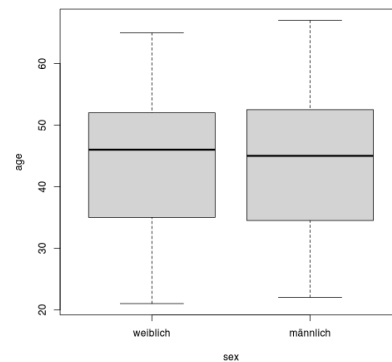


Abb. 2.51

Über die Parameter `frame`, `border`, `col` und `notch` können weitere Anpassungen erfolgen.

```
1 boxplot(age ~ sex, data=nw, frame = FALSE, border="blue", horizontal = TRUE)
2 boxplot(age ~ sex, data=nw, frame = FALSE, col=c("green", "red"), notch = TRUE)
```

2.3. DIAGRAMME PLOTTEN

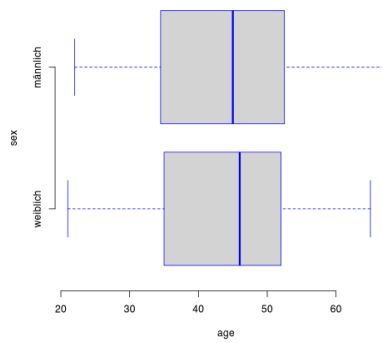


Abb. 2.52

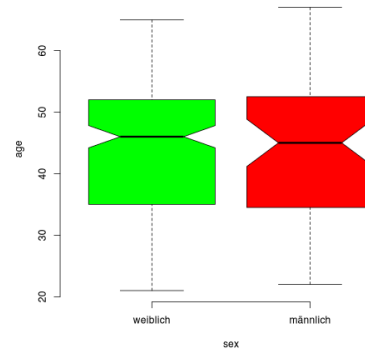


Abb. 2.53

Weitere *innere* Gruppierungen können mit einem Sternchen * übergeben werden.

```
1 boxplot(age~sex*timework, data=nw, col=c("blue", "skyblue"), notch = TRUE)
```

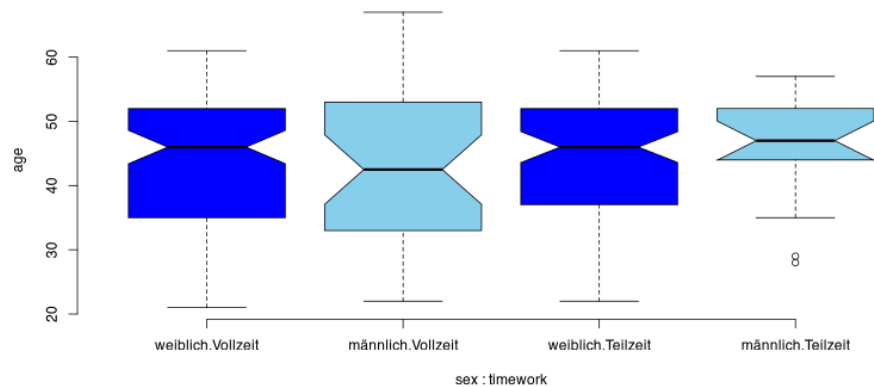


Abb. 2.54

2.3.8 Diagramme speichern

Zum Speichern der Diagramme bietet **R** die Formate `jpeg`, `png`, `bmp` und `tiff` an. Und genau so heissen auch die jeweiligen Speicherfunktionen. Dieser Befehl speichert das Diagramm im Format `png`:

```
1 # Speichere nachfolgendes als PNG
2 png("Testplot.png", width = 500, height = 500, units = "px", pointsize = 12)
3
4 # erzeuge Diagramm
5 pie(c(60,40))
6
7 # speichern abschließen
8 dev.off()
```

Wie Sie sehen wird zunächst die Funktion `png()` aufgerufen. In ihr wird der Dateiname des Diagramms sowie dessen Dimensionen angegeben. Anschließend folgt der Aufruf der Plotbefehle, in diesem Falle `pie()`. Mit dem Befehl `dev.off()` (für *device off*) wird das Speichern abgeschlossen. Die eigentlichen Diagrammbefehle müssen also *zwischen* der Speicherfunktion (`png()`) und `dev.off()` stehen.

Soll das Diagramm als `jpeg` gespeichert werden, lautet die Befehlskette entsprechend:

```
1 # Speichere nachfolgendes als JPEG
2 jpeg("Testplot2.jpg", width = 500, height = 500, units = "px", pointsize = 12)
3
4 # erzeuge Diagramm
5 barplot(c(10,40,20), names.arg=c("Dritter", "Erster", "Zweiter"))
6
7 # speichern abschließen
8 dev.off()
```

Genau so funktionieren auch die Funktionen `bmp()` und `tiff()`, aber diese Formate verwendet heute eigentlich niemand mehr.

2.4 ggplot

Das Zusatzpaket **ggplot** (für *grammar of graphics plot*) ist Hadley Wickhams[5] -Implementation der *Grammar of Graphics* von Leland Wilkinson[8]. Es ist eines der ersten Pakete des Tidyverse (welches damals noch nicht so hieß).

Die Funktion zum plotten heisst “`ggplot()`”, das Installationspaket in  lautet jedoch `ggplot2`.

Unter <https://ggplot2-book.org> ist eine detaillierte Anleitung in englischer Sprache von Hadely Wickham et al.[10] verfügbar.

```
1 # ggplot2 installieren
2 install.packages("ggplot2", dependencies=T)
3
4 # ggplot2 aktivieren
5 library(ggplot2)
```

Wenn Sie zuvor das Tidyverse (siehe Kapitel 1.6 auf Seite 147) aktiviert haben...

```
1 library(tidyverse)
```

... wird das `ggplot2`-Paket ebenfalls aktiviert.

Da `ggplot` aus dem *Tidyverse* stammt, ist es von Vorteil, wenn die zu verarbeitenden Datensätze dem Prinzip “ein Fall pro Zeile” (*tidy data* bzw. *long table*) folgen. Das bedeutet, dass jede Beobachtung (auch Wiederholungen) in einer eigenen Zeile steht, und die jeweiligen Variablen durch die Spalten repräsentiert werden.

2.4.1 Satzbau

Die Idee einer Grammatik für Diagramme ist, dass jedes Diagramm aus den selben Komponenten (*Satzteilen*) aufgebaut werden kann. Benötigt werden:

1. Daten (möglichst im *Tidy Data* Format)

2. **Aesthetische** bzw. visuelle Zuordnungen der Daten zu den Komponenten des Plots
3. **Geometrische** Objekte, die im Plot erscheinen sollen
4. **Statistische** Transformation der Daten, bevor sie geplottet werden
5. **Coordinates** (Koordinaten), um die Lage der geometrischen Objekte zu bestimmen
6. **Scales** (Skalen), um den Wertebereich der Datenzuordnung zu bestimmen
7. **Facetten**, um den Plot in Teil-Diagramme zu gruppieren

Diese Komponenten werden in `ggplot()` mit dem `+` Zeichen aneinander gereiht.

```
1  # sinnbildlich
2  ggplot(daten) +
3    visuelle Zuordnung +
4      Geometrische Objekte +
5        statistische Transformationen +
6          Koordinaten +
7            Skalen +
8              Facetten
```

Die enthaltenen Funktionen folgen dabei dieser Komponentenstruktur:

- Geome (Flächen, Linien, Punkte, usw.) beginnen mit `geom_`, z.B.:
 - `gemo_area()`, `gemo_density()`, `gemo_smooth()`, `gemo_histogram()`,
`gemo_point()`, `gemo_jitter()`, `gemo_line()`, `gemo_boxplot()`, `gemo_col()`,
`gemo_text()`
- statistische Funktionen (Häufigkeiten, Mittelwerte, Dichte, usw.) beginnen mit `stat_`, z.B.:
 - `stat_count()`, `stat_density()`, `stat_function()`, `stat_summary()`
- Koordinatensystemfunktionen beginnen mit `coord_`, z.B.:

- `coord_flip()`, `coord_trans()`
- Funktionen zum Manipulieren des grundlegenden Aussehens (Theme), beginnend mit `theme_`, z.B.:
 - `theme()`, `theme_bw()`, `theme_classic()`

In RStudio ist ein “Cheatsheet” (siehe Abschnitt 1.7.1 auf Seite 202) zu `ggplot` verlinkt. Hier finden Sie (fast) alle verfügbaren Komponenten zusammengefasst. Klicken Sie in der Menüleiste von RStudio auf `Help` → `Cheatsheets` → `Data Visualization with ggplot2`.

2.4.2 Erstes Plot

Mit dem Datensatz `jgsbook::pf8` erzeugen wir nun unser erstes `ggplot`.

```
1 # Lade Datensatz
2 load(url("https://www.produnis.de/R/pf8.RData"))
3 # oder
4 pf8 <- jgsbook::pf8
5
6 # Tidyverse aktivieren!
7 library(tidyverse)
8
9 # unser erstes Plot
10 ggplot(pf8)
```

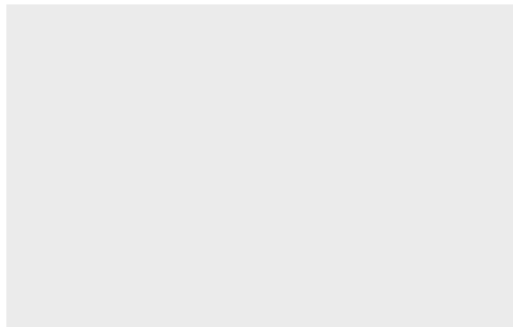


Abb. 2.55

“Wie Sie sehen, sehen Sie nichts.”

Wir sehen deshalb nichts, weil wir noch nicht bestimmt haben, *was* (welche Variablen) *wie* (Punkte, Linien, Boxplots) *wo* (x-Achse, y-Achse) geplottet werden soll. Wir müssen diese “Aesthetics” erst definieren.

2.4.2.1 Aesthetics

Angenommen, uns interessiert das **Gewicht** sowie das **Alter** der Probanden. Mit der Funktion `aes()` können wir die Daten visuell zuordnen. Das **Alter** soll auf der x-Achse liegen, das **Gewicht** auf der y-Achse.

```
1 # erster Plot mit "Aesthetics"  
2 ggplot(pf8) +  
3   aes(x = Alter,    # x-Achse  
4       y = Gewicht) # y-Achse
```

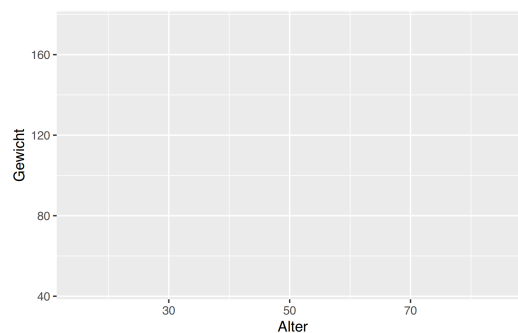


Abb. 2.56

Ggplot hat die Achsen entsprechend beschriftet und die Wertebereiche anhand der vorhandenen Daten (Minimum, Maximum) ausgewählt.

2.4.2.2 Geome

Zur Darstellung von **Gewicht** und **Alter** werden geometrische Objekte (*Geome*) benötigt. Diese werden über die Funktionen `geom_*()` erzeugt, wobei das Sternchen durch die gewünschte geometrischen Form ersetzt werden muss. Die 6 meist-genutzten Geome sind sicherlich:

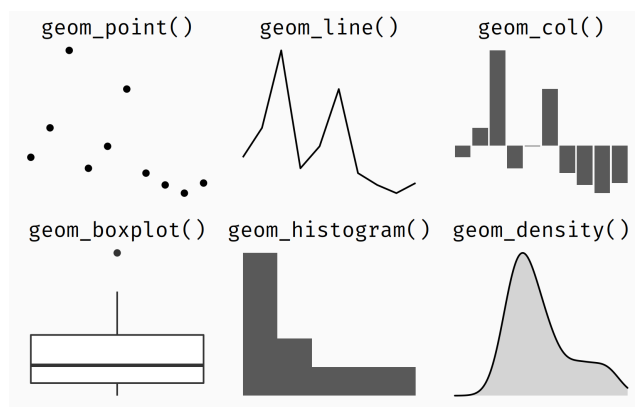


Abb. 2.57

Insgesamt sind 53 Geome verfügbar:

[1]	"geom_abline"	"geom_area"	"geom_bar"
[4]	"geom_bin_2d"	"geom_bin2d"	"geom_blank"
[7]	"geom_boxplot"	"geom_col"	"geom_contour"
[10]	"geom_contour_filled"	"geom_count"	"geom_crossbar"
[13]	"geom_curve"	"geom_density"	"geom_density_2d"
[16]	"geom_density_2d_filled"	"geom_density2d"	"geom_density2d_filled"
[19]	"geom_dotplot"	"geom_errorbar"	"geom_errorbarh"
[22]	"geom_freqpoly"	"geom_function"	"geom_hex"
[25]	"geom_histogram"	"geom_hline"	"geom_jitter"
[28]	"geom_label"	"geom_line"	"geom_linerange"
[31]	"geom_map"	"geom_path"	"geom_point"
[34]	"geom_pointrange"	"geom_polygon"	"geom_qq"
[37]	"geom_qq_line"	"geom_quantile"	"geom_raster"
[40]	"geom_rect"	"geom_ribbon"	"geom_rug"
[43]	"geom_segment"	"geom_sf"	"geom_sf_label"
[46]	"geom_sf_text"	"geom_smooth"	"geom_spoke"
[49]	"geom_step"	"geom_text"	"geom_tile"
[52]	"geom_violin"	"geom_vline"	

In unserem Beispiel möchten wir das Gewicht sowie das Alter der Probanden als Punktwolke darstellen. Das geeignete Geom ist `geom_point()`.

```
1 # unser erstes Plot, mit "Aesthetics" und "Geom"
2 ggplot(pf8) +
3   aes(x = Alter,
4       y = Gewicht) +
5   geom_point()
```

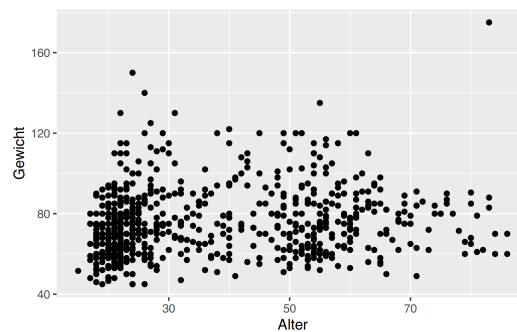


Abb. 2.58

2.4.2.3 mehr Aesthetics

Angenommen, wir möchten das **Gewicht** sowie das **Alter** der Probanden als Punktwolke darstellen, aber graphisch zwischen den Geschlechtern unterscheiden. Hierzu erweitern wir dies `aes()`-Funktion um den Parameter `color`.

```

1 # unser erstes Plot, mit "Aesthetics" und "Geom",
2 # farblich nach Geschlecht
3 ggplot(pf8) +
4   aes(x = Alter,
5       y = Gewicht,
6       color = Geschlecht) +
7   geom_point()

```

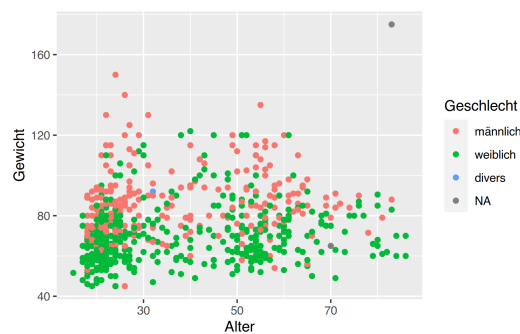


Abb. 2.59

Ggplot stellt die Werte in unterschiedlichen Farben pro Geschlechterkategorie dar. Ebenso wurde das Plot um eine Legendenbox erweitert, in welcher die Farbzugeordnungen pro Geschlechterkategorie angezeigt ist.

2.4.2.4 mehr Geome

Über das `+` Zeichen können beliebig viele Geome dem Plot hinzugefügt werden. Dabei werden die *aesthetics* automatisch an die Geome weitergegeben, sie können aber auch in jedem Geom per `aes()` neu definiert werden.

In unserem Beispiel möchten wir nun eine Regressionsgerade mit Konfidenzintervall einzeichnen. Dies kann mit `geom_smooth()` umgesetzt werden. Über den Parameter `method` können wir angeben, dass ein lineares Modell (`"lm"`) eingezeichnet werden soll.

```

1 # unser erstes Plot, mit "Aesthetics" und zwei "Geomen"
2 # und Regressionsmodell
3
4 ggplot(pf8) +
5   aes(x = Alter,
6       y = Gewicht,
7       color = Geschlecht) +
8   geom_point() +
9   geom_smooth(method="lm")

```

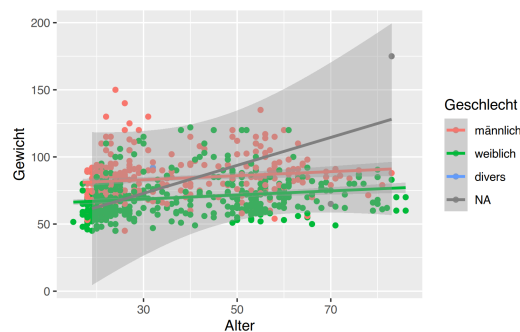


Abb. 2.60

In der Variablen `Geschlecht` sind `NA` enthalten. Diese können wir mit der Funktion `drop_na()` aussortieren und direkt an `ggplot()` weiter-pipen.

```

1 # unser erstes Plot, mit "Aesthetics" und zwei "Geomen"
2 # und Regressionsmodell und
3 # OHNE NA
4 pf8 %>%
5   drop_na(Geschlecht) %>%
6   ggplot() +
7     aes(x = Alter,
8         y = Gewicht,
9         color = Geschlecht) +
10    geom_point() +
11    geom_smooth(method="lm")

```

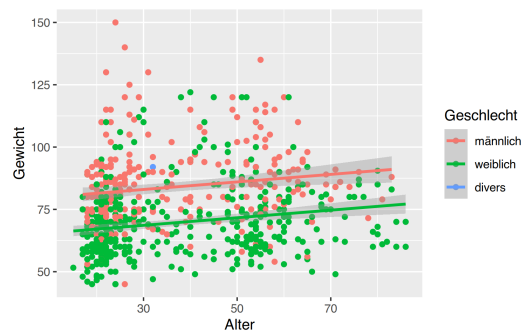


Abb. 2.61

Es wird ausdrücklich empfohlen, sich über die Hilfeseiten, z.B. `?geom_smooth`, mit den Funktionen vertraut zu machen. So erfährt man dort beispielsweise, dass der Konfidenzbereich per `se=FALSE` ausgeblendet werden kann.

```

1 # unser erstes Plot, mit "Aesthetics" und zwei "Geomen"
2 # und Regressionsmodell ohne Konfidenzbereich
3 # OHNE NA
4 pf8 %>%
5   drop_na(Geschlecht) %>%
6   ggplot() +
7     aes(x = Alter,
8         y = Gewicht,
9         color = Geschlecht) +
10    geom_point() +
11    geom_smooth(method="lm", se=FALSE)

```

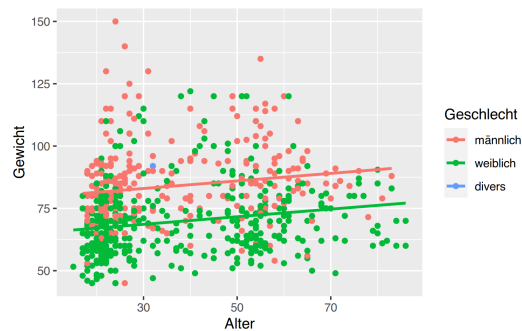


Abb. 2.62

2.4.2.5 Facetten

Mit den Funktionen `facet_wrap()` und `facet_grid()` können Teil-Diagramme innerhalb des Gesamtplots erzeugt werden.

So können wir beispielsweise ein Plot pro `Standort` erstellen.

```

1  # unser erstes Plot, mit "Aesthetics",
2  # "Geom" und Facetten
3
4  pf8 %>%
5    drop_na(Geschlecht) %>%
6    ggplot() +
7      aes(x = Alter,
8          y = Gewicht,
9          color = Geschlecht) +
10     geom_point() +
11     geom_smooth(method="lm") +
12     facet_wrap(~ Standort)

```

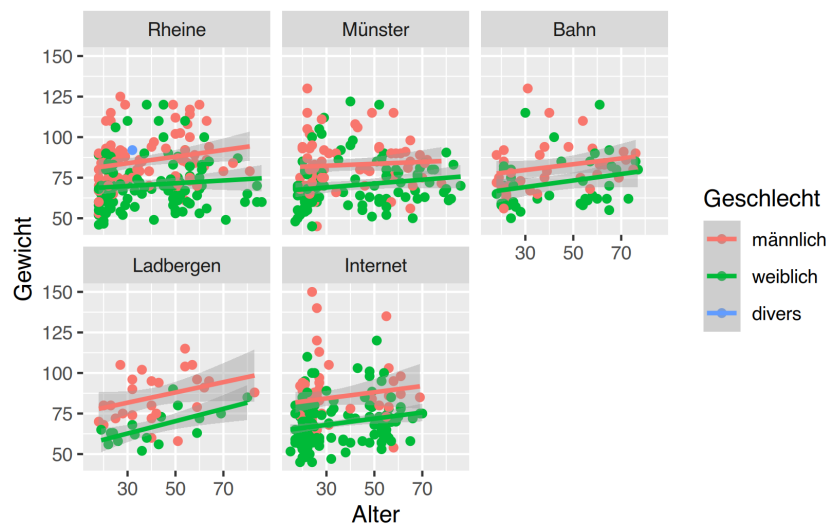


Abb. 2.63

Oder wir erstellen ein Plot pro Rauchen-Kategorie.

```

1  # unser erstes Plot, mit "Aesthetics",
2  # "Geom" und Facetten
3
4  pf8 %>%
5    drop_na(Geschlecht) %>%
6    ggplot() +

```

```

7   aes(x = Alter,
8       y = Gewicht,
9       color = Geschlecht) +
10  geom_point() +
11  geom_smooth(method="lm") +
12  facet_wrap(~ Rauchen)

```

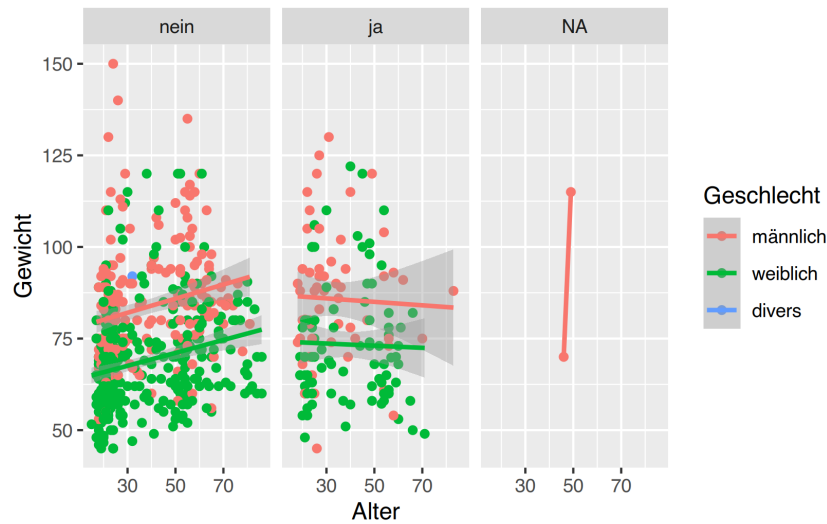


Abb. 2.64

Mit der Funktion `facet_grid()` können Plot-Gitter erzeugt werden. So können wir beispielsweise ein Plot pro Standort und Rauchen erstellen.

```

1   # unser erstes Plot, mit "Aesthetics",
2   # "Geom" und Facetten
3
4   pf8 %>%
5     drop_na(Geschlecht) %>%
6     ggplot() +
7       aes(x = Alter,
8          y = Gewicht,
9          color = Geschlecht) +
10      geom_point() +
11      geom_smooth(method="lm") +
12      facet_grid(Rauchen ~ Standort)

```

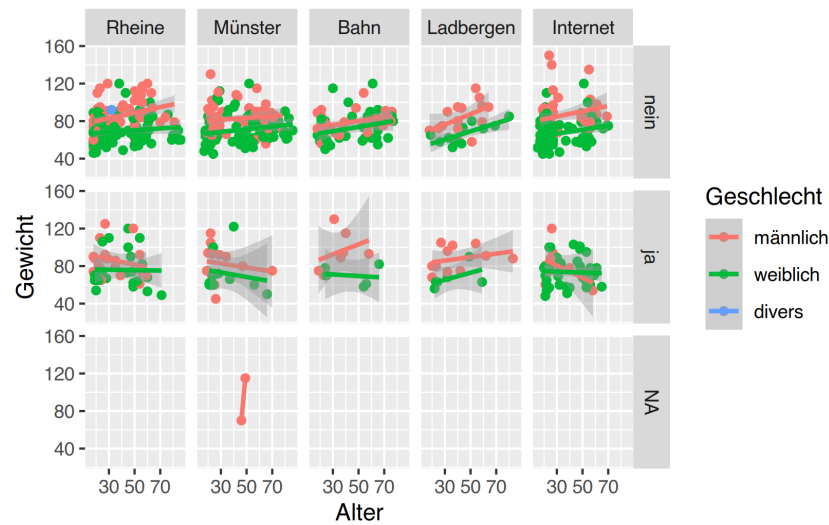



Abb. 2.65

Mittels `drop_na()` können wir uns der NA-Zeile entledigen.

```

1  # unser erstes Plot, mit "Aesthetics",
2  # "Geom" und Facetten
3
4  pf8 %>%
5    drop_na(Geschlecht, Rauchen) %>%
6    ggplot() +
7      aes(x = Alter,
8          y = Gewicht,
9          color = Geschlecht) +
10     geom_point() +
11     geom_smooth(method="lm") +
12     facet_grid(Rauchen ~ Standort)

```

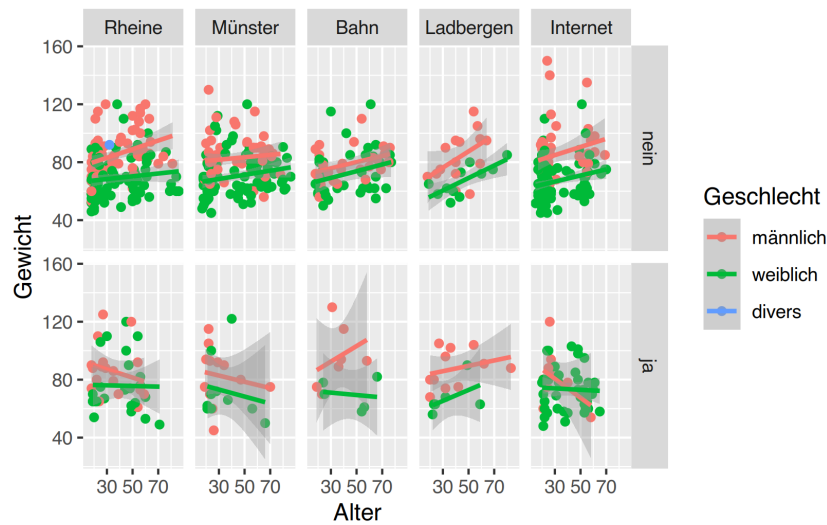


Abb. 2.66

2.4.2.6 Geom-Parameter

Geome können mit verschiedenen Parametern beeinflusst werden. Welche Parameter zur Verfügung stehen, verrät die Hilfeseite der Geome. Die meisten reagieren auf:

```
geom_*(data, aes, stat, position, PARAMETER)
```

- **data:** Geome übernehmen das Datenobjekt aus der `ggplot()`-Funktion. Sie können aber auch eigene Daten verarbeiten
- **aes:** Geome erben die Zuordnungen aus der `aes()`-Funktion. Sie können aber auch eigene aesthetics haben.
- **stat:** Geome können statistische Transformationen vornehmen, z.B. `stat='identity'`.
- **position:** die Positionierung der Objekte lässt sich anpassen, z.B. `'dodge'`, `'stack'`, `'jitter'`, `'fill'`.
- weitere PARAMETER sind:
 - **color:** (Rahmen)farbe
 - **fill:** Füllfarbe

- `size`: Objektgröße
- `shape`: Objektformen

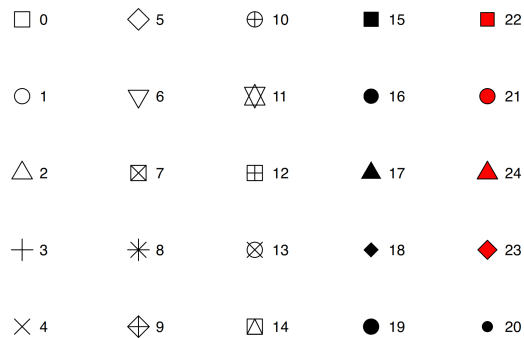


Abb. 2.67: *shape* nach Zahlen

Die Parameter können auch innerhalb der `aes()`-Funktion verwendet werden, so dass anhand der Daten entschieden wird, welche Größe, Farbe und Form die Geome haben.

- `linetype`: Linientypen

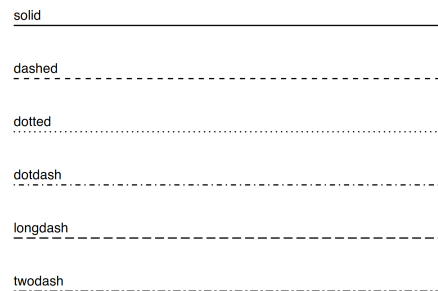


Abb. 2.68: *Linientypen*

2.4.3 Punktwolke

Punktwolken (Scatterplots) werden mit der Funktion `geom_point()` erzeugt. Wir plotten die Variablen `Größe` und `Gewicht` aus dem `pf8`-Datensatz.

```
1 ggplot(pf8) +
2   aes(x=Größe, y=Gewicht) +
3   geom_point()
```

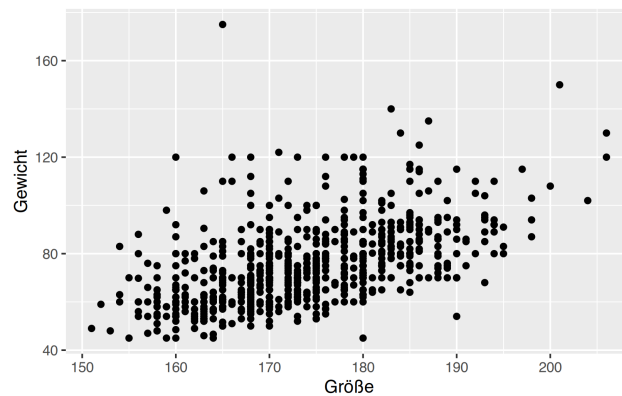


Abb. 2.69

Über die Parameter `color`, `shape` und `size` können wir die Punkte manipulieren.

```
1 ggplot(pf8) +
2   aes(x=Größe, y=Gewicht) +
3   geom_point(color="blue",
4             shape=13,
5             size=10)
```

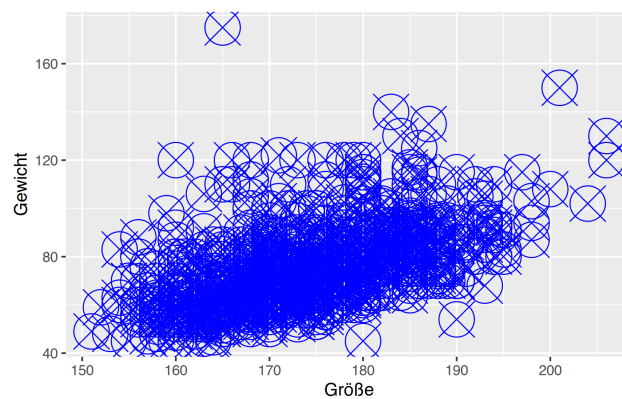


Abb. 2.70

Die Parameter können auch als *aesthetics* verwendet werden, so dass anhand der Daten entschieden wird, welche Größe, Farbe und Form die Punkte haben.

```
1 ggplot(pf8) +  
2   aes(x=Größe, y=Gewicht,  
3       color=Geschlecht,  
4       shape=Geschlecht,  
5       size=Alter) +  
6   geom_point()
```



Abb. 2.71

2.4.4 Histogramm

Histogramme werden mit der Funktion `geom_histogram()` erzeugt.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram()
```

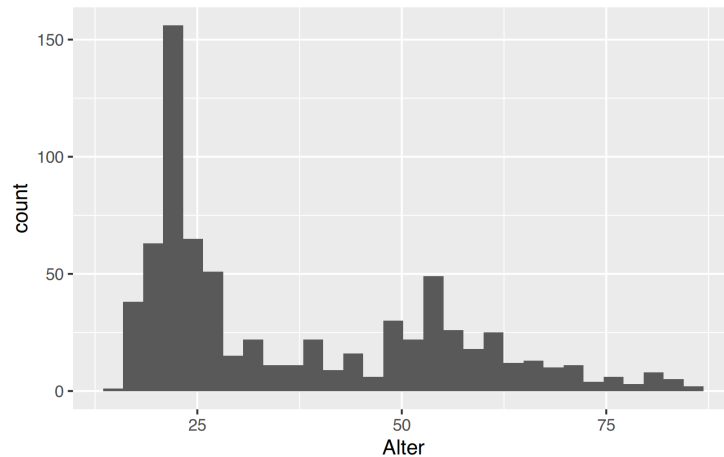


Abb. 2.72

Mit den Parametern `color` und `fill` können wir das Histogramm etwas aufhübschen.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue", # Rahmenfarbe
4                 fill="skyblue" # Füllfarbe)
```

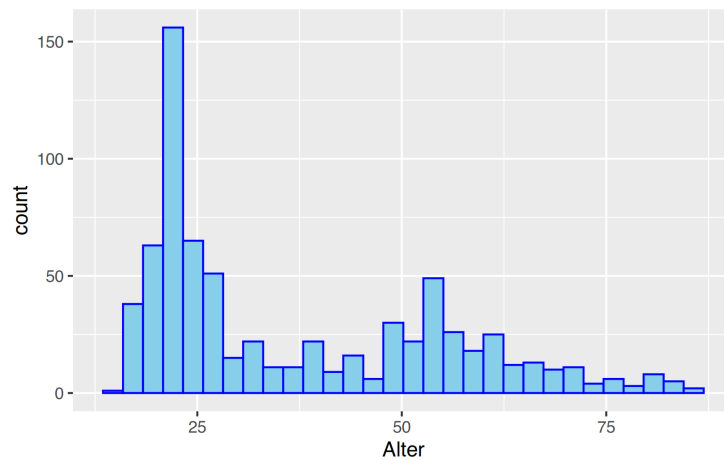


Abb. 2.73

Über den Parameter `binwidth` kann die Säulenbreite bestimmt werden.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue", # Rahmenfarbe
4                 fill="skyblue", # Füllfarbe
5                 binwidth=10) # Klassenbreite 10
```

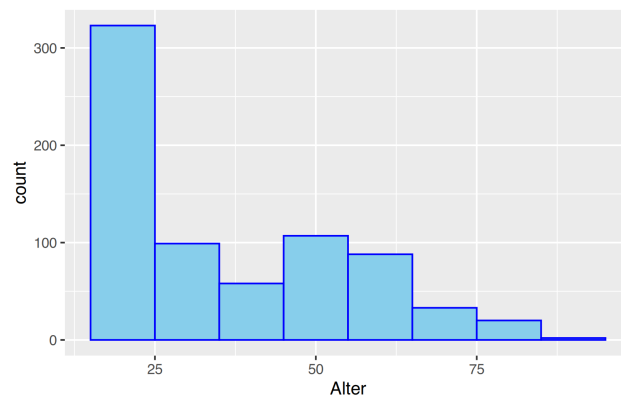


Abb. 2.74

Mit dem Parameter `bins` kann die Anzahl der Säulen bestimmt werden.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue", # Rahmenfarbe
4                 fill="skyblue", # Füllfarbe
5                 bins=5) # 5 Säulen
```

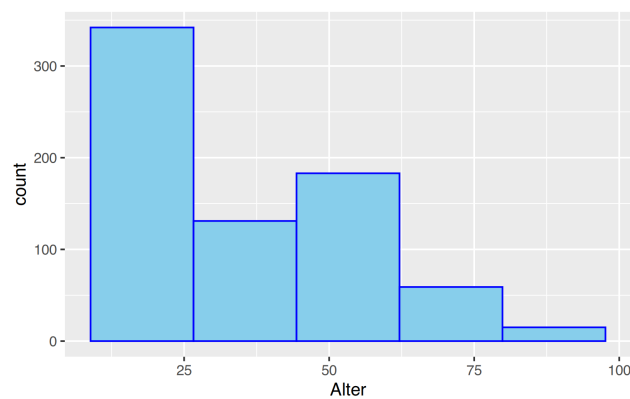


Abb. 2.75

Mit der Funktion `stat_bin()` kann das “Originalhistogramm” hinzugefügt werden. So ist leicht zu erkennen, wie grob die Klassen zusammengefasst wurden.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue", # Rahmenfarbe
4                 fill="skyblue", # Füllfarbe
5                 bins=5) +      # 5 Säulen
6   stat_bin(fill="darkblue")
```

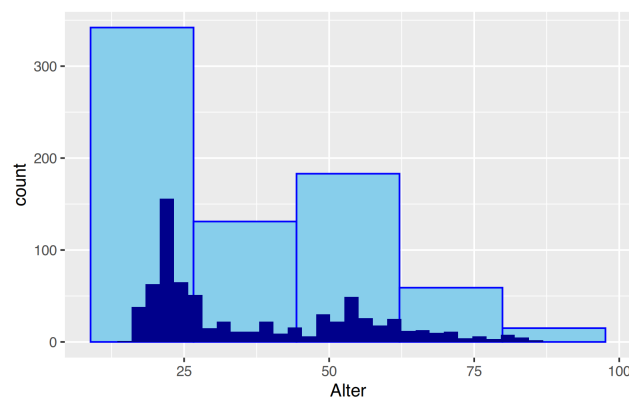


Abb. 2.76

2.4.4.1 Histogramm und Dichteverteilung

Über die Aesthetic `..density..` können wir auf die Skala der Dichtefunktion wechseln.

```
1 ggplot(pf8) +
2   aes(x=Alter,
3       y=..density..) +
4   geom_histogram(color="blue",
5                 fill="skyblue")
```

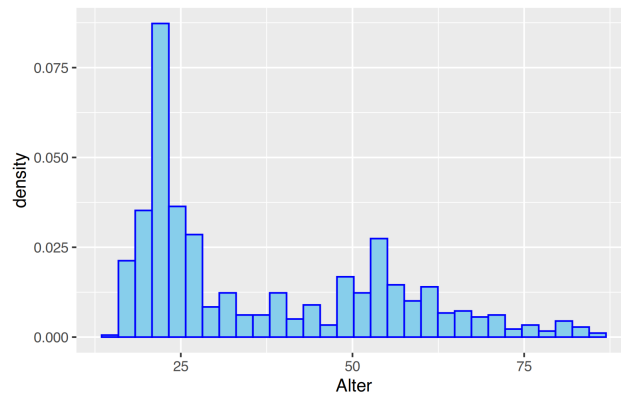



Abb. 2.77

Beachten Sie die Achseneinträge der y-Achse.
Das funktioniert auch, wenn die aesthetics im `geom_histogram()` angegeben werden.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(aes(y=..density..),
4                   color="blue",
5                   fill="skyblue")
```

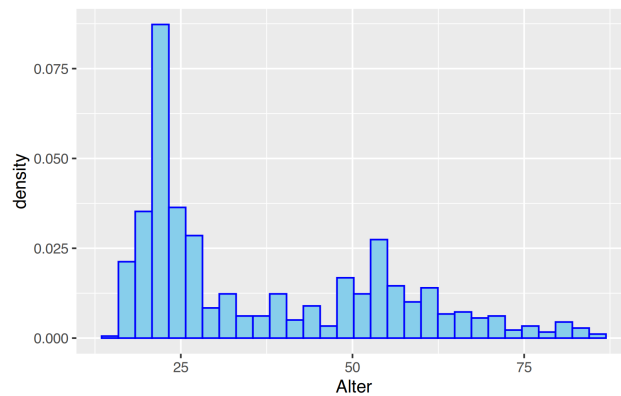


Abb. 2.78

Die Dichtefunktion des Histogramms lässt sich nun mit der Funktion `stat_density()` einzeichnen.

```

1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(aes(y=..density..),
4                 color="blue",
5                 fill="skyblue") +
6   stat_density(geom="line",
7               color="red",
8               linetype="dotted")

```

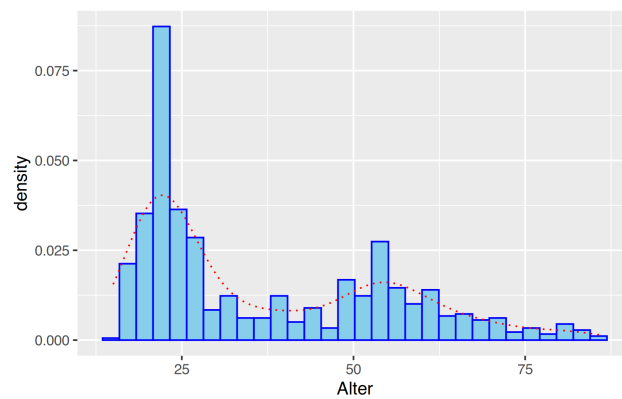


Abb. 2.79

Wir wechseln von `geom="line"` auf `geom="area"`:

```

1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(aes(y=..density..),
4                 color="blue",
5                 fill="skyblue") +
6   stat_density(geom="area",
7               color="red",
8               fill="pink",
9               linetype="dotted")

```

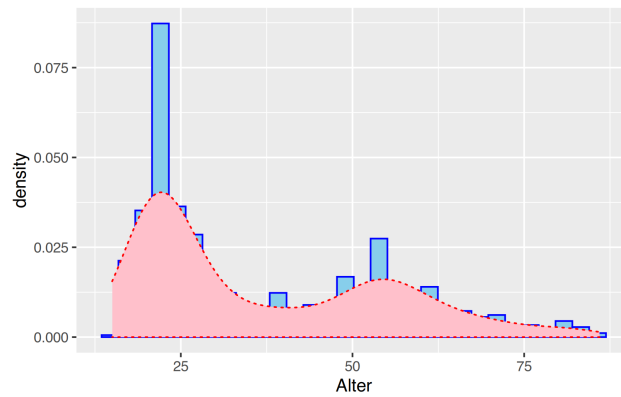


Abb. 2.80

Über den Parameter `alpha` lässt sich die Durchsichtigkeit (so genannter Alpha-kanal) des Objekts eingeben.

```

1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(aes(y=..density..),
4                 color="blue",
5                 fill="skyblue") +
6   stat_density(geom="area",
7               color="red",
8               fill="pink",
9               linetype="dotted",
10              alpha=0.5)

```

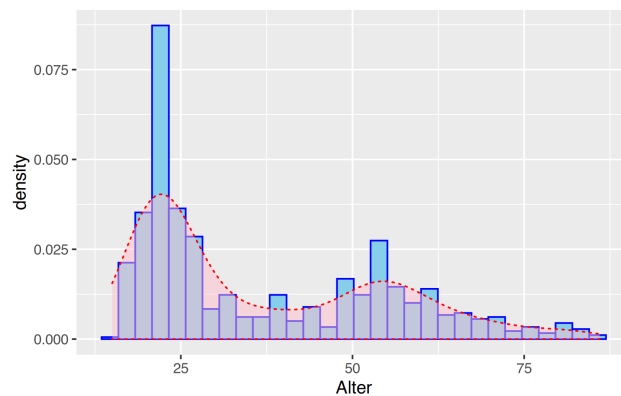


Abb. 2.81

Mittels `stat_function()` und `dnorm()` lässt sich die Normalverteilung ergänzen. Hierfür bestimmen wir zunächst Mittelwert und Standardabweichung von `pf8$Alter`.

```
1 round(mean(pf8$Alter, na.rm=TRUE))
2 round(sd(pf8$Alter, na.rm=TRUE))
```

Die Werte werden nun wie folgt übertragen:

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(aes(y=..density..),
4                 color="blue",
5                 fill="skyblue") +
6   stat_density(geom="area",
7               color="red",
8               fill="pink",
9               linetype="dotted",
10              alpha=0.5)+
11   stat_function(fun=dnorm,
12               args=(c(mean=37,sd=18)),
13               color = "darkgreen",
14               linetype = "dashed")
```

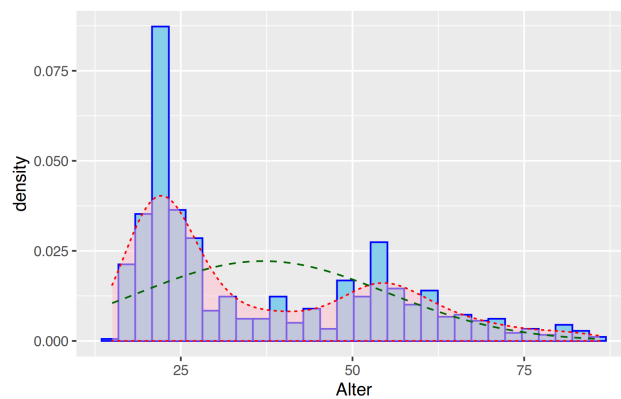


Abb. 2.82

Es ist deutlich erkennbar, dass **keine** Normalverteilung vorliegt.

2.4.5 Säulendiagramme

Säulendiagramme werden mit den Funktionen `geom_bar()` und `geom_col()` erzeugt.

- Für `geom_bar()` müssen die Variablen als Factor vorliegen. Die Größe der Säulen wird automatisch mittels `stat_count()` berechnet.
- `geom_col()` erstellt die Größe der Säulen anhand der tatsächlichen Werte im Datenframe (vgl. `stat_identity()`; absolute Häufigkeit)

Die Variable `pf8$SportWarum` ist ein Faktor und soll mittels `geom_bar()` geplottet werden.

```
1 ggplot(pf8) +
2   aes(x=SportWarum ) +
3   geom_bar(color="blue",
4           fill="skyblue")
```

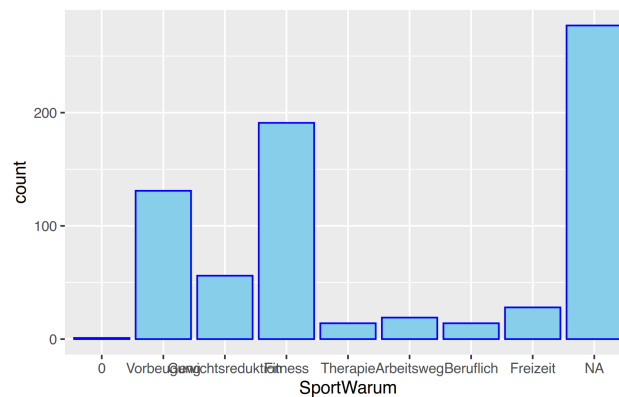


Abb. 2.83

Mittels `drop_na()` kann der große NA-Balken entfernt werden.

```
1 pf8 %>%
2   drop_na(SportWarum) %>%
3   ggplot() +
4     aes(x=SportWarum ) +
5     geom_bar(color="blue",
6             fill="skyblue")
```

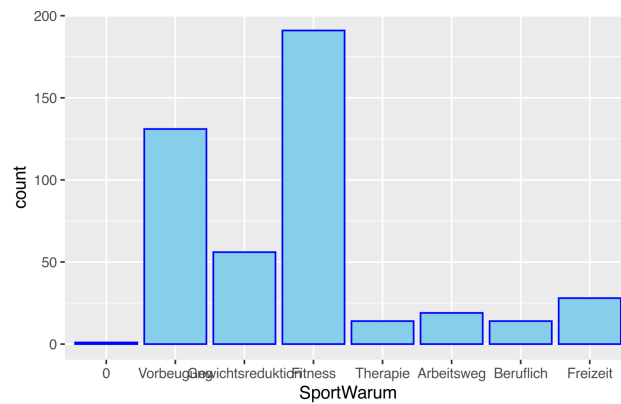


Abb. 2.84

Mit Hilfe von der Funktionen `fct_infreq()` und `fct_rev()` können wir die Levels auf- und absteigend sortieren. Hierzu erzeugen wir eine neue Spalte `S`, welche wir in der `aes()`-Funktion auf die x-Achse legen.

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   ggplot() +
5     aes(x=S ) +
6     geom_bar(color="blue",
7             fill="skyblue")

```

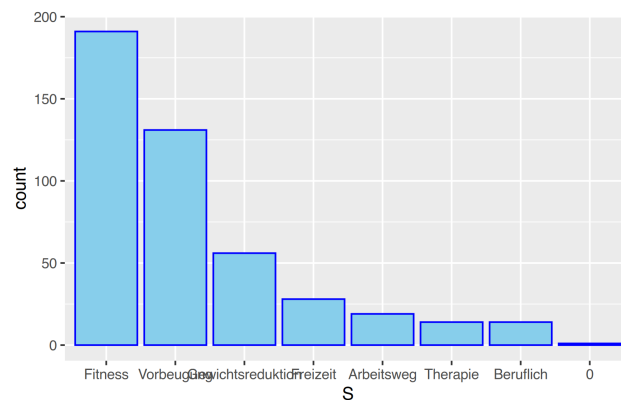


Abb. 2.85

Um aufsteigend zu sortieren erweitert sich der `mutate()`-Aufruf wie folgt:

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum),
4          S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S) +
7     geom_bar(color="blue",
8             fill="skyblue")

```

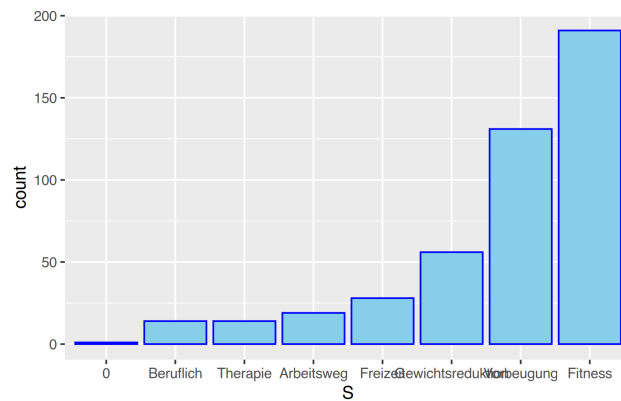


Abb. 2.86

Über die Aesthetic `fill` können wir gruppieren, z.B. nach `Geschlecht`.

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   mutate(S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S,
7        fill=Geschlecht) +
8     geom_bar(color="blue")

```

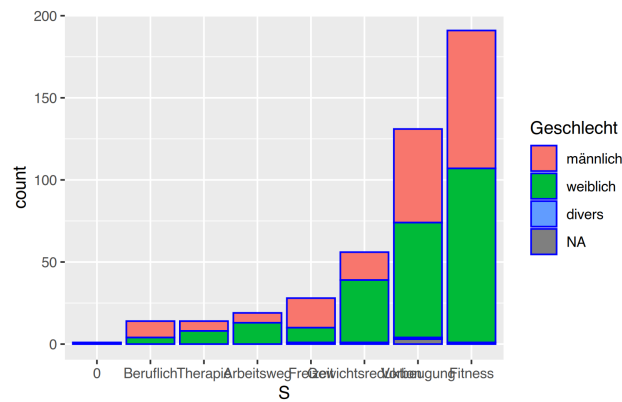


Abb. 2.87

Mit dem Parameter `position` können wir die Säulenausgabe weiter beeinflussen. Die Angabe `position="fill"` erzeugt Prozentanteile.

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   mutate(S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S,
7         fill=Geschlecht) +
8     geom_bar(color="blue",
9              position="fill")

```

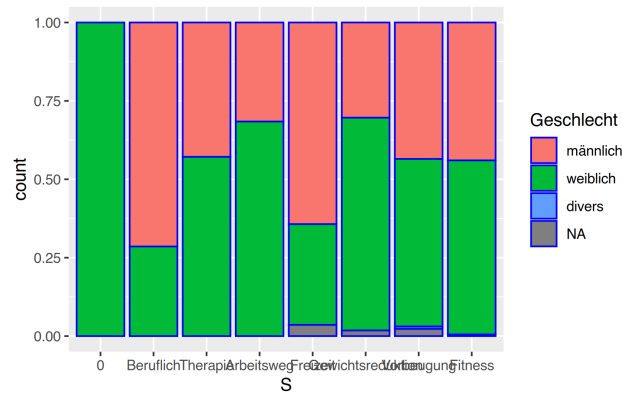


Abb. 2.88

Mit `position="dodge"` werden die Säulen der Gruppen nebeneinander gestellt.


```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   mutate(S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S,
7         fill=Geschlecht) +
8     geom_bar(color="blue",
9              position="dodge")

```

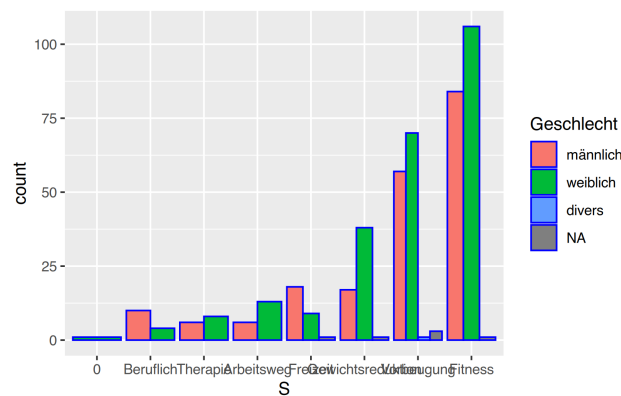


Abb. 2.89

Mit `position="jitter"` wird etwas Rauschen bei der Positionierung verwendet. Das sieht nicht immer schön aus.

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   mutate(S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S,
7         fill=Geschlecht) +
8     geom_bar(color="blue",
9              position="jitter")

```

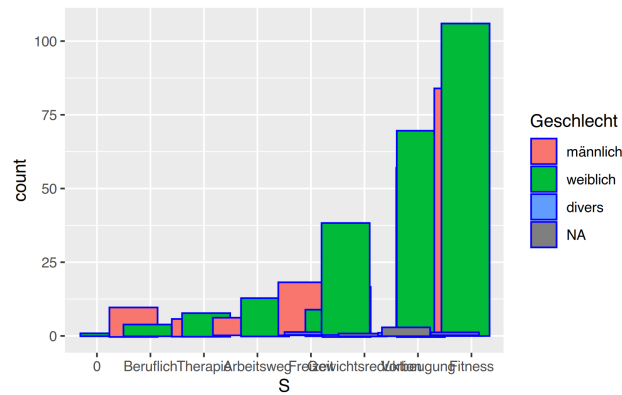


Abb. 2.90

Die Default-Einstellung ist `position="stack"`.

```

1 pf8 %>%
2   drop_na(SportWarum) %>%
3   mutate(S = fct_infreq(SportWarum)) %>%
4   mutate(S = fct_rev(S)) %>%
5   ggplot() +
6     aes(x=S,
7         fill=Geschlecht) +
8     geom_bar(color="blue",
9              position="stack")

```

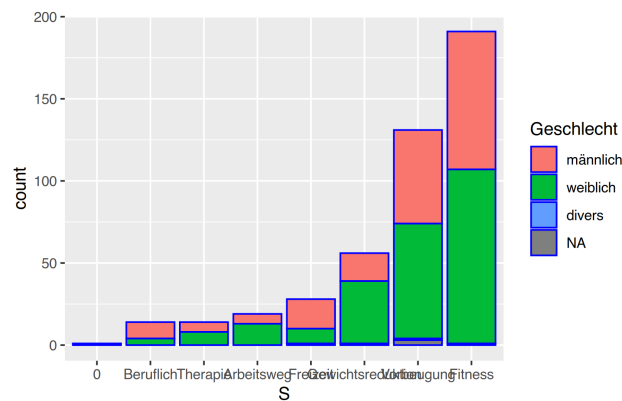


Abb. 2.91

Mit dem Parameter `stat="identity"` können vorberechnete Werte (absolute Häufigkeiten) geplottet werden.

```
1 # vorgegebene Werte
2 df <- data.frame(Gruppe=c("Vanille", "Schokolade", "Erdbeer"),
3                     Anteil=c(60, 25, 15))
4
5 # plotten
6 ggplot(df, aes(x=Gruppe, y=Anteil,
7                 fill=Gruppe)) +
8   geom_bar(stat="identity",
9            position="dodge")
```

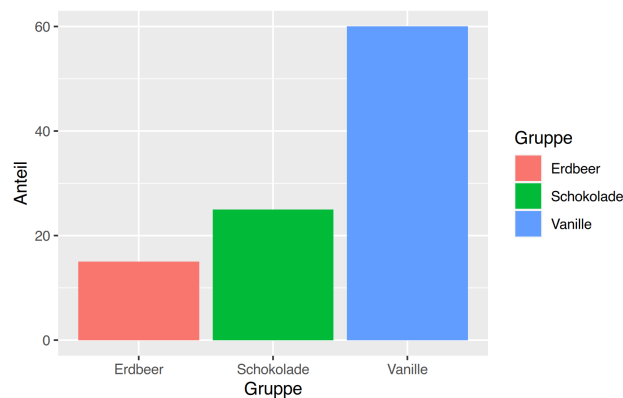


Abb. 2.92

Alternativ kann das Plot mit der Funktion `geom_col()` erzeugt werden, die von Hause aus `stat_identity()` nutzt.

```
1 # vorgegebene Werte
2 df <- data.frame(Gruppe=c("Vanille", "Schokolade", "Erdbeer"),
3                     Anteil=c(60, 25, 15))
4
5 # plotten mit geom_col()
6 ggplot(df, aes(x=Gruppe, y=Anteil,
7                 fill=Gruppe)) +
8   geom_col(position="dodge")
```

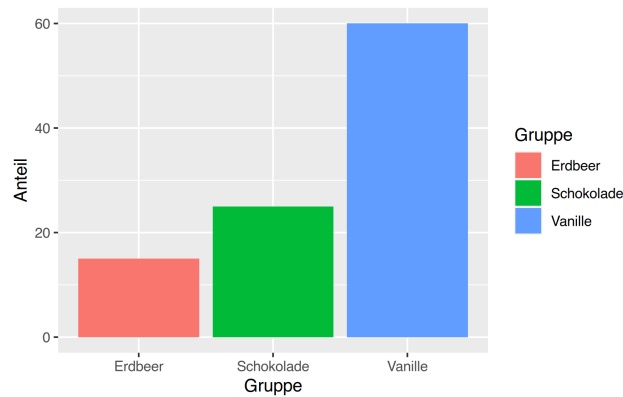


Abb. 2.93

Mit der Funktion `geom_text()` können die Werte den Plot-Balken hinzugefügt werden. Über den Parameter `vjust` kann bestimmt werden, ob die Werte über oder in den Säulen stehen sollen.

```

1 ggplot(df, aes(x=Gruppe, y=Anteil,
2               fill=Gruppe)) +
3   geom_col(position="dodge")+
4   # Werte stehen über den Balken
5   geom_text(aes(label = Anteil), vjust = -0.2)

```

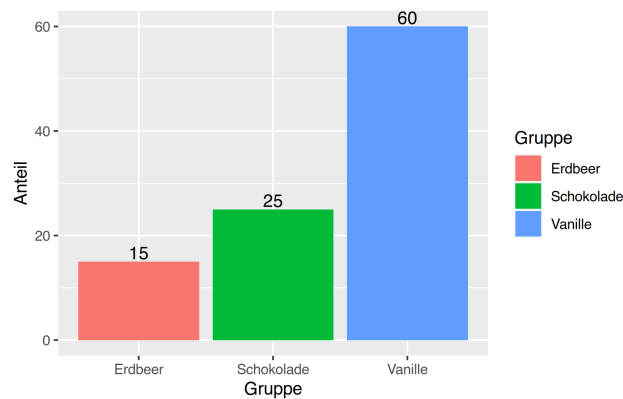


Abb. 2.94: Werte stehen über den Balken

```

1 ggplot(df, aes(x=Gruppe, y=Anteil,
2               fill=Gruppe)) +
3   geom_col(position="dodge")+
4   # Werte stehen über den Balken
5   geom_text(aes(label = Anteil), vjust = 2)

```

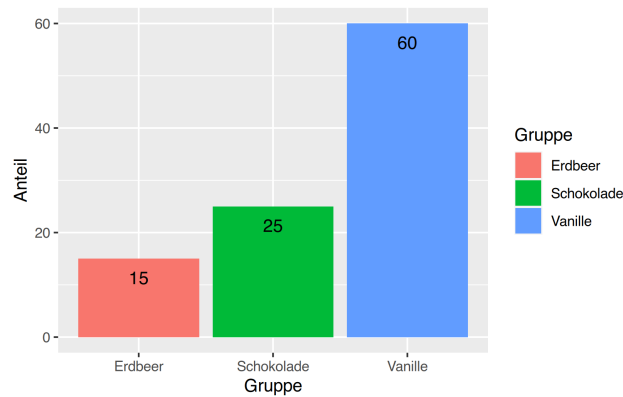


Abb. 2.95: Werte stehen in den Balken

2.4.6 Balkendiagramme

Balkendiagramme können auf zwei Arten erstellt werden. In der `aes()`-Funktion tauschen wir einfach `x` durch `y` aus:

```

1 ggplot(pf8) +
2   aes(y=SportWarum ) +
3   geom_bar(color="blue",
4           fill="skyblue")

```

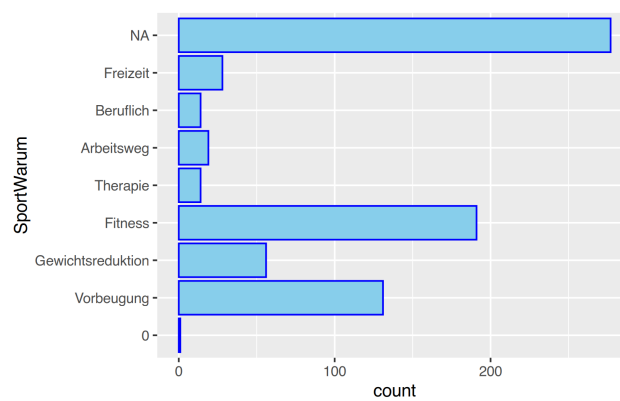


Abb. 2.96

Alternativ kann dem Plotaufwurf die Funktion `coord_flip()` angehängt werden.

```
1 ggplot(pf8) +  
2   aes(x=SportWarum ) +  
3   geom_bar(color="blue",  
4           fill="skyblue") +  
5   coord_flip()
```

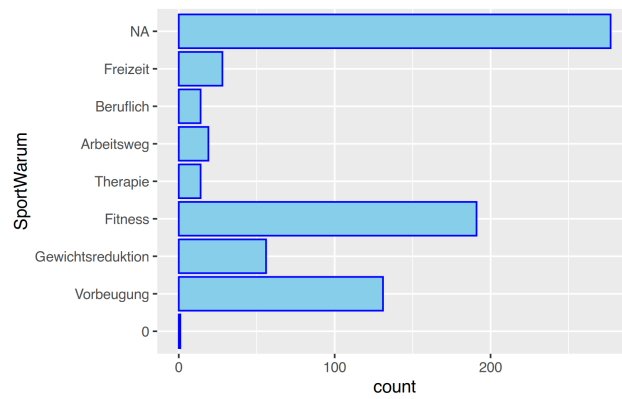


Abb. 2.97

2.4.7 Boxplots

Boxplots werden mit der Funktion `geom_boxplot()` erzeugt.

```
1 pf8 %>%
2   ggplot() +
3     aes(x = Alter) +
4     geom_boxplot(fill="seagreen3")
```

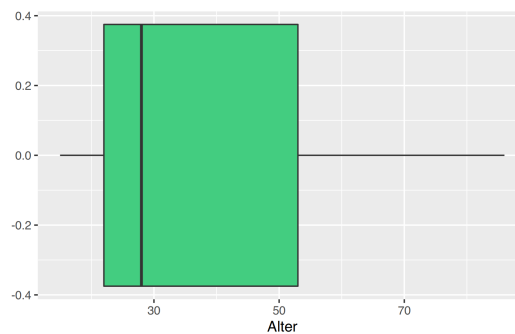


Abb. 2.98

Das Boxplot liegt auf dem Bauch, das `Alter` auf der x-Achse abgebildet werden soll. Um das Boxplot zu drehen, muss `aes()` entsprechend angepasst werden.

```
1 pf8 %>%
2   ggplot() +
3     aes(y = Alter) +
4     geom_boxplot(fill="seagreen3")
```

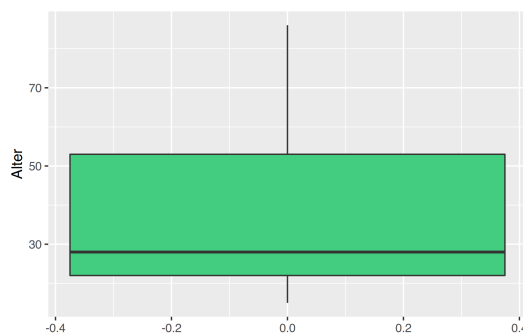


Abb. 2.99

Mit der Funktion `stat_boxplot()` können wir die Whiskers hinzugefügt werden.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter) +
4     geom_boxplot(fill="seagreen3") +
5     stat_boxplot(geom="errorbar")

```

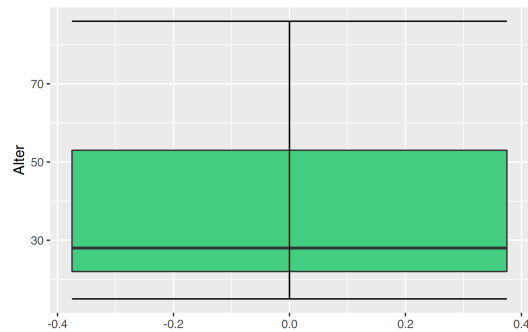


Abb. 2.100

Über den `color`-Parameter können die Farben angepasst werden.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter) +
4     geom_boxplot(fill="seagreen3",
5                 color="darkblue") +
6     stat_boxplot(geom="errorbar",
7                 color="darkgreen")

```

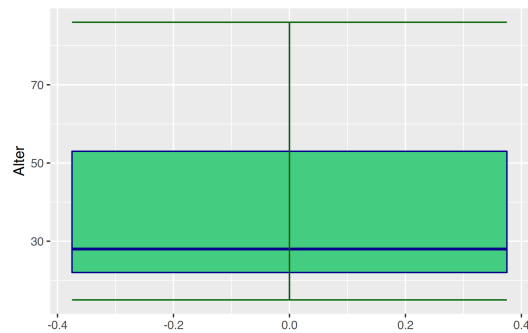


Abb. 2.101

Über die Aesthetics können wir nach **Geschlecht** gruppieren.


```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         x = Geschlecht) +
5     geom_boxplot(fill="seagreen3",
6                 color="darkblue") +
7     stat_boxplot(geom="errorbar",
8                 color="darkgreen")

```

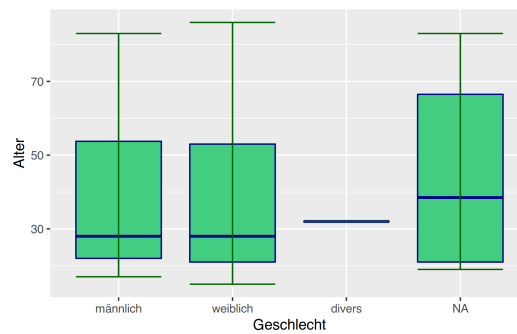


Abb. 2.102

Bzw. ohne NA:

```

1 pf8 %>%
2   drop_na(Geschlecht) %>%
3   ggplot() +
4     aes(y = Alter,
5         x = Geschlecht) +
6     geom_boxplot(fill="seagreen3",
7                 color="darkblue") +
8     stat_boxplot(geom="errorbar",
9                 color="darkgreen")

```

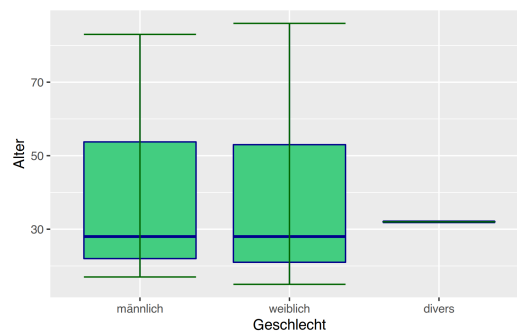


Abb. 2.103

Über die Aesthetics können wir die Boxplots auch nach **Geschlecht** färben. Dazu muss der Parameter aus `geom_boxplot()` entfernt werden.

```
1 pf8 %>%
2   drop_na(Geschlecht) %>%
3   ggplot() +
4     aes(y = Alter,
5         x = Geschlecht,
6         fill = Geschlecht) +
7     geom_boxplot(color="darkblue") +
8     stat_boxplot(geom="errorbar",
9                 color="darkgreen")
```

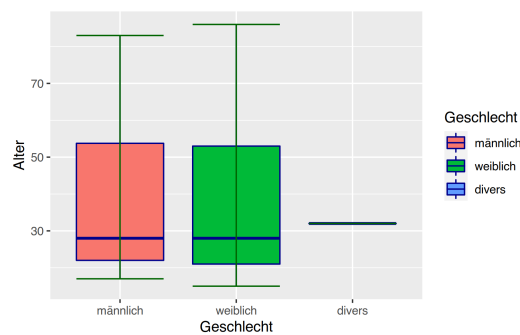


Abb. 2.104

In manchen Journals ist es üblich, ebenfalls das arithmetische Mittel per Rauten den Boxplots hinzuzufügen. Dies kann mit der Funktion `stat_summary()` umgesetzt werden.

```
1 pf8 %>%
2   drop_na(Geschlecht) %>%
3   ggplot() +
4     aes(y = Alter,
5         x = Geschlecht,
6         fill = Geschlecht) +
7     geom_boxplot(color="darkblue") +
8     stat_boxplot(geom="errorbar",
9                 color="darkgreen")+
10    stat_summary(fun=mean,
11                colour="darkred",
12                geom="point",
13                shape=18,
14                size=5,
15                show.legend = F)
```

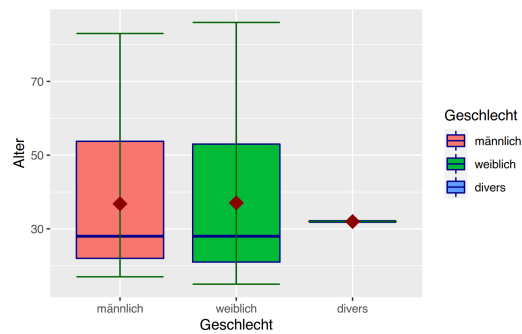


Abb. 2.105

Mit `facet_wrap()` können wir beispielweise nach `Standort` gruppieren.

```

1 pf8 %>%
2   drop_na(Geschlecht) %>%
3   ggplot() +
4     aes(y = Alter,
5         x = Geschlecht,
6         fill = Geschlecht) +
7     geom_boxplot(color="darkblue") +
8     stat_boxplot(geom="errorbar",
9                 color="darkgreen")+
10    stat_summary(fun=mean,
11                colour="darkred",
12                geom="point",
13                shape=18,
14                size=5,
15                show.legend = F)+
16    facet_wrap(~ Standort)

```

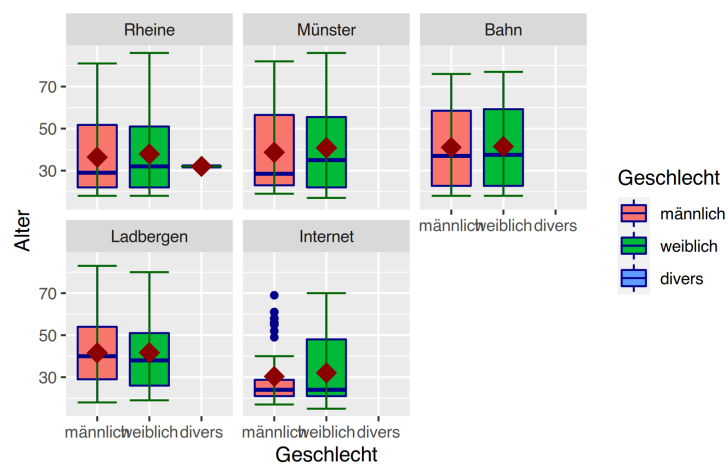


Abb. 2.106

2.4.8 Kreisdiagramme

Kreisdiagramme werden von `ggplot()` nicht direkt unterstützt, können jedoch mit einem Trick erzeugt werden. Hierbei wird das Koordinatensystem eines Balkendiagramms mittels `coord_polar()` so verbogen, dass ein Kreis dabei herauskommt.

```

1 # pfusche Pie-Chart zurecht
2 pf8 %>%
3   ggplot() +
4     aes(x="", fill=Geschlecht) +
5     geom_bar(color="white") +
6     # verbiege Koordinatensystem
7     coord_polar("y") +
8     # entferne Achsen und Ticks
9     theme_void()

```

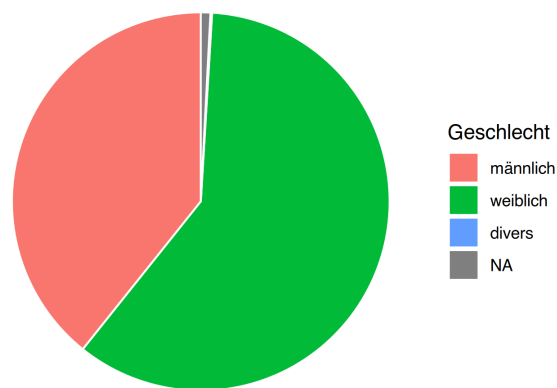


Abb. 2.107

Das funktioniert auch mit `geom_col()` und absoluten Häufigkeiten.

```
1 # vorgegebene Werte
2 df <- data.frame(Gruppe=c("Vanille", "Schokolade", "Erdbeer"),
3                       Anteil=c(60, 25, 15))
4
5 # pfusche Pie-Chart zurecht
6 ggplot(df, aes(x="",
7                 y=Anteil,
8                 fill=Gruppe)) +
9   geom_col(color="white") +
10  # verbiege Koordinatensystem
11  coord_polar("y", start=0) +
12  # entferne Achsen und Ticks
13  theme_void()
```

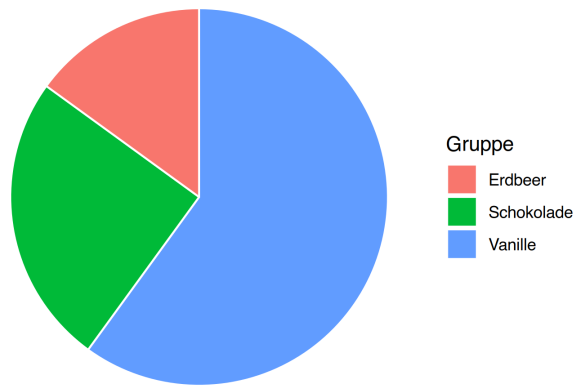


Abb. 2.108

2.4.9 Diagramme speichern

2.4.9.1 R-Objekt

`ggplot()`-Diagramme können in R-Objekte (zwischen)gespeichert werden.

```
1 # Plot in Objekt "p" speichern
2 p <- ggplot(pf8) +
3     aes(x="", fill=Geschlecht) +
4     geom_bar(color="white")
5 # Plot anzeigen
6 p
```

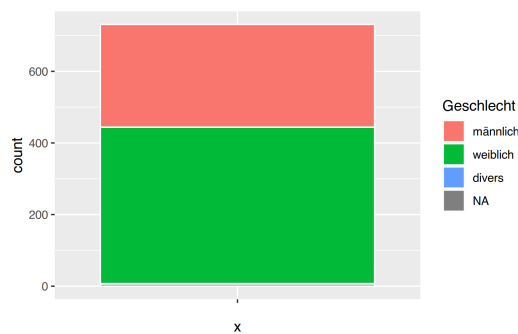


Abb. 2.109

Das Objekt kann nun weitere Funktionen verarbeiten.

```
1 # Plot in Objekt "p" speichern
2 p <- ggplot(pf8) +
3     aes(x="", fill=Geschlecht) +
4     geom_bar(color="white")
5 # Plot weiter bearbeiten
6 p + coord_polar("y", start=0) +
7     theme_void()
```

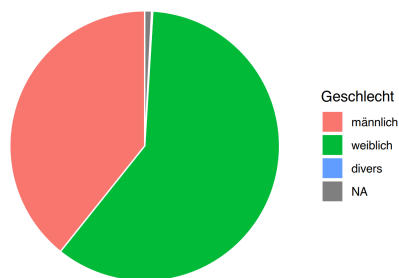


Abb. 2.110

2.4.9.2 in Datei speichern

Mit der Funktion `ggsave()` können `ggplot()`-Diagramme als Datei gespeichert werden.

```
1  # Plot erzeugen
2  ggplot(pf8) +
3    aes(x="", fill=Geschlecht) +
4    geom_bar(color="white") +
5    coord_polar("y", start=0) +
6    theme_void()
7
8  # Plot in Datei speichern
9  ggsave("MeinPlot.png",
10        units="mm",
11        width=400,
12        height=200,
13        dpi=300,
14        pointsize=7)
```

Liegt der Plot in einem R-Objekt, kann dies mit dem Parameter `plot` angegeben werden.

```
1  # Plot ist in Objekt "p" gespeichert
2  ggsave("MeinPlot.png",
3        plot= p,
4        units="mm",
5        width=400,
6        height=200,
7        dpi=300,
8        pointsize=7)
```

2.4.10 Aussehen ändern

2.4.10.1 Titel und Überschriften

Die Plottitel können mit der Funktion `labs()` verändert werden.

```

1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue",
4                 fill="skyblue") +
5   labs(title="Dies ist Überschrift",
6        subtitle="Dies ist die Unter-Überschrift",
7        caption="Dies ist die Bildunterschrift")

```

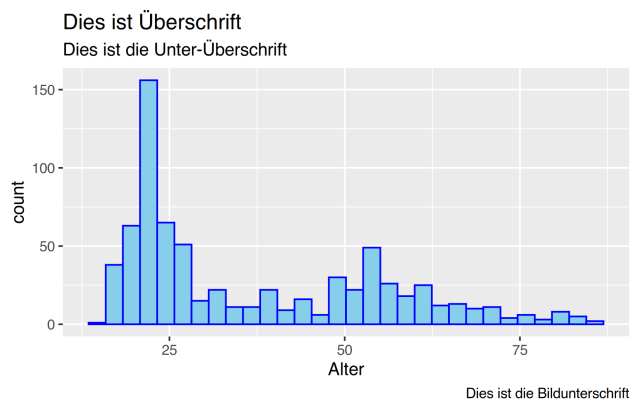


Abb. 2.111

Die Schriftart kann mittels `theme()`-Funktion geändert werden.

```

1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue",
4                 fill="skyblue") +
5   labs(title="Dies ist Überschrift",
6        subtitle="Dies ist die Unter-Überschrift in fett",
7        caption="Dies ist die Bildunterschrift in kursiv") +
8   theme(plot.title = element_text(size=18)) +
9   theme(plot.subtitle = element_text(size=10, face="bold")) +
10  theme(plot.caption = element_text(size=8, face="italic"))

```

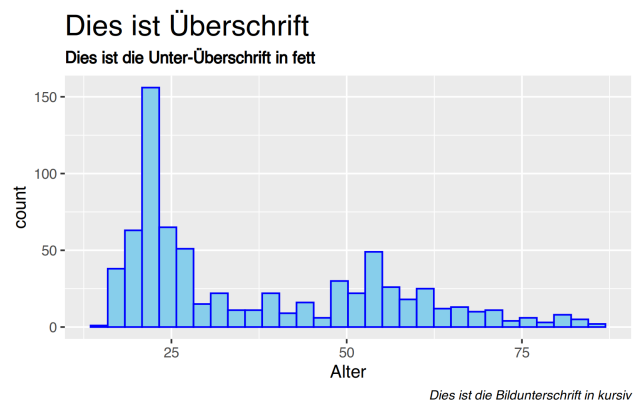



Abb. 2.112

2.4.10.2 Achsen

Die Achsenbeschriftung kann mittels `xlab()` und `ylab()` verändert werden.

```
1 ggplot(pf8) +
2   aes(x=Alter) +
3   geom_histogram(color="blue",
4                 fill="skyblue") +
5   xlab("Ich bin die x-Achse") +
6   ylab("Ich bin die y-Achse")
```

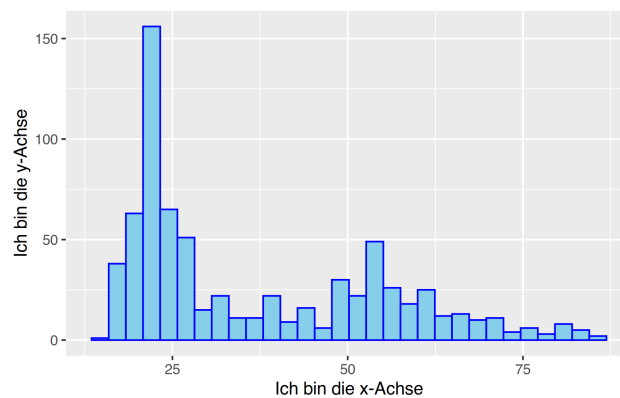


Abb. 2.113

Die Längen der Achsenausschnitte können mit `xlim()` und `ylim()` angegeben werden.

```

1  ggplot(pf8) +
2    aes(x=Alter) +
3    geom_histogram(color="blue",
4                  fill="skyblue") +
5    xlab("Ich bin die x-Achse") +
6    ylab("Ich bin die y-Achse")+
7    xlim(20, 50) +
8    ylim(-20, 75)

```

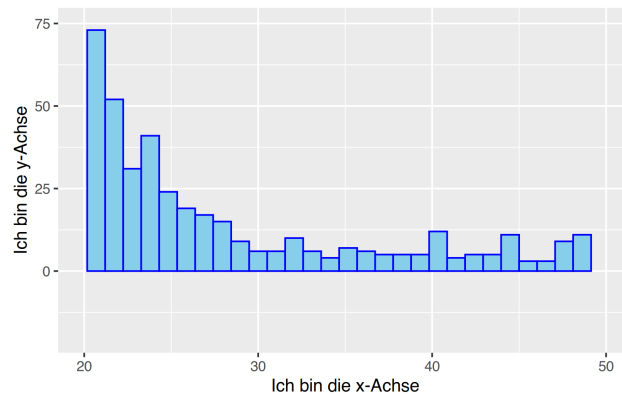


Abb. 2.114

Die Anzahl und Position der Achsenmarkierungen können mit `scale_x_continuous()` und `scale_y_continuous()` angepasst werden.

```

1  ggplot(pf8) +
2    aes(x=Alter) +
3    geom_histogram(color="blue",
4                  fill="skyblue") +
5    xlab("Ich bin die x-Achse") +
6    ylab("Ich bin die y-Achse")+
7    xlim(20, 50) +
8    ylim(-20, 75) +
9    scale_x_continuous(breaks = c(20, 25, 30, 35, 40)) +
10   scale_y_continuous(breaks = seq(-20, 100, 10) )

```

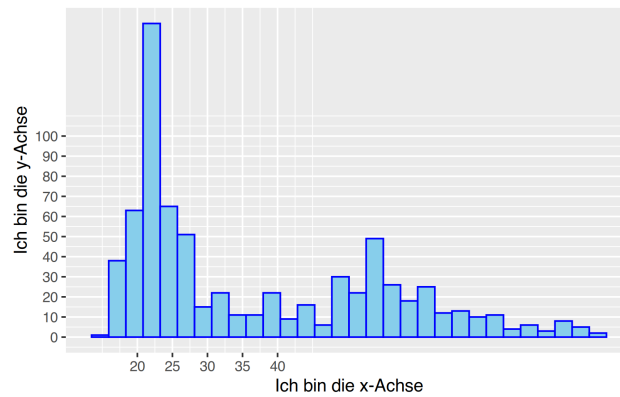


Abb. 2.115

Die Schriftarten werden ebenfalls mit der `theme()`-Funktion gesteuert.

```

1  ggplot(pf8) +
2    aes(x=Alter) +
3    geom_histogram(color="blue",
4                  fill="skyblue") +
5    xlab("Ich bin die x-Achse") +
6    ylab("Ich bin die y-Achse")+
7    xlim(20, 50) +
8    ylim(-20, 75) +
9    scale_x_continuous(breaks = c(20, 25, 30, 35, 40)) +
10   scale_y_continuous(breaks = seq(-20, 100, 10) ) +
11   theme(axis.title.x = element_text(size=18, face="bold")) +
12   theme(axis.title.y = element_text(size=14, face="italic")) +
13   theme(axis.text.x = element_text(size=8, color="red")) +
14   theme(axis.text.y = element_text(size=4, color="blue")) +
15   theme(axis.ticks.x = element_line(size=8, color="darkgreen")) +
16   theme(axis.ticks.y = element_line(size=8, color="orange"))

```

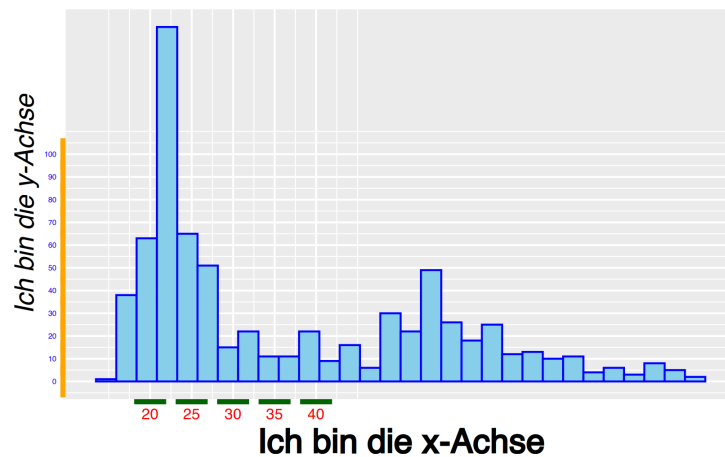


Abb. 2.116

So können die Achsenbeschriftungen auch über den Parameter **angle** gedreht werden.

```

1 ggplot(pf8) +
2   aes(x=SportWarum ) +
3   geom_bar(color="blue",
4           fill="skyblue") +
5   theme(axis.text.x = element_text(angle = 45,
6                                   hjust = 1,
7                                   size=10))

```

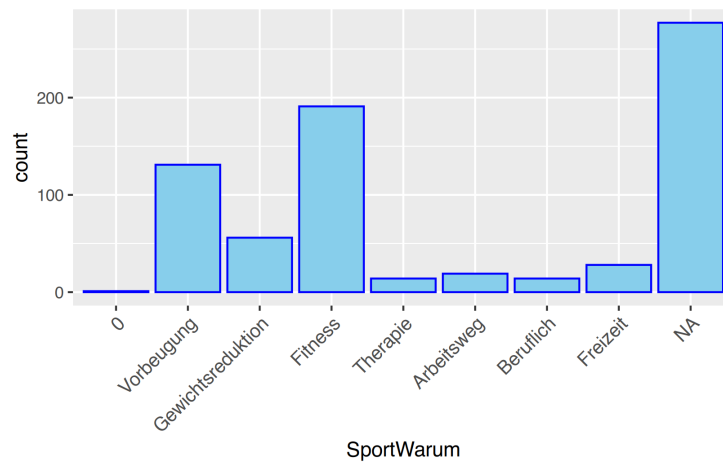


Abb. 2.117

2.4.10.3 Legendenbox

Der **Title der Legendenbox** kann prinzipiell geändert werden, indem die ursprüngliche Variable umbenannt wird. Dann ändert sich der Legendenboxtitel ebenfalls entsprechend. Sollte dies keine “gute” Lösung sein, z.B. weil mehrere Wörter als Titel verwendet werden sollen, kann der Title über die **labs()**-Funktion geändert werden. Hierbei besteht eine Abhängigkeit zur Gruppierung über die **aes()**-Funktion. Im folgenden Beispiel wird die Gruppierung über den Parameter **fill=Geschlecht** erzeugt. Daher muss der Legendentitel mittels **labs(fill="NEU")** geändert werden.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         fill= Geschlecht) +
5     geom_boxplot() +
6     labs(fill="Neuer Titel")

```

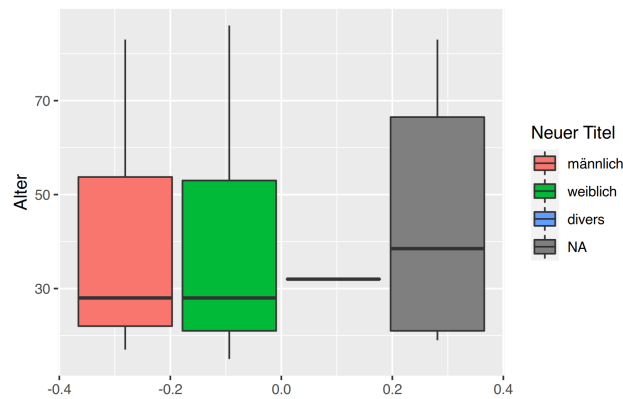


Abb. 2.118

Sollen die **Einträge der Legenbox** geändert werden, kann dies zum einen erfolgen, indem die Levels des Originalfactor entsprechend umgeschrieben werden. Dann ändern sich auch die Einträge in der Legendenbox. Sollte dies keine “gute” Lösung sein, können die Einträge von Hand über die Funktion `scale_fill_discrete()` überschrieben werden.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         fill= Geschlecht) +
5     geom_boxplot() +
6     labs(fill="Neuer Titel") +
7     scale_fill_discrete(labels=c('men', 'woman', 'divers', 'no answer'))

```

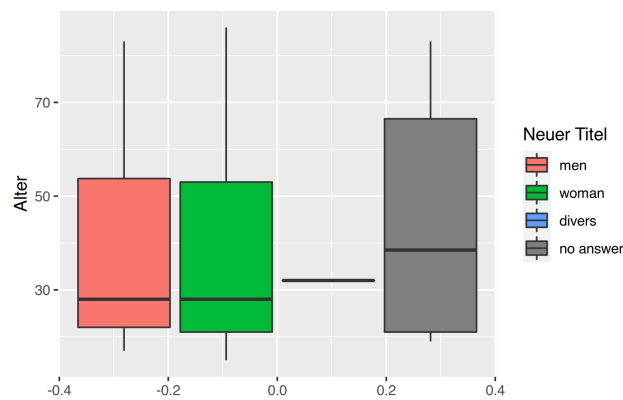


Abb. 2.119

Soll die **Reihenfolge der Levelboxelemente** geändert werden, kann ebenfalls auf die Funktion `scale_fill_discrete()` zurückgegriffen werden.

```
1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         fill= Geschlecht) +
5     geom_boxplot() +
6     labs(fill="Neuer Titel") +
7     scale_fill_discrete(breaks=c(NA, "weiblich", "divers", "männlich"))
```

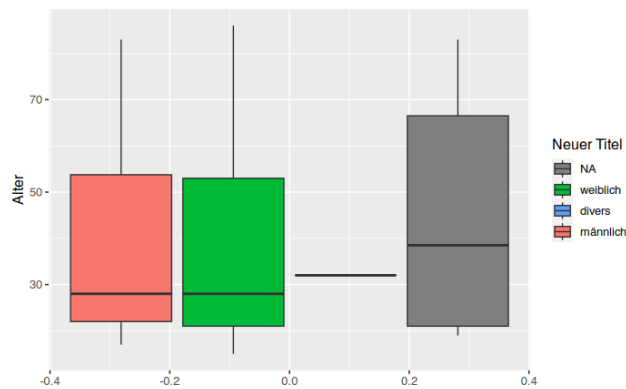


Abb. 2.120

Die **Schriftarten** der Legendenbox können ebenfalls mit der `theme()`-Funktion verändert werden.

```
1 pf8 %>%
2   ggplot() +
```

```

3   aes(y = Alter,
4       fill= Geschlecht) +
5   geom_boxplot() +
6   theme(legend.title = element_text(color="red", face="bold")) +
7   theme(legend.text = element_text(color="blue", face="italic"))

```

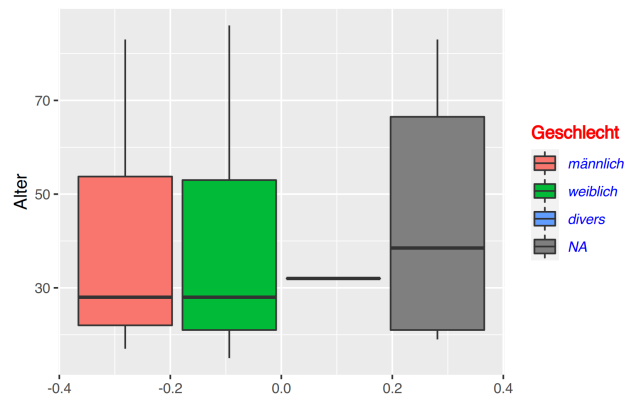


Abb. 2.121

2.4.10.4 Farben

Ohne weitere Angaben verwendet `ggplot()` seine “eingebauten” Standardfarben.

Wenn wir das `Alter` der Probanden nach `Standort` gruppieren, wählt `ggplot()` diese Farben aus:

```

1   pf8 %>%
2   ggplot() +
3   aes(y = Alter,
4       color= Standort,
5       fill= Standort) +
6   geom_boxplot()

```

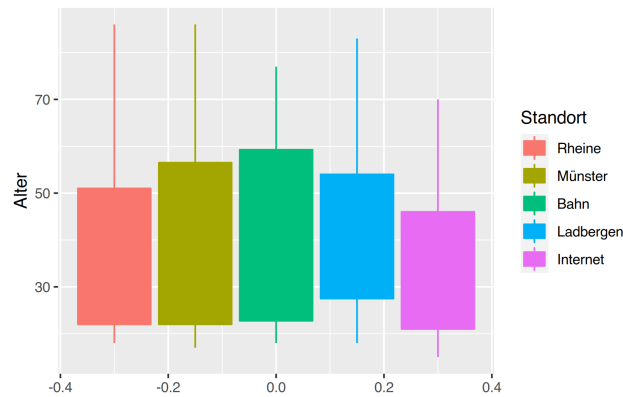


Abb. 2.122

Die Farbintensität und -helligkeit der Standardfarben können über die Funktionen `scale_color_hue()` und `scale_fill_hue()` angepasst werden. Hierbei steht der Parameter `c` für die Farbintensität, und `l` für die Helligkeit.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         color= Standort,
5         fill= Standort) +
6     geom_boxplot() +
7     scale_color_hue(l=40, c=35) +
8     scale_fill_hue(l=40, c=35)

```

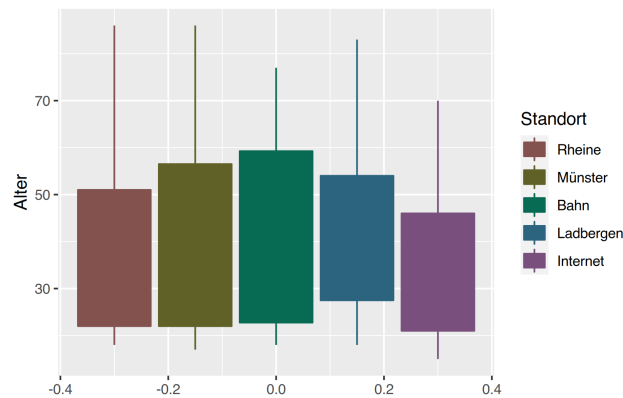


Abb. 2.123

Über die Funktionen `scale_color_manual()` und `scale_fill_manual()` können die Farben manuell ausgewählt werden (eine Liste aller Farbnamen liefert die Funktion `colors()`):


```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         color= Standort,
5         fill= Standort) +
6     geom_boxplot() +
7     scale_color_manual(values=c("darkblue", "navy", "blue",
8                                "skyblue2", "skyblue3")) +
9     scale_fill_manual(values=c("hotpink", "pink", "pink3",
10                               "red3", "red4"))

```

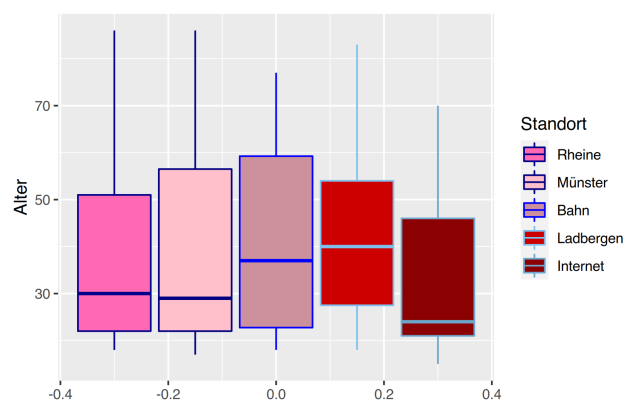


Abb. 2.124

Die Farben können über den **breaks**-Parameter auch direkt den Levelausprägungen zugewiesen werden.

```

1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         color= Standort,
5         fill= Standort) +
6     geom_boxplot() +
7     scale_color_manual(values=c("darkblue", "navy", "blue",
8                                "skyblue2", "skyblue3"),
9                        breaks=c("Internet", "Ladbergen", "Münster",
10                               "Rheine", "Bahn")) +
11     scale_fill_manual(values=c("hotpink", "pink", "pink3",
12                               "red3", "red4"),
13                      breaks=c("Internet", "Ladbergen", "Münster",
14                              "Rheine", "Bahn"))

```

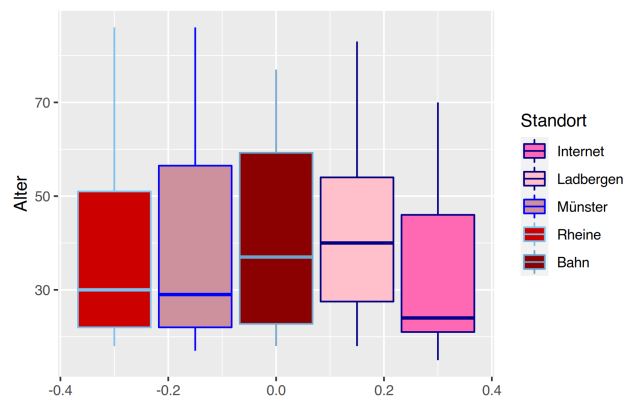


Abb. 2.125

Hierdurch hat sich auch die Reihenfolge der Levelboxelemente geändert.

Soll die Reihenfolge der Levelboxelemente geändert werden, ohne dass eigene Farben definiert werden, kann auf die Funktion `scale_fill_discrete()` zurückgegriffen werden.

```
1 pf8 %>%
2   ggplot() +
3     aes(y = Alter,
4         fill= Standort) +
5     geom_boxplot() +
6     scale_fill_discrete(breaks=c("Internet", "Ladbergen", "Bahn",
7                                   "Rheine", "Münster"))
```

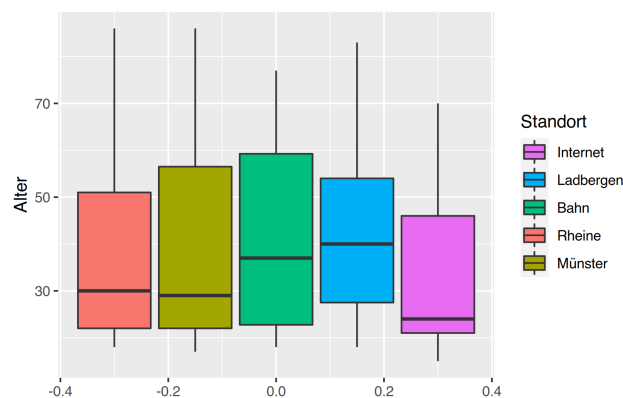


Abb. 2.126

2.4.10.5 Linien

Mit den Funktionen `geom_vline()` und `geom_hline()` können vertikale und horizontale Linien hinzugefügt werden. Mit den Parametern `xintercept` und `yintercept` wird der Schnitt durch die jeweilige Achse festgelegt.

```

1  ggplot(pf8) +
2    aes(x = Alter,
3        y = Gewicht) +
4    geom_point() +
5    geom_hline(yintercept = 60,
6               linetype = "solid",
7               color = "blue",
8               size = 3) +
9    geom_vline(xintercept=40,
10               linetype="dashed",
11               color="red")

```

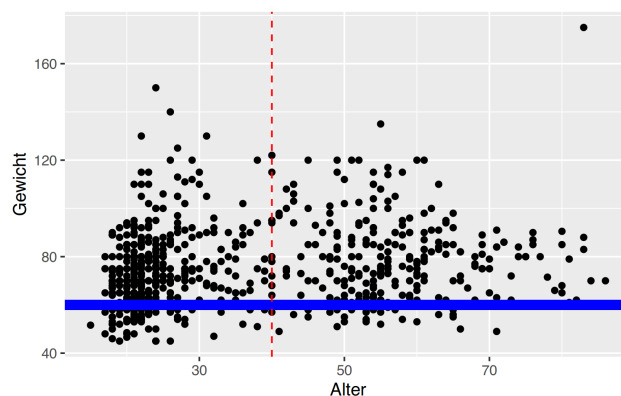


Abb. 2.127

Um zwei beliebige Punkte mit einer Linie zu verbinden, muss zunächst ein Hilfsdatenframe mit den Koordinaten der Punkte erstellt werden. Dieses neue Datenframe wird dann der Funktion `geom_line()` übergeben.

```

1  # Hilfsdatenframe
2  strich <- data.frame(x = c(30,50),
3                       y = c(10,90))
4
5  ggplot(pf8) +
6    aes(x = Alter,
7        y = Gewicht) +
8    geom_point() +
9    geom_line(data = strich,
10             aes(x=x, y=y),

```

```

11     color="darkgreen",
12     size=1)

```

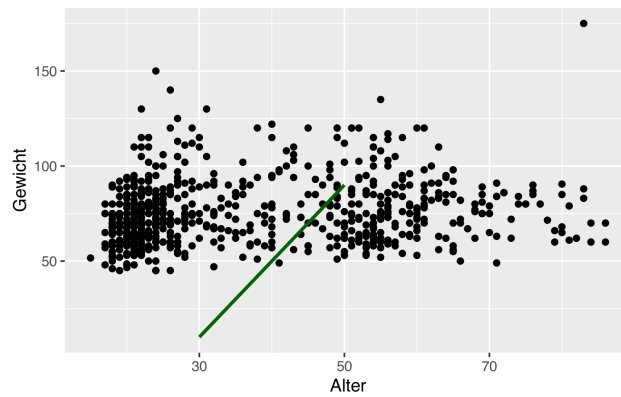


Abb. 2.128

2.4.10.6 Text

Texte können mit der `annotate()`-Funktion hinzugefügt werden.

```

1  ggplot(pf8) +
2    aes(x = Alter,
3        y = Gewicht) +
4    geom_point() +
5    annotate(geom="text",
6            x=30, y=120,
7            label="Huhu",
8            color="darkgreen",
9            size=14)

```

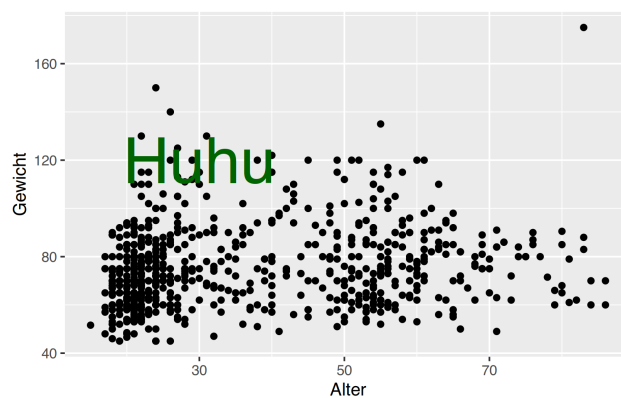


Abb. 2.129

2.4.10.7 Sonderzeichen

Sonderzeichen und Mathematiksymbole können mittels der `expression()`-Funktion hinzugefügt werden.

```

1  ggplot(pf8) +
2    aes(x = Alter,
3        y = Gewicht) +
4    geom_point() +
5    labs(title = expression("Dies ist ein Text mit"~a^2~+~b^2~="~c^2),
6         subtitle = expression(sigma~sqrt(alpha)~beta^3~"sind hier"~x[1])
7    )

```

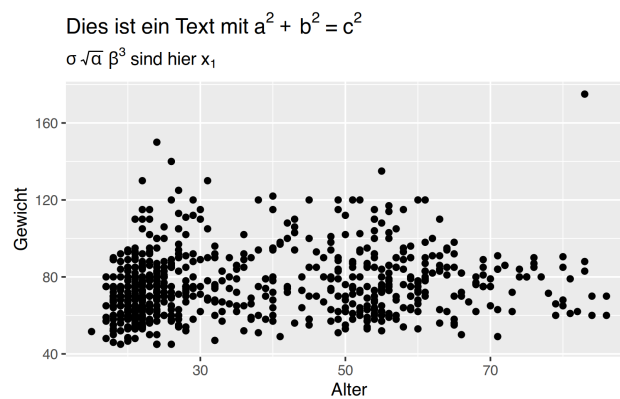


Abb. 2.130

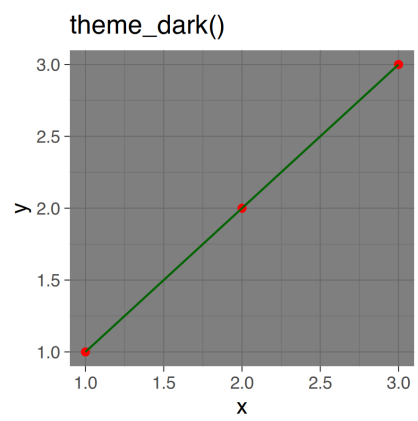
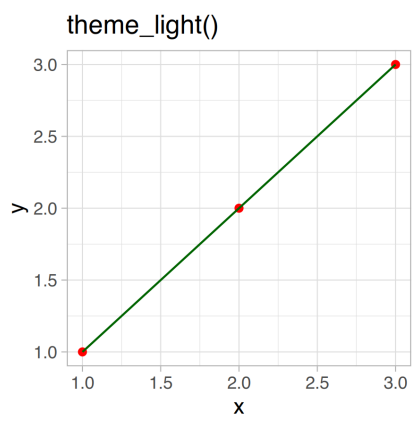
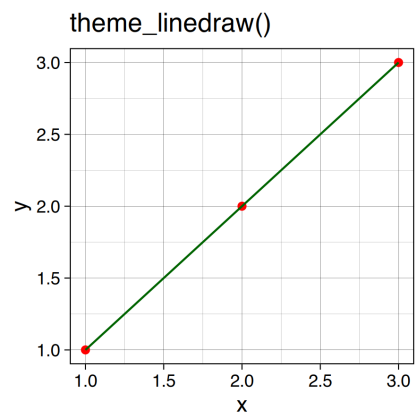
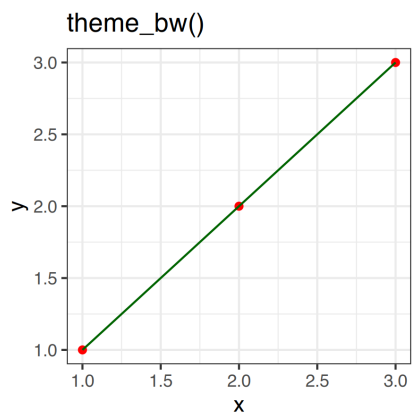
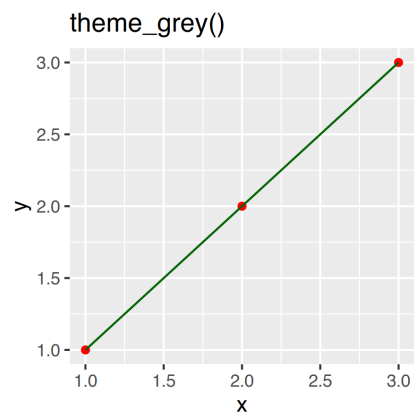
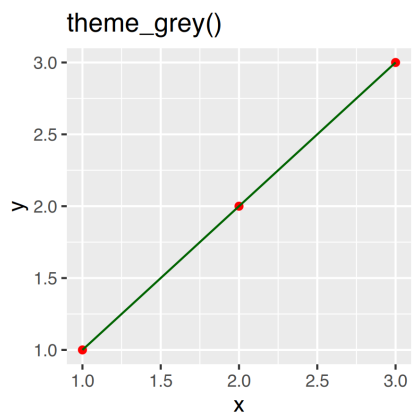
2.4.10.8 Themes

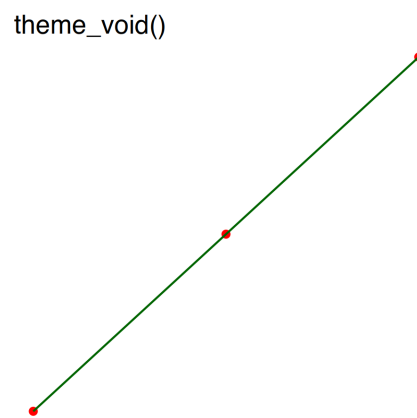
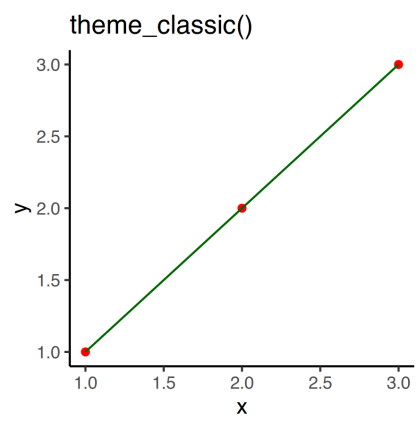
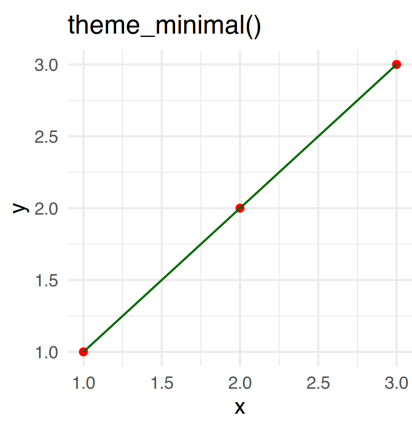
In `ggplot()` sind 8 verschiedene “Themes” implementiert, die das Aussehen der nicht-datenbezogenen Anteile des Plots festlegen. Das Standardtheme ist `theme_grey()`, die anderen sind:

- `theme_bw()`: Eine Variante von `theme_grey()` mit weißem Hintergrund und dünnen grauen Gitterlinien.

- `theme_linedraw()`: Ein Theme mit nur schwarzen Linien unterschiedlicher Stärke auf weißem Hintergrund, das an eine Zeichnung erinnert.
- `theme_light()`: Ähnlich wie `theme_linedraw()`, aber mit hellgrauen Linien und Achsen, um die Aufmerksamkeit stärker auf die Daten zu lenken.
- `theme_dark()`: Das dunkle Pendant von `theme_light()`, mit ähnlichen Linienstärken, aber dunklem Hintergrund. Nützlich, um dünne farbige Linien hervorzuheben.
- `theme_minimal()`: Ein minimalistisches Theme ohne Hintergrundanmerkungen.
- `theme_classic()`: Ein klassisch aussehendes Theme mit x- und y-Achsen-Linien und ohne Gitterlinien.
- `theme_void()`: Ein vollständig leeres Theme.

```
1  # dummy Daten
2  df <- data.frame(x = 1:3, y = 1:3)
3
4  # Basisplot
5  base <- ggplot(df, aes(x, y)) +
6    geom_point(color="red") +
7    geom_line(color="darkgreen")
8
9  # Themepark
10 base + ggtitle("theme_grey()")
11 base + theme_grey() + ggtitle("theme_grey()")
12 base + theme_bw() + ggtitle("theme_bw()")
13 base + theme_linedraw() + ggtitle("theme_linedraw()")
14 base + theme_light() + ggtitle("theme_light()")
15 base + theme_dark() + ggtitle("theme_dark()")
16 base + theme_minimal() + ggtitle("theme_minimal()")
17 base + theme_classic() + ggtitle("theme_classic()")
18 base + theme_void() + ggtitle("theme_void()")
```





3 Was willst du machen?

3.1 Daten manipulieren

Ich möchte meine Datenobjekte manipulieren.

3.1.1 Einer Kreuztabelle Prozentwerte hinzufügen

Ich möchte eine Kreuztabelle ausgeben, die sowohl absolute Häufigkeiten als auch Prozentwerte enthält.

Ich erstelle folgende Kreuztabelle:

```
1 A <- rep(c("A1", "A2", "A3"), 10)
2 B <- sample(1:5, size=30, replace=TRUE)
3 C <- sample(c("ja", "nein"), size=30, replace=TRUE)
4 df <- data.frame(rbind(table(B, A),
5                   table(C, A)))
6 df
```

	A1	A2	A3
1	0	2	3
2	1	1	1
3	1	2	1
4	6	1	4
5	2	4	1
ja	7	6	3
nein	3	4	7

Die einzelnen Werte sollen nun mit den jeweiligen Prozentwerten ergänzt werden. Dies kann mit den Funktionen `transmute()` und `across()` erfolgen.

```
1 df %>%
2   # über jede Spalte
3   transmute(across(everything(),
4   # x ausgeben und " (x%)" anhängen
5   ~ paste0(.x, " (", round(.x / sum(df), 2) * 100, "%)"))
6   )
7
```

	A1	A2	A3
1	0 (0%)	2 (3%)	3 (5%)
2	1 (2%)	1 (2%)	1 (2%)
3	1 (2%)	2 (3%)	1 (2%)
4	6 (10%)	1 (2%)	4 (7%)
5	2 (3%)	4 (7%)	1 (2%)
ja	7 (12%)	6 (10%)	3 (5%)
nein	3 (5%)	4 (7%)	7 (12%)

3.1.2 Eine gemeinsame Kreuztabelle mit mehrere Variablen erstellen

Häufig erstellen wir verschiedene Kreuztabellen mit der selben “Gruppierungsvariable”, z.B. `Geschlecht` oder Kontroll- vs. Interventionsgruppe. Nehmen wir als Beispiel den Datensatz `pf8`.

```
1 load(url("https://www.produnis.de/R/pf8.RData"))
2 # oder
3 pf8 <- jgsbook::pf8
```

Möchten wir hinsichtlich der Variable `Geschlecht` unterscheiden, würden wir die Kreuztabellen mit anderen Variablen einzeln per `xtabs()` oder `table()` erstellen.

```
1 table(pf8$Standort, pf8$Geschlecht)
```

	männlich	weiblich	divers
Rheine	90	112	1
Münster	75	103	0
Bahn	36	48	0
Ladbergen	26	17	0
Internet	60	157	0

```
1 table(pf8$Bildung, pf8$Geschlecht)
```

	männlich	weiblich	divers
keinen	2	3	0
Hauptschule	30	26	0
mittlere Reife	46	53	0
Ausbildung	10	19	0
Fachabitur	23	42	0
Abitur	64	76	0
Hochschule	50	56	1

```
1 table(pf8$Familienstand, pf8$Geschlecht)
```

	männlich	weiblich	divers
ledig	125	162	0
Partnerschaft	56	98	0
verheiratet	90	136	1
geschieden	6	23	0
verwitwet	5	13	0
getrennt	4	4	0

Eine alternative Vorgehensweise besteht darin, mit der Funktion `lapply()` eine `list` der Kreuztabellen zu erstellen.

```
1 tbl.list <- lapply(pf8[, c("Standort", "Bildung", "Familienstand")],
2                   function(x) xtabs(~ x + pf8$Geschlecht))
3
4 tbl.list
```

```
$Standort
      pf8$Geschlecht
x      männlich weiblich divers
Rheine          90      112      1
Münster          75      103      0
Bahn            36       48      0
Ladbergen        26       17      0
Internet         60      157      0

$Bildung
      pf8$Geschlecht
x      männlich weiblich divers
keinen           2        3      0
Hauptschule      30       26      0
mittlere Reife   46       53      0
Ausbildung       10       19      0
Fachabitur       23       42      0
Abitur           64       76      0
Hochschule       50       56      1

$Familienstand
      pf8$Geschlecht
x      männlich weiblich divers
ledig       125      162      0
Partnerschaft 56       98      0
verheiratet  90      136      1
geschieden   6       23      0
verwitwet    5       13      0
getrennt     4        4      0
```

Wir haben Zugriff auf die einzelnen Kreuztabellen per `tbl.list[[1]]` oder `tbl.list[[2]]`.

```
1 tbl.list[[3]]
```

```
      pf8$Geschlecht
x      männlich weiblich divers
ledig       125      162      0
Partnerschaft 56       98      0
verheiratet  90      136      1
geschieden   6       23      0
verwitwet    5       13      0
getrennt     4        4      0
```

3.1.2.1 Zeilen- und Spaltensummen

Mit der Liste können wir nun weiterarbeiten. Zum Beispiel können wir Spalten- und Zeilensummen hinzufügen...

```
1 lapply(tbl.list, addmargins)
```

```

      pf8$Geschlecht
x      männlich weiblich divers
ledig      125      162      0
Partnerschaft  56      98      0
verheiratet   90     136      1
geschieden    6      23      0
verwitwet     5      13      0
getrennt      4       4      0
> lapply(tbl.list, addmargins)
$Standort
      pf8$Geschlecht
x      männlich weiblich divers Sum
Rheine      90     112      1 203
Münster     75     103      0 178
Bahn        36      48      0  84
Ladbergen   26      17      0  43
Internet    60     157      0 217
Sum         287     437      1 725

$Bildung
      pf8$Geschlecht
x      männlich weiblich divers Sum
keinen       2       3      0   5
Hauptschule  30      26      0  56
mittlere Reife 46     53      0  99
Ausbildung   10      19      0  29
Fachabitur   23      42      0  65
Abitur       64      76      0 140
Hochschule   50      56      1 107
Sum          225     275      1 501

$Familienstand
      pf8$Geschlecht
x      männlich weiblich divers Sum
ledig      125     162      0 287
Partnerschaft  56      98      0 154
verheiratet   90     136      1 227
geschieden    6      23      0  29
verwitwet     5      13      0  18
getrennt      4       4      0   8
Sum          286     436      1 723

```

3.1.2.2 Prozenttabellen

... oder Prozenttabellen erstellen. Um die Prozentwerte *zeilenweise* zu bilden, muss der Parameter `margin=1` angegeben werden. Für *spaltenweises* Vorgehen gilt `margin=2`.

```
1 lapply(tbl.list, prop.table, margin=1)
```

```
$Standort
      pf8$Geschlecht
x      männlich  weiblich  divers
Rheine  0.443349754 0.551724138 0.004926108
Münster  0.421348315 0.578651685 0.000000000
Bahn     0.428571429 0.571428571 0.000000000
Ladbergen 0.604651163 0.395348837 0.000000000
Internet 0.276497696 0.723502304 0.000000000

$Bildung
      pf8$Geschlecht
x      männlich  weiblich  divers
keinen  0.400000000 0.600000000 0.000000000
Hauptschule 0.535714286 0.464285714 0.000000000
mittlere Reife 0.464646465 0.535353535 0.000000000
Ausbildung 0.344827586 0.655172414 0.000000000
Fachabitur 0.353846154 0.646153846 0.000000000
Abitur     0.457142857 0.542857143 0.000000000
Hochschule 0.467289720 0.523364486 0.009345794

$Familienstand
      pf8$Geschlecht
x      männlich  weiblich  divers
ledig    0.435540070 0.564459930 0.000000000
Partnerschaft 0.363636364 0.636363636 0.000000000
verheiratet 0.396475771 0.599118943 0.004405286
geschieden 0.206896552 0.793103448 0.000000000
verwitwet 0.277777778 0.722222222 0.000000000
getrennt 0.500000000 0.500000000 0.000000000
```

3.1.2.3 Signifikanztests

Zudem können wir Signifikanztests rechnen.

```
1 lapply(tbl.list, chisq.test)
```

```
$Standort
  Pearson's Chi-squared test
data:  X[[i]]
X-squared = 26.296, df = 8, p-value = 0.0009347

$Bildung
  Pearson's Chi-squared test
data:  X[[i]]
X-squared = 9.4526, df = 12, p-value = 0.6639

$Familienstand
  Pearson's Chi-squared test
data:  X[[i]]
X-squared = 10.503, df = 10, p-value = 0.3975
```

Wenn wir die Liste in ein Objekt speichern, können wir uns nur die p-Werte ausgeben lassen.

```
1 chi.list <- lapply(tbl.list, chisq.test)
2 chi.list[[1]]$p.value
```

```
[1] 0.0009347078
```

3.1.2.4 eine große Kreuztabelle

Zu guter Letzt können wir die einzelnen Kreuztabellen zu einer "großen" Tabelle vereinen.

```

1 dummy <- rbind(tbl.list[[1]],
2               tbl.list[[2]],
3               tbl.list[[3]])
4 # erzeuge ein Datenframe aus den Zeilenamen und den Werten
5 dummy <- data.frame( row.names(dummy), dummy )
6
7 # lösche die alten Rownames
8 rownames(dummy) <- NULL
9 dummy

```

	row.names.dummy.	männlich	weiblich	divers
1	Rheine	90	112	1
2	Münster	75	103	0
3	Bahn	36	48	0
4	Ladbergen	26	17	0
5	Internet	60	157	0
6	keinen	2	3	0
7	Hauptschule	30	26	0
8	mittlere Reife	46	53	0
9	Ausbildung	10	19	0
10	Fachabitur	23	42	0
11	Abitur	64	76	0
12	Hochschule	50	56	1
13	ledig	125	162	0
14	Partnerschaft	56	98	0
15	verheiratet	90	136	1
16	geschieden	6	23	0
17	verwitwet	5	13	0
18	getrennt	4	4	0

3.1.3 Zeilen und Spalten tauschen

Ich möchte bei meinem Datensatz Zeilen und Spalten vertauschen.

Dies kann mit der Funktion `apply()` gemacht werden. Angenommen das Datenframe `datensatz` sieht wie folgt aus ...

```
1 # lade Testdatensatz
2 datensatz <- read.table(url("http://www.produis.de/R/DieDaten.csv"), sep=";", header=TRUE)
3 # zeige "datensatz" an
4 datensatz
```

	Name	Geschlecht	Lieblingsfarbe	Einkommen
1	Hans	maennlich	gruen	1233
2	Caro	weiblich	blau	800
3	Lars	intersexuell	gelb	2400
4	Ines	weiblich	schwarz	4000
5	Samira	weiblich	gelb	899
6	Peter	maennlich	gruen	1100
7	Sarah	weiblich	blau	1900

...so tauschen wir Zeilen und Spalten mittels

```
1 # vertausche Spalten und Zeilen
2 apply(datensatz, MARGIN=1, FUN=function(x) {x})
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
Name	"Hans"	"Caro"	"Lars"	"Ines"	"Samira"	"Peter"	"Sarah"
Geschlecht	"maennlich"	"weiblich"	"isexeuell"	"weiblich"	"weiblich"	"maennlich"	"weiblich"
Lieblingsfarbe	"gruen"	"blau"	"gelb"	"schwarz"	"gelb"	"gruen"	"blau"
Einkommen	"1233"	" 800"	"2400"	"4000"	" 899"	"1100"	"1900"

Da die Funktion `apply()` eine Typkonversion in die Klasse `matrix` vornimmt, (und somit alle Datentypen auf den kleinsten gemeinsamen Nenner `character` zurückfallen, siehe die Anführungszeichen im Output) muss bei Bedarf zurück in die Klasse `data.frame` konvertiert werden.

```
1 # vertausche Spalten und Zeilen
2 as.data.frame(apply(datensatz, MARGIN=1, FUN=function(x) {x}))
```

	V1	V2	V3	V4	V5	V6	V7
Name	Hans	Caro	Lars	Ines	Samira	Peter	Sarah
Geschlecht	maennlich	weiblich	isexeuell	weiblich	weiblich	maennlich	weiblich
Lieblingsfarbe	gruen	blau	gelb	schwarz	gelb	gruen	blau
Einkommen	1233	800	2400	4000	899	1100	1900

3.1.4 Klassen für Variablen bilden (klassieren)

Ich möchte die Werte einer Variable in Kategorien bzw. Klassen einteilen.

Angenommen, meine Variable enthält Werte zwischen 0 und 500, und ich möchte diese Werte in die Gruppen “0-70”, “71-200”, “201-400”, “>400”, klassieren, wie gehe ich dann vor?

R base:

Im klassischen  erstellen wir zunächst zufällige Zahlen zwischen 0 und 500.

```
1 # lade tidyverse
2 library(tidyverse)
3 # erzeuge 200 Zufallszahlen von 0 bis 500
4 dummy <- sample(0:500, 200)
5 # bzw. direkt als Dataframe
6 dummy <- data.frame(x = sample(0:500, 200))
```

Nun erzeugen wir die neue Variable `xKAT`, in welcher die Klassierung angegeben werden soll.

```
1 # wir bilden Kategorien
2 # "0-70"
3 # "71-200"
4 # "201-400"
5 # "> 400"
6 # und speichern das in die neue Variable xKAT
7 dummy$xKAT[dummy$x < 71] <- "0-70"
8 dummy$xKAT[dummy$x < 201 & dummy$x > 70] <- "71-200"
9 dummy$xKAT[dummy$x < 401 & dummy$x > 200] <- "201-400"
10 dummy$xKAT[dummy$x > 400] <- "größer 400"
```

Anschließend wandeln wir die neue Variable in einen ordinalen Faktor mit korrekter Levelreihenfolge.

```
1 dummy$xKAT <- factor(dummy$xKAT, levels=c("0-70",
2                                           "71-200",
3                                           "201-400",
4                                           "größer 400"),
5                               ordered=TRUE)
6 head(dummy$xKAT)
```

```
[1] 201-400    201-400    201-400    größer 400 201-400
[6] 201-400
Levels: 0-70 < 71-200 < 201-400 < größer 400
```

Tidyverse:

Zunächst laden wir das `tidyverse`-Paket und erstellen uns zufällige Zahlen zwischen 0 und 500.

```

1 # lade tidyverse
2 library(tidyverse)
3 # erzeuge 200 Zufallszahlen von 0 bis 500
4 dummy <- sample(0:500, 200)
5 # bzw. direkt als Tibble
6 dummy <- tibble(x = sample(0:500, 200))

```

Nun erzeugen wir die neue Variable `xKAT`, in welcher die Klassierung angegeben werden soll. Hierfür nutzen wir innerhalb von `mutate()` die `ifelse()`-Funktion. Diese folgte der Logik "WELCHE - WAS -ANSONSTEN".

In einem ersten beispielhaften Schritt wählen wir all " $x < 71$ " aus, speichern für diese Fälle den `character`-Wert "0-70", und bei allen anderen Fällen ein `NA`.

```

1 dummy %>%
2   # ifelse(WELCHE, WAS, ANSONSTEN)
3   mutate(xKAT = ifelse(x < 71 , "0-70", NA))

```

Es lassen sich mehrere `ifelse()`-Ausdrücke kombinieren, indem diese vor den `NA`-Ausdruck geschrieben werden. Dabei sammeln sich die Klammer-Zu Symbole `)` an, kommen Sie hier nicht durcheinander!

```

1 # Das lässt sich erweitern,
2 # indem der ", NA"-Ausdruck nach hinten wandert
3 dummy %>%
4   mutate(xKAT = ifelse(x < 71 , "0-70",
5     ifelse(x < 201 & x > 70 , "71-200",
6       NA)))

```

Der vollständige Befehl zur Klassierung lautet demnach:

```

1 # wir bilden Kategorien
2 #   "0-70"
3 #   "71-200"
4 #   "201-400"
5 #   "> 400"
6 # und speichern das in die neue Variable xKAT
7 dummy %>%

```

```

8 mutate(xKAT = ifelse(x < 71 , "0-70",
9                     ifelse(x < 201 & x > 70 , "71-200",
10                      ifelse(x < 401 & x > 200 , "201-400",
11                       ifelse(x > 400 , "> 400",
12                            NA))))))

```

Wenn - so wie hier - mehrere Konditionen angegeben werden, ist die Funktion `case_when()` etwas einfacher zu schreiben und zu lesen als die `ifelse()`-Staffelungen.

```

1 # nutze case_when() an Stelle von ifelse()
2 dummy %>%
3   mutate(xKAT = case_when(x < 71 ~ "0-70",
4                           x < 201 & x > 70 ~ "71-200",
5                           x < 401 & x > 200 ~ "201-400",
6                           x > 400 ~ "größer 400")
7 )

```

Anschließend wandeln wir die neue Variable in einen ordinalen Faktor mit korrekter Levelreihenfolge.

```

1 # die neue Variable als ordinalen Factor mit korrekter
2 # Levelreihenfolge speichern
3 dummy <- dummy %>%
4   mutate(xKAT = case_when(x < 71 ~ "0-70",
5                           x < 201 & x > 70 ~ "71-200",
6                           x < 401 & x > 200 ~ "201-400",
7                           x > 400 ~ "größer 400"),
8   xKAT = factor(xKAT, levels=c("0-70",
9                               "71-200",
10                              "201-400",
11                              "größer 400"),
12               ordered=TRUE)
13 )
14 head(dummy$xKAT)

```

```

[1] 201-400    201-400    201-400    größer 400 201-400
[6] 201-400
Levels: 0-70 < 71-200 < 201-400 < größer 400

```

3.2 Diagramme plotten

Ich möchte Diagramme plotten!

3.2.1 Normalverteilung

Zum Plotten von Normalverteilungen (siehe auch Abschnitt 2.3.1 auf Seite 304) kann man mit der Funktion `plot()` so vorgehen:

```
1 # erstelle Werte von -4 bis 4 in 0.005er-Schritten
2 x <- seq(-4, 4, by=0.005)
3
4 # plote die Standardnormalverteilung
5 plot(x,dnorm(x))
```

Der Befehl erzeugt Abbildung 3.1.

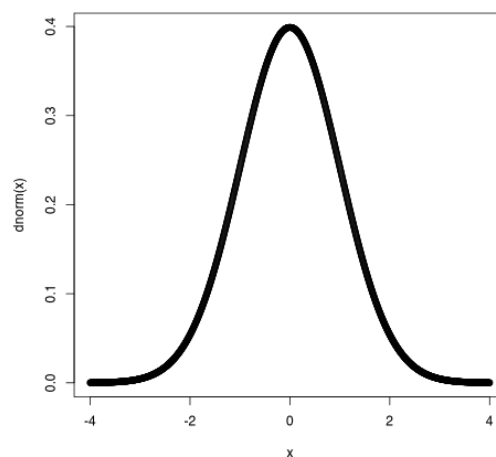


Abb. 3.1

```
1 # etwas hübscher
2 plot(x,dnorm(x), col="blue", type="l", xlab="x", ylab="f(x)",
  main="Standardnormalverteilungen")
```

Der Befehl erzeugt Abbildung 3.2.

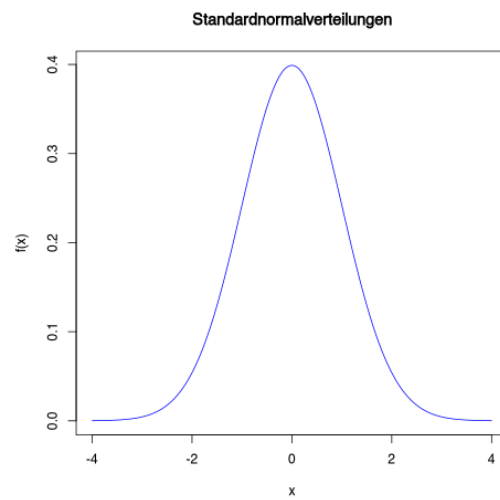


Abb. 3.2

```
1 # mit Mittelwert 2 und sd = 0,5  
2 plot(x,dnorm(x,mean=2,s=0.5), col="darkblue", type="l", xlab="x", ylab="f(x)",  
   main="Normalverteilungen")
```

Der Befehl erzeugt Abbildung 3.3.

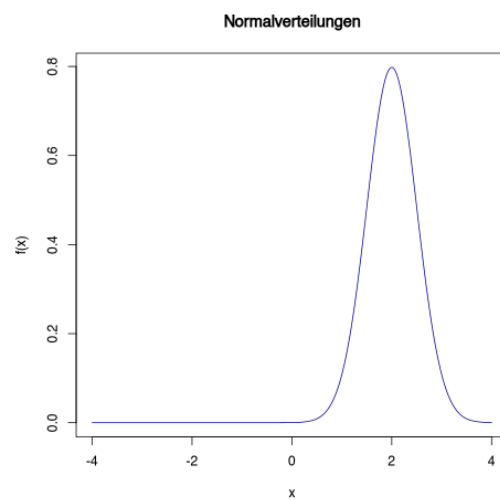


Abb. 3.3

```

1 # mit Mittelwert 2 und sd = 2
2 plot(x,dnorm(x,mean=2,s=2), col="darkorchid", type="l", xlab="x", ylab="f(x)",
  main="Normalverteilungen")

```

Der Befehl erzeugt Abbildung 3.4.

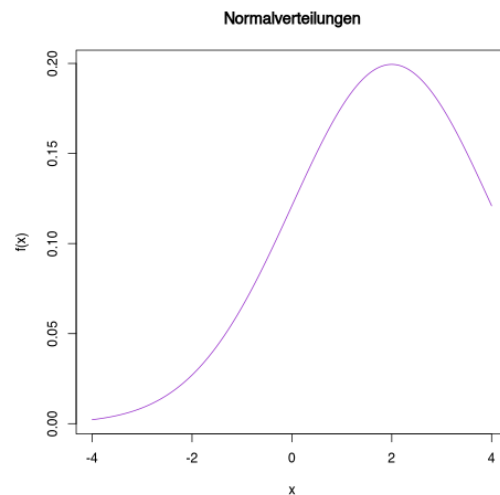


Abb. 3.4

```

1 # erzeuge neue Werte von -3 bis 6
2 x <- seq(-3,6, by=0.005)
3
4 # Alles zusammen plotten
5 plot(x,dnorm(x,mean=2,s=0.5), col="blue", type="l", xlab="x",
  ylab="f(x)",main="Normalverteilungen")
6 lines(x,dnorm(x,mean=0,s=1), col="black")
7 lines(x,dnorm(x,mean=2,s=2), col="darkorchid")
8 text(0, .45, "N(0;1)")
9 text(2.8, 0.6, "N(2;0.5)", col="blue")
10 text(5, 0.1, "N(2;2)", col="darkorchid")

```

Der Befehl erzeugt Abbildung 3.5.

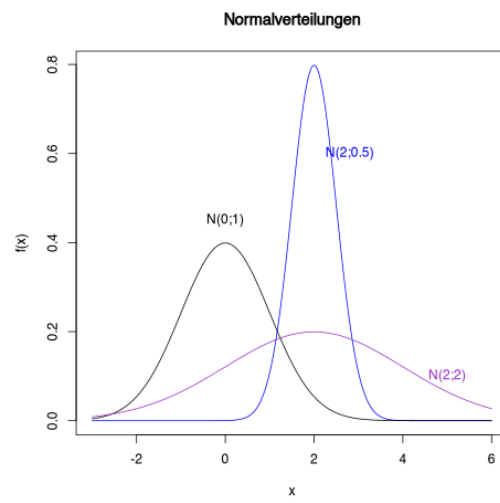


Abb. 3.5

Mit `ggplot` kann es so aussehen

```

1  # aktiviere ggplot
2  library(ggplot2)
3
4  # erzeuge Werte von -3 bis 3
5  x <- seq(-3,3, by=0.005)
6
7  # übergebe in ein data.frame
8  df <- data.frame(x)
9
10 # ggplot erstellen
11 p <- ggplot(data=df, aes(x)) +
12     xlim(-4,4) + ylim(0,0.5) +
13     xlab("x") + ylab("Dichtefunktion") +
14     ggtitle("Normalverteilungen")
15 p + stat_function(fun=dnorm, args=(c(mean=0,sd=1)), colour="black")+
16     annotate(geom="text", x=0, y=0.42, label="N(0;1)", color="black")

```

Der Befehl erzeugt Abbildung 3.6.

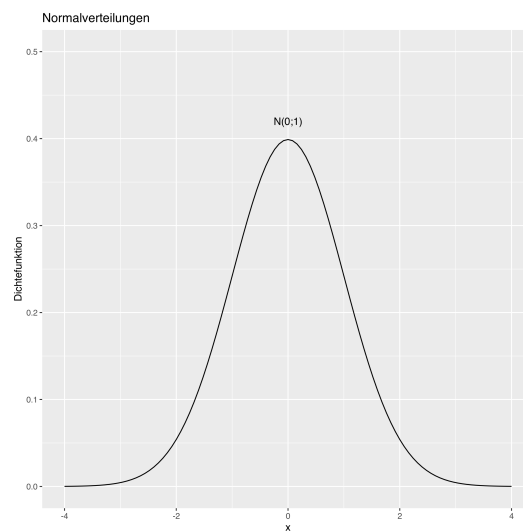


Abb. 3.6

```

1  # erzeuge Werte von -3 bis 6
2  x <- seq(-3,6, by=0.005)
3
4  # übergebe in ein data.frame
5  df <- data.frame(x)
6
7  # ggplot erstellen
8  p <- ggplot(data=df, aes(x)) +
9      xlim(-3,6) + ylim(0, 0.8) +
10     xlab("x") + ylab("Dichtefunktion") +
11     ggtitle("Normalverteilungen")
12  p + stat_function(fun=dnorm, args=(c(mean=0,sd=1)), colour="black")+
13     annotate(geom="text", x=0, y=0.42, label="N(0;1)", color="black")+
14     stat_function(fun=dnorm, args=(c(mean=2,sd=0.5)), colour="blue") +
15     annotate(geom="text", x=3, y=0.6, label="N(2;0.5)", color="blue")+
16     stat_function(fun=dnorm, args=(c(mean=2,sd=2)), colour="darkorchid") +
17     annotate(geom="text", x=5, y=0.1, label="N(2;2)", color="darkorchid")

```

Der Befehl erzeugt Abbildung 3.7.

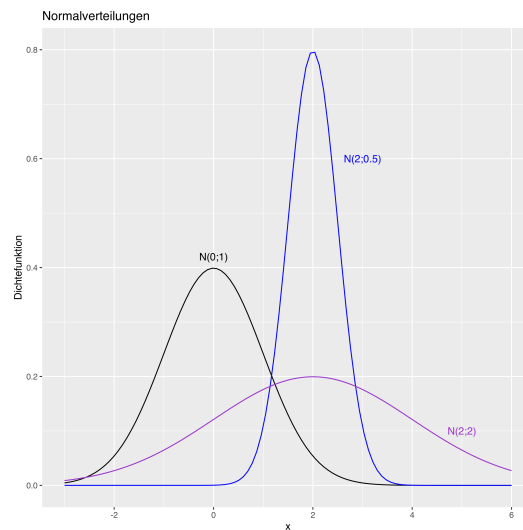


Abb. 3.7

Die Bereiche der Standardabweichung können mit `geom_area()` visualisiert werden.

```
1 # erzeuge X-Werte
2 df <- data.frame(x=seq(-3,3, by=0.005))
3 # berechne Y-Werte
4 df$y <- dnorm(df$x)
5 # Setze Cuts
6 df$sd <- cut(df$x, breaks = c(-Inf, 1 *(-2:2), Inf))
7
8 # plotte als area
9 ggplot(df, aes(x, y, fill=sd)) + geom_area()
```

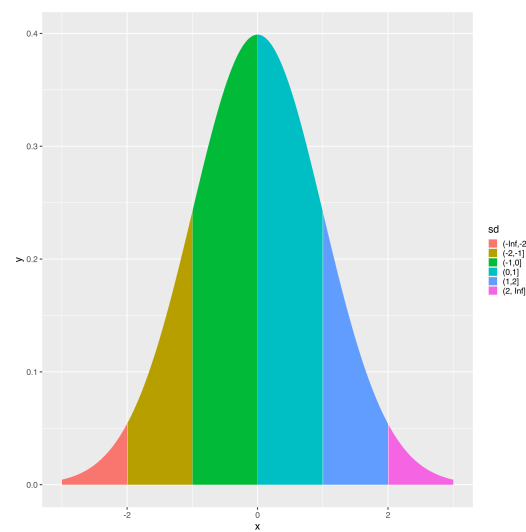


Abb. 3.8

Erzeugen wir den Bereich von 1 Standardabweichung.

```

1 df    <- data.frame(x=seq(-3,3, by=0.005))
2 df$y  <- dnorm(df$x)
3 df$sd <- cut(df$x, breaks = c(-1,1))
4 df <- rbind(df, data.frame(x=c(-1, 1), y=c(0,0),sd=c(-1,1)))
5 df$sd <- forcats::fct_explicit_na(df$sd, na_level="x")
6
7 ggplot(df, aes(x, y, fill=sd)) +
8   ggtitle("Standardnormalverteilung", subtitle = "Bereich von 1 Standardabweichung") +
9   geom_area() + theme(legend.position = "none") +
10  geom_vline(xintercept=0, linetype="dotted")+
11  geom_vline(xintercept=-1, linetype="dashed", col="blue", size=1)+
12  geom_vline(xintercept=1, linetype="dashed", col="blue", size=1)+
13  scale_fill_manual(name=c("x", "(-1,1]"), values=c("skyblue", "snow3"))

```

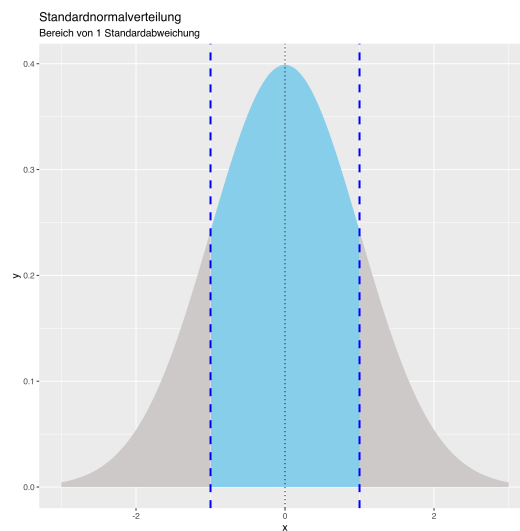


Abb. 3.9

Nun erzeugen wir den Bereich von 2 Standardabweichungen.

```

1 df <- data.frame(x=seq(-3,3, by=0.005))
2 df$y <- dnorm(df$x)
3 df$sd <- cut(df$x, breaks = c(-2,2))
4 df <- rbind(df, data.frame(x=c(-2, 2), y=c(0,0),sd=c(-2,2)))
5 df$sd <- forcats::fct_explicit_na(df$sd, na_level="x")
6
7 ggplot(df, aes(x, y, fill=sd)) +
8   ggtitle("Standardnormalverteilung", subtitle = "Bereich von 2 Standardabweichung") +
9   geom_area() + theme(legend.position = "none") +
10  geom_vline(xintercept=0, linetype="dotted")+
11  geom_vline(xintercept=-2, linetype="dashed", col="purple3", size=1)+
12  geom_vline(xintercept=2, linetype="dashed", col="purple3", size=1)+
13  scale_fill_manual(name=c("x", "(-2,2]"), values=c("purple3", "snow3"))

```

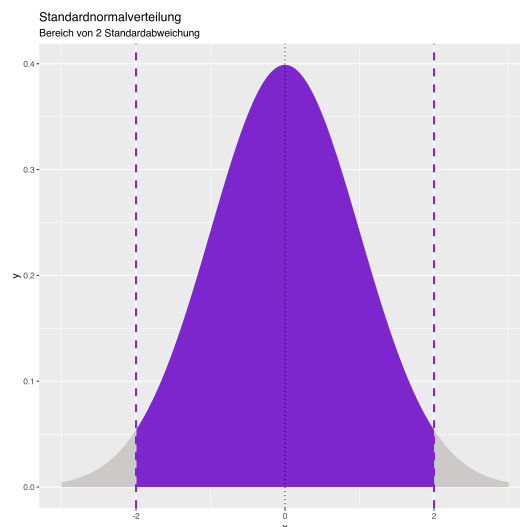


Abb. 3.10

Erzeugen wir die Fläche, in der 95% der Werte liegen, also von 1,96 Standardabweichungen.

```

1 df <- data.frame(x=seq(-3,3, by=0.005))
2 df$y <- dnorm(df$x)
3 df$sd <- cut(df$x, breaks = c(-1.96, 1.96))
4 df <- rbind(df, data.frame(x=c(-1.96, 1.96), y=c(0,0),sd=c(-1.96,1.96)))
5 df$sd <- forcats::fct_explicit_na(df$sd, na_level="x")
6
7 ggplot(df, aes(x, y, fill=sd)) +
8   ggtitle("Standardnormalverteilung", subtitle = "95% der Werte bei 1,96 sd") +
9   geom_area() + theme(legend.position = "none") +
10  geom_vline(xintercept=0, linetype="dotted")+
11  geom_vline(xintercept=-2, linetype="dotted", col="purple3", size=0.5)+
12  geom_vline(xintercept=2, linetype="dotted", col="purple3", size=0.5)+
13  geom_vline(xintercept=-1.96, col="chartreuse4", size=0.5)+
14  geom_vline(xintercept=1.96, col="chartreuse4", size=0.5)+
15  scale_fill_manual(name=c("x", "(-1.96,1.96]"), values=c("seagreen3", "snow3"))

```

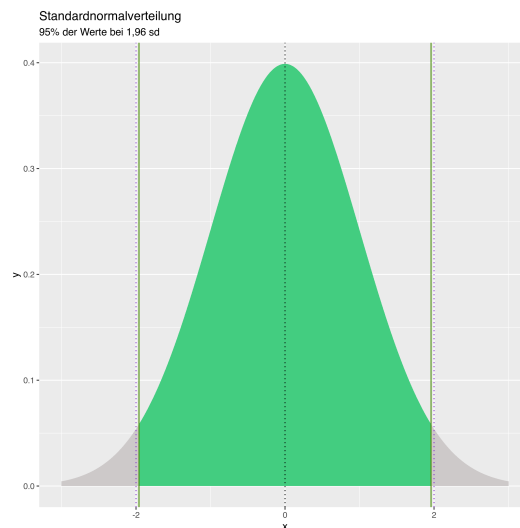


Abb. 3.11

3.2. DIAGRAMME PLOTTEN

Dieser Code kann für die Erzeugung der Grafiken von Tabelle 3.3 und 3.4 verwendet werden.

```
1 df    <- data.frame(x=seq(-3,3, by=0.005))
2 df$y  <- dnorm(df$x)
3 df$sd <- "B"
4 df$sd[df$x < 1.4] <- "A"
5
6 ggplot(df, aes(x, y, fill=sd)) +
7   geom_area() + theme(legend.position = "none") +
8   geom_vline(xintercept=0, linetype="dotted")+
9   geom_vline(xintercept=1.4, col="blue", size=0.5)+
10  ylab("") + scale_fill_manual(values=c("skyblue", "snow3"))
```

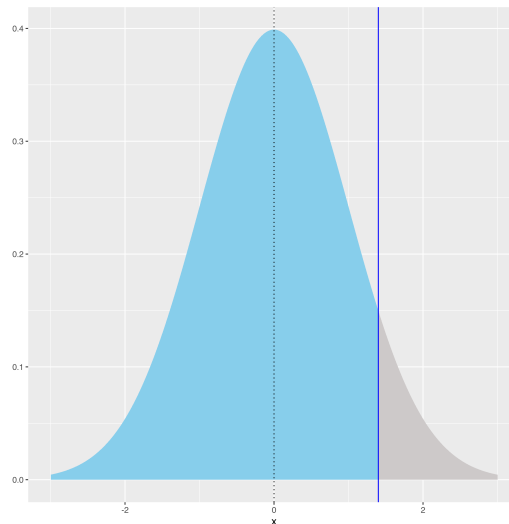


Abb. 3.12

```
1 df    <- data.frame(x=seq(-3,3, by=0.005))
2 df$y  <- dnorm(df$x)
3 df$sd <- "B"
4 df$sd[df$x < 1.4] <- "A"
5
6 ggplot(df, aes(x, y, fill=sd)) +
7   geom_area() + theme(legend.position = "none") +
8   geom_vline(xintercept=0, linetype="dotted")+
9   geom_vline(xintercept=1.4, col="blue", size=0.5)+
10  ylab("") + scale_fill_manual(values=c("snow3", "skyblue"))
```

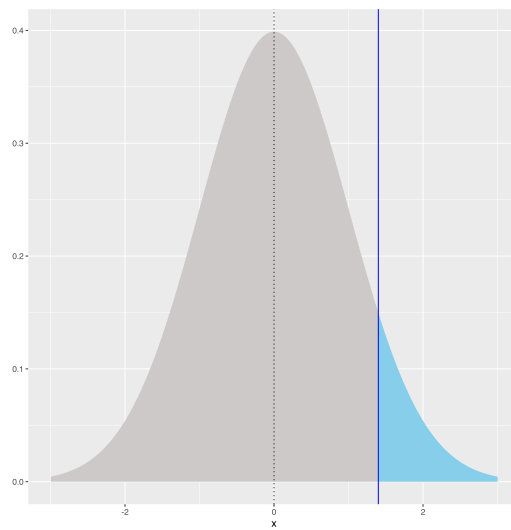


Abb. 3.13

3.2.2 t-Verteilung

Die t-Verteilung kann mit `ggplot` geplottet werden.

```

1  # Erzeuge x-werte
2  df <- data.frame(x=seq(-3,3, by=0.005))
3
4  # Grundlegende Plotangaben
5  p <- ggplot(data=df, aes(x)) +
6    # begrenze die Achsen
7    xlim(-3,3) + ylim(0, 0.4) +
8    # Achsen-Titel
9    xlab("x") + ylab("Dichtefunktion") +
10   # Plot-Titel
11   ggtitle("t-Verteilungen", subtitle = "nach Freiheitsgraden")
12
13   # t-Verteilung plotten
14   p +
15   stat_function(fun=dt, args=list(df=1), col="black") +
16   # Textfeld hinzufügen
17   annotate(geom="text", x=0, y=0.25, label="df=1", color="black")

```


3.2. DIAGRAMME PLOTTEN

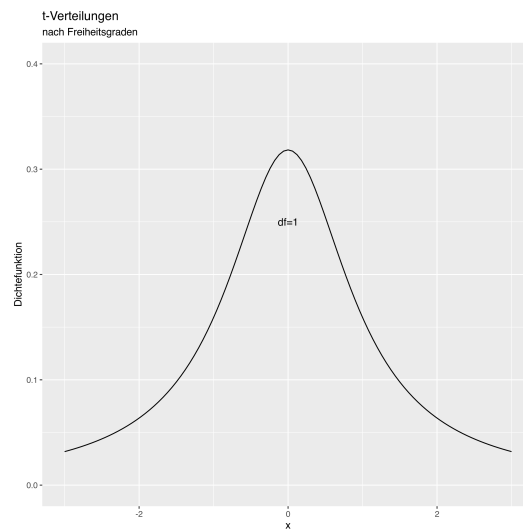


Abb. 3.14

Dem Plott können weitere Freiheitsgrade hinzugefügt werden.

```
1 # t-Verteilungen plotten
2 p +
3   stat_function(fun=dt, args=list(df=1), col="black") +
4   annotate(geom="text", x=0, y=0.25, label="df=1", color="black")+
5   stat_function(fun=dt, args=list(df=2), col="blue") +
6   annotate(geom="text", x=0, y=0.37, label="df=2", color="blue")
```

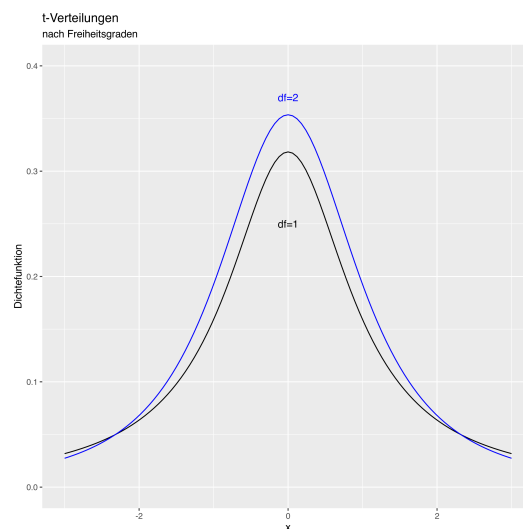


Abb. 3.15

Wenn die t-Werte bereits berechnet wurden, kann eine alternative Vorgehensweise so aussehen:

```

1  # berechne t-Werte für Freiheitsgrade 1 bis 5
2  x=seq(-3,3, by=0.005)
3  df <- data.frame(
4    x,
5    df1 = dt(x,df=1),
6    df2 = dt(x,df=2),
7    df3 = dt(x,df=3),
8    df4 = dt(x,df=4),
9    df5 = dt(x,df=5)
10 )
11
12 # wandle ins Format "long table" um
13 df <- pivot_longer(df, cols=c(df1, df2, df3, df4, df5))
14
15 # grundlegende Ploteinstellungen
16 p <- ggplot(data=df, aes(x,value)) +
17   xlim(-3,3) + ylim(0, 0.4) +
18   xlab("x") + ylab("Dichtefunktion") +
19   ggtitle("t-Verteilungen", subtitle = "nach Freiheitsgraden")
20
21 # plote t-Verteilungen
22 p + geom_line(aes(col=name))+
23   labs(col="Freiheitsgrade")

```

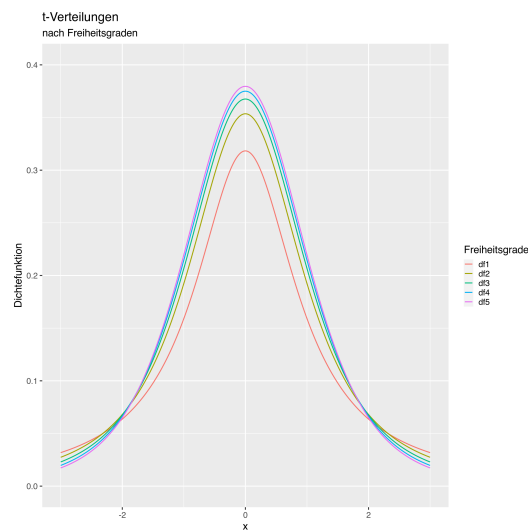


Abb. 3.16

3.2.3 χ^2 -Verteilung

Die χ^2 -Verteilung kann mit `ggplot` geplottet werden.

```

1  # Erzeuge x-werte
2  x=seq(0,25, by=0.005)
3  df <- data.frame( x,
4    df01 = dchisq(x, df=1),
5    df05 = dchisq(x, df=5),
6    df10 = dchisq(x, df=10),
7    df15 = dchisq(x, df=15)
8  )
9
10 # erzeuge long-table
11 df <- pivot_longer(df, cols=c(df01, df05, df10, df15))
12
13 p <- ggplot(data=df, aes(x, value, fill=name)) +
14   xlim(0,25) + ylim(0, 0.2) +
15   xlab("x") + ylab("Dichtefunktion") +
16   ggtitle("Chi^2-Verteilungen", subtitle = "nach Freiheitsgraden")
17 p + geom_line(aes(col=name, linetype=name))+
18   labs(col="Freiheitsgrade", linetype="")

```

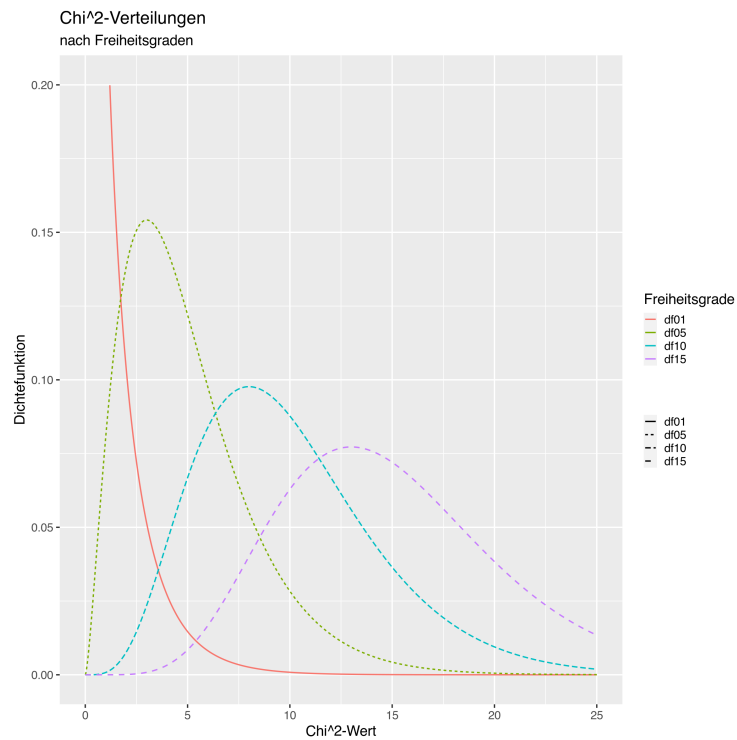


Abb. 3.17

Die Fläche unterhalb der Kurve kann mit `geom_area()` erzeugt werden. Für `df=1` lautet der Aufruf:

```
1 # erzeuge Dummy-Werte
2 x=seq(0,25, by=0.005)
3 # überführe in Datenframe
4 df <- data.frame( x, df01 = dchisq(x, df=1))
5 ggplot(data=df, aes(x, df01)) +
6   xlim(0,25) + coord_cartesian(ylim=c(0, 0.2)) +
7   xlab("Chi^2-Wert") + ylab("Dichtefunktion") +
8   ggtitle("Chi^2-Verteilung", subtitle = "mit 1 Freiheitsgrad") +
9   geom_line(col="#F8766D") + geom_area(fill="skyblue")
```

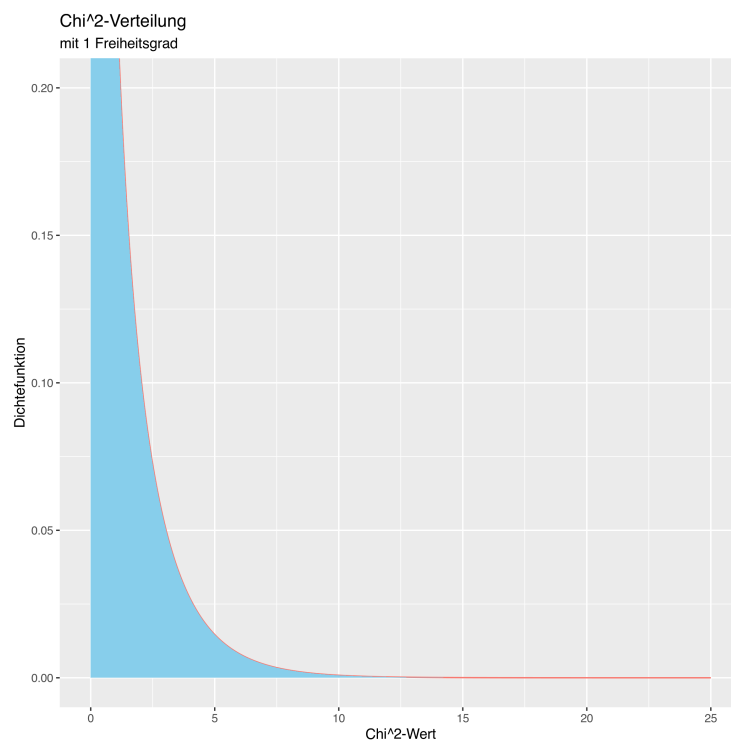


Abb. 3.18

3.2. DIAGRAMME PLOTTEN

Ähnlich wie bei der Normalverteilung können wir die Fläche für bestimmte χ^2 -Werte einfärben. Bei einem Freiheitsgrad und $\alpha = 0,05$ ergibt sich ein kritischer χ^2 -Wert von 3,84. Der Flächenanteil unterhalb dieses Wertes lässt sich wie folgt darstellen:

```
1 # Dummy-Werte erzeugen
2 df <- data.frame(x=seq(0.005,25, by=0.005))
3 # Chi^2-Werte für df=1 erstellen
4 df$y <- dchisq(df$x, df=1)
5 # Grenze bei 3.84 einziehen
6 df$sd <- cut(df$x, breaks = c(0.00, 3.84))
7 ggplot(df, aes(x, y, fill=sd)) +
8   geom_area() + theme(legend.position = "none") +
9   geom_line(aes(col="#F8766D")) +
10  xlim(0,25) + coord_cartesian(ylim=c(0, 0.2)) +
11  xlab("Chi^2-Wert") + ylab("Dichtefunktion") +
12  ggtitle("Chi^2-Verteilung", subtitle = "mit 1 Freiheitsgrad")+
13  geom_vline(xintercept=3.84, col="blue", size=0.5, linetype="dashed")+
14  ylab("") + scale_fill_manual(values=c("skyblue", "snow3")) +
15  annotate(geom="text", x=2, y=0.028, size=8,label="95%", color="blue")
```

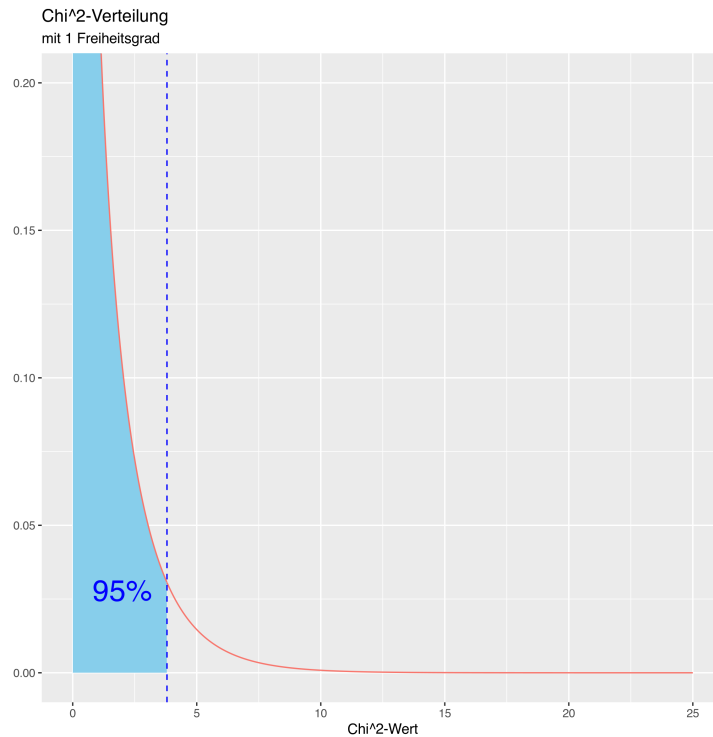


Abb. 3.19

3.2.4 Poisson-Verteilung

Die Poisson-Verteilung kann mit `ggplot` geplottet werden.

```

1  # Erzeuge x-Werte
2  x=seq(0,25)
3  df <- data.frame( x,
4                    l1 = dpois(x, 1),
5                    l2 = dpois(x, 2),
6                    l4 = dpois(x, 4),
7                    l9 = dpois(x, 9)
8  )
9
10 # erzeuge eine long-table
11 df <- pivot_longer(df, cols=c(l1, l2, l4, l9))
12
13 # plot vorbereiten
14 p <- ggplot(data=df, aes(x, value, fill=name)) +
15   xlim(0,17) + ylim(0, 0.4) +
16   xlab("x") + ylab("Dichtefunktion") +
17   ggtitle("Poisson-Verteilungen", subtitle = "nach Lambda")
18 p + geom_line(aes(col=name, linetype=name))+
19   labs(col="Lambda", linetype="")

```

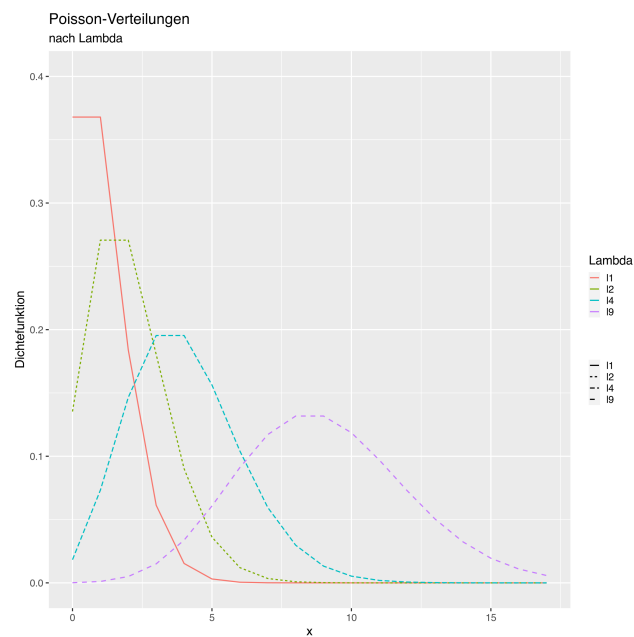


Abb. 3.20

3.2. DIAGRAMME PLOTTEN

Da wir die Plot-Grundlagen in `p` gespeichert haben, können wir ergänzen:

```
1 p + geom_bar(aes(col=name, linetype=name), stat="identity", position="dodge")+  
2     geom_line(aes(col=name, linetype=name))+  
3     labs(col="Lambda", linetype="")
```

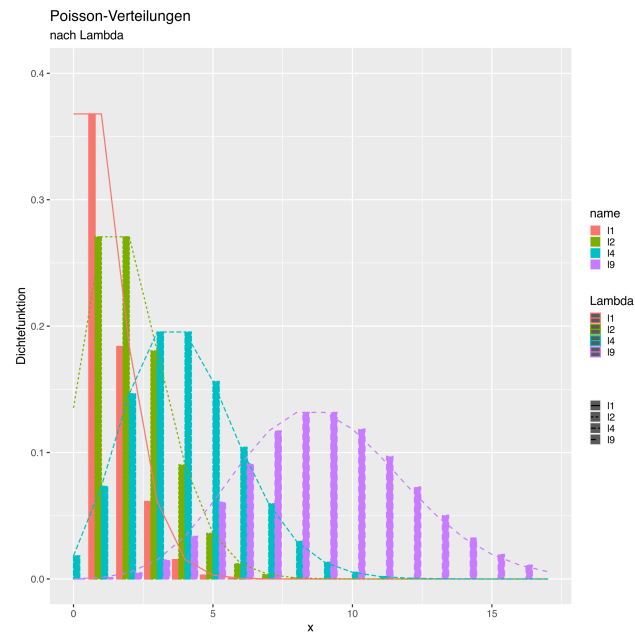


Abb. 3.21

3.2. DIAGRAMME PLOTTEN

Möchte man einen Polygonzug, müssen als Ausgangspunkt die Koordinaten (0,0) festgelegt werden.

```
1 x=seq(0,25)#, by=0.005)
2 df <- data.frame( x=c(0,x),
3                   l1 = c(0, dpois(x, 1)),
4                   l2 = c(0, dpois(x, 2)),
5                   l4 = c(0, dpois(x, 4)),
6                   l9 = c(0, dpois(x, 9))
7 )
8 df <- pivot_longer(df, cols=c(l1, l2, l4, l9))
9
10 p <- ggplot(data=df, aes(x, value, fill=name)) +
11   xlim(0,17) + ylim(0, 0.4) +
12   xlab("x") + ylab("Dichtefunktion") +
13   ggtitle("Poisson-Verteilungen", subtitle = "nach Lambda")
14 p + geom_polygon(aes(col=name,linetype=name))+
15   geom_line(aes(col=name),linetype="dashed")+
16   labs(col="Lambda", fill="Lambda", linetype="Lambda")
```

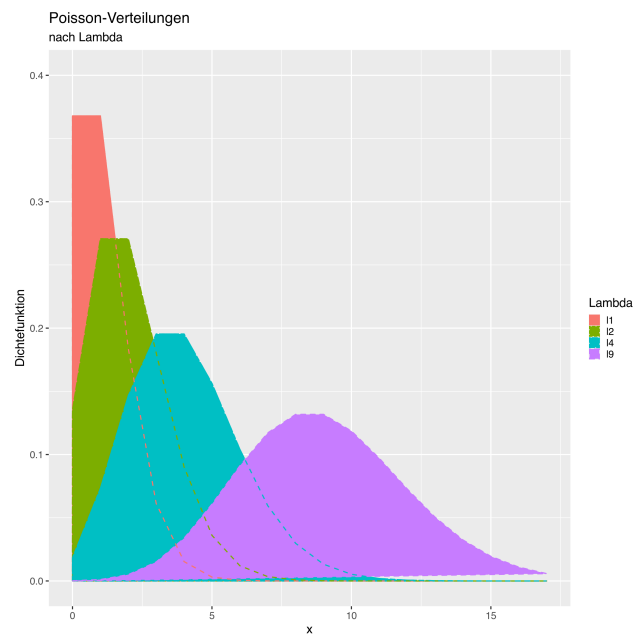


Abb. 3.22

3.3 Pakete verwalten

Ich möchte Zusatzpakete verwalten.

3.3.1 Pakete beim Start automatisch laden

Ich möchte, dass Pakete direkt beim Start von R geladen werden.

Legen Sie hierzu in Ihrem Arbeitsverzeichnis die Datei `.Rprofile` an. In diese Datei können Sie alle Befehle schreiben, die beim Start ausgeführt werden sollen. Das geht auch direkt in  :

```
1 file.edit(".Rprofile")
2
3 # Inhalt von .Rprofile
4 print("Willkommen zurück!")
5 print("Ich lade Paket tidyverse")
6 library(tidyverse)
```

3.4 RMarkdown

Ich möchte RMarkdown individualisieren.

3.4.1 Ausgabeformat per .css-Datei ändern

Ich möchte die Ausgabe der `html`-Datei ändern.

Hierfür kann eine `css`-Datei angelegt werden. Im Kopf des RMarkdown-Dokumentes wird die Datei `MeinStyle.css` wie folgt eingebunden:

```
1 ---
2 title: "Test"
3 output:
4   html_document:
5     css: MeinStyle.css
6 ---
```

In der `css`-Datei können dann wie gewohnt Style-Angaben gemacht werden, z.B. das Aussehen der Codebox. Diese sind über die Parameter `class.source` (R-Befehle) und `class.output` (R-Ausgabe) referenzierbar. In der `CSS`-Datei definieren wir unsere eigene Klasse `“meine”` und hinterlegen gewünschte Parameter.

```
1 .meine {
2   background-color: rgb(100, 0, 0);
3 }
```

Auf diese Klasse können wir in den Chunks verweisen mit

```
1 ```{r class.output="meine"}
2 # unser R Code
3 ```
```

Möchte man alle Chunks automatisch auf `class.output="meine"` setzen, kann dies im `YAML`-Kopfbereich wie folgt angegeben werden:

```
1 knitr::opts_chunk$set(echo = TRUE, class.output="meine")
```

Es gibt auch vordefinierte Klassen für die Chunks, probieren Sie `bg-primary`, `bg-success`, `bg-info`, `bg-warning`, und `bg-danger` aus.

3.5 quarto

Ich möchte `quarto` individualisieren.

3.5.1 eigene $\text{\LaTeX}$ -Vorlagen erstellen

Quarto ermöglicht es, zusätzlichen $\text{\LaTeX}$ -Code einzufügen¹ oder nur einzelne Teile des Templates auszutauschen².

Es können aber auch eigene vollständige $\text{\LaTeX}$ -Vorlagen erzeugt werden.

Folgende Punkte sind dabei zu beachten:

- Möchten Sie alle Funktionen von `quarto` verfügbar und Ihre Vorlage so flexibel wie möglich machen, sollten Sie die `quarto`-Vorlage kopieren³ und Ihre Änderungen direkt dort einfügen.
- Möchten Sie sich z.B. eine Briefvorlage erstellen, deren wesentlichen Parameter (Dokumentenklasse, Schriftgröße, Geometry, etc.) sich eh nie ändern werden, ist es durchaus legitim, eine eigene "rudimentäre" $\text{\LaTeX}$ -Datei zu erstellen.

3.5.1.1 .tex-Date

Ich habe mir einen zentralen Ordner erstellt (z.B. `Dokumente/Vorlagen/quarto`), in welchem ich alle Vorlagendateien ablege. So kann ich sie später bequem in jeder `.qmd`-Datei einbinden.

Die $\text{\LaTeX}$ -Vorlage sollte mindestens folgende Werte enthalten, damit die rudimentären `quarto`-Funktionen unterstützt werden.

```

1 \documentclass[a4paper, 10pt]{scrartcl} % Dokumentenklasse wählen
2 \usepackage[utf8]{inputenc}
3 \usepackage[ngerman]{babel}           % deutsche Sprache hardcoded
4 \usepackage{lmodern}
5 \usepackage[numbered]{bookmark}
6 \usepackage{graphicx}
7 \usepackage{longtable}
8 \usepackage{booktabs}

```

¹<https://quarto.org/docs/output-formats/pdf-basics.html>

²<https://quarto.org/docs/journals/templates.html>

³<https://github.com/quarto-dev/quarto-cli/tree/main/src/resources/formats/pdf/pandoc>

```

9 \usepackage{calc}
10 \usepackage{lastpage}
11 \usepackage{geometry}
12 \usepackage{tabularx}
13 \usepackage{colortbl}
14 \usepackage{multirow}
15 \usepackage{float}
16 \usepackage{amsmath}
17 \usepackage{hhline}
18 \usepackage{blindtext}
19 \usepackage{xcolor}
20 \usepackage[normalem]{ulem}
21
22 \definecolor{linkblue}{RGB}{17, 40, 73}
23
24 \hypersetup{
25 colorlinks,
26 linkcolor=linkblue,
27 urlcolor=blue,
28 bookmarks=true,
29 plainpages=false,
30 hypertexnames=true,
31 pdftitle={\title$},
32 pdfauthor={\author$}
33 }
34
35 \providecommand{\tightlist}{%
36 \setlength{\itemsep}{0pt}\setlength{\parskip}{0pt}}
37
38 %% Callout-Bowen
39 \makeatletter
40 \@ifpackageloaded{tcolorbox}{\usepackage[many]{tcolorbox}}
41 \@ifpackageloaded{fontawesome5}{\usepackage{fontawesome5}}
42 \definecolor{quarto-callout-color}{HTML}{909090}
43 \definecolor{quarto-callout-note-color}{HTML}{0758E5}
44 \definecolor{quarto-callout-important-color}{HTML}{CC1914}
45 \definecolor{quarto-callout-warning-color}{HTML}{EB9113}
46 \definecolor{quarto-callout-tip-color}{HTML}{00A047}
47 \definecolor{quarto-callout-caution-color}{HTML}{FC5300}
48 \definecolor{quarto-callout-color-frame}{HTML}{acacac}
49 \definecolor{quarto-callout-note-color-frame}{HTML}{4582ec}
50 \definecolor{quarto-callout-important-color-frame}{HTML}{d9534f}
51 \definecolor{quarto-callout-warning-color-frame}{HTML}{f0ad4e}
52 \definecolor{quarto-callout-tip-color-frame}{HTML}{02b875}
53 \definecolor{quarto-callout-caution-color-frame}{HTML}{fd7e14}
54 \makeatother
55
56 $if(highlighting-macros)$
57 $highlighting-macros$
58 $endif$
59
60 \begin{document}
61
62 $body$
63

```

```
64 \end{document}
```

Die Variable *body* fügt den Inhalte der *qmd*-Datei (alles unterhalb des YAML-Headers) ein.

3.5.1.2 Vorlage einbinden

Die Vorlage wird wie folgt im YAML-Header eingebunden:

```
1 ---
2 format:
3   pdf:
4     template: /pfad/zur/vorlage.tex
5 ---
```

3.5.1.3 YAML-Variablen

Im YAML-Header können Variablen gesetzt werden, welche in der *.tex*-Datei abgerufen werden können.

```
1 ---
2 title: Meine Vorlage
3 variable1: "Test"
4 variable2: "Noch ein Test"
5
6 format:
7   pdf:
8     template: /pfad/zur/vorlage.tex
9 ---
```

In der *.tex*-Datei sind die oben definierten Variablen verfügbar per *\$variable1\$* und *\$variable2\$*.

```
1 \begin{document}
2
3 $title$
4
5 $body$
6
7 Dies ist ein $variable1$.
8 Und dies ist $variable2$.
9
10 \end{document}
```

Eine Beispiel für die oben erwähnte Briefvorlage finden Sie unter <https://www.produnis.de/blog/post/quarto-briefvorlage/>.

3.5.2 quarto am Handy oder Tablet

Am Handy oder am Tablet sind `quarto` und $\text{\LaTeX}$ nicht verfügbar. Eine Möglichkeit, dennoch `.qmd`-Dateien zu rendern, besteht darin, das Rendern an einen Server auszulagern. Wenn Sie zu Hause einen kleinen Linux-Heimserver betreiben, könnten Ihnen diese beiden Bots weiterhelfen. Beide setzen voraus, dass `quarto` auf dem Server installiert ist. Ebenfalls sollte $\text{\LaTeX}$ via `quarto install tool tinytex` installiert worden sein, damit eventuell benötigte Pakete automatisch nachgeladen werden.

1. quarto-Bot für Nextcloud

- Der Bot ist ein kleines Shell-Script.
- Der Bot mountet ein Nextcloud-Verzeichnis per Webdav und schaut nach, ob in dem Verzeichnis eine `.qmd`-Datei enthalten ist.
- Sollte dies der Fall sein, führt der Bot den Befehl `quarto render DATEI.qmd` aus.
- Da die erzeugte PDF-Datei im selben Verzeichnis liegt, ist sie über die Nextcloud-Dateiansicht verfügbar und wird ebenfalls an alle Clients synchronisiert.
- <https://www.produnis.de/blog/post/simpler-quarto-bot-fuer-nextcloud/>

2. quarto-matrix-bot

- Der Bot ist in python geschrieben.
- Der Bot lauscht in vorher angegebenen Matrix-Räumen.
- Wird eine `.qmd`-Datei in den Raum gesendet, führt der Bot den Befehl `quarto render DATEI.qmd` aus.
- Wenn eine PDF-Datei erzeugt wurde, sendet der Bot diese Datei in den Matrix-Raum.
- Sollte das Rendern fehlschlagen, postet der Bot die Fehlermeldungen in den Raum.
- <https://github.com/rgomez90/matrix-bot>

3.6 Referenztabellen erstellen

Ich möchte Referenztabellen erstellen und benötige die Werte.

3.6.1 Fallzahlen nach Effektstärke und Power

$\alpha = 0.05$															
Power	Effektstärke														
	0.1	0.15	0.2	0.25	0.3	0.35	0.4	0.45	0.5	0.55	0.6	0.65	0.7	0.75	0.8
0.6	981	436	246	158	110	81	62	49	40	33	28	24	21	18	16
0.7	1235	550	310	198	138	102	78	62	50	42	35	30	26	23	20
0.8	1571	699	393	252	175	129	99	78	64	53	45	38	33	29	26
0.9	2102	935	526	337	234	173	132	105	85	70	59	51	44	38	34
0.95	2600	1156	651	417	290	213	163	129	105	87	73	62	54	47	42
0.99	3675	1634	920	589	409	301	231	182	148	122	103	88	76	66	58

Tabelle 3.1: Anzahl Probanden pro Gruppe nach Power und Effektstärke bei $\alpha = 0,05$

$\alpha = 0.01$															
Power	Effektstärke														
	0.1	0.15	0.2	0.25	0.3	0.35	0.4	0.45	0.5	0.55	0.6	0.65	0.7	0.75	0.8
0.6	1603	713	402	258	180	132	102	81	66	55	46	40	34	30	27
0.7	1924	856	482	309	215	159	122	97	79	65	55	47	41	36	32
0.8	2337	1040	586	375	261	192	148	117	95	79	67	57	49	43	38
0.9	2978	1324	746	478	332	245	188	149	121	100	84	72	62	55	48
0.95	3564	1585	892	572	398	293	224	178	144	119	101	86	74	65	57
0.99	4808	2138	1203	771	536	394	302	239	194	161	135	115	100	87	77

Tabelle 3.2: Anzahl Probanden pro Gruppe nach Power und Effektstärke bei $\alpha = 0,01$

Der Quellcode für die Werte in Tabelle 3.1 und 3.2 ist etwas komplizierter, da wir zunächst eine Funktion programmieren müssen, um die Tabelle ausgeben zu können.

```

1  # lade Paket zur Poweranalyse
2  library(pwr)
3  # erzeuge Hilfsfunktion, die nur die errechnete Fallzahl ausgibt
4  #-----
5  powertabelle <- function(alpha, d, power) { round(pwr::pwr.t.test(d = d, sig.level =
6  alpha, power = power, type = "two.sample", alternative = "two.sided")$n)}
7  # Ende der Funktion
8

```

3.6. REFERENZTABELLEN ERSTELLEN

```
9  # ein Datenframe aus allen gewünschten Kombinationen
10 # von power, alpha und d
11 scen <- expand.grid( alpha = c(0.05, 0.01),
12                     d = seq(from = 0.1, to = 0.8, by = 0.05),
13                     power = c(seq(0.6, 0.9, 0.1), 0.95, 0.99)
14                     )
15
16 # füge Datenframe "scen" eine neue Spalte "n" hinzu
17 # per apply() wird auf jede Datenreihe die Funktion "powertabelle" angewendet.
18 # line() hilft dabei, die richtigen Werte aus dem
19 # Datenframe "scen" an die Funktion "powertabelle" zu übergeben
20 scen$n <- apply(scen, 1, function(line) powertabelle(line["alpha"], line["d"],
21 line["power"]))
22
23 # zeige die ersten 20 Datenreihen an
head(scen, 20)
```

	alpha	d	power	n
1	0.05	0.10	0.6	981
2	0.01	0.10	0.6	1603
3	0.05	0.15	0.6	436
4	0.01	0.15	0.6	713
5	0.05	0.20	0.6	246
6	0.01	0.20	0.6	402
7	0.05	0.25	0.6	158
8	0.01	0.25	0.6	258
9	0.05	0.30	0.6	110
10	0.01	0.30	0.6	180
11	0.05	0.35	0.6	81
12	0.01	0.35	0.6	132
13	0.05	0.40	0.6	62
14	0.01	0.40	0.6	102
15	0.05	0.45	0.6	49
16	0.01	0.45	0.6	81
17	0.05	0.50	0.6	40
18	0.01	0.50	0.6	66
19	0.05	0.55	0.6	33
20	0.01	0.55	0.6	55

Jetzt können Teilmengen von `scen` erstellt werden, um die Werte entsprechend der gewünschten Tabellen ($\alpha = 0.05$ und $\alpha = 0.01$) zu erzeugen.

```
1  # erstelle ein subset für "alpha=0.05"
2  subset05 <- subset(scen, alpha == .05)
3
4  # benenne die Spalten um
5  # und bringe per "reshape" die Tabelle ins "wide"-Format
6  tabelle05 <- reshape(subset05, v.names = "n", timevar = "d", idvar = "power", direction
7  = "wide")
```


3.6. REFERENZTABELLEN ERSTELLEN

```
8 # gib tabelle aus
9 print(tabelle05)
```

	alpha	power	n.0.1	n.0.15	n.0.2	n.0.25	n.0.3	n.0.35	n.0.4	n.0.45	n.0.5	n.0.55	n.0.6	n.0.65	n.0.7	n.0.75	n.0.8
1	0.05	0.60	981	436	246	158	110	81	62	49	40	33	28	24	21	18	16
31	0.05	0.70	1235	550	310	198	138	102	78	62	50	42	35	30	26	23	20
61	0.05	0.80	1571	699	393	252	175	129	99	78	64	53	45	38	33	29	26
91	0.05	0.90	2102	935	526	337	234	173	132	105	85	70	59	51	44	38	34
121	0.05	0.95	2600	1156	651	417	290	213	163	129	105	87	73	62	54	47	42
151	0.05	0.99	3675	1634	920	589	409	301	231	182	148	122	103	88	76	66	58

```
1 # erstelle ein subset für "alpha=0.01"
2 subset01 <- subset(scen, alpha == .01)
3
4 # benenne die Spalten um
5 # und bringe per "reshape" die Tabelle ins "wide"-Format
6 tabelle01 <- reshape(subset01, v.names = "n", timevar = "d", idvar = "power", direction
7 = "wide")
8
9 # gib tabelle aus
10 print(tabelle01)
```

	alpha	power	n.0.1	n.0.15	n.0.2	n.0.25	n.0.3	n.0.35	n.0.4	n.0.45	n.0.5	n.0.55	n.0.6	n.0.65	n.0.7	n.0.75	n.0.8
2	0.01	0.60	1603	713	402	258	180	132	102	81	66	55	46	40	34	30	27
32	0.01	0.70	1924	856	482	309	215	159	122	97	79	65	55	47	41	36	32
62	0.01	0.80	2337	1040	586	375	261	192	148	117	95	79	67	57	49	43	38
92	0.01	0.90	2978	1324	746	478	332	245	188	149	121	100	84	72	62	55	48
122	0.01	0.95	3564	1585	892	572	398	293	224	178	144	119	101	86	74	65	57
152	0.01	0.99	4808	2138	1203	771	536	394	302	239	194	161	135	115	100	87	77

Wenn Sie den Abschnitt über *Tidy Data* auf Seite 147 gelesen haben, ist Ihnen evtl. aufgefallen, dass wir hier eine `long table` in eine `wide table` umgeformt haben. Das erwähne ich deshalb, da wir sonst immer genau das Gegenteil machen, nämlich `wide table` ins Format `long table` zu überführen.

Übrigens, dieses Buch ist mit $\text{\LaTeX}$ erstellt worden, und ich hatte keine Lust, die Werte für die Tabellen von Hand in $\text{\LaTeX}$ zu übertragen.

Mit der Funktion `xtable()` aus dem Zusatzpaket `xtable` kann die Ausgabe der Tabelle in $\text{\LaTeX}$ -Quellcode erfolgen:

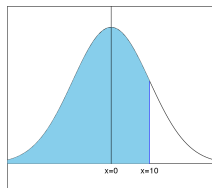
```
1 # lade Paket "xtable"
2 library(xtable)
3
4 # gib die Tabelle als LaTeX-Code aus
5 xtable(tabelle01)
```

[illegible]

3.6.2 Verteilungsfunktion und Quantile der Standardnormalverteilung

ϕ z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	.6103	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224
0.6	.7257	.7291	.7324	.7357	.7389	.7422	.7454	.7486	.7517	.7549
0.7	.7580	.7611	.7642	.7673	.7704	.7734	.7764	.7794	.7823	.7852
0.8	.7881	.7910	.7939	.7967	.7995	.8023	.8051	.8078	.8106	.8133
0.9	.8159	.8186	.8212	.8238	.8264	.8289	.8315	.8340	.8365	.8389
1.0	.8413	.8438	.8461	.8485	.8508	.8531	.8554	.8577	.8599	.8621
1.1	.8643	.8665	.8686	.8708	.8729	.8749	.8770	.8790	.8810	.8830
1.2	.8849	.8869	.8888	.8907	.8925	.8944	.8962	.8980	.8997	.9015
1.3	.9032	.9049	.9066	.9082	.9099	.9115	.9131	.9147	.9162	.9177
1.4	.9192	.9207	.9222	.9236	.9251	.9265	.9279	.9292	.9306	.9319
1.5	.9332	.9345	.9357	.9370	.9382	.9394	.9406	.9418	.9429	.9441
1.6	.9452	.9463	.9474	.9484	.9495	.9505	.9515	.9525	.9535	.9545
1.7	.9554	.9564	.9573	.9582	.9591	.9599	.9608	.9616	.9625	.9633
1.8	.9641	.9649	.9656	.9664	.9671	.9678	.9686	.9693	.9699	.9706
1.9	.9713	.9719	.9726	.9732	.9738	.9744	.9750	.9756	.9761	.9767
2.0	.9772	.9778	.9783	.9788	.9793	.9798	.9803	.9808	.9812	.9817
2.1	.9821	.9826	.9830	.9834	.9838	.9842	.9846	.9850	.9854	.9857
2.2	.9861	.9864	.9868	.9871	.9875	.9878	.9881	.9884	.9887	.9890
2.3	.9893	.9896	.9898	.9901	.9904	.9906	.9909	.9911	.9913	.9916
2.4	.9918	.9920	.9922	.9925	.9927	.9929	.9931	.9932	.9934	.9936
2.5	.9938	.9940	.9941	.9943	.9945	.9946	.9948	.9949	.9951	.9952
2.6	.9953	.9955	.9956	.9957	.9959	.9960	.9961	.9962	.9963	.9964
2.7	.9965	.9966	.9967	.9968	.9969	.9970	.9971	.9972	.9973	.9974
2.8	.9974	.9975	.9976	.9977	.9977	.9978	.9979	.9979	.9980	.9981
2.9	.9981	.9982	.9982	.9983	.9984	.9984	.9985	.9985	.9986	.9986
3.0	.9987	.9987	.9987	.9988	.9988	.9989	.9989	.9989	.9990	.9990
p =	0.5000	0.7500	0.8000	0.9000	0.9500	0.9750	0.9900	0.9950	0.9975	0.9990
z_p =	0.0000	0.6745	0.8416	1.2816	1.6449	1.9600	2.3263	2.5758	2.8070	3.0902

Tabelle 3.3: Verteilungsfunktion und Quantile der Standardnormalverteilung



Bei der z-Transformation benötigt man die Werte dieser Tabelle beispielsweise für Fragen wie “Wie hoch ist der prozentuale Anteil für Werte, die **höchstens** X sind”

Die Werte für Tabelle 3.3 werden erzeugt mit:

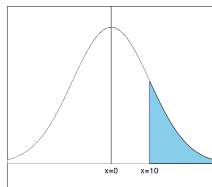
```
1  # erstelle Werte von 0.00 bis 3.09
2  z <- seq(from=0, to=3.09, by = 0.01)
3
4  # P-Werte berechnen
5  pnorm(z)
6
7  # In Matrix mit 10 Spalten übertragen
8  matrix(pnorm(z),ncol=10, byrow=TRUE)
9  # auf 4 Stellen runden
10 matrix(round(pnorm(z),4),ncol=10, byrow=TRUE)
11 # In Dataframe übertragen
12 tabelle1 <- data.frame(matrix(round(pnorm(z),4),ncol=10, byrow=TRUE))
13 # benenne die Spalten
14 colnames(tabelle1) <- c(seq(0.00, 0.09, 0.01))
15 # benenne die Zeilen
16 row.names(tabelle1) <- c(seq(0, 3, 0.1))
17 # gib fertige Tabelle aus
18 tabelle1
```

3.6. REFERENZTABELLEN ERSTELLEN

Im Gegensatz zu Tabelle 3.3 zeigt Tabelle 3.4 die Überschreitungswahrscheinlichkeiten P für Abszissenwerte z der Standardnormalverteilung ($\mu = 0$ und $\sigma = 1$). Die p-Werte entsprechen der Fläche unter der Standardnormalverteilung zwischen z und $+\infty$ und damit einer *einseitigen* Fragestellung. Sie müssen bei *zweiseitigen* Fragestellungen verdoppelt werden!

$\phi \ z$	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.4960	.4920	.4880	.4840	.4801	.4761	.4721	.4681	.4641
0.1	.4602	.4562	.4522	.4483	.4443	.4404	.4364	.4325	.4286	.4247
0.2	.4207	.4168	.4129	.4090	.4052	.4013	.3974	.3936	.3897	.3859
0.3	.3821	.3783	.3745	.3707	.3669	.3632	.3594	.3557	.3520	.3483
0.4	.3446	.3409	.3372	.3336	.3300	.3264	.3228	.3192	.3156	.3121
0.5	.3085	.3050	.3015	.2981	.2946	.2912	.2887	.2843	.2810	.2776
0.6	.2743	.2709	.2676	.2643	.2611	.2578	.2546	.2514	.2483	.2451
0.7	.2420	.2389	.2358	.2327	.2296	.2266	.2236	.2206	.2177	.2148
0.8	.2119	.2090	.2061	.2033	.2005	.1977	.1949	.1922	.1894	.1867
0.9	.1841	.1814	.1788	.1762	.1736	.1711	.1685	.1660	.1635	.1611
1.0	.1587	.1562	.1539	.1515	.1492	.1469	.1446	.1423	.1401	.1379
1.1	.1357	.1335	.1314	.1291	.1271	.1251	.1230	.1210	.1190	.1170
1.2	.1151	.1131	.1112	.1093	.1075	.1056	.1038	.1020	.1003	.0985
1.3	.0968	.0951	.0934	.0918	.0901	.0885	.0869	.0853	.0838	.0823
1.4	.0808	.0793	.0778	.0764	.0749	.0735	.0721	.0708	.0694	.0681
1.5	.0668	.0655	.0643	.0630	.0618	.0606	.0594	.0582	.0571	.0559
1.6	.0548	.0537	.0526	.0516	.0505	.0495	.0485	.0475	.0465	.0455
1.7	.0446	.0436	.0427	.0418	.0409	.0401	.0392	.0384	.0375	.0367
1.8	.0359	.0351	.0344	.0336	.0329	.0322	.0314	.0307	.0301	.0294
1.9	.0287	.0281	.0274	.0268	.0262	.0256	.0250	.0244	.0239	.0233
2.0	.0228	.0222	.0217	.0212	.0207	.0202	.0197	.0192	.0188	.0183
2.1	.0179	.0174	.0170	.0166	.0162	.0158	.0154	.0150	.0146	.0143
2.2	.0139	.0136	.0132	.0129	.0125	.0122	.0119	.0116	.0113	.0110
2.3	.0107	.0104	.0102	.0099	.0096	.0094	.0091	.0089	.0087	.0084
2.4	.0082	.0080	.0078	.0075	.0073	.0071	.0069	.0068	.0066	.0064
2.5	.0062	.0060	.0059	.0057	.0055	.0054	.0052	.0051	.0049	.0048
2.6	.0047	.0045	.0044	.0043	.0041	.0040	.0039	.0038	.0037	.0036
2.7	.0035	.0034	.0033	.0032	.0031	.0030	.0029	.0028	.0027	.0026
2.8	.0026	.0025	.0024	.0023	.0023	.0022	.0021	.0021	.0020	.0019
2.9	.0019	.0018	.0018	.0017	.0016	.0016	.0015	.0015	.0014	.0014
3.0	.0013	.0013	.0013	.0012	.0012	.0011	.0011	.0011	.0010	.0010
$p =$	0.5000	0.7500	0.8000	0.9000	0.9500	0.9750	0.9900	0.9950	0.9975	0.9990
$z_p =$	0.0000	0.6745	0.8416	1.2816	1.6449	1.9600	2.3263	2.5758	2.8070	3.0902

Tabelle 3.4: Überschreitungswahrscheinlichkeiten P für Abszissenwerte z der Standardnormalverteilung



Bei der z-Transformation benötigt man die Werte dieser Tabelle beispielsweise für Fragen wie “Wie hoch ist der prozentuale Anteil für Werte, die **mindestens** X sind”

Die Werte für Tabelle 3.4 werden erzeugt mit:

```
1  # erstelle Werte von 0.00 bis 3.09
2  z <- seq(from=0, to=3.09, by = 0.01)
3
4  # P-Werte berechnen
5  1 - pnorm(z)
6
7  # In Matrix mit 10 Spalten übertragen
8  matrix(1-pnorm(z),ncol=10, byrow=TRUE)
9  # auf 4 Stellen runden
10 matrix(round(1-pnorm(z),4),ncol=10, byrow=TRUE)
11
12 # In Dataframe übertragen
13 tabelle2 <- data.frame(matrix(round(1-pnorm(z),4),ncol=10, byrow=TRUE))
14 # benenne die Spalten
15 colnames(tabelle2) <- c(seq(0.00, 0.09, 0.01))
16 # benenne die Zeilen
17 row.names(tabelle2) <- c(seq(0, 3, 0.1))
18 # gib fertige Tabelle aus
19 tabelle2
```

3.6.3 Kritische Werte t^* der t -Verteilung

df	Irrtumswahrscheinlichkeit $\alpha/2$ für einseitige Fragestellung					
	0.10	0.05	0.025	0.01	0.005	0.0005
df	Irrtumswahrscheinlichkeit α für zweiseitige Fragestellung					
	0.20	0.10	0.050	0.020	0.010	0.0010
1	3.078	6.314	12.706	31.821	63.657	318.309
2	1.886	2.920	4.303	6.965	9.925	22.327
3	1.638	2.353	3.182	4.541	5.841	10.215
4	1.533	2.132	2.776	3.747	4.604	7.173
5	1.476	2.015	2.571	3.365	4.032	5.893
6	1.440	1.943	2.447	3.143	3.707	5.208
7	1.415	1.895	2.365	2.998	3.499	4.785
8	1.397	1.860	2.306	2.896	3.355	4.501
9	1.383	1.833	2.262	2.821	3.250	4.297
10	1.372	1.812	2.228	2.764	3.169	4.144
11	1.363	1.796	2.201	2.718	3.106	4.025
12	1.356	1.782	2.179	2.681	3.055	3.930
13	1.350	1.771	2.160	2.650	3.012	3.852
14	1.345	1.761	2.145	2.624	2.977	3.787
15	1.341	1.753	2.131	2.602	2.947	3.733
16	1.337	1.746	2.120	2.583	2.921	3.686
17	1.333	1.740	2.110	2.567	2.898	3.646
18	1.330	1.734	2.101	2.552	2.878	3.611
19	1.328	1.729	2.093	2.539	2.861	3.579
20	1.325	1.725	2.086	2.528	2.845	3.552
21	1.323	1.721	2.080	2.518	2.831	3.527
22	1.321	1.717	2.074	2.508	2.819	3.505
23	1.319	1.714	2.069	2.500	2.807	3.485
24	1.318	1.711	2.064	2.492	2.797	3.467
25	1.316	1.708	2.060	2.485	2.787	3.450
26	1.315	1.706	2.056	2.479	2.779	3.435
27	1.314	1.703	2.052	2.473	2.771	3.421
28	1.313	1.701	2.048	2.467	2.763	3.408
29	1.311	1.699	2.045	2.462	2.756	3.396
30	1.310	1.697	2.042	2.457	2.750	3.385
40	1.303	1.684	2.021	2.423	2.704	3.307
50	1.299	1.676	2.009	2.403	2.678	3.261
60	1.296	1.671	2.000	2.390	2.660	3.232
70	1.294	1.667	1.994	2.381	2.648	3.211
80	1.292	1.664	1.990	2.374	2.639	3.195
90	1.291	1.662	1.987	2.368	2.632	3.183
100	1.290	1.660	1.984	2.364	2.626	3.174
200	1.286	1.653	1.972	2.345	2.601	3.131
300	1.284	1.650	1.968	2.339	2.592	3.118
400	1.284	1.649	1.966	2.336	2.588	3.111
500	1.283	1.648	1.965	2.334	2.586	3.107
∞	1.282	1.645	1.960	2.326	2.576	3.090

Tabelle 3.5: Kritische Werte t^* der t -Verteilung

3.6. REFERENZTABELLEN ERSTELLEN

Die Werte für Tabelle 3.5 werden erzeugt mit:

```
1  # erstelle Werte von 1 bis 100, und 200, 300, 400, 500
2  df <- c(seq(from=1, to=100, by = 1), 200, 300, 400, 500)
3
4  # für alpha= 0.2
5  qt(0.9,df=df)
6
7  # für alpha= 0.1
8  qt(0.95,df=df)
9
10 # für alpha= 0.05
11 qt(0.975,df=df)
12
13 # für alpha= 0.02
14 qt(0.99,df=df)
15
16 # für alpha= 0.01
17 qt(0.995,df=df)
18
19 # für alpha= 0.001
20 qt(0.9995,df=df)
21
22 # erstelle Datenframe
23 tabelle_t <- data.frame(qt(0.9,df=df), qt(0.95,df=df), qt(0.975,df=df), qt(0.99,df=df),
24 qt(0.995,df=df), qt(0.9995,df=df))
25 # benenne Spalten
26 colnames(tabelle_t) <- c("0.01", "0.05", "0.025", "0.01", "0.005", "0.0005")
27 # benenne Zeilen
28 row.names(tabelle_t) <- df
29 # gib Tabelle aus
tabelle_t
```


3.6.4 Kritische Werte der Chiquadrat-Verteilung

df	Irrtumswahrscheinlichkeit α					
	0.10	0.05	0.025	0.01	0.005	0.0005
1	2.71	3.84	5.02	6.63	7.88	12.12
2	4.61	5.99	7.38	9.21	10.60	15.20
3	6.25	7.81	9.35	11.34	12.84	17.73
4	7.78	9.49	11.14	13.28	14.86	20.00
5	9.24	11.07	12.83	15.09	16.75	22.11
6	10.64	12.59	14.45	16.81	18.55	24.10
7	12.02	14.07	16.01	18.48	20.28	26.02
8	13.36	15.51	17.53	20.09	21.95	27.87
9	14.68	16.92	19.02	21.67	23.59	29.67
10	15.99	18.31	20.48	23.21	25.19	31.42
11	17.28	19.68	21.92	24.72	26.76	33.14
12	18.55	21.03	23.34	26.22	28.30	34.82
13	19.81	22.36	24.74	27.69	29.82	36.48
14	21.06	23.68	26.12	29.14	31.32	38.11
15	22.31	25.00	27.49	30.58	32.80	39.72
16	23.54	26.30	28.85	32.00	34.27	41.31
17	24.77	27.59	30.19	33.41	35.72	42.88
18	25.99	28.87	31.53	34.81	37.16	44.43
19	27.20	30.14	32.85	36.19	38.58	45.97
20	28.41	31.41	34.17	37.57	40.00	47.50
21	29.62	32.67	35.48	38.93	41.40	49.01
22	30.81	33.92	36.78	40.29	42.80	50.51
23	32.01	35.17	38.08	41.64	44.18	52.00
24	33.20	36.42	39.36	42.98	45.56	53.48
25	34.38	37.65	40.65	44.31	46.93	54.95
26	35.56	38.89	41.92	45.64	48.29	56.41
27	36.74	40.11	43.19	46.96	49.64	57.86
28	37.92	41.34	44.46	48.28	50.99	59.30
29	39.09	42.56	45.72	49.59	52.34	60.73
30	40.26	43.77	46.98	50.89	53.67	62.16
31	41.42	44.99	48.23	52.19	55.00	63.58
32	42.58	46.19	49.48	53.49	56.33	65.00
33	43.75	47.40	50.73	54.78	57.65	66.40
34	44.90	48.60	51.97	56.06	58.96	67.80
35	46.06	49.80	53.20	57.34	60.27	69.20
36	47.21	51.00	54.44	58.62	61.58	70.59
37	48.36	52.19	55.67	59.89	62.88	71.97
38	49.51	53.38	56.90	61.16	64.18	73.35
39	50.66	54.57	58.12	62.43	65.48	74.73
40	51.81	55.76	59.34	63.69	66.77	76.09

Tabelle 3.6: Kritische Werte der χ^2 -Verteilung

3.7. COVID19 FALLZAHLEN ANALYSIEREN

Die Werte für Tabelle 3.6 werden erzeugt mit:

```
1 # werte von 1 bis 40
2 df <- c(seq(from=1, to=40, by = 1))
3 # chi2-Werte für unterschiedliche alpha berechnen
4 tabelle_chi <- data.frame(qchisq(0.9,df=df), qchisq(0.95,df=df), qchisq(0.975,df=df),
5 qchisq(0.99,df=df), qchisq(0.995,df=df), qchisq(0.9995,df=df))
6 # benenne Spalten nach alpha-Werten
7 colnames(tabelle_chi) <- c("0.01", "0.05", "0.025", "0.01", "0.005", "0.0005")
8 # benenne Zeilen nach df-Werten
9 row.names(tabelle_chi) <- df
10 # gib Tabelle aus
tabelle_chi
```

3.7 COVID19 Fallzahlen analysieren

Ich möchte die COVID19-Fallzahlen analysieren.

Hierfür gibt es eine schöne Anleitung von der Universität Toronto:

<https://mdl.library.utoronto.ca/technology/tutorials/covid-19-data-r>

Zunächst holen wir uns die aktuellen Daten.

```
1 # aktiviere das Tidyverse
2 library(tidyverse)
3
4 # Importiere Johns Hopkins Github data
5 confirmedraw <- read.csv(
6 "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_confirmed_global.csv")
7 deathsraw <- read.csv(
8 "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_deaths_global.csv")
9 recoveredraw <- read.csv(
10 "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_recovered_global.csv")
```

Dann bringen wir sie ins richtige Format.

```
1 confirmed <- confirmedraw %>% gather(key="date", value="confirmed", -c(Country.Region,
2 Province.State, Lat, Long)) %>% group_by(Country.Region, date) %>%
3 summarize(confirmed=sum(confirmed))
4 deaths <- deathsraw %>% gather(key="date", value="deaths", -c(Country.Region,
5 Province.State, Lat, Long)) %>% group_by(Country.Region, date) %>%
6 summarize(deaths=sum(deaths))
7 recovered <- recoveredraw %>% gather(key="date", value="recovered", -c(Country.Region,
8 Province.State, Lat, Long)) %>% group_by(Country.Region, date) %>%
9 summarize(recovered=sum(recovered))
10 summary(confirmed)
```

3.7. COVID19 FALLZAHLEN ANALYSIEREN

```
Country.Region      date      confirmed
Length:83712      Length:83712    Min.   :    0
Class :character    Class :character  1st Qu.:   141
Mode  :character    Mode  :character  Median :   4521
                                   Mean  :  208962
                                   3rd Qu.:  58988
                                   Max.   :30539759
```

Jetzt kombinieren wir alles in ein Datenframe und korrigieren die Datumsangaben

```
1 # Final data: combine all three
2 country <- full_join(confirmed, deaths) %>% full_join(recovered)
3 # Date variable
4 # repariere Datumsangaben von character nach date
5 country$date <- country$date %>% sub("X", "", .) %>% as.Date("%m.%d.%y")
6
7 # Neue variable: Anzahl der Tage
8 country <- country %>% group_by(Country.Region) %>% mutate(cumconfirmed=cumsum(confirmed),
9 days = date - first(date) + 1)
```

Jetzt aggregieren wir auf Weltperspektive und Deutschland.

```
1 world <- country %>% group_by(date) %>% summarize(confirmed=sum(confirmed),
2 cumconfirmed=sum(cumconfirmed), deaths=sum(deaths), recovered=sum(recovered)) %>%
3 mutate(days = date - first(date) + 1)
4 # Extract specific country: Germany
5 germany <- country %>% dplyr::filter(Country.Region=="Germany")
```

So vorbereitet können wir Statistiken ausgeben ...

```
1 # SUMMARY STATISTICS
2 summary(country)
3 by(country$confirmed, country$Country.Region, summary)
4 by(country$cumconfirmed, country$Country.Region, summary)
5 by(country$deaths, country$Country.Region, summary)
6 by(country$recovered, country$Country.Region, summary)
7 summary(world)
8 summary(germany)
```

... und Grafiken plotten

3.7. COVID19 FALLZAHLEN ANALYSIEREN

```
1 # World confirmed
2 ggplot(world, aes(x=date, y=confirmed)) + geom_bar(stat="identity", width=0.1) +
3   theme_classic() +
4   labs(title = "Covid-19 Global Confirmed Cases", x= "Date", y= "Daily confirmed cases") +
5   theme(plot.title = element_text(hjust = 0.5))
```

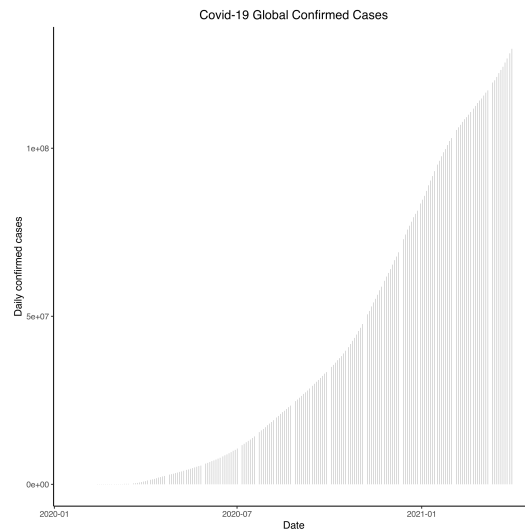


Abb. 3.23

```
1 # Germany confirmed
2 ggplot(germany, aes(x=date, y=confirmed)) + geom_bar(stat="identity", width=0.1) +
3   labs(title = "Covid-19 Confirmed Cases in Germany", x= "Date", y= "Daily confirmed
4   cases") +
5   theme(plot.title = element_text(hjust = 0.5))
```

3.7. COVID19 FALLZAHLEN ANALYSIEREN

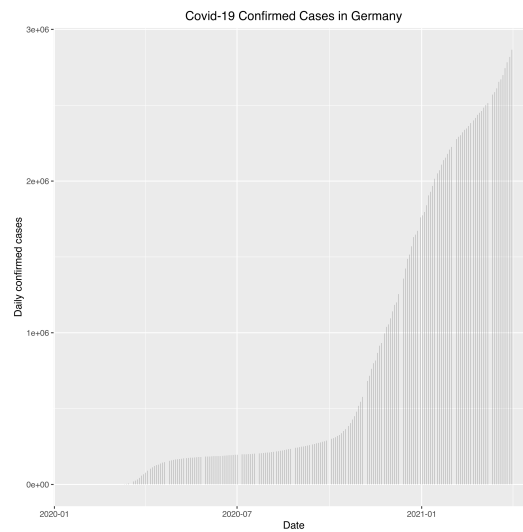


Abb. 3.24

```
1 # Line graph of cases over time
2 # World confirmed
3 ggplot(world, aes(x=days, y=confirmed)) + geom_line() +
4   labs(title = "Covid-19 Global Confirmed Cases", x= "Days", y= "Daily confirmed cases") +
5   theme(plot.title = element_text(hjust = 0.5))
6 # Ignore warning
```

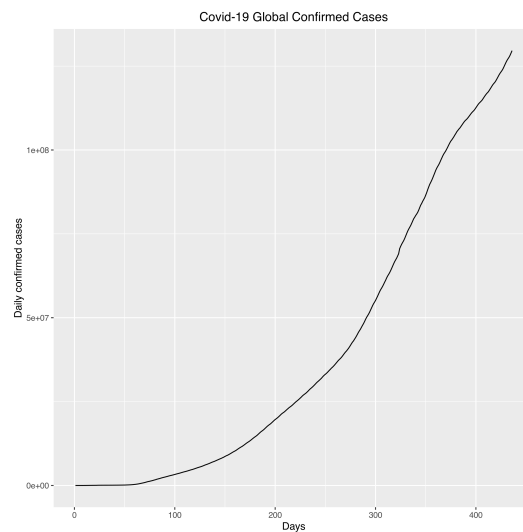


Abb. 3.25

3.7. COVID19 FALLZAHLEN ANALYSIEREN

```
1 # World confirmed with counts in log10 scale
2 ggplot(world, aes(x=days, y=confirmed)) + geom_line() +
3   labs(title = "Covid-19 Global Confirmed Cases", x= "Days", y= "Daily confirmed cases
4     (log scale)") +
5   theme(plot.title = element_text(hjust = 0.5)) +
6   scale_y_continuous(trans="log10")
```

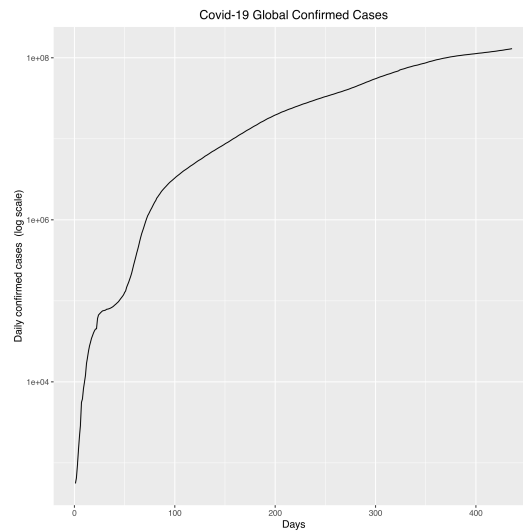


Abb. 3.26

3.7. COVID19 FALLZAHLEN ANALYSIEREN

```
1 # Confirmed by country for select countries with counts in log10 scale
2 countryselection <- country %>% filter(Country.Region==c("US", "Italy", "China", "France",
3 "United Kingdom", "Germany"))
4 ggplot(countryselection, aes(x=days, y=confirmed, colour=Country.Region)) +
5   geom_line(size=1) +
6   labs(title = "Covid-19 Confirmed Cases by Country", x= "Days", y= "Daily confirmed cases
  (log scale)") +
7   theme(plot.title = element_text(hjust = 0.5)) +
8   scale_y_continuous(trans="log10")
```

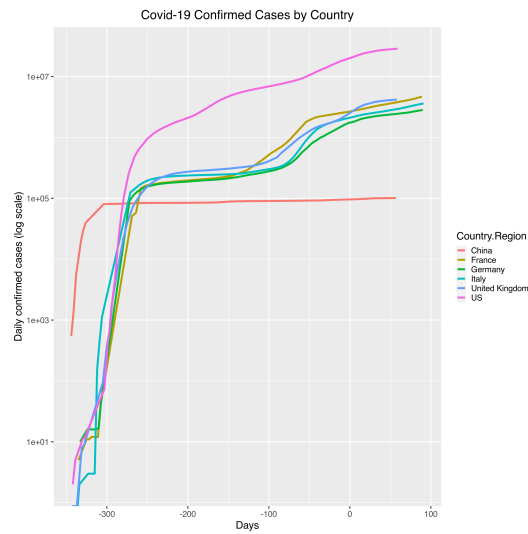


Abb. 3.27

3.7. COVID19 FALLZAHLEN ANALYSIEREN

```

1  # Matrix of line graphs of confirmed, deaths and recovered for select countries in log10
   scale
2  countryselection %>% gather("Type", "Cases", -c(date, days, Country.Region)) %>%
3    ggplot(aes(x=days, y=Cases, colour=Country.Region)) + geom_line(size=1) +
4    labs(title = "Covid-19 Cases by Country", x= "Days", y= "Daily cases (log scale)") +
5    theme(plot.title = element_text(hjust = 0.5)) +
6    scale_y_continuous(trans="log10") +
7    facet_grid(rows=vars(Type))

```

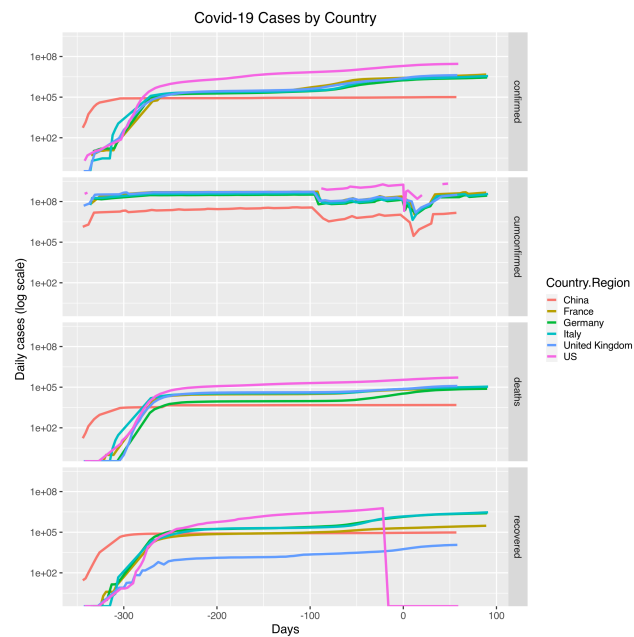


Abb. 3.28

Literaturverzeichnis

- [1] Cohen, J (1992): *A Power Primer*, Psychological Bulletin, 112(7):155-159
- [2] große Schlarmann, J; Galatsch, M (2014): *Regressionsmodelle für ordinale Zielvariablen*, GMS Medizinische Informatik, Biometrie und Epidemiologie, 10(1):2-10, doi: 10.3205/mibe000154, <https://www.egms.de/static/en/journals/mibe/2014-10/mibe000154.shtml>
- [3] Isfort, M; Rottländer, R; Weidner, F; Gehlen, D, Hylla, J, Tucman, D (2018): *Pflege-Thermometer 2018. Eine bundesweite Befragung von Leitungskräften zur Situation der Pflege und Patientenversorgung in der stationären Langzeitpflege in Deutschland*, Deutsches Institut für angewandte Pflegeforschung e.V. (DIP), Köln, <https://www.dip.de>
- [4] Kuckartz, U; Rädiker, S; Ebert, T. Schehl, J (2013): *Statistik - Eine verständliche Einführung*, Springer, ISBN 978-3-531-19889-7, S.125
- [5] Wickham, H (2010): *A layered grammar of graphics*, Journal of Computational and Graphical Statistics, 19(1):3-28, doi: 10.1198/jcgs.2009.07098, <https://vita.had.co.nz/papers/layered-grammar.html>
- [6] Wickham, H (2014): *Tidy Data*, Journal of Statistical Software, 59(10), doi: 10.18637/jss.v059.i10, <https://www.jstatsoft.org/article/view/v059i10>
- [7] Wickham, H; Grolemund, G (2017): *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*, O'Reilly Media, ISBN 978-1491910399, <https://r4ds.had.co.nz/>
- [8] Wilkinson, L (2005): *The Grammar of Graphics*, Springer, ISBN 978-0-387-28695-2
- [9] Xie, Y; Allaire, JJ; Grolemund, G (2020): *R Markdown: The Definitive Guide*, Chapman and Hall, ISBN 9781138359338 <https://bookdown.org/yihui/rmarkdown/>

- [10] Wickham, H; Navarro, D; Pedersen, TL (2022): *ggplot2: Elegant Graphics for Data Analysis*, Springer, ISBN 978-0387981406, <https://ggplot2-book.org/>

Stichwortverzeichnis

- [1], 19
- Anfängerfehler, 18, 25, 39, 46, 53, 58, 80, 127, 128, 137, 141, 182
- ANOVA, 280, 293
- arithmetisches Mittel, 209
- Balkendiagramme, 352
- Binomial-Verteilung, 110
- Bonferroni, 294
- Boxplots, 354
- Chiquadrat-Verteilung, 107
 - Plots erstellen, 406
 - Referenztabellen, 428
- Chunk, 128, 141
- Cohen's D, 272, 273
- CSV, 83, 91
- Datenframe, 51
- Datensatz, 51
 - Datensatz in Zusatzpaket, 98, 157
- dbl, 156
- Diagramme, 298
 - Balkendiagramm (R base), 317
 - Boxplot (R base), 319
 - Farben, 308
 - Histogramm (R base), 309
 - Kreisdiagramm (R base), 312
 - Punktwolke (R base), 300
 - speichern, 321
 - Säulendiagramm (R base), 315
- Dichteverteilung, 339
- Effektgröße, 272, 273
- Fallzahlen
 - Referenztabellen, 418
- Fallzahlkalkulation, 295
 - klinische Studien, 295
 - Umfragen, 296
- Farben, 308
- fct, 156
- Friedman ANOVA, 287
- Funktionen programmieren, 112
- ggplot, 322, 396
 - Cheatsheet, 324
- Histogramm, 337
- Häufigkeitstabelle, 206, 381, 382
- Interquartilsabstand, 212
- jgsbook, 97
- Kaplan-Meier-Analysen, 238
- Konfidenzintervalle, 258
 - Binomialverteilung, 264
 - Normalverteilung, 258
- Korrelation, 215
 - Pearson, 215
 - Spearman, 216
- Kreisdiagramme, 359
- Kreuztabelle, 381, 382
- Kreuztabellen, 213
- LaTeX, 145, 414, 420
- lgl, 156
- Markdown, 120
 - quarto, 132
 - RMarkdown, 121
- Median, 210
- Modalwert, 210
- NA, 68

- Normalverteilung
 - Dichtefunktion, 101
 - Plots erstellen, 392
 - Quantile, 102
 - Referenztabellen, 422
 - Verteilungsfunktion, 102
 - Zufallswerte, 103
- Pipe, 75, 151, 201
- Poisson-Verteilung, 108
 - Plots erstellen, 409
- Power, 295
 - Referenztabellen, 418
- Punktwolke, 335
- Quartile, 211
- Quartilsabstand, 212
- quarto, 132, 414
 - Bots, 417
 - Chunk, 141
 - Chunk-Optionen, 144
 - Dokumentenaufbau, 137
 - Editoransicht, 137
 - Installation, 132
 - Markdown, 120
 - R-Code integrieren, 141
 - RStudio, 133
 - Syntax, 139
- R
 - Anfängerfehler, 18, 25, 39, 46, 53, 58, 80, 127, 128, 137, 141, 182
 - Arbeitsverzeichnis, 79
 - Ausgabe in Datei, 98
 - CRAN, 92
 - Daten laden, 78
 - Daten speichern, 77
 - Datenexport, 91
 - Datenframes, 51
 - Datenimport, 80
 - Datenklassen, 35
 - Datenobjete, 32
 - Datentyp, 32
 - Faktoren, 45
 - Farben, 308
 - Installation, 2
 - Listen, 66
 - Matrizen, 38
 - NA, 68
 - Nachkommastellen, 20
 - Pipe, 75
 - Rechenoperationen, 17
 - RMarkdown, 121
 - Runden, 20
 - Umgang mit Datensätzen (klassisch), 70
 - Umgang mit Datensätzen (tidy), 172
 - Variablen und Objekte, 22
 - Vektor, 36
 - Youtube Anleitungen, 203
 - Zufallszahlen, 30
 - Zusatzpakete installieren, 92
- Referenztabellen, 418
- Regression, 219
 - linear, 219
 - logistisch, 223
 - multiple, 221
 - ordinal, 225
- rename(), 172
- RMarkdown
 - Chunk, 128
 - Syntax, 125
- RStudio
 - Arbeitsverzeichnis, 6
 - Cheatsheets, 202, 324
 - Daten importieren, 89
 - Datensätze, 61
 - Erscheinungsbild, 7
 - Installation, 2
 - Markdown, 120
 - Projekte, 13
 - quarto, 133
 - R-Zusatzpakete installieren, 94
 - Tastaturkürzel, 6
 - Youtube Anleitungen, 203
- Scatterplot, 335
- Screeplot, 249
- Signifikanztest, 270, 386

- χ^2 -Test, 270
- F-Test, 275
- Levene-Test, 275
- Mann-Whitney-U-Test, 276
- Normalverteilung, 278
- t-Test, 271
- Wilcoxon-Test, 277
- Skalenniveau
 - nominal, 45, 223
 - ordinal, 46, 50, 225
- Sonderzeichen, 376
- Spaltensummen, 384
- SPSS
 - exportiere aus R, 92
 - gelabelte Daten, 157
 - importiere nach R, 85
 - Labels, 85
- Standardabweichung, 212
- Säulendiagramme, 344
- t-Verteilung, 105
 - Plots erstellen, 403
 - Referenztabellen, 426
- Tibble, 153
- Tidy Data, 51, 63, 147
 - long table, 63, 148, 161
 - wide table, 63, 147, 162, 420
- Tidyverse, 147
 - Daten *tidy* machen, 160
 - Daten importieren, 157
 - Pipe, 151, 201
 - tibble, 153
 - Umgang mit Datensätzen, 172
- Varianz, 212
- Varianzanalyse, 280
 - ANOVA, 293
 - Friedman, 287
- Verteilungsfunktionen, 101
- Youtube Anleitungen, 203
- Z-Transformation, 214
- z-Transformation, 113
- Zeilensummen, 384
- Zufallszahlen, 30
- Zusatzpakete installieren, 92
 - Buchpaket, 97
- Überlebenszeitanalysen, 238
 - Cox-Regression, 246
 - Kaplan-Meier-Analysen, 238

Verzeichnis der verwendeten R-Befehle

?, 16

abline(), 249, 301
abs(), 20
across(), 164, 167, 381
add_count(), 189
aes(), 325
AIC(), 228, 236
annotate(), 375
anova_test(), 285
aov(), 284, 293
apply(), 388, 418
arrange(), 175
as.data.frame(), 62, 285, 388
as.Date(), 430
as.vector(), 64
as_factor(), 159
as_factor(), 86, 158
as_tibble(), 160
attr(), 85
autoplot(), 240

barplot(), 315, 317
bind_cols(), 182
bind_rows(), 182
bmp(), 321
boxplot(), 319

c(), 24, 36
case_when(), 391
cat(), 100
cbind(), 40, 55, 65, 161
ceiling(), 21
chisq.test(), 270, 386
class(), 34, 43, 49, 52, 57, 60
coef(), 227, 232, 236
cohensD(), 272, 273
cohens_d(), 286

colnames(), 43, 59
colors(), 299, 308, 371
coord_flip(), 352
coord_polar(), 359
cor(), 215, 216, 249, 255
cor.test(), 215, 217
corrplot(), 255
cortest.bartlett(), 249
cos(), 20
count(), 189
coxph(), 246
cumsum(), 206

data(), 95, 97
data.frame(), 52
dbinom(), 110
dchisq(), 107, 406
describe(), 208, 248
detach(), 93
deviance(), 228, 236
df.residual(), 230
dir(), 79
dnorm(), 101, 342
dpois(), 108, 409
drop_na(), 328
drop_na(), 187
dt(), 105

eigen(), 249
everything(), 185
exp(), 20, 223, 228
expand.grid(), 63, 418
expression(), 376
ezPlot(), 281, 288

fa.parallel(), 250
facet_grid(), 330
facet_wrap(), 330, 358

VERZEICHNIS DER VERWENDETEN R-BEFEHLE

factor(), 45
fct_infreq(), 345
fct_rev(), 345
fct_collapse(), 200
fct_explicit_na(), 399
fct_infreq(), 193
fct_lump_min(), 199
fct_lump_n(), 199
fct_recode(), 191
fct_relevel(), 197
fct_reorder(), 196
fct_rev(), 50, 194
fill(), 170
filter(), 173, 430
fisher.test(), 270
floor(), 21
format(), 22
freq(), 207
friedman_effsize(), 292
friedman_test(), 291
full_join(), 430
function(), 112, 232, 418

gather(), 429
generate_dictionary(), 159
geom_bar(), 344
geom_boxplot(), 354
geom_co(), 360
geom_col(), 344, 350
geom_histogram(), 337
geom_hline(), 374
geom_line(), 374
geom_point(), 326, 335
geom_smooth(), 328
geom_text(), 351
geom_vline(), 374
getwd(), 79
get_anova_table(), 285
ggforest(), 247
ggplot(), 322, 396
ggqqplot(), 282, 290
ggsave(), 362
ggsurvplot(), 241, 244
glimpse(), 155

glm(), 223
group_by(), 180, 188, 430

head(), 70, 78, 225
help(), 16
hist(), 309

if(), 115
ifelse(), 390
install.packages(), 92, 207, 225
installed.packages(), 117
IQR(), 212
is.character(), 34
is.data.frame(), 61
is.factor(), 49
is.logical(), 34
is.matrix(), 43
is.numeric(), 34
is.vector(), 37

jpeg(), 321

KMO(), 249

label_to_colnames(), 160
labs(), 363, 367
lapply(), 383
length(), 25, 37, 48, 206
levels(), 48
library(), 93
line(), 418
lines(), 302
list(), 66
lm(), 219, 221
load(), 78, 225
log(), 20
log10(), 20
log2(), 20
logLik(), 228
lrtest(), 230, 235
ls(), 26

matrix(), 38
mean(), 209
median(), 210
mfv(), 211
min(), 209

VERZEICHNIS DER VERWENDETEN R-BEFEHLE

`mutate()`, 86, 158, 164, 186, 390, 430
`mutate_if()`, 159, 167

`n()`, 190
`names()`, 253

`order()`, 72
`ordered()`, 50

`pairwise.t.test()`, 284, 293
`pairwise_t_test()`, 285
`par()`, 220, 222
`pbinom()`, 110
`pchisq()`, 107, 230
`pie()`, 312
`pivot_longer()`, 161, 280, 405
`pivot_wider()`, 162
`plot()`, 298, 300, 392
`png()`, 321
`pnorm()`, 102, 423, 425
`polygon()`, 303
`ppois()`, 108
`predict()`, 224
`principal()`, 256
`print()`, 99
`prop.table()`, 385
`pt()`, 105
`pull()`, 185
`pwr.t.test()`, 295, 418

`q()`, 17
`qbinom()`, 110
`qchisq()`, 107, 429
`qnorm()`, 102, 264, 267
`qpois()`, 108
`qt()`, 105, 258, 259, 427
`quantile()`, 211
`quit()`, 17

`rainbow()`, 308, 316, 318
`range()`, 209
`rbind()`, 41, 53, 145, 381
`rbinom()`, 110
`rchisq()`, 107

`read.csv()`, 84, 429
`read.table()`, 80, 83, 388
`read_excel()`, 87
`read_ods()`, 88
`read_sav()`, 85
`rep()`, 28
`replace_na()`, 170
`reshape()`, 419
`return()`, 112
`rev()`, 50
`revalue()`, 47
`rm()`, 26
`rnorm()`, 31, 103
`round()`, 21, 418
`rownames()`, 43, 60
`rpois()`, 108
`rt()`, 31, 105
`runif()`, 30

`sample()`, 30, 389, 390
`sample.size.mean()`, 296
`sample.size.prop()`, 297
`save()`, 78
`save.image()`, 77
`scale()`, 214
`scale_color_manual()`, 371
`scale_fill_discrete()`, 368, 373
`scale_fill_hue()`, 371
`scale_fill_manual()`, 371
`scale_x_continuous()`, 365
`scale_y_continuous()`, 365
`sd()`, 212
`select()`, 185
`separate()`, 163
`seq()`, 28, 423, 425, 427, 429
`setwd()`, 79
`shapiro.test()`, 278, 290
`sin()`, 20
`sink()`, 99
`slice()`, 176
`slice_head()`, 179
`slice_min()`, 176
`slice_sample()`, 179
`slice_tail()`, 179

VERZEICHNIS DER VERWENDETEN R-BEFEHLE

`sort()`, 71, 210
`source()`, 119
`sqrt()`, 17, 19
`stat_bin()`, 339
`stat_boxplot()`, 354
`stat_count()`, 344
`stat_density()`, 340
`stat_function()`, 342
`stat_identity()`, 344, 350
`stat_summary()`, 357
`stat_function()`, 106, 403
`str()`, 70
`sub()`, 430
`subset()`, 73, 419
`sum()`, 59, 68
`summarise()`, 187, 430
`summary()`, 207
`Surv()`, 239
`survdiff()`, 245
`survfit()`, 239

`t()`, 42, 64
`t.test()`, 271
`table()`, 49, 59, 206, 210, 213, 382
`tail()`, 71
`tan()`, 20
`theme()`, 363

`theme_classic()`, 430
`tibble()`, 153, 155
`tiff()`, 321
`transmute()`, 381
`tribble()`, 154

`url()`, 78, 81, 83, 388

`var()`, 212
`var.test()`, 275
`vglm()`, 226, 233

`which()`, 69
`wilcox.test()`, 276, 277
`wilcox_effsize()`, 292
`wilcox_test()`, 291
`with()`, 58, 74
`write.csv()`, 91
`write.table()`, 91
`write_sav()`, 92

`xlab()`, 364
`xlim()`, 364
`xtable()`, 420
`xtabs()`, 214, 270, 382

`ylab()`, 364
`ylim()`, 364



Prof. Dr. rer. medic. MScN, BScN, RN

Jörg große Schlarmann

Professor für Pflegewissenschaft

Fachbereich 10 - Gesundheitswesen

Campus Krefeld Süd

Hochschule Niederrhein

joerg.grosseschlarmann@hs-niederrhein.de

<https://mastodon.social/@rbuch>